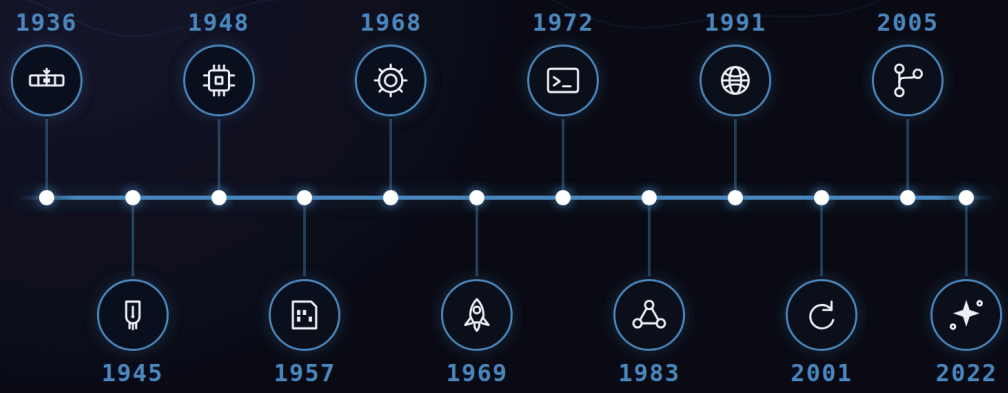
Software Engineering

Standing on the Shoulders of Giants

Python edition



Thomas Hastings

First Edition · 1.0b16 · swebook.org

Software Engineering: Standing on the Shoulders of Giants — Python Edition

- Software Engineering: Standing on the Shoulders of Giants
  - Who this book is for
  - Why this book exists
  - How the book is organized
  - Suggested paths through the book
  - Code examples in six languages
  - How this book was made (AI assistance)
  - About the author
  - If this book helped you
  - Citing this book
  - Copyright and license
- Chapter 1 — Introduction
  - 1.1 What Is Software Engineering?
  - 1.2 The Requirements Challenge
  - 1.3 Software Is Intrinsically Complex
  - 1.4 Defects Are Inevitable
  - 1.5 Balancing Constraints: The Iron Triangle
  - 1.6 Social Responsibility
  - 1.7 Conclusion
- Chapter 1 — Exercises
  - Concepts
  - Analysis
  - Ethics
- Chapter 1 — Open Resources
  - What software engineering is
  - Origins of the discipline (used in §1.1.3)
  - Complexity and defects
  - Professional responsibility (used in §1.6)
  - License note
- Chapter 2 — Software Development Processes
  - 2.1 Processes and Values Guide Development
  - 2.2 Structuring Teamwork: The Scrum Framework
  - 2.3 Agile Development: Extreme Programming
  - 2.4 Limitations of Waterfall Processes

    - 2.5 Levels of Design and Testing: V Processes
    - 2.6 Additional Project Risks
    - 2.7 Risk Reduction: The Spiral Framework
    - 2.8 Shape Up: Fixed Time, Variable Scope
    - 2.9 Conclusion
- Chapter 2 — Exercises
    - Processes and values (§2.1)
    - Scrum (§2.2)
    - XP and engineering practice (§2.3)
    - Plan-driven models and their risks (§§2.4–2.5)
    - Risk and the spiral (§§2.6–2.7)
    - Shape Up (§2.8)
- Chapter 2 — Open Resources
    - Foundational values
    - Plan-driven and risk-driven models (primary sources)
    - Course material
    - License note
- Chapter 3 — User Requirements
    - 3.1 What Is a Requirement?
    - 3.2 Developing Requirements and Software
    - 3.3 Eliciting User Needs
    - 3.4 Writing Requirements: Stories and Features
    - 3.5 Writing User-Experience Scenarios
    - 3.6 Clarifying User Goals
    - 3.7 Identifying Security Attacks
    - 3.8 Conclusion
- Chapter 3 — Exercises
    - Concepts
    - Analysis
- Chapter 3 — Open Resources
    - User stories, features, and BDD
    - Gherkin, Given/When/Then, and BDD
    - Traceability
    - Writing and specifying requirements
    - Eliciting and prioritizing needs
    - Framing the problem
    - Goals and goal modeling
    - Security requirements and attack trees
    - License note

- Chapter 4 — Requirements Analysis
  - 4.1 A Checklist Approach
  - 4.2 Relative Estimation: Iteration Planning
  - 4.3 Structured Group Consensus Estimates
  - 4.4 Balancing Priorities
  - 4.5 Customer Satisfiers and Dissatisfiers
  - 4.6 Plan-Driven Estimation Models
  - 4.7 Conclusion
- Chapter 4 — Exercises
  - Concepts
  - Estimation and forecasting
  - Prioritization
  - Effort models
- Chapter 4 — Open Resources
  - Relative estimation, story points, and velocity
  - Consensus estimation: Planning Poker and Delphi
  - Prioritization
  - Customer satisfaction: the Kano model
  - Plan-driven / algorithmic estimation: COCOMO
  - Estimation, judgment, and its limits (background)
  - Appetite: fixed time, variable scope
  - License note
- Chapter 5 — Use Cases
  - 5.1 Elements of a Use Case
  - 5.2 Alternative Flows: Conditional Behaviors
  - 5.3 Writing Use Cases
  - 5.4 Use-Case Diagrams
  - 5.5 Relationships between Use Cases
  - 5.6 Conclusion
- Chapter 5 — Exercises
  - Concepts
  - Analysis
- Chapter 5 — Open Resources
  - Foundations: what a use case is
  - Notation and diagrams
  - Templates and practice
  - License note
- Chapter 6 — Design and Architecture
  - 6.1 The Role of Architecture

    - 6.2 Designing Modular Systems
    - 6.3 Class Diagrams
    - 6.4 Architectural Views
    - 6.5 Describing System Architecture
    - 6.6 Conclusion
- Chapter 6 — Exercises
    - Concepts
    - Analysis
- Chapter 6 — Open Resources
    - Modularity, abstraction, and interfaces
    - Coupling, cohesion, and design principles
    - Class diagrams and UML notation
    - Architectural views (used in §6.4)
    - Quality-attribute scenarios
    - License note
- Chapter 7 — Architectural Patterns
    - 7.1 Software Layering
    - 7.2 Three Building Blocks
    - 7.3 User Interfaces: Model-View-Controller
    - 7.4 Dataflow Architectures
    - 7.5 Connecting Clients with Servers
    - 7.6 Families and Product Lines
    - 7.7 Conclusion
- Chapter 7 — Exercises
    - Concepts
    - Analysis
    - Choosing a pattern
- Chapter 7 — Open Resources
    - Pattern catalogs (use across the whole chapter)
    - Dataflow, streams, and big data (§7.4)
    - Client-server, broker, and distributed structure (§7.5)
    - Product lines (§7.6)
    - RESTful APIs (§7.5.4)
    - Foundational background
    - License note
- Chapter 8 — Version Control with Git
    - 8.1 Why Version Control
    - 8.2 The Everyday Workflow
    - 8.3 Branching and Merging

    - 8.4 Working with Others
    - 8.5 Merge Conflicts
    - 8.6 Habits That Make Git Work for a Team
    - 8.7 Markdown, READMEs, and Project Documentation
    - 8.8 Conclusion
- Chapter 8 — Exercises
    - Concepts
    - Analysis
- Chapter 8 — Open Resources
    - Foundations: what Git is and how it works (used in §8.1)
    - Building the right mental model (used in §8.1, §8.3)
    - Hands-on practice and everyday commands (used in §8.2, §8.4)
    - Team workflows and conflicts (used in §8.4, §8.5)
    - Commit and history discipline (used in §8.6)
    - Markdown and documentation (used in §8.7)
- Chapter 9 — Static Checking
    - 9.1 Architecture Reviews
    - 9.2 Conducting Software Inspections
    - 9.3 Code Reviews: Check Intent and Trust
    - 9.4 Automated Static Analysis
    - 9.5 Conclusion
- Chapter 9 — Exercises
    - Concepts
    - Analysis
- Chapter 9 — Open Resources
    - Code review (used in §9.3)
    - Inspection (used in §9.2)
    - Automated static analysis (used in §9.4)
    - Learning from failure (used in §9.1)
    - License note
- Chapter 10 — Testing
    - 10.1 Overview of Testing
    - 10.2 Levels of Testing
    - 10.3 Code Coverage I: White-Box Testing
    - 10.4 Input Coverage I: Black-Box Testing
    - 10.5 Code Coverage II: MC/DC
    - 10.6 Input Coverage II: Combinatorial Testing
    - 10.7 Conclusion
- Chapter 10 — Exercises

    - Concepts
    - Analysis
    - Do the work
- Chapter 10 — Open Resources
    - Foundations: what to test and how
    - Coverage criteria, white-box, and MC/DC
    - Black-box and combinatorial testing
    - Tools and practice
    - License note
- Chapter 11 — Software Security
    - 11.1 Security Is a Lifecycle Property
    - 11.2 The OWASP Top 10
    - 11.3 Finding Vulnerabilities, from Manual to Autonomous
    - 11.4 The Software Supply Chain: A World of Weak Links
    - 11.5 Building Security In
    - 11.6 Conclusion
- Chapter 11 — Exercises
    - Concepts
    - Analysis
- Chapter 11 — Open Resources
    - The OWASP Top 10 and hands-on practice
    - AI in security
    - The software supply chain
    - Standards, frameworks, and courses
    - License note
- Chapter 12 — Quality Metrics
    - 12.1 Meaningful Metrics
    - 12.2 Software Quality
    - 12.3 Graphical Displays of Data Sets
    - 12.4 Product Quality: Measuring Defects
    - 12.5 Ops Quality Improvement: A Case Study
    - 12.6 Data Dispersion: Boxplots and Histograms
    - 12.7 Data Dispersion: Statistics
    - 12.8 Confidence Intervals
    - 12.9 Simple Linear Regression
    - 12.10 Conclusion
- Chapter 12 — Exercises
    - Concepts
    - Analysis

    - Calculation
- Chapter 12 — Open Resources
    - Statistics foundations (§§12.6–12.9)
    - Software metrics and quality (§§12.1–12.5)
    - Reference and data
    - License note
- Chapter 13 — Software Engineering in the Age of AI
    - 13.1 What AI Changes — and What It Doesn’t
    - 13.2 AI Across the Lifecycle
    - 13.3 The Evidence: Productivity, Quality, Security
    - 13.4 Outcome Engineering: The o16g Manifesto
    - 13.5 Spec-Driven Development
    - 13.6 Context as Infrastructure: `CLAUDE.md`, `AGENTS.md`, and Skills
    - 13.7 Loop Engineering
    - 13.8 Principles for the AI-Augmented Engineer
    - 13.9 Conclusion
- Chapter 13 — Exercises
    - Concepts
    - Analysis
    - The o16g manifesto
    - Agentic practice
- Chapter 13 — Open Resources
    - The o16g manifesto (used in §13.4)
    - Productivity evidence (used in §13.1.4, §13.3)
    - Code quality & security (used in §13.3)
    - AI in requirements, design, testing (used in §13.2)
    - Agentic development in practice (used in §§13.5–13.7)
    - Foundations & agentic engineering
    - License note
- Chapter 14 — Delivery: CI/CD, DevOps, and Operations
    - 14.1 SaaS and the Cloud
    - 14.2 Continuous Integration Pipelines
    - 14.3 Continuous Deployment
    - 14.4 Packaging and Running a Service: Docker and Compose
    - 14.5 Reaching Users: DNS, TLS, and the Edge
    - 14.6 Continuous Security Pipelines
    - 14.7 DORA Delivery Metrics
    - 14.8 Conclusion
- Chapter 14 — Exercises

    - Concepts
    - Analysis
- Chapter 14 — Open Resources
    - Primary sources for the case studies
    - The cloud and distributed systems
    - CI/CD and deployment practice
    - Deployment mechanics: containers, DNS, and TLS
    - DORA and delivery research
    - Continuous security and the supply chain
    - License note
- Chapter 15 — Maintenance and Evolution
    - 15.1 What Makes Code Legacy
    - 15.2 Characterization Tests
    - 15.3 Refactoring Under Green Tests
    - 15.4 Repository Stewardship
    - 15.5 Technical Debt
    - 15.6 Strangler Fig versus Big-Bang Rewrite
    - 15.7 Conclusion
- Chapter 15 — Exercises
    - Concepts
    - Analysis
- Chapter 15 — Open Resources
    - Legacy code, refactoring, and technical debt
    - License note
- Appendix A — The Team Project
    - A.1 Overview
    - A.2 Project Proposal
    - A.3 Skeletal System: Status Report 1
    - A.4 Viable System: Status Report 2
    - A.5 Comprehensive Final Report
    - A.6 Grading Your Own Repository
    - A.7 Conclusion
- Running the Project on Two-Week Sprints
    - Dual hats: every team is both customer and developer
    - The cadence
    - Sprint 0: from pitch to deployed skeleton
    - Sprints 1–4: build cycles with a hardening arc
    - Every sprint: the common cycle
    - Demo Day rules

    - Appetite per sprint
    - Team and customer reviews
    - End of term
- Appendix A — Project Exercises
    - Weeks 1–2 — Team and idea
    - Weeks 3–4 — Proposal
    - Weeks 5–7 — Skeletal system (Status Report 1)
    - Weeks 8–11 — Viable system (Status Report 2)
    - Weeks 12–15 — Final report and demo
- Appendix A — Open Resources
    - Running a team project end to end
    - Requirements for the proposal (A.2)
    - Team effectiveness (A.1.2)
    - Process (A.1.1, referenced from Ch. 2)
    - Curriculum context (why the course is shaped this way)
    - Cross-team customer models (the dual-hat swap)
    - Metrics for status reports and the final report (referenced from Ch. 12)
    - License note
- Idea Pitch —
    - 1. Problem
    - 2. Appetite
    - 3. Rough solution
    - 4. Rabbit holes
    - 5. No-gos
    - 6. Customer contact
- Project Proposal — <Team / Project Name>
    - 1. Problem statement
    - 2. Users and stakeholders
    - 3. Proposed solution and value
    - 4. Key user stories / features (prioritized)
    - 5. Scope boundaries
    - 6. Intended architecture and technology
    - 7. Process and team plan
    - 8. Risks and how you'll probe them
    - 9. Definition of done for the term
- Sprint Report <1–4> — <Team / Project Name>
    - 1. Sprint goal
    - 2. What shipped (demo notes)
    - 3. Stories: planned vs. done

    - 4. Quality
    - 4a. Process evidence
    - 5. Scopes and hill chart
    - 6. Scope cuts and nice-to-haves dropped
    - 7. Risks and blockers
    - 8. Next sprint plan
    - 9. Contributions
- Status Report <1 | 2> — <Team / Project Name>
    - 1. Summary
    - 2. What works right now
    - 3. Architecture (current)
    - 4. Process and progress
    - 5. Testing and quality (Report 2 emphasizes this)
    - 6. Metrics so far (Report 2)
    - 7. Risks, blockers, and changes
    - 8. Plan for next milestone
    - 9. Individual contributions this period
- Team & Customer Review <#0–4> —
    - 1. Retrospective: keep / change / try
    - 2. Improvement committed last review — did it stick?
    - 3. New committed improvement
    - 4. Peer contribution
    - 5. Customer feedback captured
    - 6. Escalations
- Final Report — <Team / Project Name>
    - 1. Problem and users
    - 2. Users and requirements
    - 3. Architecture
    - 4. Process used
    - 5. Testing and quality metrics
    - 6. Results
    - 7. Retrospective
    - 8. AI use and provenance
    - 9. Individual contributions
    - 10. Appendix (optional)
- Individual Write-up —
    - 1. What I built
    - 2. What I learned
    - 3. Working on a team

    - 4. What I’d do differently
    - 5. Self-assessment
- Open Curriculum Resource Map
    - The four anchor courses
    - Chapter 1 — Introduction
    - Chapter 2 — Software Development Processes
    - Chapter 3 — User Requirements
    - Chapter 4 — Requirements Analysis
    - Chapter 5 — Use Cases
    - Chapter 6 — Design and Architecture
    - Chapter 7 — Architectural Patterns
    - Chapter 8 — Version Control with Git
    - Chapter 9 — Static Checking
    - Chapter 10 — Testing
    - Chapter 11 — Software Security
    - Chapter 12 — Quality Metrics
    - Chapter 13 — Software Engineering in the Age of AI
    - Chapter 14 — Delivery: CI/CD, DevOps, and Operations
    - Chapter 15 — Maintenance and Evolution
    - Appendix A — A Team Project
    - Coverage vs. the ACM/IEEE SEEK body of knowledge
    - Notes on coverage & gaps
- Course Plans
    - Variant A: 15-week milestone track
    - Variant B: 16-week semester on two-week sprints
    - Assessment (suggested weights)
    - Learning outcomes
    - Adapting the plan
- Contributing
    - Ground rules
    - Repository layout
    - House style
    - Making a change
    - Reporting issues

Software Engineering: Standing on the Shoulders of Giants

First Edition, 1.0b16.

📖 **Read the book online at www.swebook.org.**

This book is freely available and openly licensed, providing instruction in the principles and best practices of modern software engineering: how software is built in industry using agile methods, how to discover and analyze requirements, how to design and architect modular systems, how to check and test code, and how to measure quality with metrics.

The book is structured into fifteen chapters and a team project appendix, following the topic progression of the ACM/IEEE software-engineering curriculum guidelines (SE2014/SEEK).[1] Each chapter contains original explanatory prose rather than material copied from other textbooks and is accompanied by a curated map of free or openly accessible resources — MOOCs, university courseware, primary specifications, and papers — with licenses noted where known. The material is designed to support a complete software-engineering course.

> **An independent open educational resource.** This is a complete, standalone book. Except for clearly attributed quotations and linked external resources, its explanatory prose, examples, diagrams, and exercises are original, and the book is released under Creative Commons. See `curriculum/open-resources-map.md` for the mapping from each chapter to complementary open materials. The prose was drafted with AI assistance under the author's direction, review, and fact-checking — see How this book was made below.

Who this book is for

This book is intended for upper-division undergraduate students who have completed at least two programming courses and are comfortable reading modular or object-oriented code, using a command line, and working with basic data structures. Prior experience with Git, automated testing, statistics, or cloud deployment is helpful but not required — each is taught from first principles in its chapter. Instructors will find ready-made course plans in `curriculum/course-plan.md`; self-learners can follow the suggested paths below.

Why this book exists

Most significant software is developed by teams, under evolving requirements, and with inevitable defects, at a scale that exceeds the capacity of any individual to fully comprehend. A typical programming course teaches

students to write individual functions, whereas a software engineering course instructs students in building and evolving complete systems collaboratively. The discipline emphasizes enduring principles over proficiency in any single programming language:

1. **Software is complex** → we manage complexity with design and architecture.
2. **Requirements change** → we use iterative, agile methods to adapt.
3. **Defects are inevitable** → we use reviews, static checking, and testing to catch them.
4. **Teams need coordination** → we use processes, and a healthy balance of structure and flexibility.

These four foundational concepts are integrated throughout every chapter.

How the book is organized

The chapters are organized into thematic arcs:

Arc	Chapters	Question it answers
Getting started	1–2	What is software engineering, and what process do we use?
What to build	3–5	How do we discover, analyze, and specify requirements?
Design & architecture	6–7	How do we structure a system to tame complexity?
Version control	8	How do teams manage, review, and collaborate on a shared codebase?
Software quality	9–10	How do we check and test that the system works?
Security	11	How do we build software that resists attack, from the code to the supply chain?
Metrics	12	How do we measure quality and track progress?
The AI shift	13	How is AI reshaping every stage — and what stays the same?
Delivery	14	How does code get from a merged branch to running safely in production?
Evolution	15	How does code stay changeable through years of maintenance and growth?
Practice	Appendix A	How do we run a real team project alongside the concepts?

Suggested paths through the book

- **One-semester course (14–16 weeks).** Cover the complete conceptual spine: most chapters get a week, Chapter 13's AI material is distributed across the term through its per-stage sections (§13.2), and selected advanced sections work as optional or independent reading — with the Appendix A team project running in parallel from week 2. Two ready-made week-by-week plans (a milestone track and a two-week-sprint track) are in `curriculum/course-plan.md`.
- **One-quarter course (~10 weeks).** Keep the spine and compress: Chapters 1–2 (week 1), 3–4 (weeks 2–3, folding Chapter 5's use-case notation into the requirements work), 6–8 (weeks 4–5, design, patterns, and version control), 9–10 (weeks 6–7, checking and testing), 11–12 (week 8, security and metrics), and 14–15 (week 9, delivery and evolution), with Chapter 13 (AI) read alongside each topic through its per-stage sections (§13.2). Run a scoped-down project: proposal, two sprints, final demo.
- For self-study, chapters should be read sequentially. Each chapter concludes with key takeaways, exercises, and free resources for further study. Alternatively, readers may use the arc table above to focus on a specific goal, such as Chapters 9–10 for checking and testing or Chapter 14 for CI/CD. Chapters cross-reference the earlier ideas they depend on.

Code examples in six languages

Every code example in the book is provided in **Python, Java, JavaScript, Go, Ruby, and TypeScript**, plus a language-agnostic **Generic** tab written in pseudocode, behind language tabs — pick your language once and the whole site follows (or open any page with `?lang=go` to pre-select one). Each snippet also lives in `code/` as a runnable file with a test, executed by continuous integration on every change.

The book is also available as an **EPUB and a PDF**, one edition per language plus a **Generic (pseudocode)** edition — identical prose, published with each tagged release:

EPUB: Generic · Python · Java · JavaScript · Go · Ruby · TypeScript PDF: Generic · Python · Java · JavaScript · Go · Ruby · TypeScript

All editions, including past versions, are on the releases page.

How this book was made (AI assistance)

This book was written in collaboration with an AI assistant (Anthropic's Claude) under the author's direction. The author set the scope, chapter progression, and course alignment; supplied source material and corrections; fact-checked claims against the primary sources cited in each chapter's *Open Resources* page; and edited the prose throughout. The author has reviewed, and stands behind, every chapter. If you find an error, please open an issue, regardless of how it was introduced.

Chapter 13 teaches that professional AI use means disclosing the assistance, verifying the output, and owning the result. This note applies that standard to the book itself.

About the author

Thomas Hastings teaches software engineering at the University of Colorado Colorado Springs. This book started as part of his combined undergraduate and graduate course, CS 4300/5300. He also serves as an assistant professor (reservist) at the United States Air Force Academy, where he teaches in the Department of Computer and Cyber Sciences. He earned his Ph.D. in Engineering and M.Eng. in Software Engineering from UCCS, and his B.S. from Colorado Christian University. His research focuses on open-source software security and software supply chains, including the continuous verification of components.

The focus on testing, delivery, and running code in this book comes from real-world experience. Thomas has worked as a software engineer for over twenty years and is currently at Amazon Web Services. He keeps this book as an open, ongoing project. The views in this book are his own and do not reflect those of his employers.

Links: Google Scholar · tom.hastings.dev · GitHub

If this book helped you

If it helped you learn a topic, teach a course, or build something, the most useful thing you can do in return is **star the repository on GitHub** — it takes a second, and a star is how I get to see who the book is reaching: students, instructors, teams, and self-learners. Knowing where it lands is what tells me the work is worth continuing, and it shapes what I write and fix next. If you have a moment for more than a click, I would genuinely love to hear how you are using it — a quick note in an issue naming your course, team, or project makes my week.

Citing this book

If you use this book in a course or reference it in your writing, please cite it. GitHub's **"Cite this repository"** button (from `CITATION.cff`) gives APA and BibTeX directly, or copy the BibTeX below.

Whole book:

```bibtex
@book{hastings2026swe,
  author    = {Hastings, Thomas},
  title     = {Software Engineering: Standing on the Shoulders of Giants},
  year      = {2026},
  publisher = {Self-published},
  url       = {https://www.swebook.org/},
  note      = {Open textbook, licensed CC BY-SA 4.0. Source:
               \url{https://github.com/tghastings/open-swe-book}}
}
```

A single chapter (adjust `title`, `chapter`, and `url`):

```bibtex
@inbook{hastings2026swe-ch14,
  author    = {Hastings, Thomas},
  title     = {Delivery: CI/CD, DevOps, and Operations},
  booktitle = {Software Engineering: Standing on the Shoulders of Giants},
  chapter   = {14},
  year      = {2026},
  publisher = {Self-published},
  url       = {https://www.swebook.org/chapters/14-delivery/}
}
```

The `\url` in `note` needs `\usepackage{url}` (or `hyperref`) in your LaTeX preamble; with `biblatex` you can move it to a `urldate`/`addendum` field instead. If you use a classic BibTeX style (`plain`, `plainnat`), swap `@inbook` for `@incollection` — those styles ignore `booktitle` inside `@inbook`.

Copyright and license

Copyright © 2026 Thomas Hastings. Unless otherwise noted, the text and figures are licensed **CC BY-SA 4.0**; code examples are licensed **MIT**. You are free to share and adapt with attribution. All trademarks, product names, and company names are the property of their respective owners and are used for identification and educational purposes only. See also the `LICENSE` file.

[1] ACM/IEEE-CS Joint Task Force on Computing Curricula, *Software Engineering 2014: Curriculum Guidelines for Undergraduate Degree Programs in Software Engineering* (SE2014), which defines the SEEK body of knowledge. acm.org/education/curricula-recommendations.

Chapter 1 — Introduction

> **Where we are.** This opening chapter introduces the ideas that thread through the book. These include the definition of software engineering, the challenges associated with requirements, the inherent complexity of software, the inevitability of defects, and the trade-offs among project scope, cost, and time. Additionally, the chapter addresses the importance of social responsibility for software professionals. Subsequent chapters each address one or more of these foundational challenges.

Writing a program that works once, on your machine, for input you chose yourself, is programming. **Software engineering** is what you need when *other people* depend on that program — when it must keep working next year, after five other developers have changed it, on inputs no one anticipated, at a scale no single person can hold in their head. Understanding the gap between those two activities is the subject of this book.

1.1 What Is Software Engineering?

1.1.1 A Working Definition

> **Definition.** *Software engineering* is the disciplined application of principles, methods, and tools to build and evolve software systems that are useful, correct enough, and economical to change — usually by a **team**, usually under **changing requirements**, usually at a **scale** that defeats ad-hoc effort.

Three words in that definition carry a lot of weight:

- **Team.** Most real software — and nearly all software at commercial scale — is built by groups. Coordination, communication, and shared understanding become first-class engineering concerns, not afterthoughts. This is why Chapter 2 is about *process* and why an entire team project runs alongside the concepts (Appendix A).
- **Changing.** Requirements are discovered, not handed down. They shift as users learn what they actually want. Methods that assume a frozen specification tend to fail; this is why the industry moved toward *iterative, agile* methods (Chapters 2–4).
- **Scale.** Beyond a few tens of thousands of lines, hardly anyone understands the whole system in detail. We manage that with **design and architecture** (Chapters 6–7): dividing the system so each part can be understood, changed, and tested in relative isolation.

What separates software engineering from mere coding is the focus on the **why** behind the **how**. Any competent developer can be told “use dependency injection here.” An engineer understands *what problem it solves*

(coupling), *when it helps*, and *when it is overkill* along with the trade-offs — and can make the call on a design they have never seen before.

1.1.2 A Tale of Two Companies

Consider two startups building the same product — a scheduling app for clinics.

Company A hires strong programmers and tells them to "just ship." They skip written requirements ("we all know what a calendar does"), skip tests ("we'll test by using it"), and let the architecture emerge by accretion. For three months they look *faster* than anyone: demos appear weekly. Then the trouble starts. A change to time-zone handling breaks appointment reminders in a way no one notices for two weeks. Onboarding a fourth engineer takes a month because only one person understands the booking logic. Every new feature has a chance of resurrecting an old bug. Velocity collapses under the weight of the system's own complexity.

Company B spends the first two weeks talking to clinic staff, writing a handful of user stories, sketching a modular architecture, and setting up automated tests and code review. Their first demo comes a week later than Company A's. But by the third month the two teams look nothing alike: new engineers are productive in days, changes are localized, and a failing test usually catches a regression before it reaches a user.

Neither team is smarter. The difference is **engineering discipline**: Company B paid a small, early, *predictable* cost to avoid a large, late, *unpredictable* one. The rest of this book is, in effect, a catalog of the disciplines that make a team behave like Company B — and an honest account of when each is worth its cost.

1.1.3 Where the Discipline Came From

The phrase "software engineering" owes its early currency to practitioners, not to a standards committee. In the 1960s, **Margaret Hamilton** led the team at the MIT Instrumentation Laboratory that wrote the flight software for the **Apollo Guidance Computer** — the code that flew astronauts to the Moon.[1] At the time, programming was widely seen as a clerical afterthought to the "real" engineering of hardware. Hamilton pushed the term **software engineering** into use deliberately, to insist that building flight software deserved — and required — the same rigor and respect as the aerospace engineering around it.[2]

Her team's discipline paid off in the most public way imaginable. Minutes before the Apollo 11 lunar landing, the guidance computer was flooded with unexpected input and began running out of capacity, throwing alarms. Because the software had been engineered with **priority scheduling** — the ability to shed low-priority work and keep the tasks that mattered most for landing — it recovered instead of crashing, and the landing proceeded.[3] A design decision made long before launch, against a failure no one could fully predict, saved the mission. It remains one of the clearest early demonstrations of what disciplined software design can do for a project.

The rest of the industry was learning similar lessons. Through the 1960s, hardware performance improved on the exponential curves we now describe with Moore's Law,[4] while large software projects ran chronically late, blew

their budgets, and failed outright in mission-critical settings — defense, aerospace, banking. Software, not hardware, had become the bottleneck. In 1968, a NATO-sponsored conference in Garmisch, Germany gave the problem a name — the **software crisis** — and proposed a remedy in its very title: the *NATO Software Engineering* conference argued that software should be developed like an engineering discipline, with deliberate methods, rather than by ad-hoc programming and heroics.[5]

What ultimately answered the crisis, though, was not a fixed body of rules like a building code. Software turned out to be too changeable for that. The discipline eventually adopted an **empirical** approach modeled on experimental science. Iterate in small steps, seek fast **feedback**, prefer **incremental** progress you can evaluate, run **experiments** when you are unsure, and let measured evidence — not opinion or authority — settle the argument. Modern writers such as Dave Farley frame the effective software engineer as this kind of scientist.[6] Keep that framing in mind in Chapter 2: every process you will learn about there — Scrum's sprints, XP's tests, the spiral's risk-driven loops — is an institutionalized version of the same iterate, measure, and adjust cycle.

1.2 The Requirements Challenge

The hardest part of many projects is not building the system right, but figuring out **which system to build**. Fred Brooks, the author of *The Mythical Man-Month*, famously argued that the hardest single part of software work is deciding precisely what to build, because it is the one part whose mistakes most cripple the result and are hardest to fix later.[7]

1.2.1 Identifying Users and Requirements

A **requirement** is a statement of something the software must do or a quality it must have. The catch is that the people who *have* the need are usually not the people who *build* the software, and they rarely state the need precisely. Users describe solutions ("add a dropdown here") when what they have is a *problem* ("I can't tell which appointments are unconfirmed"). Part of the engineer's job is to dig past the requested solution to the underlying problem.

Requirements also come from more than the obvious end user. A clinic scheduling app has:

- **End users** (front-desk staff) who book appointments,
- **Secondary users** (clinicians) who view schedules,
- **Sponsors** who pay and care about cost and compliance,
- **Regulators** whose rules (privacy, accessibility) are non-negotiable constraints.

Chapter 3 gives concrete techniques — user stories, features, scenarios, and goal hierarchies — for eliciting and recording what all of these stakeholders actually need.

1.2.2 Dealing with Requirements Changes

Requirements *change*, and this is normal, not a failure. Users learn from early versions; markets move; regulations update. Because requirements will change, engineers focus on reducing the cost of accommodating change.

Two broad answers appear throughout the book:

1. **Process-level:** work in short iterations so that less new work accumulates before feedback arrives, limiting the likelihood and extent of rework and keeping the team aligned with its *latest* understanding rather than a stale specification (Chapter 2).
2. **Design-level:** structure the system so that a likely change touches one module, not twenty (Chapter 6).

> **Principle.** The cost of a change should be proportional to the *size* of the change, not to the *size* of the system. Achieving that is the central goal of good process and good architecture alike.

1.3 Software Is Intrinsically Complex

1.3.1 Sources of Complexity

Some complexity is **essential** — it is inherent in the problem. A tax-preparation program is complicated because tax law is complicated; no cleverness removes that. Other complexity is **accidental** — it comes from our tools, our choices, and our mistakes: tangled dependencies, unclear names, duplicated logic, leaky abstractions.[7]

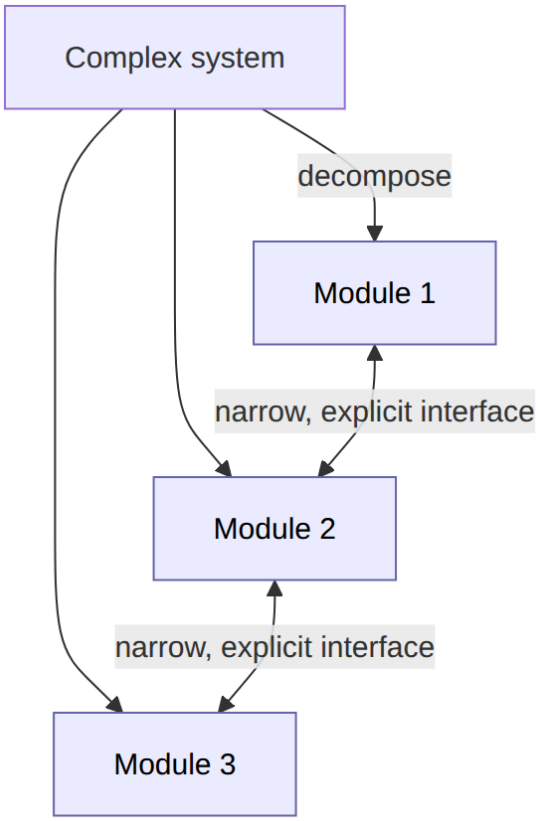
Good engineering cannot eliminate essential complexity, but it relentlessly attacks accidental complexity. Much of this book is about that: clear requirements and specifications (Chapters 3–5), design and patterns that package proven solutions (Chapters 6–7), reviews that catch needless complication (Chapter 9).

Software is also unusually complex among engineered artifacts for structural reasons:

- It is **invisible** — you cannot see its shape, so you rely on models and diagrams.[7]
- It is **discrete** — a one-character change can flip behavior completely; there is no "small error → small effect" safety net that continuous physical systems enjoy.
- It is **changed constantly** — a bridge is designed once; software is redesigned every sprint.

1.3.2 Architecture: Dealing with Program Complexity

The primary weapon against complexity is **decomposition**: split the system into parts small enough to understand, with **well-defined interfaces** between them, so that you can reason about one part without holding the others in your head. When the parts and their connections are chosen deliberately, we call the result the system's **architecture** (Chapters 6–7).



A good decomposition has **high cohesion** (each module does one thing well) and **low coupling** (modules depend on each other as little as possible, and only through their interfaces). These two ideas — introduced here, developed in Chapter 6 — are perhaps the most important in all of software design.

1.4 Defects Are Inevitable

Humans make mistakes; therefore software has defects. The engineering response is to accept this and build **multiple, overlapping** nets that catch defects early, when they are cheap to fix.

1.4.1 Fix Faults to Avoid Failures

It helps to separate three ideas that everyday speech blurs into "bug":

- A **mistake** (or *error*) is a human action that produces an incorrect result — a developer misreads a spec.
- A **fault** (the "defect" in the code) is the resulting flaw in the software — an off-by-one in a loop.
- A **failure** is the observable event when the running system deviates from correct behavior — the report prints the wrong total.

A fault is *latent* until some execution triggers it into a failure. This chain — mistake → fault → failure — explains why we attack defects at every stage: better requirements and reviews reduce **mistakes**; static checking and

inspection find **faults** before they run; testing provokes **failures** in a safe setting before users do. No single technique suffices, which is why Chapters 9 and 10 are complementary.

1.4.2 Introduction to Testing

Testing runs the software on chosen inputs and checks the outputs against what we expect. It cannot prove the absence of defects — as Dijkstra, a pioneer in computing and mathematics, observed, testing shows the *presence* of bugs, never their absence[8] — but a well-designed test suite makes it *unlikely* that a serious fault survives, and it makes change safe by catching regressions. Testing is important enough that Chapter 10 is devoted to it, including how to decide when you have tested *enough* (coverage).

1.4.3 Black-Box and White-Box Testing

Two complementary viewpoints run through all of testing:

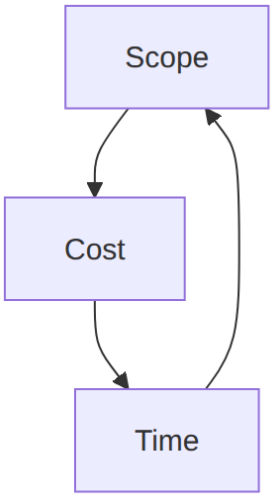
- **Black-box (specification-based)** testing chooses inputs from the *specification*, ignoring the code. You test *what the system should do*: valid and invalid inputs, boundaries, and equivalence classes of behavior. Its strength is that it finds *missing* logic and stays valid when the code is rewritten.
- **White-box (structure-based)** testing chooses inputs by looking at the *code*, aiming to exercise its statements, branches, and paths. Its strength is finding code that the specification-based tests never reach.

Neither subsumes the other, and Chapter 10 shows how to combine them and how to *measure* their thoroughness with coverage criteria.

1.5 Balancing Constraints: The Iron Triangle

Every project runs under three linked constraints:

- **Scope** — how much it does (features, quality).
- **Cost** — how much effort/money it consumes.
- **Time** — when it must ship.



1.5.1 Scope. Cost. Time. Pick Any Two!

These constraints are coupled: you cannot freely fix all three. Want more **scope** at the same **time**? Pay more **cost** (people, tools). Want to ship sooner without more cost? Cut **scope**. The classic quip — *"good, fast, cheap: pick two"* — captures the reality that **quality is not free**, and that a manager who demands more scope, sooner, for less is mistaking innumeracy for ambition.

Agile methods make a deliberate choice here: they **fix time and cost** (a fixed-length iteration with a fixed team) and let **scope flex** — the aim is to ship *something* valuable on the date, even if not everything. Plan-driven methods more often fix scope and flex time. Chapters 2 and 4 return to this trade-off with concrete estimation and prioritization techniques (story points, MoSCoW, value/cost/risk). And modern software increasingly ships as a **continuously updated service** rather than a boxed product — Chapter 14 examines how that reshapes the triangle, when releasing becomes a routine decision instead of a once-a-year event.

> **Pitfall.** Silently absorbing a scope increase by working nights ("crunch") hides the trade-off instead of resolving it. It borrows against future quality — the debt is repaid, with interest, as defects and burnout.

1.6 Social Responsibility

Software runs medical devices, cars, elections, and financial systems. Engineering decisions are therefore **ethical** decisions, and "I was just implementing the spec" is not a defense. This section grounds that claim in three cases.

1.6.1 Case Study: The Volkswagen Emissions Scandal

In 2015, regulators discovered that Volkswagen diesel vehicles contained a **defeat device**: software that detected when the car was being emissions-tested (from steering, speed, and time patterns) and switched to a cleaner,

lower-performance mode *only during the test*. On the road, the cars emitted many times the legal limit of nitrogen oxides.[9]

The software worked *exactly as designed* — and that is the problem. This was no mere testing failure — it was a *requirement* that should never have been implemented. Engineers wrote that code. **A system can satisfy its specification and still be ethically wrong.** A professional must sometimes refuse to build what they are asked to build.

1.6.2 The ACM Code

The profession has written norms. The **ACM Code of Ethics and Professional Conduct** and the joint **ACM/IEEE-CS Software Engineering Code of Ethics** state, among other principles, that software engineers shall act in the **public interest**, produce work of **high quality**, be **honest** about their work and its limitations, and maintain **competence**.[10] [11] These are not decoration: they give an engineer a shared, citable basis for saying *no*, and for raising concerns without it being merely personal opinion.

1.6.3 Case Study: The Therac-25 Accidents

Between 1985 and 1987, a radiation-therapy machine, the **Therac-25**, delivered massive radiation overdoses to at least six patients, several fatally.[12] The causes were a textbook of engineering failures: a **race condition** in concurrent control software that surfaced only when an operator typed quickly; **removal of the hardware interlocks** that earlier models used as a safety backstop, leaving software as the sole safeguard; **reused code** never designed for this configuration; misleading error messages; and a development culture that assumed the software was correct and dismissed early field reports.[12]

The Therac-25 is the canonical warning that in safety-critical systems, software defects kill, and that **process, testing, and honest incident response are matters of life and death** — not bureaucratic overhead.

1.6.4 Lessons for Software Projects

Across these cases, recurring lessons:

1. **Build the right thing, not only the thing right** — challenge requirements that would cause harm to users or the public. (Chapter 10 returns to this distinction as validation versus verification.)
2. **Don’t rely on a single safeguard** — defense in depth (interlocks *and* software checks; reviews *and* tests).
3. **Take field reports seriously** — a dismissed anomaly is a failure waiting to recur.
4. **Concurrency and reuse are risk multipliers** — treat them with extra rigor.
5. **You are accountable for what you ship**, regardless of who asked for it.

1.7 Conclusion

Software engineering exists because software is built by teams, for changing needs, at a scale and complexity that defeat improvisation, with defects that are inevitable and sometimes dangerous. The rest of this book is organized as a set of responses to exactly those pressures:

- **Process** (Ch. 2) coordinates the team and embraces change.
- **Requirements** (Ch. 3–5) discover and pin down *what to build*.
- **Design and architecture** (Ch. 6–7) tame complexity so change stays cheap.
- **Version control** (Ch. 8) gives every change a name, a history, and an owner — the substrate every later discipline builds on.
- **Static checking and testing** (Ch. 9–10) catch the inevitable defects early.
- **Security** (Ch. 11) makes the software resist attack, from the code to the supply chain it is built from.
- **Metrics** (Ch. 12) tell us, with evidence rather than opinion, how we are doing.
- **AI across the lifecycle** (Ch. 13) re-examines every one of those disciplines as coding agents take over more of the typing.
- **Delivery** (Ch. 14) carries a verified change the last mile, into production, safely.
- **Maintenance and evolution** (Ch. 15) keep the system alive and changeable through the years after launch — where most engineering effort actually goes.
- **A team project** (Appendix A) puts it all into practice.

A word about the code you will meet along the way. Concepts come first in this book and code second, but from Chapter 2 onward nearly every code-shaped idea is also shown as a short, runnable example — and each example appears in six languages — **Python, Java, JavaScript, Go, Ruby, and TypeScript** — plus a language-agnostic **Generic** tab written in pseudocode, for when you want the idea without any one language's syntax. They sit behind a row of tabs. Pick your language once and every example on every page follows; you can also open any page with `?lang=go` (or `java`, `javascript`, `python`, `ruby`, `typescript`, `generic`) to pre-select one. The tabs run alphabetically, so **Generic** leads the row; the **Ruby** tab is selected by default until you pick another — and the examples are written to read cleanly even in a language you have never used.

The others are worth your attention even so, because some lessons only appear in the contrast: information hiding looks different as a Java interface, a Go implicit interface, Ruby duck typing, and a TypeScript `interface`; the same unit tests read differently in pytest, JUnit, `node:test`, `go test`, and Minitest — and each difference says something about the language's design. Every snippet is a real program, kept as a runnable file with a test in the book's repository and exercised by continuous integration, so the examples cannot silently rot. And when an example is *supposed* to fail — a type fault the compiler rejects in Chapter 9 — the error message printed beside it is the compiler's actual output, not a paraphrase.

Keep the four cross-cutting principles in view as you read: *software is complex; requirements change; defects are inevitable; teams need coordination.* Nearly every technique in this book earns its place by answering one of

them.

Sources

[1] NASA, *Margaret Hamilton* (NASA Science biography, n.d.). science.nasa.gov.

[2] IEEE Computer Society, *Margaret Hamilton: First Software Engineer* (2018). computer.org.

[3] Don Eyles, *Tales from the Lunar Module Guidance Computer*, AAS paper 04-064, 27th AAS Guidance and Control Conference (2004). doneyles.com.

[4] Gordon E. Moore, *Cramming More Components onto Integrated Circuits*, Electronics 38(8), 1965; reprinted in IEEE Solid-State Circuits Society Newsletter (2006). doi.org.

[5] Peter Naur & Brian Randell (eds.), *Software Engineering: Report on a Conference Sponsored by the NATO Science Committee, Garmisch, Germany, 7–11 Oct. 1968* (1969). cs.ncl.ac.uk.

[6] David Farley, *Modern Software Engineering: Doing What Works to Build Better Software Faster* (Addison-Wesley, 2022). informit.com.

[7] Frederick P. Brooks, Jr., *No Silver Bullet: Essence and Accidents of Software Engineering*, Computer 20(4), pp. 10–19 (1987). doi.org.

[8] Edsger W. Dijkstra, *Notes on Structured Programming*, EWD249, p. 7 (1970). cs.utexas.edu.

[9] U.S. Environmental Protection Agency, *Notice of Violation to Volkswagen AG* (Sept. 18, 2015). epa.gov.

[10] ACM, *ACM Code of Ethics and Professional Conduct* (2018). acm.org.

[11] ACM/IEEE-CS Joint Task Force, *Software Engineering Code of Ethics and Professional Practice, v5.2* (1999). computer.org.

[12] Nancy G. Leveson & Clark S. Turner, *An Investigation of the Therac-25 Accidents*, Computer 26(7), pp. 18–41 (1993). doi.org.

- **Key takeaways** are summarized above in §1.7.
- Continue to the Exercises.
- Go deeper with the Open Resources for this chapter.

Chapter 1 — Exercises

Exercises are graded by depth: **[warm-up]** checks understanding, **[analysis]** asks you to reason.

Concepts

1. **[warm-up]** In your own words, give one difference between *programming* and *software engineering*, and one example task that belongs to each.
2. **[warm-up]** Define *mistake*, *fault*, and *failure*, and give a single concrete scenario (a few sentences of a story) that contains all three in sequence.
3. **[warm-up]** Distinguish *essential* from *accidental* complexity. Give one example of each from a system you have used or built.

Analysis

4. **[analysis]** The “iron triangle” says scope, cost, and time are coupled. A manager says: “Add these five features (more scope), ship two weeks earlier (less time), and don’t hire anyone (same cost).” Explain, using the triangle, why at most two of these three demands can be met — and only by breaking the third — and propose two honest options you could offer instead.
5. **[analysis]** Agile methods “fix time and cost and let scope flex.” Describe a kind of project where that trade-off is *wrong* — where scope must be fixed and time should flex instead — and justify it.
6. **[analysis]** Re-read the two-companies story (§1.1.2). Identify the specific moment at which Company A’s velocity begins to collapse, and name the two engineering disciplines from later chapters that would most have helped, and why.
7. **[analysis]** Black-box testing “stays valid when the code is rewritten,” while white-box testing “finds code the spec never reaches.” Construct a small example function where a black-box suite would pass but a serious fault remains that only white-box testing would catch.

Ethics

8. **[analysis]** The Volkswagen defeat device “worked exactly as designed.” Explain why better testing could not have prevented this scandal, and which principle of the ACM/IEEE code was most directly violated.
9. **[analysis]** List the *defense-in-depth* failures in the Therac-25 case. For each, name a safeguard (from anywhere in this book) that could have provided a second layer.
10. **[analysis]** You are asked to implement a feature you believe misleads users but is legal. Using the ACM code as your basis, script (in ~150 words) how you would raise the concern with your team and manager.

Chapter 1 — Open Resources

Free, open-licensed, or freely accessible materials that reinforce this chapter. Types: 📘 book/book chapter · 🎓 course · 📄 primary source · 🎥 video. Licenses vary and are noted where known.

What software engineering is

- 🎓 **UC Berkeley — Engineering Software as a Service (ESaaS), ch. 1** — saasbook.info. Motivates SE and agile from first principles.
- 📘 **MIT 6.031 Software Construction, "Reading 1"** — web.mit.edu/6.031. Frames all of SE around three goals: *safe from bugs, easy to understand, ready for change.*
- 📄 **SWEBOK Guide v4 (IEEE Computer Society)** — computer.org/education/bodies-of-knowledge. The discipline's knowledge areas at a glance.
- 📘 D. Farley, *Modern Software Engineering* (Addison-Wesley, 2022) — informit.com. The engineer-as-scientist framing cited in §1.1.3. Commercial book (not open access).

Origins of the discipline (used in §1.1.3)

- 📄 **NASA — Margaret Hamilton** biography — science.nasa.gov. Her leadership of the MIT Instrumentation Laboratory's Apollo flight-software team. Free to read.
- 📄 **IEEE Computer Society — "Margaret Hamilton: First Software Engineer"** (2018) — computer.org. Hamilton, in her own words, on why she pushed the term "software engineering". Free to read.
- 📄 D. Eyles, **"Tales from the Lunar Module Guidance Computer"** (AAS 04-064, 2004) — doneyles.com. First-hand account of the Apollo 11 1201/1202 alarms and the Executive's priority scheduling, by one of the programmers. Free to read.
- 📄 P. Naur & B. Randell (eds.), **NATO Software Engineering Conference report, Garmisch 1968** (1969) — cs.ncl.ac.uk PDF. The conference that named the "software crisis". Free to read (hosted by Brian Randell).
- 📄 G. Moore, **"Cramming More Components onto Integrated Circuits"** (*Electronics*, 1965) — doi.org. The original Moore's Law paper (official IEEE reprint catalog page; PDF may require access).

Complexity and defects

- 📄 F. Brooks, *No Silver Bullet* (essence vs. accident), *IEEE Computer* 20(4), 1987 — doi.org (official abstract page); also widely available as a PDF via library and university course pages.

- 📄 E. W. Dijkstra, **“Notes on Structured Programming”** (EWD249, 1970) — cs.utexas.edu EWD archive. Source of “testing shows the presence of bugs, never their absence” (p. 7). Free to read.
- 📘 **MIT 6.031** readings on *static checking* and *testing* develop the fault-vs-failure and black-box-vs-white-box distinctions used here.

Professional responsibility (used in §1.6)

- 📄 **ACM Code of Ethics and Professional Conduct** — acm.org/code-of-ethics.
- 📄 **Software Engineering Code of Ethics (ACM/IEEE-CS)** — computer.org/education/code-of-ethics.
- 📄 N. Leveson & C. Turner, **“An Investigation of the Therac-25 Accidents”**, *IEEE Computer* 26(7), 1993 — PDF. The definitive account.
- 📄 **Volkswagen “defeat device”** — U.S. EPA Notice of Violation (Sept. 2015), epa.gov. Primary regulatory source.

License note

Linked resources remain under their own licenses; this page is CC BY-SA 4.0.

Chapter 2 — Software Development Processes

> **Where we are.** Chapter 1 argued that software engineering is what you need when *other people* depend on your program: a team must coordinate, requirements will change, complexity will accumulate, and defects are inevitable. A **development process** is the team's standing answer to all four pressures at once — the agreed way you turn ideas into working, changeable software without stepping on each other. This chapter surveys the main families of process, from lightweight agile frameworks to heavier plan-driven models, and gives you a way to *choose* rather than cargo-cult.

A process is not paperwork. It is the set of habits that decides, on an ordinary Tuesday, who talks to the customer, when code gets reviewed, how a half-finished feature gets integrated, and what happens when someone finds a bug. Teams that never *choose* a process still *have* one — an accidental, undocumented one — and it is usually the most expensive kind. This chapter gives you named, deliberate alternatives and, more importantly, the judgment to pick among them.

2.1 Processes and Values Guide Development

2.1.1 What Is a Process?

A **software development process** is a repeatable way of organizing the activities that turn an idea into deployed, maintained software: understanding what to build, designing it, writing it, checking it, releasing it, and changing it. Every process has to account for the same core activities — the differences are in their *order*, their *size*, and how often you revisit them.

It helps to separate two things that beginners conflate. A **process model** is the abstract shape: "work in two-week iterations, each producing shippable software" or "gather all requirements, then design, then build." A **process** — the real one your team lives — is that model plus a thousand local decisions: which chat channel bugs go in, how long a code review may sit, whether Friday afternoon is for demos. A process model provides a basic structure that each team adapts to its project.

> **Principle.** A good process makes the *right thing* the *easy thing*. If your process only works when everyone is disciplined and heroic, it is not a process — it is a wish.

Why bother naming and choosing a process at all? Because the alternative is that every recurring decision gets re-litigated every time it comes up, and because a shared process is how a team of people acts like a single competent engineer instead of a crowd. A process also encodes *feedback loops* — the points at which you find out you were

wrong. As you will see repeatedly in this chapter, the single biggest thing distinguishing strong processes from weak ones is **how early and how often they expose mistakes**.

2.1.2 Two Development Cultures: Plan versus Grow

Underneath the alphabet soup of methods lie two genuinely different philosophies about how software should come into being.

The **plan-driven** culture treats software like a building. You decide what you are going to build, in detail, *before* you build it. Requirements are elicited and frozen, a design is drawn, and construction follows the design. Correctness means "conforms to the spec." This culture prizes predictability, documentation, and up-front analysis; it is strongest when the problem is well understood, the cost of a mistake is high, and change is rare — avionics, medical devices, contracts with fixed deliverables.

The **grow-it** culture treats software like a garden or an organism. You start something small that works, then evolve it through many small changes, each validated against reality before the next. Requirements are *discovered* through working software, not frozen in advance. Correctness means "does something useful for a real user, and can keep changing." This culture prizes feedback, working code over documents, and adaptability; it is strongest when the problem is poorly understood, the market is moving, and the cost of being wrong for two weeks is low.

Neither culture is "right." They are tuned for different risk profiles. Most of this chapter's methods — Scrum, XP, the spiral — are attempts to get the *feedback* benefits of growing while keeping *enough* of the discipline of planning. The waterfall and V models sit at the plan-driven end; you will study them not because you should copy them wholesale but because understanding *why* they struggle is the fastest route to appreciating what the agile methods are for.

2.1.3 Role Models: Unix Culture and Agile Values

Two well-documented value systems shaped modern practice, and both are worth internalizing because they are *values*, not procedures — they tell you what to prefer when the manual runs out.

The **Unix philosophy**, distilled from the culture that built the Unix operating system, values small tools that each do one thing well and compose through clean interfaces. Its maxims — *write programs that do one thing well; expect the output of one program to become the input of another; build a prototype as soon as possible; prefer clarity over cleverness* — are really a philosophy of **decomposition and iteration**.[1] The connection to process is direct: build small, integrate through narrow interfaces, and get something running early so reality can correct you. Those instincts reappear in every good modern process.

The **Agile Manifesto** (2001) crystallized the grow-it culture into four value statements.[2] In each, both sides have worth, but the left is valued *more*:

- **Individuals and interactions** over processes and tools.

- **Working software** over comprehensive documentation.
- **Customer collaboration** over contract negotiation.
- **Responding to change** over following a plan.

Read carefully: this is a statement about what to trust when the two sides conflict, not a case against planning or documentation. If your beautiful plan and your working software disagree, believe the software. If a document and a conversation with the customer disagree, have the conversation. The manifesto's twelve supporting principles push further: deliver working software frequently, welcome changing requirements even late, maintain a sustainable pace, and reflect regularly on how to improve.[3] Every agile framework in this chapter is one concrete way to live those values.

> **Principle.** Values outrank rules. A team that follows Scrum's ceremonies to the letter while ignoring "responding to change" has missed the point more completely than a team that improvises but keeps the feedback loops tight.

2.1.4 Selecting a Process Model

There is no universally best process, only a best process *for this project, this team, this risk profile*. A handful of questions will steer you most of the way:

- **How well is the problem understood?** Poorly understood problems reward iteration and early feedback; well-understood ones tolerate up-front planning.
- **How likely and how costly is change?** Volatile requirements favor short iterations; stable ones favor plan-driven flow.
- **What does a mistake cost?** A pacemaker's software and a marketing microsite live at opposite ends; the former justifies heavy verification, the latter does not.
- **How big and how distributed is the team?** Small co-located teams can lean on conversation; large or scattered teams need more explicit coordination.
- **What does the customer relationship allow?** Fixed-price contracts push toward plan-driven scope; ongoing partnerships enable collaboration and change.

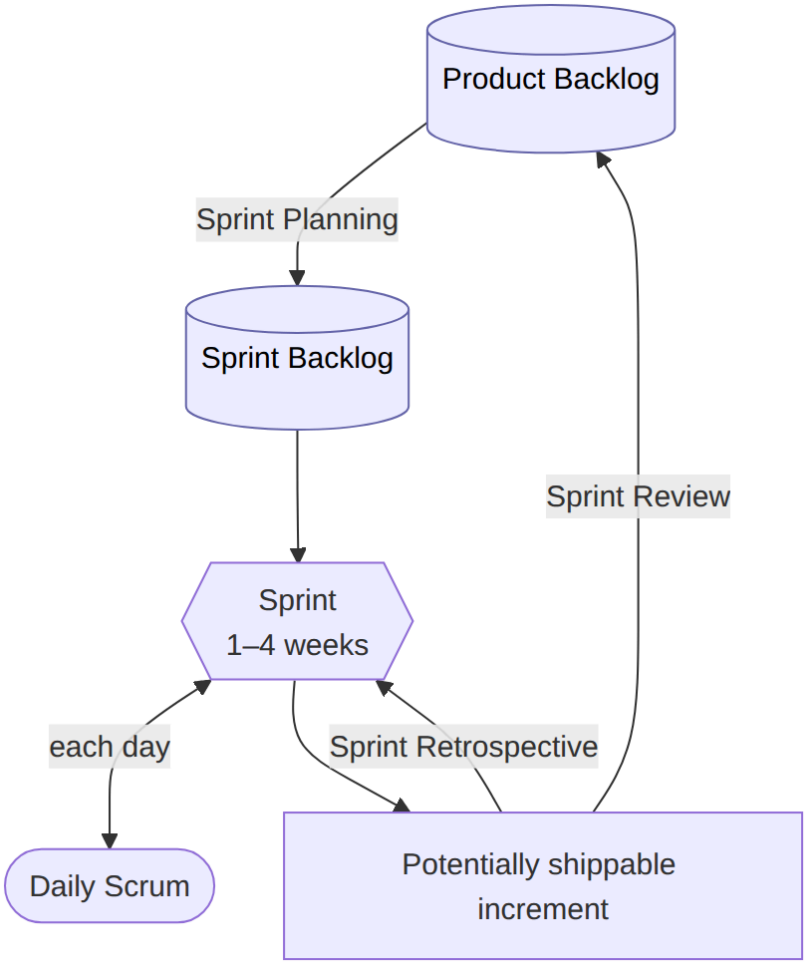
Most real teams end up **hybrids**: agile iterations wrapped in enough up-front architecture and enough documentation to satisfy their domain. Choose the process model that best *fits* the project's risks and constraints. Keep the four pressures from Chapter 1 in view — complexity, change, defects, coordination — and pick the process that best answers the ones that dominate your project.

2.2 Structuring Teamwork: The Scrum Framework

2.2.1 Overview of Scrum

Scrum is one of the most widely used frameworks for organizing agile teamwork. It is deliberately minimal: it does not tell you how to write code, design modules, or test. It tells you how to *organize the team's time and decisions* so that you deliver working software in short, regular cycles and improve as you go. You supply the engineering practices (Section 2.3 covers a strong set); Scrum supplies the rhythm.

The heartbeat of Scrum is the **sprint**: a fixed-length iteration, usually one to four weeks, that produces a potentially shippable increment of the product.[4] Everything else in Scrum exists to plan a sprint, keep it on track, ship its result, and learn from it.



The diagram shows the loop you repeat every sprint: pull the highest-value items from the product backlog into a sprint backlog, work them through the sprint with a daily check-in, produce an increment, review it with stakeholders (which reshapes the backlog), and reflect on the process itself. Scrum organizes work as a repeating

cycle of planning, delivery, review, and adaptation, and each turn of the cycle gives you a fresh chance to correct course based on real, working software.

2.2.2 Scrum Accountabilities

The 2020 *Scrum Guide* defines three **accountabilities** — the Product Owner, the Scrum Master, and the Developers — within a single Scrum Team.[4] (Earlier versions called these "roles"; the 2020 edition prefers *accountabilities* to stress that they describe responsibility, not job titles.)[5] Confusing them is a common source of dysfunction.

The **Product Owner** owns *what* and *why*. They represent the customer and stakeholders, maintain the product backlog, and — crucially — decide priority. When two features compete, the Product Owner breaks the tie. A single person holds this accountability so that the team gets one coherent voice about value, not a committee.

The **Developers** (the people building the product — programmers, testers, designers, and anyone else doing the work) own *how*. They estimate, design, build, and test. In Scrum they are **self-managing**: no one outside the group assigns tasks to individuals within it.[4] The team collectively commits to a sprint goal and figures out how to meet it.

The **Scrum Master** owns *the process itself*. They are not a project manager who assigns work; they are a coach and an obstacle-remover. They help the team live the framework, facilitate the events, and clear the impediments the team cannot clear alone — a flaky test environment, an unresponsive stakeholder, an unrealistic external deadline. A good Scrum Master's success is measured by the team needing them less over time.

> **Pitfall.** When the Scrum Master or a manager starts assigning tasks to individuals and tracking their personal output, self-organization dies and Scrum degrades into micromanagement with extra meetings. The team's *collective* ownership of the sprint goal is what makes the framework work.

2.2.3 Scrum Events

Scrum's events are the framework's feedback loops made concrete. Each has a purpose; running them as empty ritual is a classic failure.

- **Sprint Planning** opens the sprint. The team and Product Owner agree on a **sprint goal** and select backlog items the team believes it can complete, breaking them into a plan of work — the sprint backlog. The output answers two questions: *what* will we deliver, and *how* will we approach it.
- The **Daily Scrum** (or standup) is a short, time-boxed check-in — fifteen minutes — where the Developers synchronize and re-plan the next day's work toward the sprint goal.[4] It is for the *team*, not a status report to a manager. The useful frame is "are we still on track for the goal, and what is in our way?" rather than "what did I do yesterday" theater.

- The **Sprint Review** closes the sprint's *product* loop. The team demonstrates the working increment to stakeholders and gathers feedback, which flows back into the product backlog. This is where requirements get corrected by contact with reality.
- The **Sprint Retrospective** closes the sprint's *process* loop. The team reflects on how it worked — what to keep, what to change — and commits to one or two concrete improvements for next sprint. A team that skips retrospectives gives up its built-in way of getting better.

2.2.4 Scrum Artifacts

Scrum has three artifacts, each paired with a commitment that keeps it honest.[4]

The **Product Backlog** is the single, ordered list of everything that might be worth doing to the product — features, fixes, improvements. It is never "done"; it evolves as you learn. Its commitment is the **Product Goal**, the longer-term objective the backlog serves.

The **Sprint Backlog** is the subset the team pulls in for the current sprint, plus the plan for delivering it. It is owned by the Developers and updated daily. Its commitment is the **Sprint Goal**.

Some teams — especially XP-influenced ones — keep a third scope between those two: the **release backlog**, the subset of the product backlog targeted at a specific release. It sits between the product backlog (everything the product might ever do) and the sprint backlog (what the team is doing *this* iteration), and it is the natural artifact of release planning: you group stories by the release that will carry them, so everyone can see what "version 2.0" actually means without wading through the whole product backlog.

The **Increment** is the sum of completed backlog items — actual working software that meets the team's **Definition of Done**, a shared, explicit checklist of what "finished" means (compiled, reviewed, tested, documented, integrated). The Definition of Done is what stops "done" from meaning "done on my machine, probably, mostly."

> **Principle.** A backlog item is not done because someone wrote the code. It is done when it meets the Definition of Done. A weak Definition of Done is how teams accumulate a pile of "finished" work that does not actually ship.

2.2.5 Summary

Scrum is a lightweight scaffold for teamwork: three accountabilities, a handful of events, three artifacts, all serving one idea — deliver working software in short cycles and use each cycle's feedback to steer. It says nothing about engineering technique, which is both its flexibility and its danger: Scrum with weak engineering practices produces increments that *look* done but rot. The next section supplies the practices that make each increment genuinely solid.

2.3 Agile Development: Extreme Programming

Where Scrum organizes the team's *time*, **Extreme Programming (XP)** prescribes the team's *engineering discipline*. XP takes practices that are known to be good — testing, code review, integration, simple design — and turns their dials up to "extreme": if testing is good, test constantly; if review is good, review continuously by programming in pairs; if integration is good, integrate many times a day.[6] The two frameworks are complementary, and teams often combine them.

2.3.1 Listening to Customers: User Stories

XP captures requirements as **user stories**: short descriptions of a feature told from the user's point of view, written to fit on an index card. A common template is *"As a <role>, I want <capability> so that <benefit>."* For example: *"As a front-desk clerk, I want to see unconfirmed appointments highlighted so that I can call those patients before the day fills up."*

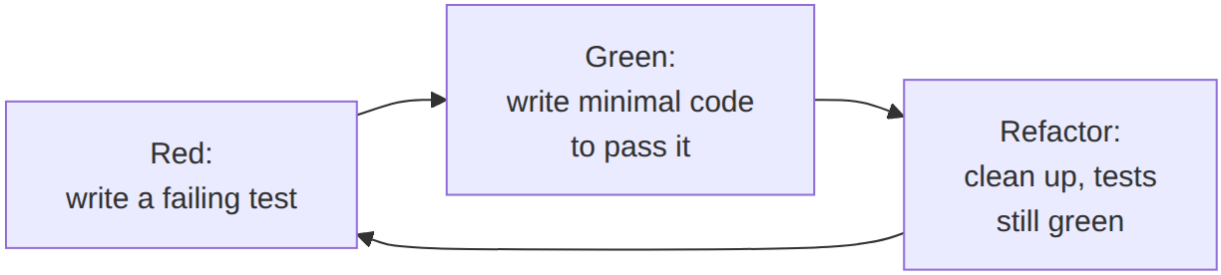
A story is deliberately *not* a specification. It is, in the well-worn phrase, a *promise to have a conversation*.[7] The card is a placeholder for a discussion between the developers and the customer about the details — details you will understand far better once you build the first version and show it. Stories keep requirements small, concrete, and grounded in user value, and their size makes them easy to prioritize, estimate, and slot into a sprint.

Good stories are often summarized by the acronym **INVEST**: Independent, Negotiable, Valuable, Estimable, Small, and Testable (each unpacked in §3.4.1).[8] That last property matters most for the next section: if you cannot state how you would *test* that a story is satisfied, you do not yet understand it well enough to build it.

2.3.2 Testing: Make It Central to Development

XP's most influential idea is that testing is not a phase that happens *after* development — it is *part of* development, written continuously, run constantly, and used as a design tool.

The signature practice is **test-driven development (TDD)**, a tight loop often summarized as **red–green–refactor**:



You write a small automated test for behavior that does not exist yet and watch it fail for the expected reason (*red*). That failure confirms three things at once: the test is actually being executed, its failure is connected to the missing behavior, and it can tell the current implementation apart from the one you intend to build. It also pins

down what “done” means for this slice. Then you write the least code that makes it pass (*green*). Then you improve the design while the test guards against breakage (*refactor*). Repeat in minutes-long cycles.[9]

One turn of this loop on the clinic app’s appointment slots starts *red* — the slot-overlap function does not exist yet, so the test run fails before a single assertion is checked (each language reports the missing function in its own way; see the comment in the code):

```
def test_overlapping_slots_conflict():
  assert slots_overlap(("09:00", "09:30"), ("09:15", "09:45"))

def test_back_to_back_slots_do_not_conflict():
  assert not slots_overlap(("09:00", "09:30"), ("09:30", "10:00"))

# Red: each test errors with "NameError: name 'slots_overlap' is not defined".
```

The least code that makes both tests pass turns the run *green*:

```
def slots_overlap(a, b):
  return a[0] < b[1] and b[0] < a[1]
```

This leads to a durable testing rule, one Chapter 10 returns to: *never trust a test you have never seen fail — and never trust a failure you have not understood.* A red phase that comes from a broken setup, a missing import, or an unrelated compiler error is not the red you are looking for.

Why work this way? Three reasons. First, tests written *first* are honest — they cannot be quietly shaped to match whatever the code happens to do. Second, the growing suite is a **regression net**: it lets you change code fearlessly, because a break announces itself immediately, which is what makes continuous refactoring safe. Third, writing the test first forces you to use your own interface before you build it, which surfaces awkward designs early. Testing, in XP, is less about catching bugs at the end and more about *driving* the design and *enabling* change — the very properties Chapter 1 said good engineering is for.

2.3.3 When to Design?

Plan-driven cultures design everything up front. XP takes the opposite stance: **design a little, all the time.** Because requirements are discovered, a comprehensive up-front design is largely a guess about a future you do not yet understand — and guesses about the future are a major source of accidental complexity.

XP’s answer rests on three habits. **Simple design**: build the simplest thing that could possibly work for the stories you have *now*, not the ones you imagine you might have later (resisting speculative generality is sometimes captured as YAGNI — *you aren’t gonna need it*).[10] **Refactoring**: continuously improve the design of existing

code without changing its behavior, so the structure keeps pace with your growing understanding. And the **test suite** that makes refactoring safe. Together these let the design *emerge*: it is always as good as your current understanding, and it improves as your understanding does, instead of being frozen at the moment you understood least.

This does not mean *no* thinking ahead. Architectural decisions that are expensive to reverse — the shape of your data, a choice of framework, a security model — still deserve deliberate up-front thought, a point the V and spiral models (Sections 2.5 and 2.7) take seriously. The XP claim is narrower and correct: do not *elaborate* a detailed design for requirements you have not yet validated.

> **Pitfall.** "Emergent design" is not a license to skip thinking. Without disciplined refactoring and a strong test suite, "we'll design as we go" becomes "we'll never design," and the codebase collapses into the tangle Chapter 1 called accidental complexity.

2.3.4 A Scrum+XP Hybrid

In practice, many strong teams run **Scrum for the process and XP for the engineering**. Scrum gives the cadence — sprints, backlog, review, retrospective — and the accountabilities that keep priorities and impediments owned. XP fills the sprint with the discipline that makes each increment genuinely done: TDD, pair or peer review, refactoring, and **continuous integration**, in which everyone merges into a shared mainline many times a day and an automated build-and-test pipeline verifies each merge.[11] (Chapter 14 dissects that pipeline stage by stage and follows it all the way to production.)

The combination is more than the sum of its parts. Scrum without engineering practice produces increments that pass the demo and fail in maintenance; XP without a coordinating rhythm can drift without clear priorities or stakeholder feedback. Together, Scrum's *what and when* meets XP's *how well*, and the Definition of Done can honestly include "tested, reviewed, integrated" because the engineering practices actually deliver those properties.

2.3.5 Summary

XP treats testing, review, integration, and design as continuous activities rather than phases, turning known-good practices up to the point where they reshape how you work. Requirements arrive as user stories — promises to converse — and design emerges under the protection of an automated test suite. Paired with Scrum's cadence, XP's practices are how a team keeps each short increment both *valuable* and *solid*.

2.4 Limitations of Waterfall Processes

To appreciate why the industry moved toward iteration, you have to understand what it moved *away from*. The **waterfall** model runs the project as a single pass through a strict sequence of phases — requirements, design, implementation, verification, maintenance — with each phase completed and signed off before the next begins,

like water flowing down a series of steps and never back up. (One historical note before we critique it: this strict, single-pass model is the *textbook* waterfall. Royce's 1970 paper — the usual citation — presented the strict sequence and then argued it was risky and needed feedback and iteration.[12] The industry largely adopted the diagram and skipped the warning; this section critiques the model as it was actually practiced.)

Its appeal is real: it is simple to explain, easy to plan and bill against, and it front-loads the thinking. For well-understood problems with stable requirements, a waterfall-ish flow can work fine. The trouble is that most software is *not* well understood up front, and waterfall's structure hides that trouble until the most expensive possible moment.

2.4.1 The Perils of Big-Bang Integration and Testing

Waterfall's deepest flaw is *when* it finds out it is wrong. Because verification is a late phase, the pieces built in isolation are first brought together — **integrated** — near the end, and the system is first exercised as a whole near the end. This is **big-bang integration**: months of separately developed components meeting for the first time at once.

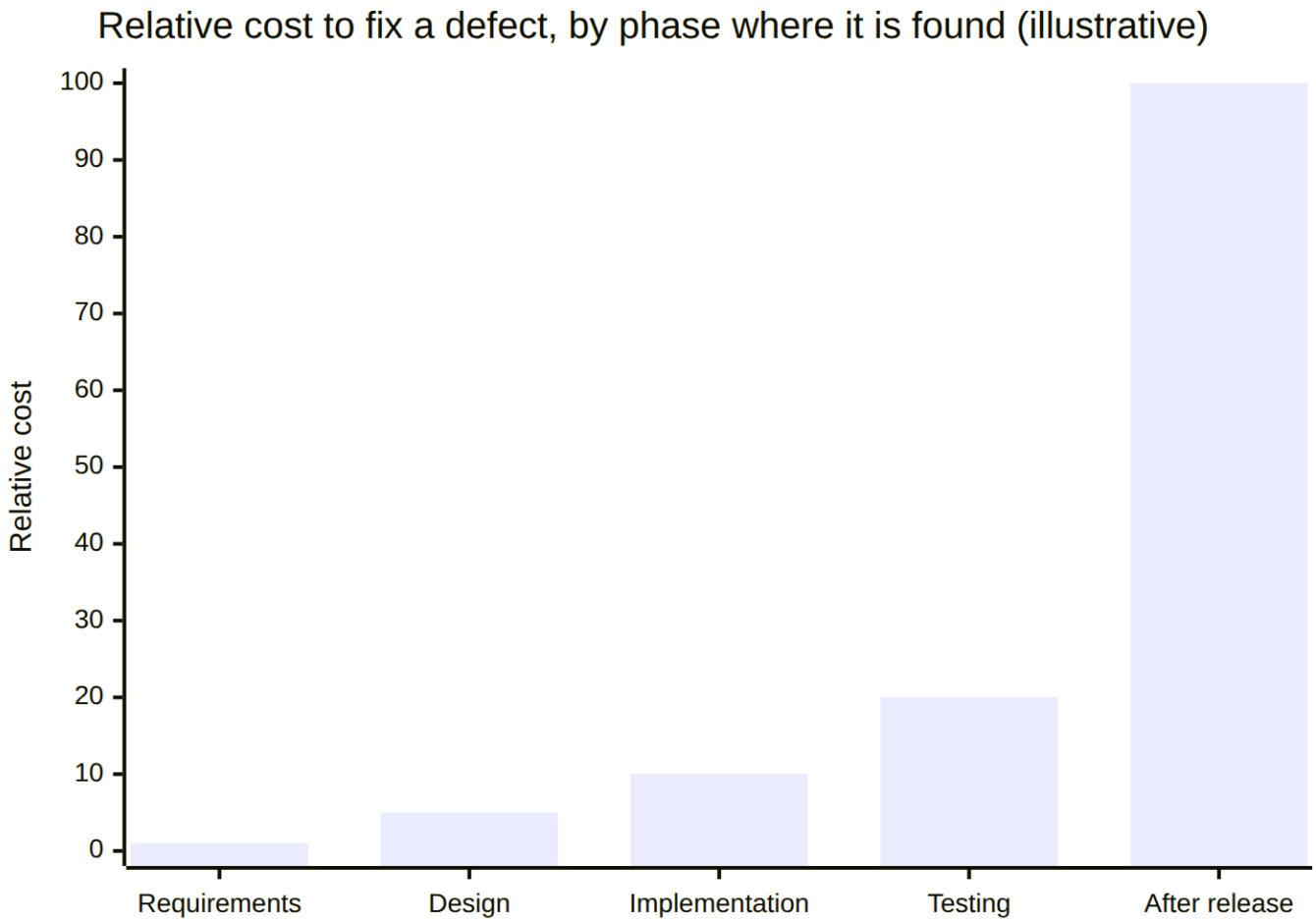
Two bad things happen at once. Integration surfaces every mismatched assumption between components simultaneously — module A expected meters, module B produced feet; A assumed the list was sorted, B never promised it — and untangling many interacting failures at once is far harder than fixing them one at a time. Meanwhile, testing at the end means that requirements mistakes made in month one — the truly expensive kind — are discovered in month ten, after everything built on them must be unwound. The project can look healthy right up to the integration phase, then fall apart, because *all* the risk was deferred to the end.

> **Pitfall.** In a strict waterfall, the moment you first learn whether the system actually works is also the moment you have the least time and money left to fix it. Deferring integration and testing does not remove risk; it concentrates it where it does the most damage.

The agile answer is precisely the opposite: **continuous integration** merges constantly so that at most one small change's worth of mismatch surfaces at a time, and **continuous testing** exercises the system every day so mistakes are caught while they are cheap. Both are direct reactions to the big-bang pathology.

2.4.2 The Waterfall Cost-of-Change Curve

The economic argument against waterfall is captured by the **cost-of-change curve**. Studies of real projects have long observed that the cost of fixing a defect or accommodating a change rises steeply the later it is discovered — a requirements error caught during requirements is nearly free to fix; the same error caught after release can cost orders of magnitude more, because so much work has been built on top of the mistake.[13]



The exact multipliers vary by study and context, and modern tooling flattens the curve considerably — automated tests and fast deploys make late fixes cheaper than they once were. But the *shape* is robust and it drives process design: if late changes are dramatically more expensive than early ones, then a process that *defers* discovery of problems to the end is economically backwards. Every iterative method is, in part, a strategy for sliding discovery leftward — finding your mistakes while they are still cheap.

2.4.3 Managing the Risks of Waterfall Processes

Waterfall is not always the wrong choice, and when constraints force something waterfall-like — a fixed-scope contract, a regulatory sign-off gate — you can blunt its risks:

- **Prototype the risky parts first.** Build a throwaway prototype of the hardest or least understood piece before committing to the full design, so you learn early.
- **Review at every phase boundary.** Since you cannot rely on late testing to catch early mistakes, inspect requirements and designs rigorously before building on them (Chapter 9).
- **Integrate incrementally even within a plan.** You can keep the phased structure while still bringing components together in stages rather than all at once.

- **Keep the phases short and iterate the whole thing.** A series of small waterfalls is much safer than one giant one — which is, in effect, how iterative development was born.

Notice that every mitigation pulls waterfall *toward* iteration. That is the tell: the fixes for waterfall's problems are the defining features of the methods that replaced it.

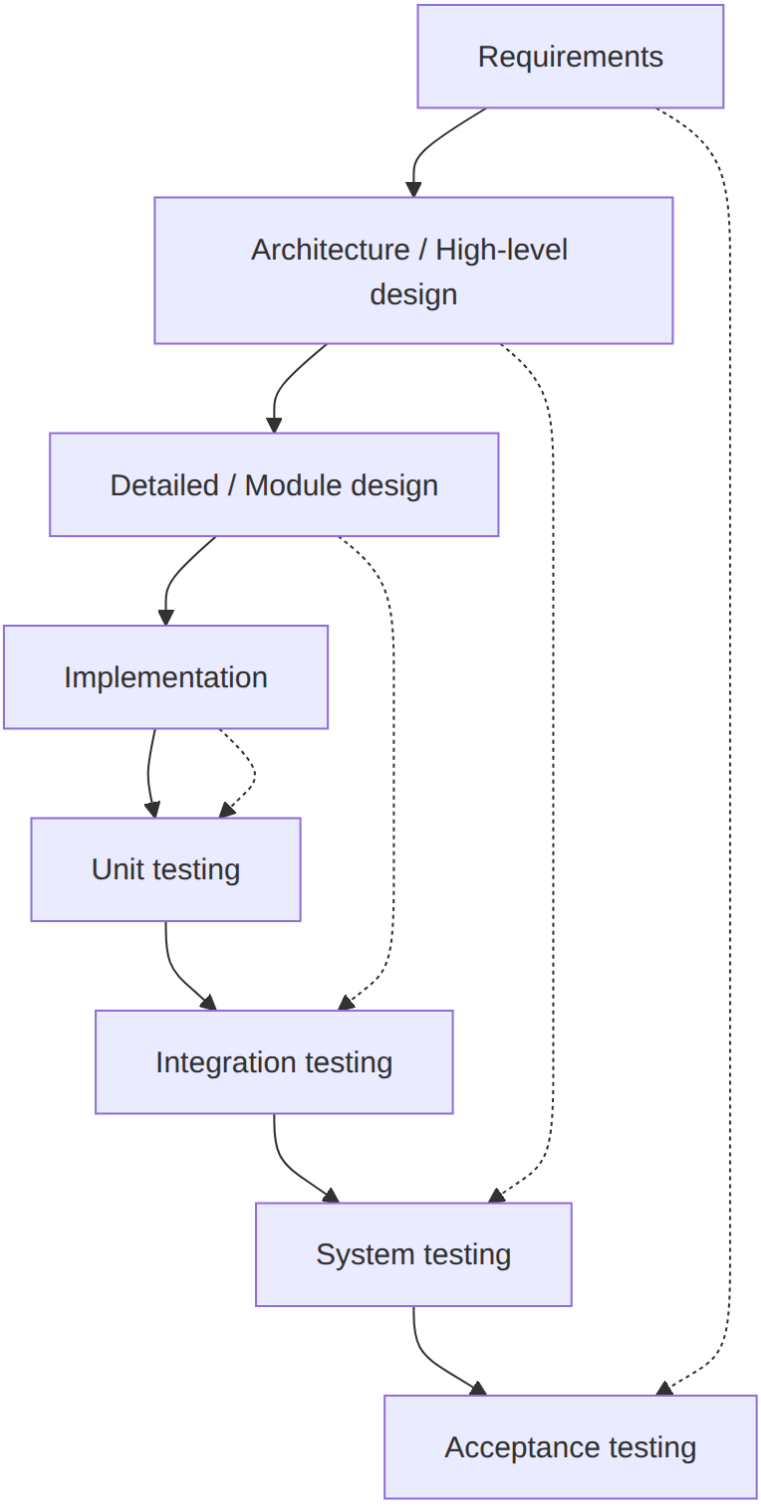
2.4.4 Summary

Waterfall's simplicity is seductive and its structure is treacherous: by deferring integration and testing to the end, it concentrates risk exactly where you can least afford it, and the cost-of-change curve makes late discovery of mistakes brutally expensive. The model is defensible mainly when requirements are genuinely stable and well understood, or when contract or regulation forces its shape — and even then, incremental integration and phase reviews are cheap insurance.

2.5 Levels of Design and Testing: V Processes

2.5.1 Overview of V Processes

The **V-model** is a refinement of waterfall that fixes one specific complaint: it makes testing a first-class citizen and ties every level of design to a matching level of testing. Draw the phases descending on the left and ascending on the right, forming a "V." The left arm is decomposition — from broad requirements down to detailed design; the right arm is integration and verification — from small units back up to the whole system.



The dashed lines carry the model's central insight: each design activity has a corresponding test activity that checks *whether that design was right*. The implementation is verified by unit tests; detailed module design by integration tests, which check that the pieces fit together the way the design claimed; the architecture by system

tests; and the original requirements by acceptance tests. This pairing means you plan the tests *while* you do the design, not as an afterthought — and it makes explicit that different mistakes are caught at different levels.

2.5.2 Levels of Testing from Unit to Acceptance

The right arm of the V names four levels of testing, each with a distinct question and scope. Getting these distinct is one of the most useful mental models in all of software engineering, and every level maps back to a decision made on the left arm.

- **Unit testing** checks the smallest pieces — a single function, class, or module — in isolation, usually by the developer who wrote them. Its question: *does this piece do what its detailed design says?* Unit tests are fast, numerous, and the backbone of TDD.
- **Integration testing** checks that units work *together* — that the interfaces and assumptions between modules actually line up. Its question: *do these pieces fit?* This is precisely the risk that big-bang integration mishandles; doing it incrementally is how you find interface mismatches one at a time.
- **System testing** exercises the *whole* integrated system against its specified behavior, including non-functional qualities like performance, security, and reliability. Its question: *does the complete system meet its specification?*
- **Acceptance testing** checks the system against the *customer's* needs, ideally with the customer involved. Its question: *did we build the right thing?* Passing system tests proves you built the system right; passing acceptance tests proves you built the right system — and the two can diverge when the specification itself was wrong.

These levels are not tied to waterfall; agile teams run all four, just continuously rather than in a final phase. The V-model's lasting contribution is its *vocabulary*: it gives you the ladder of testing levels that every process, iterative or not, has to climb.

2.5.3 Summary

The V-model repairs waterfall's neglect of testing by pairing each level of design with a matching level of verification, from unit up to acceptance. Its schedule is still essentially sequential and inherits waterfall's late-integration risks, but its ladder of testing levels — unit, integration, system, acceptance — is a permanent part of every engineer's toolkit, whatever process you adopt.

2.6 Additional Project Risks

Process choice is really **risk management** in disguise, so it pays to think directly about risk. A **risk** is a potential future problem: something that has not gone wrong yet but could, with some probability, cause some loss. Good teams do not just react to problems; they enumerate the likely ones in advance and attack the biggest first.

2.6.1 Rough Risk Assessment

You do not need a heavy methodology to reason about risk usefully. A rough assessment ranks risks by **exposure** — roughly, *probability* × *impact* — so you spend your limited attention where it matters.[14] A defect that is very likely but trivial ranks below one that is unlikely but catastrophic.

Common risk categories on software projects include:

- **Requirements risk:** you are building the wrong thing, or requirements will churn.
- **Technical risk:** an unproven technology, algorithm, or integration might not work.
- **Schedule and estimation risk:** the work is larger than estimated.
- **People risk:** key contributors leave, or the team lacks a needed skill.
- **External risk:** a dependency, vendor, or regulator behaves unexpectedly.

The list becomes useful when the team defines a concrete *response* for each important risk. For each significant risk you choose a strategy — **avoid** it (change the plan so it cannot occur), **mitigate** it (reduce its probability or impact, e.g. prototype the risky component early), **transfer** it (insurance, a vendor SLA), or knowingly **accept** it. The single most powerful risk-reduction technique in software is the one this whole chapter has been circling: **iterate**, so that you confront each risk early and cheaply instead of late and expensively.

2.6.2 An Iterative Project That Succeeded

Consider how iteration handles risk in practice. Imagine a team building a new patient-portal feature that lets people book their own clinic appointments — a genuinely uncertain project, since no one is sure patients will use self-service or that it will integrate cleanly with the existing scheduling system.

An iterative team attacks the uncertainty head-on. In the first two-week sprint they build a deliberately thin slice: one appointment type, no payments, booking against a copy of the real schedule, shown to a dozen real patients. That tiny increment answers the project's two biggest questions almost immediately — *will patients use it?* and *does the integration work?* — while the cost of being wrong is two weeks, not two quarters. Each subsequent sprint adds a slice (more appointment types, reminders, cancellations), and each sprint review lets real usage reshape the backlog. Requirements that looked important on paper get dropped when usage data contradicts them; unforeseen needs get added. The team did not predict the final system at the start. Short feedback cycles let it identify mistakes early, revise the product at low cost, and converge on a useful result. This is the grow-it culture paying off exactly as advertised.

2.6.3 A Troubled Project

Now a cautionary tale, drawn from a widely discussed episode in the history of web browsers. In the mid-1990s, a leading browser maker shipped a hugely successful early version of its product. For the next major version, rather

than evolving the working codebase, the organization undertook a sweeping rewrite of large parts of the system at once — a big, ambitious, mostly up-front effort to build the successor.[15]

The rewrite proved far harder and slower than planned. Because so much was rebuilt simultaneously, the familiar big-bang pathology appeared: integration was painful, the new system was unstable for a long stretch, and the release slipped badly while competitors kept shipping steady improvements to their own products. By the time the ambitious successor stabilized, the organization had bled schedule, quality, and market position. Commentators later treated the episode as a textbook argument against throwing away working software for a grand rewrite — you discard hard-won knowledge embedded in code that already handles a thousand edge cases, and you take on enormous integration and schedule risk all at once.[15]

The contrast with Section 2.6.2 shows how process choices affected the outcome. The successful project reduced risk by evolving a working system in small, validated steps. The troubled one *concentrated* risk into a single large leap — the same mistake, structurally, as big-bang integration and strict waterfall. Whether the risk is technical, schedule, or requirements, the pattern repeats: large, late-validated bets are dangerous; small, early-validated ones are far safer.

> **Principle.** Prefer evolving working software to rewriting it wholesale. A rewrite throws away embedded knowledge and takes on all its risk at once — exactly the concentration of risk that good process exists to avoid.

2.7 Risk Reduction: The Spiral Framework

2.7.1 Overview of the Spiral Framework

The **spiral model** is an explicitly **risk-driven** process framework.[16] Its central claim is that the *risks* of your particular project — not a fixed schedule — should decide what you do next. It is best understood as a meta-framework rather than a rival to waterfall or agile: at each turn it tells you *which* approach to apply based on where your biggest uncertainties lie.

You proceed in a series of loops, each spiraling outward as the project grows more concrete and more expensive. Every loop passes through four kinds of activity:

1. **Determine objectives, alternatives, and constraints** for this round — what are we trying to achieve, and how might we achieve it?
2. **Evaluate alternatives and identify and resolve risks** — this is the heart of the model. Find the biggest current risk and attack it, often by building a **prototype** or running an experiment to turn an unknown into a known.
3. **Develop and verify** the deliverable for this round.

4. **Plan the next round** — and, critically, decide whether to continue at all.

Because each loop starts by confronting the biggest remaining risk, the spiral front-loads learning about whatever is most likely to sink the project. A wildly uncertain project spends early loops on prototypes and feasibility studies; a maturing one settles into more waterfall-like build-and-verify loops. The model also builds in explicit *go/no-go* decisions: after each loop you may decide the risks are too high and stop, having spent only that loop's budget rather than the whole project's.

The spiral's spirit and the agile spirit are close cousins. Both iterate, both prize early feedback, both refuse to bet everything on an up-front plan. The spiral emphasizes *explicit risk analysis and prototyping* and suits larger, higher-stakes projects; agile methods emphasize *frequent working software and customer collaboration* and suit smaller, faster ones. Both are answers to the same question the whole chapter has asked: how do you find out you were wrong while it is still cheap to be wrong?

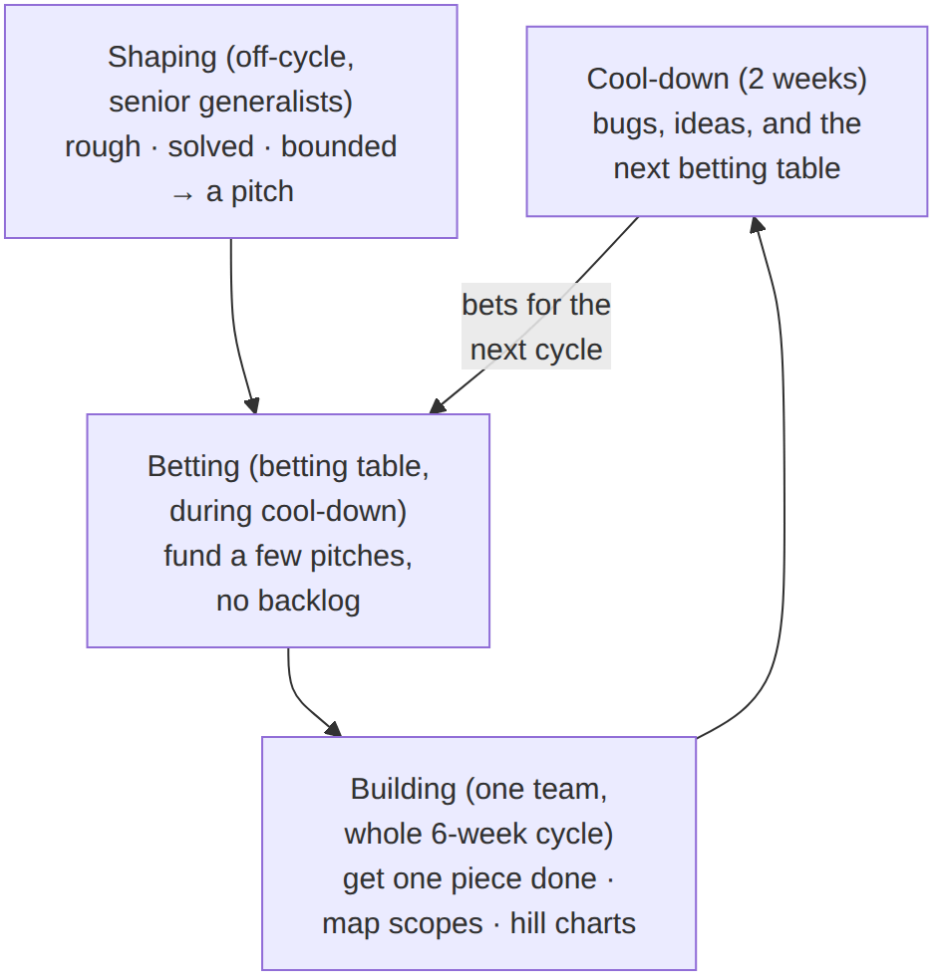
2.7.2 Summary

The spiral model makes risk the steering wheel: each loop identifies the biggest current uncertainty, resolves it (often by prototyping), delivers and verifies a piece, and decides whether to keep going. It generalizes both plan-driven and agile instincts into a single risk-first frame, and it is the clearest statement of this chapter's core idea — that a good process is, above all, a machine for reducing risk early.

2.8 Shape Up: Fixed Time, Variable Scope

Scrum, XP, and the spiral all iterate in short cycles and carry unfinished work forward on a backlog. **Shape Up** — the method Basecamp published as a free online book — keeps the fixed cadence but makes three sharp bets that set it apart: it fixes *time* and flexes *scope*, it refuses to keep a backlog, and it hands a team a whole shaped problem rather than a list of tasks.[17] It belongs in this chapter because it questions assumptions the other models share.

Shape Up runs in three phases that overlap across the calendar:



Shaping (what to build, and how much it's worth). Before any team is committed, senior people *shape* the work: they set an **appetite** — the fixed time the problem is worth (see §4.2.4) — and design a solution that is deliberately **rough** (leaves room for the builders' judgment), **solved** (the main elements are worked out, not vague), and **bounded** (it says explicitly what is *out* of scope). The output is a **pitch**: problem, appetite, solution sketch, rabbit holes to avoid, and no-gos. Shaping is where **rabbit holes** — unsolved design problems or untested technical assumptions — get found and removed *before* anyone commits.[18]

Appetites come in two standard batch sizes. A **small batch** is a designer and one or two programmers for one to two weeks; a **big batch** is that same small team for the full six-week cycle. An idea that will not fit even a big batch is not given more time — it is narrowed until it fits, because the appetite, not the idea, is fixed.[18]

Shaping itself follows four steps, in order:

1. **Set boundaries.** Decide how much time the raw idea is *worth* — the appetite — and what the problem actually is.
2. **Rough out the elements.** Sketch a solution at a level higher than wireframes, moving fast and exploring alternatives while the drawing is still cheap.

3. **Find risks and rabbit holes.** Hunt for holes and unanswered questions, then amend the solution and spell out the tricky spots before they can swallow a team.
4. **Write the pitch.** A formal write-up of problem, appetite, solution, rabbit holes, and no-gos, ready for the betting table.

Who does this work? A **shaper** is a technically literate generalist doing strategic design work. They need not be the team's best programmer — they may not write production code at all — but they must know what is cheap and what is expensive to build, or their sketches will make promises the appetite cannot keep.

Betting (deciding, without a backlog). During the two-week **cool-down** between cycles, a small **betting table** of senior people reviews the shaped pitches and *bets* on a few for the next cycle. A **bet** commits one team to one project for the whole cycle, uninterrupted, with the expectation of finishing.[19]

> **Principle — bets, not backlogs.** Shape Up keeps *no backlog*. Unchosen ideas simply lapse; if one really matters, it comes back and gets re-pitched. This trades the comfort of a tracked list for freedom from the "always behind" guilt and the grooming overhead of a backlog that only ever grows.

No backlog also implies an etiquette for new **raw ideas**. A raw idea usually receives a provisional response — *"interesting — maybe someday"* — rather than an on-the-spot yes or no. The team first *shapes* the idea enough to decide whether it deserves a bet; only shaped work can be bet on. Do not shut down an idea you do not understand, and keep a poker face even about ideas you love — visible enthusiasm commits you before anyone has checked for rabbit holes. Bugs get no privileged lane either; Shape Up handles them three ways: fix them during cool-down (that slack time exists partly for this), pitch a big bug at the betting table like any other project, or run an occasional dedicated **bug-smash** cycle where the whole team pays down accumulated defects.[19]

Building (hand over the whole problem). The team gets the pitch, *not* a task breakdown — "splitting the project into tasks up front is like putting the pitch through a paper shredder."[20] They define their own tasks and own how the pieces fit. Three building practices are worth borrowing regardless of your process:

- **Get one piece done.** Finish one *vertically integrated* slice early — front end wired to back end so *something works* in week one — instead of building all of one layer then all of the next (where nothing works until the end). Pick a first slice that is **core, small, and novel** so it proves the concept and kills the biggest uncertainty first.
- **Map the scopes.** Organize the project into **scopes** — parts that can be built, integrated, and finished *independently* — named by function, not by layer or person. Scopes become the shared language for status.
- **Hill charts for progress.** Track each scope as a dot on a hill: **uphill** is *figuring it out* (unknowns remain), **downhill** is *just execution* (all unknowns solved). A dot that stops moving is a raised hand. This exposes uncertainty in a way a to-do list cannot — see §12.3.

The circuit breaker. When the cycle ends, the project ships or it is *dropped* — it does **not** automatically get an extension. The most you can lose is one cycle. Extending is allowed only if what remains is genuine must-have work that is *entirely downhill*; any remaining uphill work means the shaping was wrong, so the project goes back to shaping rather than dragging on.[21]

Shape Up vs. Scrum. The contrast sharpens both. Scrum re-plans every 1–4-week sprint and rolls unfinished stories forward on a *backlog*; Shape Up makes a *one-shot bet* over a longer six-week cycle with *no* backlog and *no* auto-extension. Scrum layers daily standups, reviews, and retrospectives; Shape Up gives one integrated team full autonomy for the whole cycle and reads status from hill charts instead of ceremonies. Neither is universally right — but Shape Up is a clean demonstration of this chapter's theme that a process is a set of *bets about where your risk lives*.

2.9 Conclusion

Strip away the terminology and every process in this chapter is answering one question: **how do you find out you were wrong while being wrong is still cheap?** Plan-driven models — waterfall and the V — bet on getting the answer right up front and pay dearly when the bet fails, because they defer integration and testing to the end where the cost-of-change curve is steepest. Agile frameworks — Scrum for cadence, XP for engineering discipline — bet instead on short cycles that expose mistakes constantly, while the spiral makes that same instinct explicit by steering each loop toward the biggest remaining risk.

The practical takeaways connect straight back to Chapter 1's four pressures:

- **Coordination:** a named, deliberate process lets a team of people act like one competent engineer instead of a crowd; Scrum's accountabilities, events, and artifacts are one proven scaffold.
- **Change:** short iterations make change cheaper by limiting how much work can pile up before feedback arrives — shrinking the likelihood, and usually the size, of expensive rework — and by always adapting to your *latest* understanding.
- **Defects:** continuous testing and integration, structured by the V-model's levels of verification, catch faults while they are cheap instead of concentrating them in a final big bang.
- **Complexity:** simple design plus disciplined refactoring under a strong test suite lets structure keep pace with understanding, instead of freezing it at the moment you knew least.

There is no universally best process — only a best process for *this* project's risks, team, and constraints. An engineer should evaluate the project's risks, select a process that addresses them early, and adapt that process as conditions change.

Sources

[1] M. D. McIlroy's Unix maxims (1978), as collected in Eric S. Raymond, *The Art of Unix Programming*, ch. 1 (2003). catb.org.

[2] Kent Beck et al., *Manifesto for Agile Software Development* (2001). agilemanifesto.org.

[3] *Principles behind the Agile Manifesto* (2001). agilemanifesto.org.

[4] Ken Schwaber and Jeff Sutherland, *The Scrum Guide* (2020). scrumguides.org.

[5] Ken Schwaber and Jeff Sutherland, *Scrum Guide Revisions* (2020). scrumguides.org.

[6] Kent Beck with Cynthia Andres, *Extreme Programming Explained: Embrace Change*, 2nd ed. (2004). informit.com.

[7] Ron Jeffries, *Essential XP: Card, Conversation, Confirmation* (2001); the phrase "a promise for a conversation" is Alistair Cockburn's. ronjeffries.com.

[8] Bill Wake, *INVEST in Good Stories, and SMART Tasks* (2003). xp123.com.

[9] Kent Beck, *Test-Driven Development: By Example* (2002). informit.com.

[10] Martin Fowler, *Yagni* (2015). martinfowler.com.

[11] Martin Fowler, *Continuous Integration* (2024 revision). martinfowler.com.

[12] Winston W. Royce, *Managing the Development of Large Software Systems*, Proceedings of IEEE WESCON (1970). cs.umd.edu.

[13] Barry Boehm and Victor R. Basili, *Software Defect Reduction Top 10 List*, IEEE Computer 34(1) (2001), summarizing data reported in Barry W. Boehm, *Software Engineering Economics* (Prentice-Hall, 1981). cs.umd.edu.

[14] Barry W. Boehm, *Software Risk Management: Principles and Practices*, IEEE Software 8(1) (1991). cs.virginia.edu.

[15] Joel Spolsky, *Things You Should Never Do, Part I* (2000). joelonsoftware.com.

[16] Barry W. Boehm, *A Spiral Model of Software Development and Enhancement*, IEEE Computer 21(5) (1988). cse.msu.edu.

[17] Ryan Singer, *Shape Up: Stop Running in Circles and Ship Work that Matters* (Basecamp, 2019). basecamp.com/shapeup.

[18] Ryan Singer, *Shape Up*, Part One: Shaping, chs. 2–6 (2019). basecamp.com.

[19] Ryan Singer, *Shape Up*, Part Two: Betting, chs. 7–9 (2019). basecamp.com.

[20] Ryan Singer, *Shape Up*, Part Three: Building, chs. 10–13 (2019). basecamp.com.

[21] Ryan Singer, *Shape Up*, ch. 14, "Decide When to Stop" (2019). basecamp.com.

- **Key takeaways** are summarized above in §2.9.
- Continue to the Exercises.
- Go deeper with the Open Resources for this chapter.

Chapter 2 — Exercises

Work these in order within a theme; each theme builds from recall toward judgment and practice. Tags: **[warm-up]** checks understanding, **[analysis]** asks you to reason about trade-offs.

Processes and values (§2.1)

1. **[warm-up]** In your own words, distinguish a *process model* from the *process a team actually lives*. Give one concrete decision (from an ordinary workday) that the model does *not* specify but the lived process must.

2. **[warm-up]** For each of the four Agile Manifesto value pairs, write one sentence explaining what the *right-hand* side is good for. The manifesto values the left more — argue why that does not make the right side worthless.

3. **[analysis]** The chapter contrasts a *plan-driven* culture and a *grow-it* culture. Name two real project situations from your own experience or reading — one that clearly fits each culture — and justify the match using the selection questions in §2.1.4.

Scrum (§2.2)

4. **[warm-up]** Match each Scrum accountability to the question it owns — *what/why*, *how*, or *the process itself* — and explain why the Product Owner accountability is held by a single person rather than a committee.

5. **[analysis]** A manager reads the Daily Scrum minutes each morning and privately assigns the next day's tasks to individual developers based on them. Which Scrum principle is being violated, and what concrete harm does the chapter predict? Rewrite the manager's behavior so it supports the framework instead.

6. **[analysis]** A team reports that every item in last sprint's backlog is "done," yet the demo reveals features that crash on bad input and were never code-reviewed. Diagnose the failure in terms of the *Definition of Done*, and write four checklist items you would add to prevent a repeat.

XP and engineering practice (§2.3)

7. **[warm-up]** Write a user story for a feature of an app you use often, using the *"As a <role>, I want <capability> so that <benefit>"* template. Then state one *testable* acceptance criterion for it — the property from INVEST that §2.3.1 says matters most for building it.

8. **[analysis]** Explain the red–green–refactor loop, then give two distinct reasons the chapter offers for writing the test *before* the code. Which of the two reasons is about *design* rather than *catching bugs*?

9. **[analysis]** “Emergent design means we don’t need to think ahead.” Refute this using the §2.3.3 pitfall. In your answer, name one kind of decision that XP still says deserves deliberate up-front thought, and explain why it is an exception.

Plan-driven models and their risks (§§2.4–2.5)

10. **[warm-up]** Draw the V-model and label, for each of the four testing levels, the single question it answers. Explain why *system testing can pass while acceptance testing fails*.

11. **[analysis]** Explain *big-bang integration* and connect it to the cost-of-change curve. Then describe precisely how *continuous integration* attacks the same risk, and why “one small mismatch at a time” is so much cheaper than “all mismatches at once.”

12. **[analysis]** A client insists on a fixed-scope, fixed-price contract that forces a largely waterfall flow. Using §2.4.3, propose three concrete mitigations you would build into the plan, and note how each one pulls the project *toward* iteration.

Risk and the spiral (§§2.6–2.7)

13. **[analysis]** Compare the hypothetical iterative project (§2.6.2) with the troubled rewrite (§2.6.3). Identify the *single structural difference* that the chapter says explains the different outcomes, and connect it to the spiral model’s central claim in §2.7.1.

Shape Up (§2.8)

14. **[analysis]** Shape Up fixes time and flexes scope, keeps no backlog, and applies a circuit breaker when the cycle ends. For a project you know, argue which one of these three bets would help the team most and which would hurt it most, using the §2.8 “Shape Up vs. Scrum” contrast to justify both picks.

Chapter 2 — Open Resources

Free, open-licensed, or freely accessible material for going deeper on software development processes. Legend: 📘 book/book chapter · 🎓 course/courseware · 📄 paper or standard · 🎥 video/talk. Licenses vary and are noted where known.

Foundational values

- 📄 **Manifesto for Agile Software Development** — the four values and twelve principles that define the agile movement, plus the list of original signatories. https://agilemanifesto.org/ · Free to read; the manifesto text is freely reproducible with attribution to its authors.
- 📄 **The Scrum Guide (2020)**, Ken Schwaber and Jeff Sutherland — the definitive, vendor-neutral 13-page definition of Scrum's accountabilities, events, and artifacts; the site's revision history documents the changes from earlier editions. https://scrumguides.org/scrum-guide.html · Released under Creative Commons Attribution-ShareAlike (CC BY-SA 4.0).
- 📘 **Extreme Programming**, Portland Pattern Repository (C2 wiki) — the original community wiki where XP's practices (TDD, pair programming, simple design, continuous integration) were discussed and refined. http://wiki.c2.com/?ExtremeProgramming · Free to read.
- 📘 **The Art of Unix Programming**, Eric S. Raymond (2003) — chapter 1 collects the Unix philosophy's maxims, including Doug McIlroy's "do one thing well" dicta and the rule that clarity is better than cleverness. http://www.catb.org/esr/writings/taoup/html/ · Free to read online (full text posted by the author).
- 📘 **Extreme Programming Explained: Embrace Change (2nd ed.)**, Kent Beck with Cynthia Andres (2004) — XP's values and practices from their originator; the primary source for §2.3. https://www.informit.com/store/extreme-programming-explained-embrace-change-9780321278654 · Commercial book (publisher page); widely held in libraries.
- 📘 **Test-Driven Development: By Example**, Kent Beck (2002) — the red–green–refactor loop worked through complete examples; the primary source for TDD in §2.3.2. https://www.informit.com/store/test-driven-development-by-example-9780321146533 · Commercial book (publisher page); widely held in libraries.
- 📄 **"Essential XP: Card, Conversation, Confirmation"**, Ron Jeffries (2001) — the classic "three Cs" statement of what a user story is (and is not); companion to Alistair Cockburn's "promise for a conversation." https://ronjeffries.com/xprog/articles/expcardconversationconfirmation/ · Free to read.

- 📄 **“INVEST in Good Stories, and SMART Tasks”**, Bill Wake (2003) — the article that coined the INVEST checklist for user stories used in §2.3.1 and Chapter 3. https://xp123.com/articles/invest-in-good-stories-and-smart-tasks/ · Free to read.
- 📄 **“Continuous Integration”**, Martin Fowler (2024 revision) — the standard description of CI as an XP-born practice: integrate at least daily, verify every merge with an automated build and tests. https://martinfowler.com/articles/continuousIntegration.html · Free to read.
- 📄 **“Yagni”**, Martin Fowler (2015) — a careful unpacking of “you aren’t gonna need it” and its relationship to XP’s simple-design practice. https://martinfowler.com/bliki/Yagni.html · Free to read.

Plan-driven and risk-driven models (primary sources)

- 📄 **“Managing the Development of Large Software Systems”**, Winston W. Royce (1970) — the paper universally cited as the origin of the “waterfall” diagram. Read it carefully: Royce presents the strict sequential model and then argues it is *risky* and must be iterated, which is the opposite of how it is usually invoked. https://www.cs.umd.edu/class/spring2003/cmsc838p/Process/waterfall.pdf · Free to read (archival scan mirrored on university course pages).
- 📄 **“A Spiral Model of Software Development and Enhancement”**, Barry W. Boehm (1988) — the original, highly readable presentation of the risk-driven spiral framework, from IEEE *Computer* 21(5). https://www.cse.msu.edu/~cse435/Homework/HW3/boehm.pdf · Free to read (university course mirror).
- 📄 **“Software Risk Management: Principles and Practices”**, Barry W. Boehm (1991) — defines risk exposure as probability times loss and lays out the identify–analyze–respond cycle behind §2.6, from IEEE *Software* 8(1). https://www.cs.virginia.edu/~sherriff/papers/Boehm%20-%201991.pdf · Free to read (university course mirror).
- 📄 **“Software Defect Reduction Top 10 List”**, Barry Boehm and Victor R. Basili (2001) — the concise empirical backing for the cost-of-change curve: finding and fixing a problem after delivery is often 100 times more expensive than during requirements and design, from IEEE *Computer* 34(1). https://www.cs.umd.edu/projects/SoftEng/ESEG/papers/82.78.pdf · Free to read (author-affiliated university copy).
- 📄 **“Things You Should Never Do, Part I”**, Joel Spolsky (2000) — the widely cited essay on the mid-1990s browser rewrite behind §2.6.3, and why rewriting working software from scratch concentrates risk. https://www.joelonsoftware.com/2000/04/06/things-you-should-never-do-part-i/ · Free to read.

Course material

- 📘🎓 **Engineering Software as a Service (ESaaS)**, Armando Fox and David Patterson, UC Berkeley — Chapter 1 surveys agile versus plan-and-document lifecycles, and Chapter 10 covers Scrum, pairing, and project management in depth alongside a running project. Companion site with free materials, slides, and MOOC links. https://www.saasbook.info/ · Free companion materials; MOOCs offered free to audit on edX.
- 🎓 **MIT OpenCourseWare, 6.005 “Software Construction”** — open readings and problem sets on test-first development, specifications, and designing for change that complement this chapter’s engineering practices; the successor course 6.031’s readings are openly available at https://web.mit.edu/6.031/. https://ocw.mit.edu/ · Creative Commons Attribution-NonCommercial-ShareAlike.
- 🎥 **“Agile Product Ownership in a Nutshell”**, Henrik Kniberg — a widely used 15-minute whiteboard talk explaining backlogs, prioritization, and the Product Owner accountability. Searchable on YouTube. · Free to view.
- 📘 **“Shape Up,” Ryan Singer (Basecamp)** — the free online book behind §2.8: shaping, six-week cycles plus a two-week cool-down, betting instead of backlogs, the circuit breaker, and hill charts. https://basecamp.com/shapeup · Free to read online.

License note

Linked resources remain under their own licenses; this page is CC BY-SA 4.0.

Chapter 3 — User Requirements

> **Where we are.** Chapter 1 argued that the hardest part of many projects is deciding *which* system to build, and Chapter 2 gave you a process to build it in short, adaptive iterations. This chapter zooms in on the front of that process: how you discover what real people need, how you write it down so a team can act on it, and how you keep those written needs honest as understanding shifts. Everything downstream — design, tests, metrics — inherits the quality of the requirements you produce here. Get this wrong and you will build a beautiful, well-tested answer to the wrong question.

A **requirement** is the bridge between a person with a problem and a team with the skill to solve it. That bridge is where most projects quietly succeed or fail. You can recover from a bad database choice; you refactor it. You rarely recover cheaply from having built the wrong feature for six months, because the cost is not just the wasted code — it is the wasted *time*, the eroded trust of the people who told you what they wanted and watched you miss it. This chapter is about narrowing that gap between what people need and what you believe they need, and doing so continuously, because the gap never fully closes.

3.1 What Is a Requirement?

A requirement is a statement of something the software must **do** for its users, or a **quality** it must have while doing it. That sounds simple, and the trouble hides in the simplicity. Requirements are written in natural language, by and for humans, about needs that the humans themselves often cannot articulate. They are simultaneously the most important artifact in a project and the least precise. Learning to work skillfully in that tension — to extract precision from ambiguity without pretending the ambiguity is gone — is the core skill of this chapter.

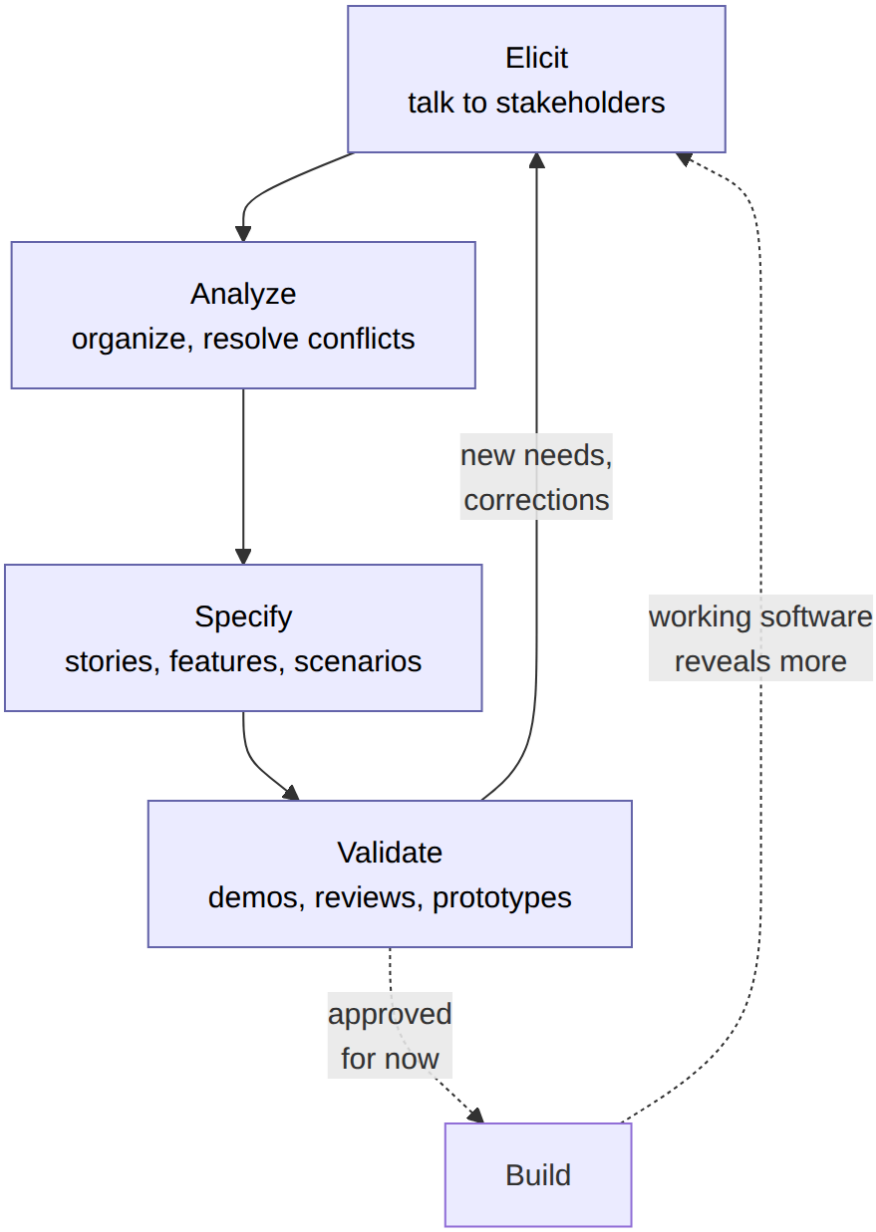
> **Definition.** A *requirement* is an agreed statement of a need that a stakeholder has of the system: either a **behavior** the system must exhibit or a **property** it must possess. A requirement is *not* a design; it says *what* and *why*, not *how*.

That last sentence matters more than it looks. When a user says “add a dropdown,” they have handed you a design dressed up as a need. Your job is to reach past it to the need underneath — “I need to pick one clinic from a short list without typos” — because the need admits *better* designs than the one the user happened to imagine. Requirements work is largely the discipline of not accepting the first solution offered as if it were the problem.

3.1.1 The Basic Requirements Cycle

Requirements are not gathered once and filed away. They move through a repeating cycle, and the health of a project is largely the health of this loop. You **elicit** needs from stakeholders, **analyze** and organize them into

something coherent, **specify** them in a form the team can act on, and **validate** that what you specified is actually what people need — which surfaces new needs and misunderstandings, sending you around again.



Notice two things about this diagram. First, the loop never terminates during an active project; it merely *slows* as understanding stabilizes. Second, **Build** feeds back into **Elicit**. Users learn what they want by using something, so the most powerful requirements-elicitation tool you own is often a working (or even fake) version of the product. This is why Chapter 2's iterative process and this chapter's requirements work are the same activity viewed at two zoom levels: each iteration is one trip around this cycle.

> **Principle.** Requirements are *discovered*, not *collected*. You do not walk into a room and receive a complete list; you co-create understanding with stakeholders over time, and the earliest versions are always partly wrong. Plan for that, don't resent it.

3.1.2 Requirements Challenges

Why is this hard? Because every step of the cycle fights human nature and the limits of language. A handful of recurring challenges account for most requirements failures:

- **Tacit knowledge.** Experts cannot fully articulate what they know. A nurse triaging patients follows rules she has never written down and would struggle to state. Ask "how do you decide?" and you get a simplified, partly fictional answer. The real rules live in practice, not in interviews.
- **The unstated obvious.** Stakeholders omit what is obvious *to them*. "Of course the system must handle two clinicians sharing a patient" goes unsaid — until it breaks.
- **Solutions masquerading as needs.** As above, users hand you designs. Accept them uncritically and you inherit everything the user overlooked.
- **Conflicting stakeholders.** The front desk wants speed; compliance wants audit trails; the sponsor wants low cost. These pull in different directions, and no phrasing makes the conflict disappear. Someone must *decide*, and requirements work makes the conflict visible enough to decide well.
- **Ambiguous language.** "The report should load quickly." How quickly? For whom? Under what load? "Quickly" is a placeholder for an argument no one has had yet.
- **Change.** Even a perfectly captured requirement expires. Regulations update, markets move, users mature. A frozen specification is a specification that is slowly going stale.

None of these are solved by working harder at the same technique. They are managed by *combining* techniques — interviews plus observation plus prototypes plus review — so that what one misses another catches.

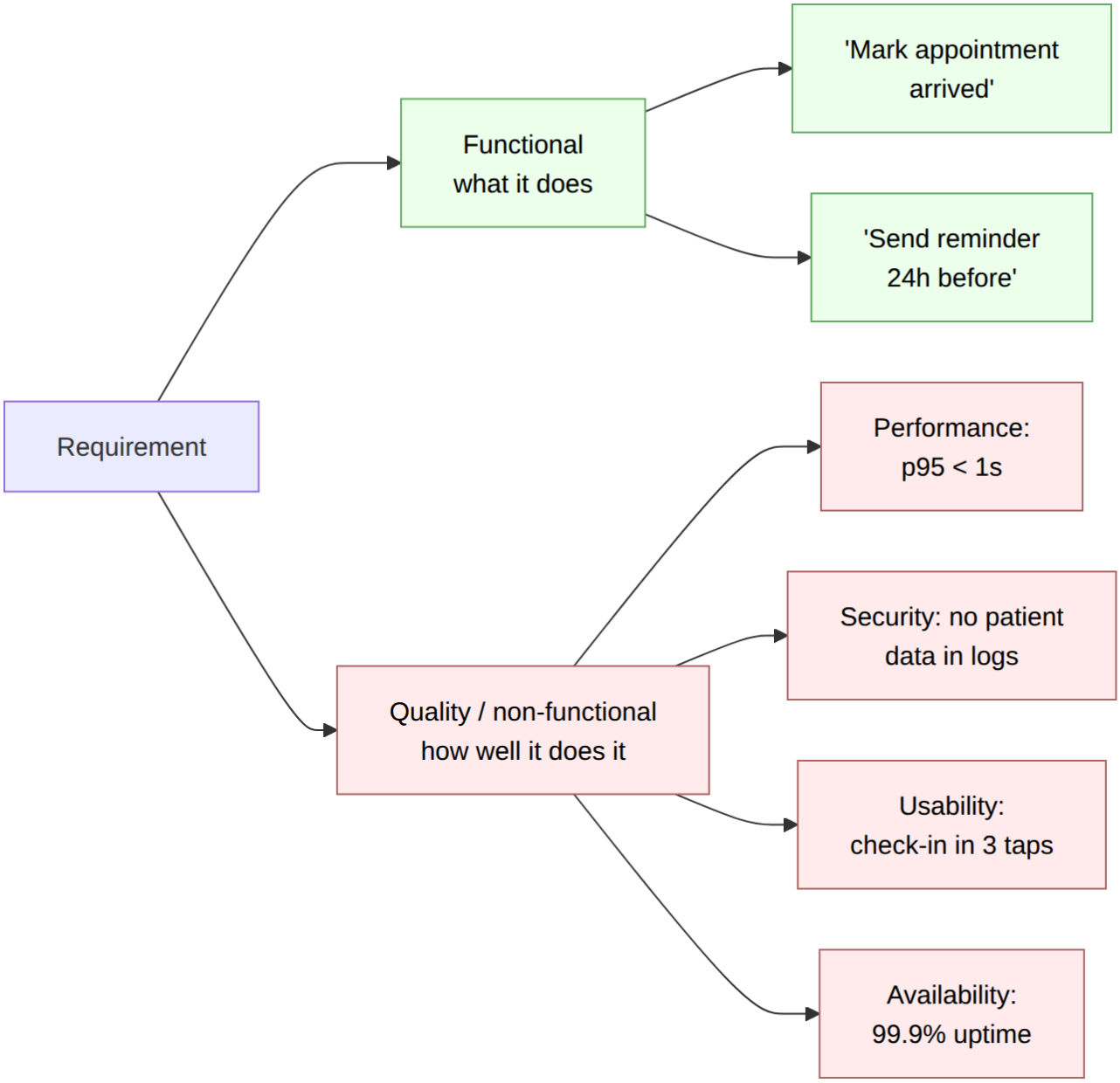
3.1.3 Kinds of Requirements

It helps enormously to sort requirements into two families, because they are elicited, written, and tested in different ways.

A **functional requirement** describes a *behavior*: something the system does in response to input. "When a patient checks in, the system marks their appointment as *arrived* and notifies the assigned clinician." You can usually test a functional requirement with a concrete example: given this input, expect this output. Most user stories (§3.4) capture functional requirements.

A **non-functional requirement** — better called a **quality requirement** — describes a *property* the system must have while it behaves: how fast, how secure, how available, how usable, how maintainable. "Check-in must complete in under one second for 95% of requests when 200 clinics are active." Quality requirements are the ones

teams most often botch, because they are easy to state vaguely (“it should be fast”) and hard to state testably (“p95 latency under 1s at 200 concurrent clinics”). The discipline is to attach a **measurable fit criterion** to every quality requirement, so that “done” is a fact and not an argument.[1]



> **Pitfall.** Quality requirements are frequently *invisible* until violated. No one writes a story titled “the app should not leak patient data,” so no one budgets for it, and it surfaces only as an incident. Make quality requirements explicit and measurable, or they will be discovered by your users on your worst day.

3.2 Developing Requirements and Software

How you handle requirements depends on how you build software, and the two dominant philosophies — agile and plan-driven — take genuinely different stances. Neither is universally right. Understanding *why* each makes sense in its context lets you choose deliberately rather than by fashion.

3.2.1 Agile Methods Validate Working Software

Agile methods treat requirements as a hypothesis to be tested by shipping. Rather than try to specify everything up front — which the challenges in §3.1.2 say you cannot do reliably — agile teams specify a *little*, build it, put it in front of users, and let their reaction correct the next round. The unit of truth is **working software**, not a document. If the demo delights the clinic staff, the requirement was right; if they frown, you learn cheaply and adjust before you have built a mountain on a bad foundation.

This is a bet, and the bet is that *feedback is more reliable than foresight*. In domains where users cannot tell you what they want until they see it — which is most consumer and business software — the bet usually pays off. The cost is that you must be genuinely willing to change: an agile team that treats its first backlog as fixed has adopted the ceremony without the substance.

3.2.2 An Agile Emphasis on Requirements

A common misreading is that agile means "no requirements." The opposite is true. Agile teams often do *more* requirements work than plan-driven teams — they just do it in small, continuous doses rather than one large batch. Every iteration begins with refining and selecting requirements (the backlog), and ends with validating them against a running system. Requirements conversations happen constantly, at the moment they are needed, with the person who has the answer in the room.

The agile move is to shift requirements from a *document* to a *conversation plus a thin written reminder*. A user story (§3.4) is deliberately too short to be a full specification; it is a **promise to have a conversation** later, when the team is about to build it and knows the most it will ever know.[2] This defers the detailed decisions to the last responsible moment, when they are cheapest and best-informed.

> **Principle.** In agile work, the written requirement is a placeholder for a conversation, not a substitute for it. The story reminds you *what* to talk about; the value is created when the developer, the customer, and the tester talk it through together — the "three amigos."[3]

3.2.3 Plan-Driven Methods Validate a Specification

Plan-driven (sometimes called "specification-first" or, historically, "waterfall") methods invest heavily in writing a complete, precise, agreed specification *before* significant building begins. Validation happens against the

document: reviews, sign-offs, and traceability checks confirm that the specification is complete and consistent, and later that the built system matches it.

This looks old-fashioned next to agile, and for a scheduling app it usually is. But there are domains where it is exactly right. When the cost of a late change is enormous or the system is safety-critical — avionics, medical devices, spacecraft, systems bound by a regulatory certification — you *cannot* "ship and see." You must reason carefully about the requirements up front, because a failure in the field is catastrophic and a change after certification is ruinously expensive. In those worlds, a rigorous specification is the cheapest place to catch a life-threatening mistake.

Most real projects blend the two: a stable, carefully specified core (the things that must not be wrong) surrounded by an agile, iterative shell (the things you will learn by trying). Choosing where to draw that line is an engineering judgment, and the right cut depends on the cost of being wrong.

3.3 Eliciting User Needs

Elicitation is the active work of drawing out needs that stakeholders cannot simply hand you. The word is chosen carefully: you *elicit* needs the way a good teacher elicits an answer — with questions, prompts, and situations that help people discover what they know but had not stated.

3.3.1 A Classification of Needs

Not all needs are equal, and it helps to classify them by how the user relates to them. A useful three-way split:

- **Stated needs** — what the user explicitly asks for. "I want to search appointments by patient name." These are the easiest to collect and the most likely to be half-solutions.
- **Implied needs** — what the user assumes without saying, because it is obvious to them. "Of course the search shouldn't show patients from other clinics." Miss these and users feel the system is broken even though it does everything they *said*.
- **Latent needs** — what the user does not know they want until they see it, but which delights them when they do. "Oh — it can flag patients who haven't confirmed? I didn't know I needed that, but now I can't live without it."

The gap between stated and implied needs is where most "the software technically works but everyone hates it" failures live. The latent needs are where competitive advantage lives. A team that only satisfies stated needs builds something adequate; a team that uncovers implied and latent needs builds something people love.

3.3.2 Accessing User Needs

There is no single technique that reaches all three kinds of needs, so skilled teams triangulate. Each of the main methods has a characteristic strength and a matching blind spot:

- **Interviews.** Direct and flexible; great for stated needs and stakeholder goals. Weakness: people rationalize and misremember what they actually do. What people *say* they do and what they *do* often differ.
- **Observation (contextual inquiry).**[4] Watch users do the real task in their real environment. Superb for tacit and implied needs — you see the sticky note on the monitor, the workaround, the double-entry no one mentioned. Weakness: expensive, and observation can change behavior.
- **Surveys.** Cheap at scale; good for prioritizing among *known* options and quantifying how common a need is. Weakness: useless for discovering needs you did not already think to ask about.
- **Prototypes and demos.** Show something concrete and watch the reaction. The single best tool for latent needs, because a person who cannot describe what they want can almost always react to what is in front of them. Weakness: users may fixate on the prototype's incidental choices.
- **Brainstorming (blue-sky sessions).** Gather many stakeholders and think big: during the session, *no idea is ruled out or criticized*, and everything is captured — the goal is quantity first, judgment second, because premature filtering kills the unusual idea that turns out to matter. Keep the focus on the core of what the software is supposed to do, then reconvene later, with cooler heads, to filter and prioritize what the session produced. Weakness: without that second, critical pass, a brainstorm yields a wish list, not requirements.
- **Role play.** A developer *pretends to be the software* while the customer walks through using it: "I'd click here… no, it should already show today's schedule." The customer's instructions — and especially their corrections — are requirements spoken aloud. Write everything down during the session, then split the notes afterward into requirements and open questions. Weakness: you only exercise the workflows the customer thinks to perform.
- **Studying existing systems and data.** Support tickets, competitor products, and usage logs are elicitation gold — they are records of real needs and real friction, unfiltered by anyone's memory.

Use several elicitation methods, because each corrects a different weakness: interviews form hypotheses, observation tests them, prototypes reveal omissions, and surveys help prioritize the findings.

> **Case study.** A team building the clinic scheduler interviewed front-desk staff, who asked for faster search. Only when the team *watched* a morning rush did they see the real problem: staff kept a paper list of "difficult" patients beside the keyboard because the system had no way to flag them. No interview surfaced this — it was too normal to mention. Observation found in one hour what interviews had missed for weeks.

Frame the problem before the solution. Users — and stakeholders — hand you *solutions* ("add a dropdown," "redesign the reports page"). A useful habit from Basecamp's *Shape Up* is to flip every such request from *"what should we build?"* back to **"what's really going wrong?"** — the specific moment in the workflow where today's experience fails.[5] Two ideas sharpen that:

- **The baseline** is what users do *today* without the feature. Stated plainly ("right now they keep a paper list beside the keyboard"), it both proves the problem is real and becomes the reference for judging any solution: a fix only has to beat the baseline, not reach perfection.

- A **grab-bag** is the anti-pattern — a vague mandate ("do something about scheduling") with no driving problem, so no one can tell where it starts or ends. Grab-bags are how scope explodes; the cure is to narrow to one concrete pain point before designing.

> **Pitfall.** A requirement phrased as a solution smuggles in an unexamined assumption about *how*. Trace it back to the problem and you often find a smaller, cheaper, better answer than the one you were asked for.

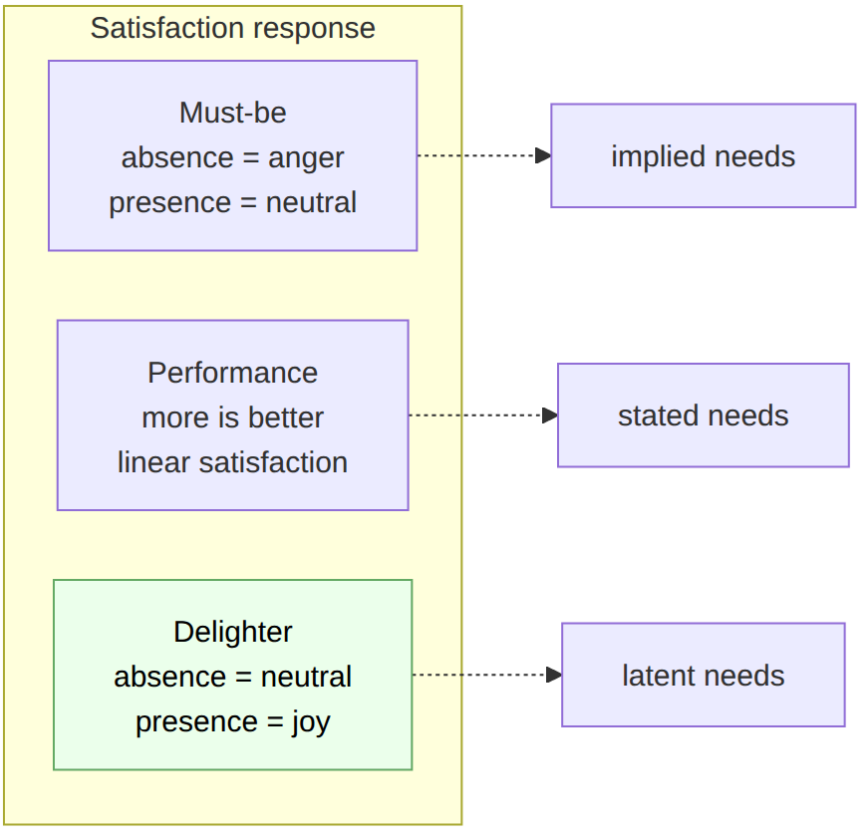
Whatever mix of techniques you use, there will always be a gap between what the customer said and what you understood — and that gap fills itself with **assumptions**. Every unstated assumption buried in a story is untracked risk: you will build on it, and if it is wrong, everything on top of it is suspect.

> **Technique — track your assumptions.** Write assumptions down *explicitly*, the moment you notice yourself making one ("we assume clinicians never edit each other's appointments"). Then convert each into a question for the customer at the next meeting — an assumption is just a question you have not asked yet. If an assumption cannot be clarified yet, treat that as a prioritization signal: deprioritize the work that depends on it until it can be, and spend the iteration on stories that stand on firmer ground (see the prioritization techniques in Chapter 4, §4.4).

3.3.3 Design for Delight

Satisfying stated needs makes a product *acceptable*. Delighting users — meeting latent needs they did not know to ask for — makes a product *loved*, and loved products tend to win. A useful mental model here — the **Kano model**,[6] developed fully in §4.5 — distinguishes categories of features by how user satisfaction responds to them:

- **Must-be features** (basic expectations). Their presence is unnoticed; their absence is a dealbreaker. A scheduler that loses appointments has failed no matter how pretty it is. These map to *implied* needs. You get no credit for them, but you cannot skip them.
- **Performance features** (the more, the better). Satisfaction scales with how well you do them: faster search, more report options. These map to *stated* needs. This is where users compare products, so doing them well matters.
- **Delighters** (unexpected value). Their absence is not missed, because no one expected them — but their presence produces disproportionate delight. These map to *latent* needs. The patient-confirmation flag above is a delighter.



> **Pitfall.** Delighters decay into expectations. The first scheduler with automatic reminders delighted users; today a scheduler *without* reminders feels broken. What was a delighter becomes a must-be. Elicitation continues throughout development because user expectations rise as products and competitors improve.

Two practical consequences. First, spend your must-be budget ruthlessly but seek no glory there — just don't fail. Second, invest deliberately in one or two delighters per release, because they are where love and loyalty are earned, and they are precisely the needs your competitors also failed to ask about.

3.4 Writing Requirements: Stories and Features

Having elicited needs, you must write them down in a form the whole team can act on. Two complementary formats dominate modern practice: **user stories** (small, user-centered, conversational) and **system features** (larger capabilities that group related stories).

3.4.1 Guidelines for Effective User Stories

A **user story** is a short statement of a need from a user's point of view, written to provoke a conversation rather than to settle every detail. The most widely used template, often called the **Connextra format** after the company where it originated,[7] is:

> **As a** <*role*>, **I want** <*capability*>, **so that** <*benefit*>.

For example:

> *As a* front-desk clerk, *I want* to flag a patient as "needs interpreter," *so that* the clinician is prepared before the visit begins.

Each clause has a job. The **role** keeps you honest about *who* benefits, and forces you to notice when a story serves no real user. The **capability** states the behavior. The **so that** clause is the most important and most often omitted: it captures the *why*, the underlying goal that §3.1 said you must reach past the solution to find. When a story's benefit clause is vague or circular ("…so that I can flag patients"), it is a warning that no one understands why the feature exists — and a feature no one can justify is a feature to cut.

To judge whether a story is *ready* to build, teams use the **INVEST** checklist.[8] A good story is:

- **I — Independent.** It can be built and shipped without depending on other stories being done first. Dependencies make planning and prioritizing brittle.
- **N — Negotiable.** It is a starting point for a conversation, not a fixed contract. The details are worked out with the customer, not dictated by the card.
- **V — Valuable.** It delivers something a user or customer can perceive as valuable. "Refactor the database layer" is real work but is not a user story — it has no user-facing value on its own.
- **E — Estimable.** The team can roughly size it. If they can't, the story hides too much uncertainty and needs to be split or researched first (a *spike*).
- **S — Small.** It fits comfortably within a single iteration. Big stories ("epics") are split until each piece is small enough to finish and demo.
- **T — Testable.** You can state, in advance, how you will know it works. If you can't write an acceptance test, you don't yet understand the story well enough to build it.

> **Principle.** If a story fails INVEST, don't force it into the sprint — *fix the story*. An unestimable story means you need a spike; an untestable one means the requirement is still ambiguous; a huge one means you should split it. INVEST is a diagnostic, not a bureaucratic gate.

Attach **acceptance criteria** to each story — concrete conditions that must hold for the story to count as done. These are the seed of the tests you will write (Chapter 10) and the answer to "how will we know?"

Given / When / Then: writing acceptance criteria as scenarios

A widely used, highly readable form for acceptance criteria is **Given / When / Then**: you name the starting context (**Given**), the action or event (**When**), and the expected, observable outcome (**Then**). Each such triple is a **scenario** — one concrete example of the story working:

> **Given** a checked-in patient flagged for an interpreter, **When** the clinician opens the visit, **Then** an interpreter-needed banner is shown.

This structured-English format is called **Gherkin**,[9] and it comes from **Behavior-Driven Development (BDD)** — an approach that writes requirements as concrete examples of behavior, in language a customer can read.[10] In a BDD workflow the scenarios live in plain `.feature` files and look like this:

```
Feature: Interpreter alerts
  So that clinicians are prepared, staff want a visible interpreter flag.

  Scenario: Patient needs an interpreter
    Given a checked-in patient flagged for an interpreter
    When the clinician opens the visit
    Then an "interpreter needed" banner is shown

  Scenario: Patient does not need an interpreter
    Given a checked-in patient with no interpreter flag
    When the clinician opens the visit
    Then no interpreter banner is shown
```

> **Why it matters.** The same words that specify the requirement can *run* as an automated test. Every mainstream language has a BDD framework — the Cucumber family, among others — that maps each `Given`/`When`/`Then` step to code, turning the scenario into an **executable specification**.[11] Write the scenarios with the customer, and they double as your **acceptance tests** (Chapter 10, §10.2.3).

Your BDD framework binds each scenario line to a **step definition** — a function that calls the clinic app and asserts what the user would see (each tab below names its framework in its imports or comments):

```python
from behave import given, when, then
# behave matches each scenario line to its step by exact text

@given("a checked-in patient flagged for an interpreter")
def flagged(context):
  context.patient = check_in(interpreter=True)

@given("a checked-in patient with no interpreter flag")
def not_flagged(context):
  context.patient = check_in(interpreter=False)

@when("the clinician opens the visit")
def opens(context):
  context.visit = open_visit(context.patient)

@then('an "interpreter needed" banner is shown')
def banner_shown(context):
  assert "interpreter needed" in context.visit["banners"]

@then("no interpreter banner is shown")
def no_banner(context):
  assert not context.visit["banners"]
```

A few guidelines for good scenarios: keep them **declarative** (describe *what*, not which buttons to click); use **one** `When` per scenario (multiple actions usually means you have multiple scenarios); cover the **happy path and the important alternatives** (mirroring the alternative flows of a use case, Chapter 5); and reuse a shared `Given` with a `Background` when several scenarios start the same way. Keep the vocabulary in the **problem domain** — "interpreter needed," not "row 7 turns yellow."

3.4.2 Guidelines for System Features

Stories are small by design, which makes them wonderful for building and terrible for seeing the forest. A **system feature** is a larger, coherent capability that a stakeholder would recognize and value as a unit — "appointment reminders," "insurance verification," "waitlist management." Features group related stories and give the product a shape that executives, marketers, and users can talk about.

Good features share several properties:

- **User-meaningful.** A feature names something a stakeholder cares about, not an internal component. "Caching layer" is not a feature; "instant search" might be.

- **Independently valuable.** Shipping the feature (even partially) delivers value on its own, so it can be prioritized against other features honestly.
- **Decomposable into stories.** A feature that cannot be broken into small, INVEST-able stories is probably too vague to plan.
- **Traceable to a goal.** Every feature should map to a stakeholder goal (§3.6). A feature with no goal behind it is a candidate for the cut list.

Features and stories form a two-level hierarchy: features answer "what capabilities does the product have?" for planning and communication, while stories answer "what do we build this week?" for execution. Keeping both views lets you zoom between the roadmap and the sprint without losing either.

3.4.3 Perspective on User Stories

User stories are a tool, not a religion, and it is worth being honest about their limits. A story is deliberately incomplete — that is its virtue for conversation and its vice for anything requiring precision. Stories are excellent for the flexible, discoverable parts of a system. They are *poor* at capturing:

- **Cross-cutting quality requirements.** "The system shall never log patient identifiers" does not fit "as a…I want…so that." Quality requirements often need their own explicit statements with fit criteria (§3.1.3).
- **Complex flows.** A multi-step interaction with branches and error cases overflows a story card. That is what **scenarios** (§3.5) are for.
- **Regulatory or contractual precision.** When wording is legally binding, a conversational placeholder is not enough; you need a specification (§3.2.3).

The mature view is that stories are one instrument in an ensemble. Reach for them when the need is user-facing, discoverable, and best refined by conversation. Reach for features when you need a roadmap, scenarios when you need to show a flow, explicit quality requirements when you need a testable property, and a specification when the cost of ambiguity is unacceptable. Using the right form for each need is itself a skill.

3.4.4 Tracing Requirements to Tests and Backlog Items

Every artifact this section has introduced — story, acceptance criteria, executable scenario — is a link in a chain, and the chain has a name. **Requirements traceability** is the ability to follow a requirement *forward* to the backlog items, code, and tests that realize it, and *backward* from any of those artifacts to the requirement that justifies it.[12] Without it, two questions that decide releases become guesswork: *is every requirement we promised actually covered by a passing test?* and *when this requirement changes, what else has to change with it?*

You have already built most of the chain without noticing. Give each story a stable key when it enters the backlog, and the rest is discipline about references:

Link in the chain	Where it lives	How it points back
Story `STORY-143`	the backlog	— (the anchor)
Acceptance criteria	`interpreter_alerts.feature`	story key in the feature file
Step definitions and tests	the test suite (Chapter 10)	they execute the criteria
Commits and pull requests	version control	`STORY-143` in the message

Each direction pays differently. Forward traceability turns "are we done?" into a checkable claim: every *Must* in the release plan (§4.4) has at least one acceptance scenario, and every scenario passes in the pipeline (Chapter 14) — a requirement with no test is a promise nobody is keeping. Backward traceability pays when things change or break: a failing scenario names the requirement it protects, and a proposed change to a requirement leads you — through its key — to exactly the scenarios, tests, and code that must move with it.

How heavy to make this is a judgment call. Safety-regulated industries mandate complete **traceability matrices** — tables mapping every requirement to every design element, test, and verification result — and auditors read them. For most teams, that ceremony is wasted; the lightweight chain above, maintained through artifacts that already exist, gives you the two answers that matter.

> **Pitfall.** A traceability matrix maintained by hand, after the fact, is fiction within a month. Trace through artifacts that *execute* (scenarios that run as tests) and links that tools maintain for you (story keys in commit messages, pull requests that close backlog items). If keeping the trace requires a separate document nobody runs, the trace is already broken.

3.5 Writing User-Experience Scenarios

A **scenario** tells a concrete story of a specific person using the system to accomplish a goal, step by step, in a specific situation. Where a user story is a one-line promise, a scenario is a narrative — and that narrative form exposes the gaps, branches, and human realities that terse formats hide.

3.5.1 Guidelines for User-Experience Scenarios

Scenarios earn their keep because they are *concrete*. Abstract requirements ("the system supports check-in") let everyone imagine a different thing; a scenario with a named person, a real goal, and specific steps forces the vagueness into the open, where you can argue about it productively. Good scenarios follow a few guidelines:

- **Use a concrete persona, not "the user."** Give them a name, a role, a level of skill, and a motivation. "Kati, a new front-desk clerk two weeks into the job" behaves very differently from "an experienced office manager," and the difference drives design.

- **Start from a trigger and a goal.** What situation begins the scenario, and what is the person trying to achieve? A scenario without a goal is just a click-by-click script.
- **Walk the happy path first, then the exceptions.** Write the smooth case, then deliberately branch: what if the patient isn't in the system? what if the network drops mid-check-in? Exceptions are where most requirements are missing.
- **Stay in the problem, mostly.** A scenario should describe the *experience* and the *need*, leaning away from prescribing exact UI. Some interface detail is unavoidable and useful, but a scenario dictating pixel positions has stopped being a requirement and become a design.
- **Include the person's emotional and situational context.** "Kati is being watched by an impatient patient and three people in line" is a requirement in disguise: it tells you the interaction must be fast and forgiving under stress.

Scenarios pair naturally with stories: a scenario reveals the *flow* and the missing steps; each step then becomes one or more INVEST-able stories to build. Together they give you both the narrative whole and the buildable parts.

3.5.2 A Medical Scenario Example

Here is a scenario for the clinic scheduler, written to the guidelines above:

> **Scenario: Dorothy checks in an arriving patient during the morning rush.**
>
> Dorothy is an experienced front-desk clerk. It is 8:55 a.m., the waiting room is filling, and she has a line of five patients. Her goal is to check each one in quickly and make sure the clinician has what they need before the visit.
>
> Laurel arrives for a 9:00 appointment. Dorothy greets her and searches by last name. Two patients share it; she confirms the date of birth and selects the right one. The system shows Laurel's appointment and a reminder that her insurance verification is *pending*. With one tap Dorothy marks her *arrived*; the assigned clinician's screen updates, and the pending-insurance flag is passed along so the clinician isn't surprised. Total time: under fifteen seconds. Dorothy moves to the next patient.
>
> **Exception A — patient not found.** The next arrival, Harper, does not appear in search. She has an appointment but her name was misspelled at booking. Dorothy needs to find her by phone number and correct the record without leaving the check-in flow, because the line is growing.
>
> **Exception B — patient needs an interpreter.** Behind her, a patient is flagged *needs-interpreter*. The system must surface this prominently at check-in *and* on the clinician's screen, early enough that an interpreter can be arranged before the visit starts, not discovered inside the exam room.

Read what this one scenario produced. The happy path yields stories about search, disambiguation by date of birth, one-tap arrival, and clinician notification. Exception A reveals a whole missing capability — *correcting a mis-booked record inline* — that no one had put in the backlog. Exception B surfaces a cross-cutting need

(interpreter flags that travel with the patient) and a quality requirement (surfaced *early enough* to act). And the context — the growing line, the fifteen-second budget — is a performance requirement stated as human reality rather than as a number, which you then make testable. A single well-written scenario routinely uncovers more real requirements than a day of abstract brainstorming, because it refuses to let anything stay vague.

3.5.3 Storyboards: Drawing the Scenario

A scenario is a story told in prose; a **storyboard** is the same story told in pictures — a short sequence of rough sketches, one frame per step, showing what the user sees and does at each moment. Borrowed from film,[13] storyboarding earns its place in requirements work because a drawing forces decisions that a sentence lets you dodge: what is actually *on* the screen when Dorothy searches? Where does the *pending-insurance* flag appear? What does she tap? If you cannot draw the frame, you have not yet imagined the requirement.

Storyboards at this stage are **lo-fi** (low fidelity) on purpose: boxes, arrows, stick figures, and hand lettering, drawn in minutes on paper or a whiteboard. The roughness is a feature twice over. It keeps *you* from sliding into visual design when the question is still *what happens*, and it signals to stakeholders that everything is still changeable — people freely correct a scribble but hesitate to criticize something that looks finished. This is the same right-fidelity discipline as the fat-marker sketches and breadboards of §6.1.2; a storyboard is essentially a breadboard with time running left to right.

> **Technique — storyboard an exception.** Include failure cases and difficult paths in the storyboard alongside the ideal demonstration path. Drawing §3.5.2's Exception A frame by frame — Harper at the desk, the empty search result, Dorothy switching to phone-number lookup, the inline correction, the line still moving — immediately raises the questions that matter: where does the correction happen? does Dorothy lose her place in check-in? what does the system show the *next* patient while she fixes it? Each frame you cannot draw is a requirement you have not discovered yet.

A practical rhythm for a team: write the scenario (§3.5.1), storyboard its key flows in four to eight frames, walk the storyboard with a real user while they narrate what they'd expect, then convert what you learned into stories and acceptance criteria (§3.4). The storyboard itself becomes a cheap, durable artifact — teams pin them up and check designs against them for the rest of the project.

3.6 Clarifying User Goals

Beneath every feature and story lies a **goal** — a state of the world a stakeholder wants to bring about. "Reduce patient no-shows," "protect patient privacy," "get patients seen faster." Goals are the *why* that stories' "so that" clauses gesture at, and working explicitly with goals is how you keep a backlog coherent, spot conflicts early, and know what to cut. If requirements are the bridge, goals are the far bank you are trying to reach.

3.6.1 Properties of Goals

Good goals share properties that make them useful for reasoning:

- **A goal describes an end, not a means.** “Send SMS reminders” is a means; “reduce no-shows” is the end. Stating the end keeps your options open — maybe email or a phone call serves the goal better for some patients.
- **A goal can be satisfied to a degree.** Unlike a functional requirement (done or not), many goals — especially quality goals like “fast” or “secure” — are matters of *more or less*. These are sometimes called **soft goals**, and you satisfy them by argument and measurement, not by a checkbox.[14]
- **A goal is measurable, at least in principle.** “Reduce no-shows to under 8%” can be checked against reality. A goal you cannot measure even in principle is a slogan, and it will not help you decide anything.
- **A goal has an owner.** Someone cares about it and can speak for it. Ownerless goals drift and get traded away silently.

3.6.2 Asking Clarifying Questions

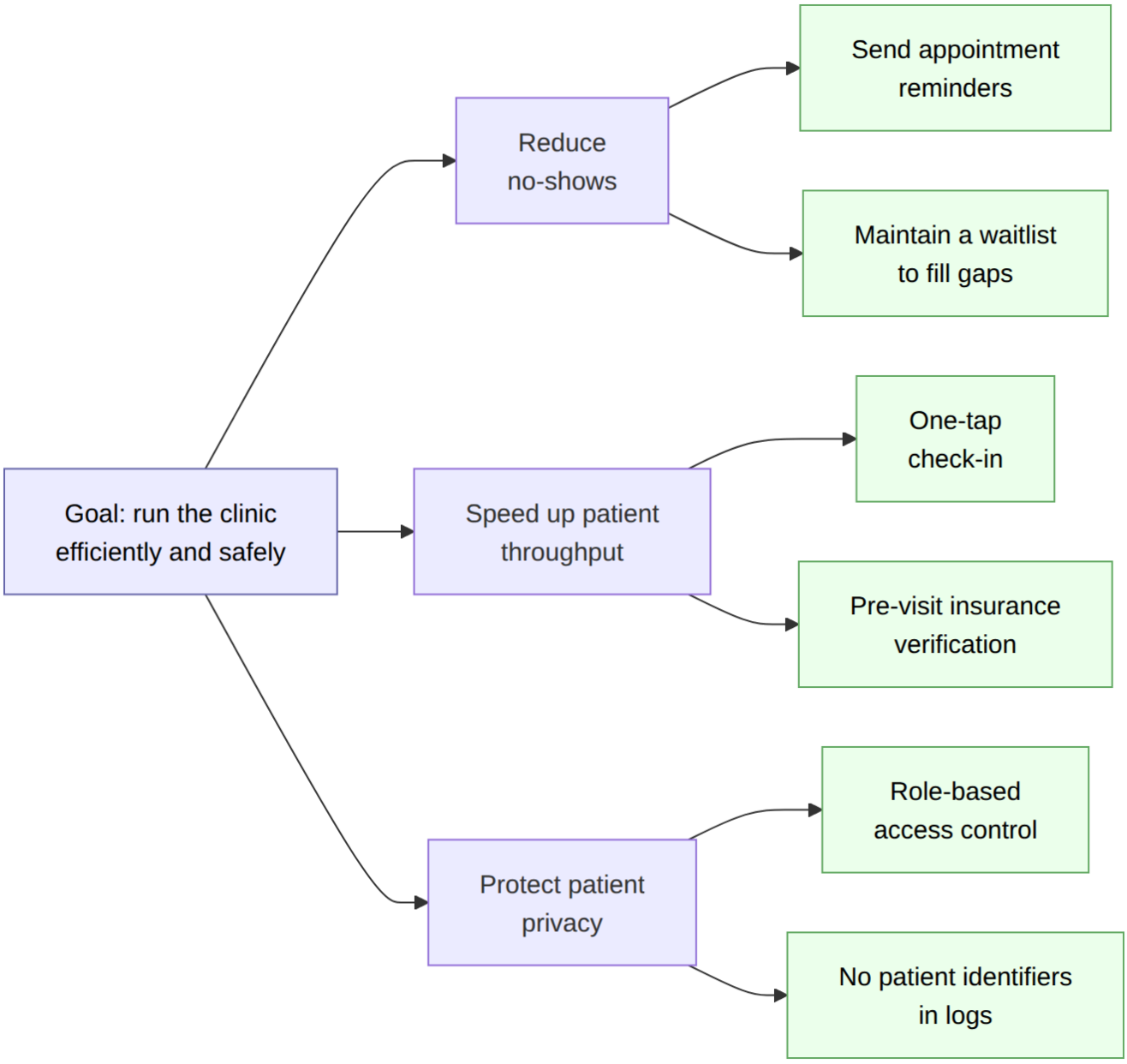
Stakeholders state goals vaguely, and your job is to sharpen them without putting words in their mouth. A few reliable moves:

- **Ask “why?” to go up.** “Why do you want SMS reminders?” → “So fewer patients miss appointments.” Now you have the real goal, and reminders are just one candidate solution.
- **Ask “how?” to go down.** “How would we reduce no-shows?” → reminders, overbooking, waitlists, deposits. Now you have candidate sub-goals and features.
- **Ask “how much?” to make it measurable.** “How much would count as success?” turns “faster check-in” into “under fifteen seconds per patient.”
- **Ask “for whom?” and “when?”** to expose hidden context. “Fast — for the experienced clerk or the new one? During the morning rush or midday?”
- **Ask “what would go wrong if we didn’t?”** to test whether the goal is real. If nothing bad happens without it, it may not be a goal worth funding.

These questions do more than clarify; the “why” and “how” chains literally build the goal hierarchy of the next section.

3.6.3 Organizing Goals into Hierarchies

Asking “why” moves you toward broader goals; asking “how” moves you toward narrower ones. Do this systematically and goals organize into a **goal hierarchy** (or goal tree): a high-level goal at the top, refined downward into sub-goals, and finally into concrete features and stories at the leaves.[14] The tree makes the *rationale* of your backlog visible — every leaf traces up to a goal someone owns, and every goal traces down to work that serves it.

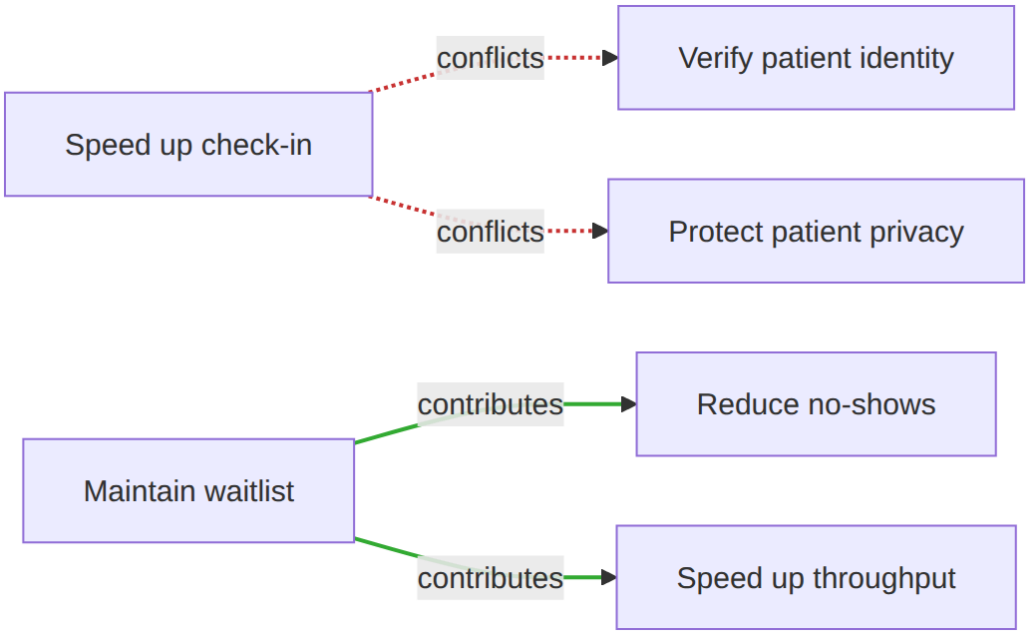


The hierarchy is a working tool, not decoration. When someone proposes a feature, you ask which goal it serves; if it fits nowhere, either you have found a missing goal or an unjustified feature. When you must cut scope, you cut from the least important *goal*, which tells you which whole *cluster* of stories can go — a far better conversation than arguing story by story.

3.6.4 Contributing and Conflicting Goals

The hierarchy above is tidy, but real goals interact, and the interactions are where the hard decisions live. Two relationships matter most:

- **Contribution.** One goal *helps* another. A waitlist both reduces no-shows' impact (empty slots get filled) *and* speeds throughput. Features that serve multiple goals are high-value and should rise in priority — you get more for the same cost.
- **Conflict.** One goal *works against* another. "Speed up check-in" pushes toward fewer confirmation steps; "protect patient privacy" and "avoid mis-identification" push toward *more* verification. You cannot maximize both. The tension is real — the stakeholders live with it — and the analysis has done its job by making it visible.



The value of naming conflicts is that it turns an implicit, unwinnable tug-of-war into an explicit *trade-off decision* that the right person can make on the record. "We will require date-of-birth confirmation even though it adds three seconds, because a mis-identified patient is a safety event and the sponsor accepts the speed cost." That sentence — a goal conflict, resolved deliberately, with an owner and a rationale — is worth more than any amount of vague agreement. Unresolved conflicts do not disappear; they get resolved by accident, in code, by whoever happens to write the feature, usually badly.

> **Principle.** Surface goal conflicts *before* you build, and resolve them *explicitly* with the stakeholder who owns the trade-off. A conflict resolved on purpose is a decision; a conflict resolved by default is a bug with a backstory.

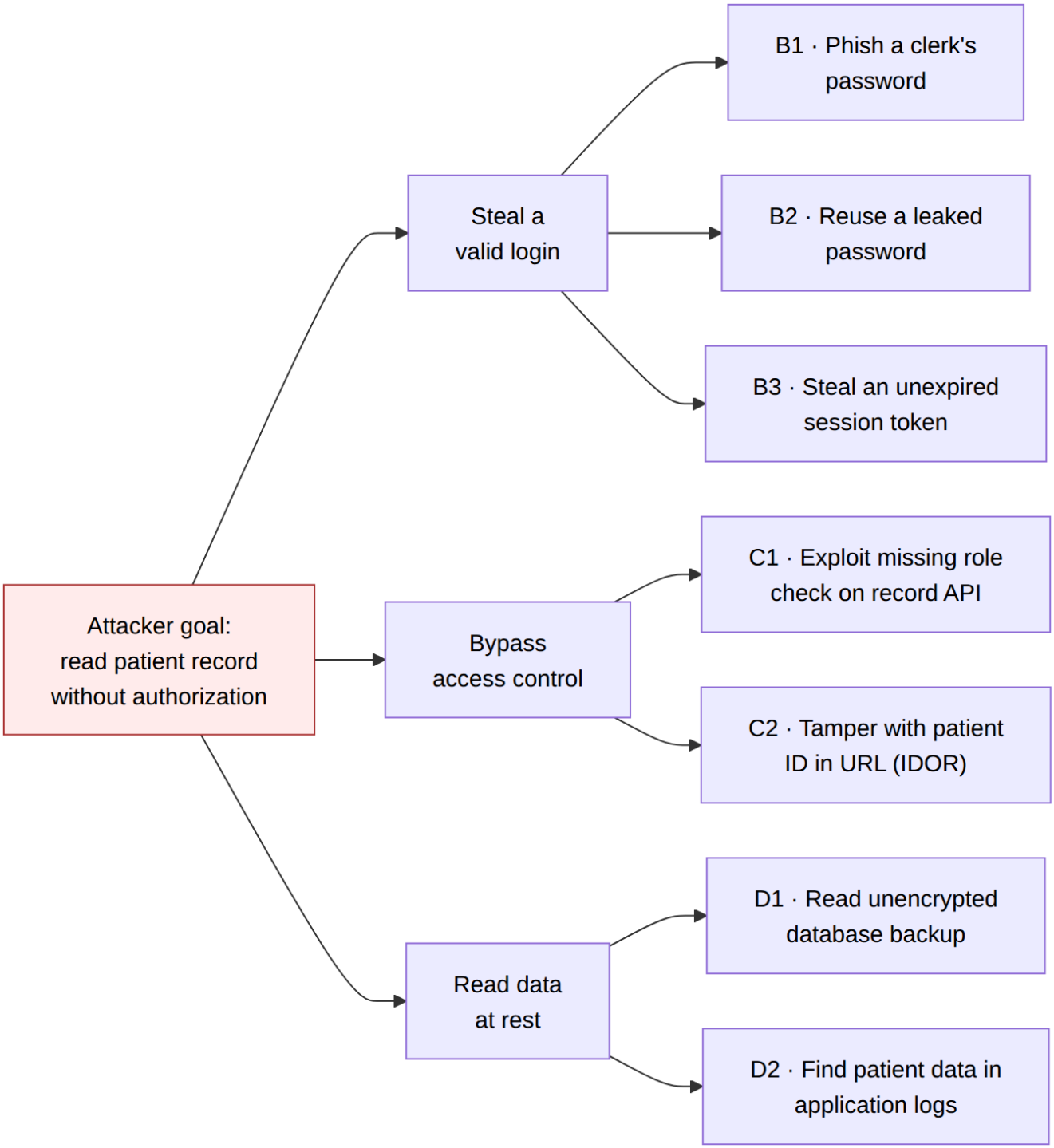
3.7 Identifying Security Attacks

Everything so far has asked "what do users *want* the system to do?" Security requirements demand the opposite question: "what do *adversaries* want the system to do, and how do we stop them?" This inversion is why security

is so often missed — it does not show up when you ask cooperative users what they need. You have to deliberately adopt the mindset of someone trying to break your system.

3.7.1 Attack Trees: Think Like an Attacker

An **attack tree** is a goal hierarchy for the enemy.[15] The root is the attacker's goal — "read a patient's medical record without authorization." The branches are the ways to achieve it, refined downward into ever more concrete attacks, exactly as a goal tree refines a user goal into stories. The power of the technique is that it forces *systematic* thinking: instead of imagining a random hack, you decompose the attacker's goal and make sure every branch is defended.



Read the leaves as a to-do list of defenses. Each one becomes a security requirement: enforce multi-factor auth (defeats B1/B2), expire and bind session tokens (B3), check authorization on *every* record access server-side (C1/C2), encrypt backups (D1), and keep identifiers out of logs (D2 — which you may recognize as a quality requirement from §3.1.3, now justified by a concrete threat). The tree connects each defense to the attack it stops, which is exactly the justification a skeptical sponsor asks for when you request time to build security.

3.7.2 Finding Possible Attacks

Building the tree is a skill, and a few systematic prompts help you find branches you would otherwise miss:

- **Enumerate the assets.** What is worth stealing, corrupting, or denying? Patient records, credentials, availability of the system itself. Each valuable asset is the root of a tree.
- **Walk the trust boundaries.** Every place data crosses from less-trusted to more-trusted — a browser to your API, a third-party integration, an admin console — is a place to attack. Draw them and ask what could cross that shouldn't.
- **Use a threat checklist.** A common one groups threats as **STRIDE**: *Spoofing* (pretending to be someone), *Tampering* (altering data), *Repudiation* (denying an action), *Information disclosure* (leaking data), *Denial of service* (blocking access), and *Elevation of privilege* (gaining rights you shouldn't have).[16] Walking each category against each asset surfaces branches brainstorming misses.
- **Include social and procedural weaknesses, not only flaws in code.** The cheapest attack is often a convincing phone call to the front desk. Social engineering, weak password-reset flows, and over-broad admin rights are attack branches too.
- **Prune by feasibility, then defend the rest.** Not every leaf is worth defending equally. Estimate each attack's *cost to the attacker* and *damage to you*, and invest where cheap attacks meet high damage first.

> **Pitfall.** Security is a set of requirements you discover by asking a different question, not a feature you add at the end. A team that only elicits from cooperative users risks shipping a system that does exactly what good people want — and exactly what bad people want, too. Build one attack tree per critical asset *while* you write your user stories, not after your first breach.

Attack trees and STRIDE are where security *requirements* begin; Chapter 11 picks the thread back up on the engineering side, from the OWASP Top 10 to securing the open-source supply chain your system is built from.

3.8 Conclusion

Requirements are where a project decides whether it is solving a real problem, and this chapter has been a catalog of ways to make that decision well. You saw that a requirement is a bridge from a person's need to a team's action, and that the need is rarely handed over clean: it is tacit, partial, dressed as a solution, and always changing. The **requirements cycle** — elicit, analyze, specify, validate — is how you close the gap between believed and real needs, and it never fully terminates, because working software teaches users what they actually want.

The techniques compound. You **classify** needs (stated, implied, latent) and **elicit** them by triangulating interviews, observation, prototypes, and data — because each method reaches needs the others cannot. You **write** them in the right form for each job: user stories with the *as a…I want…so that* template and the INVEST qualities for discoverable, user-facing behavior; features for the roadmap; scenarios when a concrete narrative is the only thing

that exposes the missing steps; explicit, measurable statements for quality requirements. You **clarify** the *why* by building goal hierarchies, and — crucially — you surface **conflicting goals** so the right owner resolves the trade-off on purpose rather than by accident in the code. And you **invert** the whole exercise for security, using attack trees to ask what an adversary wants, because cooperative users will never tell you.

None of this produces certainty, and it is not meant to. The purpose is to identify errors early, while correction is inexpensive, by investing a small amount of deliberate effort before committing to the wrong solution. This applies Chapter 1's engineering discipline to deciding *what* the team should build — the most consequential question in the project. The next chapters take these requirements and turn them into plans, designs, and tests — but each of those inherits the quality of the understanding you build here.

Sources

1 Suzanne Robertson & James Robertson, *Volere Requirements Specification Template* (Atlantic Systems Guild). volere.org.

2 Ron Jeffries, *Essential XP: Card, Conversation, Confirmation* (2001). ronjeffries.com.

3 Agile Alliance, *Three Amigos* (glossary entry; term credited to George Dinwiddie). agilealliance.org.

4 Hugh Beyer & Karen Holtzblatt, *Contextual Design: Defining Customer-Centered Systems* (Morgan Kaufmann, 1998). dl.acm.org.

5 Ryan Singer, *Shape Up*, ch. "Set Boundaries" (Basecamp, 2019). basecamp.com/shapeup.

6 Noriaki Kano, Nobuhiku Seraku, Fumio Takahashi & Shin-ichi Tsuji, "Attractive Quality and Must-Be Quality," *Journal of the Japanese Society for Quality Control* 14(2) (1984). jstage.jst.go.jp.

7 Mike Cohn, *Why the Three-Part User Story Template Works So Well* (Mountain Goat Software; credits the template to Rachel Davies and the Connextra team, early 2000s). mountaingoatsoftware.com.

8 Bill Wake, *INVEST in Good Stories, and SMART Tasks* (2003). xp123.com.

9 Cucumber, *Gherkin Reference* (documentation). cucumber.io.

10 Dan North, "Introducing BDD," *Better Software* (2006). dannorth.net.

11 Cucumber, *Behaviour-Driven Development* (documentation). cucumber.io.

12 Orlena C. Z. Gotel and Anthony C. W. Finkelstein, "An Analysis of the Requirements Traceability Problem," *Proceedings of the First International Conference on Requirements Engineering* (IEEE, 1994), pp. 94–101. doi.org.

13 Interaction Design Foundation, *What are Storyboards?* (traces storyboarding to Walt Disney Studios in the 1930s). interaction-design.org.

14 Axel van Lamsweerde, "Goal-Oriented Requirements Engineering: A Guided Tour," *Proceedings of RE'01* (IEEE, 2001). author PDF.

15 Bruce Schneier, "Attack Trees," *Dr. Dobb's Journal* (1999). schneier.com.

16 Microsoft, *The STRIDE model* (Microsoft Threat Modeling Tool documentation). learn.microsoft.com.

- **Key takeaways** are summarized above in §3.8.
- Continue to the Exercises.
- Go deeper with the Open Resources for this chapter.

Chapter 3 — Exercises

Exercises are graded by depth: **[warm-up]** checks understanding, **[analysis]** asks you to reason.

Concepts

1. **[warm-up]** In your own words, distinguish a *functional requirement* from a *quality (non-functional) requirement*. Write one of each for a photo-sharing app, and give the quality requirement a *measurable fit criterion*.
2. **[warm-up]** Name the four stages of the requirements cycle and, in one sentence each, say what goes wrong if a team skips that stage.
3. **[warm-up]** Classify each of the following as a *stated*, *implied*, or *latent* need for a food-delivery app, and justify the label: (a) "let me track the driver on a map," (b) "don't show me restaurants that are closed," (c) "reorder my usual meal in one tap before I even open the menu."
4. **[warm-up]** Rewrite this vague requirement into a testable one: "The search should be fast." State what extra information you had to invent, and why each piece matters.

Analysis

5. **[analysis]** Take a real need you have with some app you use and write it as a Connextra user story (*as a…I want…so that*). Then check it against all six INVEST letters, and for any letter it fails, describe the specific fix (split it, add acceptance criteria, schedule a spike, etc.).
6. **[analysis]** A teammate says "agile means we don't need to write requirements." Using §3.2, write a two-paragraph rebuttal that explains what agile teams actually do with requirements and *why* deferring detail to "the last responsible moment" is a feature, not laziness.
7. **[analysis]** Interviews and observation reach different kinds of needs. Describe a concrete situation where an interview would *mislead* you but observing the real task would reveal the truth, and explain the human reason for the gap.
8. **[analysis]** Pick a feature you consider a *delighter* in a product you love. Argue how it will eventually decay into a *must-be* expectation, and what that implies for a team that stops eliciting needs after version 1.
9. **[analysis]** Take the medical check-in scenario in §3.5.2 and write one *new* exception branch that the chapter did not cover. Then list the specific requirement(s) — including at least one quality requirement — that your exception uncovers.
10. **[analysis]** Sketch a small goal hierarchy (root goal, two sub-goals, three or four leaf features) for a bike-share app. Then identify one pair of *conflicting* goals and one pair of *contributing* goals, and write the one-sentence trade-off decision a stakeholder would need to record for the conflict.

11. **[analysis]** Build a three-level attack tree for the goal “post a message as another user” on a social app. Use at least three STRIDE categories to find branches, then turn two leaves into concrete security requirements, naming the attack each one defeats.
12. **[analysis]** A team’s backlog holds three *Must* stories (`ST-4` flag interpreter, `ST-9` cancel appointment, `ST-11` print day schedule). Their repo has feature files for `ST-4` and `ST-11`, a passing scenario for a story `ST-2` that no longer exists, and recent commit messages `"fix cancel flow"` and `"ST-11 print header"`. Using §3.4.4, list every traceability gap you can find (forward and backward), and state the one-line repair for each.

Chapter 3 — Open Resources

Free, open-licensed, or freely accessible materials that reinforce this chapter. Types: 📘 open text · 🎓 course · 📄 primary source/standard · 🎥 video. Licenses vary and are noted where known.

User stories, features, and BDD

- 🎓 **UC Berkeley — Engineering Software as a Service (ESaaS), ch. 7 (BDD & User Stories)** — saasbook.info. Develops user stories, behavior-driven development, and the Connextra "as a…I want…so that" template with worked examples. Course materials are freely available; the associated MOOC runs on edX.
- 📄 **INVEST in Good Stories** — Agile Alliance glossary, agilealliance.org/glossary/invest. The canonical short definition of Independent/Negotiable/Valuable/Estimable/Small/Testable , with links to Bill Wake's original 2003 article.
- 📄 **Bill Wake — "INVEST in Good Stories, and SMART Tasks" (2003)** — xp123.com. The original article that coined the INVEST checklist (§3.4.1). Free to read on the author's site.
- 📄 **Ron Jeffries — "Essential XP: Card, Conversation, Confirmation" (2001)** — ronjeffries.com. The classic statement that a story card is a token for a conversation, confirmed by acceptance tests — the "three Cs" behind §3.2.2. Free on the author's site.
- 📄 **Agile Alliance — "Three Amigos" (glossary)** — agilealliance.org/glossary/three-amigos. Short definition of the business/developer/tester conversation used in §3.2.2; credits the term to George Dinwiddie. Freely accessible.
- 📘 **Mountain Goat Software — User Stories & story writing (Mike Cohn)** — mountaingoatsoftware.com/agile/user-stories. Free articles on story structure, splitting epics, and acceptance criteria.
- 📄 **Mike Cohn — "Why the Three-Part User Story Template Works So Well"** — mountaingoatsoftware.com. Free article on the *as a… I want… so that* template; credits its origin to Rachel Davies and the team at Connextra (§3.4.1).

Gherkin, Given/When/Then, and BDD

- 📄 **Gherkin reference (Cucumber docs)** — cucumber.io/docs/gherkin/reference. The definitive syntax for `Feature` / `Scenario` / `Given` / `When` / `Then` / `Background` / `Scenario Outline`.
- 📘 **Behavior-Driven Development (Cucumber docs)** — cucumber.io/docs/bdd. What BDD is and how scenarios become executable specifications.
- 📄 **Dan North — "Introducing BDD"** — dannorth.net/introducing-bdd. The essay that named BDD and the Given/When/Then form.

- 🎓 **Berkeley ESaaS — BDD & user stories chapter** — saasbook.info. Ties Gherkin/Cucumber scenarios to acceptance testing in a full course.

Traceability

- 📄 **Orlena Gotel & Anthony Finkelstein — "An Analysis of the Requirements Traceability Problem" (ICRE 1994)** — doi.org. The classic statement of why traces decay and what forward/backward traceability must answer (used in §3.4.4). Publisher page; author preprints are widely indexed.

Writing and specifying requirements

- 📄 **Volere Requirements Specification Template** — volere.org. A widely used, freely downloadable template that organizes functional and non-functional requirements and popularized attaching a measurable *fit criterion* to each requirement.
- 📄 **Karl Wiegers — "Goodies" (free requirements resources)** — processimpact.com/goodies.shtml. Free templates, checklists, and articles on requirements elicitation, specification, and prioritization from a leading requirements author.
- 📄 **ISO/IEC/IEEE 29148:2018 — Requirements engineering** — iso.org/standard/72089.html. The international standard defining requirements processes and the qualities of a good requirement (unambiguous, verifiable, traceable, etc.). The standard itself is paid, but its vocabulary and criteria are summarized on many open university course pages — search "ISO 29148 characteristics of good requirements."

Eliciting and prioritizing needs

- 📄 **Noriaki Kano — the Kano model of customer satisfaction.** The origin of the must-be / performance / delighter distinction used in §3.3.3. The original 1984 study, Kano, Seraku, Takahashi & Tsuji, "Attractive Quality and Must-Be Quality" (*Journal of the Japanese Society for Quality Control* 14(2)), is archived on J-STAGE (abstract free; article in Japanese). Openly summarized in many free write-ups.
- 📄 **Hugh Beyer & Karen Holtzblatt — *Contextual Design: Defining Customer-Centered Systems* (Morgan Kaufmann, 1998)** — dl.acm.org. The book that defined *contextual inquiry*, the field-observation technique behind §3.3.2. The book is commercial; the ACM listing and many summaries are freely accessible.
- 🎓 **Interaction Design Foundation — user research & personas (free article set)** — interaction-design.org/literature. Open articles on contextual inquiry, interviews, and persona/scenario writing that back §3.3 and §3.5.
- 🎓 **Interaction Design Foundation — "What are Storyboards?"** — interaction-design.org/literature/topics/storyboards. Open topic page on storyboarding in design, including its film origins at Walt Disney Studios in the 1930s (§3.5.3).

Framing the problem

- 📘 **Ryan Singer — *Shape Up*, ch. “Set Boundaries” (Basecamp, 2019)** — basecamp.com/shapeup. Problem-framing, the *baseline*, and avoiding “grab-bag” requests (used in §3.3.2). Free to read online.

Goals and goal modeling

- 📄 **Axel van Lamsweerde — goal-oriented requirements engineering (KAOS).** The academic foundation for goal hierarchies, soft goals, and contributing/conflicting goals (§3.6). His tutorial paper “Goal-Oriented Requirements Engineering: A Guided Tour” (RE’01, IEEE) is posted openly by the author: webperso.info.ucl.ac.be/~avl/files/RE01.pdf.

Security requirements and attack trees

- 📄 **Bruce Schneier — “Attack Trees”** (*Dr. Dobb’s Journal*, 1999) — schneier.com/academic/archives/1999/12/attack_trees.html. The original, very readable introduction to modeling threats as trees (§3.7).
- 📄 **Microsoft — The STRIDE threat model** — Microsoft Threat Modeling Tool docs, learn.microsoft.com. Explains the six threat categories (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege) used in §3.7.2. Freely accessible.
- 📄 **OWASP — Threat Modeling & Application Security Verification Standard (ASVS)** — owasp.org/www-community/Threat_Modeling. Free, community-licensed guidance and checklists for turning threats into concrete security requirements.

License note

Linked resources remain under their own licenses; this page is CC BY-SA 4.0.

Chapter 4 — Requirements Analysis

> **Where we are.** Chapter 3 taught you to *elicit and record* what stakeholders need — user stories, features, scenarios. But a backlog of good stories is not a plan. Now comes the harder question every team faces on Monday morning: *given more work than we can possibly do, in what order, and how long will it take?* This chapter is about **requirements analysis** — sizing work, estimating effort, ranking by value, cost, and risk, and turning a pile of wishes into a defensible plan. These techniques help teams produce evidence-based forecasts, communicate uncertainty, and revise commitments responsibly as new information appears.

You have talked to users, and the whiteboard is full. There are forty things the clinic scheduling app "must" do, six people who each want theirs first, and a sponsor asking a question you cannot yet answer: *when will it be done, and what will it cost?* Guessing feels dishonest; refusing to answer feels unprofessional. The discipline in this chapter sits exactly in that gap. The method cannot eliminate uncertainty. It can produce **estimates that the team can explain, update as it learns, and use for explicit trade-offs.**

4.1 A Checklist Approach

The simplest analysis technique is also the most underrated: before you estimate or build anything, walk the requirement against a **checklist** of questions you have learned, the hard way, to ask. Checklists convert hard-won experience into a procedure that a tired person can follow at 4 p.m. without heroics. Atul Gawande's surgeons and Boeing's pilots rely on them for the same reason engineers should: not because the steps are difficult, but because *skipping* one is easy and expensive.[1]

A requirement is ready to be estimated with confidence only when you can answer questions like these:

- **Who** asked for it, and who else is affected? (A change to appointment reminders touches front-desk staff, patients, *and* the SMS-cost line item.)
- **Why** — what underlying goal does it serve? If you cannot state the goal, you cannot tell whether a cheaper solution would do.
- **How will we know it works?** What is the acceptance test? A requirement with no acceptance criterion is only a hope.
- **What does it depend on?** Does it assume a login system, a payment gateway, a data migration that does not exist yet?
- **What could go wrong?** Privacy, performance, edge cases, regulatory limits.

> **Principle.** A requirement you cannot *test* you cannot *estimate*, because you do not yet know what "done" means. The first job of analysis is to make each item concrete enough that reasonable people would agree when it is finished.

The payoff is twofold. First, the checklist surfaces hidden work early — the "small" reminder feature secretly requires a patient-consent flag, which requires a database column, a migration, and a privacy review. Second, it makes estimates *comparable*: when every story has been run through the same questions, the team is sizing like-for-like rather than comparing a well-understood task to a vague one. Keep your checklist short (five to ten questions), specific to your domain, and *living* — every nasty surprise in a retrospective is a candidate for a new line.

4.2 Relative Estimation: Iteration Planning

Humans are notoriously bad at estimating absolute quantities — ask people how many minutes a task will take and the answers vary widely. But we are surprisingly good at **relative** judgments: *this task is about twice as big as that one*.[2] Relative estimation exploits that strength. Instead of asking "how many hours?", you ask "how big is this compared to something we already understand?" This is the foundation of agile iteration planning.

4.2.1 Anchoring Can Bias Decisions

Before we use numbers, respect how easily they mislead. **Anchoring** is a cognitive bias: the first number mentioned drags every subsequent estimate toward it, even when it is arbitrary.[3] In a famous demonstration, people asked whether Gandhi died before or after age 140 later guessed a much higher age at death than people asked about age 9 — the absurd anchor still pulled them.[4]

On a team, anchoring is everywhere. The lead says "this should take about three days," and suddenly no one estimates two or eight; the whole discussion orbits three. The sponsor writes "$50k" in an email, and every proposal mysteriously lands near it. The danger is that the anchor carries the *authority* of a number without any of the *analysis* behind one.

> **Pitfall.** Never let a loud voice or an early figure set the anchor before the team has thought independently. The estimation methods later in this chapter — Planning Poker, Wideband Delphi — are *specifically engineered* to defeat anchoring by having people commit to a private estimate before anyone speaks.

4.2.2 Agile Story Points

A **story point** is a unit of *relative size*, not time.[2] When you say a story is 5 points and another is 2, you are claiming the first is about two-and-a-half times as much work — accounting for effort, complexity, and

uncertainty together. Points deliberately hide the hours, which frees the team from false precision and from the anchoring trap of committing to a clock.

Most teams draw point values from a restricted scale — commonly a Fibonacci-like sequence:

> **1, 2, 3, 5, 8, 13, 20, 40, 100**[5]

The gaps widen on purpose. You can tell a 2 from a 3, but no one can honestly tell a 21 from a 22, so the scale refuses to offer that false choice. A large gap also sends a signal: if a story looks like a 20, it is probably too big to plan confidently and should be **split** into smaller stories you understand better.

Points are calibrated against a **reference story** — a small, well-understood item the whole team agrees to call, say, a 2. Every other estimate is "bigger or smaller than the reference, by how much?" Because points are relative to *your* team's reference, a 5 on one team is not comparable to a 5 on another, and that is fine: points are an internal currency, not an industry constant.

Consider four stories for the clinic app, already run through the §4.1 checklist:

Story	Points
Show a read-only daily schedule (reference)	2
Book a new appointment with conflict detection	5
Send SMS reminders (needs consent flag + gateway)	8
Export a schedule to PDF	3
Total	**18**

It is worth knowing what points *replaced*. The traditional approach is **bottom-up, time-based estimation**: estimate each story in ideal days across its activities — design, code, test, deliver — then sum the story estimates into a total for the whole project. It is a perfectly logical two-step process, and teams moved away from it for three empirical reasons: the day-level numbers carry **false precision** (nobody can distinguish a 21-day story from a 22-day one, but the arithmetic pretends they can); ideal days ignore interruptions, meetings, and the sheer variance of real work, so the sums are systematically optimistic; and every number stated in days is an **anchor** (§4.2.1) that hardens into a commitment the moment a manager hears it. The approach is not dead, though — where a contract demands a full up-front bid, plan-driven teams still estimate bottom-up, usually with the formal models of §4.6.

4.2.3 Velocity of Work

Points become a *forecasting* tool the moment you measure **velocity** — the number of points a team actually completes in one iteration (sprint).[2] Velocity is measured, not decreed: you finish a sprint, add up the points of the

stories that met the definition of done, and that sum is your velocity for that sprint. Only *completed* work counts; a story that is 90% done contributes zero to velocity, because it has not met the definition of done and may not yet deliver usable customer value — you cannot bank a partial point.

Because a single sprint is noisy — someone was sick, the API was down — you smooth it. Take a rolling average of the last few sprints. Suppose your team's last three sprints closed:

> Sprint 1: **22** points Sprint 2: **18** points Sprint 3: **26** points

The average velocity is (22 + 18 + 26) / 3 = **22 points per sprint**. Now you can forecast. Imagine the whole clinic-app backlog totals **130** points. At 22 points per sprint:

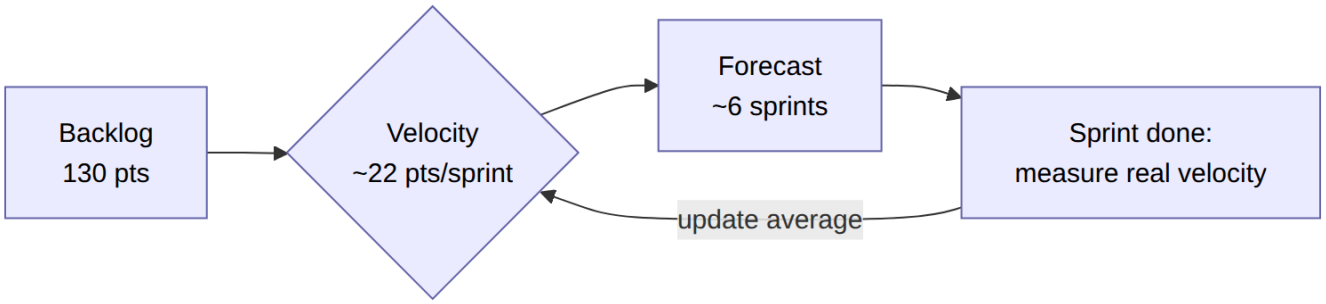
> 130 ÷ 22 ≈ **5.9 sprints**, so you round up to **6 sprints** of work remaining.

If a sprint is two weeks, that is 12 weeks, and with a start date you can name a completion range. Honest teams forecast a *range*, not a point: use your slowest recent sprint (18) for the pessimistic case and your fastest (26) for the optimistic one.

> Optimistic: 130 ÷ 26 ≈ 5.0 → **5 sprints** (10 weeks)
> Pessimistic: 130 ÷ 18 ≈ 7.2 → **8 sprints** (16 weeks)

So you tell the sponsor "10 to 16 weeks, most likely 12" — a defensible statement backed by data, not a heroic promise of an exact day. As sprints complete, velocity data accumulates, the range narrows, and the forecast sharpens. That is estimation in one sentence: **plan with what you know now, and re-plan every iteration as reality corrects you.**

> **Pitfall.** Velocity is a *planning* measure, never a *performance* target. The moment a manager rewards higher velocity, teams inflate their point estimates — a 3 quietly becomes a 5 — and the number stops meaning anything. Points are a ruler; do not bribe people to report longer inches.



4.2.4 Appetite: Fixed Time, Variable Scope

Everything so far *estimates* work: you look at a feature and ask "how big is this?" There is a provocative alternative from **Shape Up** (Basecamp's product-development method): don't estimate at all — set an **appetite**.[6]

An estimate and an appetite point in opposite directions:

- An **estimate** starts from a *design* and produces a *number*: "this feature, as specified, is about 13 points / three weeks."
- An **appetite** starts from a *number* and constrains the *design*: "this problem is worth about two weeks — now what solution fits in two weeks?"

An appetite is a **budget, not a prediction.** It flips the iron triangle (Chapter 1): instead of fixing scope and letting time flex, you **fix time and let scope flex**. The deadline is a **circuit breaker**: when the budget runs out, you ship what you have or drop the project rather than extending the schedule automatically.[6] A fixed limit brings the scope trade-offs forward, before the end-of-project rush.

> **Principle.** Fixed time, variable scope. A budget you cannot exceed turns "how long will it take?" into the far more useful question "what is the best thing we can build in the time this is worth?"

Making that work requires two disciplines:

- **Scope hammering.** Because time is fixed, you continually attack scope: for each use case and each piece of implementation, ask *is this essential? can we ship without it? is this a pre-existing problem we don't have to solve now?* This is sharper than the passive "cut if we have time" — you go looking for things to cut. Mark genuinely optional work as **nice-to-haves** (Shape Up prefixes them with a "~") and let them fall off the edge of the budget without guilt.[6] It connects directly to MoSCoW prioritization (§4.4.1): the appetite is the box, and MoSCoW decides what goes in it.
- **Judge against the baseline, not the ideal.** "Is it good enough to ship?" has no answer against a perfect vision — software is never done by that standard. Ask instead: *is it better than what users have today* (the **baseline**, introduced in §3.3.2)? That reframes an endless "not yet" into a shippable "yes, this beats the status quo."

Appetite and risk. An appetite is only trustworthy if the work reliably *fits* it. Shape Up's insight is to reduce the *variance* of a project before committing: a well-understood project has a **thin-tailed** duration (it lands near its appetite), while one hiding an unsolved design problem or an untested technical assumption — a **rabbit hole** — has a **fat tail** (it can run 3× over).[6] You de-risk *before* betting by finding those rabbit holes and either solving them or ruling them out of scope. (This is the shaping work discussed as a process in Chapter 2, §2.8.)

When appetites fit — and when they don't. Appetites shine for **discretionary, shapeable** features where "how good?" is negotiable and you'd rather have *something* valuable on a known date. They fit poorly where scope is essentially fixed and non-negotiable — a tax calculation must be *complete and correct*, not "80% of it by Friday." Much real work is discretionary, which is why fixed-time/variable-scope is a powerful default; the estimation techniques earlier in this chapter remain the right tool when scope truly cannot flex.

4.3 Structured Group Consensus Estimates

An estimate from one person is usually a guess. An estimate from a group *if you gather it well* is markedly better, because different people see different risks: the database expert knows about the migration, the front-end developer knows the UI is fiddly, the tester knows the edge cases. The trick is to combine their knowledge **without** letting the loudest or most senior person anchor everyone else. The following techniques are built to do just that.

4.3.1 Wideband Delphi and Planning Poker

Wideband Delphi (Barry Boehm's adaptation of the classic Delphi method for software) is a structured cycle: experts estimate privately, the estimates are revealed together, the outliers *explain their reasoning*, and everyone re-estimates.[7] The magic is in that middle step — the person who guessed 13 when everyone else guessed 3 usually knows something the others don't ("that feature needs a HIPAA audit log"), or has misunderstood the story. The estimate improves when the group examines the *reasons* behind disagreement; simply averaging the numbers loses that information.

Planning Poker is Wideband Delphi made fast and fun for agile teams.[8] Each estimator holds a deck of cards printed with the point scale (1, 2, 3, 5, 8, 13, …). For each story:

1. The product owner reads the story and answers questions.
2. Everyone privately picks a card, face down. *This defeats anchoring:* you commit before you hear anyone else.
3. On a count, all cards flip at once.
4. If the cards agree, record the number and move on.
5. If they diverge, the **highest and lowest** estimators explain their thinking. Then the team re-votes.

Repeat until the estimates converge — usually in two or three rounds.[5] Here is a walkthrough for the SMS-reminders story:

> **Round 1.** Cards flip: 3, 5, 5, **13**. The 13 is an outlier. The developer who played it explains: “Reminders need a patient-consent flag, and legally we can’t text someone who opted out — that’s a new database field, a migration, *and* a compliance check.” The 3 player, who had only pictured calling an SMS API, says, “I hadn’t thought about consent.”
>
> **Round 2.** Now that everyone shares the same picture, cards flip: 8, 8, 8, 13. Close. The 13 player admits the compliance check might be reused from an existing module, so it may be smaller than feared.
>
> **Round 3.** Cards flip: 8, 8, 8, 8. **Consensus: 8 points.**

Notice what happened. The final number matters less than the *conversation* it forced: the team discovered hidden work (consent) and a hidden opportunity (reuse) that a single estimator would have missed. The session’s main result is a shared understanding of the work; the point value records that understanding in a compact form.

4.3.2 The Original Delphi Method

Planning Poker’s ancestor is the **Delphi method**, developed at the RAND Corporation in the 1950s to forecast the effects of technology on warfare — questions with no data and no experts who could be safely gathered in one room.[9] Its designers, Olaf Helmer and Norman Dalkey, identified a real problem with panels: face-to-face groups are distorted by seniority, volume, and the bandwagon effect.[10] People converge on the boss’s view, or on whoever spoke first and loudest — the anchoring bias of §4.2.1, weaponized by social pressure.

Delphi’s answer was **anonymity plus iteration plus feedback**:

- A facilitator poses the question to experts *separately*; no one knows who said what.
- Responses are collected, summarized statistically (often just the median and the range), and fed back to the whole panel.
- Experts revise their answers in light of the summary, and the cycle repeats until the spread stabilizes.

Anonymity strips out status and volume; iteration lets people change their minds without losing face; statistical feedback replaces “the CEO thinks X” with “the median is X, and here is why the outliers disagree.” The result is a group judgment that reflects *knowledge* rather than *dominance*. Every consensus-estimation technique in this chapter — Wideband Delphi, Planning Poker — is a descendant of that insight, tuned for the tempo of a software team.

4.4 Balancing Priorities

Estimation tells you what work *costs*. It says nothing about what work is *worth*. A 90-point feature that three users want is a worse investment than a 5-point feature every user needs daily. Prioritization is the discipline of ordering the backlog so that you build the *most valuable* things first — which matters enormously, because if you run out of time or money (and you will), you want the *unfinished* work to be the *least* important.

4.4.1 Must-Should-Could-Won't (MoSCoW) Prioritization

MoSCoW — from the DSDM agile framework — sorts requirements into four blunt but powerful buckets.[11] The capital letters spell the name; the lowercase o's are filler.

- **Must have.** Non-negotiable. Without it the release is useless or illegal. If a Must slips, you slip the release.
- **Should have.** Important and painful to omit, but the product works without it. You will fight for these, but they can wait for the next release if forced.
- **Could have.** Desirable, low-cost, nice — the first to be dropped when time runs short. These are your *scope buffer*.
- **Won't have (this time).** Explicitly out of scope *for now*. Writing these down prevents scope creep and records a promise to revisit them later.

Requirement	MoSCoW	Why
Book/cancel an appointment	Must	The app's entire reason to exist
Detect double-booking conflicts	Must	Wrong data is worse than no app
SMS appointment reminders	Should	Big value, but staff can call if needed
Export schedule to PDF	Could	Nice for reporting; a workaround exists
Multi-language UI	Won't (yet)	No current demand; revisit next quarter

A healthy release is mostly *Should* and *Could*, with *Musts* held to roughly **60% or less** of the total effort — a rule of thumb, not a law.[11] Why the cap? Because if every item is a Must, you have no flexibility — the first delay forces a broken promise. The *Coulds* are the shock absorber that lets you hit the date with something shippable. A backlog with no Coulds is a plan with no margin.

> **Principle.** Prioritization is only real if it can *hurt*. If you are not willing to move something you like into "Won't," you have not prioritized — you have made a wish list, and a wish list cannot survive contact with a deadline.

4.4.2 Balancing Value and Cost

MoSCoW is fast but coarse. To order work *within* a bucket, weigh **value against cost**. A useful heuristic is the ratio:

> **priority score = value ÷ cost**

Estimate value on a simple scale (say 1–10, from stakeholder input) and cost in the story points you already have. Rank by the ratio and you get the most "bang per point" first — the classic strategy behind agile's preference for small, high-value slices. Here is the clinic backlog ranked:

Feature	Value (1–10)	Cost (pts)	Value ÷ Cost	Rank
Conflict detection	9	5	**1.80**	1
PDF export	4	3	**1.33**	2
SMS reminders	8	8	**1.00**	3
Multi-language UI	5	13	**0.38**	4

The ranking is instructive. SMS reminders carry a *high raw value* (8) — second only to conflict detection — yet they place third, because they cost as much value as they deliver. Humble PDF export, worth only 4, outranks them: it is cheap enough that its value-per-point is higher. **This is the core insight of value/cost analysis — the best next thing to build is the one with the best return on effort, and it is rarely the flashiest.**

4.4.3 Balancing Value, Cost, and Risk

Value and cost still miss a third dimension: **risk** — the probability that a story blows up in ways your estimate did not capture. A brand-new payment integration might be high value and modest cost, but if it depends on a bank's flaky sandbox API you have never touched, it carries real risk of overrunning or failing outright.

Risk cuts two ways, and *both* argue for tackling risky work early:

- **Threat.** A risk that could sink the project should be confronted early, while you still have time and budget to react — the *fail-fast* principle. Discovering in week two that the payment gateway can't do refunds is survivable; discovering it in week eleven is not.
- **Learning.** Risky stories are where you *learn the most*. Doing them early retires uncertainty and makes every later estimate more accurate.

A common technique is to fold risk into the score — where "risk" enters the formula not as danger for its own sake but as **risk-reduction value**: how much uncertainty doing the story *now* would retire. Rate that 1–10 (10 = doing it early burns down the most uncertainty) and compute a **weighted priority**. One transparent formula rewards value, penalizes cost, and adds a *bonus* for retiring risk early:

> **weighted priority = (value + risk reduction) ÷ cost**

Adding risk reduction to the numerator deliberately *pulls uncertain work forward* — you want to face it now, not last. (Read it any other way and the formula would appear to reward a feature merely for being dangerous.) Re-

ranking the backlog with a risk-reduction column — the low-value multi-language row has dropped off the short list, and a new payment-integration request has arrived:

Feature	Value	Risk red.	Cost	(V+R) ÷ Cost	Rank
Conflict detection	9	2	5	**2.20**	1
Payment integration	7	9	8	**2.00**	2
PDF export	4	1	3	**1.67**	3
SMS reminders	8	5	8	**1.63**	4

Recompute carefully: conflict detection scores (9+2)/5 = **2.20**, payment (7+9)/8 = **2.00**, PDF (4+1)/3 = **1.67**, SMS (8+5)/8 = **1.63**. So the order becomes **conflict detection, payment integration, PDF export, SMS reminders.** The risky payment work leaps from the bottom of the pack to near the top — exactly the intent. Conflict detection still leads because it is both valuable *and* cheap, but risk has promoted the payment work above two safer, comfier features. Pulling risk forward also does not have to mean building the whole feature next: often the right first move is a small, time-boxed **spike** — an investigation whose only deliverable is knowledge — which banks most of the risk-reduction value at a fraction of the cost.

> **Pitfall.** Do not let these formulas masquerade as objective truth. The inputs are judgments, and the formula's shape encodes a *strategy* (here, "confront risk early"). Use the number to *structure the conversation*, then let the team override it when they know something the model doesn't. A spreadsheet is a thinking aid, not an oracle.

Industry has named two widely used variants of this same idea. **WSJF (Weighted Shortest Job First)** prioritizes by *cost of delay ÷ job size* — how much you lose per unit time by not having the feature, divided by how big it is.[12] The (value + risk reduction) ÷ cost ranking above is a WSJF-family scheme: value and retired risk stand in for cost of delay, and story points for job size. The **risk–value matrix** trades the arithmetic for a 2×2 quadrant view: plot each feature by value and risk, then read the strategy off the quadrants — high-value/high-risk items go *first*, to burn off risk early; high-value/low-risk items can be scheduled with confidence; and low-value/high-risk items are candidates to drop outright. Both are the same principle you saw in the risk-first sequencing of §2.6 and the spiral framework of §2.7: face the dangerous work while you can still afford to be wrong about it.

4.5 Customer Satisfiers and Dissatisfiers

So far we have treated "value" as a single number. But not all value behaves the same way. Some features earn no praise yet cause fury if missing; others delight when present yet are never missed. Confusing the two wastes effort on the wrong things. The **Kano model** — whose three categories you first met in §3.3.3 — gives us the vocabulary to tell them apart.

4.5.1 Kano Analysis

In the 1980s, Noriaki Kano challenged the assumption that customer satisfaction rises linearly with how much you deliver.[13] He showed that satisfaction depends on the *kind* of feature, and that you learn the kind by asking customers **two** questions about each one:

- The **functional** question: “How do you feel if this feature is *present*?”
- The **dysfunctional** question: “How do you feel if this feature is *absent*?”

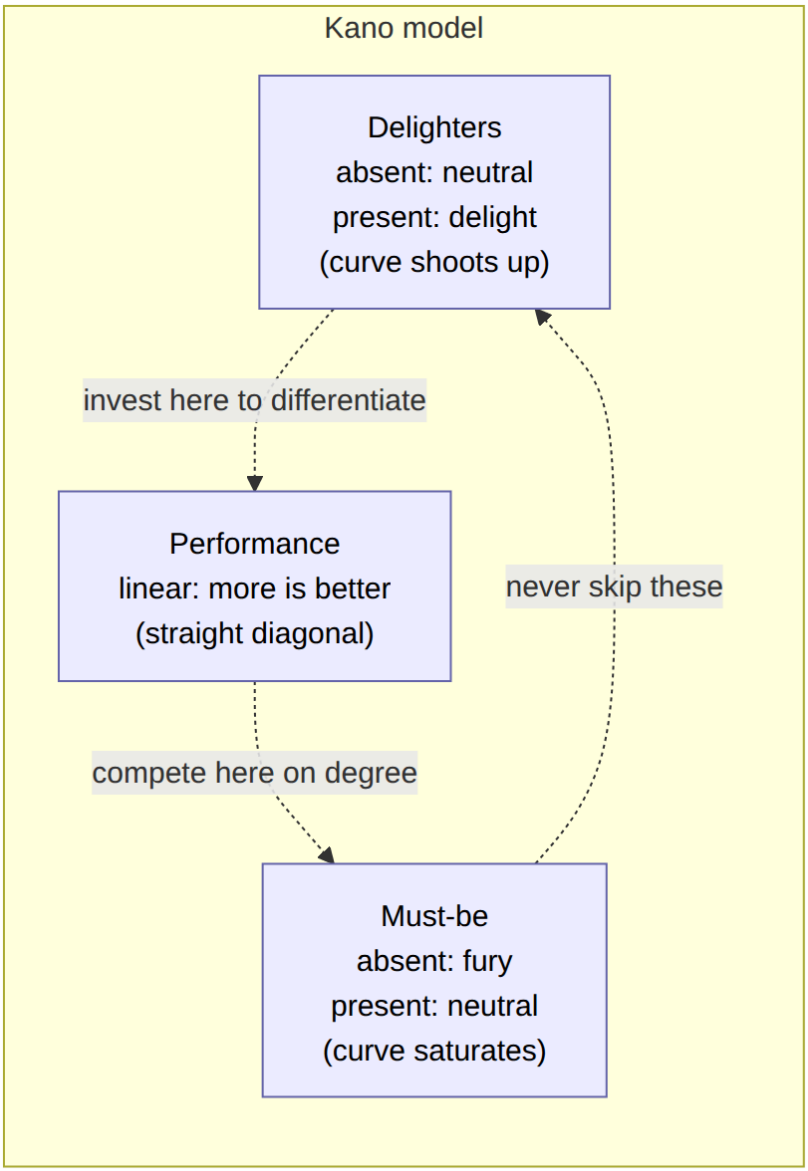
The *pair* of answers classifies the feature. Someone who is neutral about a feature being present but *furious* about its absence is describing a Must-be. Someone delighted by its presence but indifferent to its absence is describing a Delighter. A single question could never distinguish these; the two-question structure is Kano’s central methodological move.

4.5.2 Classification of Features

Kano’s questions sort features into categories.[13] The three that matter most for planning:

- **Must-be (basic) qualities.** Expected, unspoken, taken for granted. Their presence earns *nothing* — no one thanks you — but their absence causes strong dissatisfaction. For the clinic app: the schedule shows the *correct* date. No patient praises correct dates; every patient is enraged by a wrong one.
- **Performance (one-dimensional) qualities.** Satisfaction rises *linearly* with them, and customers will name them when asked. “How fast does search return?” More is better, proportionally. These are the features people comparison-shop on.
- **Attractive (delighter) qualities.** Unexpected features that *delight* when present but are not missed when absent — customers don’t know to ask for them. A one-tap “add this appointment to my phone calendar” button might delight. Their absence costs nothing; their presence differentiates you.

Two lesser categories round out the model: **Indifferent** (customers don’t care either way — a warning sign that a feature is wasted effort) and **Reverse** (some customers actively *dislike* it — one person’s helpful automation is another’s loss of control).



Picture the three categories as curves on shared axes: the horizontal axis is *how well the feature is delivered* (absent → fully present), the vertical axis is *customer satisfaction* (dissatisfied → delighted). **Must-be** features trace a curve that starts deep in dissatisfaction and merely rises to neutral — you can never make a customer happy with them, only avoid making them miserable. **Performance** features are the straight diagonal: deliver more, get more satisfaction, proportionally. **Delighters** hug the bottom (their absence does not hurt) then curve sharply upward (their presence thrills).

The planning lesson writes itself: **fund every Must-be first (they are the price of admission), compete on Performance features, and sprinkle in a Delighter or two to stand out** — but never chase Delighters while a Must-be is still broken.

4.5.3 Life Cycles of Attractiveness

Kano's subtlest insight is that categories **decay over time**.[14] Yesterday's delighter is today's expectation and tomorrow's basic requirement. A front-facing camera on a phone was once an *attractive* surprise; then, as every phone had one, it became a *performance* feature (megapixels mattered); now it is a *must-be* — ship a phone without one and customers will call it a defect.

The clinic app's SMS reminders followed the same arc. A decade ago, an automated text was a delighter. Today patients *expect* it; its absence generates complaints. It has aged from Delighter to Must-be.

> **Principle.** Because attractiveness decays, a product that stops innovating slides downhill even if it never changes. Yesterday's delighters silently become today's obligations, so you must keep discovering *new* delighters just to stay in place. Your Kano classification is a *snapshot*, not a permanent truth — re-survey periodically.

4.5.4 Degrees of Sufficiency

Kano also reframes the meaning of "enough." For **Must-be** qualities there is a *threshold* of sufficiency, and past it, more effort is wasted: a schedule that shows the correct date is sufficient; making it "extra correct" is meaningless. Over-investing in a basic quality buys you nothing, because it saturates at neutral.

Performance qualities have no hard ceiling — faster search is often better, until the gain drops below what users can perceive or what the improvement costs — so "sufficiency" here is an economic decision: keep investing while the marginal satisfaction exceeds the marginal cost, and stop when it doesn't. **Delighters** are all-or-nothing in a different sense: a half-built delighter often delights no one, so either fund it enough to actually thrill or don't start.

Kano analysis and value/cost analysis complement one another: Kano describes the *shape* of a feature's payoff curve, and value/cost shows *where on that curve to stop spending*. Used together, they show both which item should come next and what level of implementation is sufficient.

4.6 Plan-Driven Estimation Models

Story points and velocity work beautifully *once a team has history*. But what if you have none — a brand-new team, a fixed-bid contract, a proposal that must name a cost before a single sprint has run? For decades before agile, and still today for large or contractual projects, engineers used **algorithmic, plan-driven models** that estimate effort from the predicted *size* of the software. They are cruder in some ways and more rigorous in others, and every engineer should understand how they work.

4.6.1 How Are Size and Effort Related?

The intuitive guess is that effort scales *linearly* with size: twice the code, twice the work. Decades of project data say otherwise.[7] Effort grows **faster than linearly** — *super-linearly* — because of communication and integration overhead. A program twice as big has more than twice as many interactions between its parts, and a team twice as big has *far* more than twice as many communication paths (recall Brooks: n people have $n(n-1)/2$ channels).[15] Complexity compounds.

Models capture this with a **power law**:

> **Effort = a × Size^b**

where **Size** is measured in thousands of lines of code (**KLOC**) or function points, and a and b are constants fitted to historical projects. The exponent **b** is the crucial term. If $b = 1$, effort is linear. Real projects show **b slightly greater than 1** (typically 1.05–1.2), which is the mathematical signature of *diseconomy of scale*: unlike building identical widgets, where bigger batches get cheaper per unit, bigger software costs *more* per line.[7] Doubling the size more than doubles the effort. That single fact — that software exhibits diseconomies of scale — is the reason "just add more code" and "just add more people" both so often disappoint.

4.6.2 The COCOMO Family of Estimation Models

The best-known plan-driven model is **COCOMO** (the COnstructive COst MOdel), published by Barry Boehm in 1981[7] and revised as **COCOMO II** in the 1990s.[16] It is a family of power-law equations calibrated on real project data, and it makes the abstract "Effort = a × Size^b" concrete.

The original **Basic COCOMO** used three settings for a and b depending on project type:[7]

Project type	Description	a	b
Organic	Small team, familiar problem, stable reqs	2.4	1.05
Semi-detached	Medium size, mixed experience	3.0	1.12
Embedded	Tight constraints, hardware, high complexity	3.6	1.20

Effort comes out in **person-months** (one person working one month). Let us work the clinic scheduling app. Suppose your design suggests roughly **20,000 lines of code**, i.e. **KLOC = 20**, and it is a straightforward business app with a small experienced team — **organic** (a = 2.4, b = 1.05):

> Effort = 2.4 × 20^1.05
> 20^1.05 = 20 × 20^0.05. Now 20^0.05 = e^(0.05 × ln 20) = e^(0.05 × 3.00) = e^0.150 ≈ 1.16.
> So 20^1.05 ≈ 20 × 1.16 = 23.2.
> **Effort ≈ 2.4 × 23.2 ≈ 55.7 person-months.**

Now watch the diseconomy of scale bite. Double the size to **KLOC = 40**:

> Effort = 2.4 × 40^1.05
> 40^1.05 = 40 × 40^0.05 = 40 × e^(0.05 × ln 40)
> = 40 × e^(0.05 × 3.69) = 40 × e^0.184 ≈ 40 × 1.202 = 48.1.
> **Effort ≈ 2.4 × 48.1 ≈ 115.4 person-months.**

The code redoes the arithmetic at full precision:

```
# full precision: the hand math rounded 20^0.05 to 1.16, so the 55.7 above prints here as 55.8
def effort(kloc, a=2.4, b=1.05):          # Basic COCOMO, organic
  return a * kloc ** b

def schedule(e):
  return 2.5 * e ** 0.38                  # calendar months

e20, e40 = effort(20), effort(40)
print(f"20 KLOC: {e20:5.1f} person-months, {schedule(e20):.1f} months")
print(f"40 KLOC: {e40:5.1f} person-months, {schedule(e40):.1f} months")
print(f"doubling factor: {e40 / e20:.2f}")
```

```
20 KLOC:  55.8 person-months, 11.5 months
40 KLOC: 115.4 person-months, 15.2 months
doubling factor: 2.07
```

Doubling the code (20 → 40 KLOC) raised effort from ~56 to ~115 person-months — a factor of **2.07**, not 2.0. The extra 7% is the exponent *b* = 1.05 at work; on an *embedded* project (b = 1.20) the same doubling would cost a factor of 2^1.20 ≈ **2.30**, a 30% penalty. The model has turned "bigger software costs disproportionately more" from a slogan into a number you can put in a proposal.

COCOMO also estimates **schedule** (calendar months) from effort, again as a power law (roughly Time ≈ 2.5 × Effort^0.38 for the basic organic case), which lets you derive team size: dividing effort by schedule gives the average staff needed.[7] **COCOMO II** refines all of this with size in function points, *scale factors* for team maturity

and requirements volatility, and seventeen *effort multipliers* for things like personnel skill and tool support — but the core is still Effort = a × Size^b.[16]

> **Pitfall.** A COCOMO estimate is only as good as its *size* input, and estimating KLOC before you have written the code is itself hard and error-prone — you have pushed the uncertainty from "effort" onto "size," not removed it. Treat algorithmic estimates as *one input*, ideally cross-checked against expert judgment (Delphi) and, once available, real velocity. The best forecasts triangulate multiple methods that fail in different ways.

4.7 Conclusion

Requirements analysis is where a heap of stakeholder wishes becomes a plan you can defend. You met a toolkit, and its parts fit together:

- A **checklist** (§4.1) makes each requirement concrete enough to estimate at all — no acceptance test, no estimate.
- **Relative estimation** with **story points and velocity** (§4.2) turns a team's history into a forecast, and taught you to report a *range* and re-plan every sprint.
- **Consensus techniques** — Planning Poker and its ancestor, the **Delphi method** (§4.3) — combine many experts' knowledge while defeating anchoring and the bandwagon effect.
- **Prioritization** — MoSCoW, then value/cost, then value/cost/risk (§4.4) — orders the backlog so that whatever goes unfinished is the *least* important, and pulls risky work early.
- **Kano analysis** (§4.5) explains *why value is not one number*: must-bes, performance features, and delighters pay off in different shapes, and those shapes decay over time.
- **Plan-driven models** like **COCOMO** (§4.6) estimate effort from size via a power law when you have no velocity yet — and make the diseconomy of scale a number, not a hunch.

No single method is trustworthy alone. Points assume history; COCOMO assumes you can guess size; Kano assumes you can survey users; every formula rests on human judgment. The mature engineer uses several, watches where they *disagree*, and treats that disagreement as a signal to go learn more. Above all, remember the through-line from Chapter 1: estimates exist to make trade-offs **visible and negotiable** — scope, cost, and time argued in the open, with data, rather than settled by whoever talks loudest. That is the difference between a promise and a hope.

Sources

[1] Atul Gawande, *The Checklist Manifesto: How to Get Things Right* (Metropolitan Books, 2009). atulgawande.com.

[2] Mike Cohn, *Agile Estimating and Planning* (Prentice Hall, 2005). mountaingoatsoftware.com.

[3] Amos Tversky and Daniel Kahneman, “Judgment under Uncertainty: Heuristics and Biases,” *Science* 185(4157) (1974). science.org.

[4] Fritz Strack and Thomas Mussweiler, “Explaining the Enigmatic Anchoring Effect: Mechanisms of Selective Accessibility,” *Journal of Personality and Social Psychology* 73(3) (1997). doi.org.

[5] Mountain Goat Software (Mike Cohn), “Planning Poker: An Agile Estimating and Planning Technique.” mountaingoatsoftware.com.

[6] Ryan Singer, *Shape Up: Stop Running in Circles and Ship Work that Matters* (Basecamp, 2019). basecamp.com/shapeup.

[7] Barry W. Boehm, *Software Engineering Economics* (Prentice-Hall, 1981). dl.acm.org.

[8] James Grenning, “Planning Poker or How to Avoid Analysis Paralysis While Release Planning” (2002). wingman-sw.com. Popularized by Mike Cohn in *Agile Estimating and Planning* (2005).

[9] RAND Corporation, “Delphi Method” (topic overview). rand.org.

[10] Norman Dalkey and Olaf Helmer, “An Experimental Application of the Delphi Method to the Use of Experts,” *Management Science* 9(3) (1963); RAND memorandum RM-727/1. rand.org.

[11] Agile Business Consortium, “MoSCoW Prioritisation,” *DSDM Agile Project Framework Handbook* (2014). agilebusiness.org.

[12] Donald G. Reinertsen, *The Principles of Product Development Flow: Second Generation Lean Product Development* (Celeritas, 2009); the cost-of-delay ÷ job-size form follows Scaled Agile’s WSJF. scaledagile.com.

[13] Noriaki Kano, Nobuhiko Seraku, Fumio Takahashi, and Shinichi Tsuji, “Attractive Quality and Must-Be Quality,” *Journal of the Japanese Society for Quality Control* 14(2) (1984). doi.org.

[14] Noriaki Kano, “Life Cycle and Creation of Attractive Quality,” *Proceedings of the 4th International QMOD Conference* (2001). semanticscholar.org.

[15] Frederick P. Brooks, Jr., *The Mythical Man-Month: Essays on Software Engineering* (Addison-Wesley, 1975; anniversary ed. 1995). informit.com.

[16] Barry Boehm et al., *COCOMO II Model Definition Manual, version 2.1* (USC Center for Software Engineering, 2000). rose-hulman.edu mirror.

- **Key takeaways** are summarized above in §4.7.
- Continue to the Exercises.
- Go deeper with the Open Resources for this chapter.

Chapter 4 — Exercises

These exercises build the judgment behind requirements analysis: sizing work, forecasting from data, and ranking by value, cost, and risk. Tags indicate the kind of effort: **[warm-up]** checks understanding, **[analysis]** asks you to reason or compute. Several exercises require real arithmetic — do the math, and show your work.

Throughout, we reuse the **clinic scheduling app** from Chapters 1 and 3.

Concepts

1. **[warm-up]** For each vague requirement below, rewrite it so it passes the §4.1 checklist — in particular, give it a concrete **acceptance test** and name at least one dependency.

   - "The app should be fast."
   - "Patients should get reminders."
   - "Reports should look professional."

   Then explain, in one sentence each, why the original wording could not be estimated.

2. **[warm-up]** A teammate says, "Let's just call one story point one day of work, so 5 points = 5 days." Explain two concrete problems this creates, referring to *anchoring* (§4.2.1) and to the reason points are deliberately *relative* (§4.2.2).

3. **[warm-up]** Read this exchange:

   > **Lead:** "The export feature is trivial — maybe half a day?" **Junior dev:** "Yeah, that sounds about right, half a day."

   Identify the cognitive bias at work, explain how Planning Poker's *private, simultaneous* voting (§4.3.1) would have prevented it, and describe what the junior dev *should* have done differently.

Estimation and forecasting

4. **[analysis]** *Forecast from velocity.* Your team's completed points for the last four sprints were:

   > Sprint 1: **19** Sprint 2: **24** Sprint 3: **21** Sprint 4: **28**

   The remaining backlog totals **156 points**. Sprints are **two weeks** long.

a. Compute the average velocity. b. Forecast the number of sprints remaining (round up) and the calendar weeks. c. Give an *optimistic* and a *pessimistic* forecast using the fastest and slowest of the four sprints, and state the completion range you would tell a sponsor. d. Mid-Sprint-5, the sponsor adds 30 points of new "must-have" work. Redo the most-likely forecast. In one sentence, explain what this shows about fixed-scope promises.

5. **[analysis]** At sprint's end, a 13-point story is "almost finished — just needs testing." Your teammate argues it should count as, say, 11 points toward velocity since it is nearly done. Explain why agile counts 90%-done as **0**, and how counting partial credit would corrupt the forecast in Exercise 4.

6. **[analysis]** *Run a Planning Poker round.* Below are the Round-1 cards four estimators played for the story *"Let patients reschedule their own appointment online."* The scale is 1, 2, 3, 5, 8, 13.

   > Cards: **3, 5, 5, 13**

   a. Which two estimators must explain their reasoning first, and why those two? b. Invent a *plausible* reason the 13-player saw that the others missed, and a *plausible* reason the 3-player was too low. (Hint: think about consent, conflicts, notifications.) c. Write a believable Round-2 result and the final consensus, in the style of the walkthrough in §4.3.1. Explain why the *conversation*, not the number, was the real output.

Prioritization

7. **[analysis]** You have a fixed release date and can clearly not finish everything. Sort these into **Must / Should / Could / Won't-this-time**, and justify each placement in a sentence.

   - Book and cancel appointments
   - Detect scheduling conflicts
   - Dark-mode theme
   - HIPAA-compliant audit log of who viewed a record
   - Waitlist with automatic promotion when a slot opens
   - Multi-language interface

   Then explain why keeping *Musts* to roughly 60% of the effort (§4.4.1) protects your release date — what specifically goes wrong if 95% of the work is "Must"?

8. **[analysis]** Rank these features by **value ÷ cost** and give the build order. Show the ratio for each.

Feature	Value (1–10)	Cost (pts)
Waitlist auto-promote	7	8
Conflict detection	9	5
Dark mode	3	2
Insurance-card scan	8	13

Which feature has a high *raw* value (8) yet ranks **last**, and what does that teach about “build the flashiest thing first”?

9. **[analysis]** Take the same four features from Exercise 8 and add a **risk-reduction** rating (10 = doing it early retires the most uncertainty). The insurance-card scan needs an unfamiliar OCR service; the waitlist needs new notification plumbing.

Feature	Value	Risk red.	Cost
Waitlist auto-promote	7	6	8
Conflict detection	9	2	5
Dark mode	3	1	2
Insurance-card scan	8	9	13

a. Compute the **weighted priority = (value + risk reduction) ÷ cost** for each and give the new order. b. Compute how much each feature’s score changed from Exercise 8 (both the difference and the ratio). Notice that the *order* did not change — explain how the insurance-card scan’s large cost denominator swamps even a risk-reduction score of 9, and why the formula nevertheless encodes “tackle risky work early” (§4.4.3). c. The formula still ranks the two genuinely risky features last. Argue for or against overriding it to pull the insurance-card scan forward anyway — for example, spending one sprint on an OCR spike purely to retire its risk.

10. **[analysis]** For each feature, decide its Kano category (**Must-be / Performance / Attractive / Indifferent**) and justify it using the functional/dysfunctional question pair (§4.5.1).

    - The displayed appointment time is correct.
    - Search returns results faster.
    - A one-tap “add to my phone’s calendar” button.
    - The app’s loading spinner is a custom animation.

    Then pick one **Attractive** feature you named and argue how it might *decay* into a Performance or Must-be feature over five years (§4.5.3).

Effort models

11. **[analysis]** *COCOMO effort estimate.* Your team estimates the clinic app at **35 KLOC**. Using **Basic COCOMO**, compute the effort in person-months for:

    a. an **organic** project (a = 2.4, b = 1.05), and b. an **embedded** project (a = 3.6, b = 1.20).

    Show the arithmetic (you may use Effort = a × KLOC^b directly). Then:

    c. Explain the *ratio* between the two answers in terms of both *a* and *b*. d. The project actually grows to **70 KLOC**. Recompute the *organic* estimate and show that doubling the size raised effort by a factor of about **2.07**, not 2.0. Name the property of software this demonstrates (§4.6.1).

12. **[analysis]** *Triangulate three estimates.* For one real feature, you now have three independent estimates: Planning Poker says **8 points** (which your team's history suggests is about 2 person-weeks of work); COCOMO, from a KLOC guess, says **3 person-weeks**; a senior developer's gut says **1 person-week**. They disagree by 3×.

    a. Rather than averaging them, what should you *do* with the disagreement (§4.6.2 pitfall)? b. For each method, name one assumption that, if wrong, would explain its being an outlier. c. Which single method would you trust *most* once the team has run six sprints, and why?

Chapter 4 — Open Resources

Curated, freely available materials for going deeper on requirements analysis: estimation, consensus techniques, prioritization, satisfaction modeling, and algorithmic cost models. Legend: 📘 reading/article · 🎓 course/courseware · 📄 paper/standard · 🎥 video. Each entry notes access terms; "free to read" means available at no cost, though not always under an open license. Prefer these over paywalled textbooks.

Relative estimation, story points, and velocity

- 📘 **Mike Cohn — "What Are Story Points?" and related estimation articles** (Mountain Goat Software blog). The most widely cited practitioner writing on story points, velocity, and relative sizing. https://www.mountaingoatsoftware.com/blog/what-are-story-points *Free to read; © Mountain Goat Software. Concepts are freely reusable; do not copy text.*

- 📘 **Mike Cohn — main blog** (agile planning, splitting stories, release forecasting). https://www.mountaingoatsoftware.com/blog *Free to read.*

- 📘 **Mike Cohn — *Agile Estimating and Planning* (Prentice Hall, 2005).** The standard book-length treatment of story points, relative sizing, velocity, and release planning. https://www.mountaingoatsoftware.com/books/agile-estimating-and-planning *Commercial book; official page free to read. Check your library for the full text.*

- 🎓 **Atlassian Agile Coach — "Estimation" and "Sprints/Velocity" guides.** Clear, vendor-neutral tutorials with worked examples of forecasting from velocity. https://www.atlassian.com/agile/project-management/estimation *Free to read; © Atlassian.*

Consensus estimation: Planning Poker and Delphi

- 📘 **Planning Poker — overview.** The canonical description of the technique and its rules. https://www.mountaingoatsoftware.com/agile/planning-poker *Free to read; © Mountain Goat Software.*

- 📄 **James Grenning — "Planning Poker or How to Avoid Analysis Paralysis While Release Planning" (2002).** The original paper introducing Planning Poker, hosted by its author. https://wingman-sw.com/papers/PlanningPoker-v1.1.pdf *Free to download; © James Grenning.*

- 📘 **RAND Corporation — "Delphi Method" topic page.** RAND's own overview of the method's origin (1950s, Olaf Helmer and Norman Dalkey, forecasting technology's effect on warfare). https://www.rand.org/topics/delphi-method.html *Free to read; © RAND.*

- 📄 **Barry Boehm — Wideband Delphi** (described in *Software Engineering Economics*, 1981, and summarized in many open lecture notes). For an openly accessible summary, see the Wikipedia article, which cites primary sources: https://en.wikipedia.org/wiki/Wideband_delphi *Wikipedia text is CC BY-SA 4.0.*

- 📄 **Dalkey & Helmer — "An Experimental Application of the Delphi Method"** (RAND, 1963). The foundational report on the original Delphi method. RAND makes many historical reports freely downloadable. https://www.rand.org/pubs/research_memoranda/RM727z1.html *Free to download from RAND.*

Prioritization

- 📘 **DSDM / Agile Business Consortium — MoSCoW Prioritisation.** The source framework that defines Must/Should/Could/Won't and the effort-balancing guidance (e.g., capping Musts at roughly 60% of effort). https://www.agilebusiness.org/dsdm-project-framework/moscow-prioritisation.html *Free to read; © Agile Business Consortium.*

- 📘 **Scaled Agile — "WSJF (Weighted Shortest Job First)."** The cost-of-delay ÷ job-size prioritization scheme, adapted from Donald G. Reinertsen, *The Principles of Product Development Flow* (Celeritas, 2009). https://framework.scaledagile.com/wsjf *Free to read; © Scaled Agile, Inc. Reinertsen's book is commercial.*

- 📘 **Roman Pichler — "Product Prioritization" articles** (value vs. cost vs. risk, and when to sequence risky work early). Practical, well-illustrated. https://www.romanpichler.com/blog/ *Free to read; © Roman Pichler.*

Customer satisfaction: the Kano model

- 📘 **Kano model — overview and how-to** (Folding Burritos / Daniel Zacarias, "Complete Guide to the Kano Model"). A thorough, well-diagrammed practitioner explanation of the categories, the two-question survey, and category decay. https://foldingburritos.com/blog/kano-model/ *Free to read.*

- 📄 **Kano et al. (1984) — "Attractive Quality and Must-Be Quality."** The original paper introducing the model (*Journal of the Japanese Society for Quality Control* 14(2)). Free to read on J-STAGE: https://doi.org/10.20684/quality.14.2_147 *Copyrighted (in Japanese); the Wikipedia article (CC BY-SA) is a fair open starting point:* https://en.wikipedia.org/wiki/Kano_model

- 📄 **Kano (2001) — "Life Cycle and Creation of Attractive Quality"** (4th International QMOD Conference). Kano's follow-up showing categories decay over time — delighters age into must-bes. https://www.semanticscholar.org/paper/Life-Cycle-and-Creation-of-Attractive-Quality-

Kano/488788f1b49e92ee9ffb3bae6164c24374e1b2e4 *Copyrighted; abstract and citation record free via Semantic Scholar.*

Plan-driven / algorithmic estimation: COCOMO

- 📄 **Barry W. Boehm — *Software Engineering Economics* (Prentice-Hall, 1981).** The book that introduced COCOMO (and Wideband Delphi); the source of the Basic COCOMO a/b coefficients and schedule equations. https://dl.acm.org/doi/10.5555/539425 *Commercial book; ACM catalog page free. Check your library for the full text.*

- 📄 **Boehm et al. — "Cost Models for Future Software Life Cycle Processes: COCOMO 2.0"** (USC-CSE technical report, 1995). The primary open specification of COCOMO II, including the power-law form and scale/effort factors. The USC CSSE site is offline; archived copy: https://web.archive.org/web/20160910062006/http://csse.usc.edu/TECHRPTS/1995/usccse95-500/usccse95-500.pdf *Free to download (Internet Archive).*

- 📄 **Boehm et al. — *COCOMO II Model Definition Manual*, v2.1 (USC CSE, 2000).** The full COCOMO II specification: function-point sizing, five scale factors, seventeen effort multipliers. Widely mirrored by universities, e.g.: https://www.rose-hulman.edu/class/csse/csse372/201410/Homework/CII_modelman2000.pdf *Free to download; © USC.*

- 🎓 **USC Center for Systems and Software Engineering — COCOMO II tool page** (archived; the live site is offline). https://web.archive.org/web/20220319202437/https://csse.usc.edu/tools/COCOMOII *Free to use; © USC CSSE.*

- 📘 **Frederick P. Brooks, Jr. — *The Mythical Man-Month* (Addison-Wesley, 1975).** Source of the n(n−1)/2 intercommunication formula behind the diseconomy-of-scale discussion in §4.6.1. https://www.informit.com/store/mythical-man-month-essays-on-software-engineering-anniversary-9780201835953 *Commercial book; publisher page free. Check your library for the full text.*

Estimation, judgment, and its limits (background)

- 📘 **Steve McConnell — *Software Estimation: Demystifying the Black Art* companion materials / articles** (Construx resources). Practical guidance on estimate-vs-target, ranges, and re-estimation. https://www.construx.com/resources/ *Free to read; © Construx.*

- 📄 **Tversky & Kahneman (1974) — "Judgment under Uncertainty: Heuristics and Biases,"** *Science*. The primary source on anchoring — essential background for §4.2.1. Widely available as a free PDF via university course pages. https://www.science.org/doi/10.1126/science.185.4157.1124 *Copyrighted; free preprints commonly hosted by universities.*

- 📄 **Strack & Mussweiler (1997) — "Explaining the Enigmatic Anchoring Effect,"** *Journal of Personality and Social Psychology* 73(3). The Gandhi age-9/age-140 anchoring demonstration cited in §4.2.1. https://doi.org/10.1037/0022-3514.73.3.437 *Copyrighted (APA); abstract free at the DOI.*

- 📘 **Atul Gawande — *The Checklist Manifesto* (Metropolitan Books, 2009).** How surgery and aviation turned hard-won experience into checklists — background for §4.1. https://atulgawande.com/book/the-checklist-manifesto/ *Commercial book; author's page free. Check your library for the full text.*

Appetite: fixed time, variable scope

- 📘 **Ryan Singer — *Shape Up: Stop Running in Circles and Ship Work that Matters* (Basecamp, 2019).** The free online book behind §4.2.4: *appetite* vs. estimate, fixed-time/variable-scope, the circuit breaker, scope hammering, and judging against the baseline. See especially the Introduction and "Bets, Not Backlogs." https://basecamp.com/shapeup *Free to read online; © Basecamp.*

License note

Linked resources remain under their own licenses; this page is CC BY-SA 4.0.

Chapter 5 — Use Cases

> **Where we are.** Chapter 3 gathered raw needs as user stories and features, and Chapter 4 sized and prioritized them into a plan. Those artifacts are deliberately small and fragmented — a story fits on an index card. But a card tells you *that* a user wants to withdraw cash; it does not tell you what the machine says when the account is overdrawn, what happens if the network drops mid-transaction, or in what order the card and the money come out. A **use case** fills that gap. It tells the whole story of one goal, start to finish, including everything that can go wrong along the way. This chapter shows you how to write use cases that are precise enough to design and test against, yet readable enough that a nurse, a cashier, or a bank teller can confirm you understood them.

A user story says *who*, *what*, and *why* in a single sentence. That compression is a feature when you prioritize a backlog and a liability when you build. The moment a developer sits down to implement "As a customer, I want to withdraw cash so that I have money on hand," a swarm of questions appears: How does the customer prove who they are? What if the amount exceeds the balance? What if the machine is out of bills? Who gets the card back, and when? A **use case** answers exactly these questions by narrating a complete interaction between a user and the system, from the user's opening move to the moment their goal succeeds — or fails in a controlled, understood way.

Use cases occupy a sweet spot in the requirements toolkit: more structured and complete than a story, but cheaper and more readable than a formal specification. They are written in the language of the domain, not of code, so the people who have the need can validate them. And because a use case enumerates its own failure modes, it doubles as a checklist for design and a source of test scenarios — one artifact that pays rent in three later chapters.

5.1 Elements of a Use Case

Before writing one, you need the vocabulary. A use case is built from a small, fixed set of parts: **actors** who interact with the system, a **goal** those actors are trying to reach, and one or more **flows** — sequences of steps that describe how the interaction unfolds. Get these three ideas straight and the rest is largely a matter of disciplined writing.

5.1.1 Actors and Goals Outline a System

An **actor** is any entity *outside* the system that interacts with it to accomplish something. The most important word in that sentence is *outside*. An actor is a role played by someone or something that sits across the system's boundary and exchanges information with it; nothing inside your own software qualifies. When you name the actors, you are implicitly drawing that boundary — deciding what is "the system" and what is the world it serves.

Actors are **roles, not people**. The same human being may be a `Cardholder` when withdrawing cash and a `Bank Employee` when reconciling the till at day's end; those are two actors even though it is one person. Conversely, many different people play the single role `Cardholder`. Naming the *role* rather than the *person* keeps the use case stable when the org chart changes.

Actors come in two broad kinds, and the distinction matters:[1]

- **Primary actors** initiate a use case to reach a goal *of their own*. The `Cardholder` wants money; that desire is why the use case exists. Every use case should have one.
- **Supporting actors** (sometimes called secondary actors) are external systems or people the system calls on to get the job done, but whose *own* goals are not the point. When the ATM asks the `Bank Authorization System` to approve a withdrawal, that back-end service is a supporting actor: essential to the flow, but not the one whose goal drives the story.

A **goal** is what the primary actor wants the interaction to achieve — stated as a result in the world, not as a feature of the software. "Withdraw cash" is a goal. "Click the withdraw button" is merely a step toward one, and phrasing goals as UI actions bakes premature design decisions into requirements. A good test: a goal should still make sense if you replaced the entire user interface. People withdrew cash from human tellers long before ATMs existed; the goal outlived the implementation, and so should your phrasing.

> **Principle.** Name actors by the *role* they play and goals by the *result* they seek. Both should survive a complete redesign of the user interface. If a change of buttons would force you to reword the goal, you have described a screen, not a requirement.

Listing the actors and their goals — before writing a single flow — gives you a lightweight outline of the whole system.[2] It is often the first thing a team produces, and it fits on one page. The **actor–goal list** is simply a table: each row a goal, tagged with its primary actor.[3] For the ATM:

Primary actor	Goal (use case)
Cardholder	Withdraw cash
Cardholder	Check account balance
Cardholder	Deposit funds
Cardholder	Change PIN
Bank Employee	Reload cash cassettes
Bank Employee	Retrieve captured cards

That table is worth an afternoon of meetings. It surfaces goals nobody mentioned (who refills the machine?), reveals which actors matter, and gives the team a unit of work to estimate and schedule — one use case at a time.

5.1.2 Flows and Basic Flows

A **flow** is a numbered sequence of steps describing an interaction. Each step is a single, complete exchange: either the actor does something the system can perceive, or the system responds. The steps alternate like a conversation, and each one moves the goal forward.

The most important flow is the **basic flow** — also called the *main success scenario* or the *happy path*.[3] It describes what happens when everything goes right: the card is valid, the PIN is correct, the account has funds, the machine has cash, and the network stays up. The basic flow is deliberately optimistic. It is the shortest, cleanest story of the goal being achieved, and it should read as a smooth narrative with nothing branching off it.

Why start with the optimistic path? Because it defines the *spine* of the use case. Every complication you handle later is a deviation from some specific step of the basic flow, so you cannot describe the deviations until the spine exists — and writing the happy path first forces agreement on the essential shape before anyone gets lost in edge cases.

Here is the basic flow for *Withdraw Cash from ATM*, written the way a use case wants it:

1. The Cardholder inserts a bank card.
2. The system reads the card and prompts for a PIN.
3. The Cardholder enters a PIN.
4. The system validates the PIN with the Bank Authorization System.
5. The system displays the menu of services and the Cardholder selects **Withdraw Cash**.
6. The system prompts for an amount.
7. The Cardholder enters an amount.
8. The system confirms with the Bank Authorization System that the account holds sufficient funds and that the amount is within limits.
9. The system dispenses the requested cash.
10. The system returns the card.
11. The system prints a receipt and records the completed transaction.

Read those steps and notice their properties. Each is a whole interaction (not “the system sends a byte to the card reader” — that is design). Each names *who* acts. None mentions a screen layout, a database table, or a programming language. And the sequence tells a story a bank customer could read and confirm: *yes, that is how I withdraw money.* A basic flow must be clear enough for the domain expert to verify and correct.

> **Pitfall.** Steps that describe the system's *internals* ("the controller polls the dispenser queue") make a use case impossible for stakeholders to validate and tie your requirements to one implementation. Keep every step at the level of *observable behavior at the system boundary* — what an actor sends in and what they get back.

5.2 Alternative Flows: Conditional Behaviors

The basic flow is a lie of omission. In reality PINs are mistyped, accounts run dry, and machines run out of twenties. The parts of a use case that make it genuinely useful — and genuinely hard — are the **alternative flows** that describe what happens when the happy path is interrupted. This is where use cases earn their keep over user stories: a story hides these conditions; a use case drags every one you can find into the light and decides, on the record, what the system will do.

An alternative flow always attaches to a specific point in another flow. It answers: "At step *N*, if condition *C* holds instead, then this happens." Because each alternative names the step it branches from and the condition that triggers it, the reader sees exactly where reality departs from the happy path and where — if anywhere — it rejoins.

5.2.1 Specific Alternative Flows

A **specific alternative flow** branches at one identified step, handles a particular condition, and then either rejoins the basic flow or ends the use case. "Specific" means it is tied to a single step: the condition can only arise there, so the alternative lives next to that step and nowhere else.

Continuing the ATM example:

A1. Incorrect PIN *(at step 4)*

> If the Bank Authorization System reports the PIN is invalid:
>
> 1. The system tells the Cardholder the PIN was not recognized and how many attempts remain.
> 2. The system prompts for the PIN again.
> 3. The Cardholder re-enters a PIN, and the flow resumes at **step 4**. After three failed attempts, the flow continues at alternative **A2** (*Three incorrect PINs*: retain the card and end the use case — shown in the full template in §5.3.1).

A3. Insufficient funds *(at step 8)*

> If the account balance is below the requested amount:
>
> 1. The system informs the Cardholder that the account lacks sufficient funds.
> 2. The flow resumes at **step 6**, where the system prompts for an amount again.

Notice how each alternative states its trigger step, its condition, its steps, and its *resumption point* — the step number where control returns to the basic flow. That resumption point is the seam that spares the alternative from re-narrating the rest of the transaction. “Resume at step 6” is a promise that everything from step 6 onward behaves exactly as already written.

5.2.2 Extension Points

Sometimes you want to say “at this moment, additional behavior *may* occur” without committing, in the basic flow, to what that behavior is. An **extension point** is a named location in a flow where such optional behavior can be attached.[4] It is a labeled hook: the basic flow declares “here is a place where things may branch,” and one or more extending use cases (§5.5.3) plug in at that label.

Extension points shine when the extra behavior is optional and conceptually separate. In the ATM, after the amount is confirmed but before cash is dispensed, you might declare an extension point named `AmountConfirmed`. A separate use case, *Assess Withdrawal Fee* — relevant only for out-of-network cards — hooks onto that point. The core withdrawal flow stays clean; the fee logic lives in its own use case and attaches only where and when it applies.

The value is *separation of concerns in the requirements themselves*. The moment you find yourself writing “and if this is a premium customer, also do X, unless it’s a holiday, in which case Y,” you are overloading one flow. An extension point lets you name the hook and move the conditional story elsewhere, so each flow stays about one thing.

5.2.3 Bounded Alternative Flows

Some conditions are not tied to a single step — they can strike almost anywhere. A **bounded alternative flow** applies across a *range* of steps rather than at one point. The bound tells the reader “this can happen at any step from *i* to *j*,” which is far more honest, and far more compact, than copying the same alternative under every step in that range.

The canonical example is a resource that can vanish at any time. Suppose the network link to the Bank Authorization System can drop at any point during the withdrawal, whether or not the system is actively talking to it at that instant — a link that fails at step 5 is simply discovered at step 8:

B1. Authorization link lost *(at any step from 4 through 8)*

> If the connection to the Bank Authorization System is lost:
>
> 1. The system cancels the pending transaction.
> 2. The system displays an “out of service” message.
> 3. The system returns the card and ends the use case; no cash is dispensed.

Without the bound, you would repeat this handling under steps 4, 5, 6, 7, and 8 — five copies that drift out of sync the first time someone edits one. The bounded flow states the shared behavior once and names its scope. Use bounded flows for cross-cutting failures — timeouts, cancellation, loss of power, a user who walks away — that are indifferent to exactly which step you were on.

> **Principle.** Choose the *narrowest* structure that fits the condition. A condition that can arise at exactly one step is a specific alternative; one that ranges over several is a bounded alternative; optional behavior you would rather describe elsewhere gets an extension point. Matching the structure to the condition keeps each flow short and keeps duplication out.

5.3 Writing Use Cases

Vocabulary in hand, the practical question is how to produce good use cases. The answer has two parts: a **template** that gives every use case the same skeleton, and a **discipline** for filling that skeleton in — writing steps at the right level and building the document up in the right order.

5.3.1 A Template for Use Cases

A template is a checklist wearing the costume of a document. Its job is to make sure you never forget the preconditions, never leave the postconditions implicit, and never ship a use case whose failure modes went unconsidered. Teams differ on the fields, but a solid, widely used template contains the following — shown here fully filled in for the ATM example, so you see a complete artifact rather than an empty form:[3]

Use case: Withdraw Cash from ATM

Primary actor: Cardholder

Supporting actors: Bank Authorization System

Scope: The ATM terminal software.

Level: User goal — a single sitting at the machine.

Brief: A cardholder authenticates at an ATM, requests an amount of cash, and — if the account and machine can satisfy the request — receives the cash, their card, and a receipt.

Stakeholders and interests:

- *Cardholder* — wants correct cash quickly, and their card back.
- *Bank* — wants every disbursement authorized and accurately recorded; no money out without a matching debit.
- *Regulator* — wants an auditable record of each transaction.

Preconditions:

- The ATM is in service and has cash available.
- The Cardholder possesses a valid bank card.

Trigger: The Cardholder inserts a card into the ATM.

Basic flow (main success scenario):

1. The Cardholder inserts a bank card.
2. The system reads the card and prompts for a PIN.
3. The Cardholder enters a PIN.
4. The system validates the PIN with the Bank Authorization System.
5. The system displays the service menu; the Cardholder selects **Withdraw Cash**.
6. The system prompts for an amount.
7. The Cardholder enters an amount.
8. The system confirms sufficient funds and within-limit amount with the Bank Authorization System.
9. The system dispenses the cash.
10. The system returns the card.
11. The system prints a receipt and records the completed transaction.

Alternative flows:

- **A1. Incorrect PIN** *(step 4)* — inform the Cardholder, re-prompt; resume at step 4.
- **A2. Three incorrect PINs** *(step 4)* — retain the card, end the use case.
- **A3. Insufficient funds** *(step 8)* — inform the Cardholder; resume at step 6.

- **A4. Amount exceeds machine cash on hand** *(step 9)* — inform the Cardholder, offer a smaller amount; resume at step 6.
- **B1. Authorization link lost** *(steps 4–8)* — cancel, show “out of service”, return the card, end the use case.

Postconditions (on success):

- The requested amount has been debited from the account exactly once.
- The Cardholder holds the cash, the card, and a receipt.
- The transaction is recorded in the bank’s ledger.

Postconditions (on failure):

- No money has left the machine and no debit has been recorded.
- The Cardholder holds the card, unless it was retained under **A2**.

Study the two *postcondition* sections, because they are where beginners are weakest. Every use case has both a success guarantee and a failure guarantee, and the failure guarantee is often the more important: “no money out without a matching debit” is a promise the bank cares about far more than the happy path. Writing both forces you to decide, in advance, what must remain true even when things go wrong — precisely the invariant your error handling and your tests will have to enforce.

The failure guarantee is testable before any real bank or dispenser exists — drop the authorization link at step 8 and check the postcondition, word for word:

```
class LostLinkBank:                          # the fake — the link drops at step 8
  debits = []
  def authorize(self, amount): raise ConnectionError("link lost")

class Atm:
  dispensed = 0
  def dispense(self, amount): self.dispensed += amount

def withdraw(atm, bank, amount):             # steps 8-11 of the basic flow
  try: bank.authorize(amount)
  except ConnectionError: return           # B1 — cancel, return the card
  atm.dispense(amount)
  bank.debits.append(amount)

atm, bank = Atm(), LostLinkBank()
withdraw(atm, bank, 200)
assert atm.dispensed == 0 and bank.debits == []  # the failure postcondition, verbatim
```

5.3.2 From Actor Intentions to System Interactions

A use case is a translation. On one side is the actor's **intention** — a goal expressed in human terms: "I want cash." On the other side is a sequence of **interactions** at the system boundary: insert card, enter PIN, select service, and so on. Writing a use case is the act of turning the first into the second without leaking implementation on the way.

The reliable technique is to alternate between two questions as you write each step:

1. *What does the actor intend here?* — This keeps the step goal-directed. The Cardholder does not "enter digits into field two"; they *identify themselves* so the bank will trust them.
2. *What is the smallest observable interaction that serves that intention?* — This keeps the step concrete and testable. Identifying oneself becomes "enters a PIN," a real exchange at the boundary.

Hold each step at this **intention level**: concrete enough to be an observable exchange, abstract enough to avoid dictating the interface. Compare three ways to write the same step:

- *Too abstract:* "The Cardholder authenticates." (What actually happens? Untestable.)
- *Too concrete:* "The Cardholder types four digits into the PIN field and taps ENTER, and the system SHA-256-hashes them and compares against the stored hash." (This is design; it forbids a PIN of other lengths, a keypad, biometrics — decisions requirements should leave open.)
- *Just right:* "The Cardholder enters a PIN; the system validates it with the Bank Authorization System." (Observable, testable, implementation-free.)

The middle level survives contact with design. It tells the builders *what must happen* and leaves them free to decide *how* — the whole point of separating requirements from design.

5.3.3 How to Build Use Cases

Do not try to write a use case top to bottom in one pass. That way lies analysis paralysis — you will be arguing about a rare timeout before you have agreed on the main story. Build each use case **incrementally**, in three passes, widening your focus each time:

Pass 1 — Actors and goals. Before any prose, list the actors and their goals as in §5.1.1. Decide which goals are worth a full use case (user-goal level) and which are merely steps. Output: the actor–goal table. This pass is about *breadth* — the whole system at a glance — and it is cheap enough to redo daily as understanding shifts.

Pass 2 — The basic flow. Pick one goal and write only its happy path: the numbered main success scenario, assuming everything cooperates. Resist every temptation to handle errors here. Output: a spine of 3–10 steps that a domain expert can read and confirm. If the basic flow runs past a dozen steps, your use case is probably two use cases wearing a trench coat — split it.

Pass 3 — The alternatives. Now walk the basic flow step by step and, at each one, ask: *what else could happen here?* Wrong PIN at step 4. Empty account at step 8. Empty machine at step 9. Dead network across steps 4–8. Each answer becomes a specific alternative flow, a bounded alternative flow, or behavior attached at an extension point (§5.2). This pass is where the use case becomes trustworthy, because it is a systematic sweep for failure rather than a hope that you thought of everything.

This order is not arbitrary. It moves from *breadth* (all goals) to *depth on one goal* (the spine) to *robustness* (every deviation) — and at each pass you have something complete and reviewable, so you can stop, get feedback, and adjust before investing in the next layer. It is the same iterative instinct that runs through the whole book: reach a coherent whole early, then refine it, rather than perfecting one corner before you know the shape.

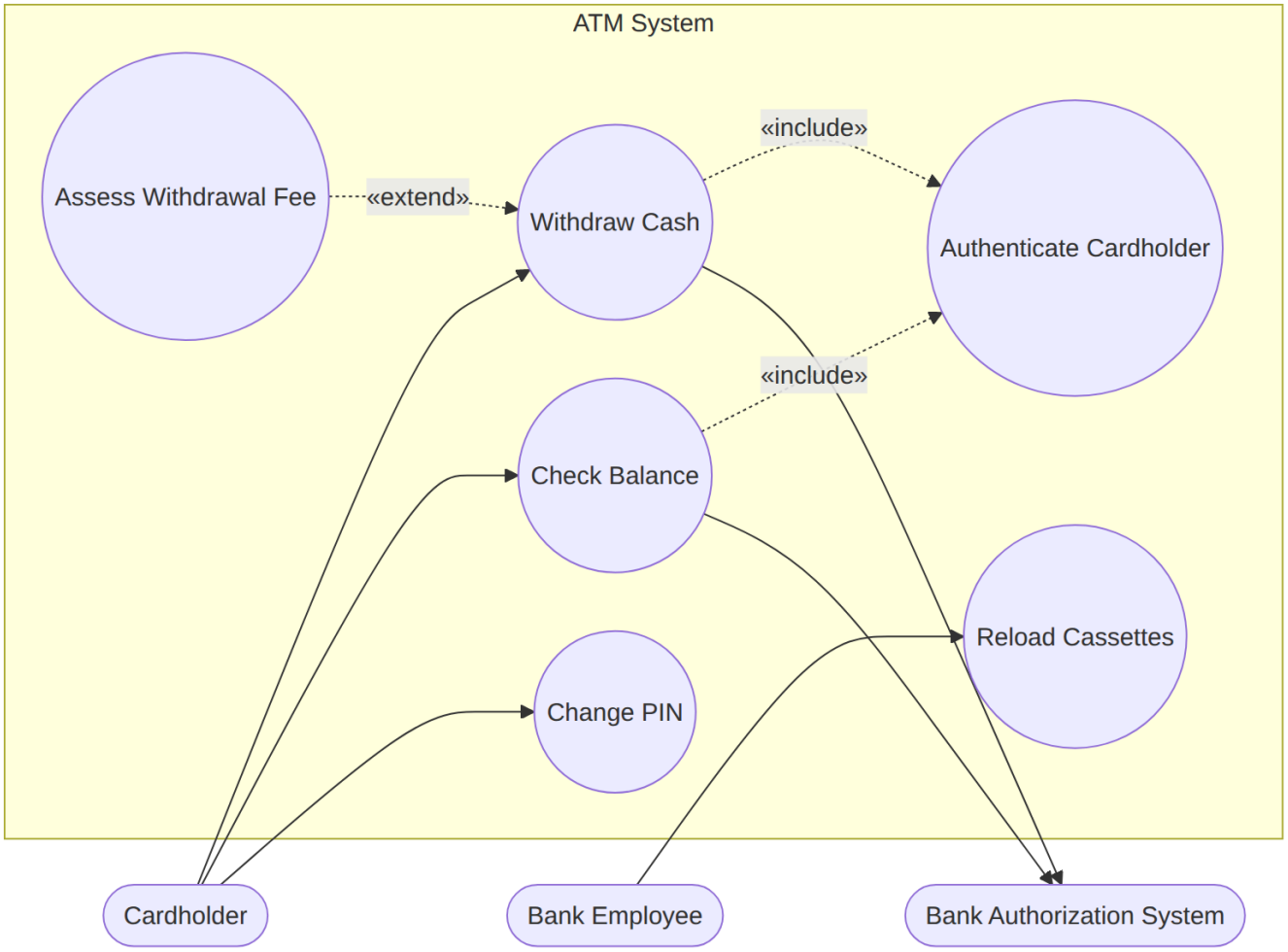
> **Pitfall.** Writing alternatives before the basic flow is stable is wasted work. Every time you reorder or reword a main-flow step, its attached alternatives shift with it. Stabilize the spine, *then* hang the exceptions on it.

5.4 Use-Case Diagrams

Everything so far has been text, because the *content* of a use case is a story and stories are made of words. But a project may have dozens of use cases and many actors, and at that scale you want a map. A **use-case diagram** is that map: a one-glance picture of who the actors are, what goals the system offers them, and how those goals relate.

5.4.1 Diagrams Highlight Goals and Actors

A use-case diagram uses a tiny visual vocabulary. **Actors** are drawn as stick figures (or labeled boxes for non-human systems). **Use cases** are ovals labeled with a goal. An **association** is a plain line connecting an actor to each use case in which it participates.[4] This minimal notation keeps the diagram focused on *scope and relationships*; the detailed flows remain in the use-case text.



Read that diagram as an index, not a specification. It tells you the `Cardholder` has three goals (`Deposit Funds`, the third goal in §5.1.1's actor–goal list, is omitted here to keep the diagram readable, as is the Bank Employee's `Retrieve captured cards`), that `Withdraw Cash` and `Check Balance` both depend on `Authenticate Cardholder` (the dashed `«include»` arrows), and that `Assess Withdrawal Fee` optionally attaches to `Withdraw Cash` (the `«extend»` arrow). It does *not* tell you what happens on a wrong PIN — for that you read the text. A common and costly mistake is to encode control flow, loops, and conditions into the diagram with dozens of arrows; the result is an unreadable tangle that says less than the plain list of use cases it replaced.

> **Pitfall.** A use-case diagram is not a flowchart and not a data-flow diagram. If your diagram has arrows showing the *order* steps happen in, you are misusing the notation. Order lives in the numbered flows; the diagram shows only *which actors reach which goals* and *how goals relate*.

5.5 Relationships between Use Cases

As a system grows, its use cases start to share material. Two goals both require login; five flows all contain the same three-step payment ritual; one goal is really the base goal plus an optional twist. Left unmanaged, this shared material gets copied, and copies drift. The last piece of the toolkit is a small set of **relationships** that let use cases share behavior without duplicating it: subflows, `«include»`, and `«extend»`.

5.5.1 Subflows

Within a single use case, a **subflow** is a named, reusable chunk of steps that the main flow can invoke by name. It is the use-case equivalent of a local helper function: purely an organizing device *inside* one document, going nowhere else. When the basic flow would otherwise spell out the same five steps in three places, you factor them into a subflow — say, `Print Receipt` — and refer to it by name where needed.

Use a subflow when a flow gets long or repeats itself. "Then perform *Print Receipt*" reads better than five inlined steps, and if the receipt process changes you edit it in one spot. But keep subflows *local*: the moment a chunk of behavior is wanted by a *different* use case, a subflow is the wrong tool and you reach for `«include»` instead.

5.5.2 Inclusion of Use Cases «include»

An **include** relationship factors shared behavior out into its own use case that other use cases invoke. When use case A `«include»`s use case B, it means "at this point, A performs all of B, then continues."[4] It is textual composition across use-case boundaries — the same idea as calling a shared function, raised to the level of requirements.

Use `«include»` when several use cases share the same stretch of behavior. Both `Withdraw Cash` and `Check Balance` begin by authenticating the cardholder; rather than copy the PIN-entry-and-validation steps into each, you extract them into an `Authenticate Cardholder` use case and have both includers invoke it:

- In *Withdraw Cash*, steps 2–4 become: "**Include** *Authenticate Cardholder*."
- In *Check Balance*, the same line appears.

Now the authentication rules — retry limits, card capture, lockout — live in exactly one place. Change them once and every including use case inherits the change. The tell-tale sign that you want `«include»` is duplication you must maintain in parallel: the same behavior, needed *unconditionally* by two or more base use cases. The included use case runs every time the base one reaches that point; inclusion is not optional.

5.5.3 Extensions of Use Cases «extend»

An **extend** relationship is the mirror image of include. Where include is *mandatory behavior pulled out of* a base use case, extend is *optional behavior added onto* one — and the base use case does not know its extender exists. When use case E `«extend»`s use case A at extension point P, it means "*if* E's condition holds when A reaches point P, then E's steps run; otherwise A proceeds unchanged."[4] The base flow is complete and sensible on its own; the extension is an optional overlay.

This is the `AmountConfirmed` extension point from §5.2.2. `Withdraw Cash` is a complete, correct use case with no fee logic in it at all. The *Assess Withdrawal Fee* use case extends it at `AmountConfirmed`, contributing its steps only when the card is out-of-network. If tomorrow the bank drops all fees, you delete one extending use case and never touch `Withdraw Cash`.

The direction of dependency matters enough to state plainly:[4]

- With **include**, the *base* names the *included* use case ("Withdraw includes Authenticate"). The base depends on the shared part.
- With **extend**, the *extending* use case names the *base* ("Assess Fee extends Withdraw"). The base is oblivious; the extension depends on it.

That difference decides which relationship you want. If the base flow would be *incomplete or wrong* without the behavior, it is essential — use `«include»`. If the base flow is *complete on its own* and the behavior is an optional, conditional, or specialized add-on you would like to keep separable, use `«extend»`. Choosing correctly keeps your common cases readable and your rare cases out of the way.

> **Principle.** Reach for a relationship only to remove real duplication or to isolate genuinely optional behavior — never for its own sake. Three well-chosen `«include»`s can clarify a system; twenty speculative ones bury it. If a relationship does not save you from maintaining the same steps in two places, delete it and inline the steps.

5.6 Conclusion

A use case is the bridge between *what a user wants* and *what a system must do*. It takes the compressed intent of a user story and unfolds it into a complete, validated narrative: the actors and their goals, the happy path that reaches each goal, and — most valuably — the alternative flows that say exactly what happens when reality refuses to cooperate. Along the way you met a template that guarantees you consider preconditions, postconditions, and failure guarantees; a discipline for writing steps at the intention level, neither too vague to test nor too concrete to design against; and an incremental build order — actors and goals, then the basic flow, then the alternatives — that keeps something reviewable in front of you at every stage.

Three ideas are worth carrying forward. First, **the failure modes are often the most revealing part of the requirements**; anyone can describe the happy path, but the value of a use case is its honest catalog of what can go wrong and what the system promises anyway. Second, **keep every step at the system boundary** — observable behavior in the language of the domain — so stakeholders can validate it and designers stay free to build it well. Third, **structure only to remove duplication**: use subflows, `«include»`, and `«extend»` to share behavior, and use the diagram only to map the terrain, never reaching for a mechanism you do not need.

Use cases are the handoff from requirements work to design. Because a use case names its actors, its data, and its every branch, it feeds directly into the domain model and architecture of Chapter 6, and its alternative flows and postconditions become ready-made scenarios for the tests of Chapter 10. Written well, a use case is the same artifact that lets a bank teller confirm you understood them, lets an architect decompose the system, and lets a tester know when it is done — which is a great deal of value from one disciplined story.

Sources

[1] Ivar Jacobson and Alistair Cockburn, *Use-Case Foundation*, v1.1 (2024). alistaircockburn.com.

[2] Alistair Cockburn, “Structuring Use Cases with Goals,” *Journal of Object-Oriented Programming* (1997). web.archive.org.

[3] Alistair Cockburn, *Writing Effective Use Cases* (Addison-Wesley, 2001). informit.com.

[4] Object Management Group, *OMG Unified Modeling Language (UML)*, v2.5.1, clause 18 “UseCases” (2017). omg.org.

- **Key takeaways** are summarized above in §5.6.
- Continue to the Exercises.
- Go deeper with the Open Resources for this chapter.

Chapter 5 — Exercises

Exercises are graded by depth: **[warm-up]** checks understanding, **[analysis]** asks you to reason.

Concepts

1. **[warm-up]** Define *actor*, *primary actor*, and *supporting actor* in your own words. For the ATM example, name one of each and say what distinguishes the primary from the supporting one.
2. **[warm-up]** Explain the difference between a *goal* and a *step*. Rewrite each of these as a proper goal (a result in the world, not a UI action): "click the Submit button," "scroll to the bottom of the terms," "log in."
3. **[warm-up]** What is the *basic flow* (main success scenario), and why do you write it before any alternative flow? Give the one-sentence reason in terms of what the alternatives depend on.
4. **[warm-up]** Match each condition to the right structure — *specific alternative*, *bounded alternative*, or *extension point*: (a) the user cancels at any screen; (b) the entered coupon code is expired; (c) an optional gift-wrap step offered only in December.

Analysis

5. **[analysis]** Take the ATM step *"The Cardholder enters a PIN; the system validates it with the Bank Authorization System."* Rewrite it once *too abstractly* and once *too concretely* (leaking design), then explain in a sentence what each version costs you.
6. **[analysis]** A teammate's use case has a success postcondition but no failure postcondition. Using the ATM's "no money out without a matching debit" guarantee as a model, explain why the missing failure postcondition is a real defect, and write a failure postcondition for a use case *Transfer Funds Between Accounts*.
7. **[analysis]** You are told to model a shared *login* step used by five different use cases, and an optional *apply loyalty discount* step that only applies to members during a sale. Decide whether each should be an `«include»` or an `«extend»`, and justify each choice by naming which use case depends on which.
8. **[analysis]** Critique this use-case diagram decision: a colleague has drawn arrows between ovals to show that "Withdraw Cash" happens *before* "Print Receipt," with a labeled condition on the arrow. Explain what is wrong, and state where that ordering and condition information actually belongs.
9. **[analysis]** Convert this user story into a full use case using the template from §5.3.1: *"As a patron, I want to renew a borrowed library book online so that I can keep it longer without visiting the branch."* Include a primary actor, at least one supporting actor, preconditions, a numbered basic flow, at least two alternative flows (one specific, one bounded), and both success and failure postconditions.
10. **[analysis]** Write a use case from scratch for **"Book a Clinic Appointment"** by an online patient. Produce the actor–goal list first (Pass 1), then the basic flow (Pass 2), then at least three deviations from the happy

path (Pass 3), labeling each as a specific alternative, a bounded alternative, or behavior attached at an extension point. Note explicitly which step each deviation branches from.

11. **[analysis]** Take your answer to Exercise 10 and identify one stretch of behavior it shares with a second plausible use case for the same clinic system (e.g., *Reschedule Appointment*). Factor that shared behavior into an `«include»`d use case, and show the single line that now replaces the shared steps in each includer.

Chapter 5 — Open Resources

Free, open-licensed, or freely accessible materials that reinforce this chapter. Types: 📘 open text · 🎓 course · 📄 primary source / reference · 🎥 video · 🛠️ tool. Licenses vary and are noted where known.

Foundations: what a use case is

- 📄 **Ivar Jacobson & Alistair Cockburn — *Use-Case Foundation* (v1.1)** — alistaircockburn.com. A short, freely downloadable joint statement of the core concepts — system of interest, primary and supporting actors, goals, and scenarios — from the two people most associated with the practice.
- 📄 **Alistair Cockburn — "Structuring Use Cases with Goals" (JOOP, 1997)** — archived copy at web.archive.org. Cockburn originated much of the modern practice, including goal levels, the fully dressed vs. casual template, and the "main success scenario / extensions" structure this chapter follows. The paper is freely readable at the archived link; his current site is alistaircockburn.com.
- 📄 **Alistair Cockburn — *Writing Effective Use Cases* (Addison-Wesley, 2001)** — official catalog page at informit.com. The definitive book-length treatment behind §5.1–§5.3. The book itself is commercial, but the publisher posts a substantial free sample-pages PDF covering the writing process and a fully dressed example.
- 📘 **UC Berkeley — Engineering Software as a Service (ESaaS)** — saasbook.info. The requirements chapters connect user stories (this book's Chapter 3) to more structured behavioral descriptions, with an agile framing. Companion course materials are free, and the associated MOOC is free to audit on edX.
- 📘 **MIT 6.005 / 6.031 — Software Construction, "Specifications" readings** — MIT OpenCourseWare. Complements this chapter by showing how the behavior a use case describes becomes a precise specification to design and test against. CC BY-NC-SA.

Notation and diagrams

- 📄 **uml-diagrams.org — Use Case Diagrams** — uml-diagrams.org/use-case-diagrams.html. A careful, example-rich reference for actors, associations, and especially the `«include»` and `«extend»` relationships and extension points. The clearest free walkthrough of the notation used in §5.4–§5.5.
- 📄 **OMG — Unified Modeling Language (UML) Specification** — omg.org/spec/UML. The authoritative, openly published standard. See the *Use Cases* clause for the formal definitions of actor, use case, extend, include, and extension point. Dense, but the source of truth when tools disagree.
- 🛠️ **Mermaid live editor** — mermaid.live. Paste and render the diagrams in this chapter, or draft your own use-case maps. Mermaid is MIT-licensed.

Templates and practice

- 📄 **Cockburn's use-case template (free sample pages)** — the publisher's sample-pages PDF from *Writing Effective Use Cases* shows the "fully dressed" template filled in, with the same fields used in §5.3.1.
- 📘 **Wikibooks — Systems Analysis and Design** — en.wikibooks.org/wiki/Systems_Analysis_and_Design. An openly licensed (CC BY-SA) community text with worked use-case and diagram examples for extra practice at no cost.

License note

Linked resources remain under their own licenses; this page is CC BY-SA 4.0.

Chapter 6 — Design and Architecture

> **Where we are.** Chapters 3 through 5 answered *what* to build: you elicited user requirements, analyzed them, and captured behavior as use cases. This chapter is the hinge where the project turns from problem to solution. Before anyone writes a line of production code, someone decides how the system will be carved into parts, how those parts will talk to one another, and which qualities the whole must exhibit. That set of decisions is the system's **architecture**, and living with it — well or badly — is most of what "maintaining software" means. Chapter 7 will catalog reusable architectural *patterns*; here you learn the underlying vocabulary and judgment.

A house can be built without blueprints. People do it — a room added here, a porch bolted on there — and for a while it works. Then someone wants to move a load-bearing wall, and it turns out no one knows which walls are load-bearing. Software accretes the same way, except that its "walls" are invisible, so the discovery that you cannot safely change something usually arrives as a production incident rather than a sagging roof. **Architecture is the discipline of deciding, on purpose and in advance, where the walls go** — and of writing that decision down so the next person can respect it.

6.1 The Role of Architecture

6.1.1 What Is Architecture?

Ask five engineers to define software architecture and you will get five answers, but they cluster around a single idea. The architecture of a system is the set of **structures** needed to reason about it: the parts it is made of, the externally visible properties of those parts, and the relationships among them.[1] Two phrases in that sentence carry the weight.

First, **externally visible properties.** Architecture is deliberately about the *outside* of each part — what it offers, what it requires, what guarantees it makes — and not about the private machinery inside. When you say "the payments module talks to the ledger only through a documented interface," you have made an architectural statement. When you say "the ledger sorts its entries with quicksort," you have made an implementation statement. The line between them is exactly the line between what other parts can depend on and what they must not.

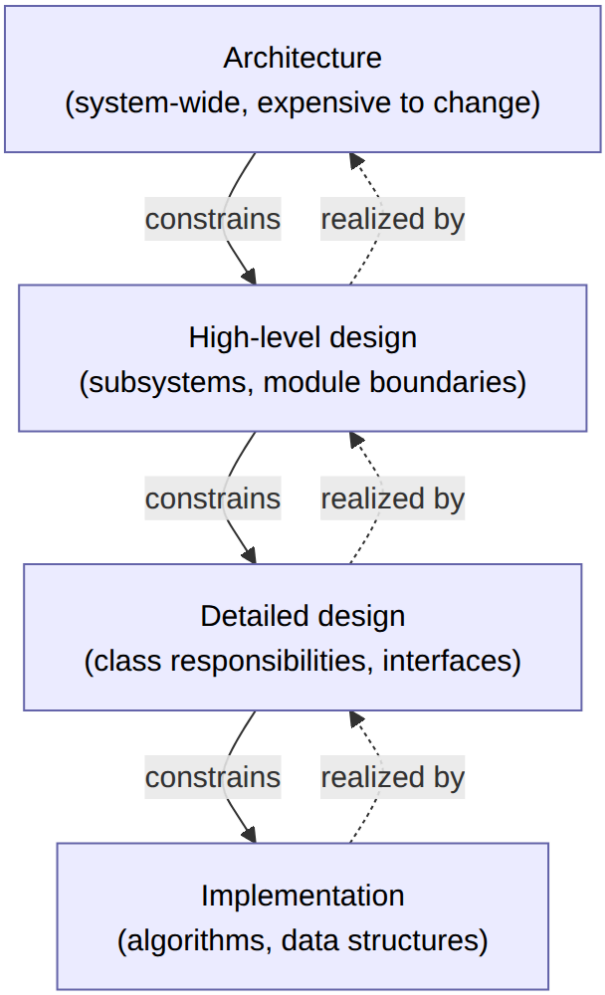
Second, **structures.** A system has more than one architecture depending on which structure you look at. The way code is grouped into files and packages is one structure. The way running processes communicate at runtime is another. The way software is deployed onto machines is a third. All three are "the architecture," and confusing them is a common source of muddled design discussions (§6.4 gives you a way to keep them straight).

> **Definition.** A system's **architecture** is the set of significant design decisions about how it is structured — its components, their responsibilities and interfaces, and the relationships and constraints among them — chosen to satisfy the system's most important quality requirements.

The word *significant* is doing real work. Not every decision is architectural. Whether a helper function takes two arguments or three is a local matter you can change on a Tuesday afternoon and no one else will notice. Whether the system stores state in a single shared database or gives each service its own store is a decision that, once made and built upon, may take a year and a rewrite to reverse. **Architectural decisions are the ones that are expensive to change** — and that is why they deserve thought before, not after.

6.1.2 Design Includes Architecture

People sometimes talk as though "design" and "architecture" name two different activities, or two different phases separated by a fence. They do not. **Design is a continuum of decisions at different scales**, and architecture is simply its coarsest, most consequential end.



At the top, you decide that the system is a web client, an API server, and a background worker sharing a database — a handful of decisions that shape everything else. In the middle, you decide how the API server divides into modules: authentication, messaging, user profiles. Lower still, you decide that the messaging module exposes a `Conversation` class with certain operations. At the bottom, you choose the data structure that stores messages in memory. Each level *constrains* the levels below it and is *realized by* them. A brilliant sorting algorithm cannot rescue an architecture that put the sort in the wrong process; a clean architecture cannot survive detailed code that ignores its boundaries.

The practical consequence is that you cannot fully separate "the architects" from "the programmers." The people writing code make micro-architectural decisions constantly, and if those decisions ignore the intended structure, the real architecture drifts away from the drawn one. Good teams keep the two aligned by writing the architecture down (§6.5) and by reviewing changes against it.

Designing at the right fidelity. If design is a continuum, one recurring mistake is working at the *wrong* level of detail for the decision at hand. Jump to pixel-perfect mockups or fully fleshed-out classes too early and you make expensive commitments before you understand the problem — and you rob the people downstream of the judgment that is their job. Stay too vague ("redesign the reports section") and there is no shared object to reason about, so scope balloons unnoticed. Basecamp's *Shape Up* names a useful target between these failures: a design should be **rough** (visibly unfinished, so it invites change), **solved** (the main elements and their connections are worked out, not hand-waved), and **bounded** (it states what is explicitly *out* of scope).[2]

> **Technique — breadboards and fat-marker sketches.** To design at that level, work in deliberately low-fidelity forms. A **breadboard** (borrowed from electronics) captures only *places* (screens, states), *affordances* (things a user can act on — a button, a field), and *connection lines* between them — the functional wiring, with no visual styling. A **fat-marker sketch** is drawn with strokes so thick that fine detail is *impossible*, which forces attention onto layout and relationships rather than polish.[3] The same instinct applies to system design: a boxes-and-arrows component sketch (§6.4) is the architectural breadboard — enough to reason about structure, not so much that you have secretly written the code in a diagram.

6.1.3 What Is a Good Software Architecture?

There is no single "correct" architecture for a problem, only architectures that serve some goals better than others. So we judge an architecture the way we judge a plan: by how well it delivers the qualities that matter for *this* system, at an acceptable cost. Concretely, a good architecture tends to have these properties.

- **It fits the important quality requirements.** If the system must handle ten thousand concurrent users, the architecture must have an answer for load and failure. If it must keep working while individual parts are redeployed, the architecture must isolate those parts. Architecture exists mainly to satisfy the *non-*

functional requirements — performance, availability, security, modifiability — because functional requirements can usually be met by many structures, but quality requirements rule most of them out.

- **It makes likely changes cheap.** No architecture makes *every* change cheap; that is impossible, because reducing the cost of one kind of change usually raises the cost of another. The trick is to predict which changes are likely and arrange the walls so those changes stay local. An architecture that isolates the parts most likely to change — the user interface, the tax rules, the payment provider — earns its keep for years.
- **It can be understood.** An architecture that no one on the team can hold in their head is, for practical purposes, no architecture at all: people will violate boundaries they cannot see. Conceptual integrity — a small number of consistent ideas applied throughout — is worth more than local cleverness.[4]
- **It is buildable and testable.** A structure that looks elegant on a diagram but forces every test to spin up the entire system is a bad architecture, because it makes the feedback loop that catches defects slow and expensive (Chapter 10).

> **Principle.** You cannot maximize all qualities at once. A good architecture records deliberate trade-offs among quality attributes: the team chooses which qualities to prioritize, understands the costs imposed on others, and avoids letting incidental code structure make those decisions.

6.1.4 Quality-Attribute Scenarios: Making “Good” Testable

“The system should be fast” rules out no architecture at all, because every architect believes their design is fast. Before a quality requirement can shape a design — or judge one — it has to become concrete. The standard tool is the **quality-attribute scenario**: a short, structured statement that turns a vague quality into an observable event with a measurable outcome.[5] A full scenario names six parts; four carry most of the weight:

- **Stimulus** — the event that arrives: a request, a failure, a spike in load, a change request, an attempted intrusion.
- **Environment** — the conditions under which it arrives: normal operation, peak load, degraded mode, mid-deployment.
- **Response** — what the system visibly does about it.
- **Response measure** — the number that says whether the response was good enough.

(The other two parts name the *source* of the stimulus and the *artifact* it strikes.) For the clinic app of Chapter 3:

> *During peak morning check-in* (environment), *a clerk submits a booking* (stimulus); *the booking is confirmed on screen* (response) *within one second for 95% of submissions* (response measure).

> *While the payment provider is unreachable* (environment), *a patient completes a visit* (stimulus); *the visit is recorded and billing is queued* (response) *with no data loss, and the queue drains within ten minutes of recovery* (response measure).

This format turns a vague quality goal into a scenario that can be reviewed and tested. Each scenario is an architecture test you can run on paper — walk the design and ask *where* each stimulus lands and *which* structures produce the response in time — and later a real test you can run in Chapter 10's sense. The response measure is exactly the **fit criterion** of §3.1.3 wearing its architectural clothes, and the same scenarios reappear as the concrete cases an architecture review walks through (§9.1.1). If a quality requirement matters enough to influence the structure of your system, it matters enough to get a scenario; a quality with no scenario is an opinion, and opinions do not rule out designs.

6.2 Designing Modular Systems

The single most powerful idea in software design is **modularity**: build the system out of parts that can each be understood, built, tested, and changed largely on their own. The whole of §6.2 unpacks what makes a decomposition into modules good rather than merely present. (Every non-trivial program *has* modules; the question is whether they are the *right* ones.)

6.2.1 The Modularity Principle

A **module** is a unit of software with a boundary: a set of things it provides to the rest of the system and a set of things it keeps to itself. The value of a module comes almost entirely from what it *hides*.

Information hiding is the practice of designing each module around a decision — often a design decision likely to change — and hiding that decision behind an interface, so that the rest of the system depends on the *interface* and not on the decision.[6] The classic example: a module that stores user records should expose operations like `find(id)` and `save(record)` while hiding whether the records live in a SQL database, a file, or a remote service. Because callers never learned how records are stored, you can switch from files to a database without touching a single caller. The hidden decision was free to change because it was, in fact, hidden.

Here is that module, its secret — how the records are laid out on disk — behind the interface. (Each listing names its exact secret in a comment.)

```
import json, os

class UserStore:
  def __init__(self, path):
    self._path = path                    # the secret: a JSON file

  def find(self, id):
    return self._load().get(id)

  def save(self, record):
    users = self._load()
    users[record["id"]] = record
    with open(self._path, "w") as f:
      json.dump(users, f)

  def _load(self):
    if not os.path.exists(self._path):
      return {}
    with open(self._path) as f:
      return json.load(f)
```

When disk storage becomes the bottleneck, the secret changes and the promise does not.

```
class UserStore:
  def __init__(self):
    self._records = {}                   # the new secret: an in-memory table

  def find(self, id):
    return self._records.get(id)

  def save(self, record):
    self._records[record["id"]] = record
```

A caller that relied only on the promise runs unchanged against either version.

```
store.save({"id": "u7", "name": "Kati"})
assert store.find("u7")["name"] == "Kati"
```

Encapsulation is the mechanism that enforces information hiding: the language keeps the private parts of a module unreachable from outside, so that clients *cannot* accidentally depend on internals even if they wanted to. A field marked `private`, a function not exported from its file, a class whose only public surface is a small set of methods — these are encapsulation. Information hiding is the *intent* ("callers should not know how records are stored"); encapsulation is the *enforcement* ("and the compiler will stop them").

The reason this matters is not tidiness. **A module's interface is a promise, and its implementation is a secret.** As long as you keep the promise, you may change the secret freely, and no one who relied only on the promise can break. The size of the promise you make — the "surface area" of the interface — is therefore the size of the commitment you can never quietly walk back. Small, deliberate interfaces are small, deliberate commitments. This is why experienced designers fight to keep interfaces narrow: every public method is a hostage to fortune.

> **Principle.** Design each module around a secret. Ask "what decision does this module hide, such that if that decision changed, only this module would change?" A module that hides nothing is just a place where code happens to sit.

6.2.2 Coupling and Cohesion

Two measures, introduced together decades ago, tell you whether a decomposition is good: **coupling** between modules and **cohesion** within each.[7]

Coupling is the strength of the dependency between two modules — how much one must know about the other, and how badly a change to one ripples into the other. You want it **low**. It also comes in kinds, and the kinds form a ladder from worst to best.[8] Knowing the ladder lets you diagnose *why* a dependency hurts, not just that it does.

- **Content coupling (worst).** One module reaches inside another and manipulates its internals directly — modifying its private data, or jumping into the middle of its code. Any change to the target's internals can break the intruder without warning. This is the coupling encapsulation exists to prevent.
- **Common coupling.** Modules share global mutable state — a global variable, a shared singleton. Now any module can change data any other module reads, so behavior depends on execution order and no module can be understood alone. Bugs here are notoriously non-local.
- **Control coupling.** One module passes a flag that tells another *what to do* — a control signal — so the caller must understand the callee's internal logic and branches. A function called `process(data, mode)` where `mode` selects between unrelated behaviors couples caller to callee's internal structure.
- **Stamp coupling.** A module passes a whole compound record when the callee needs only a field or two. The callee is now needlessly coupled to the shape of the whole record, and changing that record's structure can ripple into modules that never used the changed part.
- **Data coupling (best).** Modules communicate only through simple parameters — the exact data the callee needs, and nothing more. This is the loosest, most honest form of connection, and the one to aim for.

Cohesion is how well the elements *inside* a single module belong together — how focused the module is on one job. You want it **high**. It, too, comes in kinds, from worst to best:[8]

- **Coincidental (worst):** the parts are grouped for no real reason — a "utils" grab-bag.
- **Logical:** parts are grouped because they are the same *category* of thing (all the input routines) though they do unrelated jobs.
- **Temporal:** parts are grouped because they run at the same *time* (everything done "at startup"), not because they relate.
- **Procedural / communicational:** parts share a flow of control, or operate on the same data.
- **Functional (best):** every part contributes to one well-defined task, and the module does that task completely and does nothing else.

Coupling and cohesion pull in the same direction, which is why they are always taught together: **when you raise cohesion, coupling tends to fall, and vice versa.**[8] If each module does exactly one thing (high cohesion), related logic sits together and does not need to reach across boundaries (low coupling). If a module does five unrelated things (low cohesion), pieces of those five jobs almost inevitably entangle with five other modules (high coupling). Chasing one gets you the other.

> **Pitfall.** A "utility" or "helpers" module is usually a cohesion failure wearing a friendly name. Because it has no single responsibility, everything ends up depending on it, and it ends up depending on everything — the worst of both measures. When you notice a module you cannot describe in one sentence without saying "and," split it.

6.2.3 Design Guidelines for Modules

The two measures above become actionable through a handful of guidelines. Treat them as heuristics rather than laws: each pays off far more often than not.

1. **Give each module one responsibility.** You should be able to state a module's job in a single sentence with no "and." This is the *single responsibility* idea, and it is really "high cohesion" restated as a design rule.
2. **Make interfaces small and explicit.** Expose the fewest operations that let clients do their job. Every additional public operation is another promise you must keep forever (§6.2.1).
3. **Depend on interfaces, not implementations.** When module A needs a service, let it depend on an abstract description of that service, so that any conforming implementation can be substituted. This keeps coupling at the data/interface level and makes testing (with fakes) and change (with new providers) cheap.
4. **Push details down and out.** Keep policy — the important, stable decisions — in the core, and push volatile details (the specific database, the specific UI toolkit) to the edges where they can be swapped. Stable things should not depend on volatile things.

5. **Avoid cyclic dependencies.** If A depends on B and B depends on A, they are not two modules; they are one module with a line drawn through it, and you can no longer build, test, or reason about either alone. Break the cycle by introducing an interface one side owns.
6. **Design for the changes you expect.** You cannot isolate everything, so spend your modularity budget on the decisions most likely to change: external providers, business rules, presentation. Do not pay for flexibility you have no reason to expect (that is over-engineering, its own kind of accidental complexity).

Several of these guidelines circulate in industry under memorable names, and you should recognize them when colleagues use them. The best-known bundle is **SOLID** — five object-oriented design principles Robert C. Martin catalogued,[9] later named for their initials:

- **S — Single responsibility:** guideline 1 above — one stateable job per module.
- **O — Open-closed:** a module should be **open for extension, closed for modification** — you add behavior by adding new code (a new implementation of an interface, a new subclass), not by editing code that already works and that other modules already depend on.[10]
- **L — Liskov substitution:** a subtype must be usable anywhere its supertype is expected, with no surprises.[11] If code that works on every `Conversation` breaks when handed a `GroupConversation`, the generalization arrow (§6.3.2) is a lie.
- **I — Interface segregation:** many small, client-specific interfaces beat one fat one — guideline 2 taken seriously. A client forced to depend on operations it never calls is coupled to changes it never needed to see.
- **D — Dependency inversion:** guidelines 3 and 4 above — depend on abstractions, not implementations, and never let stable policy depend on volatile detail.

A companion rule with its own name is **DRY — Don't Repeat Yourself**: every piece of knowledge in the system should have a single, authoritative representation.[12] With duplicated logic, the wasted keystrokes are the least of it: a future change must now be found and fixed in two places, and eventually someone will fix only one.

> **Comments deserve design too.** Write comments *with* the code, not afterward — a comment added later is a guess. Never restate what a line obviously does (`i += 1 # add one to i` helps no one). Use comments to raise the level of abstraction above the code: the *why* behind a non-obvious choice, the constraint being honored, the units of a value. And prune them ruthlessly during maintenance, because a wrong comment is worse than none — comments rot faster than the code they describe.

6.3 Class Diagrams

To design and discuss modular structure, you need a notation. The **class diagram** from the Unified Modeling Language (UML) is the lingua franca for describing the static structure of object-oriented systems: the classes,

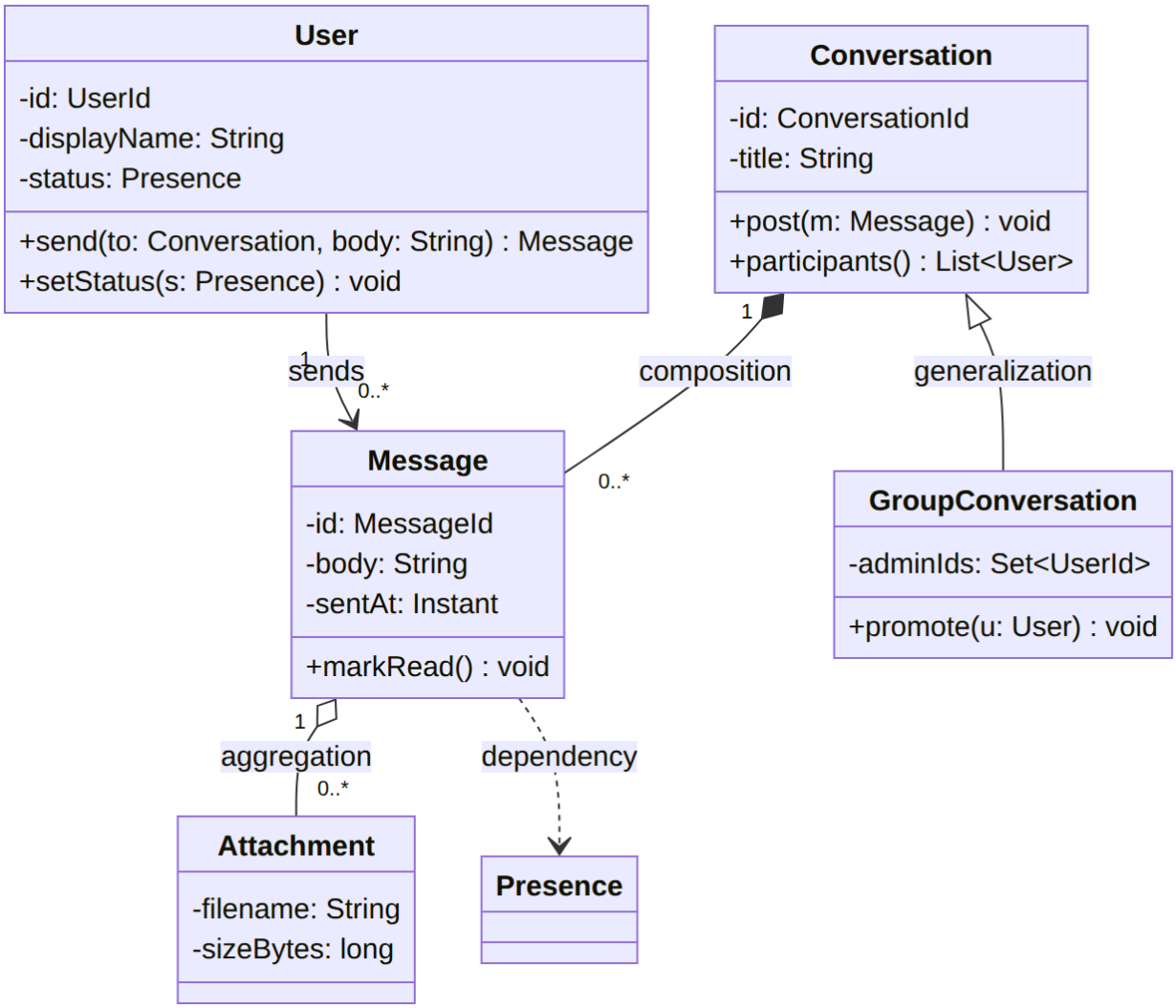
what each holds and does, and how they relate.[13] You will not draw every class; you draw the ones whose relationships someone needs to understand.

6.3.1 Representing a Class

A class is drawn as a box with up to three compartments: the **name** on top, the **attributes** (data each object holds) in the middle, and the **operations** (what each object can do) at the bottom. A **visibility** marker precedes each member:

- `+` **public** — visible to everyone (part of the interface, the promise),
- `-` **private** — visible only inside the class (the secret),
- `#` **protected** — visible to the class and its subclasses,
- `~` **package** — visible within the same package.

The convention is `visibility name : type` for attributes and `visibility name(parameters) : returnType` for operations. Here is a small messaging model you can read at a glance.



Read the boxes first: a `User` hides its `id` and `status` (private, the `-`) but promises a public `send` operation (the `+`). The lines between class boxes show the relationships that give the diagram its structure; the next section examines those relationships.

6.3.2 Relationships between Classes

The lines in a class diagram carry precise meanings — each *shape* tells you how two classes are connected and how tightly.[13] Getting them right is how a diagram communicates coupling.

- **Association** (a plain solid line) says two classes are connected: objects of one refer to objects of the other. "A `User` *sends* `Message`s" is an association. It is the most general relationship; the others are associations with extra meaning.
- **Aggregation** (a solid line with a *hollow* diamond at the "whole" end) is a "has-a / part-of" relationship where the part can outlive the whole and can be shared. A `Message` aggregates `Attachment`s: the same

attachment could be referenced elsewhere, and deleting the message need not destroy the file. Aggregation is a weak, informal grouping.

- **Composition** (a solid line with a *filled* diamond at the "whole" end) is a stronger "part-of" where the part's lifetime is bound to the whole and the part is not shared. A `Conversation` is *composed of* its `Message`s: destroy the conversation and its messages go with it; a message belongs to exactly one conversation. Composition encodes ownership.
- **Generalization** (a solid line with a *hollow triangle* pointing at the parent) is inheritance: "a `GroupConversation` *is a* `Conversation`." The subclass inherits the parent's interface and can be used wherever the parent is expected.
- **Dependency** (a *dashed* arrow) is the weakest link: one class *uses* another transiently — as a parameter type, a return type, or a local — without holding a long-lived reference. `Message` depends on `Presence` if one of its methods takes a `Presence` argument. Dependencies are the couplings you most want to keep few and one-directional.

Multiplicity annotates the ends of a line with how many objects participate: `1` (exactly one), `0..*` or `*` (zero or more), `1..*` (one or more), `0..1` (optional). In the diagram above, `Conversation "1" *-- "0..*" Message` reads "one conversation is composed of zero or more messages, and each message belongs to exactly one conversation." Multiplicities matter: `1` versus `0..1` versus `0..*` is often the difference between a null-pointer bug, a missing foreign key, and a correct schema.

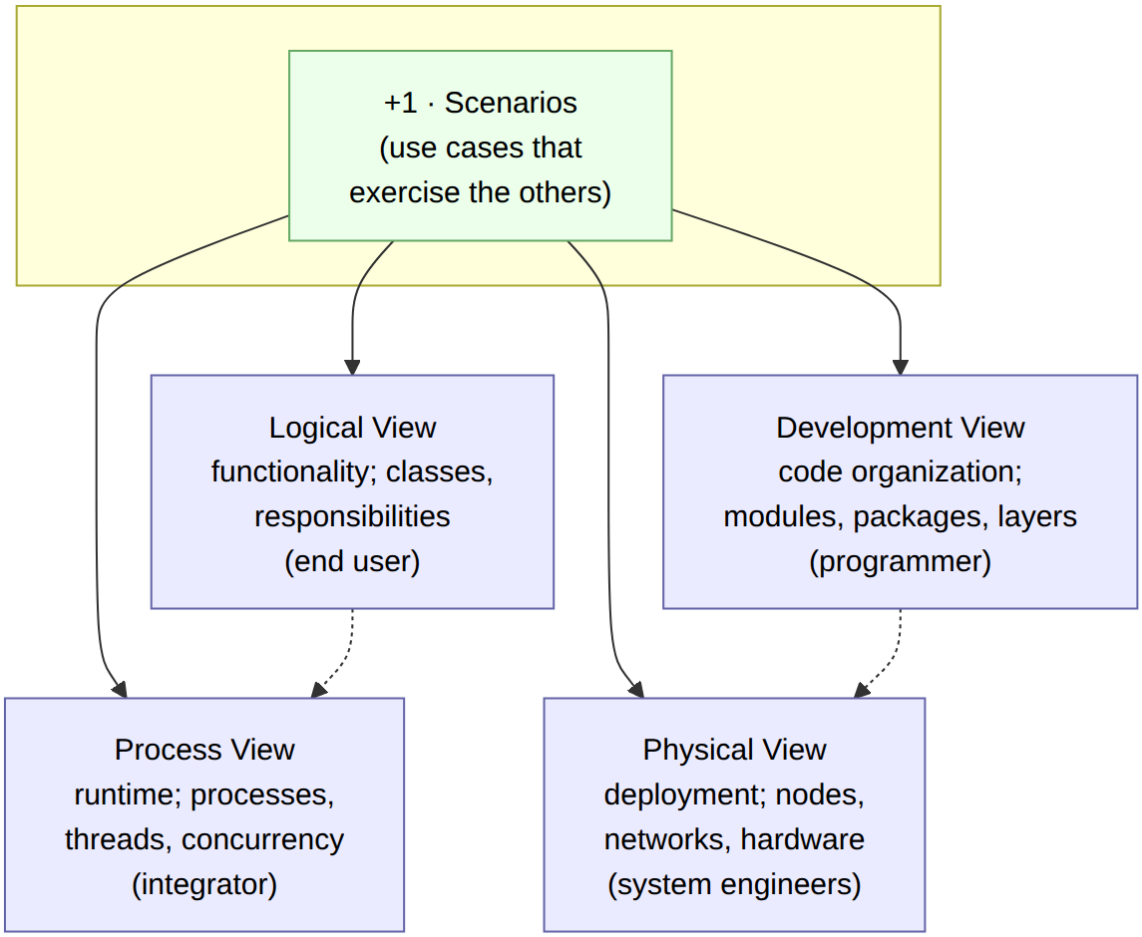
> **Pitfall.** Do not spend excessive time choosing between aggregation and composition — even the UML specification leaves aggregation's precise semantics open.[13] Start with the *lifetime and ownership* questions: when the whole is deleted, does the part also disappear, and can multiple wholes share the same part? Choose the diamond after answering those questions.

6.4 Architectural Views

A single diagram cannot capture a whole system any more than a single photograph captures a building. Stakeholders ask different questions — "what are the responsibilities?", "how does it perform under load?", "how do I check out and build the code?", "which server runs what?" — and each question is best answered by a different picture. A **view** is a representation of the system from one such perspective.[14]

6.4.1 The 4+1 Grouping of Views

A widely used way to organize views, introduced by Philippe Kruchten, groups them into four plus one.[15] The four cover distinct concerns; the "+1" ties them together.



- **Logical view** — the *functionality* the system offers to end users, expressed as the key abstractions: classes, their responsibilities, their relationships. Class diagrams (§6.3) live here. It answers "what are the concepts and what can they do?"
- **Process view** — the system *at runtime*: the processes and threads, how they communicate, and how concurrency, performance, and availability are handled. It answers "what is executing, and how do the running pieces coordinate?" This is where you reason about deadlocks, throughput, and failover.
- **Development view** — the *code as organized for building*: the modules, packages, libraries, and layers, and their dependency structure. It is the programmer's map (§6.5.3) and answers "where does this code live and what may depend on what?"
- **Physical view** (also called the *deployment* view) — the *software mapped onto hardware*: which components run on which nodes, and how those nodes are networked. It answers "what do we actually deploy, and where?"
- **+1: Scenarios** — a handful of important **use cases** (Chapter 5) that thread through all four views. Scenarios are the glue: walking a use case through the logical, process, development, and physical views is how you check that the four are *consistent* with one another and that, together, they actually deliver the behavior. They also make the views discoverable — a reader follows a familiar story and sees each view play its part.

The power of the 4+1 grouping is that it *separates concerns among readers*. An operations engineer reads the physical view and ignores the logical one; a new programmer reads the development view. Nobody has to understand everything at once, which is the same complexity argument that motivated modularity — applied now to the documentation.

6.4.2 Structures and Views

It helps to distinguish a **structure** from a **view**. A structure is something real in the system — the actual set of modules and their dependencies, the actual set of runtime processes. A view is a *representation* of a structure, drawn for an audience. The three big families of structure are worth naming because almost every architectural confusion is a failure to keep them apart:[1]

- **Module structures** — how the system is divided into units of *code*: modules, packages, classes, layers, and the "is-part-of," "depends-on," and "is-a" relations among them. (Development and logical views show these.)
- **Component-and-connector structures** — how the system is divided into units that have *runtime* presence: processes, services, clients, servers, and the connectors — channels, calls, message queues — through which they interact. (The process view shows these.)
- **Allocation structures** — how software elements *map onto* their environment: onto hardware nodes, onto teams that own them, onto files in a repository. (The physical view shows these.)

The reason to keep these straight is that a single box on one diagram can be several things on another. A "messaging" module in the development view might, at runtime, be split across three processes in the process view and deployed on two machines in the physical view. If you try to force all of that into one diagram, you get a picture that is wrong from every angle. **Draw one structure per view, name the view, and say who it is for.**

6.5 Describing System Architecture

On a team, an architecture decision is worthless if it lives only in the head of the person who made it. Architecture must be *described* — written down in a form the team can read, question, and hold changes accountable to. This section gives you a practical outline and a worked example.

6.5.1 Outline for an Architecture Description

An architecture description need not be a book. For most systems, a living document of a few pages, kept next to the code, does the job. A serviceable outline:

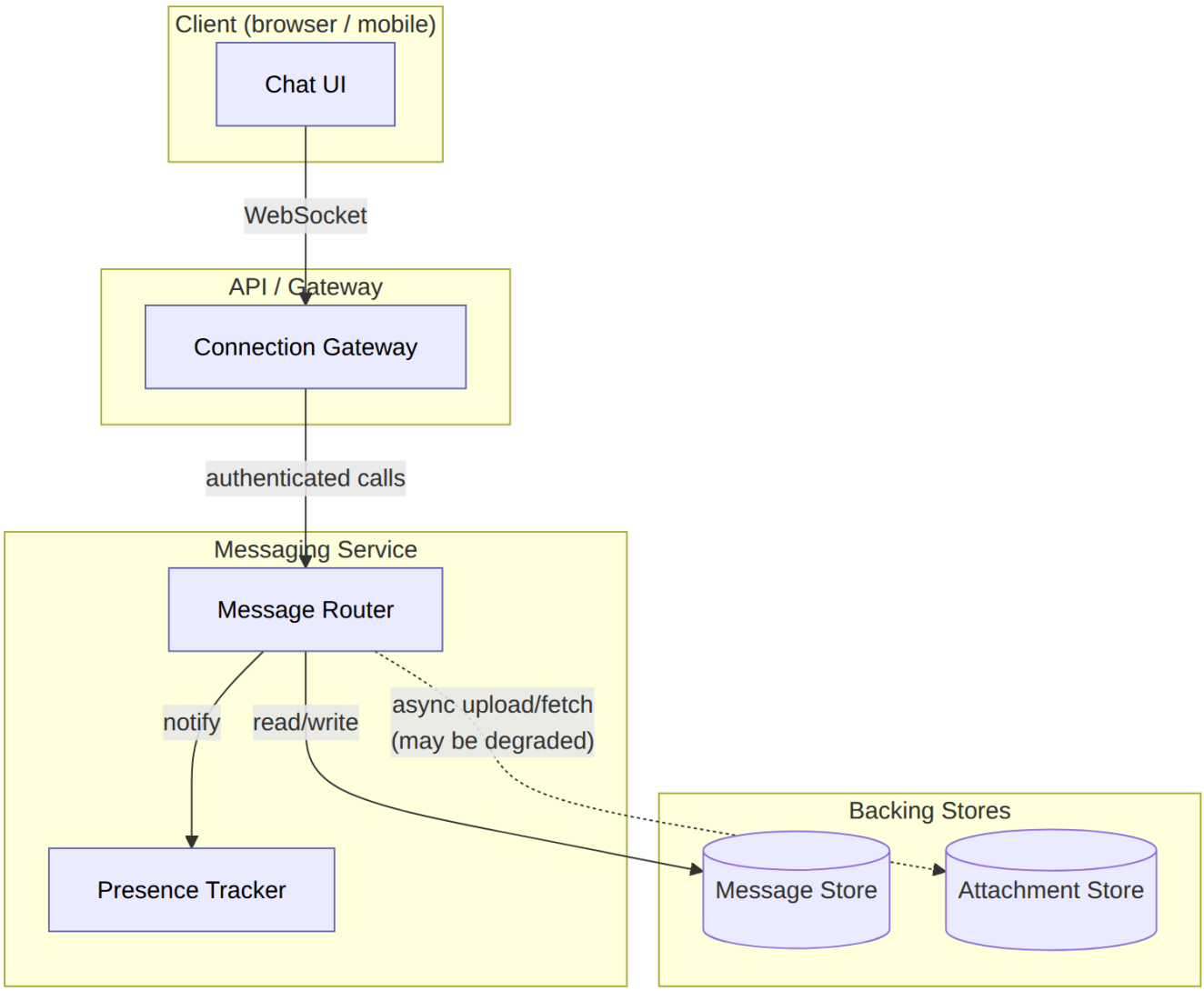
1. **Context and goals.** What the system is, who its stakeholders are, and — crucially — the *quality requirements* that drive the architecture (the target load, the availability goal, the security posture, the changes you expect). Everything after this section exists to satisfy this section.

2. **Constraints and decisions.** The fixed constraints (must run on the customer's cloud; must integrate with an existing identity provider) and the *significant decisions* you made, each with a one-line rationale. Recording *why* is what lets a future engineer safely revisit the *what*.
3. **The views.** One section per view you chose to document (§6.4): typically a logical view (a class or component diagram), a development view (the module hierarchy), a process view if concurrency matters, and a physical view if deployment is non-trivial.
4. **Key scenarios.** Two or three use cases walked through the views, showing how the parts cooperate to deliver behavior — the "+1" made concrete.
5. **Risks and open questions.** Honesty about what you are unsure of. An architecture document that admits its risks is far more useful than one that pretends to certainty.

6.5.2 System Overview of a Communications App

Make it concrete. Suppose you are building **a communications app** — one-to-one and group messaging, with presence ("online / away") and file attachments. Its important quality requirements are: messages should be delivered within a second under normal load; the system should stay usable if the attachment storage is temporarily unavailable; and the team expects to change the *transport* (today WebSockets, perhaps something else later) and to add new *message types* often.

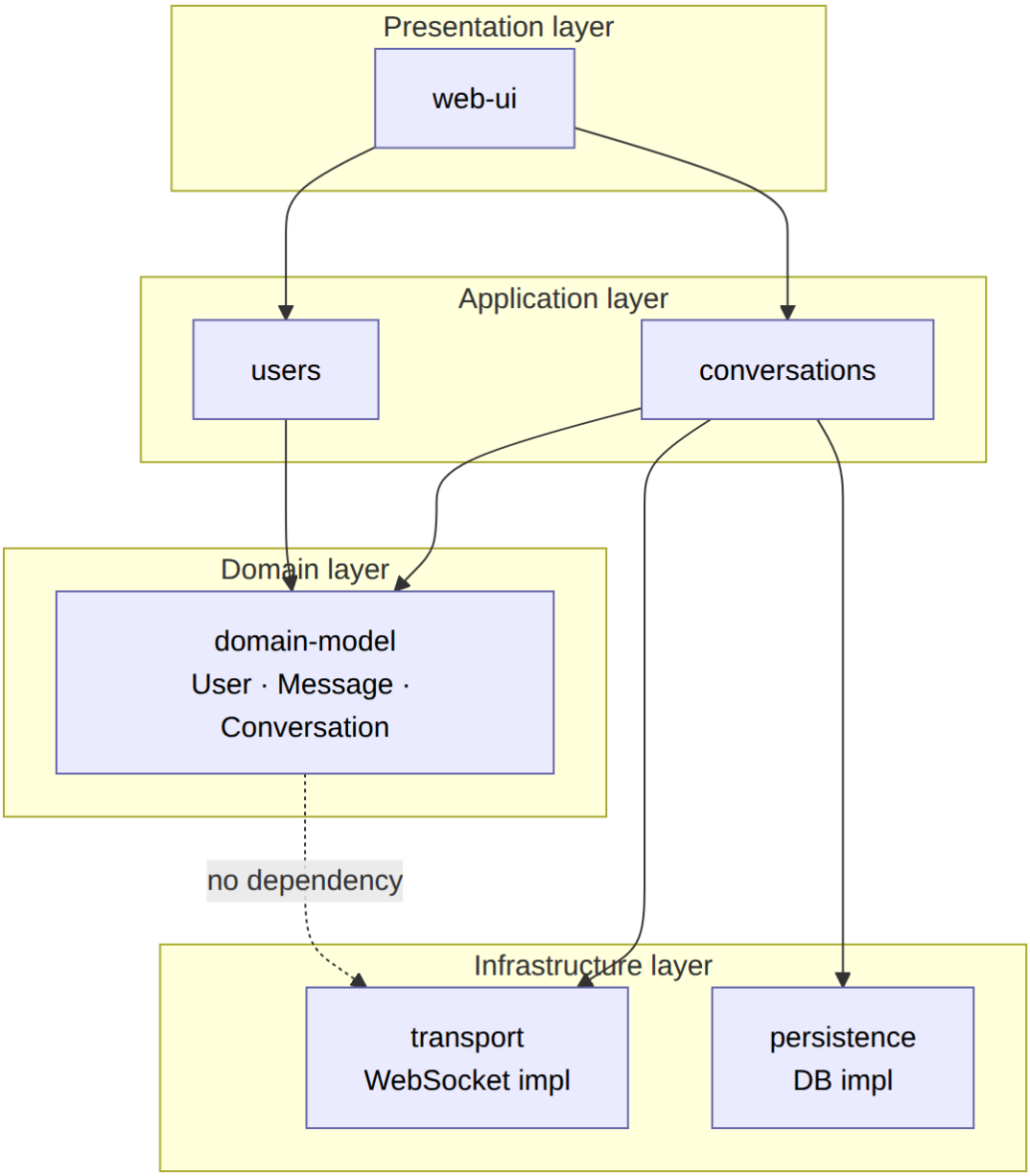
Those quality requirements dictate the shape. The need to swap transports says: hide the transport behind an interface. The need to survive attachment-storage outages says: make attachments a separate component the messaging core does not synchronously depend on. Here is a component-and-connector (process) view for that overview.



Notice how the diagram *encodes the quality requirements*. The transport (WebSocket) is confined to the edge between the UI and the Connection Gateway, so changing it touches two boxes, not the whole system. The Attachment Store is reached by a *dashed, asynchronous* connector, signaling that the Message Router does not block on it — if attachments are down, text messages still flow, satisfying the "stay usable" requirement. The overview remains readable by displaying the decisions readers need and omitting lower-level detail.

6.5.3 A Development View: Module Hierarchies

The process view above shows runtime pieces. The **development view** shows how the *code* is organized and — most importantly — which modules are *allowed* to depend on which. The usual discipline is a **layered** module hierarchy: higher layers may depend on lower ones, never the reverse, and volatile details sit at the edges.



Two things stand out in this hierarchy. First, **the domain model depends on nothing below it.** The `domain-model` module — the classes from §6.3 — knows nothing about databases or WebSockets. That is deliberate: the domain is the most stable, most valuable part of the system, so it must not be dragged along when a volatile detail changes. The dashed "no dependency" line is a design *rule*, not an observation, and a good build setup enforces it (Chapter 9's static checks can fail the build if `domain-model` ever imports `transport`).

Second, **the interface belongs to the layer that needs it, and the implementation lives below.** The application layer *owns* the interface each of these requires — what a store or a transport must *do* — while the `persistence` and `transport` modules in infrastructure hide a concrete implementation behind it. This is dependency inversion in the concrete: the important, stable code (application and domain) depends on abstractions, and the volatile code (the actual database driver, the actual socket library) depends on those same abstractions from below. Swapping

the database becomes a change confined to one module — exactly the “make likely changes cheap” property from §6.1.3, realized in the code structure rather than merely wished for on a slide.

In code, the inversion is small: the application layer owns the definition of what a transport must do — deliver a body to a recipient — and the real transport and a test fake both conform to it from below.

```
from typing import Protocol

class Transport(Protocol):                  # owned by the application layer
  def deliver(self, to: str, body: str) -> None: ...

class MessageRouter:                        # application code sees only the interface
  def __init__(self, transport: Transport):
    self._transport = transport

  def route(self, message: dict) -> None:
    self._transport.deliver(message["to"], message["body"])

class WebSocketTransport:                   # infrastructure, conforming from below
  def __init__(self, socket):
    self._socket = socket

  def deliver(self, to: str, body: str) -> None:
    self._socket.send(f"{to}:{body}".encode())

class FakeTransport(list):                  # a two-line test double
  def deliver(self, to, body): self.append((to, body))

fake = FakeTransport()
MessageRouter(fake).route({"to": "eloise", "body": "you are on call"})
assert fake == [("eloise", "you are on call")]
```

The `MessageRouter` can now be exercised in a test without opening a socket — the same substitution that lets the team change transports later.

6.6 Conclusion

Architecture is what you get when you make the expensive decisions on purpose. Its core is a single, endlessly applied idea — **decompose the system into modules with high cohesion and low coupling, each hiding a decision behind a small, explicit interface** — and everything else in this chapter is that idea seen from a different

angle. Coupling and cohesion give you a vocabulary to *diagnose* a decomposition; information hiding tells you *how to choose* the boundaries; class diagrams and multiplicities let you *record* the static structure; the 4+1 views let you *describe* the system to different audiences without drowning any of them; and an architecture document makes the whole thing *durable* — a decision the team can respect instead of a story only one person remembers.

Carry two ideas into the rest of the book. First, architecture serves *quality requirements*: you choose a structure to buy performance, availability, security, or — above all — cheap change, and you pay for it consciously in the qualities you did not choose. Second, architecture is only real to the extent that the code obeys it; a drawn boundary that the build does not enforce and the reviews do not defend will erode. Chapter 7 takes the next step, showing that you rarely invent these structures from nothing: recurring problems have well-understood **architectural patterns** — layered, client-server, pipe-and-filter, publish-subscribe, and more — that package hard-won decisions so you can reuse the judgment, not just the diagram.

Sources

1 Len Bass, Paul Clements, and Rick Kazman, *Software Architecture in Practice*, 4th ed. (2021). sei.cmu.edu.

2 Ryan Singer, *Shape Up: Stop Running in Circles and Ship Work that Matters*, ch. “Principles of Shaping” (Basecamp, 2019). basecamp.com.

3 Ryan Singer, *Shape Up: Stop Running in Circles and Ship Work that Matters*, ch. “Find the Elements” (Basecamp, 2019). basecamp.com.

4 Frederick P. Brooks, Jr., *The Mythical Man-Month: Essays on Software Engineering* (1975; anniversary ed. 1995). informit.com.

5 Len Bass, Paul Clements, and Rick Kazman, *Software Architecture in Practice*, 4th ed. (Addison-Wesley, 2021), ch. 3 — the six-part quality-attribute scenario (source, stimulus, artifact, environment, response, response measure). informit.com.

6 David L. Parnas, “On the Criteria To Be Used in Decomposing Systems into Modules,” *Communications of the ACM* 15(12) (1972). dl.acm.org.

7 W. P. Stevens, G. J. Myers, and L. L. Constantine, “Structured Design,” *IBM Systems Journal* 13(2) (1974). dl.acm.org.

8 Edward Yourdon and Larry L. Constantine, *Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design* (1979). archive.org.

9 Robert C. Martin, “Design Principles and Design Patterns” (2000). objectmentor.com via web.archive.org.

10 Bertrand Meyer, *Object-Oriented Software Construction* (1988; 2nd ed. 1997). bertrandmeyer.com.

11 Barbara H. Liskov and Jeannette M. Wing, “A Behavioral Notion of Subtyping,” *ACM Transactions on Programming Languages and Systems* 16(6) (1994). dl.acm.org.

12 Andrew Hunt and David Thomas, *The Pragmatic Programmer* (1999; 20th-anniversary ed. 2019). pragprog.com.

13 Object Management Group, *OMG Unified Modeling Language (OMG UML)*, version 2.5.1 (2017). omg.org.

14 ISO/IEC/IEEE, *ISO/IEC/IEEE 42010:2022 — Software, systems and enterprise — Architecture description* (2022). iso.org.

[15] Philippe Kruchten, “Architectural Blueprints — The 4+1 View Model of Software Architecture,” *IEEE Software* 12(6) (1995). cs.ubc.ca.

- **Key takeaways** are summarized above in §6.6.
- Continue to the Exercises.
- Go deeper with the Open Resources for this chapter.

Chapter 6 — Exercises

Exercises are graded by depth: **[warm-up]** checks understanding, **[analysis]** asks you to reason about a design. Where an exercise asks for a diagram, a hand sketch or a Mermaid block is fine — the thinking is what is graded.

Concepts

1. **[warm-up]** In your own words, explain the difference between *information hiding* and *encapsulation*, and give one concrete example where a language provides encapsulation but a design still fails at information hiding (that is, the interface leaks a secret).

2. **[warm-up]** For each pair, say which relationship it is — association, aggregation, composition, generalization, or dependency — and justify the choice in one sentence: (a) an `Order` and the `LineItem`s that make it up and die with it; (b) a `Playlist` and the `Song`s it references, where songs exist independently; (c) a `SavingsAccount` and an `Account`; (d) a `ReportGenerator` whose `render()` method takes a `Theme` argument.

3. **[warm-up]** Name the four "4+1" views plus the "+1," and for each write the one question it answers and the one stakeholder who cares most about it.

4. **[warm-up]** Rank these couplings from worst to best and give a one-line reason for the worst and the best: *stamp*, *content*, *data*, *common*, *control*.

Analysis

5. **[analysis]** You are handed a module named `Utilities` containing: `sendEmail`, `formatCurrency`, `retryWithBackoff`, `parseCsvFile`, and `MAX_UPLOAD_SIZE`. Diagnose its cohesion (name the type), explain the specific maintenance problems this module will cause, and propose a re-decomposition into cohesive modules. State the responsibility of each new module in a single "and"-free sentence.

6. **[analysis]** Read the following code and critique its coupling and cohesion.

```
LAST_TOTAL = 0.0                          # global: written by PriceEngine, read by Receipt

class PriceEngine:
  def compute(self, cart, is_b2b):
    global LAST_TOTAL
    cart._items.sort(key=lambda i: i["qty"])
    total = 0.0
    for item in cart._items:
      if is_b2b:
        total += item["wholesale"] * item["qty"] * 0.9
      else:
        total += item["retail"] * item["qty"]
    LAST_TOTAL = total

class Receipt:
  def render(self):
    return f"Total: ${LAST_TOTAL:.2f}"
```

Identify each coupling type present (there are at least three), explain the concrete risk of each, and rewrite the design in a paragraph so that the modules communicate through data coupling and depend on interfaces.

7. **[analysis]** Draw a UML **class diagram** for a small library-loan system with these facts: a `Member` may have zero or more `Loan`s; each `Loan` is for exactly one `BookCopy` and belongs to exactly one `Member`; a `BookCopy` is one physical copy of a `Book`, and a `Book` may have one or more `BookCopy`s that are destroyed if the `Book` record is removed; a `ReferenceBook` is a kind of `Book` that cannot be loaned. Show attributes with visibility, at least one operation per class, correct multiplicities, and use composition, association, and generalization at least once each. Then write two sentences explaining which relationships you chose composition versus aggregation for and why.

8. **[analysis]** Take the communications-app *development view* from §6.5.3. Suppose a new requirement arrives: messages must now be end-to-end encrypted. In which layer(s) and module(s) does this change belong, and why? Identify one *wrong* place to put it that would violate the “domain depends on nothing below it” rule, and explain the harm.

9. **[analysis]** A teammate argues that because their service “has a clean class diagram,” the architecture is documented. Using the structure-versus-view distinction from §6.4.2, explain what the class diagram does *not* tell an operations engineer or a new programmer, and name the two additional views you would ask them to add before you would call the architecture “described.”

10. **[analysis]** Cyclic dependencies are called out in §6.2.3 as something to avoid. Given modules `Auth`, `User`, and `Notification` where `Auth` needs to notify on login, `Notification` needs the user’s preferences from

`User`, and `User` needs `Auth` to check permissions — draw the dependency cycle, then show how to break it by introducing one interface owned by the right module. Explain why your choice of interface owner matters.

Chapter 6 — Open Resources

Free, open-licensed, or freely accessible materials that reinforce this chapter. Types: 📘 open text · 🎓 course · 📄 primary source · 🎥 video. Licenses vary and are noted where known; when a resource is free to read but not openly relicensable, it is marked *free to read*.

Modularity, abstraction, and interfaces

- 🎓 **MIT 6.031 — Software Construction (specs, ADTs, abstraction functions)** — web.mit.edu/6.031. The readings on *specifications*, *abstract data types*, and *abstraction functions & rep invariants* are the best free treatment of information hiding and interface design anywhere. *License:* course materials under **CC BY-SA** (see the site footer).
- 📘 **Software Engineering at Google** — abseil.io/resources/swe-book. Especially the chapters on *dependencies*, *large-scale change*, and *deprecation*, which show coupling, modularity, and "design for change" at industrial scale. *License:* free to read online; individual figures **CC BY 4.0** (see the book's front matter).
- 📄 **D. L. Parnas, "On the Criteria To Be Used in Decomposing Systems into Modules"** (1972) — the paper that introduced information hiding. Official abstract page: dl.acm.org/doi/10.1145/361598.361623; also widely available as a PDF via university course pages. *License:* ACM copyright, *free to read* via many .edu mirrors.
- 📄 **F. P. Brooks, Jr., *The Mythical Man-Month: Essays on Software Engineering*** (1975; anniversary ed. 1995) — the source of *conceptual integrity* (§6.1.3). Publisher page: informit.com. *License:* commercial book; catalog page *free to read*.

Coupling, cohesion, and design principles

- 📘 **Refactoring.Guru — Design Principles & SOLID** — refactoring.guru/design-patterns and refactoring.guru/refactoring/smells/couplers. Clear, illustrated explanations of coupling smells and the principles behind low coupling / high cohesion. *License:* free to read; content is the site's own (not openly relicensable).
- 📘 **MIT 6.005/6.031 "Reading: Designing specifications / Abstraction"** — web.mit.edu/6.031 (see the *Specifications* and *Abstraction Functions & Rep Invariants* readings). Covers cohesion and coupling implicitly through spec strength and representation independence. *License:* **CC BY-SA**.
- 📄 **W. P. Stevens, G. J. Myers, and L. L. Constantine, "Structured Design"**, *IBM Systems Journal* 13(2), 1974 — the paper that introduced coupling and cohesion (§6.2.2). Official page: dl.acm.org/doi/10.1147/sj.132.0115. *License:* IBM and ACM copyright, *free to read* via .edu mirrors.

- 📄 **E. Yourdon and L. L. Constantine, *Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design*** (1979) — the book-length treatment of the coupling and cohesion ladders used in §6.2.2. Borrowable scan: archive.org. *License:* commercial book; *free to borrow* via the Internet Archive.
- 📄 **R. C. Martin, "Design Principles and Design Patterns"** (2000) — the paper that collected the five principles later known as SOLID (§6.2.3). Archived PDF via the Wayback Machine. *License:* © Robert C. Martin, *free to read*.
- 📄 **B. Meyer, *Object-Oriented Software Construction*** (1988; 2nd ed. 1997) — origin of the open-closed principle's "open for extension, closed for modification." Author's page: bertrandmeyer.com/OOSC2. *License:* commercial book; the author's page is *free to read*.
- 📄 **B. H. Liskov and J. M. Wing, "A Behavioral Notion of Subtyping"**, *ACM TOPLAS* 16(6), 1994 — the formal statement behind the Liskov substitution principle. Official page: dl.acm.org/doi/10.1145/197320.197383. *License:* ACM copyright, *free to read* via .edu mirrors.
- 📘 **A. Hunt and D. Thomas, *The Pragmatic Programmer*** (1999; 20th-anniversary ed. 2019) — origin of DRY, "every piece of knowledge must have a single, unambiguous, authoritative representation." Publisher page: pragprog.com. *License:* commercial book; extracts on the publisher page are *free to read*.

Class diagrams and UML notation

- 📘 **UML-diagrams.org — Class Diagrams** — uml-diagrams.org/class-diagrams-overview.html. A precise, example-rich reference for association, aggregation, composition, generalization, dependency, and multiplicity notation. *License:* free to read (site's own content).
- 📄 **OMG, *OMG Unified Modeling Language (OMG UML)*, version 2.5.1** (2017) — the normative specification for the class-diagram notation in §6.3 (visibility markers, association, aggregation, composition, generalization, dependency, multiplicity). omg.org/spec/UML/2.5.1. *License:* specification is *free to download* from the OMG.
- 📘 **Mermaid — Class Diagram syntax** — mermaid.js.org/syntax/classDiagram.html. The notation used for the diagrams in this chapter, so you can reproduce and extend them. *License:* **MIT** (Mermaid is open source).

Architectural views (used in §6.4)

- 📄 **P. Kruchten, "Architectural Blueprints — The 4+1 View Model of Software Architecture"**, *IEEE Software*, 1995 — PDF. The primary source for §6.4; short and very readable. *License:* IEEE copyright, *free to read* via a university course page.
- 📄 **ISO/IEC/IEEE 42010 — Architecture description** — the standard behind "views," "viewpoints," and "concerns." Overview and definitions at archived overview site (original site now offline); official catalog page: iso.org/standard/74393.html. *License:* standard is paid; the overview page is *free to read*.

- 📘 **Software Architecture in Practice / SEI resources** — L. Bass, P. Clements, and R. Kazman's book (4th ed. 2021, insights.sei.cmu.edu) defines architecture as the set of structures needed to reason about a system and names the module / component-and-connector / allocation structure families (§6.1.1, §6.4.2); the Software Engineering Institute also publishes free technical reports on *views and beyond* and *architecture documentation* at resources.sei.cmu.edu. *License:* book is commercial; SEI reports are free to download (U.S. government–funded FFRDC reports).
- 📘 **"Shape Up," Ryan Singer (Basecamp), chs. "Principles of Shaping" and "Find the Elements"** — shaped work as rough / solved / bounded, breadboarding (places/affordances/connections), fat-marker sketches, and designing at the right fidelity (used in §6.1.2). https://basecamp.com/shapeup/1.1-chapter-02 · https://basecamp.com/shapeup/1.3-chapter-04 · *Free to read online; © Basecamp.*

Quality-attribute scenarios

- 📘 **"Software Architecture in Practice" (4th ed.), Bass, Clements & Kazman** — the standard treatment of quality-attribute scenarios (§6.1.4) and architecture evaluation. https://www.informit.com/store/software-architecture-in-practice-9780136886099. *License:* commercial book; the sample chapters and companion material on the publisher and SEI sites are *free to read*.

License note

Linked resources remain under their own licenses; where a resource is only *free to read* rather than openly licensed, treat its text as read-only and cite it rather than copying it. This page itself is licensed **CC BY-SA 4.0**.

Chapter 7 — Architectural Patterns

> **Where we are.** Chapter 6 argued that the way to tame a large system is to decompose it into modules with high cohesion and low coupling. That advice is true but abstract: it tells you *what good structure looks like* without telling you *which structure to reach for* when you sit down to design a real system. This chapter closes that gap. An **architectural pattern** is a reusable, named arrangement of modules and connectors that has repeatedly solved a recurring design problem.[1] [2] Learning the catalog means you rarely start from a blank page — and, better still, that you can *reason about the trade-offs* of a structure before you commit code to it.

A pattern is not a library you install or a framework you inherit. It is a *shape* — a recurring answer to a recurring question about how responsibilities and dependencies should be arranged. When someone says "we'll put a message broker in the middle" or "the UI is just a view over the model," they are naming a pattern, and everyone who knows the vocabulary immediately shares a picture of the structure, its participants, and the compromises it implies. The main benefit is that participants leave with a shared view of the system. Patterns give a team a *design language* precise enough to argue in.

One honest caveat before the catalog: the patterns in this chapter do not all live at the same altitude, even though tradition files them under one name.[3] Layering, shared data, and client-server shape *entire systems*; MVC organizes a *subsystem*; the observer is an object-level *design pattern* that architectures merely lean on; REST is a *convention for interfaces* between systems; and product lines are a *strategy across a family* of systems. Keep the altitude in mind as you read — "should we use observer or a product line?" is not a choice anyone faces, and knowing which level a pattern addresses tells you which decisions it can and cannot make for you.

Every pattern in this chapter is presented the same way, so you can compare them fairly: the **problem** it solves, its **structure** (with a diagram), the **participants** and their responsibilities, the **trade-offs** you accept by adopting it, and a concrete **example** from systems you have probably used. Read for the trade-offs above all. The mark of an engineer, as Chapter 1 put it, is knowing *when a pattern earns its cost and when it is overkill*, not merely that it exists.

> **Principle.** A pattern is a *contract with consequences*. Adopting it buys you some qualities (flexibility, testability, scalability) and charges you others (indirection, latency, more moving parts). If you cannot name what you are buying and what you are paying, you are cargo-culting, not designing.

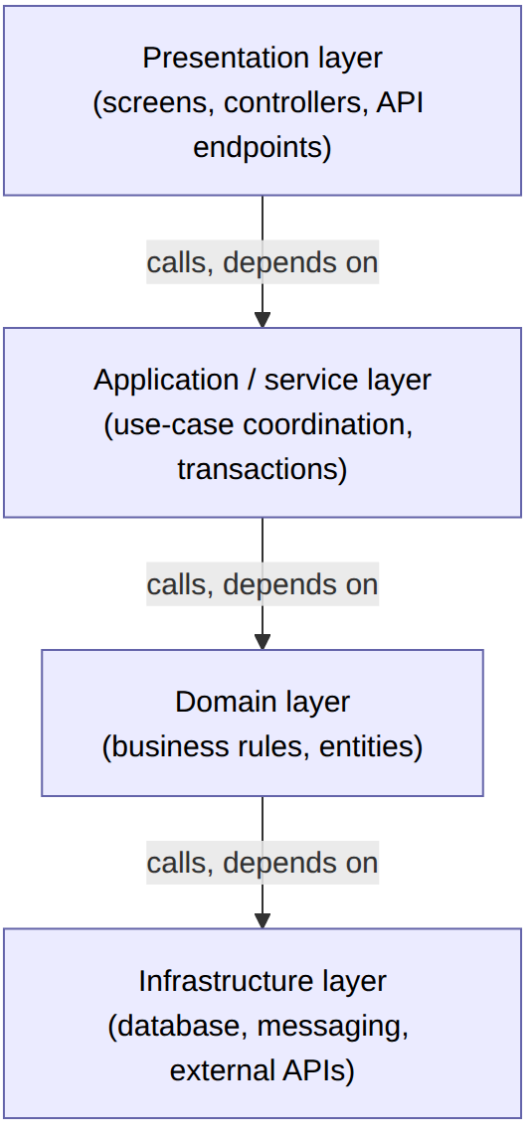
7.1 Software Layering

7.1.1 The Layered Pattern

Almost every substantial system you will build is organized, at the coarsest level, into **layers**. The **layered pattern** stacks the system into horizontal tiers, where each layer offers services to the layer directly above it and consumes services from the layer directly below.[1] A layer is a group of modules that operate at the same level of abstraction: the top speaks the language of the user's problem, the bottom speaks the language of the machine, and the middle layers translate progressively between them.

Problem it solves. A system spans a huge range of abstraction. At the top, you care about "confirm the appointment"; at the bottom, you care about "write these bytes to a socket." If every part of the code were free to talk to every other part, a change to the database driver could ripple into the appointment-booking logic, and you could not reason about either in isolation. Layering imposes order on that range of abstraction by insisting that dependencies point *downward only*. High-level policy depends on low-level mechanism through a stable interface, never the reverse.

Structure. A classic business application uses three or four layers. The **presentation** layer renders the interface and interprets user input. The **application** (or service) layer coordinates use cases and enforces workflow. The **domain** layer holds the core business rules and entities. The **infrastructure** (or persistence) layer talks to databases, message queues, and the outside world.



Participants. Each layer is a participant with one responsibility: to provide a coherent set of services at its level of abstraction and to hide the layer beneath it. The *interface* of a layer — the set of operations it exposes upward — is itself a participant, because it is the contract that lets the upper layer stay ignorant of the lower layer's internals.

A design decision you must make explicitly is whether layers are **strict** (a layer may call *only* the layer immediately below it) or **relaxed** (a layer may call any layer below it). Strict layering maximizes isolation — you could replace the domain layer's notion of persistence without the presentation layer noticing — but it costs you "pass-through" code, where a layer exposes an operation for no reason but to forward it down. Relaxed layering removes that boilerplate but weakens the guarantee, because now the presentation layer might reach past the application layer straight into infrastructure, coupling the top of the stack to the bottom.

A real example. The seven-layer OSI model of network protocols is the textbook case: your application code hands a message down the stack without knowing whether it travels over copper, fiber, or radio, because each

layer presents a clean service to the one above.[4] Closer to daily work, the standard three-tier web application — browser, application server, database — is layering you already rely on, and any framework organized as "controllers → services → repositories" is layering by another name.

7.1.2 Design Trade-offs

Layering is popular because its benefits are real and general. It gives you **substitutability**: because a layer depends only on the *interface* below it, you can swap one implementation for another — an in-memory store for a real database in tests, a new payment provider for an old one — without touching the layers above. It gives you **separation of concerns**, so a developer can work on presentation without understanding persistence. And it gives you a **shared mental model**: new engineers can find their way because "where does this code go?" usually has an obvious layered answer.

But the pattern is not free, and pretending otherwise is a common source of pain.

> **Pitfall.** *The pass-through tax.* In strict layering, a field the user enters at the top must often be threaded through every layer to reach the database — a change to add one field touches presentation, application, domain, and infrastructure. When most of your layers merely forward data without transforming it, the layering is buying you little and costing you edits in four files per change.

There is also a **performance** cost. Each layer boundary is a call, sometimes a serialization, occasionally a network hop. For most applications this is negligible; for a latency-critical inner loop it is not, and you may deliberately collapse layers in the hot path. And layering says nothing about horizontal scaling — a beautifully layered monolith is still one deployable unit.

This decision balances **isolation against directness**. Strict layers isolate change but add indirection; relaxed layers are direct but let coupling leak. The right call depends on which changes you expect. If you expect to swap infrastructure often, pay for strictness. If your layers are thin and stable, relax them. Notice that this is the Chapter 1 principle in action: *make likely changes cheap*, and accept a small standing cost to do it.

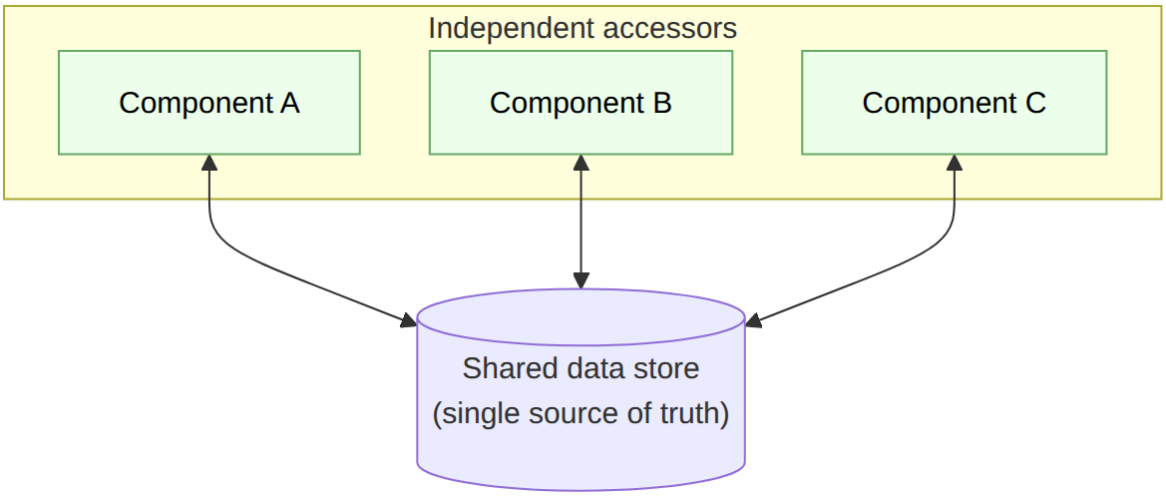
7.2 Three Building Blocks

Before the larger architectures, you need three smaller building blocks that appear *inside* almost all of them. They are the mortar between the bricks: how modules share state without becoming tangled (shared data), how they react to each other's changes without tight coupling (the observer), and — implicit in both — how a stable interface lets independent parts evolve. Master these and the big patterns become recombinations of ideas you already understand.

7.2.1 The Shared-Data Pattern

Problem it solves. Many components need to read and write a common body of data: several tools operate on the same document, several services on the same customer record. If each component kept its own copy, you would face the treacherous job of keeping the copies consistent. The **shared-data** (or **repository**) pattern gives the components a single, central data store that they all access through a defined interface, making the store the authoritative source of truth.[2]

Structure. A central **data store** sits at the hub. Around it, independent **accessor** components read and write, but they do not talk to each other directly — they communicate *through* the shared data. The store owns concerns that would otherwise be duplicated: consistency, concurrency control, integrity constraints, and durability.



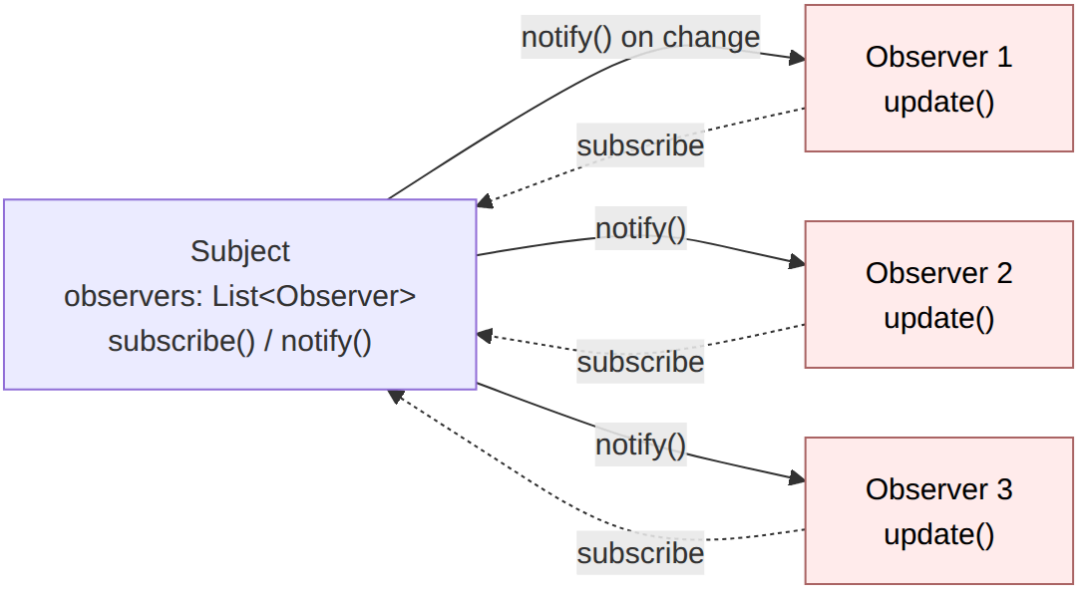
Participants. The **data store** is responsible for holding state and enforcing its invariants. The **accessors** are responsible for the actual computation; they are decoupled from each other because their only shared knowledge is the store's schema and interface. Often one accessor's writes become another's inputs — a compiler's symbol table, written by the parser and read by the type checker, is a shared-data design.

Trade-offs. The great advantage is that accessors are mutually independent: you can add a new one without touching the others, because they coordinate only through the store. The great danger is that the store becomes a **coupling magnet** and a **bottleneck**. Because everyone depends on the schema, a schema change can break every accessor at once — the store's interface is now a system-wide contract. And because every access funnels through one place, the store can become the scalability limit and a single point of failure. Relational databases, blackboard systems in AI, and the Redux "single store" in front-end apps are all shared-data designs, and all of them live with this same tension between convenient centralization and dangerous centralization.[1] [5]

7.2.2 Observers and Subscribers — the Observer Pattern

Problem it solves. One object holds state that others care about, and those others must react when it changes — a spreadsheet cell changes and every chart derived from it must redraw. The naive solution has the state-holder call each interested party directly, but that means the state-holder must *know about* every interested party, hard-coding a dependency on things that logically depend on *it*. The **observer** pattern inverts that dependency: interested parties **subscribe**, and the state-holder notifies its subscribers without knowing who or what they are.[6]

Structure. A **subject** maintains a list of **observers** and offers `subscribe` and `unsubscribe` operations. When the subject's state changes, it iterates its list and calls a uniform `notify`/`update` operation on each observer. The subject knows only that observers implement that operation; it knows nothing else about them.



In the clinic scheduler, the waiting-room display must redraw when an appointment's status changes:

```
class Appointment:
  def __init__(self, status="booked"):
    self.status = status
    self._observers = []

  def subscribe(self, observer):
    self._observers.append(observer)

  def set_status(self, new_status):
    self.status = new_status
    for obs in self._observers:      # any object with update() will do
      obs.update(self)

class WaitingRoomDisplay:
  def update(self, appointment):
    print(f"display: appointment is now {appointment.status}")

appt = Appointment()
appt.subscribe(WaitingRoomDisplay())
appt.set_status("arrived")              # prints: display: appointment is now arrived
```

Nowhere does `Appointment` name a concrete observer type — it iterates whatever subscribed and invokes each one's notification callback.

Participants. The **subject** (or *observable*) owns the state and the subscriber list. The **observer** (or *subscriber*) implements the callback that runs on change. The crucial property is the direction of the dependency: observers depend on the subject's *notification interface*, and the subject depends on the observers' *update interface* — neither depends on the other's concrete type. Because the subject depends only on the observer interface, a new observer can be added without recompiling the subject.

At larger scale the same idea becomes **publish-subscribe**: publishers emit *events* to named channels or topics, and subscribers register interest in topics rather than in a specific subject.[7] A message broker (§7.5.3) sits in the middle so that publishers and subscribers need not even know the other exists or be running at the same time. Observer is publish-subscribe in miniature, in one process; pub/sub is observer scaled across a network.

Trade-offs. You gain **loose coupling** and **extensibility**: the subject is closed to modification but open to new observers, which is precisely the flexibility a GUI toolkit or an event system needs. You pay in **traceability** and **surprise**. Control flow becomes implicit — reading the subject's code, you cannot see what happens when it fires an event, because the reactions live elsewhere and are wired up at runtime. Bugs hide in the ordering of notifications, in observers that trigger further changes (cascade and cycles), and in **lapsed listeners** that forget to

unsubscribe and leak memory. Use the observer when the set of reactions is open-ended and you want the source to stay ignorant of them; avoid it when a plain direct call would be clearer and the reactions are fixed.

7.3 User Interfaces: Model-View-Controller

7.3.1 Design Decisions

Interactive applications share a hard problem: the same underlying information must be *displayed* in one or more ways, *edited* through user gestures, and *stored* somewhere durable — and each of those concerns changes on a different schedule and for different reasons. Visual design changes when a designer redoes the look. Interaction changes when you add a keyboard shortcut. Business rules change when the domain changes. If all three concerns live in one tangled blob — the "smart widget" that draws itself, handles clicks, *and* enforces business rules — every change risks breaking the others, and none of it can be tested without a running screen.

The governing design decision, then, is a **separation of concerns**: keep the *data and rules* apart from the *presentation* apart from the *input handling*. A second decision follows: because one piece of data may drive several displays, the data should not depend on any particular display — displays should depend on the data. That is the observer relationship of §7.2.2, and it is the engine inside every UI architecture in this family.

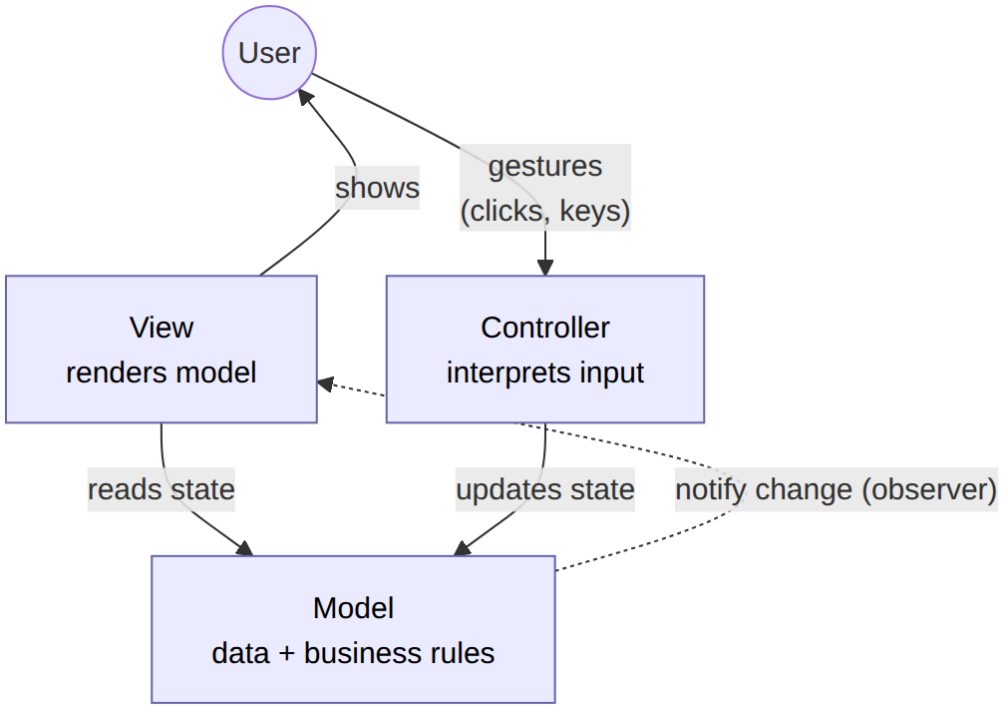
7.3.2 The Basic Model-View-Controller Pattern

Problem it solves. How do you build an interactive application so that its appearance, its input handling, and its core logic can each change independently and be tested independently? **Model-View-Controller (MVC)** answers by splitting the application into three roles.

Structure and participants.

- The **model** holds the application's data and business rules. It knows nothing about the screen. It is a plain, testable object that could run in a batch job with no UI at all. When its state changes, it announces the change to whoever is listening.
- The **view** renders the model for the user. It reads from the model and draws; it holds no business rules of its own. Several views may show the same model differently — a table and a chart of the same numbers.
- The **controller** interprets user input (clicks, keystrokes, gestures) and translates it into operations on the model. It contains the *interaction* logic that connects gestures to intentions.

The wiring is what makes MVC more than three boxes. The view **observes** the model: when the model notifies its views of a change, each view re-reads the model and redraws — so editing the data in one place causes every view to refresh.



A real example. The pattern was born in desktop UI toolkits and now underlies most web frameworks.[8] In a server-side web app, an incoming request is dispatched to a *controller* action, which manipulates *model* objects (and their database records) and then selects a *view* (a template) to render the response. The exact division of labor varies — UI frameworks have spawned close cousins called **MVP** (Model-View-Presenter) and **MVVM** (Model-View-ViewModel), which differ mainly in how much logic sits between the view and the model and in which direction the wiring points.[9] What they share, and what you should carry away, is the core commitment: *the model does not depend on the view*.

Trade-offs. MVC buys you **independent evolution and testability**. Because the model has no UI dependencies, you can unit-test all your business rules without simulating a screen — often the single biggest testability win available in an interactive app. You can restyle the view or add a second view without touching the model. The cost is **structure for its own sake in small cases**: a trivial screen split into three collaborating objects is more indirection than a beginner expects, and the boundary between "controller logic" and "model logic" is a perennial source of debate that spawns those competing variants. As with layering, the pattern pays off in proportion to how often, and how independently, the three concerns actually change.

7.3.3 Keep Views Simple

The single most valuable discipline within MVC is to keep views **thin** — sometimes called the *humble view*.[10] A view should contain as little logic as you can manage: ideally it only reads values from the model (or a view-model prepared for it) and maps them onto widgets. It should contain no business rules, no decisions that matter, and no state that cannot be trivially reconstructed from the model.

Why so insistent? Because the view is the *one part you cannot easily test*. Automated tests can exercise a model directly, but exercising a rendered screen usually means slow, brittle UI automation. Every branch, calculation, or rule you allow to creep into the view is a branch that escapes your fast, reliable tests and hides where only a human clicking around will find its bugs. Pushing that logic down into the model — or into a dedicated **presenter/view-model** whose job is to pre-compute exactly what the view should show — moves it back under test.

In the clinic app's invoice list, a fat view buries the overdue rule where no unit test reaches it:

```
class InvoiceWidget:
  def render(self, invoice, today):
    text = invoice.patient_name
    if not invoice.paid and (today - invoice.sent_on).days > 30:
      text += " — OVERDUE"         # a business rule, trapped on screen
    return text
```

The humble version moves the rule into a view-model and leaves the view nothing to decide:

```
def is_overdue(invoice, today):          # the rule, extracted where tests reach it
  return not invoice.paid and (today - invoice.sent_on).days > 30

def invoice_view_model(invoice, today):
  badge = " — OVERDUE" if is_overdue(invoice, today) else ""
  return {"text": invoice.patient_name + badge}

class InvoiceWidget:
  def render(self, vm):                # maps a precomputed field to a widget
    return vm["text"]
```

The extracted rule now tests in a few short lines, with no widget in sight:

```
def test_unpaid_31_days_is_overdue():   # the whole test: two lines
  assert is_overdue(Invoice("Adilyn", sent_on=date(2026, 6, 1), paid=False), date(2026, 7, 2))
```

> **Principle.** *Put logic where it can be tested.* The humble-view rule is a testability strategy, not a matter of aesthetics. Treat the boundary of your test coverage as a design constraint, and keep the untestable layer as thin as physically possible.

> **MVC in the wild.** Real frameworks realize the pattern under shuffled names. Django calls its variation **MTV (Model-Template-View)**: Django's "view" fills the controller role, and its "template" fills the view role.[11] Rails uses conventional server-side MVC.[12] **Single-page applications (SPAs)** built with React or Angular place the same separation in the browser, often using a **unidirectional data-flow** store (Flux/Redux): a view dispatches an action, the store updates, and the view renders again.[5] Large products have been built with each approach — Instagram with Django, Shopify with Rails, Airbnb with React.[13] [14][15] The architectural separation transfers across frameworks even when the names and update mechanisms differ.

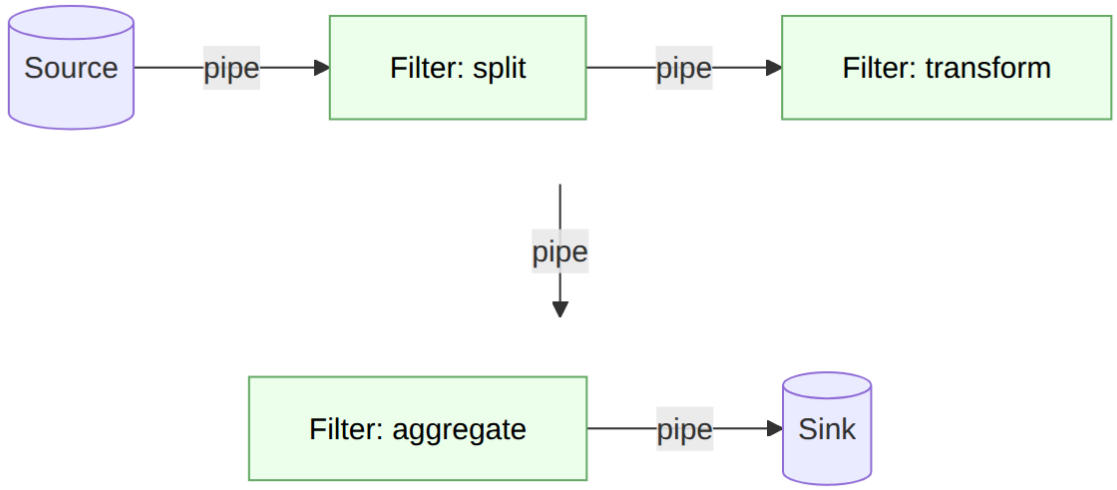
7.4 Dataflow Architectures

The patterns so far organize systems around *objects that call each other*. A different family organizes systems around the *flow of data through transformations*. In a **dataflow architecture**, you think first about the data — where it enters, how it is transformed step by step, and where it exits — and the components fall out as the stages that do the transforming. This inversion is powerful whenever the essence of a system is "take this stream of stuff and progressively turn it into that stream of stuff."

7.4.1 Dataflow Pipelines — Pipes and Filters

Problem it solves. You must perform a *sequence* of independent transformations on a body of data, and you want each transformation to be a self-contained unit you can build, test, reuse, and reorder without knowing about its neighbors. The **pipes-and-filters** pattern arranges the computation as a linear chain: each **filter** reads from an input **pipe**, transforms what it reads, and writes to an output pipe that feeds the next filter.[1] [2]

Structure and participants. A **filter** is a stateless-by-default component with one job: consume input, produce output. A **pipe** is the connector that carries data from one filter's output to the next filter's input — it is a buffer and a decoupler. Because filters communicate *only* through pipes and share no other state, a filter knows nothing about what comes before or after it; it only knows the shape of the data on its pipes.



A real example. The Unix shell pipeline is the canonical case: six independent filters chain into a "most frequent sources of 404 errors" report, and each command was written by someone who never imagined this particular pipeline.

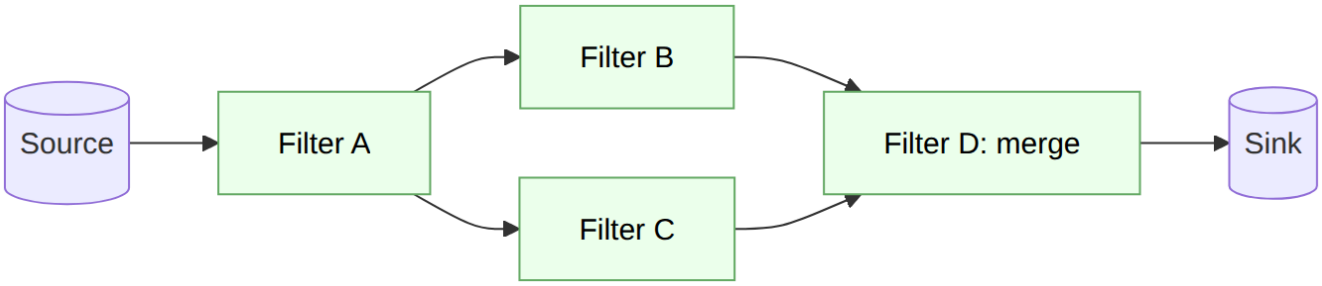
```
cat access.log  |   # read
  grep 404      |   # select
  cut -d' ' -f1 |   # project
  sort          |   # order
  uniq -c       |   # count
  sort -rn          # rank
```

Compilers are built the same way — lexer, parser, optimizer, code generator — and so is any image or audio processing chain.

Trade-offs. The wins are **reuse and composability** (filters recombine into new pipelines like Lego), **understandability** (each stage is simple in isolation), and easy **parallelism** (adjacent filters can run concurrently, each working on data as it arrives). The costs: pipes impose a **lowest-common-denominator data format** — often plain text or byte streams — so filters spend effort parsing and re-serializing at every boundary, and rich structure is awkward to pass through. A purely linear pipeline also handles **errors** poorly: there is no natural place for a filter deep in the chain to report a problem back upstream. And interactive, back-and-forth workloads simply do not fit the one-directional flow.

7.4.2 Dataflow Networks

A strict pipeline is linear, but real transformations often **branch and merge**. A **dataflow network** generalizes the pipeline into a directed graph: a filter may have several inputs and several outputs, data may fan out to parallel branches and fan back in, and the topology can express joins, splits, and conditional routing.



Problem it solves and trade-offs. Networks handle transformations whose natural shape is a graph, not a line — enriching a record from two sources and combining the results, or routing different record types down different processing paths. You keep the filters' independence and composability, but you pay in **complexity of coordination**: a node that merges two inputs must decide how to synchronize streams that arrive at different rates, and the graph can deadlock if a cycle of full buffers waits on itself. Stream processing frameworks, build systems (whose dependency graph *is* a dataflow network), and spreadsheet recalculation engines are all dataflow networks. This is the structure underneath modern **reactive** programming, where you declare how outputs are computed from inputs and the runtime propagates changes through the graph for you.

7.4.3 Unbounded Streams

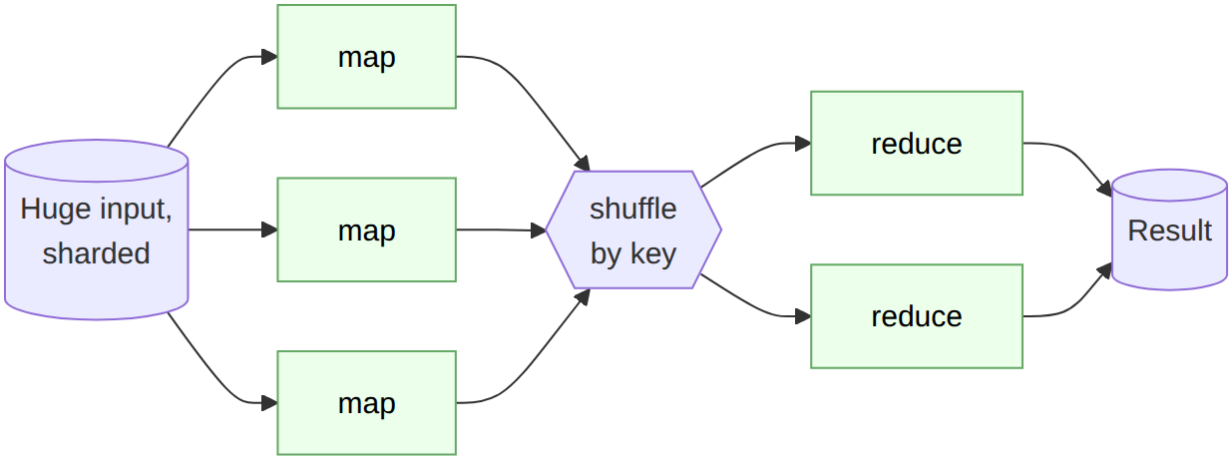
The pipelines above quietly assumed the data *ends* — a file has a last line, a document a last page. Many important systems process data that never ends: sensor readings, user clickstreams, financial ticks, log lines flowing in forever. An **unbounded stream** changes the rules, because a filter can no longer "read all the input, then produce all the output." It must produce results *incrementally*, from data it has seen so far, without ever waiting for an end that will not come.

Problem it solves and consequences. Stream processing lets you compute continuously and react in near-real-time instead of in nightly batches. But unboundedness forces design decisions the batch world could ignore. You must reason in **windows** — "the count of events in the last five minutes" — because "the total count" is never final.[16] You must handle **out-of-order and late data**, because events generated at one time may arrive later, and you must decide how long to wait. You must bound memory, because you cannot buffer an infinite stream, which means most computations must be *incremental* and approximate. And **backpressure** becomes essential: when a downstream filter cannot keep up, it must signal upstream to slow down — or the design must shed load deliberately — or buffers overflow and the system falls over.[17]

> **Case study.** A fraud-detection system watches a never-ending stream of card transactions and must flag a suspicious one within seconds. It cannot wait for a "batch" to close; it maintains a sliding window of recent activity per account, updates running statistics as each event arrives, and applies backpressure to the ingestion feed when the scoring stage saturates. Every hard part of this design is a direct consequence of the stream being *unbounded*.

7.4.4 Big Dataflows — MapReduce-Style

When the data is not just endless but *enormous* — too large for one machine — dataflow becomes a strategy for **distributing** computation across a cluster. The **MapReduce** style captures a pattern that recurs across huge batch jobs: express the computation as a **map** step that transforms each record independently, followed by a **reduce** step that aggregates the mapped results by key.[18] Because the map step treats records independently, the framework can shard the data across hundreds of machines and run the maps in parallel; because the reduce step aggregates by key, the framework can route all records with the same key to the same reducer.



Problem it solves and trade-offs. In MapReduce, *you* supply two small, sequential functions — map and reduce — and the *framework* handles the hard distributed concerns: partitioning the data, scheduling work, moving map outputs to the right reducers (the **shuffle**), and re-running tasks that fail on a flaky machine. You get massive parallelism and fault tolerance almost for free, provided your problem fits the map-then-reduce shape. The costs are real: the rigid two-phase structure forces awkward encodings for computations that are naturally iterative (many machine-learning algorithms) or that need many stages, and the shuffle — moving data across the network between phases — is often the dominant expense.[18] Later engines generalize the idea into arbitrary dataflow graphs of transformations to reduce that cost, but the core insight is the same: *describe the dataflow, let the platform distribute it.*[19]

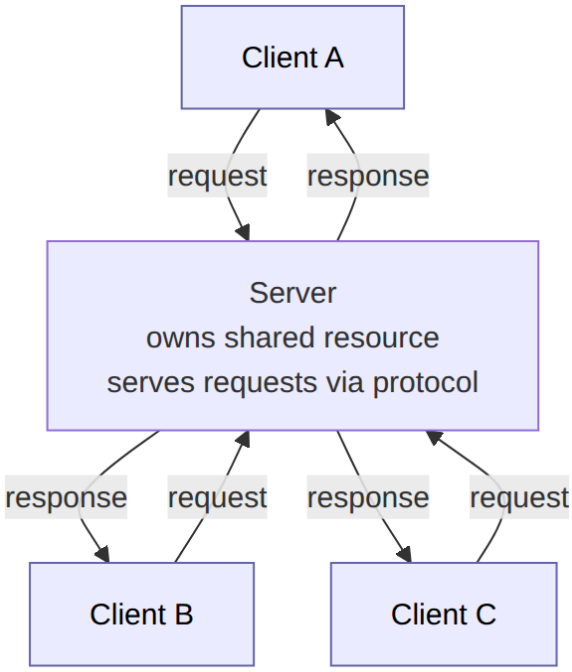
7.5 Connecting Clients with Servers

The previous patterns mostly organize a single program. The next family organizes systems that span **multiple machines**, where the fundamental questions become who initiates contact, who holds the shared state, and how parties find one another across a network.

7.5.1 The Client-Server Pattern

Problem it solves. Many users, on many machines, need shared access to common data or computation — a shared calendar, a bank ledger, a search index. You cannot put a copy on every device and hope they agree. The **client-server** pattern designates one role, the **server**, as the keeper of the shared resource, and another role, the **client**, as the many requesters that ask the server to do work on their behalf.[2] Clients initiate; the server responds.

Structure and participants. The **server** owns the authoritative resource and waits for requests, handling each according to a published **protocol** (the request/response contract). The **client** knows the protocol and the server's address, sends requests, and consumes responses; clients generally do not talk to each other. The protocol is the interface that lets the two sides evolve independently, just as a layer's interface does in §7.1.



A real example. The web is client-server at planet scale: your browser (client) sends HTTP requests to web servers that own the pages and data. A database server serving many application instances, an email server, a game server — all follow the same shape. **Trade-offs:** you gain a single authoritative source of truth and centralized control of security and consistency, but the server is a **bottleneck and single point of failure**, which is why real systems replicate and load-balance servers. The pattern also raises the question of *how thick* the client should be

— a thin client (just a display) keeps logic central and easy to update but demands constant connectivity; a thick client works offline and offloads the server but is harder to keep consistent and to upgrade.

7.5.2 Deploying Test Servers

A practical consequence of client-server structure is that you can *substitute the server*. Because the client depends only on the **protocol**, not on any particular server implementation, you can point the client at a stand-in server during development and testing — a **test server** (often called a *mock*, *stub*, or *fake*).

Why this matters. Testing a client against the real production server is slow, flaky, and dangerous: the real server may be unavailable, may hold irreplaceable data you do not want to mutate, or may be a paid third-party API. A test server implements just enough of the protocol to exercise the client's behavior — you can program it to return specific responses, simulate errors and timeouts, and record what the client sent. This lets you test client behavior that is nearly impossible to trigger against the real thing, such as "what does the UI do when the server returns a 500?" A short stub built from nothing but the standard library answers that question for the clinic client:

```
import http.server, threading, urllib.error, urllib.request
# stub server, client, and failing-path check: about twenty lines of stdlib

class AlwaysFail(http.server.BaseHTTPRequestHandler):
  def do_GET(self):
    self.send_error(500)
  def log_message(self, *_):           # keep test output clean
    pass

def fetch_appointments(base_url):
  try:
    with urllib.request.urlopen(base_url + "/appointments") as resp:
      return resp.read()
  except urllib.error.HTTPError:
    return []                          # fallback: empty schedule, not a crash

stub = http.server.HTTPServer(("127.0.0.1", 0), AlwaysFail)
threading.Thread(target=stub.serve_forever, daemon=True).start()

assert fetch_appointments(f"http://127.0.0.1:{stub.server_port}") == []
stub.shutdown()
```

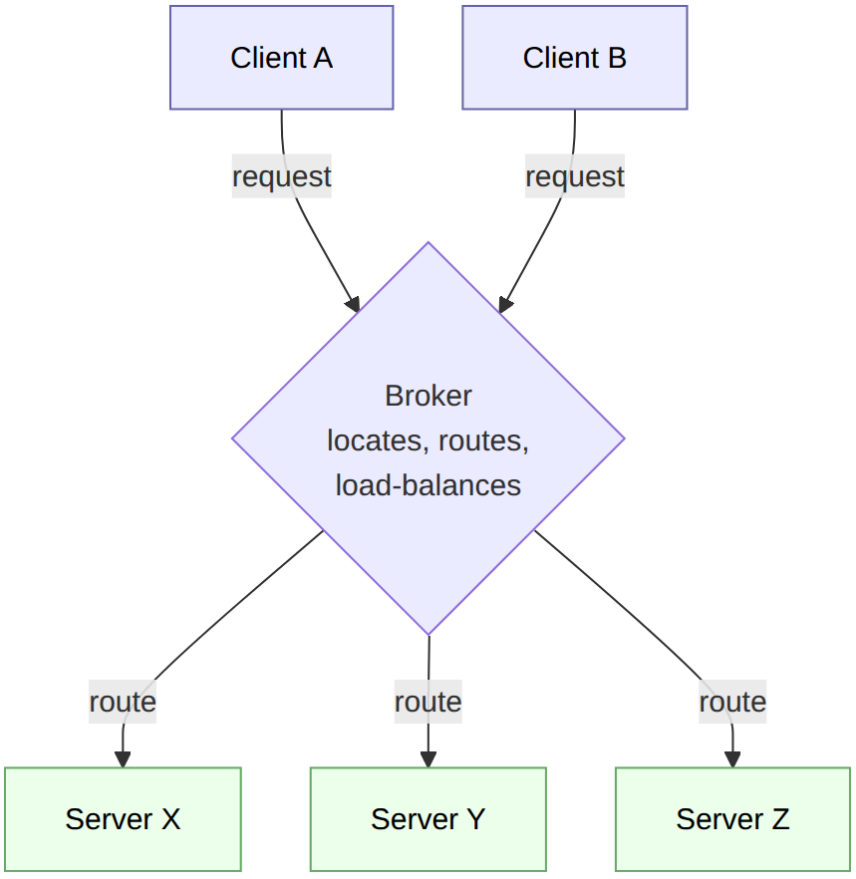
The substitution is the network-tier echo of the layering benefit in §7.1.2: *depend on an interface, and you can swap the implementation.* The trade-off is that a test server is only as faithful as you make it — tests can pass against a fake that behaves differently from reality, so a smaller number of full **integration tests** against a real

server remain essential (Chapter 10). The same substitution idea, scaled to production, powers the blue-green and canary deployment strategies of Chapter 14.

7.5.3 The Broker Pattern

Plain client-server assumes the client knows *which* server to call and *where* it lives. That assumption breaks down when there are many servers, when they come and go, or when you want clients and servers to be ignorant of each other's location and identity. The **broker** pattern inserts an intermediary — a **broker** — between clients and servers to decouple them.[1]

Problem it solves. As a system grows into many services, hard-wiring every caller to every callee's address creates a brittle web of point-to-point dependencies: move or rename a service and every caller breaks. A broker centralizes the connective tissue. Clients send requests to the broker, which is responsible for *locating* an appropriate server, *routing* the request to it, and *returning* the result — so clients address the broker, not the server, and servers can be added, moved, or replaced without any client knowing.



Participants and examples. The **broker** registers available servers, resolves each request to a capable server, and may add load-balancing, retries, and monitoring. The **client** and **server** are relieved of finding each other. Message brokers (the engine behind publish-subscribe at scale, §7.2.2), API gateways, service meshes, and the

classic CORBA object request broker are all instances.[1] **Trade-offs:** you gain **location transparency**, dynamic membership, and a natural home for cross-cutting concerns like load-balancing and observability. You pay in **latency** (an extra hop), in **complexity** (the broker itself must be built, deployed, and scaled), and in **risk concentration** — the broker can become the very single point of failure it was meant to help you avoid, so in practice it too must be made redundant.

7.5.4 RESTful APIs

Client-server tells you *who talks to whom*; it does not tell you what the conversation looks like. The dominant answer on today's web is **REST** (Representational State Transfer) — less a new pattern than a set of conventions that make the client-server pattern scale across the internet.[20] An API that follows them is called **RESTful**.

Problem it solves. If every server invents its own vocabulary of operations (`getPatient`, `fetch_appt`, `makeBookingV2`…), every client must learn every server's private language, and generic infrastructure — caches, proxies, browsers, load balancers — can help with none of it. REST standardizes the conversation so that *any* client can talk to *any* server using the same small grammar.

Structure. Four conventions do the work:

- **Resources, named by URIs.** Everything interesting is a **resource** with a stable name: `/patients/143`, `/patients/143/appointments`, `/appointments/987`. Nouns, not verbs — the URI names a *thing*, not an *action*.
- **A fixed set of verbs.** Instead of unlimited custom operations, HTTP's methods act on resources. By common convention **GET** reads (safely, with no side effects), **POST** often creates, **PUT/PATCH** update, and **DELETE** removes.[21] The clinic app's "mark patient arrived" becomes `PATCH /appointments/987` with body `{"status": "arrived"}` — no bespoke `markArrived` endpoint to document or learn.
- **Representations.** The client never touches the server's internal objects; it exchanges **representations** of them — today almost always JSON. The server's database schema can change freely as long as the representation stays stable (an interface, in Chapter 6's sense, at network scale).
- **Statelessness.** Each request carries everything the server needs (identity, parameters); the server keeps **no per-client session state** between requests.[20] That is what lets any of the broker's servers (§7.5.3) answer any request — the foundation of horizontal scaling.

Example. A slice of the clinic scheduler's API:

Request	Meaning
`GET /patients/143/appointments`	List patient 143's appointments
`POST /patients/143/appointments`	Book a new appointment for patient 143
`GET /appointments/987`	Fetch one appointment's representation
`PATCH /appointments/987`	Update it (e.g., `{"status": "arrived"}`)
`DELETE /appointments/987`	Cancel it

POST, PUT, or PATCH? The three writing verbs are the ones students confuse, and the choice turns on two questions. First, *who names the new resource?* Reach for **POST** when the *server* assigns the URL: `POST /patients/143/appointments` creates a fresh `/appointments/987`, and posting again makes a *second* appointment — POST is not **idempotent**. Reach for **PUT** when the *client* already knows the URL and wants to create or wholly *replace* what lives there, sending the complete representation; repeating the same `PUT /appointments/987` leaves the resource in the same state, so PUT *is* idempotent. Second, *full replacement or a partial change?* Reach for **PATCH** to alter part of an existing resource, sending only the fields that change — `PATCH /appointments/987` with `{"status": "arrived"}` touches nothing else. The one-line version: **POST** creates under a server-chosen name, **PUT** replaces in full, and **PATCH** changes in part.

> **Pitfall.** Idempotency is not a nicety — the network will retry. If a client times out and resends, an idempotent `PUT` or `DELETE` is harmless, but a resent `POST` can double-book the appointment. Design writes so that the safely-repeatable verb is used wherever you can, and protect the ones that cannot be (e.g., with an idempotency key) rather than assuming each request arrives exactly once.

Trade-offs. You gain **uniformity** (one grammar for every service; tooling, caching, and testing all get cheaper), **evolvability** (representations decouple clients from server internals), and effortless fit with the web's infrastructure. You pay when the domain is not noun-shaped — long-running operations, batch actions, and "verbs with no natural resource" (send reminder? merge records?) force modeling contortions — and statelessness pushes real state you *do* need (sessions, carts) into tokens or the shared data layer (§7.2.1). When a client needs many resources at once, REST's one-resource-per-request grain can also mean chatty interfaces.

> **Principle.** REST is the interface discipline of Chapter 6 applied across a network: stable names, a small uniform contract, and hidden internals. Every real API still needs documentation — schemas, authorization rules, error formats — but if every endpoint requires its own bespoke interaction model that consistent resource-and-HTTP semantics cannot explain, the API is gaining little from REST's uniform interface.

One more pair of names to recognize: **service-oriented architecture (SOA)** and its modern descendant, **microservices**, describe systems built as fleets of small services calling one another — the services-calling-

services lineage that client-server, broker, and REST together make workable at scale.[22]

7.6 Families and Product Lines

Everything so far treats architecture as the design of *one* system. But organizations rarely build one system in isolation; they build a *family* of related systems — the same product for different platforms, tiers, or customers. A phone maker ships dozens of handset models running variations of one software base. A car company runs related but distinct control software across an entire fleet. Treating each as a from-scratch project wastes the enormous overlap between them. **Software product-line engineering** means building a family of related products from a shared set of assets in a planned way, and it changes what "architecture" is for.[23]

7.6.1 Commonalities and Variabilities

The core analytical move is to separate what all family members share from what distinguishes them. A **commonality** is a feature, behavior, or component present in *every* product of the family — every handset has a dialer, every edition of the app authenticates users. A **variability** is a point where products *differ* — one edition has video calling and another does not; one region requires a different tax calculation.

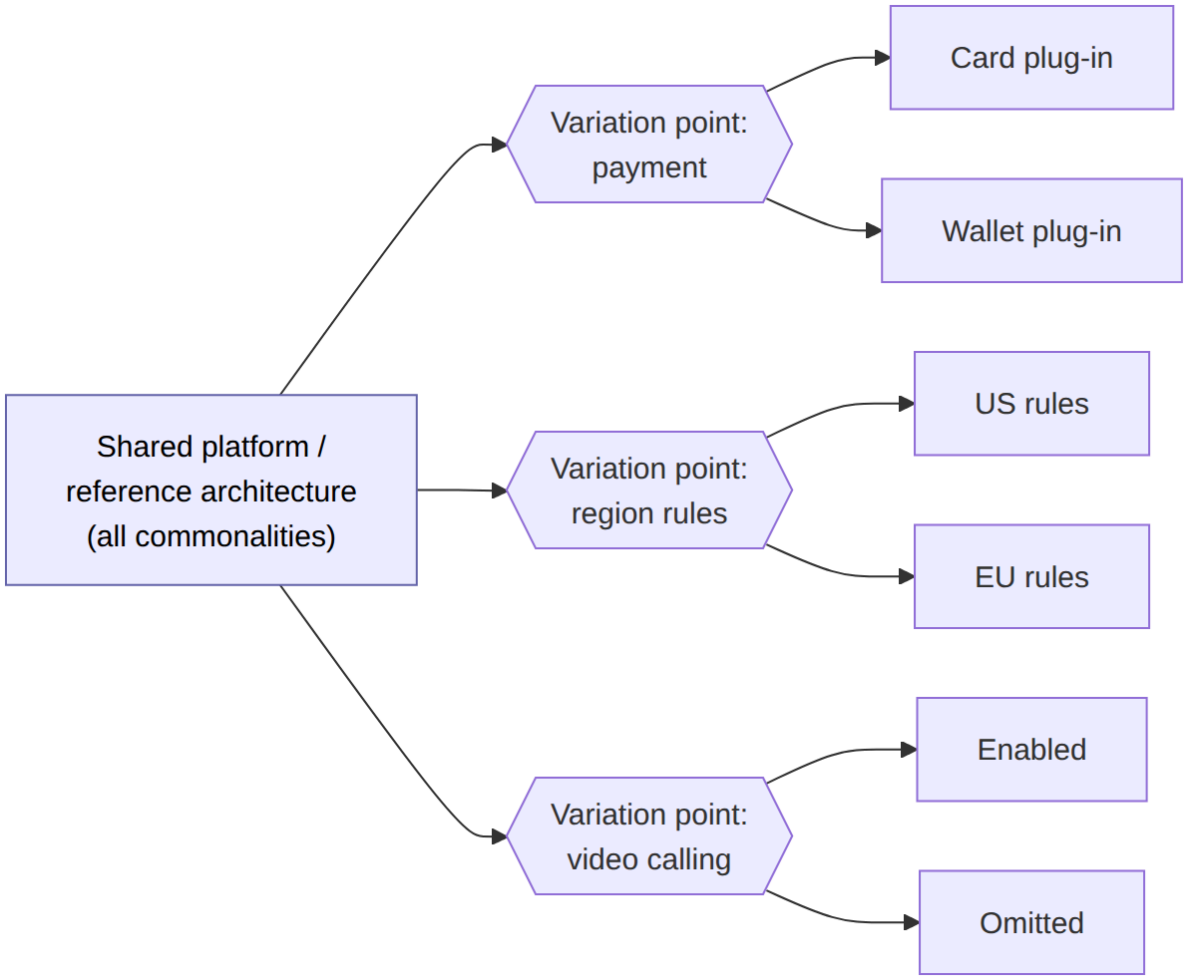
Why the distinction matters. Commonalities are built *once* and reused without change across the family, so effort spent making them solid is amortized over every product. Variabilities are the points you must design to *vary*, so each one needs an explicit mechanism — a configuration flag, a plug-in slot, a swappable module, a compile-time option — that lets a given product select its variant without modifying shared code. The design work of a product line is largely the work of **enumerating the variabilities and choosing a variation mechanism for each**. A **feature model** is the common tool for recording this: it captures which features are mandatory, which are optional, and which are mutually exclusive, so that each valid product is a permitted selection of features.[24]

> **Pitfall.** Guessing wrong about which axis will vary is expensive. If you hard-code something that later must differ across products, retrofitting variability into shared code is painful; if you build elaborate variation machinery for an axis that never varies, you paid for flexibility you never use. Product-line design lives or dies on predicting the right variabilities — which is why it rewards domains you understand deeply.

7.6.2 Software Architecture and Product Lines

A product line needs a special kind of architecture: a **reference architecture** (or *platform*) that is shared by every product in the family and that makes the planned variabilities easy to exercise. This raises the stakes on the patterns earlier in the chapter. Every variation point is, in the end, an *interface behind which alternatives plug in* — and the patterns you have learned are the machinery for building such interfaces:

- **Layering** lets you vary one layer (say, the platform-specific infrastructure) across products while the layers above stay identical.
- The **observer** and **broker** patterns let a product add or swap behavior by registering different components, without editing the shared core.
- **Plug-in** structures — a stable core that discovers and loads optional modules — are the workhorse variation mechanism, and they are just the open/closed discipline of the observer pattern applied at the scale of whole features.

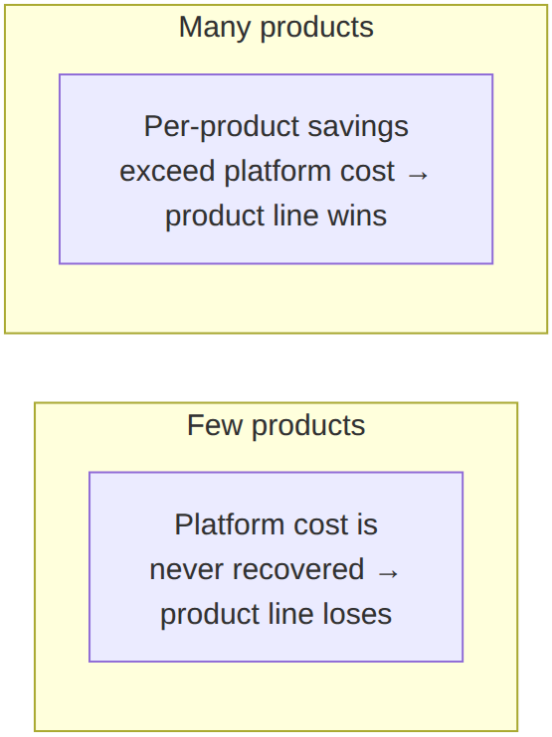


The organizational consequence is a split between two activities: **domain engineering** builds the reusable platform and its variation points, and **application engineering** assembles a specific product by selecting variants and adding only what is unique to it.[25] Getting this split right is what turns “we copy-pasted last year’s product and hacked on it” into a genuine, maintainable family.

7.6.3 Economics of Product-Line Engineering

Product lines are an *economic* bet, and you should be honest about the arithmetic. Building a reusable platform with well-designed variation points costs **more up front** than building a single product — you are engineering for

change that has not happened yet.[25] That investment pays back only when you build *enough* products from the platform that the per-product savings exceed the extra platform cost.



Until you build enough products from the platform to reach break-even, you would have been better off building each system directly. The savings, when they come, are not only in initial construction but in **maintenance and quality**: fix a bug in a commonality once and every product inherits the fix; a feature hardened in one product is available to all. The risks mirror the rewards — an over-engineered platform, a mis-predicted variability (§7.6.1), or a domain too unstable to have durable commonalities can leave you paying the up-front cost without collecting the return. The engineering judgment is therefore the same one that governs every pattern in this chapter: *invest in flexibility exactly where change is likely, and not one place more.*

7.7 Conclusion

Architectural patterns are the vocabulary of large-scale design. Each is a named answer to a recurring structural question, and each answer is a trade-off — you adopt it to buy a quality you need and you pay for it in some other coin. The through-line of the whole chapter is the Chapter 1 principle restated at every scale: *structure the system so that the changes you expect are cheap, and do not pay for flexibility you will not use.* Layering makes it cheap to swap an implementation behind an interface. The observer makes it cheap to add a new reaction to a change. MVC makes it cheap to evolve appearance, interaction, and rules on separate schedules. Dataflow makes it cheap to reorder and parallelize transformations. Client-server, broker, and product lines push those same ideas across machines and across whole families of products.

The table below is your quick-reference map. Treat it as a catalog: choose the *smallest* pattern that addresses the problem in front of you, and make its costs explicit.

Pattern	Problem it solves	Key trade-off
Layering	Span a wide range of abstraction without tangling high- and low-level code	Isolation vs. pass-through indirection and per-boundary cost
Shared data	Many components must operate on one authoritative body of data	Convenient central truth vs. coupling magnet and bottleneck
Observer / pub-sub	Parties must react to a change without the source knowing them	Loose coupling vs. hidden, hard-to-trace control flow
Model-View-Controller	Appearance, input, and rules must evolve and test independently	Testable separation vs. extra structure and boundary debates
Pipes and filters	Apply a sequence of reusable, recombinable transformations	Composability vs. lowest-common-denominator data format
Dataflow network	Transformations whose natural shape branches and merges	Expressive graph vs. coordination and deadlock complexity
Unbounded streams	Process never-ending data with near-real-time results	Continuous, incremental results vs. windowing, late data, backpressure
MapReduce-style	Batch-process data too large for one machine	Free parallelism/fault tolerance vs. rigid two-phase shape and shuffle cost
Client-server	Many users need shared access to one authoritative resource	Central control vs. bottleneck and single point of failure
Broker	Many services must find each other without hard-wired locations	Location transparency vs. extra hop, complexity, and risk concentration
RESTful API	Every server invents its own vocabulary, defeating generic clients and tooling	Uniform grammar and evolvability vs. contortions for non-resource-shaped operations
Product line	Build a family of related products from shared assets	Reuse and consistency vs. up-front platform cost and misprediction risk

No system of any size uses just one pattern. A real application layers presentation over domain over infrastructure (layering), renders through MVC (which runs on the observer), talks to a database (shared data) across the network (client-server over a REST API, perhaps via a broker), ingests events as a stream (dataflow), and — if the company is lucky enough to sell many editions — does all of this on a shared platform (product line). Learning the patterns individually is how you learn to *combine* them, and combining them well is what architecture is.

Sources

[1] Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad & Michael Stal, *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns* (1996). wiley.com.

[2] Mary Shaw & David Garlan, *Software Architecture: Perspectives on an Emerging Discipline* (1996). dl.acm.org.

[3] Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad, and Michael Stal, *Pattern-Oriented Software Architecture, Volume 1: A System of Patterns* (Wiley, 1996) — which files recurring structures into three levels: architectural patterns, design patterns, and idioms. wiley.com.

[4] ISO/IEC, *ISO/IEC 7498-1:1994 — Open Systems Interconnection — Basic Reference Model* (1994). iso.org.

[5] Redux project, *Three Principles*, Redux documentation (accessed 2026). redux.js.org.

[6] Erich Gamma, Richard Helm, Ralph Johnson & John Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software* (1994). informit.com.

[7] Microsoft, *Publisher-Subscriber Pattern*, Azure Architecture Center (accessed 2026). learn.microsoft.com.

[8] Trygve Reenskaug, *MVC: Xerox PARC 1978–79* (1979; the inventor's notes). folk.uio.no.

[9] Martin Fowler, *GUI Architectures* (2006). martinfowler.com.

[10] Martin Fowler, *Humble Object* (2020; after Michael Feathers' "Humble Dialog Box" and Gerard Meszaros' *xUnit Test Patterns*). martinfowler.com.

[11] Django Software Foundation, *FAQ: General*, Django documentation (accessed 2026). docs.djangoproject.com.

[12] Rails team, *Getting Started with Rails*, Rails Guides (accessed 2026). guides.rubyonrails.org.

[13] Instagram Engineering, *Static Analysis at Scale: An Instagram Story* (2019). instagram-engineering.com.

[14] Shopify Engineering, *Deconstructing the Monolith* (2019). shopify.engineering.

[15] Airbnb Engineering, *Rearchitecting Airbnb's Frontend* (2018). medium.com/airbnb-engineering.

[16] Tyler Akidau, *Streaming 101: The World Beyond Batch* (2015). oreilly.com.

[17] Jonas Bonér, Dave Farley, Roland Kuhn & Martin Thompson, *The Reactive Manifesto* (2014). reactivemanifesto.org.

[18] Jeffrey Dean & Sanjay Ghemawat, *MapReduce: Simplified Data Processing on Large Clusters*, OSDI (2004). research.google.

[19] Matei Zaharia et al., *Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing*, NSDI (2012). usenix.org.

[20] Roy T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, ch. 5 (2000). ics.uci.edu.

[21] R. Fielding, M. Nottingham & J. Reschke (eds.), *RFC 9110: HTTP Semantics*, IETF (2022). rfc-editor.org.

[22] James Lewis & Martin Fowler, *Microservices* (2014). martinfowler.com.

[23] Carnegie Mellon Software Engineering Institute, *Software Product Lines Collection* (accessed 2026). sei.cmu.edu.

[24] Kyo C. Kang, Sholom G. Cohen, James A. Hess, William E. Novak & A. Spencer Peterson, *Feature-Oriented Domain Analysis (FODA) Feasibility Study*, CMU/SEI-90-TR-021 (1990). sei.cmu.edu.

[25] Klaus Pohl, Günter Böckle & Frank van der Linden, *Software Product Line Engineering: Foundations, Principles and Techniques* (2005). link.springer.com.

- **Key takeaways** are summarized above in §7.7.
- Continue to the Exercises.
- Go deeper with the Open Resources for this chapter.

Chapter 7 — Exercises

Exercises are graded by depth: **[warm-up]** checks understanding, **[analysis]** asks you to reason about a design and its trade-offs. For every pattern you choose, the grading rubric is the same one an engineer applies in practice: *name what the pattern buys you and what it costs you.*

Concepts

1. **[warm-up]** In one sentence each, state the *problem* solved by the layered pattern, the observer pattern, and the client-server pattern. Then name the single quality each one is primarily trying to protect (for example, substitutability, or shared truth).

2. **[warm-up]** Explain the difference between *strict* and *relaxed* layering. Give one concrete situation where you would accept the pass-through cost of strict layering, and one where relaxed layering is the better call, and say why.

3. **[warm-up]** The observer pattern "inverts a dependency." Which object would depend on which in the naive (non-observer) design, and which way does the dependency point after you apply the observer? Draw the two arrows.

4. **[warm-up]** MVC insists that "the model does not depend on the view." Give one concrete benefit you lose the moment a business rule leaks into a view, and connect it to the humble-view principle from §7.3.3.

Analysis

5. **[analysis]** A colleague proposes storing all shared application state in one central store that every component reads and writes (the shared-data pattern). List two specific ways this will help the team as the app grows, and two specific ways it will hurt. Propose one design change that keeps most of the benefit while reducing the biggest risk.

6. **[analysis]** You are asked to add "email the customer whenever an order ships" to a system, and you expect more such notifications (SMS, push, webhook) to be requested soon. Compare two designs: (a) a direct call from the shipping code to the email code, and (b) an observer/publish-subscribe design where shipping emits an event. Argue which you would choose *today* and how your answer changes if only the email notification will ever exist.

7. **[analysis]** Take the Unix pipeline

```
cat log | grep ERROR | sort | uniq -c
```

Identify the filters and the pipes. Then describe one transformation that the pipes-and-filters structure makes *easy* (give the new pipeline) and one requirement that this structure makes *awkward*, explaining why the awkwardness is inherent to the pattern rather than to Unix. A fifteen-line sample log to experiment on lives at `code/ch07/data/access.log` — its interesting lines say `404` rather than `ERROR`, so adjust the `grep`.

8. **[analysis]** A batch report currently runs every night over a file of the day's transactions. Product now wants results "within ten seconds of each transaction." Explain, using §7.4.3, three specific design problems that switching from a bounded file to an unbounded stream forces you to confront that the nightly batch could ignore.

9. **[analysis]** Your mobile app talks to a payment provider's server over a documented HTTP protocol. Explain how the client-server structure lets you test the app's behavior for a *declined card* and a *server timeout* without ever touching the real provider. Then state one class of bug this test-server approach *cannot* catch, and what kind of test (Chapter 10) you would add to cover it.

10. **[analysis]** A team is splitting a monolith into many services and debating whether to let services call each other directly or route all calls through a broker. Give the two strongest arguments for the broker and the two strongest against. Under what growth conditions does the broker become worth its cost?

Choosing a pattern

For each scenario, choose the *most appropriate* pattern (or small combination) from this chapter, sketch its structure in two or three sentences, and — this is the graded part — justify the choice by naming what you buy and what you pay. There is often more than one defensible answer; the justification is what matters.

11. **[analysis]** A data-science team must process 40 terabytes of clickstream logs overnight to compute per-user visit counts. The job is embarrassingly parallel per record, then aggregated by user. Pick a pattern, and explain what the framework does for you and where your main cost will be.

12. **[analysis]** A desktop drawing program shows the same shapes in a canvas, a layers-list sidebar, and a properties panel; editing any one must instantly update the others. Pick a pattern (or combination), show the wiring, and explain how it keeps the three displays consistent without any of them knowing about the others.

Chapter 7 — Open Resources

Free, openly-licensed or freely-readable materials that reinforce this chapter. Types: 📘 open text/report · 🎓 course · 📄 primary source/catalog · 🎥 video. Each entry notes how freely it can be reused; unless stated, a resource is *free to read* but remains under its author's copyright.

Pattern catalogs (use across the whole chapter)

- 📄 **Martin Fowler — *Patterns of Enterprise Application Architecture*, online catalog** — martinfowler.com/eaaCatalog. Concise entries for layering, repository/shared-data, MVC and its relatives, and the service/domain split. *Free to read online; text is © the author.*
- 📄 **Martin Fowler — "GUI Architectures"** — martinfowler.com/eaaDev/uiArchs.html. The clearest free treatment of MVC, MVP, MVVM, the observer wiring behind them, and the humble-view idea used in §7.3.3. *Free to read; © the author.*
- 📄 **Martin Fowler — "Humble Object"** — martinfowler.com/bliki/HumbleObject.html. The humble-view discipline of §7.3.3 in pattern form, crediting Michael Feathers' "Humble Dialog Box" and Gerard Meszaros' *xUnit Test Patterns*. *Free to read; © the author.*
- 📄 **E. Gamma, R. Helm, R. Johnson & J. Vlissides — *Design Patterns* (1994), publisher's page** — informit.com. The original catalog entry for the observer pattern of §7.2.2. *Catalog page free to read; the book itself is a paid text.*
- 📄 **F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad & M. Stal — *Pattern-Oriented Software Architecture, Vol. 1: A System of Patterns* (1996), publisher's page** — wiley.com. The catalog that documents layers, broker, and blackboard as named patterns. *Catalog page free to read; the book itself is a paid text.*
- 📄 **MVC in real frameworks** — Django's own account of its MTV naming (Django FAQ), Rails' MVC code organization (Rails Guides), and Redux's single-store, one-way data flow (Three Principles), as used in §7.3.3's "MVC in the wild." *Free official documentation.*
- 📄 **Patterns at production scale** — engineering write-ups from Instagram on Django ("Static Analysis at Scale"), Shopify on Rails ("Deconstructing the Monolith"), and Airbnb on React ("Rearchitecting Airbnb's Frontend") — the systems named in §7.3.3. *Free to read; © the companies.*
- 📘 **Mark Richards — *Software Architecture Patterns*** (O'Reilly report) — a short, free report covering layered, event-driven, microkernel/plug-in, and other styles; register for the free download at O'Reilly. Excellent companion to §7.1 and §7.6. *Free with registration; © O'Reilly.*
- 📄 **Microsoft — Cloud Design Patterns** — learn.microsoft.com/azure/architecture/patterns. Practical, vendor-documented patterns for client-server, broker/gateway, pub-sub, and streaming problems, each with problem/solution/considerations. *Free to read; © Microsoft, many samples under permissive licenses.*

Dataflow, streams, and big data (§7.4)

- 📄 **The Reactive Manifesto** — reactivemanifesto.org. A short, free statement of the principles behind responsive, message-driven, back-pressured systems — directly relevant to dataflow networks and unbounded streams. *Free to read.*
- 📄 **J. Dean & S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters"** (OSDI 2004) — the original paper behind §7.4.4, free from Google Research: research.google. *Free to read; © the authors/publisher.*
- 📄 **M. Zaharia et al., "Resilient Distributed Datasets" (NSDI 2012)** — usenix.org. The Spark paper: how later engines generalized MapReduce into arbitrary dataflow graphs (§7.4.4). *Open access via USENIX.*
- 🎥 **Tyler Akidau — "The World Beyond Batch: Streaming"** (talk/companion articles) — a free, accessible introduction to windows, event time, and late data that underpins §7.4.3. O'Reilly Radar: Streaming 101. *Free to read.*

Client-server, broker, and distributed structure (§7.5)

- 📘 **MIT 6.031 Software Construction — readings on interfaces, concurrency, and client/server** — web.mit.edu/6.031. Develops the "depend on an interface, swap the implementation" idea that powers test servers (§7.5.2). *Free courseware, Creative Commons licensed.*
- 📄 **Microsoft — Publisher-Subscriber, Gateway Aggregation, and Ambassador patterns** (within the Cloud Design Patterns catalog above; direct link: Publisher-Subscriber) — concrete, documented instances of the broker pattern in modern systems. *Free to read; © Microsoft.*
- 📄 **James Lewis & Martin Fowler — "Microservices"** — martinfowler.com/articles/microservices.html. The essay that defined the microservice style named at the end of §7.5.4. *Free to read; © the authors.*

Product lines (§7.6)

- 📄 **Carnegie Mellon Software Engineering Institute — Software Product Lines** — sei.cmu.edu. Definitions of commonality/variability, the domain- vs. application-engineering split, and product-line economics used in §7.6. *Free to read; © CMU/SEI.*
- 📄 **K. Kang, S. Cohen, J. Hess, W. Novak & A. S. Peterson — "Feature-Oriented Domain Analysis (FODA) Feasibility Study" (CMU/SEI-90-TR-021, 1990)** — sei.cmu.edu. The report that introduced feature models (§7.6.1). *Free to read; © CMU/SEI.*
- 📄 **K. Pohl, G. Böckle & F. van der Linden — *Software Product Line Engineering* (Springer, 2005), publisher's page** — link.springer.com. The standard text behind the domain- vs. application-engineering split of §7.6.2. *Catalog page free to read; the book itself is a paid text.*
- 📄 **SEI — "A Framework for Software Product Line Practice"** — a free online catalog of the practice areas an organization needs to run a product line; a good map of §7.6 at organizational scale. *Free to read;*

© CMU/SEI.

RESTful APIs (§7.5.4)

- 📄 **Roy Fielding — "Architectural Styles and the Design of Network-based Software Architectures," ch. 5** — the doctoral dissertation that defined REST. ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm. *Free to read; © Roy Fielding.*
- 📘 **MDN Web Docs — HTTP methods and status codes** — developer.mozilla.org/docs/Web/HTTP. The practical reference for the verb grammar REST builds on. *CC BY-SA.*
- 📄 **IETF — RFC 9110, *HTTP Semantics*** — rfc-editor.org/rfc/rfc9110. The definitive specification of the HTTP methods in §7.5.4, including which are safe. *Freely available; © IETF Trust.*

Foundational background

- 📄 **M. Shaw & D. Garlan — architectural styles** — the academic origin of thinking about systems as pattern/style choices (pipes-and-filters, layered, repository, client-server). Overview papers and lecture notes are freely available via university course pages; the book record for *Software Architecture: Perspectives on an Emerging Discipline* (1996) is at dl.acm.org. *Free to read; © the authors.*
- 📄 **Trygve Reenskaug — "MVC: Xerox PARC 1978–79"** — folk.uio.no. The inventor's own notes on the origin of Model-View-Controller (§7.3.2). *Free to read; © the author.*
- 📄 **ISO/IEC 7498-1:1994 — the OSI Basic Reference Model** — iso.org. The seven-layer model used as the textbook case of layering in §7.1.1. *Catalog page free to read; the standard text is paywalled.*
- 📘 **Wikipedia — "Architectural pattern"** and linked style articles — en.wikipedia.org/wiki/Architectural_pattern. A reliable, openly-licensed starting map with references. *Text under CC BY-SA.*

License note

Linked resources remain under their own licenses; check each before reuse. This page and all original prose in this chapter are licensed **CC BY-SA 4.0**.

Chapter 8 — Version Control with Git

> **Where we are.** Chapters 1–7 taught you to decide *what* to build and how to structure it: process, requirements, use cases, design, architecture. All of that produces *code* — and the moment code exists, a problem appears that no amount of good design solves on its own. More than one person will change it. Even working alone, you will change it across weeks, take a wrong turn, and need to get back to something that worked. You will want to know who wrote a line and why, try a risky idea without endangering what already runs, and combine your work with a teammate's without either of you losing an afternoon. A **version control system** makes all of that safe and routine, and for essentially the whole industry that system is **Git**. Every discipline in the rest of this book sits on top of it: the code reviews of Chapter 9 happen on Git branches, the delivery pipeline of Chapter 14 runs on every Git commit, and the team project of Appendix A is impossible to coordinate without it.

Git is close to universal. In recent Stack Overflow developer surveys roughly 93–94% of professional developers report using it,[1] and the platforms built on it — GitHub, GitLab, and others — are where modern software is written, reviewed, and shipped. Yet Git also has a reputation for being hard to learn, and the difficulty is partly deserved: its command interface is inconsistent and exposes internal concepts that beginners do not yet understand.[2] This chapter therefore begins with the underlying *mental model*, so that the commands have a coherent explanation. Once you know what Git actually stores, the commands stop being incantations and start being obvious.

> **Principle.** Version control is the substrate of collaboration. You cannot review, integrate continuously, test on every change, or ship safely as a team until every change has a name, a history, and an owner. That is precisely what version control gives you, and it is why this chapter comes before the quality and delivery chapters that depend on it.

8.1 Why Version Control

8.1.1 Life Without It

Picture a shared folder that has grown these files over a week:

```
report.docx        report_final.docx          report_FINAL_jing_use_this.docx
report_v2.docx     report_final_ACTUAL.docx
```

Everyone has lived some version of this. It is the manual, error-prone answer to a real need — keeping old versions around in case you have to go back — and Pro Git calls the copy-into-a-timestamped-folder approach exactly that: *error-prone*, because "it is easy to forget which directory you're in and accidentally write to the wrong file or copy over files you don't mean to."[3]

Now make it code, and make it a team. Two people edit the same file and email copies back and forth; the second person to save wins, and the first person's afternoon is gone. A change three weeks ago introduced a bug, but nobody can say which change, or why it was made, or what the file looked like before. Someone wants to try a bold refactor but is afraid to, because there is no safe way back if it fails. These are not tooling annoyances; they are the daily friction that makes software built by more than one person hard. A **version control system (VCS)** is, in Pro Git's words, "a system that records changes to a file or set of files over time so that you can recall specific versions later."[3] It gives you four things the shared folder cannot: a complete **history** you can travel back through, **attribution** so you can see who changed a line and read why, **isolation** so you can experiment without endangering working code, and **integration** so several people can change the same project at once and combine the results on purpose rather than by luck.

Early systems solved pieces of this. Local tools such as RCS tracked a single developer's file history on one machine. Centralized systems — CVS, then Subversion (SVN), then Perforce — put all the versioned files on one server that clients check out from, which finally let a team collaborate and gave administrators fine-grained control.[4] But a central server is a single point of failure: if it is down, as Pro Git notes, "nobody can collaborate at all or save versioned changes," and if its disk is lost without a backup, "you lose absolutely everything — the entire history of the project."[4] That weakness is what the next generation was built to remove.

8.1.2 What Git Is

Git is a **distributed** version control system. When you `git clone` a project you do not check out the latest files the way you would from a central server; you "fully mirror the repository, including its full history."[4] Every clone is a complete, independent backup. Any developer's copy can restore a dead server, you can commit and browse history on a plane with no network, and there is no single machine everyone must be able to reach to get work done.

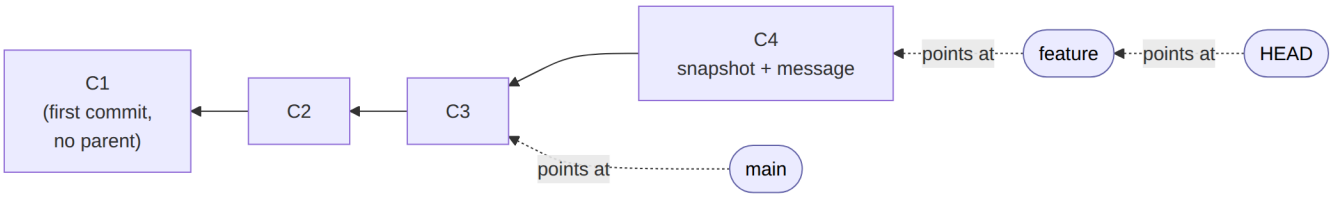
Git was written in a hurry for a serious job. In early 2005 the Linux kernel project lost free use of BitKeeper, the proprietary distributed system it had relied on, after a license dispute.[5] Linus Torvalds started writing a replacement on 3 April 2005, announced it on the 6th, had it hosting its own source the next day, and used it to manage the Linux kernel 2.6.12 release that June; he handed maintenance to Junio Hamano — still the maintainer today — a few months later.[5] Torvalds set four goals: avoid everything he disliked about CVS, support a distributed BitKeeper-style workflow, provide "very strong safeguards against corruption," and be *fast*.[5] (He also, by his own cheerful admission, named it after himself; the manual page calls Git "the stupid content tracker.")

Two design decisions from that origin explain almost everything about how Git behaves. The first is that **Git stores snapshots, not differences.** Older systems store, per file, a list of changes over time — a base version plus a stack of diffs you add up to reconstruct any version. Git instead "thinks of its data more like a series of snapshots": every commit records a picture of *all* your files at that moment, and for any file that did not change, it stores "a link to the previous identical file it has already stored."[6] The second is that **everything is checksummed.** Git names each piece of content by the SHA-1 hash of the content itself — a 40-character fingerprint — and stores it under that name rather than under a filename.[6] You cannot change a file, or a commit, without changing its hash, so corruption cannot hide, and identical content is automatically stored only once. (Git is migrating from SHA-1 to SHA-256 following a 2017 demonstration that SHA-1 collisions are feasible, but the idea is unchanged.)

8.1.3 How to Think About Git: A Graph of Snapshots

Begin with a graph of snapshots; the later commands follow from this representation. Underneath, Git is a key-value store: hand it content, it hashes the content and hands back the fingerprint. Three kinds of object are built on that store.[7] A **blob** holds the contents of one file. A **tree** maps names to blobs and other trees — it is a snapshot of a directory. A **commit** points to one top-level tree (the snapshot of your whole project), carries an author, a timestamp, and a message, and points back to its **parent** commit — the state the project was in just before. The first commit has no parent; a normal commit has one; a merge has two or more.

Follow those parent pointers backward and you are walking the project's history. Because a commit can have several children (two people build on the same starting point) and a merge has several parents, the shape this makes is not a straight line but a **directed acyclic graph (DAG)** of snapshots. "History" in Git is just this graph, read backward.



The arrows between commits run from child to parent: each commit knows where it came from. A **branch** is the last piece, and it is smaller than beginners expect. A branch "is simply a lightweight movable pointer" to one commit — literally a small file containing a 40-character hash.[8] Creating a branch costs about 41 bytes; that is why Git users make branches constantly and older tools, which copied the whole project into a new folder to branch, made people dread it. **HEAD** is one more pointer: it points at the branch you are currently on. When you commit, the commit is created with your current commit as its parent, and the branch HEAD points at slides forward to the new commit. Branches do not "contain" commits and have no built-in parent-child relationship to each other; they are just sticky notes you peel off and re-stick onto newer commits as you work.

> **Pitfall.** The single most common misconception is that *Git stores diffs.* It stores whole snapshots keyed by content hash. Viewing commits as snapshots connected in a graph explains why branches are cheap, how merges and rebases operate, and how Git can recover work that seems lost.[2]

8.2 The Everyday Workflow

8.2.1 Setting Up Once

Before your first commit, tell Git who you are, because that name and email are stamped onto every commit you make:

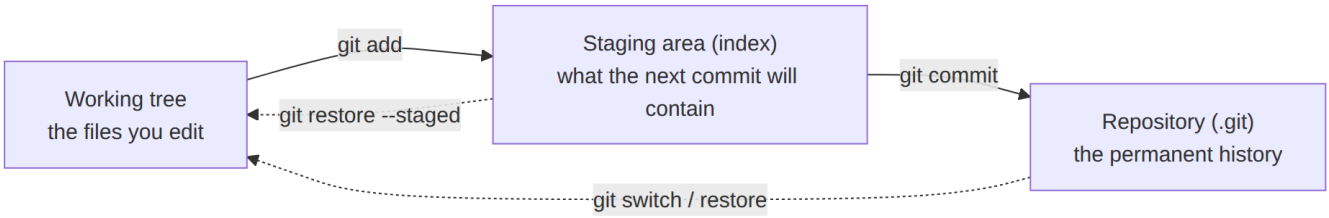
```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

You get a repository in one of two ways. If the project already exists on a server, copy it and its whole history to your machine with `git clone <url>`.[9] If you are starting fresh locally, run `git init` inside your project directory to create the hidden `.git/` subdirectory that holds the object database — then, when you are ready to share it, connect it to a remote with `git remote add origin <url>` and `git push -u origin main`.

A note on the default branch name. Historically Git's first branch was called `master`; modern Git and GitHub create it as `main` instead. Both are ordinary branches with no special powers — wherever this chapter says `main`, use whatever your repository's default branch is called. Finally, pushing to a service like GitHub requires authentication: GitHub removed account-password authentication in 2021, so the two normal paths are HTTPS with a **personal access token** (which you paste in place of a password) or an **SSH key** you generate once and add to your account.

8.2.2 The Three Areas

Every change flows through three places, and understanding them is the thing that most separates people who are comfortable with Git from people who fight it. The **working tree** is the ordinary directory of files you edit. The **repository** (`.git/`) is the committed history. Between them sits the piece with no equivalent in Dropbox or a "Save" button: the **staging area**, also called the **index**, which holds exactly the changes that will go into your next commit.[10]



Why an intermediate step? Because a commit should be one coherent change, and your working tree is usually a mess of several. Staging lets you compose the next commit deliberately — these three files, this part of a fourth — rather than dumping everything you happened to touch. The everyday loop is: **edit** files in the working tree, **stage** the pieces you want with `git add`, and **commit** the staged snapshot to history.

8.2.3 Recording Changes

`git status` is the command you will run more than any other; it shows what is untracked, what is modified but unstaged, and what is staged and ready to commit.[11] `git add <file>` stages a file's *current* content into the index. `git commit` records the staged snapshot as a new commit — either opening an editor for the message or taking it inline with `git commit -m "..."`.

```
git status                          # what changed, and what is staged
git add src/login.py                # stage this file's current content
git add -p                          # stage selected chunks, interactively
git commit -m "Fix redirect loop on expired session (#143)"
git log --oneline --graph           # see history as a labeled graph
git diff                            # unstaged changes (working tree vs. index)
git diff --staged                   # what the next commit will record
```

The staging area is also where beginners get their first genuine surprise, so it is worth seeing once on purpose. If you `git add` a file and then edit it again, the *newer* edits are not in the commit — the commit records what was staged at add-time, not what is in the working tree now. `git status` will helpfully list the file as both "staged" and "modified" to tell you so. This is not a bug; it is the whole point of staging being a separate step. The shortcut `git commit -a -m "..."` auto-stages every already-*tracked* modified file and commits in one move, which is convenient but skips exactly the deliberate selection that makes staging useful — and it silently omits brand-new files, which are not yet tracked.[11]

> **Pitfall.** "`git commit` saved my files" and "`git commit -a` captures everything" are both false in ways that bite. A commit records only what is *staged*, and `-a` stages only files Git is already tracking. Run `git status` before you commit until reading it is second nature.

8.2.4 What Not to Track: `.gitignore`

Not everything in your working directory belongs in history. Compiled output, dependency folders like `node_modules/`, editor scratch files, and — most importantly — secrets such as API keys and passwords should never be committed. You tell Git to ignore them with a `.gitignore` file listing patterns (`*.log`, `build/`, `.env`) at the repository root.[12]

Two facts about `.gitignore` cause real damage when learned too late. First, it only affects files Git is *not already tracking*. Adding a path to `.gitignore` does nothing to a file you already committed; to stop tracking it you must `git rm --cached <file>` and commit that removal. Second, and far more serious: **Git history is forever.** Deleting a secret in a new commit does not remove it — it still sits in every earlier commit, every clone, and every fork. If you commit a credential, the credential is compromised the moment it is pushed; rotate it immediately, and only then scrub it from history with a tool such as `git filter-repo`.[13]

> **Principle.** Create `.gitignore` *first*, before your first `git add .`. It is far cheaper to never commit a secret or a build artifact than to purge one from history after the fact.

8.2.5 Undoing Things Safely

Git's reputation for danger is mostly undeserved, but it has a real basis: a few commands change your working files and a few rewrite history, and beginners reach for the dangerous ones by accident. The commands sort cleanly by *which of the three areas they touch*.[14]

To…	Use	Touches	Safe?
Discard unstaged edits to a file	`git restore <file>`	working tree	destructive (edits gone)
Unstage a file, keep the edits	`git restore --staged <file>`	index	safe
Fix the *last* (unpushed) commit	`git commit --amend`	last commit	safe if not pushed
Undo an old commit with a new one	`git revert <hash>`	adds a commit	safe, shareable
Move a branch back, keep edits	`git reset --soft` / `--mixed`	history + index	recoverable
Move a branch back, discard edits	`git reset --hard <hash>`	all three	destructive

Two ideas make this manageable. **`git revert`** is the collaboration-safe undo: instead of erasing an old commit, it adds a *new* commit that reverses it, so history stays intact for everyone who already has it. And the safety net

beneath all of it is **`git reflog`**, which records every position `HEAD` has held. Because Git "generally only adds data," committed work is almost impossible to truly lose; when you think you have destroyed something, the answer is usually `git reflog` to find the commit's hash and `git reset --hard <hash>` (or `git branch recover <hash>`) to get it back — not deleting the folder and re-cloning in despair.[15]

> **Pitfall.** As a default team policy, never `--amend`, `reset`, or force-push a commit you have already shared. You are not editing your copy; you are rewriting history other people have built on, and their next pull becomes a mess. The rare exceptions — a coordinated `--force-with-lease` on your own feature branch, scrubbing a leaked secret — are announced, not improvised. Rewrite freely *before* you push; treat pushed history as public record.

8.3 Branching and Merging

8.3.1 A Branch Is a Pointer

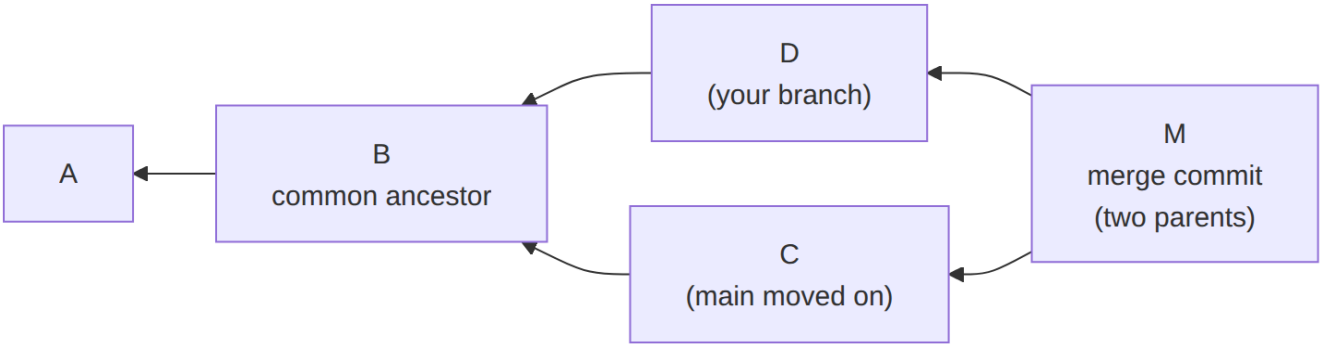
Because a branch is just a movable pointer (§8.1.3), making one is instant and free, and that cheapness is what enables Git's whole style of work: open a branch for every feature or experiment, and keep the shared `main` branch clean. The commands split the two things the old, overloaded `git checkout` used to do — creating/switching branches and discarding file changes — into the clearer modern pair `git switch` and `git restore`, introduced in Git 2.23.[16]

```
git branch                # list branches; * marks the one you are on (HEAD)
git switch -c add-search   # create a new branch and move onto it
# ...edit and commit; the add-search pointer advances, main stays put...
git switch main           # back to main (working tree updates to its snapshot)
```

When you switch, `HEAD` moves to the named branch and your working tree is rewritten to that branch's snapshot; when you commit, the branch you are on advances to the new commit while every other branch stays exactly where it was.

8.3.2 Merging

Eventually you want the work on a branch back in `main`. `git merge` does this, and it takes one of two shapes.[17] If `main` has not moved since you branched, Git can simply slide the `main` pointer forward to your branch's latest commit — a **fast-forward**, no new commit needed. If `main` *has* moved (a teammate merged something meanwhile), Git performs a **three-way merge**: it finds the common ancestor of the two branches, combines the two sets of changes, and records the result as a new **merge commit** with *two* parents — one on each branch.



The merge commit is not clutter; it is an honest record that two lines of development came together here, and it is what lets Git combine work that others based on either side.

8.3.3 Merge or Rebase?

There is a second way to combine branches, and the choice between them is one of the few genuine debates in everyday Git. `git rebase` takes the commits on your branch and *replays* them, one at a time, on top of the latest `main`, as if you had started your work from there.[18] The result is a straight, linear history with no merge commit — which many teams find easier to read. The cost is that rebasing creates *new* commits (same changes, new hashes): it rewrites history. That leads to the nearest thing everyday Git has to an iron rule.

> **Principle — the Golden Rule of Rebasing.** Unless the whole team has coordinated the rewrite, never rebase commits that exist outside your own local repository — that is, commits you have already pushed and others may have.[19] Rebase your *own* unpushed work to tidy it before sharing; use *merge* to integrate work that is already public. Break this rule and you force everyone else into a painful reconciliation of two divergent histories.

A useful default for a student team: rebase your short-lived feature branch onto the latest `main` before you open a pull request (so your changes sit cleanly on top of current work), and let the final integration into `main` be a merge.

8.4 Working with Others

8.4.1 Remotes

A **remote** is another copy of the repository, usually the shared one on GitHub, and by convention it is named `origin`. Four commands move commits between your machine and it.[20] `git clone` copies a remote and its history down for the first time. `git fetch` downloads new commits from the remote but does *not* change your working files — it just updates your view of where the remote's branches are (the `origin/*` "remote-tracking"

branches). `git pull` is `git fetch` followed by a merge (or rebase) of those new commits into your current branch. `git push` uploads your commits to the remote.

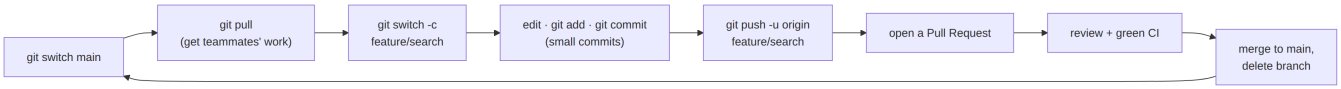
The moment every beginner hits is the **rejected push**:

```
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to '...'
hint: Updates were rejected because the remote contains work that you
hint: do not have locally.
```

This is not an error so much as Git protecting a teammate's work: someone pushed to `main` after you last pulled, and blindly overwriting the remote would discard their commits. The fix is always the same — `git pull` (integrate their work into yours, resolving any conflict), then `git push`. Never reach for `git push --force` to make the message go away on a shared branch; that is exactly the overwrite Git was preventing.

8.4.2 The Daily Team Loop

Put the pieces together and a team's day has a rhythm. Almost every task follows the same loop, and internalizing it as one motion is more valuable than any single command.



1. Start from an up-to-date `main` (`git switch main`, then `git pull`).
2. Branch for the task with a descriptive name (`git switch -c feature/search`).
3. Do the work in small commits; push the branch early (`git push -u origin feature/search`), both to back it up and to let others see it.
4. Open a **pull request**, get it reviewed and past CI, and merge it into `main`.
5. Delete the branch and return to step 1.

8.4.3 Pull Requests and Code Review

A **pull request (PR)** — GitLab calls it a *merge request* — is the ritual that turns "I changed some code" into "the team accepted this change." You push a branch, open a PR proposing to merge it into `main`, request reviewers, and a conversation happens on the diff: reviewers comment, may "request changes," and you push more commits to the same branch until it is approved. The PR is where the code review of Chapter 9 actually takes place, and where the continuous-integration pipeline of Chapter 14 reports whether your change passes the tests and checks.

There are two shapes of PR depending on your access. If you have write access to the shared repository — a normal student team — you push a branch to it directly and open the PR from there (the **branch-and-PR** model). If you do not — the usual case when contributing to someone else's open-source project — you **fork** the

repository to your own account, push your branch there, and open the PR across from your fork to the original (the **fork-and-PR** model).

This is also the answer to a question every team asks: *why can't I just push to `main`?* Because `main` is a **protected branch**. On a well-run project, GitHub is configured to reject direct pushes to `main` and to require that every change arrive through a PR that has an approving review (§9.3) and a passing CI run (§14.2) before it can merge.[21] That single setting is what turns "we should review our code" from a good intention into something the system actually enforces.

8.4.4 Team Workflows and Their Trade-offs

A **workflow** is your team's shared convention for how work leaves a laptop and becomes part of the product. Four are worth knowing, and — this matters — they are *not* equally good for a modern web project.

Trunk-based development keeps one shared mainline (the *trunk*, i.e. `main`) that every developer integrates into at least daily, using only very short-lived branches that live hours to a day or two.[22] Incomplete work is hidden behind **feature flags** rather than kept on a long branch. This is the workflow the research literature associates with elite delivery performance, precisely because frequent integration keeps merges tiny.[23]

GitHub Flow (feature-branch workflow) is the lightly-more-structured version most student teams should use: `main` is always deployable, you branch for each change, open a PR, review, and merge.[24] It keeps the PR-and-review ritual while staying close to trunk-based development if the branches stay short.

Gitflow deserves a specific warning. For years it was *the* answer — a memorable diagram with five kinds of branch (`main`, `develop`, `feature`, `release`, `hotfix`).[25] It suits software that ships in discrete, versioned releases, but it is heavyweight for anything continuously deployed, and its own author added a note in 2020 walking it back for web teams that "deliver continuously."[25] If your instinct is that "the Git workflow" means Gitflow, unlearn that; it is a specialized tool, not the default.

The forking workflow is how open source runs: contributors have no write access, so each forks the canonical repository, works in their fork, and proposes changes by PR.[26]

Workflow	Branches	Best for	Watch out for
Trunk-based	trunk + hours-long	continuously deployed products, strong CI	needs feature flags to hide incomplete work
GitHub Flow	`main` + short feature branches	most teams, including student projects	let branches live too long and merges hurt
Gitflow	`main`, `develop`, +3 kinds	versioned, scheduled releases	overkill and slow for continuous delivery
Forking	one fork per contributor	open source, untrusted contributors	extra step to sync your fork with upstream

> **Pitfall.** Long-lived feature branches are the single most common cause of "merge hell." A branch that diverges from `main` for three weeks accumulates three weeks of conflicts to untangle at the worst possible moment. Whatever workflow you choose, keep branches short and integrate often — this is the most reliable conflict-prevention technique there is.

8.5 Merge Conflicts

8.5.1 What Causes a Conflict

A **merge conflict** is Git admitting it cannot safely combine two changes on its own, so it stops and hands the decision to you. It happens when the two branches being merged changed the *same lines* of the same file in different ways, or when one side edited a file the other deleted.[27] When Git can see that two changes touch different regions, it merges them silently; a conflict is specifically the case where the changes overlap and only a human can say what the combined result should be.

It is important to know what Git does *not* catch. It detects *textual* overlap, not *semantic* incompatibility. If you rename a function on your branch while a teammate adds a new call to the old name on theirs, and the two edits are in different files, Git merges them without complaint — and the build breaks. Conflicts are the visible disagreements; the invisible ones are why review and tests still matter after a clean merge.

8.5.2 Reading the Markers

When a conflict happens, Git pauses the merge and rewrites the conflicted region of the file with markers showing both versions:[27]

```
<<<<<<< HEAD
price = subtotal * 1.08          # our change: 8% tax
=======
price = subtotal * 1.095         # their change: 9.5% tax
>>>>>>> feature/tax-update
```

Read it as three parts. Everything between `<<<<<<< HEAD` and `=======` is **“ours”** — the version on the branch you are currently on. Everything between `=======` and `>>>>>>>` is **“theirs”** — the version coming in from the branch you are merging. The label after `>>>>>>>` names that incoming branch. A one-time configuration, `git config --global merge.conflictstyle zdiff3`, adds a third section showing the common **ancestor** — what the line said *before* either change — which usually makes the right resolution obvious.

> **Pitfall.** “Ours” and “theirs” are relative to what you checked out, and they *flip* between a merge and a rebase (because a rebase replays your commits onto their branch, so “ours” becomes the branch you are landing on). When in doubt, do not guess from the label — read the actual content and the ancestor.

8.5.3 Resolving One, Step by Step

The resolution is less mysterious than the markers make it look:[28]

1. Run `git status` — conflicted files are listed under “Unmerged paths.”
2. Open each one and edit the conflicted region into the single result you actually want — which may be ours, theirs, a blend, or something new. **Delete all three marker lines** (`<<<<<<<`, `=======`, `>>>>>>>`) as you go.
3. Stage the resolved file with `git add <file>` — staging *is* how you tell Git a file is resolved.
4. When every file is staged, finish with `git commit` (Git pre-fills a merge message) or `git merge --continue`.

Most editors make this friendlier. VS Code and other IDEs show a three-way merge view with “Accept Current,” “Accept Incoming,” and “Accept Both” buttons and a live preview of the merged result.[29] They are a real help — but treat “Accept Both” with suspicion: it simply concatenates the two versions, which often produces duplicated or contradictory code that compiles and is still wrong.

And the escape hatch, which is the first thing to learn, not the last: if a merge goes sideways and you want to be back exactly where you started, run **`git merge --abort`** (or `git rebase --abort` during a rebase). Nothing is lost; both sides were preserved the whole time. Knowing this makes conflicts stop feeling dangerous.

> **Pitfall.** `git add` will happily stage a file that still contains `<<<<<<<` markers, and the broken text sails into your commit and then the build. After resolving, search the file for the marker characters before you stage.

8.5.4 Preventing Conflicts on a Team

You cannot eliminate conflicts, but a team can make them rare and small, and the techniques follow directly from *what causes* conflicts — overlapping edits that sit unmerged for a long time.[30]

- **Integrate often.** Pull `main` into your work (or rebase onto it) at least daily. The longer two branches diverge, the more they overlap.
- **Keep branches short-lived and changes small.** A branch open for a day conflicts far less than one open for three weeks; a small pull request (§9.3) touches fewer lines to collide.
- **Divide the work by module.** Good design pays off here directly: if the architecture (Chapters 6–7) gives people separate, cohesive modules to own, they edit different files and rarely collide. Conflict frequency is a quiet signal about coupling.
- **Communicate ownership.** A thirty-second "I'm refactoring the auth module this afternoon" prevents the conflict that a day of silent parallel edits all but guarantees.
- **Agree on formatting.** A shared auto-formatter (§9.1) means diffs contain real changes, not whitespace and reflowing that manufacture conflicts out of nothing.

> **Principle.** The size and pain of a merge scale with how long branches diverge and how much they overlap. Every effective conflict-prevention practice — short branches, frequent integration, modular ownership — is an attack on one of those two variables.

8.6 Habits That Make Git Work for a Team

Everything above is mechanism. What makes Git genuinely *work* for a team is a handful of habits that keep history readable, changes reviewable, and mistakes recoverable.[31]

Commit small, coherent changes. Each commit should do one thing — one fix, one feature step, one refactor — so that history can be read, a bad change can be reverted cleanly, and a bug can be traced to a single commit. A commit that mixes a feature, a reformat, and an unrelated fix is impossible to review or undo in isolation.

Write commit messages for the next reader. The convention that has settled across the industry is a short subject line in the **imperative mood** — "Fix redirect loop," not "Fixed" or "Fixes" — kept under about 50 characters, and, when the change deserves it, a blank line and a body that explains *why* rather than *what* (the diff already shows the what).[32] The imperative reads as an instruction the commit carries out, which is also how Git's own generated messages are phrased. Many teams formalize this with **Conventional Commits** (`feat:`, `fix:`, `docs:` …), a prefix scheme that makes history scannable and can drive automated versioning.[33]

Commit and push often. Small, frequent commits are cheaper to review and safer to integrate; pushing often backs your work up and lets teammates see it. **Never commit secrets or large binaries** (§8.2.4). **Protect `main`**

and require review plus a green pipeline before merge (§8.4.3). Tag releases with annotated tags and a **semantic version** so a specific shipped state has a name you can return to.[34] And keep `git reflog` in mind as the safety net that means an afternoon's work is essentially never truly lost.

8.7 Markdown, READMEs, and Project Documentation

Teams create documentation alongside code. Every repository carries a README, every pull request carries a description, every issue and design note is prose — and almost all of it is written in one lightweight format: **Markdown**. Learning it is a small investment that pays off in every chapter after this one and in every deliverable of the project (Appendix A).

8.7.1 Why Markdown

Markdown is a plain-text writing format — a handful of formatting conventions (created by John Gruber in 2004) that a tool renders into HTML.[35] Its whole appeal for software is that it is *plain text*, so it lives in the repository right beside the code, shows up as a clean line-by-line diff, and is reviewed in a pull request like any other change. The same "docs-as-code" property that version control gives your source, Markdown gives your writing.[36] It is the native format of READMEs, issues, pull requests, wikis, chat, and documentation sites — including this book, which is written in Markdown. The flavor you will actually type is **GitHub Flavored Markdown (GFM)**, which is what GitHub and GitLab render.[37]

8.7.2 The Syntax You Need

You can write almost everything with about a dozen constructs. Here is the working set as you would type it:

```
# Heading   (use ## and ### for sub-headings)

A blank line separates paragraphs. *italic*, **bold**, and `inline code`.

- a bullet list          1. a numbered list
- indent to nest          2. second item

[link text](https://example.com)   ![image alt text](diagram.png)

> a blockquote, useful for notes and warnings

| Column | Column |
| ------ | ------ |
| cell   | cell   |

- [x] a finished task     - [ ] an unfinished task
```

For a block of code, open a line with three backticks and a language name (```` ```python ````) and close with three backticks — that is what produces the highlighted code boxes throughout this book. The one rule beginners trip on is the **blank line**: a list, heading, or code block that is not preceded by a blank line often will not render as one.[38]

8.7.3 The README: Your Project's Front Door

A repository's `README.md` is the first thing GitHub shows a visitor, rendered automatically on the landing page,[39] so it is where a stranger — a teammate, a grader, a future employer — decides in a minute whether they understand your project. For a student team project, a good README answers all of these:

- **Project name** and a one-line description of **what it does and why**.
- **Team members** (and who owns what).
- **Setup instructions** — how to install dependencies and configure the project.
- **How to run it locally**, and **how to run the tests** (Chapter 10).
- The **deployment link** to the live application (Chapter 14).
- **Known limitations** — what does not work yet, honestly stated.
- A link to your **AI-use / provenance log** (Chapter 13, and the individual write-up).

8.7.4 Writing Pull Request Descriptions

The description of a pull request (§8.4.3) is where you explain a change to the teammate who will review it. A blank "updated stuff" tells a reviewer nothing; a short, structured description tells them what to look at and how to check it. A reliable template:

```
## What changed
Added password-reset validation.

## Why
Users need a safe way to recover access without exposing accounts.

## How to test
- Run `npm test`
- Try a reset with an expired token
- Try a reset with mismatched passwords

## Screenshots
(before / after images of the reset screen)
```

Written this way, the description makes the review faster and, months later, explains the change to anyone reading the history.

8.7.5 Markdown as Evidence

There is a reason this section sits inside the version-control chapter. In the individual write-up and the final report (Appendix A), you are asked to show *evidence* of what you did — and that evidence is almost entirely Markdown-and-Git artifacts: your commit messages, your pull request descriptions and review comments, your README, and the issues you opened and closed. Clear commits and well-written pull requests create a durable, timestamped record of who changed the repository, what changed, and why — and that record is what lets you, and your grader, see each member's contribution. Write it as though it is evidence, because it is.

8.8 Conclusion

Git looks like a pile of arbitrary commands until you see the small idea underneath: a graph of content-addressed snapshots, with branches as cheap movable pointers into it. Hold that picture and the everyday workflow — stage a coherent change, commit it, branch to isolate work, merge or rebase to bring it back — becomes routine, and even the scary moments (a wrecked merge, a lost commit, a rejected push) become recoverable and ordinary.

The deeper reason this chapter comes before the ones on quality and delivery is that version control is what makes *team* engineering possible at all. It gives every change a name, a history, and an owner, which is the precondition for reviewing changes (Chapter 9), running a pipeline on every one of them (Chapter 14), and coordinating four people around a single codebase for a semester (Appendix A). Set up your repository, protect `main`, keep your branches short, and commit like someone will read it — because someone will, and often that someone is you.

Sources

[1] Stack Overflow, *Annual Developer Survey* (version-control usage), summarized in *Beyond Git: the other version control systems developers use* (2023). stackoverflow.blog.

[2] Anish Athalye et al., *MIT Missing Semester — Version Control (Git)* ("Git's interface is a leaky abstraction"). missing.csail.mit.edu.

[3] Scott Chacon & Ben Straub, *Pro Git*, §1.1 "About Version Control" (2nd ed.). git-scm.com.

[4] Scott Chacon & Ben Straub, *Pro Git*, §1.1 — local, centralized, and distributed version control. git-scm.com.

[5] Linus Torvalds, *Kernel SCM saga.*. and follow-ups, linux-kernel mailing list (April 6–8, 2005) — the primary record: the announcement, the design goals (a distributed workflow, CVS as an example of what *not* to do, protection against corruption, speed), and the early timeline. lkml.org. See also Scott Chacon and Ben Straub, *Pro Git*, 2nd ed., §1.2 "A Short History of Git" — the Git project's own account of the BitKeeper dispute and Git's goals. git-scm.com.

[6] Scott Chacon & Ben Straub, *Pro Git*, §1.3 "What is Git?" — snapshots not differences, and integrity by checksum. git-scm.com.

[7] Scott Chacon & Ben Straub, *Pro Git*, §10.2 "Git Internals: Git Objects" — blobs, trees, and commits. git-scm.com.

[8] Scott Chacon & Ben Straub, *Pro Git*, §3.1 "Branches in a Nutshell" — a branch as a lightweight movable pointer. git-scm.com.

[9] Scott Chacon & Ben Straub, *Pro Git*, §2.1 "Getting a Git Repository" (`init` vs `clone`). git-scm.com.

[10] Julia Evans, *How Git Works* and *Confusing git terminology* — the working tree, the index/staging area, and building an accurate mental model. wizardzines.com, jvns.ca.

[11] Scott Chacon & Ben Straub, *Pro Git*, §2.2 "Recording Changes to the Repository." git-scm.com.

[12] GitHub Docs, *Ignoring files* (`.gitignore` patterns) and the *gitignore* manual page. docs.github.com.

[13] GitHub Docs, *Removing sensitive data from a repository* — rotate the secret first; history is public. docs.github.com.

[14] Scott Chacon & Ben Straub, *Pro Git*, §2.4 "Undoing Things" and §7.7 "Reset Demystified." git-scm.com.

[15] *git-reflog* manual page — recovering lost commits. git-scm.com.

[16] GitHub Blog, *Highlights from Git 2.23* — the new `git switch` and `git restore` commands. github.blog.

[17] Atlassian, *Git Merge* — fast-forward versus three-way merges. atlassian.com.

[18] Scott Chacon & Ben Straub, *Pro Git*, §3.6 "Rebasing." git-scm.com.

[19] Atlassian, *The Golden Rule of Rebasing*. atlassian.com.

[20] Scott Chacon & Ben Straub, *Pro Git*, §3.5 "Remote Branches" (`fetch`, `pull`, `push`). git-scm.com.

[21] GitHub Docs, *About protected branches* — required reviews and status checks before merge. docs.github.com.

[22] *Trunk-Based Development* — one mainline, very short-lived branches. trunkbaseddevelopment.com.

[23] DORA, *Capabilities: Trunk-based development* — its correlation with elite delivery performance. dora.dev.

[24] GitHub Docs, *GitHub flow*. docs.github.com.

[25] Vincent Driessen, *A successful Git branching model* (Gitflow), including the author's 2020 note of reflection for continuously delivered software. nvie.com.

[26] Atlassian, *Forking Workflow*. atlassian.com.

[27] *git-merge* documentation — how conflicts arise and how they are presented. git-scm.com.

[28] GitHub Docs, *Resolving a merge conflict using the command line*. docs.github.com.

[29] VS Code Docs, *Resolve merge conflicts* — the three-way merge editor. code.visualstudio.com.

[30] Atlassian, *Trunk-Based Development* and *Merge Conflicts* — short-lived branches and frequent integration as conflict prevention. atlassian.com.

[31] Scott Chacon & Ben Straub, *Pro Git*, §5.2 “Contributing to a Project” — commit and collaboration discipline. git-scm.com.

[32] Chris Beams, *How to Write a Git Commit Message* — the seven rules and the imperative-mood convention. cbea.ms.

[33] *Conventional Commits 1.0.0*. conventionalcommits.org.

[34] Tom Preston-Werner et al., *Semantic Versioning 2.0.0*, and *Pro Git* §2.6 “Tagging.” semver.org.

[35] John Gruber, *Markdown* — the plain-text writing format and its readability-first design goal. daringfireball.net.

[36] *Docs as Code* — Write the Docs (documentation written with the same tools as code). writethedocs.org.

[37] *GitHub Flavored Markdown Spec* — the CommonMark superset GitHub and GitLab render. github.github.com/gfm.

[38] *Basic Syntax* — Markdown Guide (headings, lists, code, and the blank-line rule). markdownguide.org.

[39] GitHub Docs, *About READMEs* — the repository’s front door and what it should say. docs.github.com.

- **Key takeaways** are summarized above in §8.8.
- Continue to the Exercises.
- Go deeper with the Open Resources for this chapter.

Chapter 8 — Exercises

Exercises are graded by depth: **[warm-up]** checks understanding, **[analysis]** asks you to reason about how Git behaves and how a team should use it.

Concepts

1. **[warm-up]** In one or two sentences each, define *working tree*, *staging area (index)*, and *repository*, and name the command that moves a change from each one to the next.

2. **[warm-up]** Explain, in your own words, why creating a branch in Git is nearly instantaneous when creating one in an older tool like Subversion was slow. Your answer should use the phrase "a branch is a pointer."

3. **[warm-up]** A classmate says, "Git stores the diffs between versions of my files." Correct the statement and explain what Git actually stores, and why the distinction matters.

4. **[warm-up]** Distinguish `git fetch` from `git pull`. Which one changes the files in your working tree, and which one does not?

5. **[warm-up]** Give one example each of a file that *should* be committed to a repository and a file that should be listed in `.gitignore`, and say why.

6. **[warm-up]** In Markdown (§8.7), write the source for: a top-level heading, a sentence with one **bold** and one *italic* word, a three-item bulleted list, a link, and a fenced code block with a language tag. Explain what the blank-line rule is and give one example where forgetting it breaks the output.

Analysis

7. **[analysis]** You run `git add report.py`, then edit `report.py` again, then `git commit`. Which version of `report.py` is in the commit — the one you staged or the one now in your working tree? Explain using the three-areas model, and say what `git status` would show between the second edit and the commit.

8. **[analysis]** A teammate accidentally ran `git reset --hard` and believes a morning's committed work is gone forever. Explain why it very likely is *not* gone, name the command you would use to find it, and outline the recovery. Then state the one situation in which work genuinely is unrecoverable.

9. **[analysis]** Draw (by hand or in Mermaid) the commit graph for this sequence: commit A on `main`; branch `feature` off A; commit B on `feature`; switch to `main` and commit C; then merge `feature` into `main`. Label the merge commit and its parents, and explain why this is a three-way merge rather than a fast-forward.

10. **[analysis]** You resolve a merge conflict, but you accidentally leave one `=======` marker line in the file, run `git add`, and commit. What happens, and at which later stage (Chapter 9 review, Chapter 14 CI, or production) is this most likely to be caught? What habit would have prevented it?

11. **[analysis]** Your team keeps hitting large, painful merge conflicts near every deadline. Using the causes of conflicts from §8.5, propose three concrete changes to how the team works that would make conflicts smaller and rarer, and justify each in terms of *branch lifetime* or *overlap*.

12. **[analysis]** Explain the Golden Rule of Rebasing and construct a short scenario in which breaking it damages a teammate's repository. Then describe a use of rebase that is perfectly safe, and say what distinguishes the two.

13. **[analysis]** Your instructor requires that `main` be a protected branch. Explain what a protected branch is, list two rules a team might require before a merge is allowed, and connect each rule to a practice from another chapter (code review in §9.3, continuous integration in §14.2).

14. **[analysis]** Draft a `README.md` for your team project that answers every field in §8.7.3 (name and purpose, team, setup, run locally, run tests, deployment link, known limitations, AI-use log link). Then write a pull request description for one real change, using the *What changed / Why / How to test* structure from §8.7.4, and explain why each section helps the reviewer.

Chapter 8 — Open Resources

Free, open-licensed, or freely accessible materials that reinforce this chapter. Types: 📘 open text · 🎓 course · 📄 primary source · 🎥 video. Licenses vary and are noted where known.

Foundations: what Git is and how it works (used in §8.1)

- 📘 **Scott Chacon & Ben Straub — *Pro Git* (2nd ed.)** — git-scm.com/book. The canonical, complete reference and the primary source for most of this chapter: version control (§1.1), the snapshot model (§1.3), Git objects (§10.2), branching and merging (§3), and undoing things (§2.4, §7.7). *License: Creative Commons BY-NC-SA 3.0 — free to read and reuse.*
- 🎓 **MIT "The Missing Semester of Your CS Education" — Version Control (Git)** — missing.csail.mit.edu. Teaches Git data-model-first, the same approach this chapter takes. *License: CC BY-NC-SA 4.0.*
- 📄 **Linus Torvalds — "Kernel SCM saga.." (linux-kernel mailing list, April 2005)** — lkml.org. The primary record of Git's origin: the announcement, the design goals, and the early timeline, in Torvalds' own words. *Access: free archive.*
- 📘 **Chacon & Straub — *Pro Git,* §1.2 "A Short History of Git"** — git-scm.com/book. The Git project's own account of the BitKeeper dispute and how Git came to manage the Linux kernel. *License: CC BY-NC-SA 3.0.*

Building the right mental model (used in §8.1, §8.3)

- 📘 **Julia Evans — *How Git Works* (Wizard Zines)** — wizardzines.com/zines/git. Short, illustrated explanations aimed exactly at the mental-model gap that makes Git confusing; her free blog posts on branches and HEAD are excellent companions. *Access: blog posts free; the zine is a paid PDF.*
- 📄 **De Rosso & Jackson — "What's Wrong with Git?" and the conceptual redesign** — onward13.pdf, oopsla16.pdf. The research behind the claim that Git's difficulty is a conceptual-model problem, not a you problem. *Access: free PDFs.*

Hands-on practice and everyday commands (used in §8.2, §8.4)

- 🎓 **Atlassian Git Tutorials** — atlassian.com/git/tutorials. Clear, example-driven walkthroughs of the everyday commands, merging vs. rebasing, and comparing team workflows. *Access: free.*
- 📄 **GitHub Docs — Git basics, GitHub Flow, and pull requests** — docs.github.com. The mechanics of branches, pull requests, and protected branches. *Access: free.*

- 🎥 **“Oh Shit, Git!?!”** — ohshitgit.com (and the PG-rated dangitgit.com). A recovery cookbook for the specific messes beginners create; keep it open next to your terminal. *Access: free.*

Team workflows and conflicts (used in §8.4, §8.5)

- 📄 **Trunk-Based Development** — trunkbaseddevelopment.com. The reference for short-lived branches and daily integration, with the DORA evidence for why it correlates with elite delivery. *Access: free.*
- 📄 **Martin Fowler — “Patterns for Managing Source Code Branches”** — martinfowler.com. A careful, vocabulary-building survey of branching patterns and their trade-offs. *Access: free.*
- 📄 **Vincent Driessen — “A successful Git branching model” (Gitflow), with the 2020 note** — nvie.com. Read it *including* the author’s own caveat about continuous delivery, discussed in §8.4.4. *Access: free.*

Commit and history discipline (used in §8.6)

- 📄 **Chris Beams — “How to Write a Git Commit Message”** — cbea.ms/git-commit. The seven rules and the imperative-mood convention this chapter recommends. *Access: free.*
- 📄 **Conventional Commits 1.0.0** — conventionalcommits.org and **Semantic Versioning 2.0.0** — semver.org. The prefix scheme and the version grammar that structured histories and releases build on. *Access: free.*

Markdown and documentation (used in §8.7)

- 📄 **Markdown Guide — Basic and Extended Syntax** — markdownguide.org. The most practical reference for the syntax in §8.7, with a printable cheat sheet. *Access: free.*
- 📄 **CommonMark** — commonmark.org and **GitHub Flavored Markdown** — github.github.com/gfm. The standardized spec and the GitHub superset (tables, task lists) you will actually type. *Access: free.*
- 📄 **GitHub — Basic writing and formatting syntax** and **About READMEs** — docs.github.com. How Markdown renders on GitHub, and what a README should contain. *Access: free.*
- 🎓 **Write the Docs — Docs as Code** — writethedocs.org and **Make a README** — makeareadme.com. The philosophy behind keeping docs in the repo, and a template for the project README. *Access: free.*

Chapter 9 — Static Checking

> **Where we are.** Chapter 1 separated *mistakes*, *faults*, and *failures*, and promised that we attack defects at every stage. This chapter is about the family of techniques that find faults *without running the code*: architecture reviews, formal inspections, everyday code review, and automated static analysis. Chapter 10 covers the complementary half — *testing*, which finds defects by running the code. Neither replaces the other, and a mature team leans on both.

There is a persistent myth that the only way to know whether software works is to run it. Running it certainly helps, and Chapter 10 is devoted to doing so well. But a great deal of what is wrong with a program is visible *on the page* — a null that is never checked, a lock acquired but never released, an interface that two teams understand differently, an architecture that quietly assumes a database will never be slow. **Static checking** is the discipline of finding those problems by *examining* the software — its design documents, its source code, its types — rather than by *executing* it. The word "static" is the opposite of "dynamic": nothing is running, no test input is chosen, no clock ticks.

Why bother, when we could just write tests? Three reasons, each of which this chapter develops. First, **static techniques find defects earlier**, often before a single line of code exists — an architecture review can catch a scalability flaw during a design meeting, when fixing it costs an afternoon rather than a rewrite. Second, **static techniques find defects testing cannot easily reach**: a race condition that surfaces once in ten thousand runs, a security hole on an error path no test exercises, a misleading name that will cause a *future* bug. Third, **static techniques transfer knowledge**: when a senior engineer reviews your change, you both walk away understanding the system better, which no automated test can do. The through-line of the chapter is that reviews, inspection, and static analysis are *complementary to testing, not substitutes for it*, and that they operate on a spectrum from slow-and-human to fast-and-automated.

9.1 Architecture Reviews

The earliest and cheapest place to catch a defect is before it is built. An **architecture review** is a structured examination of a system's design — its major components, their responsibilities, and the connections between them (Chapters 6–7) — conducted by people other than the designers. The goal is to *stress* the diagram, not admire it: to ask the questions that the design's authors, being close to their own work, have stopped asking.

Architecture reviews matter because architectural mistakes are usually the most expensive kind. A poorly named variable is a five-minute fix in code review. A decision to route every request through a single synchronous service, or to store money as a floating-point number, or to couple two modules that should have been independent, can require months of rework once the system is built around it. These decisions are made early, are

hard to reverse, and constrain everything downstream — so they deserve a dedicated review before code is written.

9.1.1 Guiding Principles for Architecture Reviews

A few principles keep an architecture review productive rather than performative.

- **Review against concrete scenarios, not in the abstract.** "Is this a good design?" has no answer. "How does this design handle a burst of 10,000 concurrent users?" or "What happens when the payment service is down for thirty seconds?" has an answer you can evaluate. Bring the system's quality requirements — its expected load, its latency budget, its failure modes — and walk the design through each one.
- **The authors should not run the room.** People defend their own decisions instinctively. A review where the designer both presents and judges is a monologue with extra steps. Someone else should facilitate, and the designer's job is to *explain and listen*, not to win.
- **Focus on the risky and the irreversible.** Review time is scarce. Spend it on the decisions that are hard to change and costly if wrong — data models, trust boundaries, concurrency, external dependencies — not on choices that a later refactor can fix cheaply.
- **End with recorded decisions, assigned owners, and follow-up actions.** A review that ends with "good points, we'll think about it" has failed. Every significant concern should end as a recorded item with an owner and a disposition: fix now, accept the risk (and why), or investigate further by a date.

> **Principle.** The purpose of an architecture review is to surface *risks and assumptions* while they are still cheap to change. If your review only confirms what the designers already believed, you scheduled it too late or invited the wrong people.

> **Pitfall.** Reviewing an architecture with only its authors in the room. They share the same blind spots — that is precisely why they didn't catch the problem. Invite an engineer from a *different* team, someone who has operated a similar system in production, and someone who will have to *integrate* with this one.

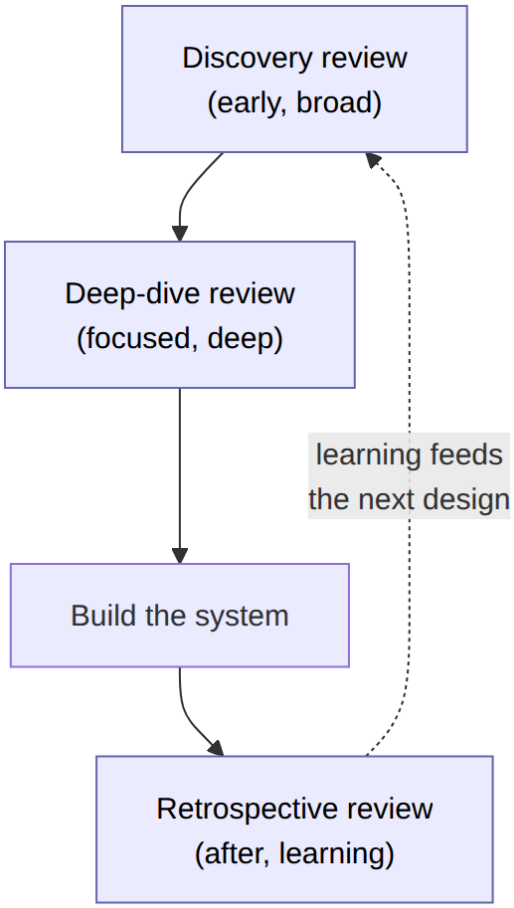
9.1.2 Discovery, Deep-Dive, and Retrospective Reviews

Not all architecture reviews happen at the same time or ask the same questions. It helps to distinguish three moments in a system's life, each with a different purpose.

A **discovery review** happens *early*, when the design is still forming. Its aim is breadth: to map the problem space, name the major components, identify the big risks, and make sure no whole category of requirement (security, privacy, cost, operability) has been forgotten. A discovery review is exploratory and generative — you are trying to find the questions as much as answer them. The right output is a prioritized list of concerns and a set of deeper investigations to schedule.

A **deep-dive review** happens once the shape is settled but a *specific* area carries outsized risk. Suppose the discovery review flagged the data-consistency model as the scariest part of the design. A deep-dive review takes that one area and examines it exhaustively: the exact failure scenarios, the concurrency interleavings, the migration path, the edge cases. Where a discovery review is a mile wide and an inch deep, a deep-dive is the reverse. You bring in specialists — a database expert, a security engineer — and you do not leave until the risk is either resolved or explicitly accepted.

A **retrospective review** happens *after* the system is built and running, and asks a different question entirely: *did the architecture actually deliver what we hoped?* Where did reality diverge from the design? Which assumptions held and which broke? A retrospective review is about *learning* rather than blame, both to guide the next iteration of this system and to make the *team* better at designing. The Therac-25 disaster from Chapter 1 is, among other things, a story of a team that never conducted an honest retrospective on its own assumptions about software reliability.[1]



The three are not mutually exclusive — a long-lived system will have all three, repeatedly. State *which* review you are running at the start: a deep-dive that turns into discovery loses its focus, while a discovery that demands deep-dive rigor exhausts everyone before the major risks are found.

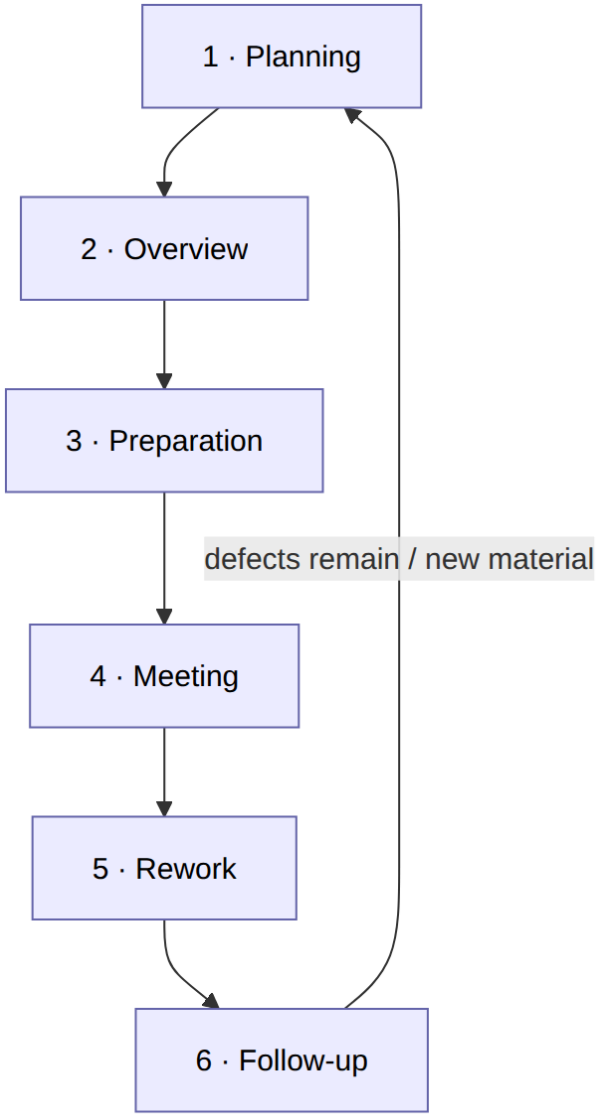
9.2 Conducting Software Inspections

Where an architecture review examines a design, a **software inspection** examines a concrete work product — most often source code, but also a specification, a test plan, or a design document — line by line, with a defined process, for the explicit purpose of finding defects. Inspection is the most formal member of the static-checking family. It is slow and it is disciplined, and decades of data show that, done well, it is startlingly effective: teams routinely find *more than half* of a document's defects in inspection, before any of that code runs.[2]

The formal version was pioneered by Michael Fagan at IBM in the 1970s, and a **Fagan inspection** remains the archetype: a small group, a defined sequence of phases, assigned roles, and — crucially — *measurement* of the process itself.[3] What distinguishes an inspection from someone "taking a look" is that it is repeatable and its effectiveness can be quantified.

9.2.1 The Phases of a Traditional Inspection

A traditional inspection moves through six phases.[4] The discipline is in *not skipping any of them*, because each guards against a specific way that informal review fails.



1. **Planning.** The author and moderator decide *what* will be inspected, confirm it meets entry criteria (it compiles, it is complete enough to review), choose the participants, and schedule the meeting. Planning also means keeping the chunk small — inspecting more than a few hundred lines in one sitting reliably degrades into skimming.[5]

2. **Overview.** The author briefly educates the reviewers on the context: what this code is for, how it fits the larger system, any background they need. This phase is optional when reviewers already know the area, but omitting it when they don't all but guarantees shallow review.

3. **Preparation.** Each reviewer examines the work product *individually and in advance*, noting suspected defects and questions. This is the phase where most defects are actually found.[6] The meeting exists to consolidate what individual, focused reading has already turned up, not to read the code for the first time.

4. **Meeting.** The group convenes. The reader walks through the material (see the roles below), reviewers raise the issues they found in preparation, and the scribe records each confirmed defect. The critical rule: the meeting *finds and records* defects; it does not *solve* them. Debating fixes in the meeting is the classic way inspections run over time and lose their value — solutions are the author's job during rework.

5. **Rework.** The author takes the logged defect list away and fixes each item. Some "defects" turn out to be misunderstandings; the author notes the disposition of every one.

6. **Follow-up.** The moderator verifies that every logged defect was actually addressed — not just that the author *intended* to. If the rework was substantial, or too many defects remain, the material may re-enter the process for another pass.

> **Pitfall.** Turning the inspection meeting into a redesign session. The moment the group starts brainstorming *how to fix* a defect rather than *confirming that it is one*, the meeting balloons, most participants disengage, and you cover a fraction of the material. Log the defect, move on, let the author solve it in rework.

9.2.2 Case Study: Using Data to Ensure Effectiveness

The feature that separates inspection from informal review is that inspection *measures itself*. Because the process is repeatable, you can collect numbers and use them to tell whether the process is healthy — and the numbers are often surprising.

> **Case study.** *(A composite team, with illustrative numbers.)* A team inspecting a mission-critical module tracks two quantities for every inspection: the **preparation rate** (lines of code each reviewer read per hour before the meeting) and the **defect density** (defects found per thousand lines). Over twenty inspections they notice a strong pattern: when reviewers prepare at 500 lines per hour or less, they find roughly 8 defects per thousand lines. When the preparation rate climbs above 1,000 lines per hour — reviewers skimming, under time pressure — the yield collapses to 2 per thousand. The defects did not vanish; the reviewers simply moved too fast to see them. Armed with this data, the team sets a policy: no inspection meeting proceeds unless every reviewer's preparation rate was under 500 lines per hour. Their post-release defect rate on inspected modules drops by half.

The lesson generalizes. A too-high preparation or review rate means reviewers are skimming, and a suspiciously *low* defect count is then bad news, not good — it usually means the review was shallow, not that the code was clean.[5] By recording review rates, defect densities, and where in the lifecycle each defect was found, a team can answer questions that opinion alone cannot: *Are our inspections worth the time? Which kinds of defects slip through? Should we inspect this class of module at all?* Measurement turns inspection from a ritual into a controllable process, and connects directly to Chapter 12's treatment of quality metrics.

> **Principle.** Slow down to speed up. In inspection, a *lower* review rate finds *more* defects, and finding a defect on the page is far cheaper than finding it in production.

9.2.3 Organizing an Inspection

An inspection works because the participants play *distinct roles*, not because more eyeballs are inherently better.[7] Assigning roles prevents the classic failure where everyone assumes someone else is doing the careful reading.

- **Author.** The person who wrote the work product. Their job in the meeting is to *listen*, answer questions of fact, and take defects away for rework — *not* to defend the code or argue it is fine. An author who treats the inspection as a trial to be won destroys it.
- **Reviewer (inspector).** The people whose job is to find defects. They do the essential preparation work individually beforehand and raise issues in the meeting. A good inspection has two to four reviewers with relevant expertise; many more than that and the meeting bogs down without finding proportionally more.[3]
- **Reader.** One reviewer designated to *paraphrase* the material aloud during the meeting — restating what each piece of code does in their own words, rather than reading it verbatim. This is subtly powerful: when the reader's paraphrase diverges from what the author intended, you have found a place where the code does not say what it means, which is a defect (or a clarity problem that will *cause* a future defect).
- **Moderator.** The facilitator who keeps the meeting on process: enforcing the "find, don't fix" rule, watching the pace, ensuring everyone prepared, and confirming follow-up. The moderator is deliberately *not* the author, so that no one is refereeing their own work.
- **Scribe (recorder).** The person who logs each confirmed defect precisely — location, description, severity — so that rework and follow-up have an unambiguous list to work from. A defect that isn't written down didn't happen.

One person can sometimes hold two roles (the moderator often also scribes in a small team), but the *author should never* be the moderator or the reader of their own code — the entire value comes from other people examining it. If your "inspection" is just the author walking everyone through their own reasoning, you have recreated the very problem you were trying to escape.

9.3 Code Reviews: Check Intent and Trust

Full Fagan inspection is heavyweight, and most teams do not inspect every change that way — the cost is justified only for the riskiest material. What virtually every professional team *does* do, on nearly every change, is a lighter-weight **code review**: before a change is merged, at least one engineer other than the author reads it and must approve it. This is one of the most widely practiced static-checking techniques in the industry, and it is the daily face of everything the inspection literature discovered.[8]

Modern code review through pull requests is less about hunting for every defect (automated tools, §9.4, are better at the mechanical ones) and more about two human questions that no tool can settle for you: **intent** and **trust**.[8] *Intent*: does this change do what the author *meant* it to do, and is what they meant actually the right thing? A reviewer who understands the system can see that a change is technically correct but solves the wrong problem, or introduces a subtle inconsistency with how the rest of the codebase works. *Trust*: code review is how a team collectively takes responsibility for its codebase. Once your change is approved and merged, it is not "your" code anymore — it is the team's, and the reviewer who approved it has vouched for it. That shared ownership is the cultural payoff of review, and it is why "the reviewer will catch it" and "I'm just approving to unblock them" are both corrosive.

> **Principle — repository stewardship.** The primary question in a code review is not "is this how *I* would have written it?" but "will this change make the repository *healthier over time*?"[9] Reviewers should identify risks and maintain standards without imposing personal preferences. If a change clearly improves the system, it should be approved even when the reviewer can imagine a marginally better version.
>
> This principle extends beyond the lines in the diff. A healthy change may also add a missing test, clarify nearby documentation, improve a misleading name, remove duplication, or strengthen the automation that verifies the code. The improvement should remain relevant and proportionate to the work at hand. Do not turn a focused feature or defect fix into an unrelated rewrite; when the cleanup is large, risky, or separable, record it and submit it as its own small change.

9.3.1 Invested Expert Reviewers

Not all reviewers are equal, and the difference is not raw skill — it is *investment* and *context*. The most valuable reviewer for a change is usually someone who knows that part of the system deeply and who has a stake in its long-term health: the person who will have to maintain it, who was burned by the last bug in this area, who understands the invariants that aren't written down anywhere. An **invested expert reviewer** brings knowledge that is not in the diff — the history of why the code is shaped this way, the failure that a strange-looking guard clause exists to prevent, the plan for where this module is heading.

This has practical consequences for how teams assign reviews. A change to the authentication system should be reviewed by someone who understands the security model, not by whoever happens to be free. Many teams encode this with **code ownership**: files or directories have designated owners whose approval is required for changes there, precisely to guarantee that an invested expert sees every change to sensitive code. It also means that as a reviewer, if you *don't* have the context to evaluate a change — if you cannot tell whether it is correct — the honest move is to say so and route it to someone who can, not to rubber-stamp it. An approval from someone who didn't understand the change provides trust that isn't real, which is worse than no approval at all.

9.3.2 Reviewing Is Done within Hours

Modern practice also emphasizes **review latency**: teams should usually respond within *hours rather than days*.[10] Fast review supports the throughput of the whole development system, even though a slower review may appear more thorough in isolation.

When a change sits unreviewed for days, several bad things happen. The author, blocked, either switches to other work (incurring the cost of context-switching back later) or, worse, starts building new changes on top of the unreviewed one, so a problem found in review now cascades. The change itself drifts out of date as the codebase moves under it, creating merge conflicts. And the author's own memory of *why* they made each decision fades, so their responses to review comments get vaguer. Fast review keeps the author in flow, keeps changes small and current, and keeps the feedback loop tight enough that the author actually learns from it.

"Within hours" does not mean "instantly and carelessly." The practice that squares speed with quality is **small changes**: a reviewer can give a 50-line change a genuinely careful read in minutes and turn it around the same day, whereas a 2,000-line change cannot be reviewed well *or* quickly — the reviewer skims, approves out of fatigue, and the review becomes theater.[5] Small, frequently reviewed changes are the mechanism that makes fast, thorough review possible at once. This is the everyday, industrialized descendant of the inspection data in §9.2.2: keep the chunk small, keep the reviewer engaged, and the defects surface.

> **Pitfall.** The giant pull request. A change touching forty files with 1,500 lines of diff cannot be reviewed with care — the reviewer's attention is exhausted long before the end, so the last half gets a glance and a thumbs-up. If you want real review, break the work into small, independently reviewable changes. If you *receive* a giant PR, it is legitimate to ask for it to be split.

9.4 Automated Static Analysis

Human review is expensive and imperfect at mechanical checks — people are bad at reliably noticing that *every* code path closes a file handle, but computers are excellent at it. **Automated static analysis** is the use of tools that examine source code (or compiled artifacts) *without running it* and report likely defects, style violations, or dangerous patterns. These tools run in seconds on every change, never get tired, and never skip the boring parts — which is why they complement human review rather than competing with it. Let the machine catch the mechanical faults so the humans can spend their attention on intent and design.

9.4.1 A Variety of Static Checkers

"Static analysis" is an umbrella over several distinct kinds of tools, each answering a different question. It helps to know the categories so you can assemble the right set.

- **Type checkers** verify that operations are applied to compatible kinds of data — that you don't add a number to a string, call a method that doesn't exist, or pass an argument of the wrong type. In statically typed languages (Java, Rust, Go) the compiler *is* a type checker, and it rejects whole classes of defect before the program ever runs. Bolt-on type checkers bring the same guarantees to dynamically typed languages — for example, gradual type checkers for Python and JavaScript-family code. A type error caught at compile time is a failure that can never reach a user. Here a price arrives as a string and flows into arithmetic:

  ```
  def line_total(price: float, quantity: int) -> float:
    return price * quantity

  price = "9.99"                # read from a CSV row, still a string
  total = line_total(price, 3)
  print(total)                  # no crash: prints 9.999.999.99
  # mypy: Argument 1 to "line_total" has incompatible type "str"; expected "float"
  ```

  In each fence, the trailing comment quotes its checker's actual verdict — the same fault, caught before the code ever runs.

- **Linters** flag stylistic issues, suspicious constructs, and small correctness hazards: unused variables, shadowed names, missing `break` in a switch, comparison that is always true. Individually minor, these findings keep a codebase consistent and readable, and they catch the occasional real bug hiding as a typo. Linters are the cheapest static analysis to adopt and usually the first a team turns on.

- **Data-flow analyzers** trace how values move through a program to find defects that span multiple lines: a variable used before it is assigned, a resource opened on one path but not closed on another, a value that can be null reaching a dereference, tainted user input flowing into a database query (a SQL-injection risk). These are the checks humans are worst at, because they require mentally simulating every execution path — exactly what a machine is built to do. This exporter closes its file on the normal path but leaks it on the error path:

```
def export_prices(catalog, discounts, path):
  out = open(path, "w")
  out.write("item,price\n")
  for item, price in sorted(catalog.items()):
    pct = discounts.percent_for(item)
    if pct < 0 or pct > 100:
      return None                 # error path: `out` is never closed
    final = round(price * (1 - pct / 100), 2)
    out.write(f"{item},{final}\n")
  out.close()
  return path
# pylint: R1732: Consider using 'with' for resource-allocating operations
```

The closing comment in each fence shows how the leak surfaces for that language — a linter flagging the fragile acquisition, a path-sensitive analyzer reporting the leaking path itself, or a runtime warning when the abandoned handle is finally collected. The acquisition and the early return are the guilty pair. They sit a few lines apart here and are often a screen or more apart in production code — a human tracing every path by eye misses the one that skips the `close`, while an analyzer walks each path mechanically.

- **Bug-pattern finders** encode a catalog of known-bad code shapes and scan for them: ignoring a method's return value that must be checked, comparing strings with reference equality, integer overflow in a size calculation, a lock acquired without a matching release. Each pattern is a mistake that has bitten enough programmers that someone wrote a detector for it. Security-focused variants (often called SAST tools) specialize in patterns that lead to vulnerabilities — the security applications of static analysis get their own treatment, alongside dynamic and AI-driven testing, in Chapter 11.

Most teams run several of these together in **continuous integration** (Chapters 2 and 14), so that every proposed change is automatically type-checked, linted, and pattern-scanned before a human reviewer ever looks at it. The tools handle the mechanical layer; the humans handle judgment. That division of labor is the whole point.

9.4.2 False Positives and False Negatives

Static analyzers are not oracles. Because they reason about *all possible* executions without running any of them, they must approximate, and approximation produces two kinds of error you must understand to use these tools well.

A **false positive** is when the tool reports a defect that *is not actually a defect* — it cries wolf. The code is fine, but the analyzer, unable to prove it is fine, flags it anyway. A **false negative** is the opposite and more dangerous case: a *real* defect that the tool *fails to report* — the wolf slips past. The code is genuinely broken, but the analysis missed it, so you get false reassurance.

It is worth laying these out against the truth, in the same confusion-matrix form you will see again for testing and classification:

	Tool reports a defect	**Tool reports nothing**
Defect really exists	True positive (correct alarm)	**False negative** (missed defect)
No defect exists	**False positive** (false alarm)	True negative (correctly quiet)

The two error types trade off against each other, and this trade-off has names. **Precision** is the fraction of the tool's alarms that are real: of everything it flagged, how much was an actual defect? Low precision means many false positives — lots of noise. **Recall** is the fraction of real defects the tool catches: of everything actually wrong, how much did it find? Low recall means many false negatives — lots of misses. You generally cannot maximize both at once. Make a tool more aggressive so it catches more real bugs (higher recall) and it will also flag more innocent code (lower precision). Tune it to only report what it is sure about (higher precision) and it will stay silent about defects it can't prove (lower recall).

Why does this trade-off decide whether a tool succeeds in practice? Because **false positives have a human cost that compounds**. If a tool floods reviewers with false alarms, people stop reading its output — and once ignored, it also misses the *real* defects buried in the noise. A checker with 90% recall is worthless if its 60% false-positive rate trains the whole team to click "dismiss" reflexively. This is why practical static-analysis tools often deliberately sacrifice recall for precision: a tool that reports *fewer* defects but is *right* almost every time earns the team's trust and gets acted upon, while a "thorough" tool that is frequently wrong gets turned off. Google's experience with static analysis is explicit about this — they target a false-positive rate low enough (roughly one in ten or better) that developers treat warnings as worth fixing rather than as noise.[11]

> **Principle.** For a static analyzer, *trust is the scarce resource*. A tool the team trusts gets acted upon; a tool the team distrusts gets suppressed, and then even its true findings are lost. High precision buys trust; chasing recall at the cost of precision spends it.

> **Pitfall.** Treating a clean static-analysis report as proof of correctness. False negatives are real: the analyzer only finds the *categories* of defect it was built to find, and says nothing about whether the code does the *right thing*. "The linter is happy" is a floor, not a ceiling — it is why static analysis complements, and never replaces, review and testing.

9.5 Conclusion

Every technique in this chapter finds defects *without running the code*, and each occupies a different point on a spectrum from slow-and-human to fast-and-automated. **Architecture reviews** catch the most expensive defects

earliest, while the design is still cheap to change. **Inspections** apply disciplined, measured, role-based scrutiny to the riskiest work products and reliably find a large share of their defects before release. **Code review** is the everyday, industrialized version — fast, lightweight, focused on intent and shared ownership, and effective in direct proportion to how small the changes are. **Automated static analysis** handles the mechanical checks tirelessly and in seconds, freeing human attention for judgment, as long as we manage its false positives so the team keeps trusting it.

These techniques are complementary to each other and, above all, complementary to *testing* (Chapter 10). Static checking excels at finding defects that are visible on the page, that lurk on rarely executed paths, and that concern design and intent — but it cannot tell you whether the assembled system actually behaves correctly when it runs. Testing can, but only for the inputs you think to try, and only after the code exists. The mistake → fault → failure chain from Chapter 1 is best broken at every link at once: reviews and inspection reduce the faults that reach the codebase, static analysis catches whole categories automatically, and testing provokes the failures that slip through. No single net catches everything, which is exactly why we cast several.

Sources

1 Nancy G. Leveson and Clark S. Turner, *An Investigation of the Therac-25 Accidents*, IEEE Computer 26(7) (1993). ieeexplore.ieee.org.

2 Barry Boehm and Victor R. Basili, *Software Defect Reduction Top 10 List*, IEEE Computer 34(1) (2001). cs.umd.edu.

3 Michael E. Fagan, *Design and Code Inspections to Reduce Errors in Program Development*, IBM Systems Journal 15(3) (1976). dl.acm.org.

4 Michael E. Fagan, *Advances in Software Inspections*, IEEE Transactions on Software Engineering SE-12(7) (1986). research.ibm.com.

5 SmartBear Software, *Best Practices for Peer Code Review* — findings from a study of a Cisco Systems team (2006). smartbear.com.

6 Lawrence G. Votta, *Does Every Inspection Need a Meeting?*, ACM SIGSOFT Symposium on Foundations of Software Engineering (1993). dl.acm.org.

7 NASA, *Software Formal Inspections Standard*, NASA-STD-8739.9 (2013). standards.nasa.gov.

8 Alberto Bacchelli and Christian Bird, *Expectations, Outcomes, and Challenges of Modern Code Review*, ICSE (2013). microsoft.com.

9 Google, *The Standard of Code Review*, Engineering Practices documentation. google.github.io.

10 Google, *Speed of Code Reviews*, Engineering Practices documentation. google.github.io.

11 Caitlin Sadowski, Edward Aftandilian, Alex Eagle, Liam Miller-Cushon, and Ciera Jaspan, *Lessons from Building Static Analysis Tools at Google*, Communications of the ACM 61(4) (2018). cacm.acm.org.

- **Key takeaways** are summarized above in §9.5.
- Continue to the Exercises.
- Go deeper with the Open Resources for this chapter.

Chapter 9 — Exercises

Exercises are graded by depth: **[warm-up]** checks understanding, **[analysis]** asks you to reason.

Concepts

1. **[warm-up]** In one or two sentences each, explain what makes a technique *static* rather than *dynamic*, and give one example of each from this book. Then state the single biggest thing static checking *cannot* tell you that testing (Chapter 10) can.

2. **[warm-up]** Name the six phases of a traditional inspection in order, and give a one-sentence purpose for each. Which phase finds most of the defects, and why is it *not* the meeting?

3. **[warm-up]** List the five inspection roles (author, reviewer, reader, moderator, scribe) and, for each, state the one thing the inspection breaks down without. Explain why the author must never also be the moderator or reader of their own code.

4. **[warm-up]** Define *false positive* and *false negative* for a static analyzer in your own words, and give a concrete one-line example of each. Which one gives you *false reassurance*, and why is that the more dangerous error?

5. **[warm-up]** Distinguish a *discovery* review from a *deep-dive* review from a *retrospective* review by when each happens and what question it answers.

Analysis

6. **[analysis]** A teammate proposes replacing your team's code reviews entirely with an aggressive static analyzer, arguing "the tool never gets tired and never misses a null check." Using the ideas of *intent*, *trust*, and *false negatives*, write a ~150-word response explaining what the tool would catch, what it would miss, and why review and analysis are complementary rather than substitutes.

7. **[analysis]** Your team's static analyzer has 95% recall but a 55% false-positive rate; developers have started dismissing its warnings without reading them. Explain, using precision/recall, why the *high recall* is now worthless in practice, and describe two concrete changes you would make to restore the team's trust in the tool. Would you accept *lower recall* to get there? Justify it.

8. **[analysis]** Fill in and interpret this confusion matrix. A checker was run on a module known (from later production incidents plus a hand audit) to contain 20 real defects of the kind the checker targets. The checker raised 30 alarms; 12 of them corresponded to real defects. Compute the true positives, false

positives, and false negatives, then compute the tool's precision and recall. In one sentence, say whether you'd trust this tool and why.

9. **[analysis] Perform a code review.** Read the function below — the defects are real, and you can point a linter or type checker at it as well as your own eyes. List every defect and concern you would raise as a reviewer, and for each, label whether it is about *correctness*, *intent/design*, or *style/clarity*. Then state which one or two of these an ordinary automated linter or type checker would likely have caught on its own, and which require a *human* reviewer who understands intent.

```
# Applies a discount code to a shopping cart and returns the new total.
def apply_discount(cart, code):
  total = 0
  for item in cart.items:
    total = total + item.price * item.quantity
  discount = discounts.lookup(code)        # returns None if code is unknown
  total = total - total * discount.percent / 100
  if total < 0:
    total == 0
  cart.total = total
  return total
```

10. **[analysis]** In an inspection meeting, a reviewer starts sketching, on the whiteboard, a better algorithm to replace the one under inspection, and a fifteen-minute design debate breaks out. As the moderator, what rule is being violated, what do you say, and where does that redesign work properly belong? Explain *why* the meeting protects this boundary.

11. **[analysis]** The §9.2.2 case study found that raising the preparation rate above ~1,000 lines/hour cut defect yield from ~8 to ~2 per thousand lines. A manager reads this as "great news — the faster reviews find fewer problems, so the code must be getting cleaner." Explain why this interpretation is exactly backwards, and describe the one metric you would add to the team's dashboard to expose the real story.

Chapter 9 — Open Resources

Free, open-licensed, or freely accessible materials that reinforce this chapter. Types: 📘 open text · 🎓 course · 📄 primary source · 🎥 video. Licenses vary and are noted where known.

Code review (used in §9.3)

- 📘 **Google Engineering Practices — "How to do a code review"** — google.github.io/eng-practices/review. Google's public guide for both reviewers and authors: what to look for, how fast to review, how to handle disagreement, and why "will this improve the codebase's health over time?" is the central question. *Licensed CC BY 3.0 — freely reusable with attribution.*
- 📘 **MIT 6.031 *Software Construction* — "Code Review" reading** — web.mit.edu/6.031. A concrete, example-driven reading on reviewing for correctness and clarity, with a catalog of common smells. *Course materials offered openly by MIT OpenCourseWare (CC BY-NC-SA).*
- 📘 **"Software Engineering at Google," Ch. 9 "Code Review"** — abseil.io/resources/swe-book. Why Google reviews nearly every change, the roles of reviewers, the value of small changes, and review latency. *Free to read online; book text CC BY-NC-ND.*
- 📄 **A. Bacchelli & C. Bird, "Expectations, Outcomes, and Challenges of Modern Code Review," *ICSE*, 2013** — microsoft.com/research. The Microsoft study behind §9.3: finding defects is the top *stated* motivation for review, but the realized value leans toward knowledge transfer, team awareness, and better solutions. *PDF freely available from Microsoft Research.*
- 📘 **SmartBear, "Best Practices for Peer Code Review"** — smartbear.com. Findings from a SmartBear study of a Cisco Systems team: review no more than 200–400 lines at a time, at under ~500 lines per hour — the data behind this chapter's small-change and review-rate advice. *Freely accessible vendor guide.*

Inspection (used in §9.2)

- 📄 **M. E. Fagan, "Design and Code Inspections to Reduce Errors in Program Development," *IBM Systems Journal* 15(3), 1976** — dl.acm.org. The founding paper on formal inspection, defining the phases, roles, and the discipline of measuring the process. *Official record paywalled; PDF widely available from IBM and university course pages — search "Fagan Design and Code Inspections 1976 PDF."*
- 📄 **M. E. Fagan, "Advances in Software Inspections," *IEEE Transactions on Software Engineering* SE-12(7), 1986** — research.ibm.com. Fagan's follow-up a decade later: the six-operation process (planning, overview, preparation, inspection, rework, follow-up) and reported quality results. *Abstract page free; full text via IEEE or course-page PDFs.*
- 📄 **B. Boehm & V. R. Basili, "Software Defect Reduction Top 10 List," *IEEE Computer* 34(1), 2001** — cs.umd.edu. A compact survey of defect-reduction evidence, including the finding that peer reviews catch

about 60% of defects — the basis for §9.2's "more than half." *Free author-hosted PDF.*

- 📄 **L. G. Votta, "Does Every Inspection Need a Meeting?," *ACM SIGSOFT FSE*, 1993** — dl.acm.org. The empirical study showing that individual preparation, not the meeting, finds most inspection defects — the source for §9.2.1's claim about the preparation phase. *Abstract free; full text paywalled.*
- 📄 **NASA Software Formal Inspections Standard (NASA-STD-8739.9)** — standards.nasa.gov. A public, practical standard for running inspections, including participant roles, entry/exit criteria, and data collection. *U.S. government work — public domain.*

Automated static analysis (used in §9.4)

- 📘 **SonarQube / SonarSource documentation** — docs.sonarsource.com. Docs for a widely used static-analysis platform: rule categories (bugs, vulnerabilities, code smells), quality gates, and how findings are surfaced in CI. *Docs free to read; the tool has open-source and commercial editions.*
- 📄 **"Lessons from Building Static Analysis Tools at Google," *CACM* 61(4), 2018** — cacm.acm.org. The primary source behind §9.4.2's argument that *false-positive rate*, not raw recall, decides whether developers act on a tool. Explains their target of keeping false positives low enough that warnings get fixed.
- 📘 **Clang Static Analyzer — user documentation** — clang-analyzer.llvm.org. Openly documented data-flow / bug-pattern analyzer for C/C++/Objective-C, useful for seeing what path-sensitive analysis actually reports. *Open source (Apache-2.0 with LLVM exceptions).*

Learning from failure (used in §9.1)

- 📄 **N. G. Leveson & C. S. Turner, "An Investigation of the Therac-25 Accidents," *IEEE Computer* 26(7), 1993** — ieeexplore.ieee.org. The definitive investigation of the Therac-25 overdoses, including the manufacturer's unexamined assumptions about software reliability — the source behind §9.1.2's retrospective-review point. *Official record paywalled; an updated version is freely available from the author at sunnyday.mit.edu.*

License note

Linked resources remain under their own licenses (noted above where known); this page is CC BY-SA 4.0.

Chapter 10 — Testing

> **Where we are.** Chapter 9 attacked defects *before* the program runs, with static checking, types, and review. This chapter attacks them by *running* the program on chosen inputs and comparing what it does against what it should do. Testing cannot prove a program correct, but it is the single most practical net for catching the defects that reviews miss — and the ones a future change will introduce. The central skill is deciding *which* tests to write and knowing when you have written *enough*.

You have already met testing in miniature: run the code, look at the output, decide if it is right. The trouble is that a real program has more possible inputs than there are atoms in the universe, so "run it and look" does not scale and does not tell you when to stop. This chapter turns testing from an art into an engineering activity with **criteria**: systematic ways to select inputs, to measure how thoroughly you have exercised the code and its specification, and to justify — to yourself, your team, and sometimes a regulator — the claim that the software has been tested well enough to ship.

10.1 Overview of Testing

At its simplest, a **test** records three things: an *input* or setup, an *expected property or result*, and the *actual behavior* observed when you run it. The test **passes** when the observed behavior matches what was expected and **fails** otherwise. A **test suite** is a collection of tests run together. This sounds trivial, and for one test it is; the engineering lives in the four questions the rest of this section raises.

> **Principle.** Testing can reveal the *presence* of defects but never their *absence*.[1] A passing suite means "no defect was triggered by these inputs," not "no defect exists." Everything in this chapter is about making that first statement as strong as it can practically be.

10.1.1 Issues during Testing

Four hard problems recur every time you test anything. Name them now, because the rest of the chapter is a set of answers to them.

- **Selection.** The input space is astronomically large. Which handful of inputs should you actually run? (§10.1.2, and all of §§10.3–10.6.) And once a suite exists: which of *its* tests does this change need to run? (§10.1.6.)
- **Adequacy.** Having run some tests, how do you know whether you have run *enough*? What is the stopping rule? (§10.1.3.)

- **Oracle.** For each input, how do you decide whether the observed output is *correct*? Sometimes that is harder than producing the output. (§10.1.4.)
- **Automation and repeatability.** A test you cannot re-run cheaply is nearly worthless, because its main long-term value is catching *regressions* — old defects reintroduced by new changes. Manual "click around and see" testing fails this test.

Keep these four in view. A criticism like "your tests are weak" almost always resolves into one of them: you picked the wrong inputs, you stopped too early, your oracle was too lax, or your tests do not run automatically.

10.1.2 Test Selection

Because you cannot try every input, you must *sample* the input space — and a random or lazy sample usually misses the inputs most likely to break. Good selection is **criterion-driven**: you choose a rule that partitions the space of possibilities and then pick inputs to satisfy the rule. Two great families of criteria run through this chapter, and they look at the program from opposite sides:

- **Black-box (specification-based)** selection reads the *spec* and ignores the code. You pick inputs that represent distinct required behaviors — valid and invalid classes, boundaries, combinations (§10.4, §10.6). Its virtue: it finds *missing* code and survives a rewrite.
- **White-box (structure-based)** selection reads the *code* and picks inputs that exercise its statements, branches, and paths (§10.3, §10.5). Its virtue: it finds code the spec-based tests never reached, and it *measures* thoroughness precisely.

Neither subsumes the other. A black-box suite can achieve 100% of every code-coverage metric and still miss a feature that was never implemented (there is no code to cover). A white-box suite can cover every branch of the code that *exists* and never notice the branch that *should* exist. Mature teams use both and measure with both.

10.1.3 Test Adequacy: Deciding When to Stop

"We tested it" is not a stopping rule; "we covered every branch, every equivalence class, and every boundary, and all tests pass" is. A **test-adequacy criterion** (also called a **coverage criterion**) is a checkable condition that tells you whether a suite is done *with respect to that criterion*. It converts the vague question "have we tested enough?" into a countable one.

Most criteria have the shape: *there is a set of things to cover, and the suite is adequate when it covers all of them.* The "things" might be statements, branches, paths, equivalence classes, boundaries, or condition-outcome pairs. Coverage is then the fraction covered:

$$\text{coverage} = \frac{\text{items exercised by the suite}}{\text{total items required by the criterion}}$$

Coverage is a *floor, not a ceiling*: 100% branch coverage means every branch ran, not that every branch is correct. But low coverage is a genuine alarm — code that never ran under any test is code you know nothing about empirically.

> **Pitfall.** Do not manage to a coverage number for its own sake. A team told "hit 90%" can write assertion-free tests that *execute* code without *checking* it, gaming the metric while learning nothing. Coverage is a diagnostic, not a goal; pair it always with meaningful oracles (§10.1.4).

10.1.4 Test Oracles: Evaluating the Response to a Test

An **oracle** is whatever decides, for a given input, whether the actual output is correct. In a textbook example the oracle is obvious — you write `assert add(2, 3) == 5`, and `5` is the oracle. In real systems it is often the hardest part of testing, a difficulty known as the **oracle problem**.[2] If you are testing a function that renders a map, or ranks search results, or predicts weather, what exactly should you compare against?

Several oracle strategies, from strongest to weakest:

- **Exact expected value.** You know the right answer and hard-code it: `assert reverse("abc") == "cba"`. Strongest, but only available when you can compute the answer independently.
- **Property / invariant oracle.** You cannot name the exact answer but you know a property it must satisfy. For a sort: the output is a permutation of the input and is non-decreasing. For encryption: `decrypt(encrypt(x)) == x` (a *round-trip* oracle). These power **property-based testing**, where a tool generates many random inputs and checks the property on each.
- **Cross-check / differential oracle.** Compare against an independent implementation — a slow reference version, a previous release, or a competitor. If they disagree, at least one is wrong.
- **Human judgment.** A person inspects the output. Necessary for look-and-feel, but slow, non-repeatable, and the enemy of automation — use it sparingly.

Consider a function `median(xs)`. An exact oracle requires you to know the median of each test list. A cheaper property oracle: after sorting, the result equals the middle element; and `median(xs) == median(reversed(xs))` (order independence). Property oracles let you throw thousands of generated inputs at the function without hand-computing each answer — often the difference between a suite that finds real defects and one that only confirms the three cases you already thought of.

With a property-based testing tool — every ecosystem has one — both properties become one test run against hundreds of generated lists:

```
# Hypothesis generates the inputs: pip install hypothesis
from hypothesis import given, strategies as st

def median(xs):
  return sorted(xs)[len(xs) // 2]          # middle element (upper median)

@given(st.lists(st.integers(), min_size=1))
def test_median_properties(xs):
  m = median(xs)
  assert m == median(list(reversed(xs)))   # order independence
  assert m in xs                           # the median is one of the inputs
```

10.1.5 Mutation Testing: Grading Your Suite

Coverage tells you what your tests *ran*; it cannot tell you what they would *catch*. The pitfall in §10.1.3 — assertion-free tests that execute everything and check nothing — scores 100% on any coverage criterion while detecting no defects at all. **Mutation testing** closes that gap by testing the tests.[3]

> **Principle — never trust a test you have never seen fail.** A passing test is evidence only if you know the test can detect the defect it claims to guard against. Before you trust a new test, temporarily remove or break the behavior under test and confirm the test turns red *for the reason you expected*; then restore the behavior and watch it return to green.[4] Seeing a test fail once does not prove it catches every defect, but *never* seeing it fail leaves a more basic question unanswered — does the test test anything? An assertion may never execute, the runner may discover no tests, a mock may intercept the wrong boundary, or the expected value may reproduce the very mistake the implementation makes. Test-driven development (§2.3.2) runs this check at birth — red first, then green; mutation testing applies the same idea to an *established* suite by inserting defects and asking whether the tests notice.

A mutation tool takes your working code and plants small, deliberate defects called **mutants**: flip a `<` to `<=`, change a constant by one, negate a condition, delete a statement. Each mutant is a plausible bug — exactly the kind a tired programmer writes. The tool then reruns your suite against each mutant in turn:

- If at least one test **fails**, the suite noticed the defect: the mutant is **killed**.
- If every test still **passes**, the mutant is a **survivor** — your suite executed the broken line and did not notice.

The **mutation score** — mutants killed divided by mutants generated — grades your suite's defect-detecting strength in a way coverage never can. Better, nearly every survivor is *actionable*: it points at a specific line where an assertion is weak or missing.

Watch it work on the discount function, the worked example you will meet in §10.2.1. Suppose the tool mutates the guard `price < 0` into `price <= 0`, so a price of exactly zero is now (wrongly) rejected. Rerun the suite: no test ever passes a zero price, so every test still passes and the mutant **survives**. The survivor names the missing test precisely:

```
def test_free_item_allowed():
  assert apply_discount(0.0, 50) == 0.0    # kills the `price <= 0` mutant
```

Add it, rerun, and the mutant dies — and, not coincidentally, you have just written the boundary-value test (§10.4.2) your suite was missing. Mature tools exist in every ecosystem: **go-mutesting** for Go, **PIT** for Java, **Stryker** for JavaScript and TypeScript, **mutmut** for Python, **mutant** for Ruby.

> **Pitfall — mutation testing is expensive.** Each mutant means rerunning the suite, and a modest module can generate hundreds of mutants, so a full run can take hours. Do not put it on every commit. Use it selectively — on critical modules, on changed code, or in a periodic job — and treat the survivors it reports as a curated to-do list for your suite, not a number to chase.

10.1.6 Selecting from the Suite: Regression Selection and Prioritization

Everything so far selects *inputs* — it builds the suite. A second selection problem appears once the suite exists and keeps growing: on every change, *which of the tests you already have should run, and in what order?* Early in a project the answer is "all of them, every time," and you should keep that answer for as long as you can. But suites grow with their codebase, and eventually the full run no longer fits the feedback loop it exists to power — one industrial team reported a suite that took seven weeks to run in full.[5] The moment there is a budget, three techniques share the work, and the vocabulary is worth knowing:

- **Minimization** permanently deletes tests that no longer earn their runtime: redundant (everything they exercise, another test also exercises) or obsolete (the behavior they check no longer exists).
- **Selection** picks, per change, the subset of tests relevant to what changed. A CSS change does not need the payroll calculation suite — *if* you can establish which tests a change can possibly affect.
- **Prioritization** keeps every test but orders the run so likely failures come first. Feedback arrives when the first test *fails*, not when the last one passes; researchers score an ordering by its rate of fault detection (APFD).[5]

The subtle point is that a time budget changes what "best" means. Suppose your merge gate has a ten-minute budget (§14.2.4), and history gives you a value estimate for each test — say, the distinct regressions each one has caught over the past year:

Test	Regressions caught	Runtime
`T1`	9	9 min
`T2`	4	3 min
`T3`	4	4 min
`T4`	3	3 min

The obvious greedy rule — most valuable test first — spends nine of the ten minutes on `T1`, for nine regressions' worth of protection. Selecting *against the budget* runs `T2`, `T3`, and `T4` instead: eleven regressions' worth in the same ten minutes. This is the **0/1 knapsack problem** — each test an item with a weight (its runtime) and a value; maximize value within the budget — which is NP-complete, and the real problem is harder still, because test values *overlap*: two tests that exercise the same code catch the same defects, so a selection's value is less than the sum of its parts. Practical approaches therefore lean on heuristics and search — knapsack approximations and genetic algorithms have both been studied[6] — and large CI systems apply the same idea as **predictive test selection**, weighting each test by its failure history and its relevance to the changed files.

> **Principle.** A test budget is a constraint to design against, not permission to skip testing quietly. Decide explicitly what runs at the merge gate and what runs continuously behind it (§14.2.4), and never let "the suite is slow" drift into "we don't run the suite."

10.2 Levels of Testing

Testing happens at several *levels*, distinguished by how much of the system is under test at once. Each level answers a different question and catches a different class of defect, and the levels are complementary, not redundant.

10.2.1 Unit Testing

A **unit test** exercises the smallest independently testable piece — typically one function, method, or class — *in isolation* from the rest of the system. Its dependencies are replaced by **test doubles**: a **stub** returns canned answers, a **mock** additionally records how it was called and lets you assert on those calls, and a **fake** is a lightweight working implementation (an in-memory database standing in for the real one).

Each double below stands in for the `discounts` dependency of `PriceService` (§10.2.2) — the stub feeds the unit a canned percent and nothing more:

```
class StubCatalog:
  def price_of(self, item): return 12.0        # every item costs 12.0

class StubDiscounts:
  def percent_for(self, item): return 25       # canned answer, whatever the item

svc = PriceService(StubCatalog(), StubDiscounts())
assert svc.quote("mug") == 9.0                 # 12.0 * (1 - 0.25)
```

The mock also records the interaction, so the test can assert on *how* the unit used it:

```
class MockDiscounts:
  def __init__(self): self.calls = []
  def percent_for(self, item):
    self.calls.append(item)
    return 25

mock = MockDiscounts()
PriceService(StubCatalog(), mock).quote("mug")
assert mock.calls == ["mug"]                   # called exactly once, with "mug"
```

And the fake actually works — an in-memory table stands in for the discount database, so any lookup behaves the way the real component would:

```
class FakeDiscounts:
  def __init__(self, table): self.table = dict(table)      # in-memory stand-in
  def percent_for(self, item): return self.table.get(item, 0)

svc = PriceService(StubCatalog(), FakeDiscounts({"mug": 25}))
assert svc.quote("mug") == 9.0
assert svc.quote("bowl") == 12.0               # unknown item: no discount
```

Unit tests are the workhorses of a suite because they are *fast*, *precise*, and *stable*: fast because they touch no network or disk, precise because a failure points at one unit, and stable because they do not break when an unrelated part of the system changes. Here is a unit under test and a small suite:

```
def apply_discount(price, percent):
  """Reduce price by percent (0..100). Raises ValueError on bad input."""
  if price < 0:
    raise ValueError("price must be non-negative")
  if percent < 0 or percent > 100:
    raise ValueError("percent must be in 0..100")
  return round(price * (1 - percent / 100), 2)
```

```
def test_no_discount():        assert apply_discount(100.0, 0)   == 100.0
def test_half_off():           assert apply_discount(143.0, 50)  == 71.5
def test_rounds_to_cents():    assert apply_discount(9.99, 10)   == 8.99
def test_full_discount():      assert apply_discount(40.0, 100)  == 0.0
def test_rejects_bad_percent():
  import pytest
  with pytest.raises(ValueError):
    apply_discount(100.0, 150)
```

Each test names a distinct behavior and carries its own oracle (the expected value or the expected exception). Notice we are already thinking about selection: 0 and 100 are the *boundaries* of the valid percent range (§10.4.2), and 150 is an invalid class.

What makes a unit test *good*? The **FIRST** mnemonic names five properties worth enforcing on every test you write:[7]

- **F**ast — milliseconds, so the whole suite runs on every save;
- **I**ndependent — no shared state and no required order, so any subset runs alone and a failure implicates exactly one test;
- **R**epeatable — the same result on every run and every machine, with no dependence on network, clock, or leftover environment;
- **S**elf-validating — the test asserts its own pass or fail, carrying its oracle (§10.1.4) instead of printing output for a human to eyeball;
- **T**imely — written with (or before — §2.3.2) the code it tests, while the design can still respond to what testing reveals.

10.2.2 Integration Testing

Units that each pass in isolation can still fail *together* — one returns meters where the other expects feet, one returns a null value where the other never checks. **Integration testing** exercises the *interfaces and interactions*

between units, growing the scope from a pair of collaborators up toward whole subsystems. This is where mismatched assumptions, wrong data formats, and protocol errors surface.

```
class PriceService:
  def __init__(self, catalog, discounts):
    self.catalog = catalog          # unit A: name -> base price
    self.discounts = discounts      # unit B: name -> percent off

  def quote(self, item):
    base = self.catalog.price_of(item)
    pct = self.discounts.percent_for(item)
    return apply_discount(base, pct)
```

A unit test of `PriceService`'s quote method would *mock* `catalog` and `discounts`. An **integration test** instead wires the real (or fake) catalog and discount components together and checks that their contract holds end to end:

```
def test_quote_integrates_catalog_and_discounts():
  catalog = Catalog({"mug": 12.0})
  discounts = Discounts({"mug": 25})
  svc = PriceService(catalog, discounts)
  assert svc.quote("mug") == 9.0     # 12.0 * (1 - 0.25)
```

If `Discounts` returned a *fraction* (0.25) while the discount function expects a *percentage* (25), every unit test could pass while this integration test correctly fails. That interface mismatch is the defect class unit tests are blind to.

10.2.3 Functional, System, and Acceptance Testing

Higher levels test the assembled product against ever-broader notions of "correct":

- **Functional testing** checks that a complete feature behaves per its specification, treating the system as a black box: *given a shopping cart with these items and this coupon, the checkout total is X*. It spans many units but is scoped to one function of the product.
- **System testing** exercises the *entire, integrated* system in a production-like environment, including **non-functional** requirements — performance, security, reliability, usability. "The page loads in under 500 ms at 1,000 concurrent users" is a system-level test.
- **Acceptance testing** answers the customer's question, "will I sign for this?" It is run against the customer's own criteria, often *by* or *with* the customer. **User acceptance testing (UAT)** uses real users and realistic tasks; **alpha/beta** releases are acceptance testing in the wild.

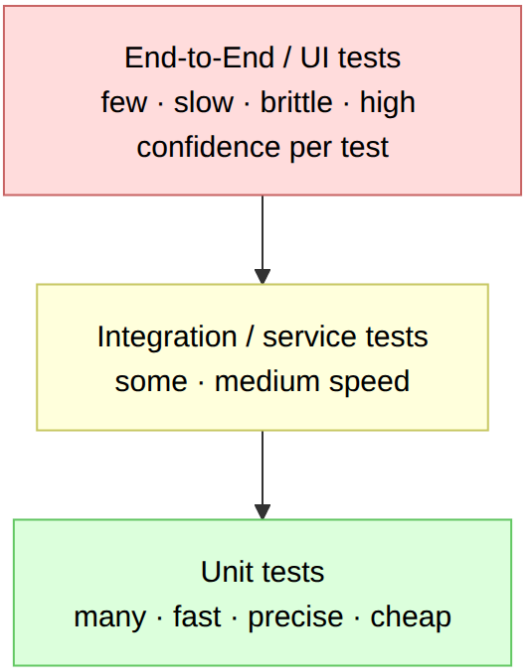
A useful way to remember the difference: functional and system testing ask *"did we build the product right?"* (verification), while acceptance testing asks *"did we build the right product?"* (validation).[8] Both matter, and passing one is no guarantee of the other.

> **From acceptance criteria to acceptance tests.** The **Given / When / Then** scenarios you wrote as a story's acceptance criteria (§3.4.1) *are* your acceptance tests. In **Behavior-Driven Development (BDD)**, those **Gherkin** `.feature` scenarios are made executable by tools like **Cucumber**, **behave**, or **SpecFlow/Reqnroll**: each `Given/When/Then` step binds to code, so the customer-readable specification runs on every build. This is functional/acceptance testing whose oracle (§10.1.4) is the agreed scenario itself.

One more system-level suite gets its own name because you will run it constantly. A **smoke test** is a fast, shallow pass that answers a single question: *is the build fundamentally alive?* The app starts, the homepage loads, a user can log in — nothing deeper. The name comes from hardware bring-up: power the board on and see whether smoke comes out before bothering with finer measurements.[9] Smoke tests run immediately after a build or a deployment, as a *gate*: if they fail, the build is dead on arrival, no deeper (and more expensive) testing is worth starting, and a deploy must not proceed.[10] Chapter 14 gives them a formal home as a stage of the delivery pipeline, run right after each deploy (§14.2.2).

10.2.4 Case Study: Test Early and Often — the Testing Pyramid

The levels are not equally cheap. A good unit test runs in milliseconds and almost never flakes; a full end-to-end (E2E) test may spin up a browser, a database, and three services, take minutes, and fail intermittently for reasons that have nothing to do with your code. This cost gradient motivates the **testing pyramid**: have *many* fast unit tests, *fewer* integration tests, and *only a handful* of slow end-to-end tests.[11]



The pyramid is an economic argument. Suppose a defect in the discount logic could be caught at any level. Caught by a unit test, the feedback arrives in seconds, on the exact line, before you even commit. Caught by an E2E test in CI, it arrives twenty minutes later as "checkout total was wrong," and you must *debug down* to the line. Caught in production, it arrives as a customer complaint and a refund. The cost of a defect typically rises substantially with each stage it survives — the exact multiplier varies by domain, tooling, and release process (§2.4.2) — so you push detection as far *down* and *early* as it will go.[12]

> **Pitfall — the ice-cream cone.** Teams that skip unit tests and lean on a big pile of slow, flaky E2E tests invert the pyramid into an "ice-cream cone." Their CI is slow, failures are hard to localize, and developers learn to ignore red builds ("probably just flaky"). An ignored test suite protects nothing. Prefer the pyramid deliberately.

Because flakiness is what poisons trust, know its causes. A **flaky test** — one that passes and fails across runs with no code change — most often traces to one of four roots: **race conditions and async timing** (the assertion runs before the work finishes); **shared mutable state** between tests (one test's leftovers change another's result); **test-order dependence** (the suite passes in one order and fails in another); or **unstable external dependencies** (real networks, real clocks, third-party services).[13] The fixes mirror the causes: isolate state so each test builds and tears down its own world; make tests order-independent (and let the runner randomize order to prove it); await asynchronous work properly with sane timeouts instead of sleeping and hoping; and replace or fake unstable dependencies. For the incorrigible case, **quarantine** the test out of the gating suite — then fix it or delete it, because a test allowed to stay red-then-green trains the team to ignore the suite.

“Test early and often” also means testing *continuously*: run the fast tests on every save, the full suite on every push, and treat a broken build as a stop-the-line event. This is the testing half of **continuous integration** (Chapter 2) — the discipline that keeps the “make change cheap” promise from Chapter 1 honest. Chapter 14 assembles these tests, with the static checks of Chapter 9, into the full delivery pipeline.

10.3 Code Coverage I: White-Box Testing

White-box testing selects inputs by looking at the code’s *structure*. To talk about structure precisely, we model a function as a graph.

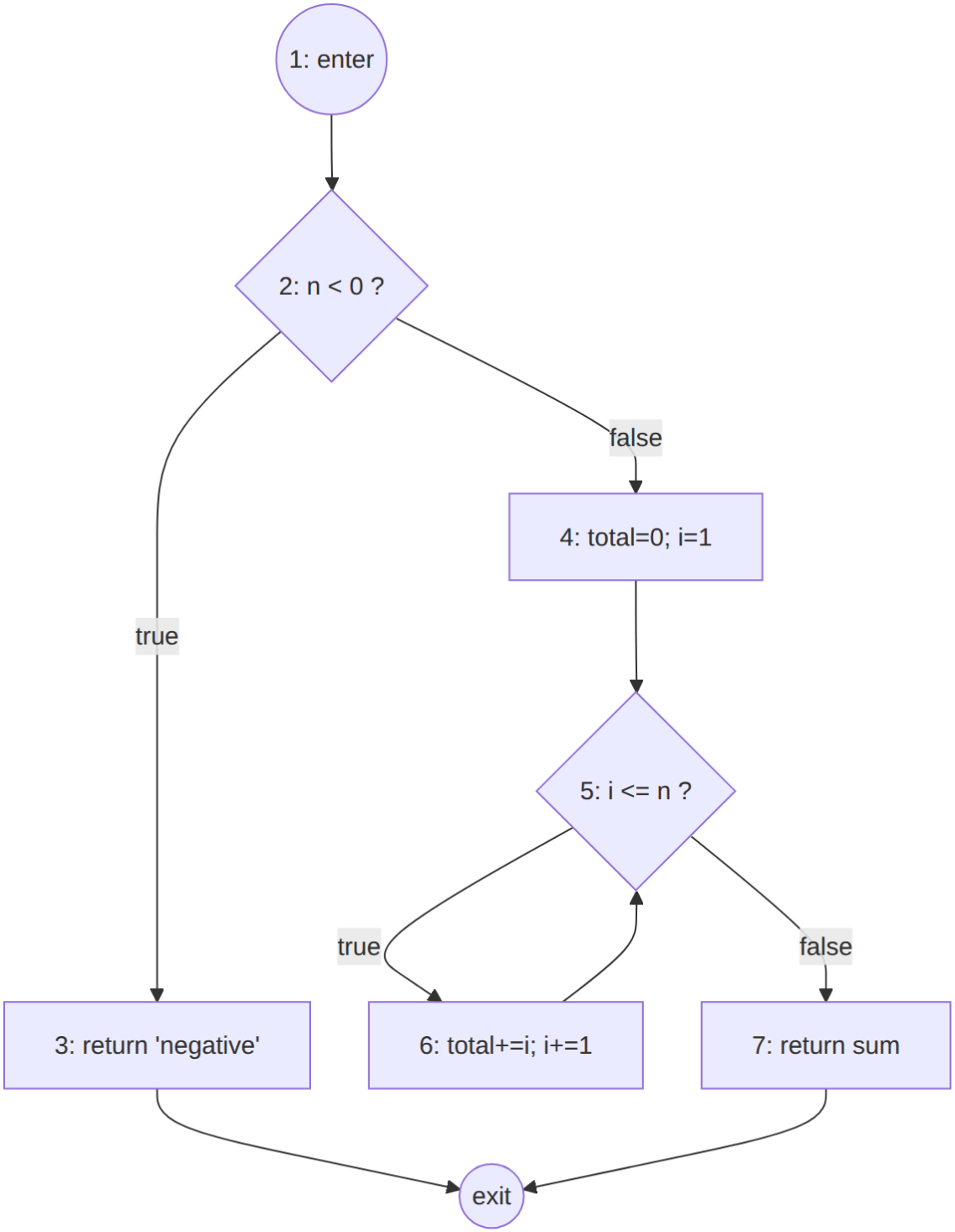
10.3.1 Control-Flow Graphs

A **control-flow graph (CFG)** represents a function as a directed graph: **nodes** are basic blocks (maximal straight-line sequences of statements), and **edges** are the possible transfers of control between them. A branch (an `if`, a loop condition) is a node with two outgoing edges; a merge is a node with two incoming edges. The CFG makes “statements,” “branches,” and “paths” into things you can *count*.

Consider a function that classifies a number and, for positive numbers, sums 1..n:

```
def classify_and_sum(n):        # node 1  (entry)
  if n < 0:                     # node 2  (decision)
    return "negative"           # node 3
  total = 0                     # node 4
  i = 1                         # node 4
  while i <= n:                 # node 5  (decision)
    total += i                  # node 6
    i += 1                      # node 6
  return f"sum={total}"         # node 7  (exit)
```

Its control-flow graph:



Two structural features matter for what follows. First, node 2 and node 5 are **decisions**, each with a true edge and a false edge. Second, node 5's true edge loops back — a **cycle** — which is why the number of *paths* through this function is unbounded (you can go around the loop 0, 1, 2, … times), while the number of *edges* is finite. That gap is the whole reason path coverage is usually impractical and weaker criteria exist.

The CFG also gives you a number worth knowing. **Cyclomatic complexity** is the count of a function's decision points plus one — equivalently, for a graph with E edges and N nodes, $E - N + 2$.[14] It measures how many independent paths thread the function, and so roughly how many tests branch coverage will demand. For the function above: two decisions (nodes 2 and 5), so complexity 2 + 1 = 3; or count the graph above, E = 9, N = 8, 9 − 8 + 2 = 3. Typical interpretation bands are **1–10** for simple, readily testable code; **11–20** for moderate complexity; **21–50** for risky code; and above **50** for code that is extremely difficult to test comprehensively and should usually be decomposed.[15] High-complexity functions tend to contain more defects and hard-to-cover branches, so the metric helps teams direct testing and refactoring effort (Chapter 15, §15.3) toward the areas of greatest risk.[16]

10.3.2 Control-Flow Coverage Criteria

Three classic criteria form a hierarchy of increasing strength. We work each one on the classify-and-sum function from §10.3.1.

Statement coverage requires every node (statement) to be executed by at least one test. What set of inputs runs every node? Node 3 needs `n < 0`; nodes 4–7 need `n >= 0`; node 6 (the loop body) needs `n >= 1`. So:

Test	Input `n`	Nodes executed	New nodes
T1	`-5`	1, 2, 3	1,2,3
T2	`3`	1,2,4,5,6,5,6,5,6,5,7	4,5,6,7

Two tests (`n=-5`, `n=3`) achieve 100% statement coverage — every node has run.

Branch (decision) coverage requires every *edge out of a decision* to be taken — both the true and false outcome of each decision. List the four decision edges: node 2 true, node 2 false, node 5 true, node 5 false. Check the tests above:

- `n=-5` takes node 2 **true**. (node 5 never reached.)
- `n=3` takes node 2 **false**, node 5 **true** (loop body runs), and node 5 **false** (loop exits).

Between them, all four edges are covered, so the *same* two tests also give 100% branch coverage here. That is a coincidence of this function's shape; branch coverage is strictly stronger than statement coverage in general. To see why, change the function so the loop can be skipped entirely and a statement after it depends on the loop having run — a test that covers every statement can still fail to take the "loop body never executes" edge.

> **Note — branch coverage subsumes statement coverage.** If every branch is taken, every node between branches must have run, so 100% branch ⇒ 100% statement. The converse fails: a single `if` with no `else` can reach 100% statement coverage with one test (condition true) while never taking the false edge. Whenever a team says "coverage," ask *which* coverage — the gap between these two is where real defects hide.

Path coverage requires every *distinct path* from entry to exit to be executed. Count the paths in the classify-and-sum function:

- The `n < 0` path (through node 3): **1** path.
- The `n >= 0` paths differ by how many times the loop runs: 0 iterations, 1 iteration, 2 iterations, … — one path per possible value of `n >= 0`, i.e. **unbounded**.

So full path coverage is *impossible* here — a decisive illustration of why we settle for branch coverage plus judgment about loops (commonly: test the loop 0 times, once, and "many" times). Even in loop-free code, path count grows as roughly 2^d for d independent decisions, so path coverage is exponential and reserved for tiny, critical routines.

The hierarchy, then: **path ⇒ branch ⇒ statement** (each implies the ones to its right).[17] Strength costs test cases; the engineering choice is how far up the hierarchy a given piece of code justifies climbing. A checkout total merits branch coverage; an autopilot demands more (§10.5).

> **Case study — measuring coverage isn't free.** Coverage tools work by *instrumenting* the program: a probe at every statement or branch records what ran. The probes cost real time and memory — overheads of 10–30% in time, with instrumented binaries 60–90% larger, have been reported — unnoticeable for a unit suite on a laptop, prohibitive on a memory-constrained phone. One line of research sidesteps instrumentation entirely: modern CPUs already record where they branch, and *sampling* those hardware monitors during a test run recovered up to ~90% of branch coverage with less overhead and no code growth — enough to measure coverage on the device a mobile app actually ships to.[6]

10.4 Input Coverage I: Black-Box Testing

Black-box testing turns the *specification* into a set of items to cover, without looking at the code. The two most valuable criteria partition the input space and then probe its edges.

10.4.1 Equivalence-Class Coverage

The insight is that inputs are not all different: a program usually treats whole *classes* of inputs identically, so one representative of a class is (almost) as good as any other. An **equivalence class (equivalence partition)** is a set of

inputs the specification implies should be handled the same way.[18] You partition the input space into classes — covering both *valid* and *invalid* classes — and pick one representative from each. This slashes the number of tests from "every input" to "one per class" while keeping strong coverage of *behaviors*.

Take a spec: *`set_volume(level)` accepts an integer `level` from 1 to 100 inclusive and sets the volume; any other value is rejected.* The classes are:

Class	Example representative	Expected behavior
C1: below range (`< 1`)	`-3`	rejected
C2: in range (`1..100`)	`55`	accepted
C3: above range (`> 100`)	`250`	rejected
C4: non-integer type	`"loud"`, `3.5`	rejected

Four tests, one per class, cover every distinct *specified* behavior. Compare this to testing 1, 2, 3, …, 100 (100 tests that all exercise the same behavior C2) — equivalence partitioning gives you the same behavioral coverage for a fraction of the cost, and it forces you to remember the *invalid* classes that ad-hoc testing skips.

10.4.2 Boundary-Value Coverage

Defects cluster at the *edges* of equivalence classes, because that is where programmers write the fragile comparisons: `<` versus `<=`, an off-by-one in a loop bound, `>` versus `>=`. **Boundary-value analysis** targets exactly those edges: for each boundary, test the value just below it, at it, and just above it.[18]

For `set_volume`, valid range 1..100, the two boundaries are 1 (lower) and 100 (upper). The boundary values to test:

Boundary	Values to test	Expected
lower = 1	`0`	rejected (just below)
	`1`	accepted (at)
	`2`	accepted (just above)
upper = 100	`99`	accepted (just below)
	`100`	accepted (at)
	`101`	rejected (just above)

Six boundary tests. Watch them do their job. Suppose a developer wrote the guard as:

```
def set_volume(level):
  if not isinstance(level, int) or level < 1 or level >= 100:   # BUG: >= should be >
    raise ValueError("level must be 1..100")
  _apply(level)
```

The equivalence-class test with representative `55` passes (it is happily accepted), and so would `250` (rejected) and `-3` (rejected). The defect hides at the top edge. But the boundary test at level `100` **fails** — the code rejects a value the spec says is valid — pinpointing the `>=`/`>` mistake. This is the payoff: boundary values find one of the most common categories of numeric defect, and they cost only a few extra tests on top of equivalence classes.

> **Note.** Combine the two criteria: equivalence partitioning tells you *which regions* exist; boundary analysis tells you *where within and between regions* to probe. In practice you write one "interior" representative per class **plus** the below/at/above triples at each boundary. For `set_volume` that is roughly 4 + 6 = a handful of tests that together cover behavior far better than dozens of random ones.

One black-box technique abandons careful selection altogether. **Fuzz testing (fuzzing)** throws huge volumes of malformed, random, or mutated inputs at a program and watches for crashes, hangs, and memory errors — no specification, no partitions, just the implicit oracle (§10.1.4) that the program *must not fall over*.[19] It is a cousin of property-based testing but adversarial in spirit: instead of checking a property on well-formed inputs, it hunts for the ill-formed input nobody thought to reject. That makes fuzzing a mainstay of security testing — buffer overflows, injection flaws, and denial-of-service defects are the bugs that surface when a parser meets input its author never imagined. Because an effective fuzzing campaign runs for hours or days, it belongs in a non-blocking pipeline stage (Chapter 14), not on the every-commit path.

10.5 Code Coverage II: MC/DC

Branch coverage checks that each *decision* comes out true and false at least once. But a decision can be a **compound** boolean expression — `(A && B) || C` — and branch coverage says nothing about the individual **conditions** inside it. A suite can flip the whole decision true/false while some condition never actually influenced the outcome. For the highest-criticality software (in avionics, DO-178C requires it at the most critical design-assurance level), we need a stronger criterion: **Modified Condition/Decision Coverage (MC/DC)**.[20]

10.5.1 Condition and Decision Coverage Are Independent

First untangle two finer criteria:

- **Condition coverage** requires each individual condition (`A`, `B`, `C`) to be both true and false at least once — but says nothing about the decision's outcome.

- **Decision coverage** (= branch coverage) requires the whole decision to be both true and false — but says nothing about individual conditions.

Neither implies the other. Consider the decision `A || B` with just two tests: `(A=true, B=false)` and `(A=false, B=true)`.

Test	A	B	`A || B`
t1	T	F	T
t2	F	T	T

Every condition takes both values (A is T then F; B is F then T), so **condition coverage is 100%**. Yet the decision is `true` in *both* tests — it never came out false — so **decision coverage is only 50%**. Conversely, testing `(A=true, B=false)` and `(A=false, B=false)` gives full decision coverage (T then F) while `B` never takes the value true — so **condition coverage is not met**. Because each criterion can be satisfied while the other is not, they are genuinely independent — and *neither alone* guarantees that every condition can actually change the result.

10.5.2 MC/DC Pairs of Tests

MC/DC demands the strongest practical property: **every condition must be shown to independently affect the decision's outcome.**[21] Concretely, for each condition you must exhibit a **pair** of tests in which *only that condition changes*, everything else is held fixed, *and the decision's result flips*. That pair proves the condition is not dead weight — it really controls the outcome on its own.

MC/DC bundles four requirements: every decision takes both outcomes (decision coverage); every condition takes both values (condition coverage); and each condition independently flips the decision. Remarkably, for a decision with **N conditions** this can usually be achieved with just **N + 1** tests — a linear number, versus 2^N for exhaustive testing of the truth table.[21] That exhaustive criterion has its own name — **multiple-condition coverage**, which requires every combination of condition truth values — and MC/DC is best understood as its practical approximation.[18] That is why MC/DC is the sweet spot for critical code: strong evidence that each condition independently affects the outcome, at a linear rather than exponential test cost.

Let us fully work the decision from the section opener:

$$D = (A \wedge B) \vee C$$

It has N = 3 conditions, so we expect 4 tests. Start from the complete truth table (row numbers will let us cite tests precisely):

#	A	B	C	`A && B`	`D = (A&&B) || C`
1	T	T	T	T	**T**
2	T	T	F	T	**T**
3	T	F	T	F	**T**
4	T	F	F	F	**F**
5	F	T	T	F	**T**
6	F	T	F	F	**F**
7	F	F	T	F	**T**
8	F	F	F	F	**F**

Now find an **independence pair** for each condition — two rows differing *only* in that condition, with different results for `D`.

Condition A. Hold B and C fixed, vary A, require `D` to flip. Compare rows 2 and 6:

#	A	B	C	D
2	**T**	T	F	T
6	**F**	T	F	F

Only A changed (T→F), B=T and C=F are held, and `D` flipped T→F. So **{2, 6}** is A's independence pair. (Intuitively: with C false, the result is decided by `A && B`; holding B true, A alone controls it.)

Condition B. Hold A and C fixed, vary B. Compare rows 2 and 4:

#	A	B	C	D
2	T	**T**	F	T
4	T	**F**	F	F

Only B changed (T→F), A=T and C=F held, `D` flipped T→F. So **{2, 4}** is B's pair.

Condition C. Hold A and B fixed, vary C. We need `A && B` to be *false* (otherwise the `|| C` is short-circuited and C cannot matter). Compare rows 3 and 4:

#	A	B	C	D
3	T	F	**T**	T
4	T	F	**F**	F

Only C changed (T→F), A=T and B=F held, `D` flipped T→F. So **{3, 4}** is C's pair.

Now assemble the minimal suite by taking the **union** of the rows used:

$$2, 6 \cup 2, 4 \cup 3, 4 = 2, 3, 4, 6$$

Just **four tests** — exactly N + 1 = 3 + 1:

Test	A	B	C	D	Proves independence of
row 2	T	T	F	T	A (with 6), B (with 4)
row 3	T	F	T	T	C (with 4)
row 4	T	F	F	F	B (with 2), C (with 3)
row 6	F	T	F	F	A (with 2)

Verify the bundle: A's pair {2,6} ✓, B's pair {2,4} ✓, C's pair {3,4} ✓ are all present. Every condition is shown to independently drive the decision, the decision is both T (rows 2, 3) and F (rows 4, 6), and each condition takes both values across the suite. This 4-test set satisfies MC/DC for `(A && B) || C`, versus 8 tests for the full truth table — the linear-vs-exponential saving that makes MC/DC affordable at scale.

> **Note — why "modified."** The name comes from the *modification* idea: for each condition, you *modify only it* and observe the decision change. The reused test (row 2 here, which serves both A and B; row 4, which serves both B and C) is why the count lands at N + 1 rather than 2N — pairs share tests. When short-circuit operators or coupled conditions prevent sharing, you may need slightly more than N + 1, though still usually far fewer than the full 2^N .

> **Pitfall.** MC/DC is defined per *condition*, not per *decision*. If you rewrite `if (a > 0 && b > 0)` as two nested `if`s, tools may report "100% branch coverage" and lull you into thinking you are done — but you have not shown each condition's independent effect. When correctness is critical, measure MC/DC explicitly with a tool that understands compound conditions.

10.6 Input Coverage II: Combinatorial Testing

Many systems are configured by several parameters at once, and defects often lurk in *combinations* rather than single values. Suppose a checkout page is parameterized by browser, operating system, and locale, each with three options:

- **Browser** ∈ {Chrome, Firefox, Safari}
- **OS** ∈ {Windows, macOS, Linux}
- **Locale** ∈ {EN, FR, JP}

Testing *every* combination is **exhaustive** and costs $3 \times 3 \times 3 = 27$ runs — and with more parameters it explodes multiplicatively (ten binary flags is already 1,024 combinations). That is infeasible, yet single-value testing ("try each browser once") misses interaction bugs like "date format breaks *only* on Safari *with* the JP locale."

Combinatorial testing exploits an empirical regularity: the large majority of interaction defects are triggered by the interaction of just **two** parameters, and almost all of the rest by three.[22] NIST studied fault databases across many domains and found that testing all *pairs* of parameter values catches the bulk of interaction faults.[23] So instead of covering all *combinations*, we cover all *pairs* — **pairwise** (a.k.a. **all-pairs**, or 2-way) testing.

The goal: choose a small set of test configurations such that for **every pair of parameters, every pair of their values appears together in at least one test.** For our three 3-valued parameters, all pairs can be covered in just **9** tests (not 27):

Test	Browser	OS	Locale
1	Chrome	Windows	EN
2	Chrome	macOS	FR
3	Chrome	Linux	JP
4	Firefox	Windows	FR
5	Firefox	macOS	JP
6	Firefox	Linux	EN
7	Safari	Windows	JP
8	Safari	macOS	EN
9	Safari	Linux	FR

Check the claim for one parameter pair, Browser × Locale. The (Browser, Locale) combinations appearing are: (Chrome,EN), (Chrome,FR), (Chrome,JP), (Firefox,FR), (Firefox,JP), (Firefox,EN), (Safari,JP), (Safari,EN), (Safari,FR) — all 9 possible pairs, each at least once. The same holds for Browser × OS and OS × Locale (verify

OS × Locale as an exercise). So every 2-way interaction is exercised, and the "Safari + JP" bug from above *cannot* slip through: test 7 pairs them. We reduced 27 runs to 9 — a 3× saving here, and the saving grows dramatically with more parameters (pairwise coverage of ten 3-valued parameters needs fewer than twenty tests, not 59,049).

> **Note.** You rarely build these tables by hand. Tools — Microsoft's PICT, the NIST **ACTS** tool, or libraries in most languages — generate a minimal (or near-minimal) covering array for you, and can extend from pairwise to 3-way or higher when the risk justifies it. What you must supply is the *model*: the parameters, their meaningful values (use equivalence classes and boundaries from §10.4 to choose them), and any constraints ("Safari implies macOS"). Combinatorial testing composes with black-box input analysis; it does not replace it.

> **Pitfall.** Pairwise is a heuristic, not a guarantee. If a defect requires a *three-way* interaction (fails only on Firefox + Linux + JP together), a 2-way suite may miss it. Use higher-strength (3-way, 4-way) coverage for the most critical or most interaction-prone subsystems, and let the empirical fault-strength data — not habit — set the level.

10.7 Conclusion

Testing is how you earn the right to say "this software works," and this chapter turned that claim from a hope into an engineering argument. The through-line is **criteria**: every hard question about testing becomes tractable once you name a set of items to cover and measure the fraction you have.

- The four issues — **selection, adequacy, oracles, automation** (§10.1) — are the questions every testing decision answers, explicitly or by accident. Answer them on purpose.
- Testing happens at **levels** — unit, integration, functional, system, acceptance (§10.2) — and the **pyramid** tells you to invest most where feedback is fastest and cheapest, catching defects early where they cost the least.
- **White-box** criteria (§10.3, §10.5) measure how thoroughly you exercised the code: statement ⇐ branch ⇐ path in strength, with **MC/DC** as the linear-cost criterion that isolates each condition's independent effect in critical compound decisions.
- **Black-box** criteria (§10.4, §10.6) measure how thoroughly you exercised the *specified behavior*: equivalence classes, boundary values, and pairwise combinations.

No single criterion is enough, and no amount of testing proves correctness. But a suite built from these criteria — black-box *and* white-box, at the right level, with honest oracles, run automatically on every change — is the most reliable, most economical net we have for catching the defects Chapter 1 promised are inevitable. The next chapter turns testing's adversarial instinct — probing the inputs and error paths where a program misbehaves —

toward an attacker who is *trying* to make it misbehave. Then Chapter 12 turns to measuring quality across a whole project, where the coverage numbers you learned to compute here are among the first metrics you will report.

Sources

[1] Edsger W. Dijkstra, *Notes on Structured Programming*, EWD249 (1970), §3 (“Program testing can be used to show the presence of bugs, but never to show their absence!”); the aphorism also appears in the 1969 NATO *Software Engineering Techniques* report. cs.utexas.edu.

[2] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo, *The Oracle Problem in Software Testing: A Survey*, IEEE Transactions on Software Engineering 41(5) (2015). doi.org.

[3] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward, *Hints on Test Data Selection: Help for the Practicing Programmer*, IEEE Computer 11(4) (1978). doi.org.

[4] Seb Rose, *BDD, Approval Testing, and VisualTest*, Cucumber Blog (February 8, 2023) — states the maxim directly: every automated test should be observed failing when written and passing only after the intended behavior is implemented. It also reflects the red–green–refactor discipline of TDD and the logic of mutation testing, which changes production code precisely to learn whether the suite detects the fault. cucumber.io/blog.

[5] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold, *Prioritizing Test Cases for Regression Testing*, IEEE Transactions on Software Engineering 27(10) (2001) — the paper that introduced APFD; the seven-week suite is its industrial case study. doi.org.

[6] Kristen R. Walcott-Justice, *Testing in Resource-Constrained Environments*, Ph.D. dissertation, University of Virginia (2012). Develops budget-guaranteed, time-aware test selection (knapsack solvers, then an overlap-aware genetic algorithm) and THeME, which recovers up to ~90% of branch coverage by sampling CPU hardware monitors instead of instrumenting the code. faculty.uccs.edu.

[7] Tim Ottinger and Brett Schuchert, *F.I.R.S.T.* (Agile in a Flash, 2009); popularized in Robert C. Martin, *Clean Code*, ch. 9 (2008). agileinaflash.blogspot.com.

[8] Barry W. Boehm, *Verifying and Validating Software Requirements and Design Specifications*, IEEE Software 1(1) (1984). doi.org.

[9] Cem Kaner, James Bach, and Bret Pettichord, *Lessons Learned in Software Testing: A Context-Driven Approach* (Wiley, 2002). wiley.com.

[10] Steve McConnell, *Daily Build and Smoke Test*, IEEE Software 13(4) (1996). stevemcconnell.com.

[11] Mike Cohn, *Succeeding with Agile: Software Development Using Scrum* (Addison-Wesley, 2009); summarized in Martin Fowler, *TestPyramid* (2012). martinfowler.com.

[12] Barry Boehm and Victor R. Basili, *Software Defect Reduction Top 10 List*, IEEE Computer 34(1) (2001). cs.umd.edu.

[13] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov, *An Empirical Analysis of Flaky Tests*, Proc. FSE 2014. doi.org.

[14] Thomas J. McCabe, *A Complexity Measure*, IEEE Transactions on Software Engineering SE-2(4) (1976). doi.org.

[15] Software Engineering Institute, *C4 Software Technology Reference Guide — A Prototype*, CMU/SEI-97-HB-001 (1997), "Cyclomatic Complexity" section. sei.cmu.edu.

[16] Arthur H. Watson and Thomas J. McCabe, *Structured Testing: A Testing Methodology Using the Cyclomatic Complexity Metric*, NIST Special Publication 500-235 (1996). nvlpubs.nist.gov.

[17] Paul Ammann and Jeff Offutt, *Introduction to Software Testing*, 2nd ed. (2016). cs.gmu.edu.

[18] Glenford J. Myers, *The Art of Software Testing* (Wiley, 1979; 3rd ed. 2011). wiley.com.

[19] Barton P. Miller, Lars Fredriksen, and Bryan So, *An Empirical Study of the Reliability of UNIX Utilities*, Communications of the ACM 33(12) (1990). doi.org.

[20] RTCA, *DO-178C: Software Considerations in Airborne Systems and Equipment Certification* (2011), Table A-7 — MC/DC is an objective only at Level A. rtca.org.

[21] Kelly J. Hayhurst, Dan S. Veerhusen, John J. Chilenski, and Leanna K. Rierson, *A Practical Tutorial on Modified Condition/Decision Coverage*, NASA/TM-2001-210876 (2001). ntrs.nasa.gov.

[22] D. Richard Kuhn, Dolores R. Wallace, and Albert M. Gallo, *Software Fault Interactions and Implications for Software Testing*, IEEE Transactions on Software Engineering 30(6) (2004). csrc.nist.gov.

[23] D. Richard Kuhn, Raghu N. Kacker, and Yu Lei, *Practical Combinatorial Testing*, NIST Special Publication 800-142 (2010). csrc.nist.gov.

- **Key takeaways** are summarized above in §10.7.
- Continue to the Exercises.
- Go deeper with the Open Resources for this chapter.

Chapter 10 — Exercises

Exercises are graded by depth: **[warm-up]** checks understanding, **[analysis]** asks you to reason. Several exercises require *actual work* — drawing a graph, deriving values, or constructing a test set — not just prose. Show the work, not only the answer.

Concepts

1. **[warm-up]** In one sentence each, state the *selection*, *adequacy*, and *oracle* problems from §10.1.1, then give a concrete example of a function for which the oracle problem is genuinely hard (you can produce inputs but cannot easily say what the right output is).

2. **[warm-up]** Explain why "we ran the app and it looked fine" fails as a stopping rule, and rewrite it as a proper test-adequacy criterion for a function that validates email addresses.

3. **[warm-up]** For the `apply_discount` function in §10.2.1, classify each of these as an *exact-value*, *property*, or *cross-check* oracle: (a) `assert apply_discount(100, 50) == 50`; (b) `assert apply_discount(p, 0) == p for all p >= 0`; (c) comparing the result against last year's pricing engine. Which is strongest, and why?

4. **[warm-up]** Give one defect that a *unit* test cannot catch but an *integration* test can, and one that an *integration* test cannot catch but a *system* test can. Explain the difference in scope that makes each blind spot exist.

Analysis

5. **[analysis]** A teammate proudly reports "95% statement coverage." Describe two distinct ways the code could still contain a serious, easily-triggered defect despite that number. For each, name the coverage criterion (from §10.3 or §10.5) that would have forced a test to catch it.

6. **[analysis]** Explain, with a two-test example of your own construction, why 100% *condition* coverage does not imply 100% *decision* coverage. Then give a different two-test example showing the reverse (100% decision, less than 100% condition). (See §10.5.1 — but use a decision *other* than `A || B`.)

7. **[analysis]** A team has inverted the testing pyramid into an "ice-cream cone": 200 slow end-to-end tests and 30 unit tests. Describe three concrete symptoms they will experience and the refactoring of their test strategy you would recommend, with justification tied to the cost-of-defect argument in §10.2.4.

8. **[analysis]** Combinatorial (pairwise) testing "reduces 27 runs to 9" in §10.6. Under what circumstance does a pairwise suite *provably* miss a defect? Construct a small three-parameter example where the bug appears

only under a specific three-way combination, and state what coverage strength you would need to guarantee catching it.

Do the work

9. **[analysis]** *Draw a CFG and give branch-coverage tests.* Consider:

```
def fizzbuzz(n):
  if n % 15 == 0:
    return "FizzBuzz"
  elif n % 3 == 0:
    return "Fizz"
  elif n % 5 == 0:
    return "Buzz"
  else:
    return str(n)
```

   (a) Draw the control-flow graph (Mermaid or by hand), labeling each decision node and each edge. (b) Give the minimum number of test inputs that achieves 100% **branch coverage**, list the input and expected output for each, and (c) state which branch edges each test covers. (d) Does your suite also achieve 100% statement coverage? Argue why, referencing the subsumption relationship from §10.3.2.

10. **[analysis]** *Derive boundary values.* A spec states: *`assign_grade(score)` accepts an integer `0..100`; it returns `"A"` for `90..100`, `"B"` for `80..89`, `"C"` for `70..79`, and `"F"` for `0..69`; any score outside `0..100` is rejected with an error.* (a) List the equivalence classes (valid and invalid). (b) For every boundary in the spec, give the below/at/above triple of values and the expected result for each. (c) Then write a buggy one-line guard or comparison (e.g., `>=` vs `>`) and identify exactly which of your boundary tests would catch it.

11. **[analysis]** *Construct an MC/DC test set.* Consider the decision `D = A && (B || C)` (note: different from the worked example). (a) Build the full 8-row truth table with a `D` column. (b) Find an independence pair for each of A, B, and C (two rows differing only in that condition, with `D` flipping). (c) Take the union to give a minimal MC/DC suite and confirm it has N + 1 = 4 tests (or explain why this decision needs a different count). (d) State, for each test in your final set, which condition(s) it helps prove independent.

12. **[analysis]** *Path counting.* For the classify-and-sum function in §10.3.1, you argued full path coverage is impossible. Now take a *loop-free* function with three independent `if` statements in sequence (write one — e.g., applying three optional surcharges). (a) How many distinct paths does it have? (b) How many tests for branch coverage? (c) Explain the exponential-vs-linear gap and what it implies about when path coverage is worth attempting.

13. **[analysis]** *Test the test* (§10.1.5). Select one test from your project that has always passed. Introduce the smallest deliberate defect the test should detect, run the test, and record three things: whether it fails, whether it fails *for the expected reason*, and whether any unrelated tests fail first. Restore the implementation afterward. If the test stayed green, it was not testing what you thought — strengthen it and repeat the experiment until a real defect turns it red for the right reason.

Chapter 10 — Open Resources

Free, open-licensed, or freely accessible materials that reinforce this chapter. Types: 📘 open text · 🎓 course · 📄 primary source / standard · 🎥 video. Each entry notes its license or access terms; when in doubt, check the linked page.

Foundations: what to test and how

- 🎓 **MIT 6.031 — Software Construction, "Testing" reading** — web.mit.edu/6.031. Excellent treatment of partitioning the input space, choosing test cases, and coverage. *License: the 6.031 materials are offered for reuse under Creative Commons (BY-SA); check the page footer.*
- 🎓 **UC Berkeley — Engineering Software as a Service (ESaaS), ch. 8 "Testing"** — saasbook.info. Test-driven development, mocks/stubs, and the testing pyramid in a full-stack context. *License: book is free to read online; see the site for terms.*
- 📘 **Software Engineering at Google — testing chapters** (Ch. 12 "Testing Overview", Ch. 13 "Unit Testing", Ch. 10 "Test Doubles", Ch. 14 "Larger Testing") — abseil.io/resources/swe-book. Industrial perspective on test size, flakiness, hermeticity, and scaling a suite. *License: CC BY-NC-ND 4.0 (free to read; no derivatives).*
- 📄 **Dijkstra — "Notes on Structured Programming" (EWD249, 1970)** — cs.utexas.edu. Source of the chapter's opening principle: "Program testing can be used to show the presence of bugs, but never to show their absence!" *Access: free in the UT Austin EWD archive.*
- 📄 **Boehm — "Verifying and Validating Software Requirements and Design Specifications"**, IEEE Software 1(1), 1984 — doi.org. Origin of the "building the product right" (verification) vs. "building the right product" (validation) distinction. *Access: abstract free; full text via IEEE Xplore (paywalled).*
- 📄 **Boehm & Basili — "Software Defect Reduction Top 10 List"**, IEEE Computer 34(1), 2001 — cs.umd.edu. The classic evidence that defects cost far more to fix the later they are found. *Access: free author copy at the University of Maryland.*
- 📄 **Barr, Harman, McMinn, Shahbaz & Yoo — "The Oracle Problem in Software Testing: A Survey"**, IEEE TSE 41(5), 2015 — doi.org. The definitive survey behind §10.1.4's oracle strategies. *Access: abstract free; full text via IEEE Xplore (paywalled); author preprints circulate freely.*
- 📄 **DeMillo, Lipton & Sayward — "Hints on Test Data Selection: Help for the Practicing Programmer"**, IEEE Computer 11(4), 1978 — doi.org. The paper that launched mutation testing (§10.1.5) and the coupling effect. *Access: abstract free; full text via IEEE Xplore (paywalled).*
- 📘 **Myers — "The Art of Software Testing"** (Wiley, 1979; 3rd ed. 2011) — wiley.com. The classic source for equivalence partitioning, boundary-value analysis, and multiple-condition coverage (§10.4, §10.5). *Access: commercial book; often in university libraries.*

- 📘 **Kaner, Bach & Pettichord — "Lessons Learned in Software Testing"** (Wiley, 2002) — wiley.com. Context-driven testing wisdom, including the hardware origin of the "smoke test" name (§10.2.3). *Access: commercial book.*

Coverage criteria, white-box, and MC/DC

- 📘 **Ammann & Offutt — "Introduction to Software Testing" companion site** — cs.gmu.edu/~offutt/softwaretest. Slides, examples, and tools for graph coverage, logic coverage (including MC/DC), and input-space partitioning — the academic backbone of §§10.3–10.6. *Access: teaching materials free on the author's site.*
- 📄 **NASA/DOT-FAA — "A Practical Tutorial on Modified Condition/Decision Coverage"** (Hayhurst, Veerhusen, Chilenski, Rierson), NASA/TM-2001-210876 — ntrs.nasa.gov. The definitive, worked explanation of MC/DC, independence pairs, and the N + 1 minimum test count. *License: U.S. government work, public domain.*
- 📄 **RTCA DO-178C** (software considerations in airborne systems) — rtca.org. The standard whose Table A-7 *mandates* MC/DC at the most critical software level (Level A). The standard itself is sold by RTCA, but many university course pages summarize its coverage objectives for free.
- 📄 **McCabe — "A Complexity Measure"**, IEEE TSE SE-2(4), 1976 — doi.org. The original cyclomatic-complexity paper behind §10.3.1. *Access: abstract free; full text via IEEE Xplore (paywalled).*
- 📄 **NIST SP 500-235 — "Structured Testing: A Testing Methodology Using the Cyclomatic Complexity Metric"** (Watson & McCabe, 1996) — nvlpubs.nist.gov. Basis-path testing and the evidence for treating complexity 10 as a working limit. *License: U.S. government work, public domain.*
- 📄 **SEI — "C4 Software Technology Reference Guide" (CMU/SEI-97-HB-001, 1997)** — sei.cmu.edu. Source of the conventional cyclomatic-complexity risk bands (1–10, 11–20, 21–50, >50) quoted in §10.3.1. *Access: free PDF from the SEI.*

Black-box and combinatorial testing

- 📄 **NIST — Automated Combinatorial Testing for Software (ACTS) project** — csrc.nist.gov/projects/automated-combinatorial-testing-for-software. The empirical fault-strength data behind §10.6, plus the free **ACTS** covering-array generator and NIST Special Publication 800-142. *License: U.S. government work, public domain; ACTS tool free for download.*
- 📄 **Microsoft PICT — Pairwise Independent Combinatorial Testing tool** — github.com/microsoft/pict. A small, scriptable covering-array generator for building pairwise test suites. *License: MIT.*
- 📄 **Kuhn, Wallace & Gallo — "Software Fault Interactions and Implications for Software Testing"**, IEEE TSE 30(6), 2004 — csrc.nist.gov. The fault-database study behind §10.6's claim that most interaction failures involve only two or three parameters. *Access: NIST page links a free preprint.*
- 📄 **Kuhn, Kacker & Lei — NIST SP 800-142, "Practical Combinatorial Testing"** (2010) — csrc.nist.gov. The full how-to for pairwise and higher-strength testing, with the empirical interaction-rule

data. *License: U.S. government work, public domain.*

- 📄 **Miller, Fredriksen & So — "An Empirical Study of the Reliability of UNIX Utilities"**, CACM 33(12), 1990 — doi.org. The study that coined "fuzz" testing (§10.4.2) by crashing a quarter of UNIX utilities with random input. *Access: abstract free; free copies via the UW–Madison fuzzing project.*

Tools and practice

- 📘 **pytest documentation** — docs.pytest.org. Fixtures, parametrization, and the `raises` idiom used in this chapter's examples. *License: docs under MIT.*
- 📄 **Hypothesis — property-based testing for Python** — hypothesis.readthedocs.io. Puts the property/invariant oracles of §10.1.4 into practice by generating inputs for you. *License: MPL-2.0.*
- 📘 **Python `unittest.mock` — official documentation** — docs.python.org/3/library/unittest.mock.html. `Mock`, `patch`, and `side_effect` — the standard-library way to build the stubs and mocks of §10.2.1. *License: Python docs, PSF license (free to read and reuse).*
- 📘 **behave — BDD for Python, documentation** — behave.readthedocs.io. Executable Gherkin as in the §10.2.3 callout: a `features/` directory of `.feature` scenarios plus a `steps/` directory whose `@given`/`@when`/`@then` functions bind each step to code. *License: BSD-2-Clause.*
- 📄 **mutmut — mutation testing for Python** — mutmut.readthedocs.io. Generates and runs the mutants of §10.1.5 against a pytest suite and reports the survivors. *License: open source; see the project page.*
- 📄 **Google OSS-Fuzz — continuous fuzzing documentation** — google.github.io/oss-fuzz. How industrial-scale fuzz testing (§10.4) is run continuously against hundreds of open-source projects, with guides for adding your own. *License: Apache-2.0.*
- 🎥 **"TDD, Where Did It All Go Wrong" — Ian Cooper** (conference talk, freely on YouTube). A widely-cited critique of testing implementation details instead of behavior; a good companion to the pyramid and oracle discussions. *Access: free to view.*
- 📄 **McConnell — "Daily Build and Smoke Test"**, IEEE Software 13(4), 1996 — stevemcconnell.com. The classic best-practices column on gating every build with a smoke test (§10.2.3). *Access: free on the author's site.*
- 📄 **Fowler — "TestPyramid"** (martinfowler.com bliki) — martinfowler.com/bliki/TestPyramid.html. A short, free summary of the testing pyramid, crediting Mike Cohn's *Succeeding with Agile* (2009), where the model appeared (§10.2.4). *Access: free to read.*
- 📄 **Ottinger & Schuchert — "F.I.R.S.T." (Agile in a Flash, 2009)** — agileinaflash.blogspot.com. The originators' write-up of the FIRST properties of §10.2.1, popularized in Robert C. Martin's *Clean Code* (2008), ch. 9. *Access: free blog post.*
- 📄 **Luo, Hariri, Eloussi & Marinov — "An Empirical Analysis of Flaky Tests"**, FSE 2014 — doi.org. The empirical taxonomy of flaky-test root causes (async waits, concurrency, order dependence) behind §10.2.4. *Access: free author copy on the authors' Illinois pages; ACM DL paywalled.*
- 📄 **Walcott-Justice — *Testing in Resource-Constrained Environments***, Ph.D. dissertation, University of Virginia, 2012 — faculty.uccs.edu. Time-aware regression test selection under a hard budget (knapsack

solvers, then an overlap-aware genetic algorithm) and THeME, which measures branch and statement coverage by sampling CPU hardware monitors instead of instrumenting the code (§10.1.6, §10.3.2). *Access: free PDF on the author's faculty page.*

License note

Linked resources remain under their own licenses (noted above where known); this page is licensed **CC BY-SA 4.0**.

Chapter 11 — Software Security

> **Where we are.** Security has surfaced in almost every chapter of this book, always as one concern among many. Chapter 3 had you draw **attack trees** and walk a STRIDE checklist to ask what an adversary wants (§3.7). Chapter 6 had you write security as a **quality-attribute scenario**, so "the system shall be secure" became something you could measure (§6.1.4). Chapter 9's static analysis traced tainted input into a query, and Chapter 10's tests probed the error paths an attacker lives on — and Chapter 14 will turn many of these checks into automated pipeline gates. Security must be addressed throughout the software lifecycle — requirements, design, implementation, dependencies, testing, and deployment. This chapter brings those practices together.

For most of this book, "does it work?" has meant "does it do what the user needs?" Security asks a sharper question: does the system do *only* that, even when someone is actively trying to make it do something else? Most other quality attributes assume a cooperative world — users who supply reasonable inputs, dependencies that behave, networks that deliver what was sent. Security assumes the opposite: an intelligent adversary studying your system for the one input, the one dependency, the one forgotten default that turns your feature into their foothold. That adversarial framing is what makes security different, and it is why the defenses in this chapter run from the first design sketch all the way to code you have already shipped.

> **Principle.** Security is not a phase. A vulnerability can be designed in during requirements, coded in during implementation, imported through a dependency, or introduced by a deployment — so every earlier discipline in this book is also a security control. The cheapest defense is the one you made impossible by design; the most expensive is the one you patch after a breach.

11.1 Security Is a Lifecycle Property

11.1.1 The CIA Triad

The classic way to name what security protects is the **CIA triad** — three properties every system holds in some measure:

- **Confidentiality** — only authorized parties can read data. A patient's diagnosis should be visible to their clinician and to no one else.
- **Integrity** — data and code are not altered by unauthorized parties, and unauthorized changes are detectable. An attacker who cannot read a record may still do real harm by silently *changing* it — a dosage, a balance, a build artifact.

- **Availability** — the system is up and responsive when legitimate users need it. This is the same availability Chapter 14 will trade against consistency in the CAP theorem; a denial-of-service attack is an availability attack, and downtime you did not choose is a security failure as much as a reliability one.

The triad is a lens for asking "secure against what?" A leaked mailing list is a confidentiality failure; a tampered firmware update is an integrity failure; a ransomware lockout is an availability failure. Naming the property under threat keeps a security conversation concrete instead of a vague wish to "be secure."

11.1.2 Threat, Vulnerability, Exploit

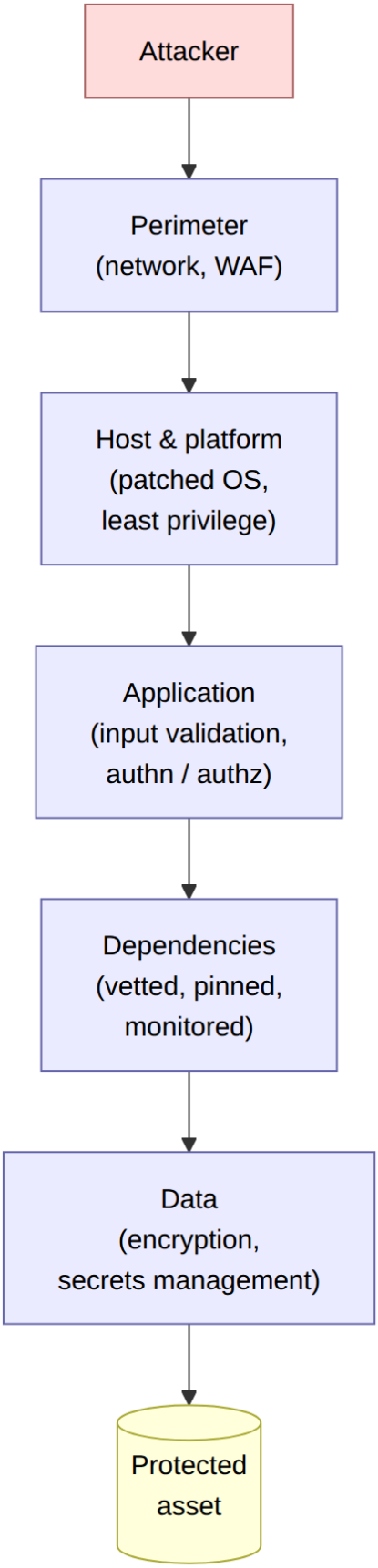
Three words get used loosely in casual talk and must be kept distinct in engineering.

A **vulnerability** is a weakness in the system — a missing bounds check, an unescaped query, an over-privileged account, a dependency with a known flaw. A **threat** is a potential source of harm: the actor or event that might act against you — a criminal crew, a disgruntled insider, an automated bot sweeping the internet. An **exploit** is the concrete technique or code that turns a specific vulnerability into an actual compromise.

The distinction is practical. You rarely get to choose your threats — the internet supplies a steady background of them — but you own every vulnerability in your system, and you can retire one before any exploit for it exists. **Risk** is where the three meet: a vulnerability that no threat can reach is low risk, and a severe threat aimed at a weakness you have already closed is harmless. Security work is the ongoing business of shrinking the overlap.

11.1.3 Defense in Depth and Shifting Left

No single control is perfect, so serious systems assume each one will eventually fail and place several independent layers between an attacker and what matters. **Defense in depth** is that layering: a network perimeter, a hardened host, an application that validates its inputs and checks authorization on every request, and encrypted data with tightly held keys. An attacker who slips past one layer meets another, and the layers are chosen to fail *independently* — a flaw that defeats the firewall should not also defeat input validation.


Attacker
Perimeter
(network, WAF)
Host & platform
(patched OS,
least privilege)
Application
(input validation,
authn / authz)
Dependencies
(vetted, pinned,
monitored)
Data
(encryption,
secrets management)
Protected
asset

The other structural idea is *when* security work happens. The same economics Chapter 9 gave for defects applies with force to vulnerabilities: the earlier you catch one, the cheaper it is to fix. A trust-boundary mistake caught in a design review costs an afternoon; the same mistake caught after a breach costs an incident, a disclosure, and sometimes the company. **Shifting left** means moving security activity toward the start of the lifecycle — threat modeling during requirements, secure design reviews before code, static and dependency scanning on every commit — instead of leaving it to a penetration test the week before launch. Everything in this chapter is arranged along that leftward pull.

11.1.4 Security as a Quality Attribute

Because security is a system property you can reason about early, it belongs in the same toolbox as the other quality attributes. Chapter 6 taught you to make a vague quality testable by writing a **quality-attribute scenario** (§6.1.4): a source, a stimulus, and a measurable response. Security scenarios read the same way — "an unauthenticated attacker submits ten thousand login attempts; the system locks the account and alerts an operator within one minute" — and they turn "be secure" into something a design can be checked against. Chapter 3's **attack trees** (§3.7.1) are the adversary's half of the same practice: model the attacker's goal as a tree of sub-goals, and each leaf is a scenario your design must answer. Threat modeling is, at its core, the requirements and design work of Chapters 3 and 6, done with an adversary in the room.

11.2 The OWASP Top 10

Design thinking tells you *how* to reason about threats; it does not tell you *which* threats are actually hurting real systems right now. For that, the field's most widely used reference is the **OWASP Top 10**, published by OWASP, a nonprofit foundation devoted to software security.[1] The Top 10 is an *awareness document*: a ranked list of the ten most critical categories of web-application security risk, assembled from empirical data — hundreds of thousands of real reported vulnerabilities — combined with a survey of practitioners. It is revised roughly every four years, and each revision is itself a snapshot of how the industry's risk profile has moved.

The current edition is the **OWASP Top 10:2025**. It was first announced as a release candidate at the OWASP Global AppSec conference in Washington, D.C., in November 2025[2] and finalized shortly afterward; as of this writing the canonical `owasp.org/Top10/` URL points to the 2025 edition and OWASP lists it as the current released version.[1] The ten categories, in rank order, are:[3]

- **A01:2025 — Broken Access Control**
- **A02:2025 — Security Misconfiguration**
- **A03:2025 — Software Supply Chain Failures**
- **A04:2025 — Cryptographic Failures**
- **A05:2025 — Injection**
- **A06:2025 — Insecure Design**
- **A07:2025 — Authentication Failures**

- **A08:2025 — Software or Data Integrity Failures**
- **A09:2025 — Security Logging and Alerting Failures**
- **A10:2025 — Mishandling of Exceptional Conditions**

Compared with the previous **OWASP Top 10:2021**,[4] the changes tell a story.[5] Two categories are new. **A03:2025 Software Supply Chain Failures** is a broadened successor to 2021's "Vulnerable and Outdated Components" — the scope now covers breakdowns anywhere in building, distributing, or updating software, not only components with a published CVE — and its promotion to third place is the clearest signal in the document that the industry's attention has moved to the supply chain (the subject of §11.4). **A10:2025 Mishandling of Exceptional Conditions** is entirely new: it names the security bugs that hide in bad error handling — fail-open logic, crashes, leaked stack traces — and it pairs naturally with Chapter 14's CrowdStrike case study, where an unhandled condition inside kernel code took down millions of machines (§14.3.5).

The rest of the diff is renaming and reranking. Server-Side Request Forgery, a standalone category in 2021, is no longer its own entry — it now lives *inside* A01 Broken Access Control. A handful of categories were renamed for emphasis ("Identification and Authentication Failures" became simply "Authentication Failures"; "Security Logging and Monitoring Failures" became "…and Alerting Failures," to stress that a log nobody acts on is worthless). And the ranks shifted: Security Misconfiguration rose to #2, while Injection — long the archetypal web vulnerability — fell to #5, a sign of how much modern frameworks now prevent it by default.

> **Pitfall.** Treating the Top 10 as a checklist to tick off. OWASP built it as an *awareness* document — the categories are broad groupings of many underlying weaknesses, and "we handled all ten" is not a security claim. Worse, memorizing the list encourages reasoning at the wrong altitude: SSRF moving under Broken Access Control and cross-site scripting living under Injection are reminders that categories are buckets of weaknesses, not one-bug-one-slot. Reason about the specific weakness in front of you, use the Top 10 to make sure no whole *class* of risk has been forgotten, and expect the list itself to change with the next edition.

11.2.1 A01: Broken Access Control

Access control enforces the policy that users can only act within their intended permissions. When it breaks, users read, modify, or destroy data that should be beyond their reach — which is why OWASP ranks it #1.[6] The failures are often mundane: an API that returns another patient's record because it trusts an ID in the URL without checking who is asking (an *insecure direct object reference*); an admin page reachable by anyone who guesses its path; a permission check performed in the browser, where the attacker controls everything, instead of on the server. The 2025 edition folds both Server-Side Request Forgery and Cross-Site Request Forgery into this category, on the reasoning that both are ultimately the system being tricked into acting outside its intended authority.

The defense is a design stance, not a patch: **deny by default**, decide authorization on the server for *every* request, and derive the acting user's identity from an authenticated session rather than from any value the client can set. Access control is also the security attribute least visible to ordinary feature testing: the system appears to work while granting access to unauthorized users. Authorization therefore needs explicit design review and dedicated scenarios such as those in §11.1.4.

11.2.2 A05: Injection

Injection is what happens when untrusted input is handed to an interpreter and part of that input gets executed as a command rather than treated as data.[7] SQL injection is the canonical case, and it remains the standard teaching example because it is easy to demonstrate, historically devastating, and completely preventable. Cross-site scripting — injecting attacker-controlled script into a page other users load — is classified here too: same mechanism, a different interpreter (the browser).

Picture the clinic scheduler looking up a patient's appointments by name. The natural, disastrous way to build the query is to paste the name straight into the SQL text:

```
# UNSAFE: the name is concatenated into the SQL text, so an input like
#   Robert'); DROP TABLE appointments; --  is parsed as SQL, not as a name.
def find_unsafe(db, name):
    query = "SELECT * FROM appointments WHERE patient_name = '" + name + "'"
    return db.execute(query).fetchall()  # A05:2025 Injection

# SAFE: the ? placeholder binds the name as a parameter; the database never
# parses it as SQL, so the same malicious input is just an unmatched name.
def find_safe(db, name):
    query = "SELECT * FROM appointments WHERE patient_name = ?"
    return db.execute(query, (name,)).fetchall()
```

The functions look nearly identical, but only the parameterized version keeps user input separate from the SQL command. In the first, the boundary between *code* (the query) and *data* (the name) is a string the attacker can rewrite; a crafted name like `Robert'); DROP TABLE appointments; --` closes the quote, ends the statement, and appends one of their own. In the second, the `?` placeholder is a **parameterized query**: the SQL text is fixed and compiled first, and the name is bound in afterward as pure data that can never change the query's structure. This is the mechanism Chapter 9's data-flow analyzers hunt for — tainted input flowing into a query — and the fix is the same everywhere, in every language and every interpreter.

> **Principle.** Never build a command by concatenating untrusted input into it. Keep code and data separate by construction — parameterized queries for databases, safe templating with output encoding for HTML, argument arrays rather than shell strings for subprocesses. Injection is a solved problem; it survives only where someone reached for string concatenation instead.

11.2.3 A06: Insecure Design

Some vulnerabilities cannot be coded away, because the flaw is in the design itself. OWASP defines **Insecure Design** as "missing or ineffective control design" — a category for weaknesses that a perfect implementation would faithfully preserve.[8] A password-reset flow that lets anyone reset an account by answering a question whose answer is public is not a coding bug; every line may be correct, and the system is still insecure because the design never included the control it needed. No amount of static analysis or line-by-line code review finds this, because there is no defect on any single line to find.

This is the category that binds security to the rest of software engineering. It says some security problems are *requirements and architecture* problems, caught earliest by the threat modeling of Chapter 3 and the design review of Chapter 9 — before code exists. Insecure Design is the standing argument against treating security as a coding-time concern: the most expensive vulnerabilities are decided at the whiteboard.

11.2.4 Learning by Doing, for Free

Reading about vulnerabilities builds recognition; exploiting and then fixing them builds understanding, and the best resources for that cost nothing. **OWASP Juice Shop** is a deliberately insecure single-page web application, free and open source, with more than a hundred hacking challenges spanning the whole Top 10 — you attack a realistic app in a safe sandbox.[9] **OWASP WebGoat** is a free, deliberately insecure application built around *guided* lessons, walking you step by step through vulnerabilities like SQL injection in an offline environment.[10] And the **PortSwigger Web Security Academy** offers a large body of free training material plus interactive labs, written by the team behind the Burp Suite testing proxy.[11]

These combine into a complete loop that costs a student nothing: read a category on the OWASP site, exploit it in Juice Shop or WebGoat, fix it using the matching page from the free **OWASP Cheat Sheet Series**,[12] and confirm your fix against a requirement from the **OWASP Application Security Verification Standard (ASVS)**, a free checklist of testable security controls.[13] Learn, attack, defend, verify — the same red-to-green rhythm Chapter 10 taught, aimed at security.

11.3 Finding Vulnerabilities, from Manual to Autonomous

Knowing the categories, how do you find *your* instances of them? Chapter 14 will introduce the three scanner families that automate the search (§14.6.1); this section previews them briefly and then turns to what is genuinely

new — security tools that use large language models to find, confirm, and even fix vulnerabilities on their own.

11.3.1 SAST, DAST, and SCA

The three families divide the work by what they look at. **Static application security testing (SAST)** is the security-focused end of the static analysis of Chapter 9 (§9.4): it reads source code without running it, hunting vulnerable *patterns* — injection sinks, unsafe deserialization, hard-coded secrets. **Dynamic application security testing (DAST)** attacks the *running* application from the outside, the way an adversary would, probing endpoints with hostile inputs and knowing nothing of the source. **Software composition analysis (SCA)** examines neither your code nor your running app but your *dependency manifest*, checking every third-party package against databases of known vulnerabilities.

The three are complementary because each sees what the others cannot, and all three carry Chapter 9's warning about **false positives and false negatives** (§9.4.2): a scanner that floods developers with false alarms trains them to ignore it, and then its true findings are lost in the noise. A security scanner earns action only by keeping its signal high enough to be worth reading.

11.3.2 AI Enters the Loop

A wave of tools now applies large language models to security. They are beginning to help *find*, *explain*, and sometimes *propose fixes for* vulnerabilities — but, as the rest of this section stresses, their outputs still require authorization, specialist security tooling, and human validation rather than standing on their own. Three of them occupy distinct points in the design space worth keeping apart.

The first *fixes* known findings. **GitHub Copilot Autofix** combines CodeQL code scanning with LLM-generated fix suggestions: when the scanner flags an alert, the model proposes a concrete patch for the developer to review. It became generally available in August 2024; GitHub reported that it produced fixes for more than 90% of alert types across JavaScript, TypeScript, Java, and Python, and that during the beta developers using it remediated vulnerabilities roughly three times faster overall — about seven times faster for cross-site scripting and twelve times faster for SQL injection — than by hand. It runs on all public repositories on GitHub.com without a separate subscription; private repositories need a code-security license.[14] Autofix does not *find* new classes of bug; it shortens the distance from a known alert to a merged fix.

The second *hunts* for unknown bugs. **Big Sleep**, a collaboration between Google Project Zero and Google DeepMind that grew out of the earlier "Project Naptime" framework, is an AI agent that searches real code for vulnerabilities. In late 2024 Google described it finding an exploitable stack buffer underflow in the widely used **SQLite** database engine — a flaw Google characterized as the first real-world vulnerability discovered by an AI agent, caught in a development branch and fixed before it reached a release.[15] In 2025 Google reported that Big Sleep, combined with threat intelligence, helped identify and pre-empt in-the-wild exploitation of a separate

critical SQLite vulnerability, CVE-2025-6965, that had been known only to attackers.[16] These are early results, but they show AI moving from fixing flagged bugs to discovering unknown ones.

The third *validates* by attacking. **Strix** is an open-source project (from the developer UseStrix) that describes itself as "autonomous AI penetration testing agents that act just like real hackers — they run your code dynamically, find vulnerabilities, and validate them through actual proofs-of-concept."[17] Its design is what makes it interesting for a software-engineering course. Strix runs a *multi-agent* architecture — specialized agents work different targets in parallel and share what they find — and it arms those agents with real tooling: an HTTP interception proxy, browser automation for client-side attacks, shell and Python environments for building exploits, plus static and dynamic analysis. It is *model-agnostic*, driving whichever LLM provider you configure, and it ships as a pip-installable command-line tool that sandboxes agent execution in Docker containers, with a mode that scopes a scan to the diff in a pull request so it can run inside CI.[17] Help Net Security's coverage corroborated the graph-based multi-agent design when the project drew attention in late 2025.[18]

The critical design difference from a static linter is *validation*. A SAST tool reports a "possible SQL injection"; Strix tries to actually exploit it and reports a working proof-of-concept or nothing. That closes the false-positive gap from the other side — a confirmed exploit is evidence, not a guess. It also opens a different set of hazards that a first course must name plainly.

> **Pitfall.** *Autonomous attack tools are power tools, and they cut both ways.* Three rules are non-negotiable. **First, authorization.** Strix's own documentation is blunt: "Only test apps you own or have permission to test. You are responsible for using Strix ethically and legally." Running an autonomous exploit agent against a system you are not authorized to test can be a crime; scope, written permission, and rules of engagement come *before* any scan. **Second, validation is still yours.** A finding — even one with a proof-of-concept — is the start of triage, not the end: reproduce it, judge its real business impact, and remember that these agents can over-report, miss logic flaws, and hallucinate. **Third, they do not replace secure design.** An agent that finds a vulnerability late is far more expensive than a design decision that made the bug impossible; treat Strix, Big Sleep, and Autofix as authorized-testing and research tools that complement threat modeling, least privilege, and input validation — never as substitutes for them. Adopt any of them the way Chapter 9 said to adopt a static analyzer: as a tool whose output you must keep trusting, which means keeping it honest.

11.4 The Software Supply Chain: A World of Weak Links

Every technique so far assumes the code under review is *your* code. It mostly is not. The single largest shift in what "your software" means is that a modern application is overwhelmingly assembled from open-source components you did not write, chosen by people who are not on your team, updated on a schedule you do not control. This section is the intellectual center of the chapter, because it is where security stops being about the code you can read and becomes about the trust you extend.

11.4.1 The Scale Problem

Start with your own habit. When you run `npm install` or `pip install`, you pull in not just the package you named but everything *it* depends on, and everything those depend on, down through a tree most developers never inspect. The numbers are stark. Hastings' 2024 dissertation on supply-chain attacks reports that open-source software is used in 99% of software applications, that a simple "hello-world" Ruby on Rails application requires 966 packages across three package managers, and that a React project pulls in 1,213 — the vast majority of them transitive dependencies nobody on the team ever chose.[19] Black Duck's 2026 Open Source Security and Risk Analysis report, based on audits of hundreds of commercial codebases, puts hard current figures on the same picture: as of the 2026 report, open source appears in 98% of codebases, **64% of the components in a typical codebase are transitive** — pulled in indirectly, selected by no one on the team — and 93% of codebases contain components with no development activity in over two years.[20] The same report found an average of 581 known vulnerabilities per codebase, with 78% of codebases carrying at least one high-risk vulnerability and 44% carrying a critical one.[20]

The reframe for a student is immediate: the package manifest at the root of your sprint project is a *trust decision*, not a convenience. Most of the code you ship is other people's, and its vulnerabilities are your vulnerabilities.

> **Pitfall.** *Automatic updates are a supply-chain attack vector.* The same **semantic-versioning** ranges you learn as good practice — accept any compatible patch or minor release — can also create supply-chain risk: your build may automatically accept a malicious patch or minor release as soon as an attacker publishes one. As Hastings' dissertation puts the inversion: "It used to be patch on a Friday to prevent a breach on Monday. Now, it is patch on Friday and a breach on Monday."[19] The defenses are cultural before they are technical: pin exact versions with a lockfile, review dependency-update pull requests like any other change, and treat a new dependency as an engineering decision with a threat model attached.

11.4.2 Weak Links

Why is the open-source supply chain so exposed? Because trust in it flows through people, and people are the weakest link. The dissertation adopts the term **weak links** from research by Zahan and colleagues, who studied the npm ecosystem and identified structural weaknesses in how packages are maintained — including maintainers whose account-recovery email ran through *expired domains* anyone could re-register — and found, strikingly, that compromising about twenty high-profile developer accounts could poison roughly half of the npm ecosystem.[21] A tiny number of overextended, under-supported maintainers hold up an enormous amount of the world's software.

The dissertation's blunt summary of the adoption mindset is worth keeping: open-source components "might be free, but they are free as in puppy."[19] A package is an ongoing responsibility to vet, monitor, and patch — a living thing you have taken into your home. OWASP recognized the whole problem in 2025 by promoting it to a dedicated Top 10 category, **A03:2025 Software Supply Chain Failures**, whose scope deliberately spans

dependencies, build systems, and distribution infrastructure — the entire attack surface from *source* to *build* to *distribution* to *dependency*.[22] The two case studies below are opposite failure modes on that same surface.

There is a subtler weak link worth naming, because it attacks the *incentives* that keep the commons healthy rather than any one package. Open source works as a bargain: you may use this code freely, and in return you honor its license — you keep the attribution, you share alike where the license says so, you give something back. A satirical site called **MALUS** dramatizes what happens when that bargain is abandoned. It advertises, deadpan, a service that feeds your dependency manifest to an AI, "recreates" each library as "legally distinct" code stripped of every attribution and copyleft obligation, and hands it back under a proprietary license — *"finally, liberation from open source license obligations."*[23] The site is a joke; the pressure it names is not. If firms could launder away the reciprocity that open source runs on — taking the value while shedding the obligation — the volunteers who maintain the world's dependencies would have even less reason to keep doing unpaid, thankless work, and the pool of overextended maintainers you will meet in the xz story (§11.4.3) would only thin further. Security tooling can verify a component's provenance; it cannot, by itself, sustain the human ecosystem that produces components worth verifying. That sustainability is a supply-chain security concern too.

> **Pitfall.** Treating open-source components as a free resource to extract rather than a commons to sustain is a slow-acting supply-chain risk. Every dependency you take on rests on a maintainer's continued willingness to maintain it; honoring licenses, reporting bugs, upstreaming fixes, and — where you can — funding the projects you depend on are how you keep the components you rely on alive and watched, not charity.

11.4.3 Two Case Studies

Case study. *Log4Shell (CVE-2021-44228), December 2021.* Apache Log4j is a logging library so common that it sits, usually as a transitive dependency, inside a large fraction of the Java software on earth. On December 10, 2021, a vulnerability was published with the maximum possible CVSS base score of **10.0**.[24] The cause was a feature almost nobody knew was on: Log4j versions from 2.0-beta9 through 2.14.1 would, by default, interpret special lookup syntax *inside the strings they logged*, and one lookup type (JNDI) could be pointed at an attacker-controlled server to download and execute arbitrary code. Because applications log untrusted input all the time — a username, a search term, a User-Agent header — an attacker could trigger remote code execution simply by getting a malicious string logged, with no privileges and no user interaction.[24]

Log4Shell was catastrophic for a supply-chain reason: the dangerous behavior lived in a *ubiquitous transitive dependency*, so countless teams were vulnerable through software they never realized contained Log4j at all. Remediation was staged — 2.15.0 disabled the behavior, 2.16.0 removed it — and the urgency was extraordinary: CISA added Log4Shell to its Known Exploited Vulnerabilities catalog with a federal remediation deadline of December 24, 2021,[25] two weeks after disclosure. The lesson is that you cannot patch a dependency you do not know you have — which is the argument for the software bills of materials in §11.4.5.

Case study. *The xz-utils backdoor (CVE-2024-3094), March 2024.* Where Log4Shell was an accident, xz was an attack — a patient, multi-year campaign to plant a backdoor in software the world runs by default. xz-utils is a compression library that ships in essentially every Linux distribution, and `liblzma` is linked, indirectly, into the OpenSSH server on many systems. The story is fundamentally a *human* one.[26]

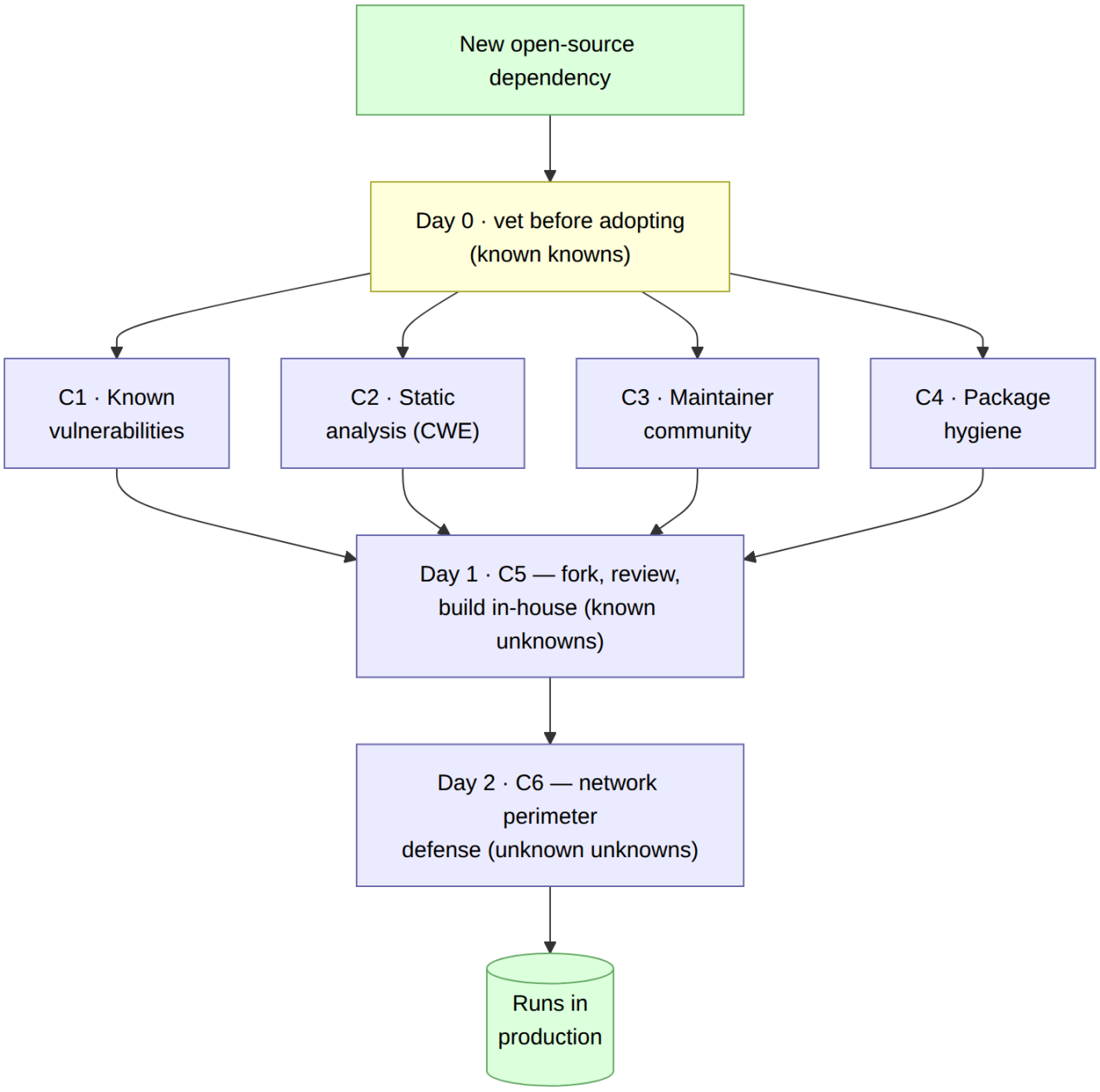
The library was maintained, for years, by one overworked volunteer, Lasse Collin. Beginning in 2021, a new contributor calling themselves "Jia Tan" started submitting patches. Over 2022, sock-puppet personas appeared in the mailing list to pressure Collin into handing over more control — complaining about slow progress and pushing for a new maintainer — while Collin, candidly, wrote that "I haven't lost interest but my ability to care has been fairly limited mostly due to longterm mental health issues."[26] Jia Tan was gradually granted trust, became a co-maintainer, and in February and March 2024 released versions 5.6.0 and 5.6.1 whose *release tarballs* — the packaged downloads, signed by Jia Tan, but not the plain git repository — contained an obfuscated backdoor hidden inside files disguised as compression test data.[27] Assembled at build time, it modified `liblzma` so that a compromised sshd would run attacker commands, before authentication.[28]

It was caught almost by accident. Andres Freund, a PostgreSQL developer at Microsoft, noticed that SSH logins on a test system were about half a second slower than they should be and that a memory-analysis tool was flagging errors in `liblzma`. He refused to shrug it off, traced the anomaly, and disclosed the backdoor on a public security mailing list on March 29, 2024.[29] The world's most sophisticated known supply-chain attack was undone by one engineer who would not ignore half a second of unexplained latency.

Read as a pair, the cases bracket the problem. Log4Shell is an *accidental* vulnerability in a ubiquitous dependency — a design failure. xz is a *deliberate* backdoor inserted through a trust-and-process failure aimed squarely at a burned-out solo maintainer. Same blast radius, opposite root causes — which is why the defenses must include both scanners that find known-bad code *and* controls that constrain how you extend trust in the first place. It also ties supply-chain security straight to this book's themes of sustainable teamwork: bus factor, code review as a security control, and observability curiosity are not soft skills here — they are the controls that failed and the one that saved the day.

11.4.4 Continuous Verification: A Framework of Six Controls

The two cases motivate a repeatable way to vet a dependency *before* and *after* you adopt it, rather than trusting it and hoping. Hastings' dissertation — the relevant chapter of which was published with Walcott as "Continuous Verification of Open Source Components in a World of Weak Links" — proposes exactly that: a framework of six controls, C1 through C6, organized across **Day 0**, **Day 1**, and **Day 2** operations and mapped to the familiar progression of *known knowns*, *known unknowns*, and *unknown unknowns*.[19] [30]



The **Day 0** controls vet a component before you adopt it, and they can be largely automated. **C1** checks the package and its dependencies for known vulnerabilities and weighs how many other projects depend on it. **C2** runs static analysis against catalogs of known weakness types (CWE). **C3** examines the maintainer *community* — how many organizations maintain it, how active it has been in the last ninety days — because a one-person project is a weak link waiting to happen. **C4** checks package *hygiene*: signed commits, signed releases, branch protection, the absence of dangerous build workflows. In the dissertation these are computed from a subset of OpenSSF Scorecard heuristics plus a custom dependents metric, and the combined score maps to a risk level — and, importantly, to an expected *time to repair*: a low-risk component's community tends to fix problems in hours, a high-risk one's in weeks or never.[19]

The **Day 1** control, **C5**, is a policy for what you let into your build: no artifact you did not build from reviewed source is included — the component is forked, code-reviewed, and built in-house, with an automated pipeline to re-review each new upstream release. The **Day 2** control, **C6**, is a production perimeter defense that inspects outbound network traffic against an allow list, to catch a component that "phones home." The dissertation's evaluation applied these controls in tabletop exercises against five real compromised packages and found the framework flagged the appropriate risk level in four of five cases, versus one of five for OpenSSF Scorecard alone — while candidly noting that the controls "excel at vetting single components" but grow costly for whole frameworks with deep recursive dependencies, and that the Day 1 and Day 2 controls offer the most protection at the highest cost.[19] The mental model is what to carry away: **vet before you adopt (Day 0), control what you ingest (Day 1), monitor in production (Day 2)** — a lifecycle arc that lines up with this book's own progression from design through delivery to operations.

11.4.5 The Modern Supply-Chain Toolbox

You do not have to build C1–C6 by hand; a growing set of free tools implements pieces of it, and each answers a different question about an artifact.

How healthy is the upstream project? **OpenSSF Scorecard** runs eighteen automated checks — code review, branch protection, pinned dependencies, signed releases, dangerous workflows, known vulnerabilities, and more — scoring any public repository on how well it follows security practices.[31] It is the tooling behind the dissertation's Day 0 controls, and you can run it against a package you are considering for your project.

What is known to be wrong with my dependencies? **OWASP Dependency-Check** is a free SCA tool that scans a project's dependencies for publicly disclosed vulnerabilities and runs in CI; **OWASP Dependency-Track** is its organization-scale companion, continuously monitoring supply-chain risk across a whole portfolio by analyzing bills of materials.[32] [33]

What is actually inside the artifact? A **software bill of materials (SBOM)** is a complete inventory of every component in a release — the answer to "do we even have Log4j?" Two open standards dominate: **CycloneDX**, an OWASP-originated format now standardized as ECMA-424,[34] and **SPDX**, a Linux Foundation format that is an ISO/IEC international standard (ISO/IEC 5962).[35] An SBOM is what turns "patch the dependency you did not know you had" from impossible into a database query.

How was the artifact built, and who produced it? **SLSA** (Supply-chain Levels for Software Artifacts) is a framework of build-integrity levels — from **L0** (no guarantees) through **L1** (provenance exists but is forgeable), **L2** (signed provenance from a hosted builder), to **L3** (a hardened build platform with strong tamper resistance) — that grades how trustworthy an artifact's origin story is.[36] And **Sigstore** makes signing that artifact practical by replacing long-lived signing keys with "keyless," identity-based signing: ephemeral keys are bound to an identity, and every signing event is recorded in an immutable public transparency log, sidestepping the key-theft and key-management problems that made traditional artifact signing so rare.[37]

Mapped onto the attack surface, the tools stop being a list and become a system: SBOMs answer *what is in it*, Sigstore answers *who made it*, SLSA provenance answers *how it was built*, and Scorecard answers *how healthy the source project is* — the same source-to-distribution progression that A03 spans.[38] All of them are free, and OpenSSF even publishes a free, certificate-bearing course, "Developing Secure Software" (LFD121), that a student can put on a résumé.[39] Supply-chain security is an employable skill, not abstract policy.

11.5 Building Security In

The through-line of this chapter is that security is decided long before an attacker shows up. The industry has codified that idea into a reference practice. The **NIST Secure Software Development Framework (SSDF)**, published as NIST Special Publication 800-218, defines a core set of high-level secure-development practices to fold into any lifecycle — with the explicit goals of reducing vulnerabilities in released software, mitigating the impact of the ones that slip through, and giving software producers and purchasers a shared vocabulary.[40] That last goal matters for the profession: because the SSDF is a common language, secure development escapes the "optional best practice" trap and becomes something a contract can require — a natural seed, too, for a research paper on how SBOM mandates and SLSA adoption are reshaping procurement.

A handful of habits carry most of the weight in day-to-day code. **Secure defaults**: ship locked down, so that the safe configuration is the one you get by doing nothing — Log4Shell was, at heart, an *insecure* default. **Least privilege**: every user, service, and token gets the minimum access it needs and no more, so that a compromise of one part cannot reach the whole. **Secrets management**: credentials belong in a secrets manager or injected configuration, never in source — the secrets-scanning gate of Chapter 14 (§14.6.3) exists because a key committed to git history must be treated as compromised the moment it lands.

Finally, assume you will be attacked anyway, and instrument for it. **A09:2025 Security Logging and Alerting Failures** addresses two related problems: without logging, a team may not detect an attack; without *alerting*, it may not respond while the breach is still in progress.[41] **A10:2025 Mishandling of Exceptional Conditions** covers fail-open logic, crashes on unexpected input, and stack traces exposed to users.[42] Careful error handling and actively monitored logs are therefore security controls. Chapter 14's CrowdStrike case shows the scale of damage that poor exceptional-condition handling can cause.

> **Principle.** Design so that the *default* path is the secure one and the *privileged* path is the exception. Every control that must be remembered will eventually be forgotten; every control that is automatic — a safe default, a scoped token, a pipeline gate — holds even on the day everyone is tired and shipping fast.

11.6 Conclusion

Security draws together practices introduced throughout the book. The attack trees and STRIDE checklists of Chapter 3 were security work; the quality-attribute scenarios of Chapter 6 made “secure” measurable; the static analysis of Chapter 9 and the tests of Chapter 10 were catching vulnerabilities all along; and the pipeline of Chapter 14 will turn those checks into gates no change can skip. What this chapter added is the adversarial frame that unifies them — the assumption that someone is actively working to make your system misbehave — and the recognition that most of the software you ship was written by strangers, so trust in the supply chain is now among the largest security decisions your team makes. The OWASP Top 10 tells you which risks are real this year; parameterized queries and secure defaults close whole categories by construction; AI tools like Copilot Autofix, Big Sleep, and Strix are beginning to find and fix bugs at machine speed, under the standing conditions of authorization and human validation; and the continuous-verification framework of §11.4 gives you a way to vet a dependency across its whole life rather than trusting it on faith. Log4Shell and xz-utils are the permanent record of what happens when you cannot see inside your own software and when a single overloaded maintainer holds up the world.

If the chapter compresses to one sentence, it is this: **treat security as a property you design in, verify continuously, and extend deliberately to every dependency you trust — because an adversary only has to find the one weak link you forgot to check.**

Sources

[1] OWASP Foundation, *OWASP Top Ten* project page (nonprofit stewardship; current released edition is 2025) and the canonical redirect. owasp.org/www-project-top-ten, owasp.org/Top10.

[2] OWASP, *OWASP Top 10* supplemental site (release-candidate announcement, OWASP Global AppSec, Washington, D.C., November 2025). owasptopten.org.

[3] OWASP, *OWASP Top 10:2025* (category list). owasp.org/Top10/2025.

[4] OWASP, *OWASP Top 10:2021* (category list). owasp.org/Top10/2021.

[5] OWASP, *OWASP Top 10:2025 — Introduction* (summary of changes from 2021). owasp.org/Top10/2025/0x00_2025-Introduction.

[6] OWASP, *A01:2025 — Broken Access Control*. owasp.org.

[7] OWASP, *A05:2025 — Injection*. owasp.org.

[8] OWASP, *A06:2025 — Insecure Design*. owasp.org.

[9] OWASP, *OWASP Juice Shop* project page. owasp.org/www-project-juice-shop.

[10] OWASP, *OWASP WebGoat* project page. owasp.org/www-project-webgoat.

[11] PortSwigger, *Web Security Academy*. portswigger.net/web-security.

[12] OWASP, *OWASP Cheat Sheet Series*. cheatsheetseries.owasp.org.

[13] OWASP, *Application Security Verification Standard (ASVS)*. owasp.org.

[14] GitHub, *Copilot Autofix for CodeQL code scanning alerts is now generally available* (changelog, August 14, 2024) and *Found means fixed* (GitHub Blog). github.blog.

[15] Google Project Zero, *From Naptime to Big Sleep: Using Large Language Models to Catch Vulnerabilities in Real-World Code* (October 2024). projectzero.google.

[16] Google, *Google's latest AI security announcements* (July 2025) — the originating announcement that Big Sleep, working from Google Threat Intelligence, identified CVE-2025-6965 (SQLite, CVSS 7.2) before in-the-wild exploitation; Google characterizes it as the first time an AI agent directly foiled an imminent exploit. blog.google. CVE record: nvd.nist.gov.

[17] usestrix/strix — GitHub repository and README (Apache-2.0; autonomous AI penetration-testing agents; self-description and ethics statement). github.com/usestrix/strix.

[18] Help Net Security, *Strix: Open-source AI agents for penetration testing* (November 17, 2025). helpnetsecurity.com.

[19] Thomas G. Hastings, *Combating Source Poisoning and Next-Generation Software Supply Chain Attacks Using Education, Tools, and Techniques*, Ph.D. dissertation, University of Colorado Colorado Springs (2024). archives.mountainscholar.org.

[20] Black Duck, *2026 Open Source Security and Risk Analysis (OSSRA) Report* (February 2026). blackduck.com.

[21] Nusrat Zahan et al., *What Are Weak Links in the npm Supply Chain?*, arXiv:2112.10165 (2022; ICSE-SEIP 2022). arxiv.org/abs/2112.10165.

[22] OWASP, *A03:2025 — Software Supply Chain Failures*. owasp.org.

[23] MALUS — *"Finally, liberation from open source license obligations"* (satirical site). malus.sh. A parody of AI "clean-room" services claiming to launder open-source licensing obligations; accessed 2026.

[24] NIST National Vulnerability Database, *CVE-2021-44228* (Log4Shell; CVSS 3.1 base score 10.0; published December 10, 2021). nvd.nist.gov.

[25] Cybersecurity and Infrastructure Security Agency (CISA), *Known Exploited Vulnerabilities Catalog* — Log4Shell (CVE-2021-44228) was added under Binding Operational Directive 22-01 with a remediation due date of December 24, 2021. cisa.gov.

[26] Russ Cox, *Timeline of the xz open source attack* (research.swtch.com), with links to the primary mailing-list messages. research.swtch.com/xz-timeline.

[27] Tukaani Project (Lasse Collin), *XZ Utils backdoor* incident page. tukaani.org/xz-backdoor.

[28] NIST National Vulnerability Database, *CVE-2024-3094* (xz-utils backdoor; CVSS 10.0; published March 29, 2024). nvd.nist.gov.

[29] Andres Freund, *backdoor in upstream xz/liblzma leading to ssh server compromise*, oss-security mailing list (March 29, 2024). openwall.com.

[30] T. Hastings and K. R. Walcott, *Continuous Verification of Open Source Components in a World of Weak Links*, IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), 2022, pp. 201–207. DOI 10.1109/ISSREW55968.2022.00068; author's copy.

[31] OpenSSF, *Scorecard* (eighteen automated security checks for open-source projects). scorecard.dev.

[32] OWASP, *Dependency-Check* (free SCA tool). owasp.org.

[33] OWASP, *Dependency-Track* (continuous SBOM-based component-analysis platform). owasp.org.

[34] CycloneDX, OWASP/Ecma International Bill of Materials standard (ECMA-424). cyclonedx.org.

[35] SPDX, Linux Foundation SBOM standard (ISO/IEC 5962:2021). spdx.dev.

[36] OpenSSF, *SLSA — Supply-chain Levels for Software Artifacts* (build track levels L0–L3). slsa.dev.

[37] Sigstore, documentation on keyless, identity-based signing (cosign, Fulcio, Rekor). docs.sigstore.dev.

[38] Google Cloud, *Software supply chain threats* (attack vectors mapped across source, build, dependency, and deployment). cloud.google.com.

[39] OpenSSF, *Developing Secure Software* (LFD121), free online course with certificate. openssf.org/training/courses.

[40] NIST, *Secure Software Development Framework (SSDF) Version 1.1*, Special Publication 800-218 (February 2022). csrc.nist.gov.

[41] OWASP, *A09:2025 — Security Logging and Alerting Failures*. owasp.org.

[42] OWASP, *A10:2025 — Mishandling of Exceptional Conditions*. owasp.org.

- **Key takeaways** are summarized above in §11.6.
- Continue to the Exercises.
- Go deeper with the Open Resources for this chapter.

Chapter 11 — Exercises

Exercises are graded by depth: **[warm-up]** checks understanding, **[analysis]** asks you to reason. Several exercises require *actual work* — running a scanner, reading a primary source, scoring a real package — not just prose. Show the work, not only the answer.

Concepts

1. **[warm-up]** Define *threat*, *vulnerability*, and *exploit* in one sentence each, then classify these three items as one of the three: a missing authorization check on an admin endpoint; a competitor who wants your customer list; a script that dumps that list by calling the unprotected endpoint (§11.1).

2. **[warm-up]** Name the OWASP Top 10:2025 category that best fits each of these, and justify in one clause: (a) a URL parameter `?userId=1043` that any logged-in user can change to read another patient's record; (b) a search box that runs `SELECT … WHERE name = '` + input + `'`; (c) a password-reset flow that emails a token but never expires it (§11.2).

3. **[warm-up]** In one sentence each, distinguish **SAST**, **DAST**, and **SCA**, and say which of the three would have had the best chance of flagging the Log4Shell dependency *before* it was exploited (§11.3, §11.4).

4. **[warm-up]** Explain "shift left" to a teammate who thinks security is the penetration test the week before launch. Give one concrete practice from an *earlier* phase that catches a class of defect the pen test would miss (§11.1).

Analysis

5. **[analysis]** Take the injection example from §11.2 and, in your project's language, write both versions: the string-concatenated query and the parameterized query. Then write one input that returns every row from the vulnerable version, and confirm the same input is harmless against the parameterized one. Submit the two queries, the malicious input, and one sentence on *why* the parameterized version is safe. **Perform this only against your own local database or a sandbox you are explicitly authorized to test** — never against a running system you do not own, including your own team's deployed project without the team's agreement.

6. **[analysis]** Run **OpenSSF Scorecard** (or read its published report) against one open-source dependency your project actually uses. Report its aggregate score and the two lowest-scoring checks. Then argue, in a paragraph, whether that score alone would tell you the package is safe to adopt — and what the six-control framework of §11.4 would add that the score does not (§11.4).

7. **[analysis]** Read Andres Freund's original oss-security disclosure of the xz-utils backdoor (linked in Open Resources). In your own words, identify the *human* weak link the attack exploited and name which of the dissertation's six controls (C1–C6), if any, would plausibly have caught it — and which would not, and why. This is the point of the case study: some attacks are social, not technical (§11.4).

8. **[analysis]** Pick one package your project depends on and walk it through the Day 0 controls (C1–C4) from §11.4: does it have known CVEs; who maintains it and how recently; is it signed; does it have branch protection? Assign it Low / Medium / High risk and state the Time-to-Repair band that implies. You do not need every answer to be perfect — the exercise is learning to *ask* the four questions before you type `install` (§11.4).

9. **[analysis]** AI pentest tools such as Strix validate findings by building a working proof-of-concept. Explain why that validation step matters more for an *autonomous* tool than for a human tester, and describe one situation in which running such a tool would be unethical or illegal even though it is technically easy (§11.3). Ground your answer in the authorization principle, not vibes.

10. **[analysis]** Choose one OWASP Top 10:2025 category and write a single **quality-attribute scenario** (§6.1.4) for it: the stimulus (an attacker's action), the environment, the response your system should make, and a measurable response criterion. This is how a vague "be secure" becomes something you can design toward and test (§11.1, §6.1.4).

Chapter 11 — Open Resources

Free, open-licensed, or freely accessible materials that reinforce this chapter. Types: 📘 open text · 🎓 course · 📄 primary source / standard · 🎥 video · 🛠️ tool. Licenses vary and are noted where known. Each entry notes its license or access terms; when in doubt, check the linked page.

The OWASP Top 10 and hands-on practice

- 📄 **OWASP Top 10:2025** — owasp.org/Top10. The current edition of the industry's shared vulnerability vocabulary, with a page per category (root causes, examples, prevention, and mapped CWEs). Start here, then read the two or three categories most relevant to your project. *License: CC BY-SA 4.0.*
- 🛠️ **OWASP Juice Shop** — owasp.org/www-project-juice-shop. A deliberately insecure single-page web app with 100+ challenges spanning the Top 10 and beyond; run it locally and try to break it. *License: MIT.*
- 🛠️ **OWASP WebGoat** — owasp.org/www-project-webgoat. A deliberately insecure Java app with *guided* lessons — a gentler on-ramp than Juice Shop for a first encounter with SQL injection and friends. *License: open source.*
- 🎓 **PortSwigger Web Security Academy** — portswigger.net/web-security. Free, hands-on labs and readings from the team behind Burp Suite; the closest thing to a canonical free web-security course. *Access: free; account only tracks progress.*
- 📄 **OWASP Cheat Sheet Series** — cheatsheetseries.owasp.org. ~150 concise, high-value guidance pages (authentication, cryptography, cloud, AI security). The reference you reach for while writing a control, not while studying. *License: CC BY-SA 4.0.*
- 📄 **OWASP Application Security Verification Standard (ASVS 5.0)** — owasp.org/www-project-application-security-verification-standard. A checklist-style standard for *testing* web-application security controls at three rigor levels. *License: CC BY-SA 4.0.*

AI in security

- 🛠️ **Strix — open-source autonomous AI pentest agents** — github.com/usestrix/strix. Agents that run a target dynamically, find vulnerabilities, and validate them with proofs-of-concept; CLI + Docker, model-agnostic, with CI/pull-request integration. **Authorized testing only.** *License: Apache-2.0.*
- 📄 **Google Project Zero — "From Naptime to Big Sleep"** — projectzero.google/2024/10/from-naptime-to-big-sleep.html. The primary write-up of the first real-world vulnerability found by an AI agent (in SQLite). *Access: free.*
- 📄 **GitHub — Copilot Autofix general availability** — github.blog changelog. The announcement with the CodeQL-plus-LLM remediation numbers cited in §11.3. *Access: free.*

The software supply chain

- 📄 **Andres Freund — "backdoor in upstream xz/liblzma"** (oss-security, 29 Mar 2024) — openwall.com/lists/oss-security/2024/03/29/4. The original disclosure of the xz-utils backdoor. Read the primary source before any blog summary. *Access: public mailing-list archive.*
- 📄 **Russ Cox — "Timeline of the xz open source attack"** — research.swtch.com/xz-timeline. A meticulously sourced timeline of the multi-year social-engineering campaign behind CVE-2024-3094. *Access: free.*
- 📄 **NVD — CVE-2021-44228 (Log4Shell)** — nvd.nist.gov/vuln/detail/CVE-2021-44228 — and **CVE-2024-3094 (xz)** — nvd.nist.gov/vuln/detail/CVE-2024-3094. The authoritative records for the two case studies. *Access: U.S. government work, public domain.*
- 🛠️ **OpenSSF Scorecard** — scorecard.dev. Automated checks (code review, branch protection, pinned dependencies, signed releases, and more) that score an open-source project's security posture — the tool the chapter's Day 0 controls build on. *License: Apache-2.0.*
- 🛠️ **OWASP Dependency-Check** — owasp.org/www-project-dependency-check — and **Dependency-Track** — owasp.org/www-project-dependency-track. Software-composition analysis: the first scans a build for known-vulnerable dependencies, the second continuously monitors an SBOM across a whole portfolio. *License: Apache-2.0.*
- 📄 **SLSA — Supply-chain Levels for Software Artifacts** — slsa.dev. The framework of build/source integrity levels (L0–L3) referenced in §11.4. *License: open.*
- 🛠️ **Sigstore** — docs.sigstore.dev. Keyless, identity-based signing of artifacts with a public transparency log — the modern answer to "who built this, and can I verify it?" *License: Apache-2.0.*
- 📄 **CycloneDX** — cyclonedx.org — and **SPDX** — spdx.dev. The two mainstream SBOM standards (ECMA-424 and ISO/IEC 5962 respectively). *License / standard: open.*
- 🎭 **MALUS** (satire) — malus.sh. A deadpan parody of an AI "clean-room" service that claims to launder away open-source license and attribution obligations. The joke makes the §11.4.2 point vivid: the open-source supply chain runs on reciprocity, and eroding it is its own slow supply-chain risk. *Read it, then read a real license.*

Standards, frameworks, and courses

- 📄 **NIST SP 800-218 — Secure Software Development Framework (SSDF) v1.1** — csrc.nist.gov/pubs/sp/800/218/final. The common vocabulary of secure-development practices referenced in §11.5. *Access: U.S. government work, public domain.*
- 🎓 **OpenSSF — "Developing Secure Software" (LFD121)** — openssf.org/training/courses. A free, self-paced online course (~14–18 hours) with a free certificate; also on edX. The single best free starting point for a student who wants breadth. *Access: free, including the certificate.*
- 📄 **Google Cloud — Software supply chain security docs** — cloud.google.com/software-supply-chain-security/docs/attack-vectors. Maps source, build, dependency, and deployment threats onto the SLSA

framework. *License: CC BY 4.0.*

- 📄 **Hastings & Walcott, “Continuous Verification of Open Source Components in a World of Weak Links,” ISSREW 2022** — doi.org/10.1109/ISSREW55968.2022.00068. The published paper behind §11.4’s six-control framework. *Access: IEEE (publisher page; author preprints are widely indexed).*

License note

Linked resources remain under their own licenses; this page is CC BY-SA 4.0.

Chapter 12 — Quality Metrics

> **Where we are.** Every earlier chapter asked you to *do* something well: elicit requirements, design for change, review code, test thoroughly, secure it. This chapter asks a different question: *how do you know whether it worked?* Opinions about quality are cheap and contradictory. Measurement replaces "I think the build got better" with "the defect-removal efficiency rose from 84% to 91%, and here is the confidence interval." This is the most quantitative chapter in the book. We build the statistical toolkit — scales of measurement, boxplots, variance, distributions, confidence intervals, and linear regression — and we ground every formula in a small dataset we carry from start to finish.

You cannot manage what you cannot see, and in software almost nothing is directly visible. You cannot hold "maintainability" in your hand or point at a cubic centimeter of "reliability." **Metrics** are how we make invisible attributes visible enough to reason about, compare, and improve. But metrics are also dangerous: a badly chosen number will be optimized at the expense of the very thing you cared about. So this chapter is as much about *choosing* and *interpreting* metrics honestly as it is about computing them. Get the statistics right, and get the humility right too.

Throughout, we carry one running dataset. A team runs code reviews and records, for a set of releases, how many **customer-found defects** each release produced in its first six months. Sorted, the eleven values are:

$$2,\ 4,\ 5,\ 5,\ 7,\ 8,\ 9,\ 10,\ 12,\ 14,\ 23$$

We will summarize this dataset with a boxplot and a histogram, compute its variance and standard deviation, wrap a confidence interval around its mean, and — using a second, paired dataset — fit a regression line. Keep these numbers in view; they anchor everything abstract that follows.

> **Two paths through this chapter.** For a first pass, §12.1–§12.5 stand alone: meaningful metrics, software quality, charts, defect measurement, and a real improvement case study — nothing beyond arithmetic. §12.6–§12.9 (dispersion, probability distributions, confidence intervals, and regression) are the quantitative backbone for courses that want students to *defend* a metric claim statistically; they assume comfort with algebra and reward a second week. If you take only the first path, return for §12.8 before you claim two releases differ in quality.

12.1 Meaningful Metrics

12.1.1 Metrics Quantify Attributes

An **attribute** is a property of some entity you care about: the *size* of a module, the *duration* of a build, the *effort* to fix a bug, the *satisfaction* of a customer. A **metric** (more precisely, a *measure*) is a rule that assigns a number or symbol to an attribute so that the assignment preserves something true about the world. Everything rides on that last clause. If module A really is more complex than module B, then a valid complexity metric must assign A a larger number than B. When the numbers stop tracking the reality, the metric has failed, no matter how carefully it is computed.

Measurement theory names three entities you should keep distinct:

- The **entity** — the thing being measured (a file, a release, a review session).
- The **attribute** — the specific property of interest (its size, its defect count).
- The **metric** — the mapping from attribute to value (lines of code, defects per KLOC).

A single entity has many attributes, and one attribute admits many metrics. A source file has size, age, complexity, and churn; "size" alone can be measured as bytes, lines, statements, or tokens. Confusing these is the root of most measurement nonsense. "The file is 400" is meaningless until you say *400 what, measuring which attribute.*

> **Definition.** A metric is **valid** if entities that differ in the attribute differ in the metric in the same direction (it *represents* the attribute), and **reliable** if repeating the measurement gives the same value. A metric can be reliable but invalid — counting lines of code is perfectly repeatable, yet it is a poor measure of "programmer productivity."

12.1.2 Selecting Useful Metrics

Since almost anything can be measured, teams must choose measures that support useful decisions. The hard part is deciding what is *worth* measuring. A useful metric passes several tests at once:

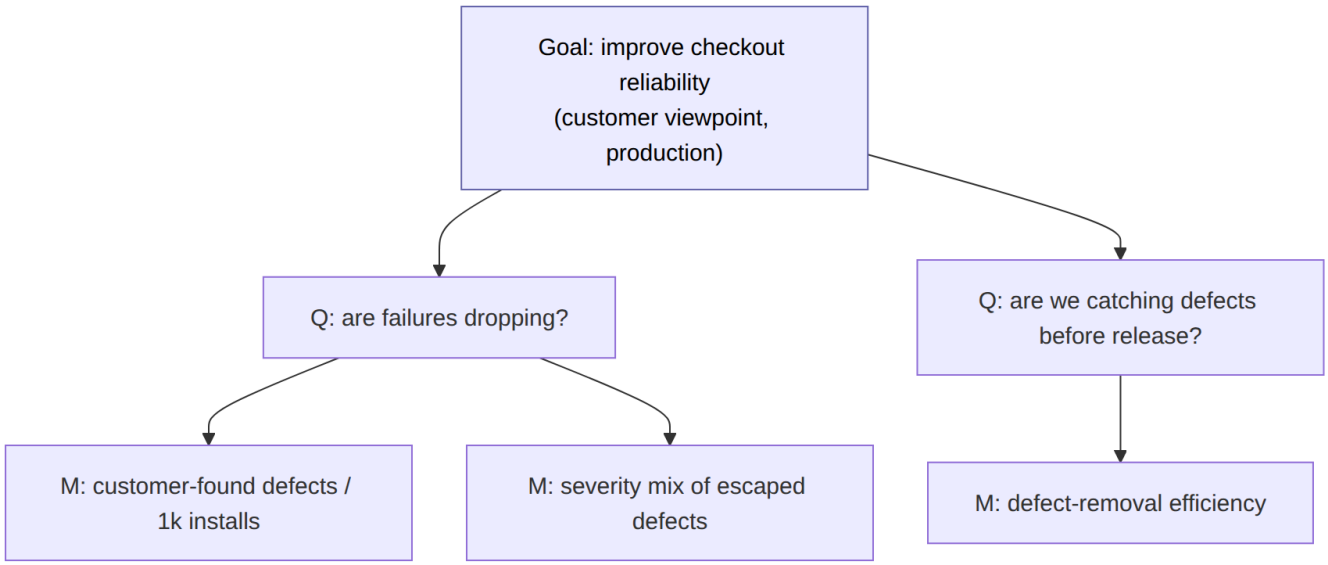
1. **Relevant.** It moves when the attribute you care about moves. Counting comments does not track code quality; if anything, heavy commenting can signal code so unclear it needs prose to explain it.
2. **Actionable.** When the number is bad, you know what to *do*. A metric you cannot act on is a spectator sport, not an engineering tool.
3. **Cheap enough.** The cost of collecting it is small relative to the value of the decision it informs. A metric that needs a week of manual effort per release will not survive contact with a deadline.
4. **Robust against gaming.** The moment a metric becomes a target for people's incentives, people begin optimizing the measured value, sometimes at the expense of the underlying goal. This is **Goodhart's Law** — the single most common way metrics programs go wrong.[1]

> **Pitfall.** Reward developers for "lines of code written" and you will get more, longer, more duplicated code. Reward testers for "number of bugs found" and you will get a flood of trivial, low-severity reports. Reward "closing tickets fast" and hard tickets get closed *unfixed*. Every metric attached to an incentive is quietly redefined by the people it measures. Prefer metrics that are hard to game and pair each metric with a *counter-metric* that would degrade if someone cheated (pair "tickets closed" with "tickets reopened").

12.1.3 Goal-Directed Measurement: GQM

The antidote to measuring the merely convenient is to start from a goal, not from a tool. The **Goal-Question-Metric (GQM)** method, introduced by Victor Basili and colleagues, gives measurement a top-down discipline:[2]

- **Goal** — a business or engineering objective, stated with an object, a purpose, a quality focus, a viewpoint, and a context. *"Improve (purpose) the reliability (focus) of the checkout service (object) from the customer's viewpoint (viewpoint) in production (context)."*
- **Question** — a concrete question whose answer would tell you whether you are meeting the goal. *"Are customers hitting fewer failures than last quarter?"*
- **Metric** — the data that answers the question. *"Customer-found defects per 1,000 active installs, tracked monthly."*



The direction matters. If you start from the metric ("we have a dashboard of 40 numbers — what do they mean?"), you drown in data and act on none of it. If you start from the goal, every number on the dashboard earns its place by answering a question you actually asked. When a metric answers no live question, delete it: an unused metric still costs collection effort and still tempts someone to game it.

12.2 Software Quality

12.2.1 The Many Forms of Software Quality

"Quality" is not one thing, and arguments about software quality usually turn out to be people talking past each other because they mean different *forms* of it. It helps to name them explicitly:

- **Functional quality** — does the software do what it is supposed to do, correctly? This is conformance to specification: the features work, the results are right, the edge cases are handled.
- **Process quality** — is the *way* the software is built repeatable, visible, and improving? A team that ships correct software by heroics has high functional quality and low process quality; next quarter's release is a coin flip.
- **Product quality** — the internal, structural health of the artifact: readability, modularity, low coupling, test coverage, absence of latent defects. Product quality is invisible to users but governs the *cost of change* (Chapter 6).
- **Operational quality** — how the system behaves in production: availability, latency, resource use, recoverability. A feature-complete service that is down two hours a week has poor operational quality.
- **Aesthetic quality** — the harder-to-quantify sense of a clean interface, a coherent API, a design that feels *right*. Real, but resistant to a single number.
- **Customer-satisfaction quality** — whether the people who use and pay for the software are actually happy with it. This is the form that ultimately pays the bills, and it is only loosely correlated with the others: users can love buggy software that solves their problem, and hate flawless software that does not.

The international standard **ISO/IEC 25010** organizes product quality into nine characteristics — functional suitability, performance efficiency, compatibility, interaction capability, reliability, security, maintainability, flexibility, and safety — each subdivided further. (The widely cited 2011 edition had eight, with *usability* and *portability* where the current edition has *interaction capability* and *flexibility*; taxonomies get revised, which is itself a lesson about measurement.)[3] Memorizing the taxonomy matters less than the habit it encodes: *before you measure quality, say which quality.*

> **Principle.** There is no single "quality score." Different stakeholders optimize different forms of quality, and improving one can degrade another (heavy security hardening can hurt usability). State which form you mean, whose viewpoint it takes, and why it matters — the discipline GQM enforces.

12.2.2 Measuring Customer Support

Customer support is where several forms of quality become visible at once, which makes it a rich source of metrics — and a treacherous one. Common support metrics include:

- **Ticket volume** — how many support requests arrive per unit time (or per install).

- **Time to first response** and **time to resolution** — how quickly the team reacts and fixes.
- **Backlog** — how many open tickets, and how old the oldest are.
- **Reopen rate** — the fraction of "resolved" tickets the customer reopens (a counter-metric to resolution speed).
- **Customer satisfaction (CSAT)** and **Net Promoter Score (NPS)** — survey-based measures of how customers *feel*.[4]

Notice how each metric, alone, misleads. Falling ticket volume might mean the software improved — or that users gave up and left. Fast resolution might mean skillful support — or tickets closed without really fixing anything (which the reopen rate would expose). This is why support metrics travel in packs: you interpret any one of them only against the others, and against the underlying goal of *keeping customers successful*. We will return to support-driven quality improvement as a full case study in §12.5.

12.3 Graphical Displays of Data Sets

12.3.1 Data Sets

A **data set** is a collection of measured values. The simplest is **univariate** — one attribute measured across many entities, like our eleven defect counts. A **bivariate** data set pairs two attributes per entity (module size *and* defect count), which is what regression needs. Before you compute a single statistic, look at the data. A number summary can hide a shape; a picture reveals it. The rest of §12.3 covers displays for *categorical* structure, *schedules*, and *uncertainty*; §12.6 covers displays for *dispersion*.

12.3.2 Scales of Measurement

Not all numbers carry the same information, and this determines which statistics and charts are even *legal*. There are four classic **scales of measurement**, from least to most informative:

Scale	What is meaningful	Example	Legal operations
Nominal	equality only; categories	defect type (UI, logic, data)	count, mode
Ordinal	order, but not distance	severity (low < med < high)	median, percentile
Interval	order *and* equal distances, no true zero	calendar date, °C	mean, standard deviation, difference
Ratio	all of the above *and* a true zero	defect count, time, KLOC	ratios, coefficient of variation

The scale tells you what you may do:

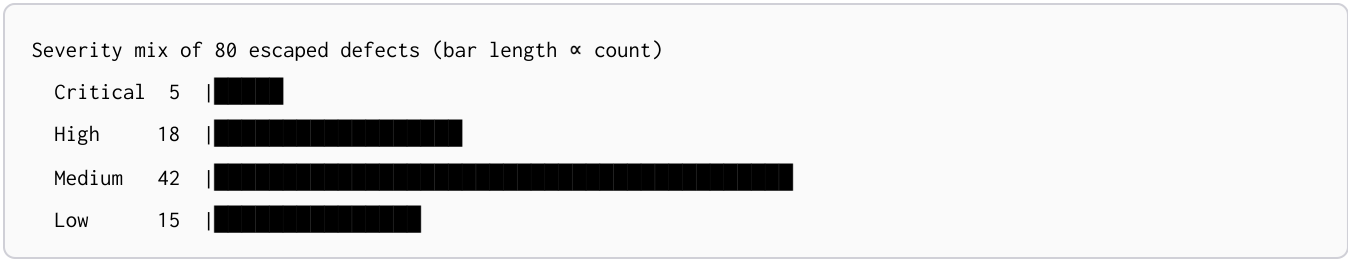
- On a **nominal** scale, “the average defect type is 2.3” is nonsense — the categories have no order to average. You may only count and find the most frequent (the **mode**).
- On an **ordinal** scale, you may find a **median** (“the median severity is High”) but *not* a mean, because the distance from Low to Medium need not equal Medium to High.
- On an **interval** scale, differences are meaningful (20°C is 10 degrees warmer than 10°C) but ratios are not (20°C is *not* “twice as hot” as 10°C, because 0°C is not true zero). Means and standard deviations become legal.
- On a **ratio** scale, everything is legal, including ratios (“release B has twice the defects of release A”), because zero really means “none.”

> **Pitfall.** Severity ratings and Likert survey scores (“rate 1–5”) are *ordinal*, yet teams routinely average them. “Average severity 2.7” treats the gap between severities as if it were constant, which it is not — one critical defect is not “worth” three trivial ones. Report the *median* and the *distribution* of ordinal data, not the mean.

Our running defect counts are **ratio-scaled** (zero defects means none, and “twice as many” is meaningful), so every statistic in this chapter is fair game for them.

12.3.3 Bar Charts Display Data by Category

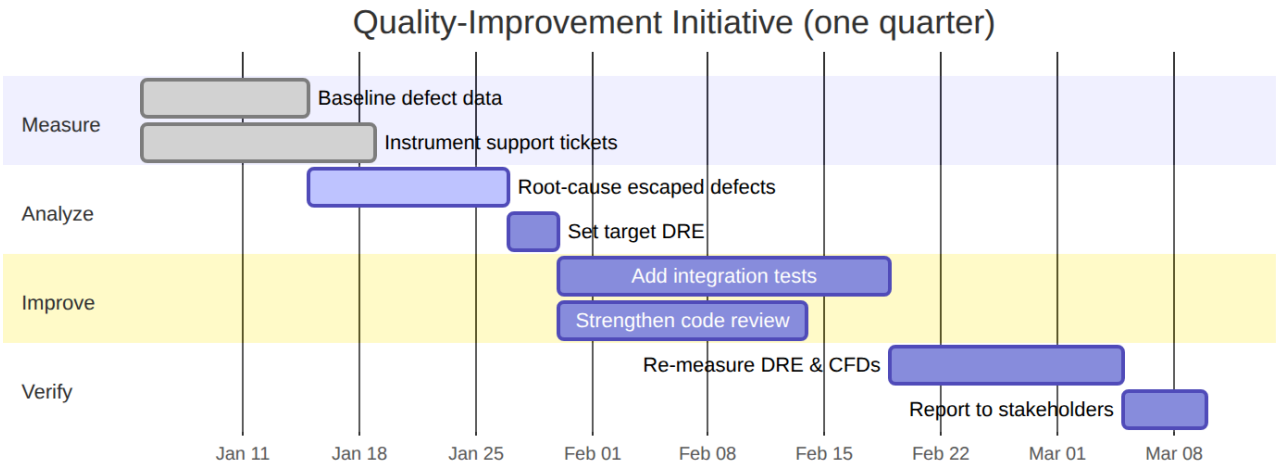
A **bar chart** shows a numeric value for each category of a nominal or ordinal attribute, using bar length to encode magnitude. It answers “how does this quantity break down by category?” Suppose the 80 customer-found defects behind one release sort by severity as: Critical 5, High 18, Medium 42, Low 15. A bar chart makes the shape immediate:



Two rules keep bar charts honest. First, **start the axis at zero** — a bar’s *length* encodes the value, so a truncated axis exaggerates differences. Second, keep the bars categorical: a bar chart’s horizontal axis has no continuous meaning, and reordering the categories changes nothing about the data (unlike a histogram, §12.6.3, whose bins *are* ordered and continuous). Confusing the two is a common error: bar charts are for *categories*, histograms are for *ranges of a continuous variable*.

12.3.4 Gantt Charts Display Schedules

A **Gantt chart** displays a schedule: tasks on the vertical axis, time on the horizontal axis, each task drawn as a horizontal bar spanning its start and end. It makes durations, overlaps, dependencies, and the **critical path** (the chain of tasks that determines the earliest possible finish) visible at a glance. Here is a quality-improvement initiative — the one we develop in §12.5 — as a Mermaid Gantt chart:

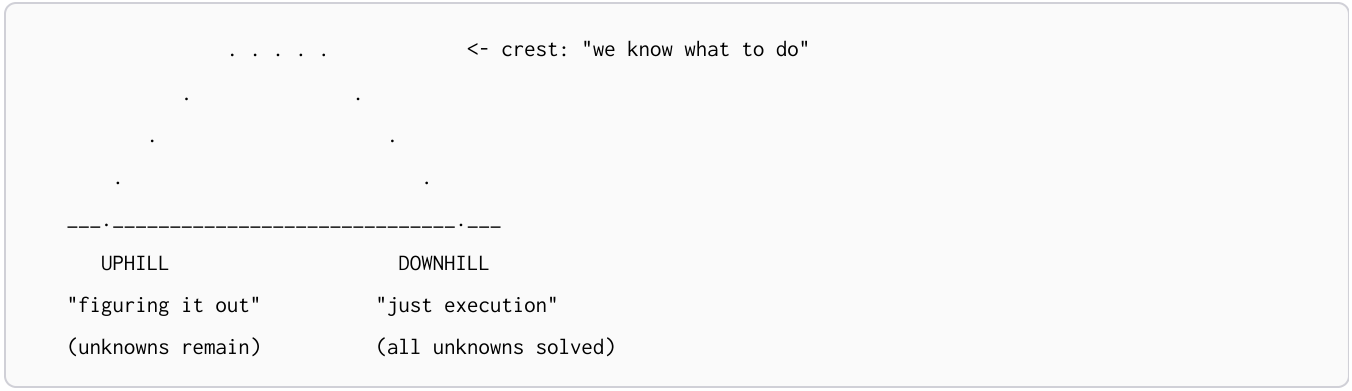


Read it the way a manager does: the *Measure* tasks run in parallel at the start; nothing in *Improve* can begin until *Set target DRE* finishes; and *Re-measure* cannot start until *Add integration tests* is done, making the analyze-test-remeasure chain the critical path. If any task on that chain slips, the whole quarter slips. Gantt charts are about *time and dependency*, not quantity — which is what distinguishes them from bar charts.

12.3.5 Hill Charts: Displaying Uncertainty, Not Just Quantity

The progress charts most teams use — a **burndown** of remaining tasks, or a "percent done" bar — report the *quantity of work remaining*, but they usually assume that the team has already discovered all of the work. The riskiest work is the work you *haven't figured out yet*, and a task list often *grows* as you discover what a problem really involves. A burndown that ticks down looks reassuring right up until the unknown you never listed blows up the schedule.

The **hill chart**, popularized by Basecamp's *Shape Up*, adds information that a burndown omits: *how much uncertainty remains* in each piece of work.[5] Represent each item as a dot moving over a hill:



- **Uphill** means *figuring it out* — open questions, unsolved design, unproven technical assumptions.
- **Downhill** means *just execution* — the unknowns are resolved and only the known, routine work remains.

Two properties make it a better *management* signal than a burndown. First, **a dot that stops moving is a raised hand**: it flags a stuck piece before a missed deadline announces it, and without interrogating anyone. Second, its language is about the work, not the person — "what would get this scope *over the hill*?" depersonalizes a status conversation that "why isn't this done?" would sour. Used well, teams push the *novel, risky* work uphill first and leave routine "screw-tightening" for last, so uncertainty falls fastest early — the same risk-first instinct behind the spiral model (§2.7) and Shape Up's building phase (§2.8).

> **Principle.** Use a progress metric that makes *uncertainty* visible alongside the amount of work remaining. A number that only counts finished tasks will look healthiest right before the unknown you never measured comes due.

12.4 Product Quality: Measuring Defects

Defects are the most measurable face of quality, so they anchor most quality programs. Measure the outcome you care about — usually "how much pain reaches the customer" — and avoid letting the result depend mainly on how aggressively the team searched for defects.

12.4.1 Severity of Defects

Not all defects matter equally, so raw counts mislead. Teams classify each defect by **severity**, an ordinal scale describing the *impact* of the failure it causes:

- **Critical (S1)** — data loss, security breach, or total outage; no workaround.
- **High (S2)** — a major feature is broken; a painful workaround may exist.
- **Medium (S3)** — a feature misbehaves but the user can proceed.
- **Low (S4)** — cosmetic or trivial; annoyance only.

Severity is distinct from **priority**, which is the business decision of *when* to fix. A low-severity typo on the login page can be high-priority because everyone sees it; a critical bug in a feature no one uses can be low-priority. Because severity is ordinal, summarize it with counts per level and a median — not a mean (§12.3.2). A release with one Critical and five Lows can be *worse* than one with ten Mediums, even though the second has more total defects; the severity distribution captures what the count destroys.

12.4.2 Defect-Removal Efficiency

The central product-quality metric is **defect-removal efficiency (DRE)**: of all the defects that existed, what fraction did you catch *before* they reached the customer?

$$\text{DRE} = \frac{D_{\text{before}}}{D_{\text{before}} + D_{\text{after}}} \times 100$$

where D_{before} is the number of defects found and removed before release (by reviews, static analysis, and testing) and D_{after} is the number the customer finds after release. The denominator is the *total* defects that were ever present; DRE is the share your internal nets caught.

> **Worked example.** During development of a release, code review, static analysis, and testing together found and removed **720** defects. In the first six months of production, customers reported **80** more.
>
> $$\text{DRE} = \frac{720}{720 + 80} \times 100$$
>
> The team caught nine of every ten defects before shipping. Some organizations use 95% DRE as an aspirational target; much lower values suggest that too many defects are escaping to users.[6]

DRE can also be computed *per phase* to show where defects leak through. If a phase receives defects, removes some, and passes the rest downstream, its **phase DRE** is defects removed in the phase divided by defects present when the phase began. Low phase DRE for, say, code review tells you *where* to invest. Two cautions keep DRE trustworthy. First, you only know D_{after} after enough time has passed — DRE for a release is provisional until its defects have had time to surface (often you fix a measurement window, like six months). Second, DRE rewards finding your own bugs, so it resists the gaming that raw "bugs found" invites: inflating D_{before} with trivial finds barely moves the ratio unless real escapes drop too.

12.4.3 Customer-Found Defects

A **customer-found defect (CFD)** is exactly what it sounds like: a defect first reported by a customer in production, i.e., the D_{after} term above. CFDs are the most *honest* defect metric in one sense — every one represents real pain that reached a real user, defects that survived every net you built. That makes CFD count, and especially the *severity mix* of CFDs, a direct measure of escaped quality. Our running dataset is a CFD series: eleven releases with 2, 4, 5, 5, 7, 8, 9, 10, 12, 14, and 23 customer-found defects respectively in their first six months.

12.4.4 CFDs Measure Installs, Not Quality

A CFD requires careful interpretation, because installation counts can make a dashboard appear healthier without showing whether product quality improved. **Raw CFD count is confounded by usage.** A release installed on ten machines and a release installed on ten million machines can have identical code quality, yet the second will generate vastly more customer-found defects, simply because more users exercise more paths on more data. A falling CFD count might mean the code got better — or that adoption cratered. A rising CFD count might mean the code got worse — or that a hit feature tripled your install base.

The fix is **normalization**: divide by exposure. Report CFDs per 1,000 active installs, or per million transactions, or per user-month. Only then does the metric track *quality* rather than *popularity*. If the release with 23 raw CFDs had ten times the installs of the release with 5, it is actually the *cleaner* release per user.

> **Principle.** Any count of production events — defects, crashes, support tickets — must be normalized by exposure before you compare across releases or over time. An unnormalized production count measures how many people used the software at least as much as it measures how good the software is. Raw CFDs measure installs; CFDs-per-install measure quality.

12.5 Ops Quality Improvement: A Case Study

12.5.1 How to Improve Software Quality

You improve what you measure, measure against a target, act, and re-measure — the loop the Gantt chart in §12.3.4 lays out (Measure → Analyze → Improve → Verify). This is just the scientific method wearing a hard hat, and it is the most reliable way to improve quality, because it forces you to state a hypothesis ("more integration tests will cut escaped defects") and then *check* it against data rather than declaring victory by feeling.

Concretely, a hosting company we will follow — call it Northwind — ships a document service and is alarmed by rising support load. Leadership wants "better quality," which means nothing until it is operationalized. The team applies GQM (§12.1.3) to turn the wish into a measurable program.

12.5.2 The Customer Quality Metric

Northwind's goal is *"reduce the reliability pain customers feel in production."* The question — *"how much pain reaches customers, per unit of usage?"* — points to a single top-line **customer quality metric**:

$$Q_{\text{cust}} = \frac{\text{severity-weighted CFDs in the quarter}}{\text{active installs (thousands)}}.$$

Weighting by severity keeps the metric from treating a cosmetic typo like an outage (Critical = 10, High = 5, Medium = 2, Low = 1, say), and normalizing by installs obeys the principle of §12.4.4. Leadership tracks this

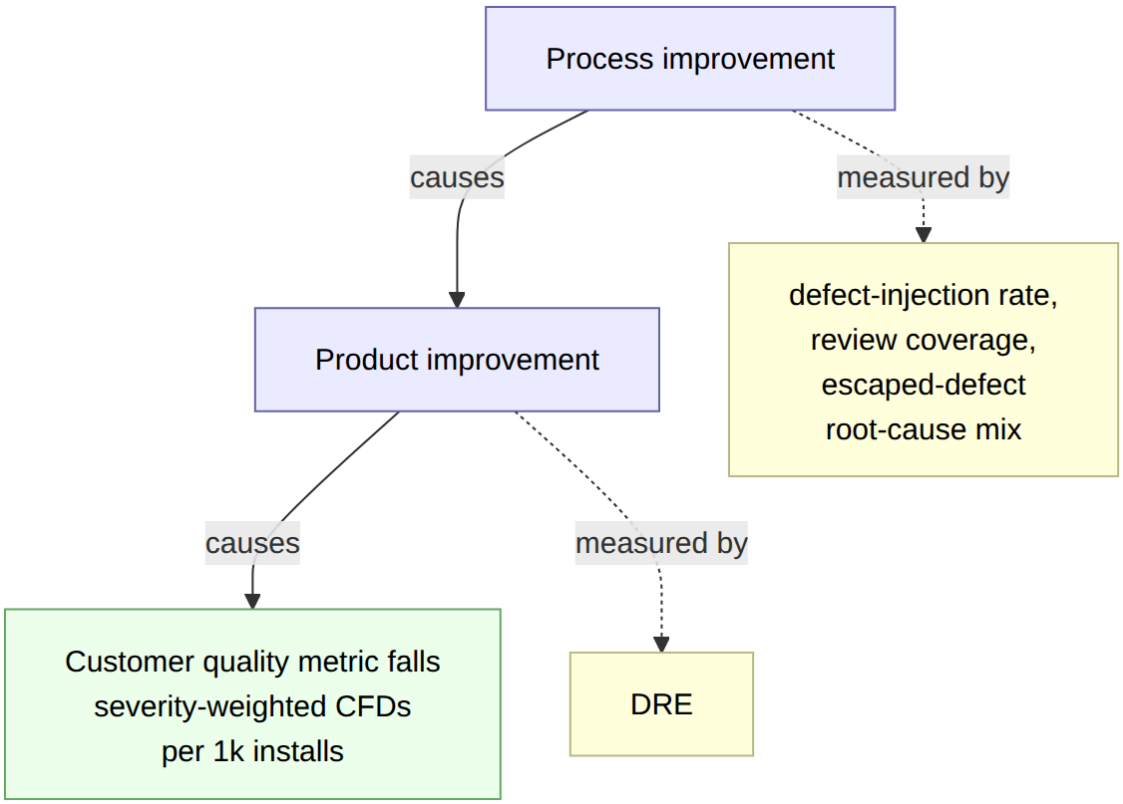
single value because it reflects customer impact. Since it measures *outcomes* instead of engineering *activity*, a team cannot improve it simply by doing more of the wrong work.

12.5.3 Subgoals: Product and Process Improvement

A top-line outcome metric tells you *whether* you are winning but not *how* to win. So Northwind decomposes the goal into two subgoals, each with its own metric:

- **Product improvement** — make each release intrinsically cleaner. Metric: **DRE** (§12.4.2). Raising DRE means fewer defects escape per release regardless of usage.
- **Process improvement** — make the *way* releases are built more effective, so that high DRE is repeatable rather than lucky. Metrics: **defect-injection rate** (defects introduced per KLOC), **review coverage** (fraction of changes reviewed), and **escaped-defect root-cause mix** (which lifecycle phase each escaped defect came from).

The relationship is a small hierarchy: process improvements *cause* product improvements (better reviews inject and escape fewer defects), and product improvements *cause* the top-line customer metric to fall.



12.5.4 Measuring Process Improvements

The dangerous move is to *assume* the process change worked. Northwind adds integration tests and strengthens code review (the *Improve* phase), then re-measures. Before the change, DRE was 84% and Q_{cust} was 7.5

weighted CFDs per thousand installs. After a quarter, DRE rose to 91% and Q_{cust} fell to 4.8. That *looks* like a win — but is the drop real, or just the noise of a naturally variable metric? A single before/after pair cannot answer that. You need the *dispersion* of the metric (how much it bounces release to release even with no real change) and a *confidence interval* around the improvement. That is precisely why the second half of this chapter builds the statistical machinery: without it, "quality improved" is a hope, not a finding. We now develop that machinery on our running CFD dataset and return to the verdict in §12.8.

12.6 Data Dispersion: Boxplots and Histograms

A single number — the mean — hides how *spread out* and how *shaped* the data are. Two release series can share a mean of 9 defects while one clusters tightly around 9 and the other swings from 0 to 30. Dispersion is often the more important story, because it tells you how *predictable* your process is. We now summarize our running dataset visually.

Recall the sorted CFD dataset (eleven releases):

$$2,\ 4,\ 5,\ 5,\ 7,\ 8,\ 9,\ 10,\ 12,\ 14,\ 23$$

12.6.1 Medians and Quartiles

The **median** is the middle value when the data are sorted — the 50th percentile. With $n = 11$ (odd), it is the 6th value: **median = 8**. The median is resistant to outliers: that lone 23 barely nudges it. The **quartiles** split the data into four parts:

- **Q1** (first quartile, 25th percentile) is the median of the lower half. Excluding the overall median, the lower half is $2, 4, 5, 5, 7$, whose median is **Q1 = 5**.
- **Q2** is the overall median, **8**.
- **Q3** (third quartile, 75th percentile) is the median of the upper half, $9, 10, 12, 14, 23$ — so **Q3 = 12**.

The **interquartile range** measures spread using the middle 50% of the data, which is why it too resists outliers:

$$\text{IQR} = Q3 - Q1 = 12 - 5 = 7.$$

12.6.2 Box Plots Summarize Data by Quartile

A **boxplot** (box-and-whisker plot) draws the five-number summary — minimum, Q1, median, Q3, maximum — as a box from Q1 to Q3 with a line at the median, and "whiskers" reaching to the most extreme non-outlier values. Points beyond the **outlier fences** are drawn individually. The standard fences are:

$$\text{lower fence} = Q1 - 1.5 \times \text{IQR}, \qquad \text{upper fence} = Q3 + 1.5 \times \text{IQR}.$$

For our data:

$$\text{lower fence} = 5 - 1.5(7) = -5.5, \qquad \text{upper fence} = 12 + 1.5(7) = 22.5.$$

Nothing falls below −5.5, but **23 exceeds 22.5, so 23 is an outlier.** The upper whisker therefore stops at the largest value *within* the fence, which is 14. Here is the boxplot described numerically, drawn to scale on a 0–24 axis:

```
0    2    4    5    7    8   10   12   14        23
|----[========|=========]----------------------o
     Q1=5    med=8    Q3=12   whisker→14      outlier
min=2                |<--- IQR = 7 --->|
```

Read at a glance: the box (5 to 12) holds the middle 50% of releases; the median sits at 8, a little left of the box's center, so the data are mildly **right-skewed** (a longer tail toward high defect counts); and one release (23) is a genuine outlier worth investigating on its own — maybe a spike in installs (§12.4.4) or a botched release. Placed side by side, Northwind's before-and-after boxplots show whether the entire distribution moved downward and how its spread changed. The mean alone cannot show either.

12.6.3 Histograms of Data Spread

A **histogram** shows the *shape* of a single continuous variable by dividing its range into equal **bins** and drawing a bar whose height is the count of values in each bin. Unlike a bar chart (§12.3.3), the horizontal axis is continuous and ordered, and bin width is a real choice: too wide hides structure, too narrow turns the picture into noise. With bin width 5, our data fall as:

```
Customer-found defects per release (bin width = 5)
  [ 0,  5)   2  |██          (values 2, 4)
  [ 5, 10)   5  |█████       (values 5, 5, 7, 8, 9)
  [10, 15)   3  |███         (values 10, 12, 14)
  [15, 20)   0  |            (none)
  [20, 25)   1  |█           (value 23)
```

The shape confirms the boxplot: a hump in the single digits, a right tail, and an isolated value far out at 23. Histograms and boxplots are complements — the boxplot gives a crisp five-number summary and flags outliers, the histogram reveals multi-modality and gaps a boxplot would smooth over. Look at both before trusting a mean.

12.7 Data Dispersion: Statistics

Pictures build intuition; numbers let you compute confidence intervals and regressions. We now put dispersion on a precise footing, still using the running dataset.

12.7.1 Variance from the Mean

The **mean** of our data is

$$\bar{x} \;=\; \tfrac{1}{n}\textstyle\sum_{i=1}^{n} x_i \;=\; \dfrac{2+4+5+5+7+8+9+10+12+14+23}{11} \;=\; \tfrac{99}{11} = 9.$$

The **variance** measures average squared distance from the mean. For a *sample* (data drawn from a larger population you want to infer about), we divide by $n-1$, not n:

$$s^2 \;=\; \frac{1}{n-1}\sum_{i=1}^{n}(x_i-\bar{x})^2 .$$

The $n-1$ (the *degrees of freedom*, and **Bessel's correction**) exists because you estimated the mean *from the same data*, which uses up one degree of freedom and makes the naive $\frac{1}{n}$ version systematically too small. Working the sum of squared deviations term by term:

x_i	$x_i-\bar{x}$	$(x_i-\bar{x})^2$
2	−7	49
4	−5	25
5	−4	16
5	−4	16
7	−2	4
8	−1	1
9	0	0
10	1	1
12	3	9
14	5	25
23	14	196
Σ	**0**	**342**

The deviations sum to zero (a useful check — they always do). So

$$s^2 \;=\; \frac{342}{11-1} \;=\; \frac{342}{10} = 34.2, \qquad s = \sqrt{34.2} \approx 5.85.$$

The **standard deviation** $s \approx 5.85$ is in the original units (defects) and says, loosely, that a typical release lands about 5.85 defects away from the mean of 9. Notice how heavily the outlier 23 dominates: its single term (196) is more than half the entire sum of squares. Standard deviation is *not* resistant to outliers — another reason to look at the boxplot's IQR alongside it. (If these eleven values were the entire population rather than a sample, you would divide by $n = 11$ to get the population variance $\sigma^2 = 342/11 \approx 31.1$.)

You can check the arithmetic in code:

```python
import statistics

cfds = [2, 4, 5, 5, 7, 8, 9, 10, 12, 14, 23]

s = statistics.stdev(cfds)       # divides by n - 1 (Bessel's correction)
sigma = statistics.pstdev(cfds)  # divides by n

print(f"s     = {s:.2f}")      # 5.85 — matches the hand computation
print(f"sigma = {sigma:.2f}")  # 5.58

# Warning: NumPy's np.std() defaults to ddof=0 and silently gives the population
# version (5.58 here); pass ddof=1 when your data are a sample.
```

One warning: statistics libraries differ on which divisor they use by default — before you trust a library's output, check whether it computes the sample or the population version.

12.7.2 Discrete Probability Distribution

To reason about data you *have not yet seen*, you need a model of how values occur: a **probability distribution**. A **discrete** distribution assigns a probability to each of countably many outcomes, with the probabilities summing to 1. Suppose that, from history, the number X of *critical* defects in a release follows:

x (critical defects)	0	1	2	3
$P(X = x)$	0.50	0.30	0.15	0.05

The **expected value** (mean of the distribution) is the probability-weighted average:

$$E[X] = \sum_{x} x, P(X = x) = 0(0.50) + 1(0.30) + 2(0.15) + 3(0.05) = 0.75.$$

Its variance uses $E[X^2] - (E[X])^2$:

$$E[X^2] \quad = 0^2\,(0.5) + 1^2\,(0.3) + 2^2\,(0.15) + 3^2\,(0.05) = 1.35,\ \mathrm{Var}(X) \quad = 1.35 - 0.75^2 \ = 0.7875.$$

So you expect about 0.75 critical defects per release, with a standard deviation of $\sqrt{0.7875} \approx 0.89$. This is the bridge from *describing* past data to *predicting* future data.

12.7.3 Continuous Distributions

Many quantities — a build's duration, a request's latency, a review's length — are **continuous**: they can take any value in a range, so the probability of *exactly* 42.0000 minutes is zero. Instead of a probability per outcome, a continuous distribution has a **probability density function** $f(x)$, and probability is *area under the curve*:

$$P(a \le X \le b) = \int_a^b f(x)\,dx, \qquad \int_{-\infty}^{\infty} f(x)\,dx = 1.$$

You never read a probability off the height of the curve; you read it off the area over an interval. This is the model behind the two distributions that dominate quality statistics: the normal and the Student's t.

12.7.4 Introduction to Normal Distributions

The **normal (Gaussian) distribution** is the symmetric, bell-shaped curve defined by its mean μ and standard deviation σ, written $N(\mu, \sigma^2)$. It matters for two reasons. First, many natural measurements are approximately normal. Second, and far more important, the **Central Limit Theorem** says that the *average* of many independent measurements is approximately normal *regardless of the shape of the original data* — which is why the normal shows up whenever you reason about sample means, even for lopsided data like our right-skewed defect counts.

The **empirical rule** (68–95–99.7) captures its spread: about 68% of values lie within 1σ of the mean, about 95% within 2σ, and about 99.7% within 3σ. Any value is located by its **z-score**, the number of standard deviations from the mean:

$$z = \frac{x - \mu}{\sigma}.$$

> **Worked example.** Historical review durations are approximately normal with μ = 45 minutes and σ = 12 minutes. A review that took 69 minutes has z = (69 − 45)/12 = 2.0 — two standard deviations above the mean. By the empirical rule, only about 2.5% of reviews run that long or longer, so 69 minutes is genuinely unusual and might warrant a look.

The **standard normal** is $N(0, 1)$: the distribution of z-scores themselves. Converting to z-scores lets you use one table (or the constants like 1.96 below) for every normal distribution.

12.7.5 Introduction to Student's t-Distributions

There is a catch. The z-score formula needs the *true* population standard deviation σ, which you almost never know — you only have the sample estimate s. Substituting s for σ adds extra uncertainty, especially with small samples, and the **Student's t-distribution** accounts for it. The t-distribution is also bell-shaped and centered at zero, but it has **heavier tails** than the normal — it admits that extreme values are a bit more likely when you had to estimate the spread from few data points.

Its shape depends on a single parameter, the **degrees of freedom**, $df = n - 1$. With few degrees of freedom the tails are fat; as df grows, the t-distribution steadily approaches the standard normal (they are nearly identical past $df \approx 30$). The practical rule follows directly:

- Population σ known (rare): use the **normal** and its critical value (1.96 for 95%).

- Population σ unknown, estimated by s (usual): use the **t-distribution** with $n - 1$ degrees of freedom, whose critical value is a bit larger than 1.96 to pay for the extra uncertainty.

12.8 Confidence Intervals

12.8.1 Definition of Confidence Interval

A sample mean is a single guess; a **confidence interval** is a guess *with error bars*. It is a range, computed from your sample, that is likely to contain the true population mean. A **95% confidence interval** is built by a procedure that, over many repeated samples, brackets the true mean 95% of the time.

> **Pitfall.** A 95% confidence interval does *not* mean "there is a 95% probability the true mean is in *this particular* interval." The true mean is a fixed (unknown) number; it is either in your interval or not. The 95% describes the long-run *success rate of the method*, not the odds for one interval. Speak of confidence in the procedure, not probability of the parameter.

Every confidence interval for a mean has the same shape:

$$\bar{x} \pm (\text{critical value}) \times (\text{standard error}), \qquad \text{standard error} = \frac{(\text{SD})}{\sqrt{n}}.$$

The **standard error** shrinks as $\sqrt{n}$ grows: to halve your error bars you need four times the data. The only question is which critical value — z or t — which depends on whether you know the population standard deviation.

12.8.2 If the Population Standard Deviation Is Known: z-interval

When σ is known (for instance, from long historical records of a stable process), use the normal distribution's critical value. For 95% confidence that value is $z^* = 1.96$ (from the empirical rule; more precisely, 95% of a normal lies within 1.96 standard deviations). The interval is:

$$\bar{x} \pm z^* \cdot \frac{\sigma}{\sqrt{n}}.$$

> **Worked example.** You time $n = 36$ code reviews and get a sample mean of $\bar{x} = 45$ minutes. From years of records the population standard deviation is known to be $\sigma = 12$ minutes. A 95% z-interval:
>
> $$45 \pm 1.96 \cdot \frac{12}{\sqrt{36}} = 45 \pm 1.96 \cdot \frac{12}{6} = 45 \pm 1.96(2) = 45 \pm 3.92.$$
>
> The 95% confidence interval is $(41.08,\ 48.92)$ minutes. You are confident the *true average* review time lies in that band. (Note $\sqrt{36} = 6$ made this clean; real samples rarely do.)

12.8.3 If the Population Standard Deviation Is Unknown: t-interval

Usually you do *not* know σ and must estimate it with the sample standard deviation s. Then use the t-distribution with $df = n - 1$:

$$\bar{x} \pm t^*_{n-1} \cdot \frac{s}{\sqrt{n}}.$$

Let us finish the story from §12.5.4 using our running CFD dataset. We have $n = 11$, $\bar{x} = 9$, and $s \approx 5.848$ (from §12.7.1). For 95% confidence with $df = 11 - 1 = 10$, the critical value is $t^*_{10} = 2.228$ (larger than 1.96, paying for the estimated spread and small sample). Then:

$$\text{standard error} = \frac{s}{\sqrt{n}} = \frac{5.848}{\sqrt{11}} = \frac{5.848}{3.317} \approx 1.763,$$

$$\text{margin} = t^*_{10} \cdot \frac{s}{\sqrt{n}} = 2.228 \times 1.763 \approx 3.93,$$

$$95\%\ \text{CI} = 9 \pm 3.93 = (5.07,\ 12.93)\ \text{customer-found defects.}$$

Now interpret Northwind's result — and resist a tempting shortcut. The "before" interval says the true mean CFD rate plausibly lies anywhere from about 5 to 13 defects per release. It does *not* follow that you can judge the improvement by computing a second interval for the "after" quarter and eyeballing whether the two overlap: overlap of two separate intervals is not a reliable test of whether two means differ. What §12.5.4 asked for was a confidence interval around the *improvement itself* — one interval for the **difference** between the means. Build it from paired differences when the observations are naturally paired (the same release measured under both processes); for two independent samples like Northwind's before and after quarters, it is

$$ (\bar{x}_{\text{before}} - \bar{x}_{\text{after}}) \pm t^* \sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}, $$

with degrees of freedom from your statistics package (or, conservatively, the smaller $n - 1$). If that interval excludes zero, the improvement is unlikely to be noise, and you can report it with quantified confidence. If it includes zero, the right conclusion is "we cannot yet distinguish the improvement from ordinary release-to-release variation; collect more data." Either way, a confidence interval turns "quality improved" from a hope into a defensible finding — or an honest *not yet*.

> **Principle.** As a default, never report a before/after change without a confidence interval or an equivalent test: metrics vary release to release even when nothing changed, and the interval is what separates a real improvement from a lucky quarter. And when your data cannot support one — a handful of observations, definitions that changed midway, measurements that aren't independent — say *that* instead. Descriptive numbers with stated limitations are honest; an interval computed from data that do not meet its assumptions is pseudo-precision.

12.9 Simple Linear Regression

Confidence intervals summarize *one* variable. **Regression** models how one variable depends on another — for example, how the number of defects in a module depends on its size. It lets you *predict* (estimate defects for a new module from its size) and *understand* (quantify how much each extra KLOC costs you in defects).

Our second, bivariate dataset pairs each module's size x (in KLOC) with its customer-found defects y:

x **(KLOC)**	**2**	**4**	**6**	**8**	**10**
y (defects)	5	9	10	13	18

12.9.1 The Simpler Case: Line through the Origin

Sometimes theory forces the line through the origin: zero size should mean zero defects, zero effort for zero work. A one-parameter model $y = b, x$ has the least-squares slope

$$b = \frac{\sum x_i \, y_i}{\sum x_i^2}.$$

For our data, $\sum x_i \, y_i$ = (2)(5) + (4)(9) + (6)(10) + (8)(13) + (10)(18) = 10 + 36 + 60 + 104 + 180 = 390 and $\sum x_i^2$ = 4 + 16 + 36 + 64 + 100 = 220, so

$$b = \frac{390}{220} \approx 1.77 \text{ defects per KLOC.}$$

The through-origin model predicts about 1.77 defects for every thousand lines. It is simpler, but it *forces* the fit to pass through $(0, 0)$ even if the data would prefer an offset — which brings us to the general case.

12.9.2 Ordinary Least-Squares Fit

Ordinary least squares (OLS) fits the line $\hat{y} = b_0 + b_1 \, x$ that minimizes the sum of squared vertical **residuals** $e_i = y_i - \hat{y}_i$. Squaring (rather than taking absolute values) makes the problem smoothly solvable and penalizes large misses heavily. Minimizing $\sum e_i^2$ by setting its partial derivatives with respect to b_0 and b_1 to zero yields the **normal equations**, whose solution is:

$$b_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}}, \qquad b_0 = \bar{y} - b_1 \, \bar{x}.$$

The intercept formula guarantees the line passes through the point of averages $(\bar{x}, \bar{y})$ — a useful sanity check. Work it through. The means are $\bar{x}$ = (2 + 4 + 6 + 8 + 10)/5 = 6 and $\bar{y}$ = (5 + 9 + 10 + 13 + 18)/5 = 11. Then tabulate the deviations:

x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$
2	5	−4	−6	24	16
4	9	−2	−2	4	4
6	10	0	−1	0	0
8	13	2	2	4	4
10	18	4	7	28	16
Σ				**60**	**40**

So $S_{xy} = 60$ and $S_{xx} = 40$, giving:

$$b_1 = \frac{60}{40} = 1.5, \qquad b_0 = 11 - 1.5(6) = 2.$$

The fitted line is:

$$\hat{y} = 2 + 1.5, x.$$

Interpret both coefficients. The slope $b_1 = 1.5$ says each additional KLOC brings about 1.5 more customer-found defects — a *rate*, the actionable number. The intercept $b_0 = 2$ is the model's prediction at zero size; here it is a mathematical artifact (a zero-size module cannot have 2 defects), a reminder not to extrapolate a regression outside the range of your data. Check the fit by residuals:

x_i	y_i	$\hat{y}_i = 2 + 1.5x_i$	**residual** e_i
2	5	5	0
4	9	8	+1
6	10	11	−1
8	13	14	−1
10	18	17	+1

The residuals sum to zero (they always do with an intercept) and are small. The **sum of squared errors** is $\text{SSE} = 0^2 + 1^2 + (-1)^2 + (-1)^2 + 1^2 = 4$. To judge fit, compare it to the total variation $\text{SST} = \sum (y_i - \bar{y})^2 = 36 + 4 + 1 + 4 + 49 = 94$. The **coefficient of determination** is

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = 1 - \frac{4}{94} \approx 0.957.$$

So size explains about 96% of the variation in defect counts here — a strong linear relationship. (Equivalently, the correlation is $r = S_{xy}/\sqrt{S_{xx}\,S_{yy}} = 60/\sqrt{40 \cdot 94} \approx 0.978$, and $r^2 = R^2$.)

> **Pitfall.** Regression measures *association*, not *causation*. A high R^2 between size and defects does not prove that shrinking modules *causes* fewer defects — both might be driven by a third factor (rushed, complex features are both large *and* buggy). And never extrapolate: our model is fit on 2–10 KLOC and says nothing reliable about a 40-KLOC module or about zero. Regression describes the data you have, in the range you have it.

12.10 Conclusion

Metrics are how software engineering earns the word *engineering*: they replace assertion with evidence. But the chapter's two halves carry one shared lesson. The measurement half (§§12.1–12.5) insists that a number is only as good as the goal behind it — choose metrics from goals (GQM), state *which* form of quality you mean, normalize production counts by exposure so you measure quality and not popularity, and expect any metric tied to an incentive to be gamed. The statistical half (§§12.6–12.9) insists that a number is only as good as its uncertainty — a mean without a spread misleads, a boxplot and histogram reveal shape and outliers a mean hides, and a before/after comparison without a confidence interval cannot tell a real improvement from a lucky quarter.

We carried one dataset through the whole apparatus: eleven customer-found-defect counts with mean 9, median 8, IQR 7, variance 34.2, standard deviation 5.85, one outlier at 23, and a 95% t-interval of $(5.07, 12.93)$; and a paired dataset whose least-squares fit $\hat{y} = 2 + 1.5x$ explains 96% of the variation in defects by size. Every one of those numbers began as an invisible attribute and ended as something you can compare, defend, and act on.

Used well, metrics close the loop that began in Chapter 1: they tell you, with evidence rather than opinion, whether your requirements, design, reviews, and tests are actually producing software that works — and whether it is getting better. Used badly, they become theater: dashboards no one reads, targets everyone games, averages that hide the story. The difference is discipline, not statistical sophistication: start from a goal, say what you mean by quality, and never report a change without its error bars.

Sources

[1] C. A. E. Goodhart, *Problems of Monetary Management: The U.K. Experience* (1975; reprinted in *Monetary Theory and Practice*, Palgrave, 1984). link.springer.com. The popular phrasing — "when a measure becomes a target, it ceases to be a good measure" — is Marilyn Strathern's, from "'Improving Ratings': Audit in the British University System," *European Review* 5(3) (1997).

[2] Victor R. Basili, Gianluigi Caldiera & H. Dieter Rombach, "The Goal Question Metric Approach," *Encyclopedia of Software Engineering* (Wiley, 1994). cs.umd.edu.

[3] ISO/IEC 25010:2023, *Systems and software engineering — SQuaRE — Product quality model* (2023) — nine characteristics; it revises ISO/IEC 25010:2011, whose product-quality model had eight. iso25000.com; iso.org.

[4] Frederick F. Reichheld, "The One Number You Need to Grow," *Harvard Business Review* (December 2003). hbr.org.

[5] Ryan Singer, *Shape Up: Stop Running in Circles and Ship Work that Matters*, ch. 13 “Show Progress” (Basecamp, 2019). basecamp.com.

[6] Capers Jones, *Software Defect Removal Efficiency* (Capers Jones & Associates, 2011) — reports a U.S. average DRE near 85% and that combining inspections, static analysis, and testing is how projects top 95%. ppi-int.com.

- **Key takeaways** are summarized above in §12.10.
- Continue to the Exercises.
- Go deeper with the Open Resources for this chapter.

Chapter 12 — Exercises

Exercises are graded by depth: **[warm-up]** checks understanding, **[analysis]** asks you to reason; a **[calculation]** tag marks exercises that require working the numbers. At least five exercises require calculation — show your work, including formulas and intermediate values, not just a final number.

Two datasets recur (the same ones used in the chapter). **Dataset A** is a univariate series of customer-found defects for eleven releases; **Dataset B** pairs module size with defects.

- **Dataset A** (sorted): $2,\ 4,\ 5,\ 5,\ 7,\ 8,\ 9,\ 10,\ 12,\ 14,\ 23$
- **Dataset B:** x (KLOC) $= 2, 4, 6, 8, 10$; y (defects) $= 5, 9, 10, 13, 18$

Concepts

1. **[warm-up]** In your own words, distinguish an *entity*, an *attribute*, and a *metric*. Give two different metrics for the same attribute of a source file, and say why one might be more *valid* than the other.
2. **[warm-up]** Classify each of the following by scale of measurement (nominal, ordinal, interval, or ratio) and name one statistic that is *legal* and one that is *illegal* for it: (a) defect severity S1–S4, (b) defect count per release, (c) the calendar date a bug was filed, (d) the module a bug lives in.
3. **[warm-up]** State the difference between a *bar chart* and a *histogram*. Give one dataset that belongs on each, and explain why swapping them would mislead.
4. **[warm-up]** Write the GQM for this goal: "reduce the time new engineers take to make their first safe production change." Supply at least one *question* and two *metrics*, and name a *counter-metric* that would catch someone gaming one of them.

Analysis

5. **[analysis]** A manager proposes paying testers a bonus per bug they file. Using Goodhart's Law and the idea of a counter-metric, predict two specific ways this will distort behavior, and propose a metric pair that resists the distortion.
6. **[analysis]** Two releases each report exactly 30 customer-found defects. Release P runs on 5,000 installs; release Q runs on 500,000 installs. Explain, with a normalized figure for each, why raw CFD count makes them look equal when their quality is very different. State which release is cleaner and by how much.
7. **[analysis]** The chapter says severity is *ordinal* and should not be averaged. A colleague insists "average severity 2.7" is fine because the labels map to numbers 1–4. Rebut this in two or three sentences, and propose what to report instead.
8. **[analysis]** Explain the difference between DRE improving because your *product* got better and the customer quality metric Q_{cust} falling because *usage* dropped. Which metric is confounded by usage and

which is not? Why does Northwind track both (§12.5)?

9. **[analysis]** You compute a 95% confidence interval (5.07, 12.93) for a release's mean CFD count. A teammate says "so there's a 95% chance the true mean is between 5.07 and 12.93." Explain precisely why this interpretation is wrong and give the correct one.

Calculation

10. **[analysis][calculation]** *(Defect-removal efficiency.)* During development, reviews and testing remove 465 defects. In the first six months of production, customers report 35 more. (a) Compute DRE. (b) The team's goal is 95% DRE; how many *fewer* escaped defects (holding D_{before} fixed) would they have needed to hit it? (c) Explain why DRE for this release is still "provisional" at six months.
11. **[analysis][calculation]** *(Quartiles and boxplot.)* For **Dataset A**, compute the five-number summary (min, Q1, median, Q3, max), the IQR, and the upper and lower outlier fences. Identify any outliers, state where each whisker ends, and say whether the distribution is left- or right-skewed and how you can tell from the summary.
12. **[analysis][calculation]** *(Variance and standard deviation.)* For **Dataset A**, compute the mean, the sample variance s^2, and the sample standard deviation s, showing the deviation table. Then recompute the variance *dividing by* n *instead of* $n - 1$. Explain which is appropriate if the eleven values are (a) a sample from an ongoing process, versus (b) the entire population of releases ever shipped.
13. **[analysis][calculation]** *(z-interval.)* You measure $n = 64$ builds and get a sample mean duration of $\bar{x} = 8.0$ minutes. From long history the population standard deviation is known to be $\sigma = 2.4$ minutes. Construct the 95% z-interval for the true mean build time (use $z^* = 1.96$). Then state what would happen to the interval's width if you quadrupled the sample size to 256, and by what factor.
14. **[analysis][calculation]** *(t-interval.)* A different eight-release sample of CFD counts has $\bar{x} = 11$ and $s = 4.0$. Using $t^*_7 = 2.365$ for 95% confidence, construct the t-interval for the mean. Then explain in one sentence why you used a t-value larger than 1.96 rather than the normal critical value.
15. **[analysis][calculation]** *(OLS regression.)* For **Dataset B**, fit the least-squares line $\hat{y} = b_0 + b_1 x$. Show $\bar{x}$, $\bar{y}$, S_{xy}, S_{xx}, then b_1 and b_0. Compute the residual for the $x = 4$ point, the SSE, and R^2. Interpret the slope in plain words, and explain why predicting y at $x = 40$ from this fit would be unwise.

Chapter 12 — Open Resources

Free, open-licensed, or freely accessible materials that reinforce this chapter. Types: 📘 open text · 🎓 course · 📄 primary source / standard · 🎥 video. Licenses vary and are noted where known; linked resources remain under their own terms.

Statistics foundations (§§12.6–12.9)

- 📘 **OpenIntro Statistics** — openintro.org/book/os. A complete, rigorous, free introductory statistics text. Covers scales of data, boxplots and quartiles, variance and standard deviation, the normal and t distributions, confidence intervals, and least-squares regression — the exact toolkit of §§12.6–12.9, with worked examples. **License: CC BY-SA 3.0** (PDF free; print available at cost).
- 📘 **Introduction to Modern Statistics (Çetinkaya-Rundel & Hardin)** — openintro-ims.netlify.app. A newer OpenIntro text emphasizing simulation-based and computational inference; strong chapters on exploratory data displays, the bootstrap, confidence intervals, and linear regression. **License: CC BY-SA.** A good second pass once the classic text's formulas are clear.
- 🎓 **Khan Academy — Statistics and Probability** — khanacademy.org/math/statistics-probability. Short videos and practice on quartiles/boxplots, variance/SD, z-scores, the normal distribution, confidence intervals, and regression. Free to use.

Software metrics and quality (§§12.1–12.5)

- 📄 **Basili, Caldiera & Rombach — "The Goal Question Metric Approach"** (*Encyclopedia of Software Engineering*, 1994) — cs.umd.edu/users/mvz/handouts/gqm.pdf. The primary source for GQM (§12.1.3), freely posted as a course handout at the University of Maryland (Basili's home institution). Read it for the original, goal-first framing of measurement.
- 📄 **Goodhart — "Problems of Monetary Management: The U.K. Experience" (1975)** — link.springer.com. The origin of Goodhart's Law (§12.1.2), reprinted in *Monetary Theory and Practice* (Palgrave, 1984). The chapter is paywalled; the popular phrasing — "when a measure becomes a target, it ceases to be a good measure" — is Marilyn Strathern's, from "'Improving Ratings'" (*European Review*, 1997), also paywalled but widely summarized.
- 📄 **Reichheld — "The One Number You Need to Grow"** (*Harvard Business Review*, Dec. 2003) — hbr.org/2003/12/the-one-number-you-need-to-grow. The article that introduced the Net Promoter Score (§12.2.2). HBR allows a limited number of free article views per month.
- 📄 **Capers Jones — "Software Defect Removal Efficiency" (2011)** — ppi-int.com (PDF). A short, free paper by the metric's best-known proponent (§12.4.2): U.S. average DRE near 85%, and why topping 95% takes inspections and static analysis on top of testing.

- 📘 **Singer, *Shape Up*, ch. 13 “Show Progress”** — basecamp.com/shapeup/3.4-chapter-13. The primary source for hill charts (§12.3.5): why to-do lists grow as work progresses, and how dots on a hill expose uncertainty a burndown hides. The full book is free to read online.
- 📘 **Fenton & Bieman, *Software Metrics: A Rigorous and Practical Approach*** — the standard reference for measurement theory, scales of measurement, and valid vs. invalid metrics (§§12.1, 12.3.2). Not open-licensed, but foundational; look for the authors’ freely posted lecture notes and papers, and library copies.
- 📄 **DORA — DevOps Research and Assessment metrics** — dora.dev. The five delivery metrics — throughput (change lead time, deployment frequency, failed-deployment recovery time) and instability (change fail rate, deployment rework rate; the “four keys” until 2024) — and the annual *State of DevOps* reports. An excellent, evidence-based example of goal-directed, hard-to-game operational metrics (§12.2, §12.5). Reports free to read.
- 📄 **ISO/IEC 25010 — Systems and software Quality Requirements and Evaluation (SQuaRE): Product quality model** — iso25000.com/en/iso-25000-standards/iso-25010. The nine product-quality characteristics referenced in §12.2.1 (the 2023 revision; the widely cited 2011 edition had eight). The summary portal is free; the formal standard is paywalled by ISO (iso.org/standard/78176.html).

Reference and data

- 📄 **NIST/SEMATECH e-Handbook of Statistical Methods** — itl.nist.gov/div898/handbook. A free, authoritative U.S.-government reference for descriptive statistics, distributions, confidence intervals, and regression, with formulas and case studies. Public domain (U.S. government work).
- 🎥 **StatQuest with Josh Starmer** — youtube.com/@statquest. Clear, free video explanations of variance, the normal distribution, confidence intervals, and linear regression — useful if a formula in §§12.7–12.9 needs a second, visual pass.

License note

Linked resources remain under their own licenses (several are CC BY-SA; the NIST handbook is U.S. public domain; ISO and Fenton & Bieman are not open). This page is CC BY-SA 4.0.

Chapter 13 — Software Engineering in the Age of AI

> **Where we are.** Every previous chapter described a durable engineering discipline — process, requirements, design, checking, testing, metrics, teamwork. This chapter asks what happens to those disciplines when generative AI and **coding agents** can draft code, tests, requirements, and designs in seconds. We separate what genuinely changes from what stubbornly does not, walk the whole lifecycle stage by stage, look honestly at the *evidence* (which is more mixed than the marketing), read the **o16g "Outcome Engineering" manifesto** as a provocation for the agentic era, and end with the concrete practices now emerging to work this way — **spec-driven development**, **agent instruction files** (`CLAUDE.md` / `AGENTS.md`), and **loop engineering**.

The headline claim of the moment is that AI writes the code now. A more useful way to describe the change is that **AI moves the engineering bottleneck.** For decades the bottleneck was *producing* correct code fast enough. As producing gets cheap, the bottleneck moves to *deciding what to build* and *proving it works* — which are the very activities this book has been teaching. The engineer who internalized Chapters 1–12 is the person best positioned to *direct* the agents.

> **Note — what will age here.** This chapter mixes two kinds of content with very different half-lives. Named tools, model capabilities, benchmark and study results, prices, and specific agent behaviors are **edition-specific**: they describe early 2026 and will drift, so treat every product name and number as a dated snapshot. The durable content is the principles — verify before you trust, keep a human accountable for what ships, write specifications precise enough to execute, and preserve traceability from requirement to change. When this edition's numbers age out, those principles are the part meant to remain.

13.1 What AI Changes — and What It Doesn't

13.1.1 Essential vs. Accidental Complexity, Revisited

Recall from Chapter 1 the split between **essential** complexity (inherent in the problem) and **accidental** complexity (from our tools and toil). Generative AI is a spectacular weapon against *accidental* complexity: boilerplate, glue code, format conversions, first drafts of tests, "how do I call this API" friction. It is far weaker against *essential* complexity: deciding what the system should actually do, resolving conflicting goals, choosing an architecture whose likely changes are cheap. That boundary is the single most important idea in this chapter. **Delegate the toil; own the essence.**

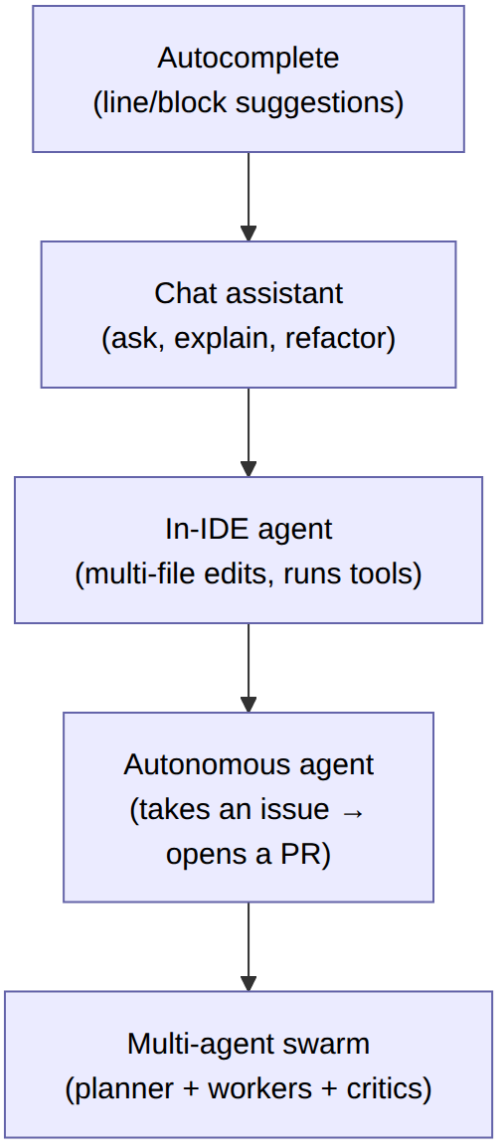
13.1.2 The Four Pressures Still Hold

The book's four cross-cutting pressures do not go away — AI just shifts them:

- **Software is complex.** Agents can *generate* complexity faster than ever, including accidental complexity you did not ask for. Architecture (Chapters 6–7) matters *more*.
- **Requirements change.** AI lowers the cost of *responding* to change, which raises the premium on knowing *which* change is worth making (Chapters 3–4).
- **Defects are inevitable.** AI produces plausible-looking defects at scale. Reviews, static checking, and tests (Chapters 9–10) become the load-bearing walls.
- **Teams need coordination.** "Teams" now include non-human members. Coordinating a *swarm* of agents is a new and unsolved organizational problem (see §13.4).

13.1.3 A Ladder of Assistance: From Autocomplete to Agents

"AI coding" is not one thing. It helps to name the rungs, because the engineering questions differ at each:



As assistance deepens from autocomplete toward a full agent swarm, the human's job moves from *typing* toward *specifying, supervising, and verifying* — and the cost of a confident-but-wrong output rises, because more happens between your instruction and your review.

13.1.4 The Productivity Paradox

Does AI make developers faster? The honest answer is *it depends, and the evidence is mixed enough to demand humility.*

- **Perceived gains are large — and the headline numbers are vendor-reported.** In GitHub's own 2022 research, surveyed Copilot users (about 2,000) reported feeling more productive, and in the companion controlled experiment — 95 developers, one well-specified task, building an HTTP server — the Copilot

group finished about 55% faster.[1] [2] These are honest but narrow measurements, reported by the vendor of the product being measured, of one task type at one moment in the tools' evolution.

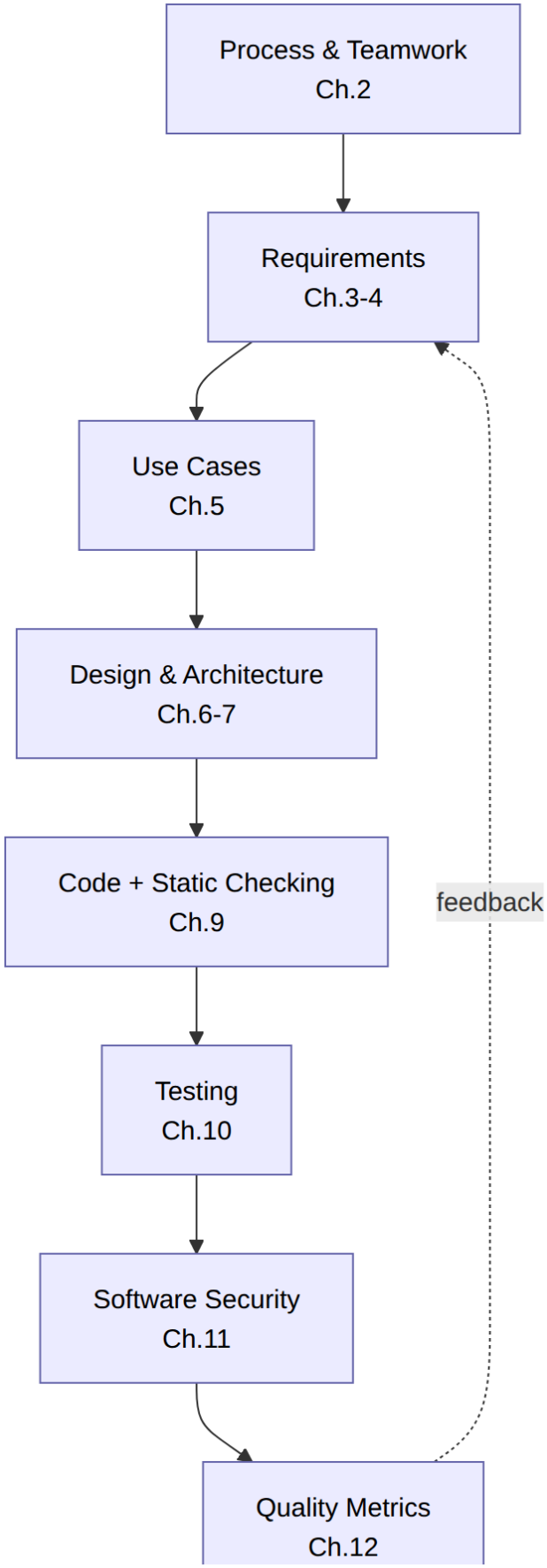
- **Measured gains are smaller and sometimes negative.** In a 2025 randomized controlled trial, METR had experienced open-source developers do real tasks in repositories they knew well. Developers *predicted* AI would make them ~24% faster; in fact they were about **19% slower** with AI — and still *believed* they'd been faster afterward.[3] The slowdown came largely from the overhead of **reviewing and integrating** generated output. Scope the finding as carefully as the researchers did: it describes *that specific setting* — experienced developers, mature and familiar repositories, high quality standards, early-2025 tools — not AI-assisted work in general, and tools have moved since.

> **Principle.** Perceived productivity is not measured productivity. On familiar, high-standards code, the time you save typing can be swamped by the time you spend verifying what the agent produced. *Verification is the new bottleneck* — a theme this whole chapter returns to.

The lesson is not "AI is useless" — gains are real for boilerplate, unfamiliar APIs, greenfield prototypes, and well-specified narrow tasks. These results suggest that **AI provides more leverage to people who can judge its output well.** Measure its effect on delivered work instead of relying on the subjective feeling of moving faster, as discussed in Chapter 12.

13.2 AI Across the Lifecycle

Here we walk the lifecycle, mapping AI's role onto each chapter's discipline. For each stage: *what AI does well now*, *where it fails*, and *what fundamental the human still owns.*


Process & Teamwork
Ch.2
Requirements
Ch.3-4
Use Cases
Ch.5
Design & Architecture
Ch.6-7
Code + Static Checking
Ch.9
Testing
Ch.10
Software Security
Ch.11
Quality Metrics
Ch.12
feedback

13.2.1 Process and Teamwork (Chapter 2)

AI compresses many process activities: drafting sprint plans, summarizing standups, writing retro notes, triaging issues, generating first-pass estimates. Agents can now take a ticket and open a pull request unattended, which strains the assumptions of Scrum and XP — what is a "sprint" when a swarm can attempt fifty backlog items overnight?

- **Fails at:** judgment about *value* and *sequencing*; reading the human dynamics of a team; owning accountability for what ships.
- **Human owns:** the definition of done, the choice of what to build next, and responsibility. The agile principle "working software is the primary measure of progress" gets *sharper*, not softer — see the manifesto's "verified reality" in §13.4.[4]

13.2.2 Requirements (Chapter 3)

Large language models are genuinely useful for **elicitation and drafting**: turning interview notes, tickets, and support logs into candidate user stories, acceptance criteria (Given/When/Then), and edge-case lists. Studies report LLM-drafted requirements that reviewers rate as complete, consistent first drafts — comparable to an entry-level engineer's — produced far faster and cheaper.[5] [6]

- **Fails at:** knowing what users actually *need* versus what is *plausible*; an LLM will cheerfully invent requirements that sound reasonable and are wrong (**hallucinated requirements**), and its fluency can manufacture false consensus.[7]
- **Human owns:** talking to real stakeholders, resolving conflicting goals (Chapter 3's goal hierarchies), and *validating* that a story reflects a real need. Use AI to widen the net of candidate requirements; use humans to decide which are true.

13.2.3 Estimation and Analysis (Chapter 4)

AI can suggest story-point estimates from historical data and surface comparable past work. But recall *why* we estimate in Chapter 4: story points model **human uncertainty** and drive **prioritization**, not stopwatch time. When agents do the building, the estimation question shifts from "how many engineer-hours?" toward "how much **compute/ budget**, and is the outcome worth it?" — precisely the manifesto's *cost, not time* reframing (§13.4). MoSCoW, value/cost/risk, and Kano analysis remain the right tools; the "cost" axis just gets a new currency.

13.2.4 Use Cases (Chapter 5)

Given a goal and a happy path, models are good at enumerating **alternative flows** and exception cases — exactly the tedious part of use-case writing that people skip. That coverage is valuable. But an agent cannot tell you which actor *goals* matter, and it tends to pad use cases with flows no user will ever take.

- **Human owns:** the actor–goal list (what the system is *for*) and pruning generated flows down to the ones that carry real risk or value.

13.2.5 Design and Architecture (Chapters 6–7)

This is where the human-owned layer is thickest. AI can propose class structures, generate **Architecture Decision Records (ADRs)**, suggest applicable patterns from Chapter 7, and even prototype competing designs to compare. Research on multi-agent systems that go from requirements to candidate architectures is active but early, and leans on role specialization to fight hallucination.[8] [9]

- **Fails at:** the *significant, expensive-to-change decisions* that define architecture (Chapter 6); it has no stake in your five-year maintenance cost and cannot feel the pain of high coupling.
- **Human owns:** modularity, coupling/cohesion trade-offs, and choosing which likely changes to make cheap. Let AI *draft and compare* designs; keep the *decision* and its rationale human and written down.

13.2.6 Static Checking and Code Review (Chapter 9)

AI both helps and harms here, sharply. AI reviewers and smarter static analyzers catch real bugs, explain them, and suggest fixes.[10] But **a growing share of code under review is now itself AI-generated** — by late 2024 Google reported more than a quarter of its new code was — which inverts the problem from Chapter 9: the scarce resource, *reviewer trust and attention*, is now spent on machine output produced faster than humans can vet it.[11]

> **Pitfall.** A 2023 controlled study found developers with an AI assistant wrote *less secure* code while being *more confident* it was secure — a false sense of safety.[12] In the AI era, static analysis and human review are the brakes that make the speed usable.

- **Human owns:** deciding *intent and trust* (does this change do what we meant?), and keeping review standards from eroding under volume. Precision/recall trade-offs for analyzers (Chapter 9) matter more as more code flows through them.

13.2.7 Testing (Chapter 10)

Testing is where agentic AI has advanced most measurably. Benchmarks like **SWE-bench** (fix a real GitHub issue) and **SWT-bench** (generate a bug-reproducing test) have driven rapid progress in automated **test generation** and **program repair**; as of 2025, agents resolve a substantial fraction of such benchmark issues end to end.[13] [14] Treat those numbers with care, though: passing a benchmark's tests is *evidence*, not proof, of a correct fix — studies of SWE-bench-style evaluation have found patches that pass the benchmark suite while failing developer-written tests or behaving differently from the true fix, so weak or incomplete test suites can inflate reported

resolution rates (the **oracle problem** again, wearing a benchmark's clothes).[15] Models are also strong at generating unit tests, property-based tests, and boundary cases (Chapter 10's black-box techniques).[16]

But the deepest problem in testing survives untouched: the **oracle problem**. A generated test encodes *what the model thinks correct behavior is* — which may simply mirror a misunderstanding also baked into the generated code. Coverage numbers can look great while the tests assert the wrong thing. Apply the same rule to an AI-generated test as to a human-written one (§10.1.5): never trust a test you have never seen fail. Introduce or restore the defect it claims to detect, confirm it fails for the expected reason, and only then count its passing result as evidence.

Chapter 10's discount example makes the failure concrete: suppose the billing spec says half-cent prices round *up*, while the generated code trips a language-specific rounding or representation trap — named in each variant's leading comment — and the generated test asserts whatever the code already returns.

```
def apply_discount(price, percent):     # AI-generated: Python round() = banker's rounding
  return round(price * (1 - percent / 100), 2)

def test_half_off():                    # AI-generated: asserts the code's own behavior
  assert apply_discount(10.25, 50) == 5.12

test_half_off()                         # passes — and every line of the unit is covered
print(apply_discount(10.25, 50))        # 5.12; the billing spec says 5.13 (half up)
```

Coverage looked great; the oracle was wrong. The suite agrees with the code because both encode the same mistake, and only a reviewer who knows the billing spec can see that the last cent rounded the wrong way — each variant's final print line shows the value the spec required.

- **Human owns:** the **oracle** — the specification of correct behavior — and the coverage *criteria* that decide when testing is enough (statement/branch/MC/DC, §§10.3–10.5). Generated tests are a starting point to be reviewed, not ground truth.

13.2.8 Security (Chapter 11)

Security is changed on both sides: AI writes vulnerabilities as fluently as it writes features, and AI-driven tools now hunt for them — autonomous pentest agents that validate findings with working exploits (§11.3). The human still owns the authorization to test, the judgment that a finding is real, and the secure-by-design decisions no scanner makes for you.

13.2.9 Quality and Metrics (Chapter 12)

AI forces a reckoning with **what we measure**. Lines of code and commit counts were always weak proxies; when a machine emits thousands of lines on request, they become actively misleading. One industry analysis reports that as AI assistance spread (2021→2024), the share of **duplicated/cloned** code rose while **refactoring** fell — a maintainability warning sign that raw output volume hides, though it awaits independent academic replication.[17]

- **Human owns:** choosing metrics that resist gaming (Chapter 12's Goodhart's-Law discipline) and that measure **outcomes** — defect-removal efficiency, customer-found defects, DORA delivery metrics (Chapter 14), verified value delivered — rather than agent *activity*. This is the empirical backbone of Outcome Engineering (§13.4).

13.2.10 The Team Project (Appendix A)

In your own project, treat agents as fast, tireless, over-confident junior teammates. Let them scaffold, draft tests, and explain unfamiliar code — but require the same evidence you would from a human: a green test suite, a passing review, and a metric that moved. Record *where* you used AI and *how you verified it*; that provenance is part of honest engineering (and of your final report).

13.3 The Evidence: Productivity, Quality, Security

A balanced reading of the current research:

Dimension	What the evidence suggests	Caveat
Productivity	Real gains on boilerplate, unfamiliar APIs, greenfield, narrow well-specified tasks.	On familiar, high-standards code, a 2025 RCT found a ~19% *slowdown*; perception overstates gains.[3]
Quality	Faster first drafts; good at tests and explanations.	Rising code duplication and falling refactoring in one industry analysis point to maintainability debt.[17]
Security	Analyzers + AI review can catch known bug patterns.	~40% of AI-generated programs in one study contained vulnerabilities; users felt *more* secure while being *less* so.[18] [12]

The through-line: **AI is a power tool, and a power tool amplifies the operator.** In skilled hands with strong verification (specs, tests, reviews, metrics), it is a real multiplier. Without those disciplines, it multiplies *output* while quietly degrading *quality* — and the operator won't feel it happening.

> **Principle.** In the AI era, the fundamentals in this book are not obsolete — they are your *verification layer*. Specs say what "correct" means; tests and reviews check it; metrics prove it in production. That layer is what turns fast generation into trustworthy software.

13.4 Outcome Engineering: The o16g Manifesto

In 2026, Cory Ondrejka (co-creator of Second Life; former engineering leader at Google and Meta; CTO of Onebrief) published the **o16g manifesto** — *Outcome Engineering* — arguing that agentic development demands a new frame.[19] [20] Its thesis: **"It was never about the code."** Code is "the incantation transforming computation into magic," a *mechanism* for delivering an idea. Once agents remove the constraint of human typing bandwidth, the manifesto argues, creation is limited by the *cost of compute*, not human capacity — and the profession should move "beyond software engineering" toward engineering **outcomes**.[19]

Treat this as an emerging framing, not a settled body of practice: at this writing the manifesto is months old, its vocabulary is still contested, and no independent evidence yet shows teams shipping better outcomes by adopting it. Read it the way you would read the Agile Manifesto in 2001 — a position paper whose worth the next decade will decide.

It is organized around four shifts and **16 principles** in two parts.[19] Whatever you make of its rhetoric, its principles map with surprising directness onto the disciplines in this book — which is itself an argument that the fundamentals endure.

The four shifts: *Creation not code. Cost not time. Capacity not backlog. Certainty not vibes.*[19]

Part I — The Goals ("superpowered creation")

1. **The Voyage — Human Intent.** Agents explore paths; humans choose the destination. Don't abdicate vision to the machine. *(This book: requirements and goals, Ch. 3.)*
2. **The Truth — Verified Reality is the Only Truth.** Code is a vanity metric; grade agents on the verified rate of positive change delivered, not lines written. *(Testing & metrics, Ch. 10, 12.)*
3. **The Teamwork — No More Single Player Mode.** Outcome engineering is a team sport; define explicit protocols for debate, decision, and delivery. *(Process, Ch. 2.)*
4. **The Liberation — The Backlog is Dead.** Never reject an idea for lack of *time*, only lack of *budget*; manage to cost, not capacity. *(Prioritization reframed, Ch. 4.)*
5. **The Joy — Unleash the Builders.** Write code only when it brings joy; delegate the toil. *(Essential vs. accidental complexity, Ch. 1.)*
6. **The Map — No Wandering in the Dark.** Never dispatch an agent without context; map the territory before building. *(Architecture description, Ch. 6.)*

7. **The Tech Island — Build It All.** When code is the cheapest resource, build to answer questions and test hypotheses. *(Prototyping to reduce risk, Ch. 2.)*
8. **The Artifacts — Failures are Artifacts.** Don't just roll back; dissect the failure and debug the *decision*, not only the code. *(Retrospectives & post-mortems, Ch. 2, 14.)*

Part II — The Building ("the iron price")

9. **The Orchestration — Agentic Coordination is a New Org.** Scaling agents mirrors scaling people, "faster, weirder, and harder"; design the org chart for the swarm. *(Team structure, Ch. 2 / Appendix A.)*
10. **The Law — Code the Constitution.** Encode mission, vision, and rules into the environment so agents can parse intent; ambiguity is the enemy of alignment. *(Specifications & acceptance criteria, Ch. 3, 5.)*
11. **The Graph — All the Context, Everywhere.** Embed context into the infrastructure (a knowledge graph), not just the prompt. *(Architectural views & documentation, Ch. 6.)*
12. **The Order — Priorities Drive Compute.** Compute is still a cost; always know the next most important task. *(Value/cost/risk prioritization, Ch. 4.)*
13. **The Documentation — Show Your Work.** Code is the *what*; reasoning is the *why* — require agents to record discoveries and rejected paths. *(ADRs & rationale, Ch. 6.)*
14. **The Immune System — Continuous Improvement.** Repeating a mistake is a system failure; spend compute on the post-mortem and inoculate against recurrence. *(Process improvement & metrics, Ch. 2, 12.)*
15. **The Gate — Risk Stops the Line.** Make risk a *blocking* function; if risk is unknown or unmitigated, the line stops. *(Static checking as a quality gate, Ch. 9.)*
16. **The Validation — Audit the Outcomes.** Trust is a vulnerability; models drift, so continuously audit agents against the domain. *(Test adequacy & monitoring, Ch. 10–11.)*

A reading: what's strong, what's open

What's compelling. The manifesto's center of gravity — *verified reality over vanity metrics, outcomes over activity, encode intent explicitly, make risk a blocking gate, audit continuously* — is essentially this book's quality philosophy, restated for a world where a machine writes the first draft. Principles 2, 10, 14, 15, and 16 are almost a summary of Chapters 9–12. Across these sources, **cheaper generation shifts more of the engineering work toward specifying desired outcomes and verifying the result.**

What's open to challenge. Treat it as a provocation, not gospel:

- *"The backlog is dead / cost not capacity"* assumes compute is cheap and value is easy to price. Prioritization under scarcity (Chapter 4) does not vanish; its currency changes.
- *"Certainty, not vibes"* is a high bar. The oracle problem (§13.2.7) means some outcomes remain expensive or impossible to verify automatically; "verified reality" has real limits.
- The **productivity paradox** (§13.1.4) cautions against assuming agents are a pure accelerator; the verification bottleneck can dominate.

- **Deskilling risk:** if engineers stop writing and reviewing code closely, who retains the judgment to *audit the outcomes*? The manifesto's own Principle 16 depends on expertise it could erode.

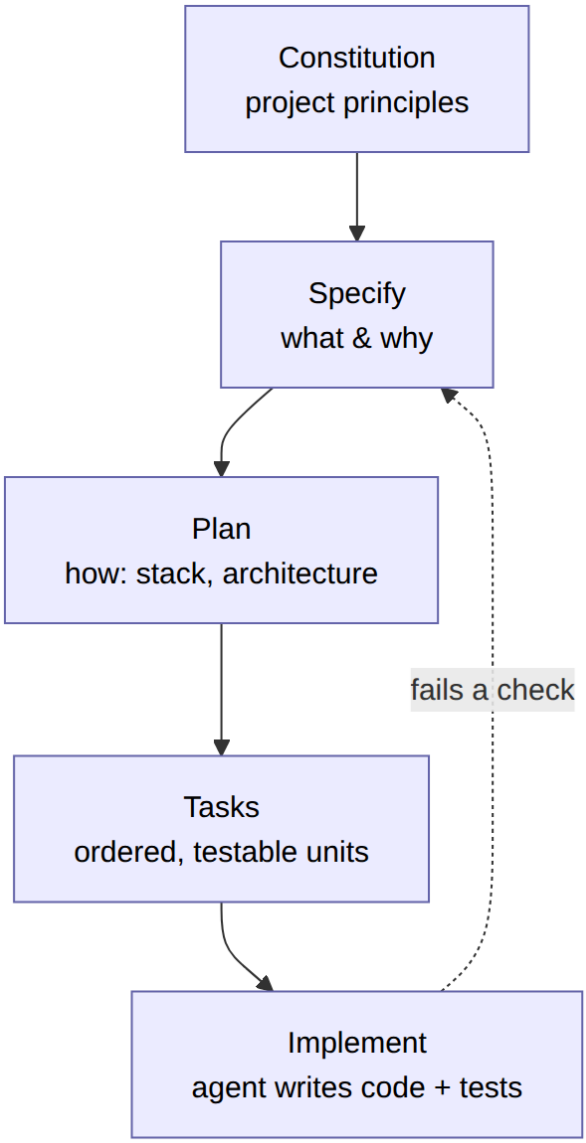
Held critically, Outcome Engineering is a useful lens: it names the shift from *producing* software to *directing and verifying* its production — which is exactly the shift this chapter has been describing.

13.5 Spec-Driven Development

The manifesto argues for encoding intent explicitly ("Code the Constitution") and treating verified reality as the only truth. **Spec-driven development (SDD)** is the concrete practice that does exactly that. Its core move is an *inversion*: in ordinary coding the specification serves the code — a document you write to explain software that already exists, and that drifts out of date the moment you stop maintaining it. In spec-driven development the code serves the specification. The spec is the durable artifact; the code is a *generated, replaceable output* of it.[21]

That inversion only became practical when a machine could regenerate the code cheaply. When typing was the bottleneck, rewriting an implementation to match a changed spec was expensive, so specs decayed. When an agent can regenerate the implementation in minutes, the spec can be the thing you maintain and the code the thing you regenerate — which is what Chapter 3 always *wanted* requirements to be.

The workflow. Open-source toolkits such as GitHub's **Spec Kit** structure SDD as a pipeline of phases with a human checkpoint between each, so the agent never runs ahead of a decision you have not made:[21]



Each phase has one job, and you do not advance until its artifact is validated:

- **Constitution** — the project's non-negotiable principles (language, testing bar, security rules, style). Written once, referenced by every later phase. This is Chapter 3's non-functional requirements and Chapter 6's constraints made machine-readable — the same idea the manifesto calls "Code the Constitution" (§13.4, P10).
- **Specify** — *what* to build and *why*, deliberately silent on the tech stack. This is user-story and acceptance-criteria work (Chapters 3, 5), often written as Given/When/Then scenarios so that "done" is executable, not a matter of taste.
- **Plan** — *how*: the architecture, stack, and interfaces the implementation will use (Chapters 6–7).
- **Tasks** — the plan decomposed into small, independently testable units, ordered by dependency: the vertical slices of Chapter 2.

- **Implement** — the agent writes code and tests against the tasks one slice at a time, so its context never fills with the whole system at once.

Between phases sit human approval gates. Their point is not ceremony; it is that a wrong decision caught in the spec is cheap to fix and the same decision discovered in production is not — §13.1.3's rising cost of a confident-but-wrong output, front-loaded to the cheapest place to catch it.

> **Principle.** In spec-driven development the specification is the *source* and the code is the *build output*. If the two disagree, the spec wins and you regenerate — which is only safe if the spec is precise enough to serve as a contract.

Early adopters report several-fold higher first-pass success from agents on non-trivial tasks when a real spec precedes the code, versus prompting from a one-line description.[21] Treat such figures as encouraging practitioner reports, not controlled measurements — but the mechanism is sound and familiar to this book: **most agent failures are underspecification failures**, and SDD front-loads the specification. The bdfinst *agentic-dev-team* project is a working template of the same shape — a `/specs → /plan → /build → /pr` loop in which acceptance-test, design, and UX *critic* agents challenge the plan before any code is written, and red-green-refactor discipline (Chapter 10) is enforced during the build.[22]

The catch is the one §13.2.7 already raised about generated tests: a spec written carelessly encodes a misunderstanding just as fluently as careless code does — and now that misunderstanding is the source of truth. SDD moves the human's effort to where it belongs, into writing an unambiguous specification. That does not remove the human from the work; it puts more weight on the part that was always the hardest.

13.6 Context as Infrastructure: `CLAUDE.md`, `AGENTS.md`, and Skills

Out of the box, an agent begins a session knowing nothing about *your* project: your conventions, your build command, which directories are off-limits, what "done" means on your team. You can paste that briefing into the prompt every time — and pay to re-explain it forever, to every agent and every session. Or you can write it down once, in a file the agent reads automatically at the start of every run. The playbook of loop engineering (§13.7) names the recurring cost of *not* doing so as **intent debt**: the price of explaining "what this project is, what the rules are, where the traps are" over and over.[23]

A small standard has emerged for paying that debt down. An **agent instructions file** — `CLAUDE.md` for Claude Code, or the tool-neutral **`AGENTS.md`** — is a Markdown briefing checked into the repository and loaded at the start of every session.[24] [25] `AGENTS.md` was formalized as an open specification in 2025 (led by OpenAI with Google, Cursor, and others) and donated to the Linux Foundation's Agentic AI Foundation in December 2025; by then more than 60,000 projects had adopted it and more than twenty agent tools could read it.[24] It is deliberately just Markdown with no required structure — "a README for agents."

Think of it as the standing brief you hand a contractor on their first morning: exactly what they need, in the order they need it, with no ambiguity tolerated. The practices that make one effective are the practices of good technical writing under a hard constraint — the file is read *every* session, so every line spends context budget:[25]

- **High-signal only.** Include what materially changes a decision — the build command, the test bar, the directories not to touch — not what the agent can infer by reading the code.
- **Imperative, not aspirational.** "Never use inline mocks; use the factories in `test/factories/`" beats "we generally try to avoid mocks."
- **Few load-bearing rules.** If every rule is marked *important*, none is. Keep the list short enough that all of it survives a skim.
- **Workflow constraints.** Say when the agent should stop and ask a human — the checkpoint the manifesto and §13.7 both insist on.

This is the manifesto's "All the Context, Everywhere" (§13.4, P11) at the scale of one repository: context embedded in the *infrastructure* rather than retyped into each prompt. It is also the mechanism behind Principle 10 — the spec-driven *constitution* of §13.5 is often exactly this file.

One standard, many dialects. `AGENTS.md` buys you portability of *facts* — the build command and the test bar are the same sentence to any agent. It does not buy portability of *tone*, because each vendor trained and tuned its model differently and each publishes its own guide for how to phrase what goes inside. When present, most agents still read their own file — `CLAUDE.md` for Claude Code, `GEMINI.md` for the Gemini CLI, `.cursor/rules/*.mdc` for Cursor, `.github/copilot-instructions.md` for Copilot — and the way a rule *lands* differs between them:

Agent	Instruction file(s)	Where its house guide lives
Claude Code (Anthropic)	`CLAUDE.md`, `.claude/skills/`	Claude Code best practices[25]
Codex (OpenAI)	`AGENTS.md`	Codex prompting guide[26]
Cursor	`.cursor/rules/*.mdc`, `AGENTS.md`	Cursor *Rules* docs[27]
Gemini CLI (Google)	`GEMINI.md`	Gemini CLI docs[27]
Copilot (GitHub)	`copilot-instructions.md`	Copilot docs[27]
portable baseline	**`AGENTS.md`**	agents.md[24]

The clearest example is **capitalization**. Anthropic's Claude Code guide explicitly lists it as a lever: "you can tune instructions by adding emphasis (e.g., `IMPORTANT` or `YOU MUST`) to improve adherence."[25] OpenAI's Codex guide says nothing of the kind; it stresses instead that instructions be *verifiable* and *decomposed* — give the agent "a specific outcome, measurable target, or test criteria," and "break [work] into smaller, focused steps."[26] So the same shouted rule that sharpens a priority for one agent may be wasted characters for another: put colloquially,

one assistant hears *emphasis* where another just hears *yelling*. The lesson is not that one guide is right — it is that instruction *phrasing does not port*, so you read the guide for the tool you actually use and tune tone in that tool's own file.

One habit does port everywhere: **explain the *why*.** A rule with its rationale — "never log raw card numbers, it violates PCI scope" — survives translation between tools and lets any model apply the principle to a case the rule never named, where a bare `NEVER LOG CARD NUMBERS` does not. That is this book's house style, and it is also the most portable prompt-writing advice there is.

> **Pitfall.** Instruction phrasing does not port cleanly between agents. A `CLAUDE.md` leaning on `IMPORTANT` / `YOU MUST` emphasis can read as noise to a different tool; a terse, verification-first `AGENTS.md` can under-specify for a model that leans on stated priorities. Keep the portable facts (commands, conventions, definition of done) in `AGENTS.md`; tune tone per tool, against that tool's published guide.

Skills push the idea one step further. Where an instructions file is standing context loaded every time, a **skill** is a named, reusable unit of project knowledge — "how we cut a release," "how we triage failing CI" — that an agent invokes only when the task calls for it. A skill can be versioned, tested, and improved like any other artifact; a prompt pasted into a script rarely is. That distinction matters for the loops of the next section: a loop that discovers its own work should trigger a *named skill*, not a wall of instructions buried in a cron job that no one will ever update.

> **Principle.** Encode project intent once, in a file the agent reads automatically, and maintain it like code. Every convention you leave only in your head is a convention the agent will violate — confidently, and at scale.

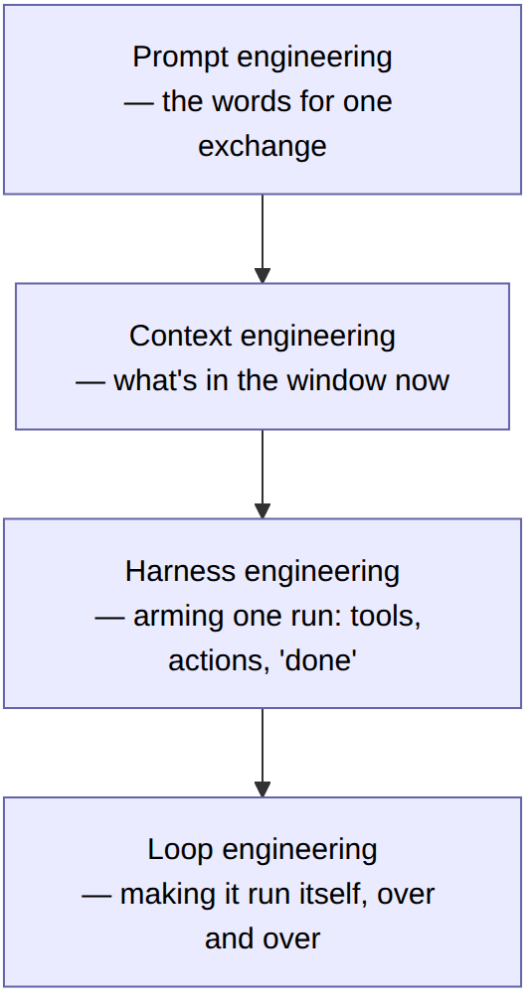
For your team project (Appendix A), an `AGENTS.md` (or `CLAUDE.md`) is a high-leverage first artifact: capture your stack, your test and lint commands, your definition of done, and your review rules. It is living documentation that agents obey and humans can read — and writing it forces the team to make its tacit conventions explicit, which is worth doing even if no agent ever reads it.

13.7 Loop Engineering

The rungs of §13.1.3 ran from autocomplete to autonomous agents. There is a further rung beyond the ones drawn there, and in 2026 it acquired a name. Within a single week that June, three practitioners independently described the same shift: Peter Steinberger argued that the relevant skill was no longer prompting agents but designing the loops that prompt them; Boris Cherny, who built Claude Code, said from the vendor side that his job was now to write the loops that drive the agent; and Addy Osmani wrote it up and named it **loop engineering**.[23] Its one-line

definition is deliberately unsettling: *loop engineering is replacing yourself* — designing the system that prompts the agent, so that you no longer feed it one instruction at a time.[23]

Four layers. Loop engineering is the widest layer of a progression the industry backed into, each layer minding something larger than the one before it:



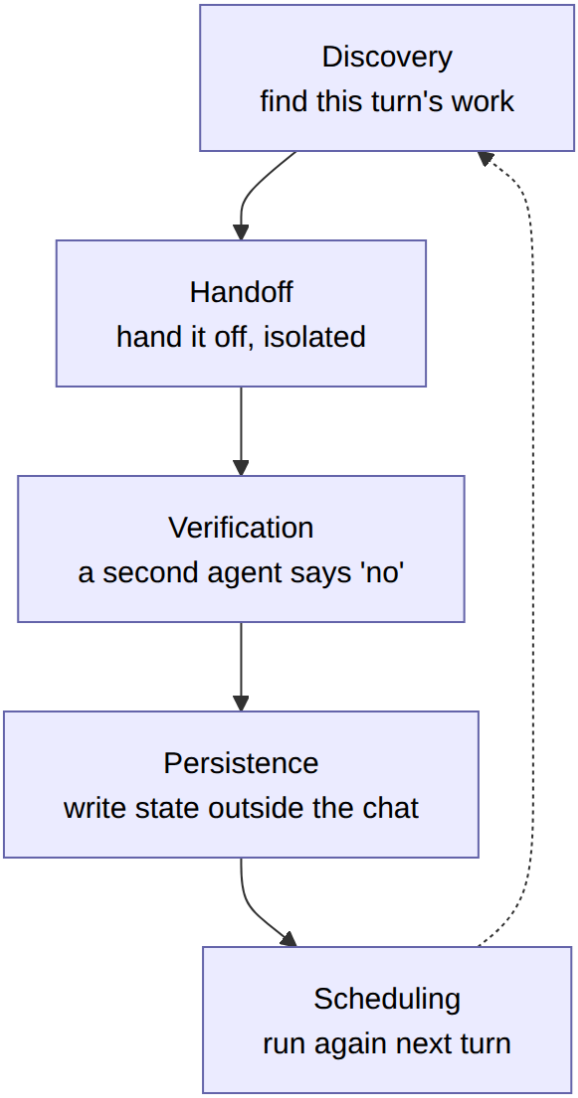
Prompt engineering minds one exchange; context engineering minds one window (§13.6); harness engineering arms one run (which tools an agent may call, and what counts as finished); loop engineering minds the widest scope of all — the system that runs the harness again and again with no human hand between turns. The first three layers still assume a person at the keyboard, directing the agent line by line. Loop engineering removes that assumption: the practitioner moves from *inside* the loop to *outside* it, building the loop rather than being it.[23]

This layer belongs in a discussion of engineering discipline because **it increases the consequences of mistakes.** As §13.1.3 noted, a confident but incorrect output becomes more costly as the system performs more actions before a person reviews it. A loop is, by construction, a machine for *maximizing* that distance — it can run all

night, change code you never looked at, and feed its own output back as tomorrow's input. Every discipline below exists to shorten the gap between a mistake and its discovery.

13.7.1 The Anatomy of a Loop

A useful loop does five things each turn, and drops any one of them at its peril:[23]



- **Discovery** — the loop finds its own work (reads failing CI, open issues, recent commits) rather than being handed a task list. Ideally this logic lives in a *skill* (§13.6), not a static prompt.
- **Handoff** — each unit of work goes to an agent in an isolated workspace. In git that means a separate **worktree** per task, so parallel agents editing the same repository do not overwrite one another.
- **Verification** — a *separate* agent, defaulting to skeptical, checks the work. This is the load-bearing move; §13.7.2 is about why.

- **Persistence** — results land somewhere that outlives the conversation: a pull request, a ticket, a state file on disk. The agent forgets when its context clears; the repository does not.
- **Scheduling** — an automation (a timer, a CI trigger, a chat reaction) makes the loop run again without anyone remembering to start it. This is what turns a script you run by hand into a loop that runs itself.

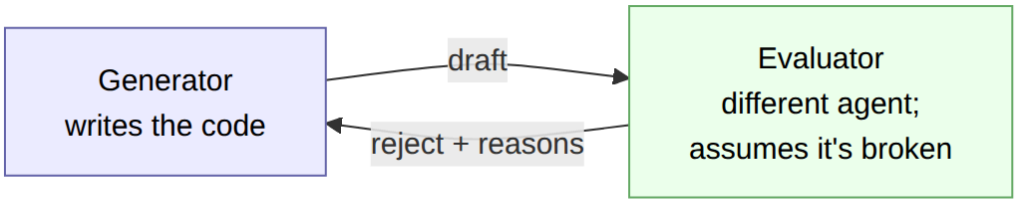
Miss one move and the loop fails in a characteristic way. The anti-patterns are the shadows of the skipped moves:[23]

Move skipped	How the loop fails
Verification	*The nodding loop* — the agent grades its own work, approves it, and accumulates plausible mistakes at machine speed. The most common failure.
Persistence	*The amnesiac loop* — good work is done, then forgotten when context clears; the next run redoes it or collides with it.
Scheduling	*The manual loop* — five good moves that a human still has to remember to trigger, and eventually won't.
Discovery	*The blind loop* — the human still hand-feeds work each morning, so the expensive part (deciding *what* to do) was never automated.
Handoff	*The tangled loop* — parallel agents share one working directory and their edits collide into a mess no one can untangle.

13.7.2 The Generator and the Evaluator

The hardest part of a loop is not getting an agent to *do* the work — it is building something that can say **"no."** And the agent that wrote the code is the worst possible judge of it.

Ask an agent to grade what it just produced and it tends to praise it, because the context in which the code was written is already full of the reasons it was written that way; looking back, the agent sees its own justification, not the result. Anthropic's engineering team, building agents that code autonomously for hours, hit this directly: agents overrate their own output, especially on subjective work.[28] Their fix — borrowed explicitly from **generative adversarial networks (GANs)** — was structural, not a matter of wording. You cannot reliably make an author self-critical, but you can hand the work to a *different* agent, with different instructions, that carries none of the first one's self-persuasion:[28]



Two refinements make the evaluator real. First, an evaluator should **execute the software and inspect its observed behavior**, not only read the code. Anthropic's evaluator used browser automation to operate the live application, capture screenshots, and grade the result against calibrated criteria.[28] Second, it should **begin from skepticism**: treat the code as unverified until a check demonstrates that it behaves correctly. This is the old **maker–checker** principle from banking — the person entering a large transfer and the person approving it must differ — applied to the stop condition of a loop.

> **Pitfall.** A loop whose only reviewer is the agent that wrote the code is a loop nodding at itself. Coverage numbers, green checkmarks, and confident self-assessment all rise together while real quality does not — the §13.2.7 oracle problem, now running unattended. The reliable fix is structural: a *separate* evaluator, on a different model where you can, that defaults to "no."

This is §13.4's Principle 2 ("Verified Reality is the Only Truth") and §13.3's through-line, arriving from the practitioners' side: **when generation is nearly free, the scarce resource is judgment**, and a loop's judgment lives in its evaluator. A strong generator with a weak evaluator produces confident garbage, faster; a modest generator with a sharp evaluator produces slow, reliable progress — and only the second compounds safely over many turns.

13.7.3 The Costs That Accrue Silently

A loop that runs itself is also a loop that makes mistakes by itself, and the more cheerfully it runs, the more quietly it errs. Four costs build up with no alarm while the loop is running, and they reinforce one another:[23]

- **Verification debt** — merged, unverified output piling up in the gap between "it ran" and "it's right," waiting for one morning when it comes due all at once.
- **Comprehension rot** — the codebase grows faster than anyone's understanding of it, because reading generated code is duller than writing it and the loop never stops to read. A bug in a corner no human has read tends to surface first as a production incident.
- **Cognitive surrender** — the gradual loss of willingness to examine or challenge the system's output. As the loop becomes more reliable, users may stop forming an independent judgment.
- **Token blowout** — the only cost that hits the bill directly. An idle bug can spin all night, burning budget on rounds that produce nothing, unless a hard cap stops it.

The defenses are inexpensive and familiar. **Read a representative sample of the loop's output each day** to preserve comprehension. **Set hard budget caps before unattended execution begins. Keep at least one human**

checkpoint where the loop pauses for review, so that people remain able and prepared to intervene.

> **Case study — Stripe's "Minions."** Stripe's internal coding agents produce roughly 1,300 pull requests each week after being triggered by an emoji reaction in Slack.[29] Their reliability comes from the *constraints* surrounding the harness, which is a fork of the open-source *Goose*. A deterministic orchestrator assembles context and runs hard-coded quality gates (a linter the agent cannot skip, then a commit step) before and after the probabilistic model does its part; anything a rule can decide is kept out of the model's hands. Every one of those PRs is still reviewed by an engineer — the humans did not leave, their time moved from writing to reviewing. Reliability came from the discipline around the loop, not the size of the model in it.[29]

13.7.4 Stay the Engineer

The unsettling framing — "loop engineering is replacing yourself" — has a sharp corollary. A loop is a faithful *multiplier* of whatever its builder brings: give it sound judgment and it executes good decisions a hundredfold; give it a lapse and it executes the lapse a hundredfold, faithfully, without ever pausing to ask whether it was right.[23] The same loop, built by two engineers, can end in opposite places — one who reads the code and holds a firm sense of direction uses it to move faster on work they have mastered; one who stops reading uses it to never have to understand again, and six months later is the gatekeeper of a machine they can no longer audit.

That is the deskilling risk of §13.4 stated operationally, and it resolves the same way: the human review point is not a temporary scaffold to remove once the loop is trusted. It is the permanent feature that *keeps* the loop trustworthy, and the day it is removed is the day comprehension rot begins in earnest. Build the loop in a way that preserves your ability and responsibility to review its work.

> **Principle.** Stop prompting the agent; design the system that prompts it. But design that system so a human can always still say "no" — a loop can execute judgment, it cannot supply it.

Start small. A first loop should be almost embarrassingly modest: one finding, discovered and handled end to end, on a timer, with a separate check and a budget cap. Add parallelism *last*, after the checks have caught real mistakes — the Stripe pipeline is the *end* of that road, hardened over years, not the place to begin.

13.8 Principles for the AI-Augmented Engineer

Synthesizing the evidence and the manifesto into working advice:

1. **Own intent and verification.** Choose the destination and define "correct"; let agents explore paths. You are accountable for what ships (Chapter 1's ethics do not delegate).
2. **Keep the fundamentals.** You cannot safely accept what you cannot evaluate. Understand the design, the tests, and the metrics well enough to catch a confident wrong answer.

3. **Shift your effort up the stack.** Spend the time AI frees on *specifying* (clear requirements, acceptance criteria, ADRs) and *checking* (reviews, coverage, metrics) — not on generating more code.
4. **Manage the new risks.** Security defects, license/provenance of generated code, hallucination, over-trust, and skill atrophy are now first-class engineering concerns.
5. **Measure outcomes, not activity.** Judge yourself and your agents by verified value delivered and defects escaped — never by lines or PR counts.
6. **Automate the loop, not the judgment.** As agents begin to run themselves, encode your intent where they read it (§13.6), specify before you generate (§13.5), and keep a separate check and a human gate in every loop (§13.7). Speed is only safe behind verification you control.

13.9 Conclusion

AI does not repeal the four pressures of software engineering; it *relocates* their weight. Producing code gets cheaper, so **deciding what to build and proving it works** — the subject of every chapter in this book — becomes the differentiator. The disciplines you learned are promoted, not made quaint. Requirements are now what you hand the agent to build from. The specification becomes the constitution it works under. Tests and reviews are the layer that decides whether the volume of generated code can be trusted, and metrics are the evidence that a real *outcome* shipped rather than mere output.

The o16g manifesto pushes this to its edge — *it was never about the code* — and whether or not you accept its every claim, it points where the profession may be heading: engineers who direct fleets of agents toward well-specified, rigorously verified outcomes. The practices are arriving to match — you write the spec, encode the constitution in a file the agents read, and design the loop that runs them — but in every one of them the human's job is the same: own the intent, keep the check, and stay able to say "no." That job is not less engineering. It is *more* of the hardest, most human parts of it.

Sources

[1] GitHub (Eirini Kalliamvakou), *Research: Quantifying GitHub Copilot's Impact on Developer Productivity and Happiness* (2022). github.blog.

[2] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer, *The Impact of AI on Developer Productivity: Evidence from GitHub Copilot* (2023). arXiv 2302.06590.

[3] METR (Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein), *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity* (2025). metr.org · arXiv 2507.09089.

[4] Kent Beck et al., *Principles behind the Agile Manifesto* (2001). agilemanifesto.org.

[5] Madhava Krishna, Bhagesh Gaur, Arsh Verma, and Pankaj Jalote, *Using LLMs in Software Requirements Specifications: An Empirical Evaluation* (2024). arXiv 2404.17842.

[6] Asma Yamani, Malak Baslyman, and Moataz Ahmed, *Leveraging LLMs for User Stories in AI Systems: UStAI Dataset* (2025). arXiv 2504.00513.

[7] Haowei Cheng et al., *Generative AI for Requirements Engineering: A Systematic Literature Review* (2024). arXiv 2409.06741.

[8] Sirui Hong et al., *MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework* (2023). arXiv 2308.00352.

[9] Ruiyin Li et al., *Bridging Requirements and Architecture: Multi-Agent Orchestration with External Knowledge and Hierarchical Memory* (2026). arXiv 2606.01385.

[10] Google Research, *Resolving Code Review Comments with Machine Learning* (2023). research.google.

[11] Sundar Pichai, Alphabet Q3 2024 earnings-call remarks — more than a quarter of new code at Google is AI-generated, then reviewed by engineers — as reported by Fortune (2024). fortune.com.

[12] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh, *Do Users Write More Insecure Code with AI Assistants?* (CCS 2023). arXiv 2211.03622.

[13] Carlos E. Jimenez et al., *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* (2023). arXiv 2310.06770 · leaderboard at swebench.com.

[14] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev, *SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents* (2024). arXiv 2406.12952.

[15] Reem Aleithan et al., *SWE-Bench+: Enhanced Coding Benchmark for LLMs* (2024). arXiv 2410.06992.

[16] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip, *An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation* (2023). arXiv 2302.06527.

[17] GitClear (William Harding et al.), *AI Copilot Code Quality: 2025 Research* (2025). gitclear.com.

[18] Hammond Pearce et al., *Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions* (2021). arXiv 2108.09293.

[19] Cory Ondrejka, *Outcome Engineering — The o16g Manifesto* (2026). o16g.com.

[20] Onebrief, *Onebrief Hires Cory Ondrejka as Chief Technology Officer* (2026). businesswire.com.

[21] GitHub, *Spec Kit — a toolkit for spec-driven development* (2025–2026). github.com/github/spec-kit · intro at github.blog. Source of the constitution → specify → plan → tasks → implement phases and the "code serves the spec" inversion; first-pass-success figures are early-adopter reports, not controlled studies.

[22] bdfinst, *agentic-dev-team* — Claude Code plugins for a spec-to-shipping workflow (2026). github.com/bdfinst/agentic-dev-team. The `/specs → /plan → /build → /pr` loop with review-critic personas and enforced red-green-refactor gates.

[23] Addy Osmani, *Loop Engineering* (2026). addyo.substack.com · overview at O'Reilly Radar. The name and one-line definition, the four-layer stack, the five moves and their anti-patterns, *intent debt*, and the four silent costs (the term surfaced the same week from Peter Steinberger and Boris Cherny, June 2026).

[24] *AGENTS.md — an open standard for agent instruction files*. agents.md. Formalized as an open spec in 2025 (OpenAI with Google, Cursor, and others) and donated to the Linux Foundation's Agentic AI Foundation in December 2025; adopted by 60,000+ projects and 20+ tools.

[25] Anthropic, *Manage Claude's memory* — Claude Code documentation. code.claude.com/docs/en/memory. `CLAUDE.md` loading and the practices that make an instructions file effective (high-signal, imperative, few load-bearing rules).

[26] OpenAI, *Codex — Prompting* and *Best practices* (2026). developers.openai.com/codex/prompting · /codex/learn/best-practices. Stresses verifiable, decomposed instructions with measurable success criteria; unlike Anthropic's guide, it gives no capitalization/emphasis advice. See also Anthropic, *Best practices for Claude Code* — code.claude.com/docs/en/best-practices — for the `IMPORTANT`/`YOU MUST` emphasis lever quoted in §13.6.

[27] Per-tool instruction conventions: Cursor, *Rules* — cursor.com/docs/rules; Google, *Provide context with `GEMINI.md` files* (Gemini CLI) — geminicli.com/docs/cli/gemini-md; GitHub, *Adding repository custom instructions for Copilot* — docs.github.com.

[28] Anthropic, *Harness design for long-running application development* (2026). anthropic.com/engineering. The generator–evaluator (and planner) architecture, its GAN inspiration, browser-driven evaluation, and the finding that agents overrate their own output.

[29] Steve Kaliski (Stripe), *How Stripe built "minions" — AI coding agents that ship 1,300 PRs weekly from Slack reactions*, *How I AI* podcast (2026). lennysnewsletter.com · coverage at InfoQ. Deterministic gates around a *Goose* fork; reliability from constraints, not model size.

- **Key takeaways** are summarized above in §13.9.
- Continue to the Exercises.
- Go deeper with the Open Resources for this chapter.

Chapter 13 — Exercises

Tags: **[warm-up]** checks understanding · **[analysis]** asks you to reason.

Concepts

1. **[warm-up]** In your own words, restate the distinction between *essential* and *accidental* complexity (Chapter 1) and give one software task at each end that AI handles well or poorly. Why does the boundary predict where AI helps?
2. **[warm-up]** Name the five rungs of the "ladder of assistance" (§13.1.3). For each, state one thing the *human* must still do.
3. **[warm-up]** What is the **oracle problem**, and why does it limit AI-generated tests even when coverage numbers look high?

Analysis

4. **[analysis]** The 2025 METR trial found experienced developers were ~19% *slower* with AI while believing they were faster. Give two plausible mechanisms for the gap, and describe an experiment (using Chapter 12 methods) you could run on your own team to measure *your* real speedup instead of trusting the feeling.
5. **[analysis]** Pick any three lifecycle stages from §13.2. For each, write one sentence on what AI does well and one on the specific fundamental a human still owns. Cite the chapter each maps to.
6. **[analysis]** §13.2.6 says AI "inverts" the code-review problem. Explain the inversion, and propose two concrete team policies that keep reviewer trust (Chapter 9) from eroding when most code under review is machine-generated.
7. **[analysis]** The security evidence (§13.3) shows users of AI assistants writing *less* secure code while feeling *more* secure. Which cognitive bias from Chapter 4 does this resemble, and what process safeguard would you add?

The o16g manifesto

8. **[analysis]** Map any **five** of the sixteen o16g principles to specific chapters of this book, and for each say whether it *restates* an existing discipline or proposes something genuinely new.
9. **[analysis]** The manifesto declares "The Backlog is Dead — manage to cost, not capacity." Argue *for* and *against* this claim using Chapter 4's prioritization tools. Under what conditions does it hold?
10. **[analysis]** Principle 16 ("Audit the Outcomes") depends on human expertise, yet heavy reliance on agents may erode that expertise (the *deskilling* risk). Explain the tension and propose how a team could stay able to audit what its agents produce.

11. **[analysis]** “Certainty, not vibes” is the manifesto’s verification slogan. Give one kind of outcome that is cheap to verify automatically and one that is expensive or impossible, and connect the second to the oracle problem.

Agentic practice

12. **[warm-up]** State the “inversion” at the heart of spec-driven development (§13.5) in one sentence. Which two chapters supply the skills the *Specify* phase depends on?
13. **[warm-up]** Name the five moves a loop makes each turn (§13.7.1). For each, name the anti-pattern that appears when it is skipped.
14. **[analysis]** Write a first-draft `AGENTS.md` (about 10–15 lines) for your team project (Appendix A). Then justify each line against the §13.6 tests: does it materially change a decision, or could the agent infer it by reading the code? Delete anything that fails.
15. **[analysis]** §13.7.2 argues the agent that wrote the code is the worst judge of it, and the fix is a *separate* evaluator that defaults to “no.” Explain why this is the same problem as the oracle problem (§13.2.7), and why swapping in a different model or agent helps where “asking the author to be more critical” does not.
16. **[analysis]** Pick two of the four silent costs of a running loop (§13.7.3) — verification debt, comprehension rot, cognitive surrender, token blowout. For each, describe a concrete symptom you would expect to see first, and the specific discipline (from this book) that guards against it.
17. **[analysis]** Stripe’s “Minions” ship ~1,300 PRs a week on a fork of a mid-tier open-source harness, not a frontier model (§13.7.3). Explain, using the five moves, where that pipeline’s reliability actually comes from — and why “use a bigger model” would not have produced it.
18. **[analysis]** §13.6 argues that an `AGENTS.md` ports *facts* between tools but not *tone* — Anthropic documents `IMPORTANT` / `YOU MUST` emphasis as a lever, while OpenAI’s Codex guide stresses verifiable, decomposed instructions instead. Take one rule from your team’s `AGENTS.md` and rewrite it two ways: once emphasis-first, once rationale-first (“do X *because* Y”). Which version would you expect to survive being read by a *different* agent than the one you wrote it for, and why?

Chapter 13 — Open Resources

Free, open-licensed, or freely accessible primary sources behind this chapter. Types: 📘 open text · 🎓 course · 📄 primary source/paper · 🎥 video. Licenses vary and are noted where known. AI moves fast — prefer primary studies over vendor blogs, and check dates.

The o16g manifesto (used in §13.4)

- 📄 **o16g — the Outcome Engineering manifesto**, Cory Ondrejka — o16g.com/manifesto. The 16 principles and the "it was never about the code" thesis. Read it in full and form your own view.
- 📄 Onebrief, **"Onebrief Hires Cory Ondrejka as Chief Technology Officer"** (2026) — businesswire.com. Background on the manifesto's author.

Productivity evidence (used in §13.1.4, §13.3)

- 📄 METR, **"Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity"** (2025) — metr.org study · arXiv 2507.09089. The randomized trial that found a ~19% slowdown and a large perception gap.
- 📄 **"The Impact of LLM-Assistants on Software Developer Productivity: A Systematic Review"** (2025) — arXiv 2507.03156.
- 📄 GitHub, **"Research: Quantifying GitHub Copilot's Impact on Developer Productivity and Happiness"** (2022) — github.blog. The perceived-productivity survey.
- 📄 Peng et al., **"The Impact of AI on Developer Productivity: Evidence from GitHub Copilot"** (2023) — arXiv 2302.06590. The vendor experiment behind the eye-catching "55.8% faster" number.
- 📄 Sundar Pichai's Alphabet Q3 2024 earnings-call remark that more than a quarter of new code at Google is AI-generated, then reviewed by engineers — reported by Fortune.
- 📄 **DORA / State of DevOps** reports on AI's effect on delivery — dora.dev.

Code quality & security (used in §13.3)

- 📄 GitClear, **"AI Copilot Code Quality: 2025 Research"** (rising code clones, falling refactoring) — gitclear.com research.
- 📄 Perry et al., **"Do Users Write More Insecure Code with AI Assistants?"** (Stanford) — arXiv 2211.03622. The "false sense of security" study.
- 📄 Pearce et al., **"Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions"** — arXiv 2108.09293.

- 📄 Google Research, **“Resolving Code Review Comments with Machine Learning”** (2023) — research.google. ML-suggested fixes for review comments at Google scale.

AI in requirements, design, testing (used in §13.2)

- 📄 **“Generative AI for Requirements Engineering: A Systematic Literature Review”** — arXiv 2409.06741.
- 📄 **“Leveraging LLMs for User Stories in AI Systems (UStAI)”** (FSE 2025) — arXiv 2504.00513.
- 📄 Krishna et al., **“Using LLMs in Software Requirements Specifications: An Empirical Evaluation”** (2024) — arXiv 2404.17842. LLM-drafted SRS quality vs. entry-level engineers, plus time/cost savings.
- 📄 Hong et al., **“MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework”** (2023) — arXiv 2308.00352. Role specialization to reduce errors in multi-agent software generation.
- 📄 Li et al., **“Bridging Requirements and Architecture: Multi-Agent Orchestration with External Knowledge and Hierarchical Memory”** (2026) — arXiv 2606.01385. Specialized agents that turn requirements into candidate architectures.
- 📄 Schäfer et al., **“An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation”** (2023) — arXiv 2302.06527.
- 📄 **“Context Matters: Evaluating Context Strategies for Automated ADR Generation Using LLMs”** (architecture decisions) — arXiv 2604.03826.
- 📄 **SWE-bench** (resolve real GitHub issues) — swebench.com · arXiv 2310.06770; and **SWT-bench** (generate bug-reproducing tests) — arXiv 2406.12952.
- 📄 Aleithan et al., **“SWE-Bench+: Enhanced Coding Benchmark for LLMs”** (2024) — arXiv 2410.06992. Solution leakage and weak test suites can inflate reported benchmark resolution rates.

Agentic development in practice (used in §§13.5–13.7)

- 📄 GitHub, **Spec Kit** — an open toolkit for spec-driven development — github.com/github/spec-kit; intro on the GitHub Blog. The constitution → specify → plan → tasks → implement pipeline.
- 📄 `AGENTS.md` — the open standard for agent instruction files — agents.md. “A README for agents”; donated to the Linux Foundation’s Agentic AI Foundation in December 2025.
- 📄 Anthropic, **“Manage Claude’s memory”** (Claude Code docs) — code.claude.com/docs/en/memory. How `CLAUDE.md` is loaded and what makes an instructions file effective.
- 📄 Addy Osmani, **“Loop Engineering”** (2026) — addyo.substack.com; overview at O’Reilly Radar. The four-layer stack, the five moves, the anti-patterns, and the four silent costs.
- 📄 Anthropic, **“Harness design for long-running application development”** (2026) — anthropic.com/engineering. The GAN-inspired planner / generator / evaluator architecture and why agents overrate their own output.
- 🎥 Steve Kaliski, **“How Stripe built ‘minions’”** (*How I AI*, 2026) — lennysnewsletter.com; coverage at InfoQ. ~1,300 PRs/week from deterministic gates around a *Goose* fork.

- 📄 `bdfinst/agentic-dev-team` — Claude Code plugins for a spec-to-shipping workflow — github.com/bdfinst/agentic-dev-team. A working `/specs → /plan → /build → /pr` template with critic agents and TDD gates.

Per-tool instruction guides (used in §13.6)

Each agent ships its own guide for how to phrase instructions — the *facts* port via `AGENTS.md`, the *tone* does not. Read the guide for the tool you actually use.

- 📄 Anthropic, **"Best practices for Claude Code"** — code.claude.com/docs/en/best-practices and **"Manage Claude's memory"** — …/memory. Source of the `IMPORTANT` / `YOU MUST` emphasis lever and the "prune ruthlessly" rule.
- 📄 OpenAI, **"Codex — Prompting"** — developers.openai.com/codex/prompting — and **"Best practices"** — …/learn/best-practices. Verifiable, decomposed instructions; no capitalization advice.
- 📄 Cursor, **"Rules"** — cursor.com/docs/rules. Glob-scoped `.mdc` rule files (and native `AGENTS.md` support).
- 📄 Google, **"Provide context with `GEMINI.md` files"** (Gemini CLI) — geminicli.com/docs/cli/gemini-md.
- 📄 GitHub, **"Adding repository custom instructions for Copilot"** — docs.github.com. The `.github/copilot-instructions.md` convention.

Foundations & agentic engineering

- 📄 **Manifesto for Agile Software Development** — agilemanifesto.org (the historical counterpoint the o16g manifesto positions itself against).
- 📄 **"Toward Agentic Software Engineering Beyond Code: Vision, Values, and Vocabulary"** — arXiv 2510.19692.
- 🎥 Talks and courses on AI-assisted development appear frequently; prefer university/primary sources and always check the publication date.

License note

Linked resources remain under their own licenses; this page is CC BY-SA 4.0. The o16g manifesto is quoted briefly for commentary and attributed to its author.

Chapter 14 — Delivery: CI/CD, DevOps, and Operations

> **Where we are.** Chapter 13 asked how AI changes the practice of software engineering; this chapter asks a question the earlier chapters quietly deferred. Chapters 1–12 taught you how to *build* software — process, requirements, design, patterns — and how to *verify* it with reviews, tests, security scanning, and metrics. But a verified commit sitting in a repository helps no one. This chapter covers the stretch of engineering between "the code is written" and "users are running it"; Chapter 15 covers the longer stretch after that — what happens to code once it has been in production for years. **Delivery** is where every earlier discipline either becomes real or stays theoretical.

Delivery used to be an afterthought — a thing operations people did after engineering was "done." Two shifts ended that. First, software moved to the cloud, where releasing stopped being a rare, ceremonial event and became something a team might do dozens of times a day. Second, a body of research (§14.7) showed that *how* a team delivers predicts its performance better than almost anything else it does. The umbrella term for the resulting culture is **DevOps**: the idea that developing software and operating it are one discipline, practiced by one team, with shared tools and shared accountability. This chapter walks the pipeline from commit to production, studies two of the most instructive deployment disasters on the public record, shows concretely how a service is packaged and put online — containers, a database, a domain, a certificate, and the edge — and closes with the research on why delivery performance predicts so much else. The fate that awaits all successful code — becoming legacy — is Chapter 15.

> **Principle.** Undeployed code is inventory, not value. Every practice in this chapter exists to shrink the time and the risk between *writing* a change and *learning* what it does in the hands of real users — the same short-feedback bet Chapter 2 made about process, now applied to the machinery of release itself.

14.1 SaaS and the Cloud

14.1.1 From Shipped Artifact to Running Service

For most of software history, delivering software meant producing an **artifact** — a binary on tape, a shrink-wrapped CD, an installer download — that customers took away and ran on *their* machines. The vendor's job ended at the factory gate. Under that model, releases are naturally rare and heavy: every artifact that leaves the building is effectively unpatchable for months, so it must be as close to perfect as testing can make it, and mistakes are answered with recalls, patch disks, and support calls.

Software as a service (SaaS) inverts the arrangement: the software runs on machines the *vendor* controls, and users reach it over the network, usually through a browser or a thin client. In its pure form, nothing ships. That single change rearranges almost everything downstream. The provider controls what runs in production — canaries, staged rollouts, and feature flags may briefly mix versions, but there is no museum of old customer installs to support. The vendor can observe actual usage through logs, errors, and performance data, reducing its dependence on support reports. And because the vendor owns the machines, an update is an internal operation rather than a request to thousands of customers: it can happen at any time, invisibly, many times a day.

Customers also benefit from the move toward SaaS: their data lives on the service, so a stolen laptop or a dead phone loses a device, not the work; several people can collaborate on the *same* data at once instead of emailing copies back and forth; and datasets that are large or constantly changing can live in one central, managed place rather than being squeezed onto whatever machine each user happens to own. For a whole class of applications — shared documents, shared calendars, anything with one truth that many people touch — the service model is both cheaper to operate and a better product.

On the user's side of the wire, the near-universal thin client is the **browser**: a program every device already has, speaking a front-end stack of **HTML** for the document's structure, **CSS** for its presentation, and **JavaScript** for behavior that runs on the client — while the server side can be written in whatever language the team prefers, because the browser never sees it. Native mobile apps are the sibling front end, trading the browser's universality for tighter platform integration; either way, the architecture is the same service behind a thin client.

14.1.2 Multi-Tenancy and Horizontal Scaling

Running the software yourself raises a design question: does each customer get their own copy? In a **multi-tenant** design, one running instance of the system serves many customers — *tenants* — whose data is kept logically separate inside shared infrastructure. Multi-tenancy is what makes SaaS economical: one fleet, one deployment, one upgrade for everyone, with the cost of operations amortized across all tenants. The price is that isolation becomes a software problem. A bug that leaks one tenant's data to another, or one tenant's heavy load that starves the rest (the "noisy neighbor"), are failure modes a shipped artifact simply could not have.

Serving many tenants also means serving unpredictable load, and the cloud's answer is **horizontal scaling**: instead of buying a bigger machine (*vertical* scaling), you run more copies of the same service behind a load balancer and add or remove copies as demand moves. Horizontal scaling only works cleanly if any copy can serve any request — which is exactly why the statelessness convention of RESTful design matters (§7.5.4). A service that keeps per-client session state on one server has silently welded each user to that server; a stateless one lets the platform treat servers as interchangeable, replaceable cattle. Architecture decisions from Chapter 7 are, it turns out, *delivery* decisions too.

14.1.3 What the Cloud Does to the Iron Triangle

Recall the iron triangle of Chapter 1 (§1.5): scope, schedule, and cost constrain each other, and the classic pressure point was the *release date* — a single, high-stakes deadline toward which scope was cut and quality was crunched. SaaS dissolves that pressure point. When deploying is cheap and continuous, **release stops being an event and becomes a decision** — a small, revocable, per-feature decision made many times a week. You no longer ask "what can we finish by the ship date?" but "is this one change ready to meet users now?" Schedule pressure does not vanish; it decomposes into many tiny pressures, each too small to justify heroics.

That is the economic root of the **deploy-early culture**. Teams that deploy each small change as it is finished get feedback while the change is still fresh in their heads, keep the gap between "works on my machine" and "works in production" permanently small, and never build up a terrifying pile of unreleased work whose first contact with reality is a big bang — the same pathology, at the release level, that §2.4.1 diagnosed at the integration level. The practices in the remainder of this chapter reduce the risk of early deployment.

14.1.4 The Cloud Landscape: Providers, Components, and Responsibility

The vendor-controlled machines of §14.1.1 have to live somewhere, and today they mostly live in a **public cloud**: vast provider-owned datacenters whose capacity is rented out over the network, by the hour or by the request. As of 2025, the market is led by Amazon Web Services (AWS), Google Cloud, and Microsoft Azure, with a long tail of smaller providers — DigitalOcean, Linode, Hetzner — that trade breadth of catalog for simplicity and price.[1] Hold the names loosely: provider names and product lists date quickly, but the concepts beneath them do not. Every cloud, whatever its branding, sells the same four component families. **Compute** is processing capacity — virtual machines, containers, or functions that run your code. **Storage** is durable data — disks, object stores, managed databases. **Network** is the plumbing — private networks, load balancers, DNS — that lets those pieces reach each other and the world. And **security and identity** is the control plane over the other three: who, and which programs, may do what to which resources.

The cloud's economic promise is threefold. You pay only for what you use, turning a large up-front hardware purchase into a metered utility bill. You can place your system in regions around the world, near your users, without signing a lease on another continent. And the hardware is someone else's problem — provisioning, failed disks, power, cooling, and physical security are all handled below the waterline of the service you rent.

What the provider explicitly does *not* handle is captured by the **shared responsibility model**: the provider secures the cloud — the facilities, the hardware, the hypervisor — while you secure what is *in* it — your operating-system configuration, your code, your data, and who can access all three.[2] This is a legal line as much as a practical one, and teams forget it at their peril: a world-readable storage bucket full of customer records is *your* breach, however secure the datacenter around it was.

None of this makes the cloud automatic. The alternatives — **on-premises** servers you own, or rented space and machines in a shared datacenter (**co-location**) — never went away, and the honest question is when each wins.

Cloud makes clear sense when you do not yet know what hardware you need, when demand is spiky or unpredictable (provision for the spike and pay for it forever, or scale for the hour it lasts), when speed to a first MVP matters more than unit cost, and when a small team has no one to spare for operations.

> **Case study.** *Cloud repatriation.* The trade cuts the other way, too. In 2022–23, 37signals — the company behind Basecamp — publicly moved its products off the cloud and onto purchased hardware, documenting seven-figure annual savings for workloads that were steady and predictable rather than spiky.[3] [4] And Amazon's own Prime Video team published an account of cutting the cost of one audio/video monitoring service by roughly 90 percent — by moving it *away* from a serverless, distributed-microservices design and back into a monolith-style process.[5] None of this means the cloud is over. The cloud is a *cost-and-flexibility trade*, not an axiom. Elastic, uncertain load is where renting wins; steady, predictable load is where owning wins. Run the numbers, not the fashion.

14.1.5 Containers, Clusters, and Kubernetes

The unit of cloud deployment that has won in practice is the **container**: a package containing an application plus everything it needs to run — language runtime, libraries, configuration — isolated from other software on the same machine while *sharing the host operating system's kernel*. Sharing the kernel is what distinguishes a container from a **virtual machine**, which carries an entire guest operating system of its own: a container starts in seconds rather than minutes, and a single host can run dozens of them.[6] The container **image** is the immutable, versioned artifact that a CI pipeline (§14.2.2) builds exactly once and then deploys everywhere — the same byte-for-byte thing on a laptop, in the test environment, and in production, which retires "works on my machine" as a category of excuse.

One host is rarely enough, so containers are run on a **cluster**: a set of network-connected machines managed as a single pool of compute, storage, and memory. And the de-facto standard software for managing that pool is **Kubernetes** (84% of organizations in CNCF's 2023 survey were using or evaluating it), often described as the *operating system for the cloud*: just as an OS schedules processes onto CPU cores, Kubernetes schedules containers onto the machines of a cluster.[7] Around that core job it handles **ingress** (routing incoming traffic to the right containers), scaling the number of running copies up and down with demand, restarting containers that crash or fail health checks, and attaching storage to containers that need it — the operational chores of §14.1.2's horizontal scaling, automated.

One caveat: Kubernetes earns its considerable complexity at fleet scale. For a small team — certainly for a class project — a single container on one host, or a platform-as-a-service that runs your container for you, delivers most of the benefit at a small fraction of the operational cost. Learn what Kubernetes is for; reach for it when you have the problem it solves.

14.1.6 Distributed Trade-Offs: The CAP Theorem

Horizontal scaling (§14.1.2) eventually reaches the data layer, and the moment state is *replicated* — living on more than one machine — a hard theoretical limit applies. Eric Brewer's **CAP theorem** concerns three properties of a distributed data system: **consistency** (every read sees the most recent write), **availability** (every request receives a response), and **partition tolerance** (the system keeps operating when the network splits and some machines cannot reach others).[8] Its real content is narrower than the popular "pick two of three": in the presence of a partition, a replicated system cannot guarantee both strong **consistency** and **availability**, so for that data it must choose which of the two to weaken until the network heals.[9]

Since partitions are a fact of real networks — switches fail, cables get cut, datacenters lose connectivity — partition tolerance is not really optional, and the practical content of the theorem is a forced choice *during* a partition: consistency or availability. A replica that cannot reach the others can either refuse to answer (staying consistent but unavailable) or answer from possibly stale data (staying available but inconsistent). Which to sacrifice is a *requirements* question, not a technical one. A bank chooses consistency: better to refuse service for a minute than to show two customers different balances for the same account. A social feed chooses availability: a like count that is a few seconds stale harms no one, but an unreachable feed is a product failure.

System	Under partition, it keeps…	What users see
Bank ledger	Consistency	Brief "service unavailable" — never a wrong balance
Social feed	Availability	The feed always loads; counts may lag and converge later

The theorem is why architecture and delivery keep meeting in this chapter: the shared-data pattern (§7.2.1) put one authoritative store at the center of a system, and scaling that store horizontally (§14.1.2) replicates it — at which point CAP stops being theory and becomes a decision your team must make on purpose, per feature, with the requirements in hand.

14.2 Continuous Integration Pipelines

Chapter 2 introduced **continuous integration (CI)** as an XP engineering practice: everyone merges into a shared mainline many times a day, and an automated build-and-test run verifies each merge (§2.3.4). That definition told you *what* CI is and *why* it exists — to surface integration mismatches one small change at a time instead of all at once. This section goes deeper: what the automation actually consists of, what branching discipline it demands, and what social contract makes it work.

14.2.1 Trunk-Based Development

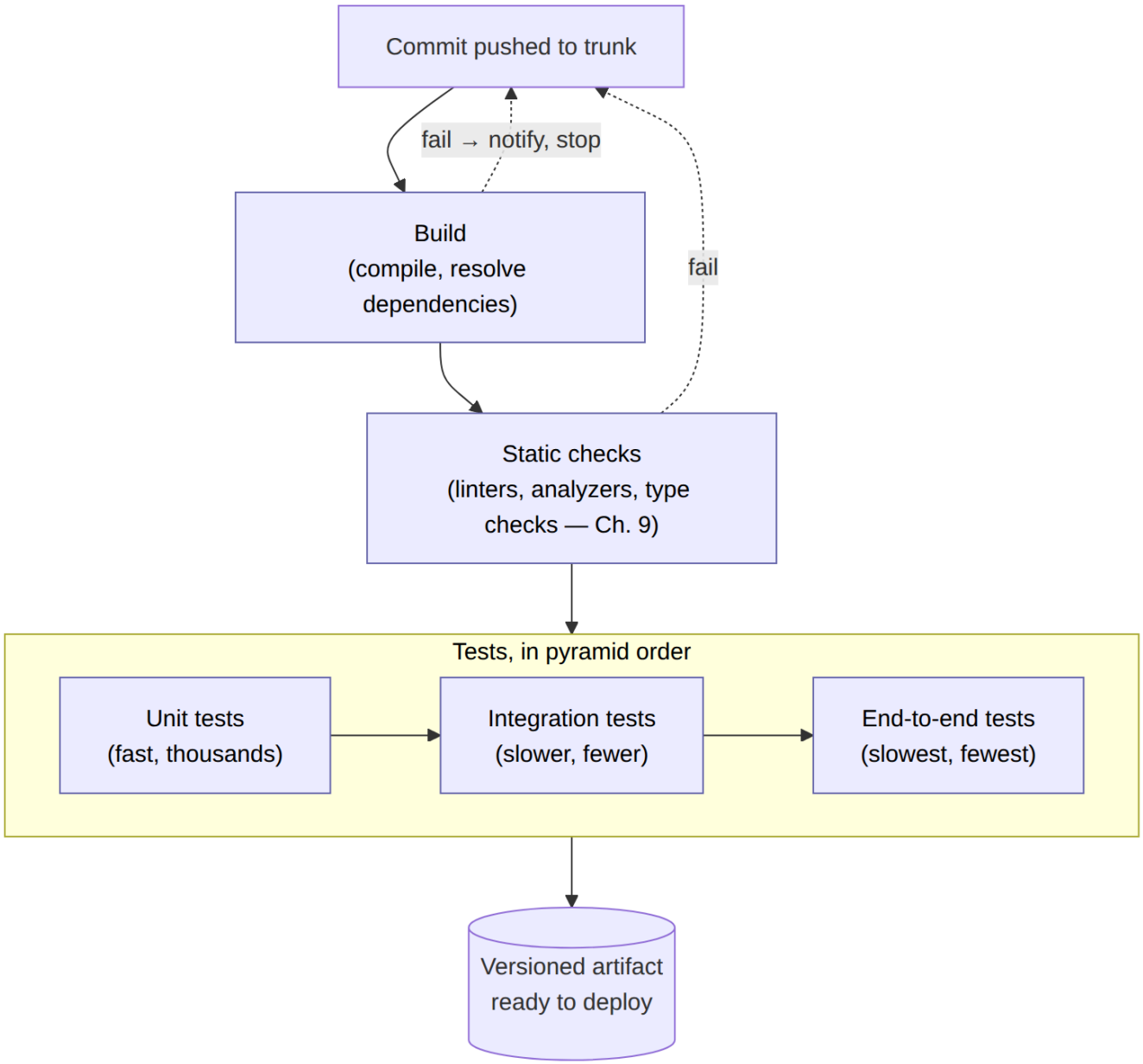
CI is first a *branching* policy, and only second a server. In **trunk-based development**, the whole team commits to a single shared mainline (the *trunk*), directly or through short-lived branches that live hours to a couple of days

before merging. The alternative — **long-lived feature branches**, where each developer or feature camps on its own branch for weeks — feels safer because nobody's half-done work disturbs anyone else. But the safety is an illusion with interest due at merge time: the longer two branches evolve apart, the more their eventual merge resembles the big-bang integration of §2.4.1, with conflicting assumptions surfacing all at once, weeks after they were made.

Integration becomes more difficult as branches *diverge*, and divergence increases with time. Trunk-based development keeps divergence permanently small by construction. Merging several times a day means each merge carries at most a few hours of divergence — small enough that conflicts are rare, and trivial when they occur. The obvious objection — "how do I commit work that isn't finished?" — has a standard answer you will meet in §14.3.3: hide unfinished work behind a **feature flag** so it can be *integrated* without being *released*. Integration and release become independent decisions, which is one of the most useful separations in this whole chapter.

14.2.2 The Stages of a Pipeline

A **CI pipeline** is the automated gauntlet every commit runs before it is declared good. The pipeline is where earlier chapters' verification techniques stop being activities a diligent person might perform and become *gates no change can skip*. A typical pipeline runs stages in increasing order of cost, failing fast on the cheap ones:



Take the stages in order. The **build** proves the change even compiles and its dependencies resolve — the cheapest possible check, so it runs first. **Static checks** run the automated analysis of Chapter 9 (§9.4) — linters, type checkers, style and bug-pattern analyzers — catching whole classes of defects without executing a line. Then the **tests by level** from Chapter 10 run in pyramid order: unit tests first because they are fast and localize failures precisely, then integration, then a thin layer of end-to-end tests. If everything passes, the pipeline produces a **versioned artifact** — a container image, a package, a binary — that is stored and never rebuilt. This last rule matters more than it looks: the artifact you tested is *byte-for-byte* the artifact you will deploy. Rebuilding "the same code" later invites the possibility that a changed compiler, dependency, or build machine produces something subtly different from what passed the tests. Mature pipelines add one more gate on the far side of deployment itself: a **smoke-test stage** — a fast is-it-alive check (does the service start, answer a trivial request,

reach its database?) run against the newly deployed version, gating the rollout before real traffic widens onto it (the testing levels these stages draw on are Chapter 10's).

14.2.3 Broken-Build Discipline

A pipeline is only as good as the team's response when it turns red. The working culture of CI rests on a small social contract. First, **a red mainline is everyone's emergency**: when the trunk build breaks, fixing it (or reverting the breaking commit) takes priority over new work, because every hour the build stays red, every other developer is either blocked or building on sand. Second, **never commit onto a broken build** — you would be stacking unverified change on unverified change, exactly the divergence CI exists to prevent. Third, **do not leave a broken build overnight**; revert if you cannot fix quickly. Reverting is not an insult. It is the cheap, always-available path back to a known-good state, and teams that treat it as routine stay green far more than teams that treat it as defeat.

A broken build interrupts the entire team's work and should be treated as an urgent operational problem. While it is green, every developer has a continuous, machine-checked guarantee that the shared codebase works and can build on top of it; while it is red, nobody knows whether the system works, and restoring it comes before other work.

> **Pitfall.** *The flaky test.* A test that fails intermittently without a code change is more corrosive than a test that always fails, because it teaches the team to ignore red. Once "just re-run it, that one's flaky" enters the vocabulary, the build has become a slot machine, and real failures start slipping through on the same shrug. Quarantine flaky tests immediately, fix or delete them promptly, and treat their existence as a defect in the suite. §10.2.4 catalogs the usual causes — races, shared state, order dependence, unstable externals — and their fixes.

14.2.4 Keeping Pipelines Fast

Pipeline speed is a hard constraint, not a convenience. Developers are supposed to merge several times a day and to *wait for green* before moving on. If the pipeline takes an hour, they will not wait — they will batch up bigger changes to amortize the wait, which re-creates the large, risky merges CI exists to eliminate. A useful target is roughly ten minutes from push to verdict for the merge-blocking stages.[10]

Achieving that is the testing pyramid of §10.2.4 applied as an engineering budget: push checks *down* the pyramid, where they are fast, and keep the slow end-to-end layer thin. Beyond that, run independent stages in parallel, cache dependencies and build outputs so unchanged parts are not rebuilt, and split the pipeline into a fast merge-blocking core plus deeper suites (full end-to-end runs, performance tests, long fuzzing) that run continuously against the trunk without holding up merges.

Order matters inside the merge-blocking core, too. The pipeline's verdict arrives when the first test *fails*, not when the last one passes — so run the tests most likely to fail first: the ones nearest the changed code, and the ones that have failed most recently. At the largest scale, teams select per change the subset of tests worth running at all — a budgeted-selection problem (the knapsack problem in disguise) that §10.1.6 examines, and that industrial CI systems implement as *predictive test selection*. Treat a slow pipeline as a process defect that changes how your team behaves, not as a tooling annoyance to live with.

14.3 Continuous Deployment

14.3.1 Delivery versus Deployment

Two similar terms name genuinely different commitments. **Continuous delivery** means every change that passes the pipeline yields an artifact that is *proven deployable* — the release decision is a business choice, but it is always available, at the push of a button, with no additional engineering work. **Continuous deployment** goes one step further: every change that passes the pipeline *is deployed to production automatically*, with no human in the loop. Delivery makes release *possible* at any moment; deployment makes it *actual* at every moment.

Continuous deployment sounds reckless until you notice what it forbids. If every green commit goes to production, then there is no such thing as a "safe to merge but not ready to ship" change without a flag, no manual pre-release checklist to lean on, and no batching of changes into a big release whose failures cannot be attributed. Every safety property must be automated, because automation is all there is. Teams that adopt it report a paradoxical result that §14.7 will make precise: deploying *more often* makes each deployment *less* risky, because each one is smaller, better attributed, and easier to undo.

14.3.2 Deployment Strategies

However often you deploy, *how* you swap new code into a live system determines the blast radius when something is wrong. Two infrastructure-level strategies dominate practice; a third approach moves the switch into code, and it gets its own section (§14.3.3).

Blue-green deployment runs two identical production environments. At any moment one (say, *blue*) serves all traffic while the other (*green*) sits idle. To release, you deploy the new version to the idle environment, verify it there against production conditions, then switch the router so all traffic flows to it. The old environment stays warm, so if the new version misbehaves, recovery is one router change back. You pay for this in doubled infrastructure and in the care demanded by anything stateful: a database schema shared by both environments must remain compatible with both versions during the switch.

Canary deployment releases the new version to a small slice of traffic first — one server, one percent of users, one region — and watches error rates, latency, and business metrics before widening. The name comes from the caged canary that warned miners of gas: the small exposed population absorbs the harm and sounds the alarm

while the damage is still bounded. Large operators generalize canaries into **staged rings**: the change rolls to ring after ring — internal users, then a small public slice, then broader populations — with automated health checks gating each promotion and halting the rollout on regression. The essential idea is *progressive exposure*: no change reaches everyone until it has demonstrably survived contact with someone.

14.3.3 Feature Flags: Decoupling Deploy from Release

Blue-green and canary deployments control exposure with *infrastructure* — routers, server pools, traffic slices. A **feature flag** (or *feature toggle*) moves the same control into *code*: a runtime conditional that turns a code path on or off without redeploying.[11] Deploying and releasing become fully independent acts. A feature's code can sit in production for weeks, dark and disabled, while the team keeps merging; when the moment comes, "releasing" it is a configuration change that takes effect in seconds, and un-releasing it is the same change in reverse. This is the endpoint of §14.1.3's argument: release is no longer even a deployment decision — it is a bit you flip.

Flags are also the standard answer to §14.2.1's objection ("how do I commit work that isn't finished?"): unfinished work merges to trunk dark and disabled, so integration stays continuous while release waits. In practice flags come in a few kinds with very different lifespans, and confusing them is where trouble starts:

- **Release flags** hide work in progress until it is ready. They should live *days to weeks* — a release flag whose feature has fully rolled out is a deletion you owe.
- **Operational flags** — including the **kill switch** — guard risky paths so an operator can disable a misbehaving feature instantly, without a deploy. These are deliberately long-lived, few in number, and tested like the safety equipment they are.
- **Experiment flags** split traffic between variants so you can *measure* a change (the A/B tests your metrics chapter made honest — §12.5). They live exactly as long as the experiment.

Targeting is what makes flags more than an on/off switch: a flag can be on for one user, one tenant, or three percent of traffic — a **percentage rollout**, which is canary deployment's progressive-exposure idea (§14.3.2) implemented in code, with no second environment to pay for.

In the clinic scheduler, both are one conditional — only the predicate changes:

```python
from zlib import crc32

def scheduler_page(user_id, flags):
  if flags["new_scheduler"]:                        # release flag: one bit, everyone
    return render_new(user_id)
  return render_old(user_id)

def scheduler_page_rollout(user_id, flags):          # same conditional, new predicate
  if crc32(user_id.encode()) % 100 < flags["new_scheduler_pct"]:  # stable bucket 0..99
    return render_new(user_id)
  return render_old(user_id)
```

How does a flip actually reach production? In real systems the flag's *state* lives outside the code, in a flag-management service — LaunchDarkly and Unleash are common platforms, and open-source flag libraries exist for every major language — while the application's SDK keeps a local, in-memory copy of every flag rule. Evaluation happens locally in microseconds, so the hot path never waits on a network call; changes stream to the SDKs within seconds of someone flipping the toggle.[12] And the failure mode is designed in advance: if the flag service becomes unreachable, SDKs keep serving the last configuration they saw — the flag system must never be the outage.[13]

The practice is older than the tooling. Flickr described running trunk-only development with flags and multiple deploys a day back in 2009, in the post that popularized the term *feature flipper*.[14] Facebook's quasi-continuous release keeps many changes "behind our Gatekeeper system," rolling out code and features independently — "if we do find a problem, we can simply switch the gatekeeper off rather than revert back to a previous version or fix forward."[15] That sentence is the next section's economics in miniature: the cheapest rollback is the one that is a config flip. And flag management has graduated from house hack to shared infrastructure — **OpenFeature**, a vendor-neutral flag API, is now a CNCF incubating project.[16]

None of this is free. Every long-lived flag doubles the configuration space your tests must consider — Chapter 10's combinatorial lesson (§10.6) applies directly, so test both states of any flag that will live past a sprint, and at least pairwise across flags that interact. And flags demand hygiene *because* they are so easy to add: every flag needs an owner, an intended lifespan, and a removal date, and a retired flag's code — both the dead branch and the conditional — must be deleted promptly. Stale flags are technical debt (§15.5) of an unusually dangerous kind: dormant behavior sitting in production, waiting for someone to trip it. The first case study below turned that danger from hypothetical to historical.

> **Pitfall.** Never repurpose an existing flag to control new behavior. The old name still points at whatever code the flag used to trigger, and that code is often dormant rather than deleted — Knight Capital's repurposed Power Peg flag (§14.3.5) is the canonical demonstration of what happens when it runs again. Retire the old flag, delete its dead code, and mint a new one; flag names are cheap, and the alternative was not.

14.3.4 Rollback versus Roll-Forward

When a deployment goes wrong, you have two exits. **Rollback** returns production to the previous version; **roll-forward** ships a new fix on top of the broken state. Rollback is usually faster and requires no new (unverified) code, so mature teams treat it as the default reflex. But a rollback path is a *mechanism*, and Chapter 10's lesson applies to mechanisms too: an untested rollback is a rumor, not a capability. Version skew can make the old code unable to read data the new code wrote; a config change may have accompanied the code; the "previous artifact" may no longer exist. Teams that take this seriously rehearse rollback routinely — some by making it the *normal* end of every canary that fails a health check, so the path is exercised weekly rather than discovered during a crisis. And some changes cannot be rolled back at all (an irreversible data migration, a security fix you must not un-ship), which is why roll-forward speed — how fast your pipeline can carry a one-line fix to production — is itself a safety property.

14.3.5 When Deployment Goes Wrong: Two Case Studies

The two case studies below are, respectively, the strongest argument on record *for* deployment automation and the strongest argument that automation *alone* is not safety. They are worth studying closely, and honestly, from the primary sources.

Case study. *Knight Capital, August 1, 2012.* Knight Capital was one of the largest equity market makers in the United States. When the New York Stock Exchange launched its Retail Liquidity Program, Knight updated SMARS — its automated order router — to participate. What follows is drawn from the findings in the SEC's later enforcement order (Release No. 34-70694, October 2013).[17]

The new code reused a **feature flag** that had previously activated "Power Peg," defunct order-routing functionality unused since 2003, whose dead code had been left in SMARS. Worse, the safety counter that once told Power Peg to stop when orders were filled had been inadvertently disabled in 2005, when the counter was moved elsewhere in the code and Power Peg was never retested. In the week before launch, a technician manually copied the new code onto SMARS's eight production servers — and missed one. No second person reviewed the deployment; there was no automated, repeatable deployment process to make the eight servers provably identical.

On the morning of August 1, orders routed to the un-updated eighth server hit the repurposed flag and woke the old Power Peg code, which began generating child orders continuously, never tracking fills, never stopping. Ninety-seven automated email messages referencing the Power Peg error had gone out *before the market opened*; no one acted on them — they were not designed as alerts, and they went to a group of personnel rather than to an owner with a duty to respond. In the roughly forty-five minutes that followed, Knight executed about four million trades across 154 stocks — on the order of 397 million shares — and, during diagnosis, engineers made the situation worse: suspecting the new code, they *rolled it back* on the seven correct servers, which put the repurposed flag's old behavior in force on all eight. There was no procedure for halting the system's aberrant activity and no documented incident-response plan. The loss exceeded $460 million. Knight survived only through an emergency investment[18] and was merged away within a year; the SEC fined it $12 million for violating market-access risk-control and related rules.

A note on honesty: the SEC order is an enforcement action about risk controls, and the deployment details above are findings within it — the record shows a chain of process failures, not the folklore version in which "one line of config" destroyed a company. The lessons are about the chain. Deployment must be automated, repeatable, and *verified* — an all-servers-identical check would have caught the eighth server in seconds. Never repurpose an old flag; delete dead code rather than leaving it armed. Alerts without owners are noise. Rollback is only a safety mechanism if it has been tested as one — here it was the step that completed the disaster. And a kill switch is required, not optional.

Case study. *CrowdStrike, July 19, 2024.* CrowdStrike's Falcon sensor is endpoint security software that runs inside the Windows kernel — the most privileged, least forgiving place code can live. To respond to new threats quickly, CrowdStrike ships "Rapid Response Content": threat-detection configuration delivered to all customers through a fully automated global push. The account below follows CrowdStrike's own external root-cause analysis, published in August 2024.[19]

In February 2024, sensor version 7.11 added a new content type defined with twenty-one input fields. The code that supplied those inputs provided twenty. The mismatch stayed latent for months because both the tests and all earlier content of that type used a wildcard match for the twenty-first field, so the missing input was never read. On July 19, a new content instance — Channel File 291 — used a non-wildcard twenty-first field for the first time. Reading the field that was not there caused an out-of-bounds memory read inside the kernel, crashing Windows. Because the sensor loads at boot, the machine crashed again on restart: a boot loop. The automated Content Validator that should have rejected the bad content had a bug of its own and passed it.

The push was global and simultaneous. Within hours, roughly 8.5 million Windows machines (Microsoft's estimate) were down: airlines (Delta alone canceled on the order of 7,000 flights), hospitals, banks, broadcasters, emergency services.[20] [21] Damage estimates ran into the billions — direct losses for the Fortune 500 alone were estimated at $5.4 billion, only a fraction of it insured.[22] Recovery was brutal precisely because the machines could not boot: in many cases a human had to start each machine in safe mode and delete the file by hand.[23] CrowdStrike's committed remediations read like this chapter's outline: staged canary rings for content, customer control over update cadence, a hardened validator, bounds checking in the interpreter, and more diverse testing.

The lessons generalize far beyond security software. **Config and content are code**: anything that changes the behavior of a running system deserves the same testing, staging, and progressive rollout as a code change, no matter how routine its format. A fully automated pipeline without progressive exposure is not safety — it is a machine for shipping a defect to every user on Earth at once. Validators are code too, with false negatives of their own (§9.4.2); a gate you never test is a gate you cannot trust. Design for a bounded blast radius *before* you need one. And recovery paths must be designed for the worst case — a fix pushed over the network is useless to a machine that cannot boot to receive it.

Read as a pair, the two cases bracket this chapter's argument. Knight shows what manual deployment costs: without automation, you cannot even guarantee that eight servers run the same code. CrowdStrike shows what automation without staging costs: with a perfect distribution machine and no progressive rollout, one latent defect reached the whole world before anyone could react. Twelve years apart, the timelines rhyme — about forty-five minutes for Knight's loss, about eighty minutes from CrowdStrike's push to its reversion.[24] Automation sets the *speed* of your outcomes; only progressive exposure and tested recovery decide their *sign*.

14.4 Packaging and Running a Service: Docker and Compose

Section 14.1.5 named the container as the unit of cloud deployment, and §14.3 covered *when* and *how often* to release. This section closes the gap between them: the concrete mechanics of turning a program that runs on your laptop into a service that runs on a server. You will build a container image, run an application together with the database and cache it depends on, and keep the whole stack reproducible. The tools are Docker and Docker Compose; the ideas they embody — immutable images, declared dependencies, externalized configuration, persistent volumes — outlast any particular tool.

14.4.1 From Dockerfile to Image

A container **image** (§14.1.5) is built from a **Dockerfile**: a text file of ordered instructions describing, step by step, how to assemble the environment your application needs.[25] Each instruction adds a **layer**, a cached filesystem diff, so a rebuild that changes only your source reuses the earlier layers that installed the runtime and dependencies and finishes in seconds. Here is a Dockerfile for a small Python web service:

```
# Start from a minimal, versioned base image. Pin the tag; never use "latest".
FROM python:3.12-slim

# Do the work inside a dedicated directory in the image.
WORKDIR /app

# Copy dependency manifests first, so "pip install" is cached until they change.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy the application source last, because it changes most often.
COPY . .

# Document the listening port and the command that starts the app.
EXPOSE 8000
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "app:app"]
```

Two ordering choices there are deliberate. Copying `requirements.txt` and installing dependencies *before* copying the source means editing a source file does not invalidate the dependency layer, so most rebuilds skip it entirely. Pinning the base image to `python:3.12-slim` rather than `python:latest` makes the build reproducible, because “latest” is a moving target that can change the runtime under you between two builds. You build the image once and run it as many times as you like:

```
docker build -t clinic-app:1.4.3 .        # build, tag with a version
docker run -p 8000:8000 clinic-app:1.4.3  # run, mapping host port to container port
```

> **Principle.** Build once, run anywhere. The image your CI pipeline (§14.2) builds is the exact artifact that runs in test and in production, byte for byte. A bug seen in production can be reproduced on your laptop by running the same tagged image, and that reproducibility is what makes the container's overhead worth paying.

A `.dockerignore` file keeps the build small and secrets out of the image: list `.git`, `node_modules`, `.env`, and local build output so they are never copied into a layer. For a compiled language, a **multi-stage build** compiles in a fat builder image and copies only the finished binary into a tiny runtime image, so the shipped image carries no compiler and no source.

14.4.2 Composing a Stack with Docker Compose

A real service is rarely one container. A typical web application is at least three: the app itself, a database that holds its state, and often a cache. Starting these by hand, creating a network and passing connection strings between them, is tedious and easy to get wrong. **Docker Compose** replaces that with one declarative file that describes the whole stack and brings it up with a single command.[26]

```yaml
# docker-compose.yml — an app, a database, and a cache as one stack.
services:
  app:
    build: .                      # build from the Dockerfile in this directory
    ports:
      - "8000:8000"               # publish the app to the host
    environment:
      DATABASE_URL: postgres://clinic:${DB_PASSWORD}@db:5432/clinic
      REDIS_URL: redis://cache:6379
    depends_on:
      db: { condition: service_healthy }
      cache: { condition: service_started }

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: clinic
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: clinic
    volumes:
      - db-data:/var/lib/postgresql/data   # persist data across restarts
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U clinic"]
      interval: 5s
      retries: 5

  cache:
    image: redis:7

volumes:
  db-data:                        # a named volume, managed by Docker
```

`docker compose up` reads this file, creates a private network so the containers can reach one another by name, and starts them. Look at the connection strings: the app reaches the database at the host `db` and the cache at `cache`. Compose resolves those service names to the right containers on the shared network, so the app never needs to know their IP addresses. The `depends_on` health condition makes the app wait until Postgres is actually ready to accept connections, not merely started, which removes a whole class of start-order race conditions.

14.4.3 Stateful Services: Postgres and Redis

Two of those services deserve a closer look, because they hold **state**, and state is where deployment gets hard. A container is **ephemeral**: stop it and everything written inside its writable layer is gone. That is exactly what you want for the stateless app container, which you replace wholesale on every deploy, and exactly what you must prevent for a database. The fix is a **volume**: storage that lives outside any container's lifecycle and is mounted into one. In the file above, `db-data:/var/lib/postgresql/data` mounts a Docker-managed volume at the directory where Postgres keeps its files, so the data survives `docker compose down` and the next deploy.

The two data services play different roles, and the difference is a direct instance of the CAP trade-off (§14.1.6) and the shared-data pattern (§7.2.1):

- **PostgreSQL** is the **system of record**: a relational database offering transactions, constraints, and durability. When correctness matters — a patient record, a payment, an appointment — the data goes here, and Postgres guarantees a committed write is not lost. It is the authoritative store §7.2.1 told you to design around.
- **Redis** is an **in-memory cache and data-structure store**: it keeps data in RAM, so reads and writes are very fast, and it holds work that tolerates loss — session tokens, rate-limit counters, computed results you can recompute, a queue of background jobs. Putting a cache in front of Postgres is the usual way to make a read-heavy service fast, and it is safe precisely because the cache is disposable: if Redis loses its data, the system reads from Postgres and refills it.

> **Pitfall.** Treating the cache as a source of truth. The moment your system *cannot* rebuild Redis's contents from the database, you have quietly made an in-memory store your system of record, and Redis by default has promised you no such durability. Keep the authoritative copy in the durable store and let the cache stay something you can throw away and rebuild at any time.

Both Postgres and Redis ship as **official images** on the public registry, maintained and security-patched by the wider community, which is why the Compose file pulls `postgres:16` and `redis:7` instead of installing and configuring database servers by hand.[27] Pin the major version, as with any dependency (§14.6.2), so an unattended pull cannot upgrade your database engine underneath you.

14.4.4 Configuration and Secrets

The Compose file contains neither a password embedded in the image nor a connection string hard-coded in the source. Those values arrive as **environment variables** (`${DB_PASSWORD}`), read from an untracked `.env` file or injected by the deployment platform. This is the externalized-configuration rule of the twelve-factor model: one image runs in every environment, and what differs between environments — credentials, hostnames, feature toggles — lives in configuration outside the image.[28]

The rule earns its keep twice. It keeps a single image promotable from test to production without a rebuild, and it keeps secrets out of the image and out of version control, where §14.6.3 shows they cause real breaches once leaked. A committed `.env` file is one of the most common ways a live credential ends up in a public repository.

> **Principle.** One image, many environments. If moving a build from staging to production requires rebuilding it, the build was never really the artifact; its configuration was. Externalize what changes between environments so the thing you tested is exactly the thing you ship.

14.5 Reaching Users: DNS, TLS, and the Edge

You now have a stack running on a server. That server has an IP address like `203.0.113.143`, but no user will type that, and no browser will trust it without a padlock. Two pieces of internet infrastructure bridge the distance between "a container is running somewhere" and "anyone can reach it safely at `https://clinic.example.com`": the **Domain Name System**, which turns a name into an address, and **TLS**, which encrypts and authenticates the connection. This section explains both, adds the layer most real deployments put in front of everything else, and closes with a complete deployment from start to finish.

14.5.1 How a Name Becomes an Address

The **Domain Name System (DNS)** is the internet's directory: a globally distributed database that maps human-readable names like `clinic.example.com` to the numeric IP addresses machines actually route to.[29] A client's first connection to an uncached hostname generally requires name resolution — commonly through DNS — and understanding the pieces clears up what is otherwise the most common source of "it works locally but not in production."

You obtain a domain from a **registrar**, the company you buy `example.com` from. The registrar records which **nameservers** are authoritative for your domain — the servers that hold the real answers. When someone visits your site, their computer asks a **resolver** (usually run by their ISP, or a public one like `1.1.1.1`), which walks the hierarchy: it asks a **root** server who handles `.com`, asks that **TLD** server who is authoritative for `example.com`, and finally asks your authoritative nameserver for the specific record. Each answer is cached along the way for a period set by the record's **TTL** (time to live), which is why a DNS change is not instant: older cached answers linger until their TTL expires. The authoritative nameserver holds **records**, each a typed mapping. The handful you actually set for a deployment:

Record	Maps a name to	Example use
A	an IPv4 address	`clinic.example.com` → `203.0.113.143`
AAAA	an IPv6 address	`clinic.example.com` → `2001:db8::143`
CNAME	another name (an alias)	`www.example.com` → `clinic.example.com`
MX	a mail server	routes email addressed to the domain
TXT	arbitrary text	domain verification, email (SPF/DKIM) policy

To point a domain at your running server, you create an **A record** from your name to the server's IP address. That is the whole mechanism: a name, a type, a value, and a TTL.

> **Pitfall.** Blaming the code for a DNS TTL. You repoint an A record to a new server, test from your own machine, and it works — yet users keep hitting the old server for minutes or hours, because their resolvers cached the previous answer for its TTL. Before a planned migration, lower the record's TTL well in advance so the cutover is quick, then raise it again afterward to cut lookup load.

14.5.2 HTTPS and TLS Certificates

A DNS lookup gets a browser to your server; **TLS** (Transport Layer Security, the protocol behind the `s` in `https`) makes the connection between them private and trustworthy. It does two things at once: it **encrypts** the traffic so no one on the network path can read or alter it, and it **authenticates** the server so the browser knows it is really talking to `clinic.example.com` and not an impostor. Modern browsers now treat plain HTTP as a defect, labeling it "Not Secure," and the large majority of page loads happen over HTTPS.[30] For a real deployment, TLS is not a nicety to add later.

Authentication rests on a **certificate**: a file, issued by a **Certificate Authority (CA)**, that binds your domain name to a cryptographic key and carries the signature of an authority the browser already trusts. When a browser connects, the server presents its certificate; the browser checks the CA's signature and that the name matches, then negotiates an encrypted session. A certificate for a name you do not control cannot normally be obtained from a trusted CA, which is what stops an attacker from impersonating your site.

Certificates were once a paid, manual chore. **Let's Encrypt** changed that: a nonprofit CA that issues certificates for free and fully automatically over the **ACME** protocol, which lets a program prove it controls a domain and receive a certificate with no human in the loop.[31] Let's Encrypt now secures over 700 million sites and issues on the order of ten million certificates a day.[32] Its certificates are deliberately **short-lived** and **auto-renewed**: a program renews them well before they expire, so a forgotten manual renewal can no longer take a site down. In practice you rarely touch a certificate directly — a **reverse proxy** such as nginx or Caddy sits in front of your app

container, obtains and renews the certificate, terminates TLS, and forwards plain HTTP to the app over the private network.

14.5.3 The Edge: CDNs and Cloudflare

That reverse-proxy idea generalizes into one of the most important pieces of modern deployment: a layer at the **edge**, between your users and your server, that handles concerns you would rather not build yourself. The dominant provider is **Cloudflare**, and the mechanism is a small DNS change. Instead of pointing your domain straight at your server, you point it at Cloudflare, which **proxies** each request to your origin server behind the scenes.[33] Your server's real address is hidden, and every request flows through Cloudflare first. Sitting in that position lets the edge do several jobs:

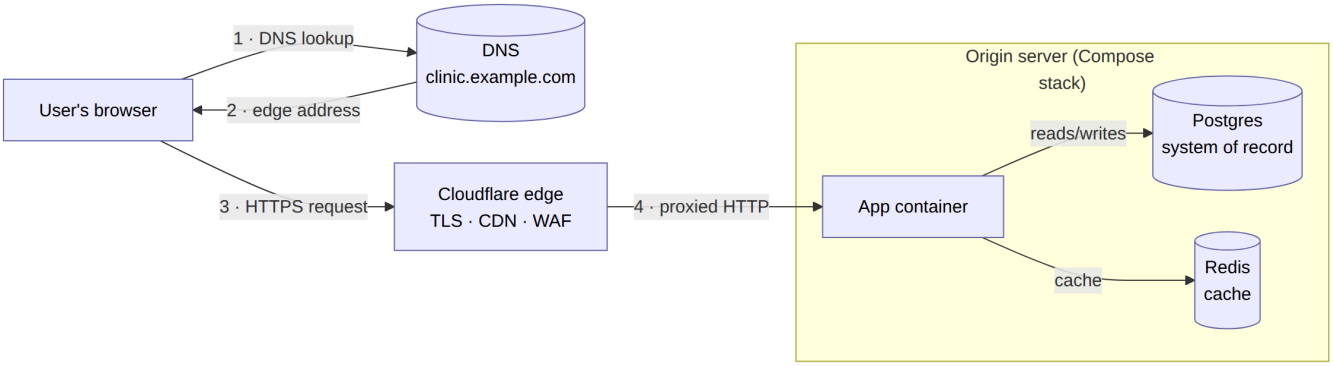
- **Content delivery (CDN).** Cloudflare runs servers in hundreds of cities and caches your static content close to users, so a visitor in Sydney is served from Sydney rather than waiting on a round trip to your origin. This cuts latency and offloads your server.
- **TLS termination.** The edge presents the HTTPS certificate and encrypts the user connection, so you get valid TLS without configuring it on the origin at all.
- **Security.** Because all traffic passes through it, the edge can absorb **distributed denial-of-service (DDoS)** attacks — floods of traffic meant to exhaust your server — and filter malicious requests with a **web application firewall (WAF)** before they reach you.

The scale is what makes this remarkable. Cloudflare sits in front of roughly **one in five of all websites**, so a large share of the web is proxied through this one network.[34] That scale is the source of both its value, because it sees enough traffic to recognize new attacks quickly, and a real concern: when a service this central has an outage, a visible slice of the web goes down with it. The concentration is worth weighing when you decide how much of your delivery to place behind a single provider.

> **Pitfall.** Assuming the edge makes your origin safe to ignore. Cloudflare hides and protects your origin only if the origin is not *also* reachable at its real IP address. If an attacker finds the origin IP and your server still accepts direct connections, they bypass the edge entirely. Configure the origin to accept traffic only from the edge, so the protection cannot be stepped around.

14.5.4 A Full Deployment, End to End

The pieces now assemble into one picture. Here is the whole path a request travels, and the sequence a team follows to put a first real service online:



Walking the deployment in order:

1. **Build the image.** Your CI pipeline (§14.2) builds the container image from the Dockerfile, tags it with the commit's version, and pushes it to an image registry.
2. **Run the stack.** On a host — a rented virtual machine, or a platform-as-a-service that runs containers for you — you bring up the Compose stack: the app image plus `postgres:16` and `redis:7`, with a volume for the database and configuration supplied through environment variables.
3. **Point the domain.** In DNS you create a record for `clinic.example.com`. Behind Cloudflare, it points at Cloudflare and Cloudflare proxies to your origin; otherwise it is an A record straight to the origin's IP.
4. **Get a certificate.** A reverse proxy on the origin obtains a Let's Encrypt certificate over ACME, or Cloudflare terminates TLS at the edge, so the site serves valid HTTPS.
5. **Release safely.** You roll the new version out with a strategy from §14.3 — behind a feature flag, or as a canary to a slice of traffic — and watch the DORA signals of §14.7: when deployment frequency is high and change-fail rate stays low, the machinery is sound.

None of these steps is exotic, and a practiced small team can stand up a minimal version of this stack in an afternoon — expect your first time through to take longer, with DNS, TLS, and container networking supplying the surprises. The chapter's argument is that doing it *repeatably* — an image built once by a pipeline, configuration externalized, the database on a durable volume, the release automated and progressively exposed, the whole path observed — is what separates a demo that happens to be online from a service a team can operate, and keep operating, as it changes.

> **Principle.** A deployment is a system, not an event. The domain, the certificate, the container, the database volume, and the release strategy are parts that must fit together and keep working without you watching. Automate each one so that shipping a change is boring, because boring, as this chapter has insisted throughout, is what you want production to be.

14.6 Continuous Security Pipelines

Chapter 9 taught static analysis as a practice; the pipeline is where it becomes policy. Modern teams extend the CI pipeline of §14.2 into a **continuous security pipeline** — a set of automated gates that check not just whether the code works, but whether it is safe to expose to an adversarial world. Three scanner families divide the work.

14.6.1 SAST, DAST, and SCA

Static application security testing (SAST) is the security-focused end of the static analysis you met in §9.4: tools that examine source code without running it, hunting injection flaws, unsafe deserialization, buffer misuse, and other vulnerable *patterns*. Everything Chapter 9 said about false positives and false negatives applies with interest — a noisy SAST gate that developers learn to rubber-stamp protects no one.

Dynamic application security testing (DAST) attacks the *running* application from outside, the way an adversary would: probing endpoints with malformed inputs, injection payloads, and authentication bypasses, knowing nothing about the source. SAST and DAST are complementary the way white-box and black-box testing were in Chapter 10: SAST sees code paths DAST may never reach; DAST sees emergent, deployed behavior — server configuration, header mistakes, the composition of services — that no source scan can.[35]

Software composition analysis (SCA) examines neither your code nor your running app but your *dependency manifest*: the inventory of third-party packages your build pulls in, checked against databases of known vulnerabilities. SCA exists because of an uncomfortable arithmetic: in a typical modern application, code you wrote is a thin layer atop orders of magnitude more code you imported. You ship your dependencies. Their vulnerabilities are your vulnerabilities, and no review of *your* code will find them.

14.6.2 Dependencies and the Supply Chain

Because dependencies drift out of date on their own — vulnerabilities are discovered in versions you already ship — SCA cannot be a one-time gate; it must run continuously. The practical pattern is the **automated update bot** (GitHub's Dependabot is the archetype): a service that watches vulnerability databases and your manifests, and when a dependency needs bumping, *opens a pull request* that updates it.[36] The elegance is in what happens next: your CI pipeline runs on that PR like any other, so the same suite that protects you from your own mistakes now proves the upgrade is safe to merge. The stronger your pipeline, the cheaper staying current becomes — one more return on the investment of §14.2.

The wider issue is **supply-chain risk**: your build is only as trustworthy as everything it downloads. Attackers have learned to poison the well — **typosquatting** packages whose names are one keystroke from a popular library, or compromising a legitimate package's maintainer account and publishing a malicious release. The 2020 SolarWinds attack planted malicious code inside a vendor's *build process*, so customers received a compromised product signed with authentic signatures;[37] the 2016 left-pad incident showed the fragility side, when the removal of an eleven-line package briefly broke builds across the industry.[38] [39] Defenses are accumulating — lockfiles

that pin exact versions, cryptographic signing and provenance attestation for artifacts (the SLSA framework),[40] and a **software bill of materials (SBOM)** enumerating everything inside a release[41] — but the first defense is the cultural one: treat adding a dependency as an engineering decision with a threat model, not a free lunch. Chapter 11 develops this into a full treatment of supply-chain security — the Log4Shell and xz-utils case studies, and a framework for continuously verifying the open-source components you depend on.

14.6.3 Secrets and Gate Placement

One more scanner belongs in every pipeline: **secrets scanning**, which searches commits for credentials — API keys, tokens, passwords, private keys — before they enter history. A secret pushed to a repository must be treated as compromised the moment it lands, because git history is effectively permanent and harvesting bots scan public commits continuously. Rotating a leaked credential is painful; a pre-commit or pre-receive scan that blocks the leak is nearly free.

Placement follows one principle: **run each gate at the earliest point it can give a correct answer**. Secrets scans and SAST need only source, so they run at commit time, inside the fast merge-blocking core. SCA needs the resolved dependency set, so it runs at build time — and again on a schedule, since the world's knowledge of your dependencies changes while your code does not. DAST needs a running system, so it runs against a staging deployment, after the artifact exists. The result is defense in depth through the pipeline itself: by the time an artifact reaches production, it has been examined as source, as a composition, and as a running target.

14.7 DORA Delivery Metrics

14.7.1 The Four Keys — Now Five

How would you know whether any of this is working? As Chapter 12 explained, metrics programs often choose convenient measurements that do not reflect the outcomes the team needs to improve. The delivery world has an unusually good answer, produced by the **DORA** research program (DevOps Research and Assessment) — a multi-year academic effort, surveying tens of thousands of professionals, published in the annual *State of DevOps* reports and the book *Accelerate* (Forsgren, Humble, and Kim).[42] [43] Its best-known finding is a set of outcome measures — the famous **four keys** — that jointly predict software-delivery performance:

1. **Deployment frequency** — how often your team deploys to production.
2. **Change lead time** — how long a commit takes to reach production.
3. **Change fail rate** — what fraction of deployments cause a failure in production (an incident, a rollback, a hotfix).
4. **Failed-deployment recovery time** — when a deployment does cause a failure, how long restoring service takes.[44]

The model has since been refined: the 2024 research added a fifth metric, **deployment rework rate** — the share of deployments that are *unplanned*, shipped in response to a production incident — and DORA's current guidance presents the five in two groups.[45] **Throughput** is change lead time, deployment frequency, and failed-deployment recovery time: how fast changes — including fixes — flow to production. **Instability** is change fail rate and deployment rework rate: how often deployments go wrong, or exist only to clean up after one that did. The regrouping is worth a moment's thought. Recovery *time* is a speed, so it now sits with throughput; and rework rate was added because a team could look stable while quietly spending its deployments patching production — reactive work the original four keys never surfaced.

Notice the shape either way: one group measures how fast value moves, the other how safely it moves. All five are *outcomes* of your whole delivery system, not activities within it — which is what makes them worth watching.

14.7.2 Why Paired Metrics Resist Gaming

Chapter 12 introduced Goodhart's Law (§12.1.2): when a measure becomes a target, people optimize the measure rather than the goal, and the chapter's advice was to pair each metric with a counter-metric that degrades when someone cheats. The DORA metrics are that advice, institutionalized. Try to game throughput — deploy half-baked changes constantly — and change fail rate rises to expose you. Try to game stability — deploy once a quarter after months of manual checking — and deployment frequency and lead time collapse. Each group is the other's counter-metric. And the fifth metric closed a cheat the original four left open: recover *fast* by shipping quick, unplanned patches, and recovery time looks elite while deployment rework rate exposes how much of your delivery is cleanup. A team can only improve all five *together* by actually getting better at delivery: smaller changes, stronger pipelines, faster recovery. There is no cheap move that improves the whole dashboard, which is the property §12.1.2 said to look for.

14.7.3 What the Research Found

Two findings from the DORA research deserve to reshape your intuitions. First, the spread between the best and the rest is not incremental — it is multiplicative. In the **2019 survey**, **elite** performers deployed on demand (many times per day) where **low** performers deployed between once a month and once every six months; elite lead times were under a day against months; elite recovery times under an hour against a week or more — differences of orders of magnitude on the throughput measures, with change fail rates lower as well.[42] The bands are re-measured each survey year and move — read them as a dated yardstick from a specific report, not a permanent threshold.

Second — and this is the finding that overturned decades of folklore — **speed and stability correlate positively**.[42] The traditional assumption was a trade-off: move fast *or* be careful. The data say the teams that deploy most often are *also* the teams that break production least and recover fastest. The mechanism should be familiar by now: high frequency forces small changes; small changes are easier to review (Chapter 9), test

(Chapter 10), and attribute; attribution makes recovery fast; and fast, safe recovery removes the fear that drives batching. Slow, careful, big-batch releases are the risky choice wearing caution's clothes.

14.7.4 Measuring Your Own Five

A student team can measure all five metrics with data it already has, and the exercise is worth doing because the numbers will be humbler than the elite benchmarks. Define "production" honestly — your deployed demo environment, or your instructor-facing release — then: **deployment frequency** is a count of deploy events per week, from your pipeline's history. **Lead time** is deploy timestamp minus commit timestamp, taken as a median over recent changes (`git log` and your CI dashboard give you both ends). **Change fail rate** requires a log discipline: record each deploy and whether it needed a revert or hotfix; failures divided by deploys. **Recovery time** is the gap from noticing a bad deploy to restored service, from the same log. **Rework rate** comes from the same log too: mark each deploy planned or unplanned (a revert or hotfix shipped because production broke is unplanned); unplanned divided by total. Review the five numbers at your retrospective, and resist the urge to set targets — use them, in GQM fashion (Chapter 12), to ask *why* lead time is three days and *which* stage of your pipeline the time hides in.

14.8 Conclusion

Delivery is the connective tissue of everything this book has taught. The CI pipeline of §14.2 is Chapters 9 and 10 made *mandatory*: reviews, static analysis, and tests by level, converted from practices a diligent team performs into gates no change can bypass. The DORA delivery metrics of §14.7 are Chapter 12 made *honest*: outcome metrics, paired against their own counter-metrics, measuring the whole system rather than rewarding activity. Continuous deployment of §14.3 is Chapter 2's short-cycle bet made *physical*: the same argument that favored small iterations over big-bang phases favors small deployments over big releases, with Knight Capital and CrowdStrike as the permanent record of what happens at either failed extreme — no automation, and automation without staging. The packaging and networking of §§14.4–14.5 are that same pipeline made *tangible*: an image built once, a database on a durable volume, a name, a certificate, and an edge in front, so the artifact a pipeline produces actually becomes a service a user can reach. What delivery cannot do is stop the clock: the system you now ship continuously will spend years being changed, and that long tail — legacy code, refactoring, technical debt — is Chapter 15's subject.

If the chapter compresses to one sentence, it is this: **make change small, make its path to users automatic and progressively exposed, and watch the outcomes.**

Sources

[1] Synergy Research Group, *Cloud Market Share Trends — Big Three Together Hold 63%* (2025). srgresearch.com.

[2] Amazon Web Services, *Shared Responsibility Model*. aws.amazon.com.

[3] David Heinemeier Hansson (37signals), *Why we're leaving the cloud* (2022). world.hey.com/dhh.

[4] David Heinemeier Hansson (37signals), *We have left the cloud* (2023). world.hey.com/dhh.

[5] Marcin Kolny (Prime Video Tech), *Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%* (2023). web.archive.org (original post now offline).

[6] Amazon Web Services, *What's the Difference Between Containers and Virtual Machines?* aws.amazon.com.

[7] Cloud Native Computing Foundation, *CNCF Annual Survey 2023* (2023). cncf.io.

[8] Eric Brewer, *Towards Robust Distributed Systems* (PODC keynote, 2000). people.eecs.berkeley.edu.

[9] Seth Gilbert and Nancy Lynch, *Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services* (ACM SIGACT News, 2002). doi.org.

[10] Martin Fowler, *Continuous Integration* (2006; revised 2024). martinfowler.com.

[11] Pete Hodgson, *Feature Toggles (aka Feature Flags)* (martinfowler.com, 2017). martinfowler.com.

[12] LaunchDarkly, *A Deeper Look at LaunchDarkly Architecture* (documentation). launchdarkly.com.

[13] Unleash, *11 best practices for building and scaling feature flag systems* (documentation). docs.getunleash.io.

[14] Ross Harmes (Flickr Engineering), *Flipping Out* (2009). code.flickr.net.

[15] Chuck Rossi (Facebook Engineering), *Rapid release at massive scale* (2017). engineering.fb.com.

[16] OpenFeature, a CNCF incubating project since 2023. openfeature.dev.

[17] U.S. Securities and Exchange Commission, *In the Matter of Knight Capital Americas LLC*, Exchange Act Release No. 34-70694 (2013). sec.gov.

[18] CNNMoney, *Knight Capital in $400 million rescue agreement* (2012). money.cnn.com.

[19] CrowdStrike, *External Technical Root Cause Analysis — Channel File 291* (2024). crowdstrike.com.

[20] David Weston (Microsoft), *Helping our customers through the CrowdStrike outage* (2024). blogs.microsoft.com.

[21] Delta Air Lines, *Form 8-K* (October 2024). sec.gov.

[22] Parametrix, *CrowdStrike to cost Fortune 500 $5.4 billion; insured loss range of $540 million to $1.08 billion* (2024). parametrixinsurance.com.

[23] CrowdStrike, *Falcon Content Update Remediation and Guidance Hub* (2024). crowdstrike.com.

[24] CrowdStrike, *Preliminary Post Incident Review — Falcon Content Update for Windows Hosts* (2024). crowdstrike.com.

[25] Docker, Inc., *Dockerfile reference* and *Get started*. docs.docker.com/reference/dockerfile, docs.docker.com/get-started.

[26] Docker, Inc., *Docker Compose* documentation. docs.docker.com/compose.

[27] Docker Official Images for *postgres* and *redis* (community-maintained, security-patched base images). hub.docker.com/_/postgres, hub.docker.com/_/redis.

[28] Adam Wiggins, *The Twelve-Factor App*, factor III: "Config." 12factor.net/config.

[29] P. Mockapetris, *Domain Names — Concepts and Facilities*, RFC 1034 (1987); readable overview: Cloudflare, "What is DNS?" rfc-editor.org/rfc/rfc1034, cloudflare.com/learning/dns/what-is-dns.

[30] Google, *HTTPS encryption on the web* (Transparency Report), which tracks the share of page loads served over HTTPS; and Chrome's move to mark plain HTTP "Not Secure." transparencyreport.google.com/https/overview.

[31] R. Barnes, J. Hoffman-Andrews, D. McCarney, and J. Kasten, *Automatic Certificate Management Environment (ACME)*, RFC 8555 (2019); Let's Encrypt, "How It Works." rfc-editor.org/rfc/rfc8555, letsencrypt.org/how-it-works.

[32] Let's Encrypt, *Let's Encrypt Stats* and "Ten Years of Let's Encrypt" (2025), reporting hundreds of millions of sites secured and roughly ten million certificates issued per day. letsencrypt.org/stats, letsencrypt.org/2025/12/09/10-years.

[33] Cloudflare, *How Cloudflare works* and *What is a reverse proxy?* developers.cloudflare.com/fundamentals/concepts/how-cloudflare-works, cloudflare.com/learning/cdn/glossary/reverse-proxy.

[34] W3Techs, *Usage statistics and market share of reverse proxy services for websites*, July 2026 — Cloudflare is used as a reverse proxy by 20.4% of all websites. (W3Techs measures website adoption, not request volume; the share of global request *traffic* is a different quantity and is not measured here.) w3techs.com/technologies/overview/proxy.

[35] OWASP Foundation, community references for the scanner families: Source Code Analysis Tools (SAST), Vulnerability Scanning Tools (DAST), and Component Analysis (SCA).

[36] GitHub, *Dependabot documentation*. docs.github.com.

[37] CISA, *Alert AA20-352A: Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations* (2020). cisa.gov.

[38] npm, *kik, left-pad, and npm* (2016). blog.npmjs.org.

[39] The Register, *How one developer just broke Node, Babel and thousands of projects in 11 lines of JavaScript* (2016). theregister.com.

[40] OpenSSF, *SLSA — Supply-chain Levels for Software Artifacts*. slsa.dev.

[41] CISA, *Software Bill of Materials (SBOM)*. cisa.gov/sbom.

[42] DORA, *Accelerate State of DevOps Report 2019* (2019). dora.dev.

[43] Nicole Forsgren, Jez Humble, and Gene Kim, *Accelerate: The Science of Lean Software and DevOps* (IT Revolution Press, 2018). itrevolution.com.

[44] DORA, *DORA's software delivery performance metrics*. dora.dev.

[45] DORA, *A history of DORA's software delivery metrics* — the transition from the original four keys to the current five-metric model; deployment rework rate ("the ratio of deployments that are unplanned but happen as a result of an incident in production") was introduced with the 2024 *State of DevOps* research. dora.dev.

- **Key takeaways** are summarized above in §14.8.
- Continue to the Exercises.
- Go deeper with the Open Resources for this chapter.

Chapter 14 — Exercises

Exercises are graded by depth: **[warm-up]** checks understanding, **[analysis]** asks you to reason. Several exercises require *actual work* — computing metrics from a log, designing a rollout, writing a real test — not just prose. Show the work, not only the answer.

Concepts

1. **[warm-up]** In one sentence each, distinguish *continuous integration*, *continuous delivery*, and *continuous deployment* (§14.2.1, §14.3.1). Then state which of the three a team practices if every green build produces a deployable artifact but a product manager clicks "release" once a week — and what single change would move them to the next level.

2. **[warm-up]** Explain why horizontal scaling (§14.1.2) depends on the statelessness convention from §7.5.4. Give one concrete example of per-client state that would break "any server can answer any request," and one standard place to move it.

3. **[warm-up]** A teammate says "we don't need rollback — we'll just fix forward, our pipeline is fast." Using §14.3.4, give one situation where roll-forward is genuinely the only option and one where an untested reliance on it would be dangerous.

4. **[warm-up]** For each scanner family in §14.6 — SAST, DAST, SCA, secrets scanning — name the *earliest* pipeline stage at which it can give a correct answer, and say why it cannot run earlier.

5. **[warm-up]** A teammate's `docker-compose.yml` writes the database password directly into the `environment:` block as a literal string, and the file is committed to the repository. Using §14.4.4, name two distinct things that go wrong with this, and rewrite the relevant lines so the password comes from an untracked source instead. Then, using §14.4.3, explain why the `db` (Postgres) service needs a `volumes:` entry but the `cache` (Redis) service can safely omit one.

Analysis

6. **[analysis]** *Compute the DORA metrics.* A student team's combined git and deploy log for two weeks is below. A deploy is "failed" if it required a revert or hotfix; recovery time is from deploy to restored service. Note that the revert and the hotfix are themselves deployments — unplanned ones, shipped because production broke.

Change	Commit time	Deployed	Outcome
C1	Mon 09:00	Mon 15:00	ok
C2	Mon 11:00	Tue 10:00	ok
C3	Tue 14:00	Thu 09:00	failed — reverted Thu 10:30
C4	Thu 11:00	Thu 16:00	ok
C5	Fri 10:00	Mon 10:00 (wk 2)	ok
C6	Mon 13:00 (wk 2)	Wed 09:00 (wk 2)	failed — hotfixed Wed 15:00
C7	Wed 10:00 (wk 2)	Wed 17:00 (wk 2)	ok
C8	Thu 09:00 (wk 2)	Fri 11:00 (wk 2)	ok

Compute (a) deployment frequency (deploys per week), (b) median change lead time, (c) change fail rate, (d) mean failed-deployment recovery time, and (e) deployment rework rate — counting the revert and the hotfix as deployments, what share of *all* deployments were unplanned (§14.7.1)? Then (f) using the 2019 elite bands quoted in §14.7.3, identify which metric is furthest from elite and propose the single pipeline change most likely to improve it.

7. **[analysis]** *Design a canary rollout.* Your team is shipping a rewritten session-handling module to a service with 200,000 daily users. Design a staged rollout plan (§14.3.2): define at least three rings with their traffic percentages, the health metrics that gate each promotion (name at least three, and give a numeric threshold for one), the soak time per ring, and the automatic action on regression. State explicitly what your plan's worst-case blast radius is, and compare it to a blue-green switch of 100% of traffic.

8. **[analysis]** *Knight versus CrowdStrike.* Using only the facts in §14.3.5, write a structured comparison of the two incidents: for each, identify (a) the latent defect and how long it lay dormant, (b) the deployment-process failure that activated or spread it, (c) the missing safeguard that would have bounded the damage, and (d) the time from trigger to full impact. Then answer the pairing question directly: why does Knight argue *for* deployment automation while CrowdStrike shows automation is not sufficient — and what one practice, common to both post-mortems, addresses each?

9. **[analysis]** The CrowdStrike case says "config and content are code." A teammate objects: "running our full test suite on every config change would be absurd — it's just data." Steelman the teammate's position, then rebut it: what *proportionate* pipeline (validation, testing, staged rollout) would you design for pure-content changes, and which properties of code changes must it preserve?

10. **[analysis]** Goodhart's Law (§12.1.2) says any single metric target gets gamed. For each of the four classic DORA keys taken *alone*, describe a way a cynical team could improve the number while making delivery worse — then show which *other* metric would expose each cheat (§14.7.2). Finally: DORA's 2024 addition

of **deployment rework rate** closed exactly one such gap — which of your cheats does it catch that the original four keys missed, and why did that cheat survive the original pairing?

11. **[analysis]** You are putting a small service online at `app.example.com`, running as a Compose stack (app + Postgres + Redis) on a single rented virtual machine. Using §14.5, write the deployment as an ordered checklist: (a) the DNS record(s) you create and what each points to; (b) how the service obtains a valid HTTPS certificate; (c) what placing the domain behind Cloudflare would add, and one risk it introduces; and (d) one thing that could still be broken for users after all of the above is correct, and how you would detect it — connecting your answer to the DORA signals of §14.7.

Chapter 14 — Open Resources

Free, open-licensed, or freely accessible materials that reinforce this chapter. Types: 📘 open text · 🎓 course · 📄 primary source / standard · 🎥 video · 🛠️ tool. Licenses vary and are noted where known. Each entry notes its license or access terms; when in doubt, check the linked page.

Primary sources for the case studies

- 📄 **SEC Release No. 34-70694 — In the Matter of Knight Capital Americas LLC** (October 2013) — sec.gov. The enforcement order behind §14.3.5's first case study: the reused flag, the eighth server, the 97 unheeded alerts, and the rollback that made things worse are all findings in this document. Read it — it is short, plainly written, and the definitive corrective to the folklore versions. *License: U.S. government work, public domain.*
- 📄 **CrowdStrike — External Technical Root Cause Analysis, Channel File 291** (August 2024) — crowdstrike.com. The vendor's own account of the 21-versus-20 field mismatch, the validator bug, and the committed remediations (staged rings, customer-controlled cadence, bounds checking). A model of what a public post-incident analysis should contain. *Access: free on the vendor's site.*
- 📄 **CrowdStrike — Preliminary Post Incident Review, and Remediation and Guidance Hub** (July 2024) — crowdstrike.com. The first public account, with the deployment timeline (content released 04:09 UTC, reverted 05:27 UTC on July 19); the companion Remediation Hub documents the manual safe-mode recovery steps. *Access: free on the vendor's site.*
- 📄 **Microsoft — "Helping our customers through the CrowdStrike outage"** (July 20, 2024) — blogs.microsoft.com. The source of the widely cited ~8.5 million affected-devices estimate ("less than one percent of all Windows machines"). *Access: free to read.*
- 📄 **Delta Air Lines — Form 8-K** (October 2024) — sec.gov. Delta's own filing on the outage: approximately 7,000 flight cancellations over five days and at least $500 million in claimed damages. *Access: SEC filing, public record.*
- 📄 **Parametrix — "CrowdStrike to cost Fortune 500 $5.4 billion"** (July 2024) — parametrixinsurance.com. The loss-estimate source: $5.4 billion in direct losses for the US Fortune 500 (excluding Microsoft), with only 10–20 percent likely covered by cyber insurance. *Access: free to read.*
- 📄 **CNNMoney — "Knight Capital in $400 million rescue agreement"** (August 6, 2012) — money.cnn.com. Contemporary account of the emergency financing that kept Knight alive after August 1; the SEC order above records the July 2013 merger into KCG Holdings. *Access: free to read.*

The cloud and distributed systems

- 📄 **Kubernetes documentation** — kubernetes.io/docs. The official concepts, tutorials, and reference for §14.1.5: containers, pods, services, ingress, and cluster architecture, with a browser-based interactive tutorial that needs no installation. *License: docs are CC BY 4.0.*
- 📘 **37signals — "We have left the cloud"** — world.hey.com/dhh. The first-person record of §14.1.4's repatriation case study; the announcement post **"Why we're leaving the cloud"** (October 2022) — world.hey.com/dhh — lays out the reasoning and the cost arithmetic for leaving the cloud on steady, predictable workloads. *Access: free to read.*
- 📘 **InfoQ — "Prime Video Switched from Serverless to EC2 and ECS to Save Costs"** (May 2023) — infoq.com. A concise writeup of the Prime Video team's account of cutting one monitoring service's cost by roughly 90 percent by consolidating a distributed serverless design into a monolith-style process (§14.1.4). The original Prime Video Tech post (Marcin Kolny, March 2023) is offline; an archived copy survives at web.archive.org. *Access: free to read.*
- 📄 **Gilbert & Lynch — "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services"** (ACM SIGACT News, 2002) — doi.org/10.1145/564585.564601. The paper that turned Brewer's conjecture into §14.1.6's CAP theorem by proving it; author copies circulate freely. Pair it with Brewer's own retrospective, **"CAP Twelve Years Later: How the 'Rules' Have Changed"** (IEEE Computer, 2012), free to read at infoq.com/articles/cap-twelve-years-later, which corrects the folklore "pick two of three" reading. *Access: both freely readable.*
- 📄 **Eric Brewer — "Towards Robust Distributed Systems"** (PODC 2000 keynote slides) — people.eecs.berkeley.edu. The keynote in which Brewer stated CAP as a conjecture, two years before the Gilbert & Lynch proof above. *Access: free on the author's site.*
- 📄 **AWS — Shared Responsibility Model, and "What's the Difference Between Containers and Virtual Machines?"** — aws.amazon.com and aws.amazon.com/compare. Vendor statements of §14.1.4's security-responsibility split ("security *of* the cloud" versus "security *in* the cloud") and §14.1.5's container/VM distinction. *Access: free to read.*
- 📄 **Synergy Research Group — cloud market share updates** — srgresearch.com. Quarterly market data behind §14.1.4's "big three" claim: Amazon, Microsoft, and Google together held 63 percent of enterprise cloud-infrastructure spending in late 2025. *Access: press releases free to read.*
- 📄 **CNCF Annual Survey 2023** — cncf.io. Adoption data behind §14.1.5's "de-facto standard" description of Kubernetes: 84 percent of respondents were using or evaluating it. *Access: free to read.*

CI/CD and deployment practice

- 📘 **Martin Fowler — "Continuous Integration"** — martinfowler.com/articles/continuousIntegration.html. The canonical long-form article behind §14.2: mainline commits, self-testing builds, broken-build discipline, and the ten-minute build. *Access: free to read; © martinfowler.com.*

- 📘 **Pete Hodgson — “Feature Toggles (aka Feature Flags)”** — martinfowler.com/articles/feature-toggles.html. The definitive long-form treatment behind §14.3.3: toggle categories (release, ops, experiment, permission), lifespans, implementation patterns, and toggle-debt hygiene. *Access: free to read; © martinfowler.com.*
- 📘 **Martin Fowler — “DeploymentPipeline”, “BlueGreenDeployment”, and “CanaryRelease”** — martinfowler.com/bliki. Short, precise entries on the deployment strategies of §14.3.2. *Access: free to read.*
- 📄 **Ross Harmes (Flickr) — “Flipping Out”** (2009) — code.flickr.net. The post that popularized the *feature flipper*: trunk-only development, flags, and multiple deploys a day, years before the tooling industry existed (§14.3.3). *Access: free to read.*
- 📄 **Chuck Rossi (Facebook Engineering) — “Rapid release at massive scale”** (2017) — engineering.fb.com. How Facebook’s quasi-continuous release uses the Gatekeeper flag system to roll out code and features independently (§14.3.3). *Access: free to read.*
- 🛠️ **OpenFeature** — openfeature.dev. The CNCF-incubating, vendor-neutral feature-flag API (§14.3.3); pairs with open-source flag backends such as Unleash and GitHub’s Flipper for Ruby. *License: Apache-2.0 (OpenFeature); vendor docs free to read.*
- 📄 **Minimum CD** — minimumcd.org. A community-maintained, vendor-neutral statement of the *minimum* practices that constitute continuous delivery — a useful checklist for auditing your own pipeline against §§14.2–14.3. *License: CC BY-SA 4.0.*
- 📘 **Bryan Finster — “5-Minute DevOps” series** — free to read on bdfinst.medium.com and bryanfinster.com. Short, blunt essays from a practitioner on trunk-based development, small batches, pipeline discipline, and why “we’re too regulated for CD” is usually wrong. Several installments pair directly with this chapter’s sections. *Access: free to read.*
- 📘 **Google — Site Reliability Engineering (the SRE book and workbook)** — sre.google/books. Free online editions covering release engineering, canarying, error budgets, and postmortem culture — the operations side of §14.3 and §14.7 at planetary scale. *License: CC BY-NC-ND 4.0 (free to read).*

Deployment mechanics: containers, DNS, and TLS

- 🛠️ **Docker — official documentation** — docs.docker.com. The Dockerfile reference, image-building guide, and *Get Started* tutorial behind §14.4.1. *Access: free to read; Docker Engine is Apache-2.0.*
- 🛠️ **Docker Compose documentation** — docs.docker.com/compose. The Compose file reference and multi-service examples used in §14.4.2. *Access: free to read.*
- 🛠️ **Docker Official Images — Postgres and Redis** — hub.docker.com/_/postgres · hub.docker.com/_/redis. Maintained, security-patched base images, with usage notes on volumes and environment configuration (§14.4.3). *Access: free; images under their projects’ licenses.*
- 📘 **The Twelve-Factor App** — 12factor.net. The widely cited methodology for cloud-native services; factor III (config) is the externalized-configuration rule of §14.4.4. *Access: free to read.*
- 📄 **Cloudflare Learning Center — DNS and reverse proxies** — what is DNS? · what is a reverse proxy?. Readable explainers for the DNS resolution chain (§14.5.1) and the edge-proxy model (§14.5.3).

Access: free to read.

- 📄 **Let's Encrypt — How It Works, and ACME (RFC 8555)** — letsencrypt.org/how-it-works · rfc-editor.org/rfc/rfc8555. The free, automated certificate authority and the protocol behind automatic HTTPS (§14.5.2). *Access: free; RFC is an IETF standard.*
- 🛠️ **Caddy — automatic-HTTPS reverse proxy** — caddyserver.com. A web server that obtains and renews Let's Encrypt certificates with no configuration, the simplest way to put TLS in front of a container (§14.5.2). *License: Apache-2.0.*
- 📄 **Google — HTTPS encryption on the web (Transparency Report)** — transparencyreport.google.com/https. Ongoing measurement of the share of web traffic served over HTTPS (§14.5.2). *Access: free to read.*

DORA and delivery research

- 📄 **DORA — research program, delivery-metric definitions, and annual State of DevOps reports** — dora.dev. The research base for §14.7: the current five-metric model (guides/dora-metrics), the four-keys-to-five history (insights/dora-metrics-history), performance-band data, capability catalog, and a quick check for assessing your own team. *Access: site content free; reports free to download.*
- 📘 **Forsgren, Humble, Kim — *Accelerate*** (IT Revolution Press, 2018). The book-length treatment of the research methodology and the speed-and-stability finding. *Not* openly licensed — listed here as the standard citation; the dora.dev material above covers the findings freely.

Continuous security and the supply chain

- 📄 **OWASP — scanner-family references** — Source Code Analysis Tools (SAST) · Vulnerability Scanning Tools (DAST) · Component Analysis (SCA). The community definitions behind §14.6.1's three scanner families, with maintained tool lists. *Free to read; OWASP content is CC BY-SA.*
- 📄 **CISA — Software Bill of Materials (SBOM)** — cisa.gov/sbom. The U.S. government's SBOM resource hub referenced in §14.6.2. *License: U.S. government work, public domain.*
- 📄 **GitHub Docs — Dependabot** — docs.github.com/en/code-security/dependabot. Reference for the update-bot workflow of §14.6.2. *License: GitHub docs are CC BY 4.0.*
- 🛠️ **OWASP Dependency-Check and Dependency-Track** — owasp.org/www-project-dependency-check and dependencytrack.org. Open-source SCA: the first scans a build for known-vulnerable components; the second tracks SBOMs across an organization. *License: Apache-2.0.*
- 📄 **SLSA — Supply-chain Levels for Software Artifacts** — slsa.dev. A leveled framework for build-pipeline integrity and artifact provenance — the structured answer to the SolarWinds-style attacks of §14.6.2. *License: spec on GitHub under Community Specification License; site content CC BY 4.0.*
- 📄 **CISA — Alert AA20-352A** (December 2020) — cisa.gov. The US government advisory on the SolarWinds compromise behind §14.6.2: malicious code inserted during the software build and signed with the vendor's legitimate code-signing certificate. *License: U.S. government work, public domain.*

- 📄 **npm — “kik, left-pad, and npm”** (March 2016) — blog.npmjs.org. The registry’s own account of §14.6.2’s left-pad unpublication and the breakage that followed; The Register’s contemporaneous report prints the eleven-line package in full — theregister.com. *Access: both free to read.*

License note

Linked resources remain under their own licenses (noted above where known); this page is licensed **CC BY-SA 4.0**.

Chapter 15 — Maintenance and Evolution

> **Where we are.** Chapter 14 carried a verified change the last mile into production and taught you to keep it flowing — pipelines, progressive deployment, containers, names, certificates, and the DORA evidence that delivery performance matters. This chapter is about everything after the celebration: the years a successful system spends being fixed, adapted, extended, and refactored, usually by people who did not write it.

Deployment begins the longest phase of a successful system's life. Most professional effort goes into **evolving** systems that have been in production for years, not into building new ones — and this chapter is about the code you will inherit.

The industry has names for that work. **Corrective maintenance** fixes defects. **Adaptive maintenance** responds to a changing environment — a new OS version, a deprecated API, a new regulation — where the code did nothing wrong but the world moved. **Perfective maintenance** adds the features and improvements users keep asking a living system for. And **preventative maintenance** — refactoring, debt paydown — improves structure now so that all the other kinds stay affordable later. A widely cited historical rule of thumb places maintenance, taken together, at roughly 60 percent of a system's lifetime cost — the share varies substantially by system, by organization, and by what gets classified as maintenance, but the direction is not in doubt.[1] Read that number again: the phase this book spent fourteen chapters preparing you for is the *minority* of the money, which is reason enough to treat evolving code as the main event of an engineering career rather than the cleanup after it.

15.1 What Makes Code Legacy

Colloquially, "legacy" means old. The working definition that matters is different: **legacy code is code without tests** — or, in its more visceral form, *code you are afraid to change*. Age is incidental. A module written last month with no tests, no documentation, and one departed author is legacy; a fifteen-year-old module with a thorough suite is not, because the suite makes change safe. The defining property is that *the system's actual behavior is not pinned down anywhere except in the code itself* — so any change might break something, you cannot know what, and you cannot know cheaply. Fear sets in, fear breeds avoidance, avoidance means changes are bolted on in the least invasive (and least clean) way possible, and the code gets worse precisely because everyone is being careful. Breaking that spiral is a skill, and it starts with an inversion of the testing you learned in Chapter 10.

The tests-first definition comes from Michael Feathers, whose *Working Effectively with Legacy Code* also names the only two ways there are to change legacy code.[2] **Edit and pray**: study the code, make the change, look around manually for anything you broke, deploy, and hope. **Cover and modify**: first build tests that cover the code you must touch, then make the change and let the tests detect any behavior you altered without meaning to. This book

has been teaching the second way all along; here it finally gets its name. Cover-and-modify starts with a search, not an edit: locate your **change points** — the specific places in the code where your change must actually land — because those are the places the test coverage has to grip before you touch anything. The next two sections are cover-and-modify in practice.

15.2 Characterization Tests

Chapter 10's tests were built from a *specification*: the oracle (§10.1.4) told you what the right answer *should* be. Legacy code has no trustworthy spec — the comments lie, the documentation describes version 2, and the original requirements are three pivots old. So you flip the direction of inference. A **characterization test** documents what the code *actually does now*: you call the function with an input, observe the output, and write that observation down as the expected value. The running system itself becomes the oracle.

This feels like cheating — you are asserting the code does whatever it does, bugs included. But the test's job is to **pin down current behavior**, not to verify correctness, so that your upcoming changes cannot alter it *unknowingly*. Users, and other code, may well depend on the current behavior, strange corners and all. The practical loop: write a test with a deliberately wrong expected value, run it, read the actual value from the failure message, and promote that actual value into the assertion. Probe the edges — empty inputs, nulls, boundary values — until you have a net of pinned behavior around everything your change might disturb. When a characterization test exposes something that is plainly a bug, resist fixing it in the same breath: record it, finish building the net, and change behavior as its own deliberate, separately reviewed step. One commit should refactor *or* fix, never ambiguously both.

Applied to a fee-code lookup inherited with the clinic scheduler, the loop leaves this trail:

```
def legacy_fee_code(visit_type):                    # inherited: no docs, no tests
  return {"exam": "E10", "lab": "L20", "vaccine": "V30"}.get(visit_type, "E10")

def test_probe_unknown_type():
  assert legacy_fee_code("phone") == "XXX"        # deliberately wrong
# FAILED: AssertionError: assert 'E10' == 'XXX'

def test_unknown_type_bills_as_exam():              # observed value, promoted
  assert legacy_fee_code("phone") == "E10"

def test_empty_type_bills_as_exam():                # edge probe: pinned, bug or not
  assert legacy_fee_code("") == "E10"
```

Characterizing assumes you can find your way around, and with an inherited codebase that takes a deliberate workflow. Read what the previous team left behind first — the tests above all: a passing test is executable

documentation that stays synchronized with the behavior it actually checks, though what it checks may be incomplete or no longer what anyone intends. Then read any design documents. On documentation, note the distinction: an architecture description (§6.5) is the *formal* design artifact, but tests, commit history, and mockups are living *informal* documentation, and agile teams weight the informal kind heavily because it stays closer to the code. Next, generate a class or dependency diagram — most languages have tools that extract one — to see the shape of the system before you dive into any single file. Then get the application and its test suite running *locally*, against a cloned or fixture copy of the database, before touching anything: a system you cannot run is a system you cannot characterize. Only then write characterization tests at your intended change points, and begin.

Modern tooling has also shifted the *comprehension* half of legacy work. Understanding what a gnarly function actually does — the prerequisite for characterizing it — has always been the slowest, loneliest part of the job. AI assistants (§13.2) are genuinely strong here: summarizing an unfamiliar module, proposing what a function's edge cases might be, drafting candidate characterization tests for you to verify against the running code. The verification discipline of Chapter 13 still governs — an AI's *account* of legacy behavior is a hypothesis, and the running system remains the only oracle — but as a hypothesis generator for code no living person understands, it removes a real bottleneck.

15.3 Refactoring Under Green Tests

With behavior pinned, you can refactor. Chapter 2 introduced **refactoring** inside the red–green–refactor loop (§2.3.2): improving the design of existing code without changing its behavior, protected by green tests. Legacy work follows the same feedback loop, but it begins by *building* the green net around existing behavior first. Because the code is difficult to change safely, the team must apply the loop more cautiously. The craft is to move in steps so small that each one is obviously behavior-preserving — rename, extract a function, inline a variable, move a method — running the suite after every step. If the bar goes red, the *last* step is the culprit; undo it and take a smaller one. Named, cataloged refactoring moves (Fowler's catalog is the standard reference) matter because each has known mechanics and known traps; a sequence of safe moves composes into a transformation you would never dare attempt as one leap.[3]

Where should you aim the moves? **Code smells** are surface symptoms that *suggest* — not prove — a deeper design problem: a long method, a large class that does too many things, a stretch of duplicated code, **magic numbers** (unexplained literals like `65` scattered through the logic), deeply nested conditionals, a long parameter list. A smell is a prompt to look closer, not a verdict; sometimes the long method really is the clearest way to write that logic. For functions in particular, a compact health checklist is **SOFA**: keep each function **S**hort, doing **O**ne thing, taking **F**ew arguments, and written at a single level of **A**bstraction. Some smells can even be measured: cyclomatic complexity counts the independent decision paths through a function — built on the control-flow analysis of Chapter 10 — turning "this method feels tangled" into a number a pipeline can watch.

The named moves map onto the smells. Beyond rename, extract, inline, and move: **replace magic number with named constant** (`speed > SPEED_LIMIT` explains itself; `speed > 65` does not); **introduce guard clauses** — early

returns for the exceptional cases — to flatten deeply nested conditionals; **remove duplication**, applying Chapter 6's DRY principle, while staying alert for code that merely *looks* similar but serves different purposes; **decompose large class**, splitting along clusters of fields and methods that change together; **replace temp with query**, turning a scattered computed variable into one well-named method; and **introduce parameter object**, bundling arguments that always travel together into a single type that can then attract the behavior that uses it.

The clinic scheduler's booking check shows two of those smells at once:

```
def can_book(patient, slot, booked_today):
  if patient is not None:
    if slot.open:
      if booked_today < 8:
        return True
      else:
        return False
    else:
      return False
  else:
    return False
```

Replace the magic `8` with a named constant, run the suite, flatten the nesting with guard clauses, run it again — the tests stay green after each move:

```
MAX_DAILY_BOOKINGS = 8


def can_book(patient, slot, booked_today):
  if patient is None:
    return False
  if not slot.open:
    return False
  return booked_today < MAX_DAILY_BOOKINGS
```

Legacy code adds a chicken-and-egg problem the catalog alone cannot solve: the worst code cannot be tested without refactoring (dependencies are hard-wired, everything talks to the database) and cannot be safely refactored without tests. To break the cycle, make only the *enabling* changes required to add a test: introduce a parameter or extract an interface around a hard-wired dependency so a test double can replace it (Chapter 10). Stop once the behavior can be tested, then proceed under that protection.

15.4 Repository Stewardship

The habit of leaving a repository better than you found it is sometimes called the **Boy Scout Rule**.[4] It does not mean perfecting every file you open or disguising a rewrite inside an ordinary change. Repository stewardship extends responsibility beyond the feature or defect named in the ticket to the condition of the surrounding system.

Small improvements compound just as small shortcuts do. A clearer name reduces the cost of the next reading. A characterization test protects the next modification. Removing a stale comment prevents the next engineer from trusting a false explanation. Simplifying one tangled branch makes the next defect easier to isolate. None transforms the system alone, but repeated across a team and over years, they determine whether the repository becomes easier or harder to change.

The guardrails are the same as the rest of this chapter: understand the behavior before changing it, protect it with tests, work in small steps, and keep each change coherent enough to review and reverse. Improve the code you already have reason to touch. Record larger problems instead of pretending they can be repaired safely on the side. This is the review-time principle of §9.3 turned into a daily working habit — deliver the change you were asked for, and leave the system better prepared for the change that comes next.

15.5 Technical Debt

The economics underneath all of this has a name. **Technical debt** is the metaphor for the future cost incurred when you take a shortcut today: like financial debt, it lets you move faster *now* in exchange for **interest** — and the interest is that *every future change to that code costs more* than it would have.[5] The metaphor's precision is its virtue. Debt is a *deal*, not simply "bad code" — and sometimes the deal is a good one. **Deliberate debt** is a conscious trade — "we hard-code the tax rule to make the pilot; we log a ticket to generalize it" — the engineering equivalent of a startup loan, rational whenever learning fast matters more than building clean, *provided you track it and service it*. **Inadvertent debt** is the interest you pay on shortcuts you never knew you took — from inexperience, from requirements that shifted under a once-correct design, or from Chapter 1's crunch pitfall, where scope silently absorbed through overtime gets paid for later in weakened structure. Nobody chose it, so nobody tracks it, so it compounds.

Unmanaged, debt's interest payments consume a team's entire capacity: each feature takes longer, which raises pressure, which invites new shortcuts, which raises interest again. The management is not "never borrow" — it is to borrow knowingly, keep the debts visible (a debt register in the backlog, reviewed like any other work), and pay down principal where you actually pay interest: the high-churn code you touch weekly, not the ugly module nobody has opened in years. Refactoring (§15.3) is the repayment mechanism, and the pipeline (§14.2) is what makes repayment safe enough to do continuously.

15.6 Strangler Fig versus Big-Bang Rewrite

What about a system so far gone that the team wants to start over? Chapter 2's troubled browser rewrite (§2.6.3) showed how a **big-bang rewrite** concentrates risk: you discard the accumulated knowledge embedded in code that handles a thousand edge cases, you run two systems (one frozen, one imaginary) for the duration, and the new system's first real validation comes at the end, all at once. The delivery-era alternative is the **strangler fig** pattern, named for the fig that grows around a host tree, roots itself, and gradually replaces the host it envelops.[6] You place an interception layer — a routing facade — in front of the legacy system, then peel off one capability at a time: build the new implementation, route that slice of traffic to it, verify it in production (a canary, §14.3.2, at the granularity of a feature), and retire the old code path. At every moment, you have one *working* system — part old, part new — and every increment of the rewrite is validated by real use within weeks of being written. The rewrite becomes a sequence of small, reversible deployments instead of one giant irreversible bet: the whole argument of Chapter 14, applied to the biggest change a team ever makes.

15.7 Conclusion

Evolution is where Chapter 6's "design for change" either pays its dividend or collects its debt: systems built with seams, interfaces, and tests bend under years of change; systems without them become the legacy code someone else must characterize, strangle, and replace. The chapter's working sequence is practical: preserve current behavior with characterization tests, refactor while the tests remain green, address technical debt deliberately, and use incremental replacement when a rewrite is justified. Underneath all of it runs the habit of §15.4 — steward the repository, leaving it a little better prepared for the next change than you found it. Chapter 14's compression was a rule about *making* change — keep it small, keep its path to users automatic; this chapter adds the long-run corollary: **keep the code changeable** — because the one certainty about a successful system is that it will have to change for longer than anyone who built it expects.

Sources

1 Robert L. Glass, *Frequently Forgotten Fundamental Facts about Software Engineering* (IEEE Software, 2001). doi.org.

2 Michael Feathers, *Working Effectively with Legacy Code* (Prentice Hall, 2004). informit.com.

3 Martin Fowler, *Catalog of Refactorings*. refactoring.com.

4 Robert C. Martin, *The Boy Scout Rule*, in Kevlin Henney (ed.), *97 Things Every Programmer Should Know* (O'Reilly, 2010) — the widely cited one-page statement of "leave the code cleaner than you found it." The collection is released under a Creative Commons license. 97-things-every-x-should-know.gitbooks.io.

5 Ward Cunningham, *The WyCash Portfolio Management System* (OOPSLA experience report, 1992). c2.com.

6 Martin Fowler, *StranglerFigApplication* (2004). martinfowler.com.

- **Key takeaways** are summarized above in §15.7.
- Continue to the Exercises.
- Go deeper with the Open Resources for this chapter.

Chapter 15 — Exercises

Exercises are graded by depth: **[warm-up]** checks understanding, **[analysis]** asks you to reason. Several require *actual work* — a real characterization suite, a real refactoring, a real repository — not just prose. Show the work, not only the answer.

Concepts

1. **[warm-up]** Classify each of these as *deliberate* or *inadvertent* technical debt (§15.5), and justify: (a) hard-coding a single currency to make a demo deadline, with a backlog ticket to internationalize; (b) a data model that made sense before the requirements pivoted; (c) copy-pasting a function at 2 a.m. during crunch to avoid touching a shared module.

Analysis

2. **[analysis]** *Write a characterization test.* You inherit this undocumented, untested function, which production code calls from several places:

   ```
   def norm_code(s, strict=False):
     if s is None:
       return "" if not strict else None
     s = s.strip().upper().replace("-", "")
     if len(s) > 8:
       s = s[:8]
     if strict and not s.isalnum():
       raise ValueError(s)
     return s or ("" if not strict else None)
   ```

   (a) Following §15.2, write a suite of characterization tests, in your language's test framework, that pins the current behavior for at least six input classes, including a missing value (`None`, `null`, or `nil`, whichever your tab uses), empty string, whitespace-only, over-length, hyphenated, and a non-alphanumeric input under both `strict` values. (b) Identify one behavior your probing reveals that looks like a bug, and explain — citing §15.2 — why you should pin it rather than fix it in the same change. (c) State which single line you would be most afraid to "clean up" without this suite, and why.

3. **[analysis]** *Refactor under green.* With the characterization suite from exercise 2 in place, refactor your language's `norm_code` in small, named, behavior-preserving moves (§15.3): at minimum, extract the normalization steps (trim/uppercase/strip hyphens) into their own function, and remove the duplicated

strict/empty handling so the "what counts as invalid" decision lives in exactly one place. Rules: one move at a time, suite green after every move, no behavior change (the suite is the referee — including any pinned bug). Report each move by its catalog name, and state after which move you would commit, and why there rather than at the end.

4. **[analysis]** *Build a technical-debt register.* The clinic-app team's codebase carries these known liabilities: (a) the authorization check is copy-pasted into three request handlers; (b) the ORM is two major versions behind and the old version stops receiving security patches next year; (c) a hand-rolled date parser has a known daylight-saving bug that two downstream call sites silently work around; (d) the billing module has no tests at all; (e) a feature flag from a promotion that ended last year still guards dead code in the checkout path. Build a **debt register** (§15.5): for each item, classify it (deliberate or inadvertent), name the *interest* it charges (what it costs the team per sprint, concretely) and the *principal* (what paying it off would take). Then order the five for paydown, justify the order — and identify the one item you would argue is *not* worth paying down this term, and why.

5. **[analysis]** Your organization proposes a two-year big-bang rewrite of a ten-year-old billing system. Using §15.6 and the browser-rewrite case of §2.6.3, write a one-page counter-proposal for a strangler-fig migration: what the interception layer would be, which capability you would peel off first (and why *that* one), how each slice gets validated, and what the organization can do at month six under your plan that it cannot do under the rewrite.

6. **[analysis]** *Maintain, strangle, or rewrite?* A regional insurer runs a 12-year-old policy-administration system: the domain rules change perhaps twice a year (regulatory updates), the last production incident was fourteen months ago, two of the original developers still maintain it, and test coverage is thin everywhere except the premium calculator — but one subsystem, the customer-facing quote portal, needs features the old architecture makes painful. Management is being pitched a full rewrite. Write a one-page recommendation that honestly weighs all *three* options — continued maintenance, a strangler migration (§15.6), and the rewrite — against criteria you state explicitly (rate of change, incident history, knowledge retention, test coverage, where the pain actually is). Note that in this scenario the cheapest option may be the right one for most of the system; your recommendation may differ by subsystem, and should say so.

7. **[analysis]** *Find the change points and seams in a real codebase.* Pick a codebase you did not write — your team project's oldest module, or an open-source project you use — and invent one plausible change request for it. Before touching anything, report: (a) the **change points** — the specific files and functions where the change must land (§15.1); (b) the **seam** you would use to get tests in place — the spot where you can alter or intercept behavior without editing the code you are about to change (a constructor parameter, an interface boundary, an environment hook); (c) the first characterization test you would write there (§15.2), with its actual input; and (d) one place you would refuse to modify until it had more coverage, and what that refusal would cost.

8. **[analysis]** *Repository stewardship.* Choose one pull request from your team project or another repository. Identify one small improvement you made — or could safely make — beyond the requested behavior: a clearer name, a stronger test, a corrected document, a simplified branch, removed duplication, or an improved build check. Explain why the improvement is *relevant* to the change (not an unrelated detour), how you verified it preserved behavior (§15.2, §15.3), and name one *larger* cleanup you deliberately left for a separate issue because including it would have made the change harder to review or reverse (§15.4, §15.5).

Chapter 15 — Open Resources

Free, open-licensed, or freely accessible materials that reinforce this chapter. Types: 📘 open text · 🎓 course · 📄 primary source / standard · 🎥 video · 🛠️ tool. Licenses vary and are noted where known. Each entry notes its license or access terms; when in doubt, check the linked page.

Legacy code, refactoring, and technical debt

- 📘 **Martin Fowler — "StranglerFigApplication"** — martinfowler.com/bliki/StranglerFigApplication.html. The original naming of §15.6's incremental-replacement pattern, with the fig-tree metaphor from its source. *Access: free to read.*
- 📘 **Refactoring catalog** — refactoring.com/catalog (Fowler's official catalog of named moves and mechanics) and refactoring.guru (a free-to-read tutorial treatment with examples). The named, small, behavior-preserving steps of §15.3. *Access: both free to read online; the full Refactoring book is commercial.*
- 📄 **Ward Cunningham — "The WyCash Portfolio Management System"** (OOPSLA 1992 experience report) — widely mirrored, e.g. via c2.com. The four-paragraph origin of the technical-debt metaphor (§15.5), worth reading to see how much the original meaning — debt as *deliberate*, to be repaid with refactoring — matches this chapter's framing. *Access: free on the author's site.*
- 📄 **Robert L. Glass — "Frequently Forgotten Fundamental Facts about Software Engineering"** (IEEE Software, 2001) — doi.org. Source for the chapter intro's maintenance-cost figure: maintenance typically consumes 40 to 80 percent (60 percent average) of software costs. *Access: paywalled at IEEE; widely assigned in courses, and author copies circulate.*
- 📘 **Michael Feathers — *Working Effectively with Legacy Code*** (Prentice Hall, 2004) — informit.com. The source of §15.1's tests-first definition of legacy code and the edit-and-pray/cover-and-modify distinction. *Not* openly licensed — listed here as the standard citation.
- 📘 **Robert C. Martin — "The Boy Scout Rule"**, in *97 Things Every Programmer Should Know* (ed. Kevlin Henney, O'Reilly, 2010) — 97-things-every-x-should-know.gitbooks.io. The one-page statement behind §15.4's repository-stewardship habit — "leave the code cleaner than you found it." *Access: the collection is released under Creative Commons.*

License note

Linked resources remain under their own licenses (noted above where known); this page is licensed **CC BY-SA 4.0**.

Appendix A — The Team Project

> **Where we are.** The chapters of this book each teach a discipline in isolation: process, requirements, design, testing, metrics. This appendix is where those disciplines stop being separate lessons and become one experience. Over a semester you and three or four teammates will build a working system for a real user. The project gives you repeated *practice* with the ideas introduced throughout the book. The project is the spine that the concepts hang from, and it is deliberately structured around four deliverables: a **proposal**, two **status reports** (a skeletal system, then a viable one), and a **comprehensive final report**.

A course project is not a bigger homework assignment. Homework has a known answer, a solitary author, and a deadline measured in days. A project has an ambiguous goal, a shifting scope, several authors who must agree, and a horizon measured in months. These differences are intentional: they expose the coordination, changing requirements, and sustained complexity that a short individual assignment cannot reproduce. Everything that makes software *engineering* hard — coordination, changing requirements, complexity that outgrows any one head — only appears at team scale and project length. You cannot learn it from a problem set. You can only learn it by living through a small, safe version of the real thing, which is what this appendix is for.

A.1 Overview

A.1.1 Goals for a Project

Before you choose *what* to build, be clear about *why* you are building it. The project has three goals, and they sometimes pull in different directions.

The first goal is **to ship something real**. By the end of the term you should have a system that a genuine user can run and benefit from — not a demo that works only on your laptop with the right input typed carefully. “Real” is the standard because only a real system exposes the problems this book is about. A toy that no one depends on never forces you to handle bad input, never accumulates the kind of coupling that makes change expensive, and never generates the defect data you will analyze in Chapter 12.

The second goal is **to practice the disciplines deliberately**. It is entirely possible to ship a working system by heroics — one strong programmer, many late nights, no tests, no design. Company A in Chapter 1 did exactly that, and it worked for three months. The project asks more of you: to write down requirements before coding them, to sketch an architecture before committing to it, to write tests that make change safe, and to measure your own process. You are being graded not only on the artifact but on the *engineering* that produced it.

The third goal is **to learn to work as a team** — which is a skill, not a personality trait, and one that employers consistently rank among the attributes they most want to see in new graduates.[1] A team that communicates well

and distributes work fairly will usually out-build a team of stronger individuals who do not.

> **Tip.** Pick a project you can *finish*, not one you can *imagine*. A scheduling tool for a campus club that you complete, test, and measure teaches you more than an ambitious "social network for X" that stalls at 40 percent. Scope is a design decision; make it early and make it small.

Choose a problem with a real user you can talk to — a campus office, a club, a small nonprofit, a lab, even a specific classmate with a recurring annoyance. A reachable user is worth more than an exciting idea, because Chapters 3 through 5 all depend on being able to *ask someone* what they need and to *show someone* what you built.

A.1.2 The Team Experience

Most of the difficulty in a team project is not technical. It is human: who does what, how you keep each other informed, and what happens when someone does not deliver. Address these on purpose, in the first week, because the default — hoping it works out — reliably does not.

Roles. A team of four or five does not need a rigid hierarchy, but it does need *ownership* so that every important responsibility has exactly one name attached to it. A lightweight set of roles that maps onto the process in Chapter 2:

- A **coordinator** (or Scrum-style facilitator) who runs standups, keeps the board honest, and unblocks people. It is a servant job that rotates well, not a boss.
- A **customer liaison** who owns the relationship with your real user, schedules the conversations, and speaks for the user when the team debates priorities (Chapters 3–4).
- An **architecture/integration owner** who keeps the pieces fitting together and guards the module boundaries you agreed on (Chapters 6–7).
- A **quality owner** who sets up CI, keeps the test suite green, and watches the metrics (Chapters 9–12).

Everyone writes code and everyone tests; roles say who *worries* about a concern, not who is allowed to touch it. Rotate at least once mid-semester so no one graduates having only ever done one thing.

Communication. Agree on three channels and their purpose: a *synchronous* one for fast questions (chat), an *asynchronous* one of record for decisions (issues, a shared doc, or pull-request discussion), and a *recurring* meeting — even fifteen minutes twice a week — where each person answers the three standup questions: what I finished, what I am doing next, what is blocking me. Write decisions down where the whole team can find them later. An architecture chosen in a hallway and remembered by one person is a liability.

> **Pitfall.** The most common failure mode is silent divergence: two people build against different assumptions for a week because neither surfaced a question. Cheap, frequent communication is not overhead — it is how you keep a small mistake from compounding into a week of rework. Software is discrete (Chapter 1): a wrong assumption does not degrade gracefully, it just breaks.

Psychological safety. Cheap communication only happens where people feel safe doing it. **Psychological safety** is the shared belief that you can raise a problem, admit a mistake, or ask a "basic" question without expecting punishment or embarrassment.[2] Research on team performance identifies it as a major factor in effective collaboration. In Google's Project Aristotle, a multi-year study of 180 internal teams, psychological safety was the most important team dynamic the researchers identified.[3] Its absence appears in familiar ways: everyone reports being "fine" in standup, delays remain hidden until a deadline, or one teammate quietly redoes another's work instead of discussing the problem. You build it with concrete habits, not slogans. Whoever is leading admits their own mistakes *first*, which licenses everyone else's honesty. Thank people for surfacing bad news early — early is exactly when bad news is cheap, and the thanks is what makes it happen again. And critique the work, never the person: "this function needs tests" opens a discussion; "you never test anything" closes one. The trust these habits produce has a name — **vulnerability-based trust**, built by admitting what you do not know — and it is precisely what this appendix's peer-feedback rituals (the visible board, the weekly log, the team reviews) are training you in.[4]

Conflict and uneven contribution. Disagreement about *technical direction* is healthy and should be resolved with evidence: a spike, a small experiment, a look at the metrics. Make the call, write down *why*, and move on. Do not let a decision stay open because the team is avoiding the discomfort of choosing.

Uneven *contribution* is the harder problem, and every team hits some version of it. Deal with it early, factually, and kindly. When someone is behind, the first assumption should be a fixable cause — they are blocked, over-committed in another course, or unclear on the task — not a character flaw. Make work *visible* (a board where every task has an owner and a status makes contribution impossible to fake or to overlook), so that "who is doing what" is a matter of record rather than resentment. If a genuine imbalance persists after you have raised it directly, escalate to your instructor *early* rather than absorbing it into a bitter final week. Most course rubrics — including the final report in this appendix — include an individual-contribution component so that fair effort is recognized and free-riding is not.

> **Tip.** Keep a shared, running log of who did what, updated weekly, not reconstructed from memory at the end. It makes the individual-contributions section of your final report honest and painless, and it turns a potential argument into a lookup.

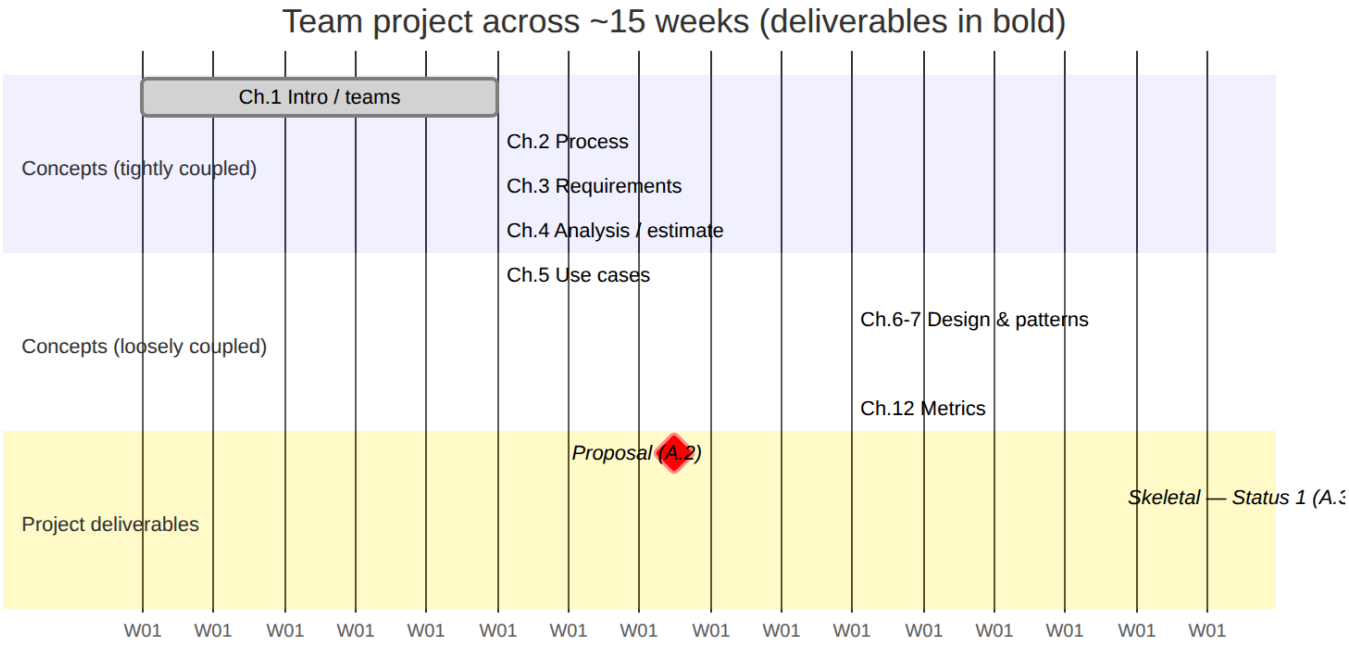
A.1.3 Coupling the Classroom and Project Experiences

This course is designed as **two parallel tracks** — a concepts track (the chapters) and a project track (this appendix) — that are *tightly coupled early and deliberately loosened later*. Understanding that shape will help you plan your semester.

For the first four weeks the tracks move in lockstep, because that is when a mistake is cheapest to correct and when you most need the theory to steer the project. In week 1 you form teams alongside Chapter 1. In week 2 you choose a process and stand up your repository and board alongside Chapter 2. In week 3 you write your **proposal** while Chapter 3 teaches user requirements, so the requirements you write are the *real* ones for *your* system, not exercises. In week 4 you build and estimate a backlog while Chapter 4 teaches analysis and prioritization. The lecture on Monday is the technique; the project on Friday is the practice. Learning and doing reinforce each other while the stakes are low.

After the requirements are stable, the coupling loosens on purpose. The concepts track keeps advancing on its own schedule — design and patterns (Chapters 6–7), static checking and testing (Chapters 9–10), metrics (Chapter 12) — while your team iterates at whatever cadence fits your process and your lives. You will pull each concept into the project as you need it rather than exactly the week it is lectured, which is how real engineering works: you learn a technique when a problem demands it. The **status reports** in weeks 7 and 11 are the checkpoints that keep the loosely-coupled tracks from drifting too far apart, and the **final report** in week 14 is where everything reconverges.

The full week-by-week mapping lives in the course plans; the timeline below shows how the four project deliverables sit on top of it. Courses that run the project on a sprint cadence should read Running the Project on Two-Week Sprints, which overlays these same four milestones on a two-week rhythm.



A.2 Project Proposal

The **proposal** (due around week 3) is where your team commits to a problem, a user, and a plausible scope. The proposal establishes that the team has identified a real problem and developed a plausible starting plan. It remains revisable as requirements change during the project (Chapter 1).

A good proposal answers five questions concretely:

1. **What problem, for whom?** A two-or-three-sentence problem statement and the specific user or stakeholders it serves. Name a real person or office you can talk to. Vague users (“students”) produce vague requirements; specific users (“the intramural sports coordinator in the rec center”) produce testable ones (Chapter 3).
2. **What will it do?** A short list of the most important user stories or features, in the users’ language, prioritized (MoSCoW or must/should/could is enough this early). This is your first backlog, and Chapter 4 will teach you to refine and estimate it.
3. **What will you build it with?** Your intended architecture in broad strokes and your technology choices, with a sentence of *why* for each significant one. You are not committing forever; you are showing you have a feasible path (Chapter 6).
4. **How will you work?** Your process (Scrum, XP, or a hybrid), your cadence, your tools (repo, board, CI), and your initial roles (Chapter 2, §A.1.2).
5. **What could go wrong?** The two or three biggest risks — a hard integration, an unavailable data source, an unfamiliar framework — and how you will probe them early. A risk you name in week 3 is a task; a risk you ignore is a week-13 emergency.

Keep it short — two to four pages. A proposal that is long is usually hiding uncertainty behind volume. Use the project proposal template as your starting form; copy it into your repository and fill it in.

> **Pitfall.** Proposing a scope you cannot finish is the single most common — and most punishing — mistake. When in doubt, halve the feature list. A smaller system built well, tested, and measured earns more than a larger one left broken. You can always add scope in a later iteration; you cannot recover the weeks a doomed scope will cost you.

A.3 Skeletal System: Status Report 1

By around week 7 your team delivers a **skeletal system** — a *walking skeleton* — and reports on it. A walking skeleton is a tiny implementation that exercises the system **end to end**: a real request enters through the real interface, flows through each architectural layer, touches the real database or service, and produces a real response — but does almost nothing useful yet.[5] If your app is a web tool, the skeleton might let a user log in, create one empty record, and see it listed. That is all. At this milestone, the important result is that a request travels successfully through *every layer* and returns a response.

The skeletal milestone exists because integration is where projects die. A team can build five beautiful components in isolation and discover in week 12 that they cannot be wired together — the assumptions do not match, the interfaces do not line up, the deployment does not deploy. Requiring the walking skeleton in week 7 reveals integration problems while the team still has time to correct them. It also stands up the machinery you will rely on all term: the repository, the build, the test harness, and continuous integration (Chapter 14) so that every commit is checked. From this point on you are *always* looking at a running system and fleshing it out, never assembling a pile of parts at the end and praying.

Your Status Report 1 should therefore demonstrate:

- A running, deployed (or trivially runnable) system a grader can exercise, however thin.
- The **architecture** made real: the modules and their interfaces from your Chapter 6–7 design, now visible in code, with a current diagram (note where reality diverged from the proposal, and why).
- Working CI that builds the system and runs at least one genuine end-to-end test.
- An honest status: what is done, what changed since the proposal, what is blocked, and your plan for the viable system.

Use the status report template and mark it as report 1.

> **Tip.** Build the skeleton along the *riskiest* path first, not the easiest. If a third-party integration or an unfamiliar deployment worries you, make *that* the thread the skeleton walks. The whole point is to buy down risk early, so aim the skeleton at the thing most likely to hurt you. Deployment itself is usually that thing — Chapter 14 shows how to make it boring.

A.4 Viable System: Status Report 2

By around week 11 you deliver a **viable system** — a *minimum viable product* — and report on it again. "Viable" is the key word and a high bar: this is a system that a real user could actually use to accomplish the core task, end to end, even if it is missing polish, secondary features, and edge-case handling. Where the skeleton proved the *structure* works, the viable system proves the *value* is there: it does the one or two most important things a user came for, and does them correctly.

Getting from skeleton to viable is the main building phase, and it is where the whole book comes together. You will flesh out the modules behind the interfaces the skeleton established. You will apply architectural patterns (Chapter 7) where they earn their keep — and resist them where they do not. Above all, you will now be **testing in earnest** (Chapter 10): the features a user depends on need black-box tests derived from their requirements and enough white-box coverage that you can change code without fear. A viable system without tests is just a demo that will break the first time you touch it.

Your Status Report 2 should demonstrate:

- The **must-have** user stories from your backlog, working end to end for a real user.
- A test suite with a stated **coverage** target and the current numbers (Chapter 10), run automatically in CI.
- An architecture description updated to match what you actually built.
- Early **metrics** (Chapter 12): velocity across your iterations, open/closed defect counts, build health — whatever your process produces. You will analyze these more rigorously for the final report; start collecting now.
- A candid gap analysis: what is left for the remaining weeks, reprioritized in light of what you learned.

Use the same status report template, marked as report 2.

> **Pitfall.** Do not confuse "viable" with "feature-complete." The temptation at week 11 is to have many half-finished features. Resist it. One fully working, tested, user-usable path beats five stubs. Cut scope ruthlessly (the iron triangle from Chapter 1 is real: fix time and cost, flex scope) so that what you *do* ship is genuinely usable.

A.5 Comprehensive Final Report

The **final report and demo** (around week 14, with presentations in week 15) is where you deliver the finished system and step back to tell its whole story — as engineers, with evidence. It is both the capstone artifact and the primary thing you are assessed on, because it is where you demonstrate that you *engineered* the software rather than merely producing it.

A comprehensive report ties the entire book together, with one section per major discipline. It should cover:

- **The problem and users** — what you set out to solve and for whom, and how that understanding evolved since the proposal (Chapters 1, 3).
- **Requirements** — the user stories and use cases you ultimately built, prioritized, with an honest note on what changed and why (Chapters 3–5).
- **Architecture** — the system's structure with a current diagram, the key design decisions, and the trade-offs behind them, using the vocabulary of cohesion, coupling, and patterns (Chapters 6–7).
- **Process** — how you actually worked: your iterations, what your board and standups looked like, and where your process adapted (Chapter 2).
- **Testing and quality metrics** — your test strategy, coverage achieved, defect data, and what the numbers *mean*, analyzed with sound statistics rather than a single hand-picked figure (Chapters 10 and 12).
- **Results** — what works, what does not, and how you know; measured against the goals you set in the proposal.
- **Retrospective** — the honest lessons: what you would do differently, where estimates were wrong, what surprised you. Graders reward insight here, not spin.
- **Individual contributions** — a fair account of who did what, backed by your running log and your version-control history.

How it is assessed: rubrics vary, but a strong report is judged on the **quality of the shipped system** (does it work, for a real user?), the **engineering evidence** (are the requirements, architecture, tests, and metrics real and coherent, or decorative?), and the **honesty and depth of reflection** (do you understand *why* things went as they did?). A modest system reported on with rigor and insight routinely outscores an ambitious one reported on with hand-waving. Use the final report template as your structure.

> **Tip.** Write the retrospective for the *next* team — yourselves in a future course, or the students who inherit your repo. The most valuable sentences in an engineering report are the ones that begin "if we did this again, we would…". They prove you learned something the deliverable alone cannot show.

A.6 Grading Your Own Repository

All term, other people grade your work. This section is about doing it to yourself first — on the same axes this book teaches, while there is still time to act on the answer.

This book's repository ships a **repository scorecard**: instructions an AI coding agent follows to work with a codebase against the nine parts of this book that cover the software lifecycle — process and teamwork (Chapters 1–2 and this appendix), requirements (3–5), design and architecture (6–7), version control (8), software quality (9–10), security (11), metrics (12), AI-assisted practice (13), and delivery and evolution (14–15). It has two modes, and you want them in this order.

Adopt writes the book's practices into your project's `AGENTS.md` — the file your coding agent reads at the start of every session. Test-driven development, Given/When/Then acceptance scenarios, commit and review standards, secrets handling, the definition of done: each rule stated concretely and cited to the chapter that explains it. Run it on the day you create the repository, and the practices are in force from the first commit rather than retrofitted in week 14. It works just as well on a project already underway, and it tailors itself — it fills in your actual test and lint commands and flags practices it did not find, so the block reads as a backlog rather than a fiction.

`AGENTS.md` is a **tool-neutral convention** (https://agents.md/): the same file is read by Claude Code, Cursor, Copilot, Codex, Gemini CLI, and others, so your team is not locked to whichever assistant you started with. If a tool insists on its own filename, keep `AGENTS.md` as the single source of truth and leave a one-line file pointing at it — two full copies drift apart within a sprint; a redirect cannot.

Score then audits what the repository actually does, reporting a letter grade per area with specific findings. A grade you cannot trace to a chapter is not worth much; every finding cites the section that teaches the fix.

You will find both under `.claude/skills/repo-scorecard/` in the book's repository, with setup instructions in that repository's `AGENTS.md`. Copy the folder into your own project, then ask your agent to set the project up with the book's principles, or to score the repository. It is plain Python and runs offline; nothing is uploaded.

> **Principle.** Adopt sets the standard; score measures against it. Writing the practices down before you need them is what makes the later grade meaningful — a team that agrees on what "done" means in week 2 is arguing about evidence in week 12, not about standards.

Using it during the project, not after

Run it **at every checkpoint**, starting with the skeletal system — not once, the night before the final report. Two reasons. First, most of what it finds is cheap to fix early and expensive to fix late: a test suite that never gated a merge is a twenty-minute pipeline change in week 6 and a rewrite of your team's habits in week 14. Second, the interesting number is not any single grade but the **delta between runs**. A repository that moves from C to B−

across two sprints is demonstrating engineering discipline, and the two scorecards are evidence you can put directly into your status report or retrospective — far stronger than asserting that quality improved.

The findings are also a ready-made work queue. Each one names a concrete change and an effort estimate, which makes them straightforward to turn into backlog items and split across the team.

What the grade is, and what it is not

Treat the output as a structured second opinion, not a verdict. Its real value is that it makes disagreement concrete: when it reports that your project has no acceptance criteria and you can point to the twelve scenarios in your issue tracker, you have learned something about where your work is *visible* — which is most of what an outside reviewer, including your instructor, can actually assess. **A finding you can refute with evidence is worth as much as one you fix.**

Be clear about its limits. It reads a repository, so it can see your tests, your commits, and your pipeline — but not whether the software is any good for the person using it, not whether your team is functioning, and not the judgment behind a design. Those are the things a demo and a retrospective are for. It is a floor, not a ceiling: passing every check it makes is the beginning of good engineering practice, not the whole of it.

> **Pitfall.** The letter it produces is not your course grade, and there is no rubric anywhere that maps one to the other. A team that optimizes for the scorecard — adding a linter nobody runs, a `SECURITY.md` nobody reads — will score better and *be* no better, which is Goodhart's law from Chapter 12 arriving on schedule. Fix the underlying practice and let the grade follow.

Practitioners can use it the same way on working code, with one adjustment: the tool offers a `practitioner` profile that shifts weight away from coursework process and toward quality, security, and delivery.

A.7 Conclusion

The team project is where this book stops being a set of ideas and becomes a way of working. Each of the four deliverables — proposal, skeletal system, viable system, final report — forces a specific engineering discipline at the moment it matters most: commit to a real problem, prove the structure integrates, prove the value is usable, and account for the whole with evidence. Do them honestly and you will finish the semester having *done* software engineering — on a team, under changing requirements, at a scale that would have defeated improvisation. That experience, more than any exam, is what makes an engineer.

Sources

[1] National Association of Colleges and Employers, *Job Outlook 2025* (2025). naceweb.org.

[2] Amy C. Edmondson, *Psychological Safety and Learning Behavior in Work Teams*, Administrative Science Quarterly 44(2) (1999). doi.org.

[3] Google re:Work, *Guide: Understand Team Effectiveness* (2016). rework.withgoogle.com.

[4] Patrick Lencioni, *The Five Dysfunctions of a Team* (2002). tablegroup.com.

[5] Alistair Cockburn, *Crystal Clear: A Human-Powered Methodology for Small Teams* (2004). alistair.cockburn.us, archived.

- **Key takeaways** are summarized above in §A.7.
- Continue to the Exercises.
- Go deeper with the Open Resources for this appendix.
- Running on sprints? See Running the Project on Two-Week Sprints.
- Templates: idea pitch · proposal · sprint report · status report · team review · final report · individual write-up.

Running the Project on Two-Week Sprints

> **Where we are.** Appendix A structures the team project around four deliverables: a proposal (§A.2), a skeletal system (§A.3), a viable system (§A.4), and a comprehensive final report (§A.5). This page is an adaptation guide for courses that run the project on a **two-week sprint cadence** with a **cross-team customer model** — say, a 16-week semester with four build sprints. Nothing about the four milestones changes; they simply become **checkpoints you reach via sprints** rather than dates you aim at directly. If your course uses this variant, read Appendix A first, then use this page as your operating schedule.

Dual hats: every team is both customer and developer

The single hardest thing to make real in a classroom project is the **gap between the customer and the developers**. When a team builds its own idea, requirements work becomes theater: everyone already "knows" what the product should do, elicitation is a conversation with yourself, and acceptance means grading your own homework. Courses solve this in different ways — some recruit external or industrial customers,[1] some rotate a customer role among students each sprint.[2] This variant uses a **swap**:

- Every team **authors a project pitch** — and *another team builds it*.
- Every team **is assigned a different team's pitch** — and builds *that*, all semester.
- **You never build your own idea.**

So each team wears two hats for the whole term:

- **The developer hat.** For the project you were assigned, you are the engineering team: you elicit requirements from your customer, maintain the backlog, build, test, deploy, and demo — everything in the sprint tables below.
- **The customer hat.** For the idea you pitched, you are the customer: you answer the developers' questions, help select and prioritize stories at sprint planning (§2.2.2 — you are, functionally, their product owner's voice of the customer), attend their Demo Days, and give honest feedback on whether what shipped is what you meant.

> **Principle.** The swap makes Chapter 3 real. The developers genuinely *do not know* what the customer wants — the idea lives in another team's heads. Tacit knowledge, the unstated obvious, solutions masquerading as needs (§3.1.2) stop being textbook warnings and start being Tuesday. And because the customer did not build the thing, their acceptance at Demo Day is honest in a way self-assessment never is.

Practical mechanics that make the swap work:

- **A standing channel per developer–customer pair** (a shared chat channel works well). Developers post what shipped and the deployed URL each sprint; customers respond with feedback and priorities. Silence from the customer hat is a failure of that hat.
- **A customer meeting inside every sprint planning** (the cadence below): the developers bring candidate stories; the customer says which matter most and why.
- **Customer feedback closes every demo.** It goes on the record (the team review template has a section for it) and feeds the next sprint's backlog.

The cadence

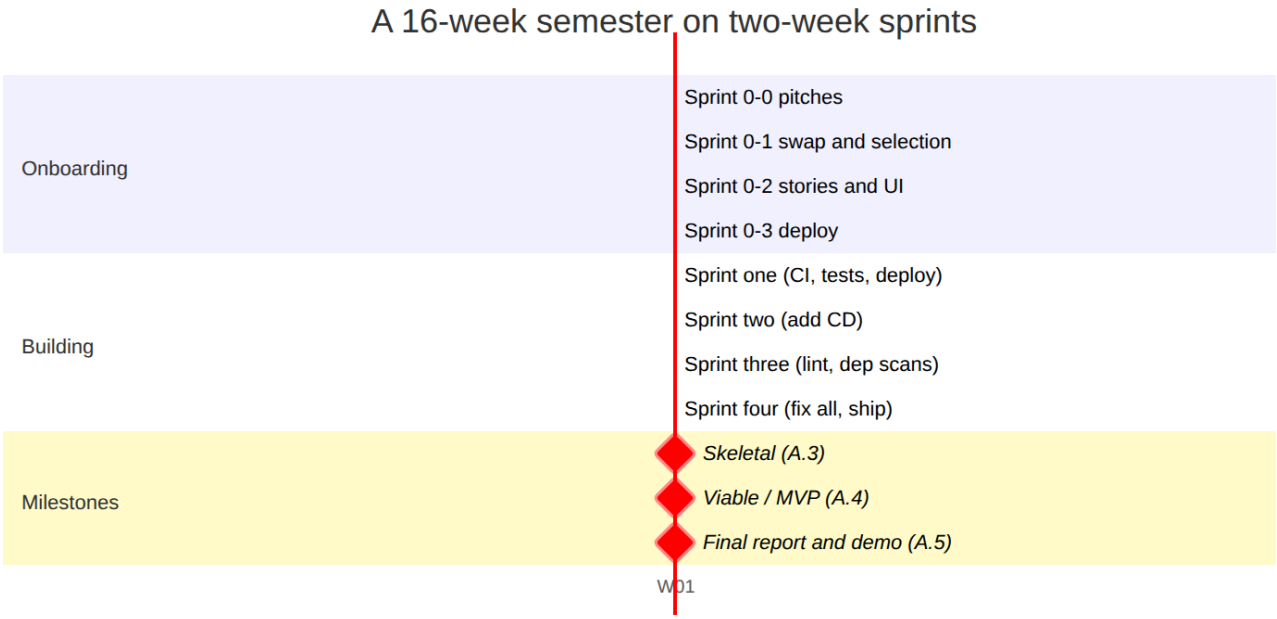
The semester has two phases.

Sprint 0 — onboarding (about four weeks). Before a team can sprint, it needs a problem, a customer, a backlog, and a running skeleton. Sprint 0 is a sequence of four roughly one-week deliverables — pitches, the swap and selection, stories and sketches, and a first deployment — that ends with the walking skeleton of §A.3 live on a real URL. Sprint 0 is deliberately *not* a build sprint: its job is to make the first build sprint possible.

Sprints 1–4 — building (eight weeks). Four two-week build cycles. Each sprint ends with two events:

- **Demo Day** — the sprint review of §2.2.3: the team demonstrates the deployed system to the class and to its customer, and the feedback goes straight into the next sprint's backlog.[3]
- **A team and customer review** — the retrospective half of the cycle, recorded with the team review template, plus peer evaluations.

The build sprints also carry a deliberate **engineering-hardening arc**: each sprint adds one layer of the delivery pipeline from Chapter 14, so that by the end of the term the team is operating a small but genuinely professional SaaS delivery process. The milestones overlay the cadence as before: Sprint 0-3 *is* the skeletal-system checkpoint (§A.3); the end of Sprint 2 should look like the viable system (§A.4); Sprint 4 rolls into the final report, presentations, and individual write-ups (§A.5).



The exact weeks flex with a given calendar (breaks, exams); what does not flex is the *order* and the rule that every build sprint ends with a demo of deployed software.

Sprint 0: from pitch to deployed skeleton

Sprint	Deliverable	Read first	Done means
0-0	**Idea pitches — customer hat.** Every member pitches at least one project idea in Shape Up pitch form: problem, appetite, rough solution, rabbit holes, no-gos.[4] You are writing the pitch **another team will build**; you will be its customer.	§2.8 on pitches and appetite	Each idea pitch names a real, reachable user, fits on a page, and is clear enough for strangers to build from.
0-1	**The swap + selection.** Pitches are exchanged across teams (however your course assigns them). The developer team reads its assigned pitch, meets its customer, and records scope decisions — what the first version will and won't attempt, and why (§4.4.3) — then completes the proposal (§A.2) *with the customer's sign-off*.	§3.1 — you are now on the far side of the requirements gap	Project proposal + decision notes, acknowledged by the customer team.
0-2	**User stories + lo-fi UI — elicited, not invented.** The first backlog as INVEST-shaped stories with **Gherkin acceptance criteria** (§3.4.1), plus storyboards of the main flows (§3.5.3) — built from interviews with your customer team, and walked past them before it counts.	§3.3 on elicitation; §§3.4–3.5	Must-have stories are testable and customer-prioritized; a stranger can follow the sketched flow.
0-3	**Initial view, DEPLOYED — through a pipeline.** The walking skeleton of §A.3: one thin path, end to end, live on a real URL — "get one piece done" (§2.8).[5] Stand up the **CI skeleton** in the same sprint: a pipeline that builds and deploys on every merge, even before there is much to test — wiring it around one thin path is far easier than retrofitting it later (§14.2).	§A.3; Chapter 14 on deployment and pipelines	Status report 1; a grader can open the URL and exercise one real path.

Sprints 1–4: build cycles with a hardening arc

Every build sprint runs the same common cycle; each one also *adds* a permanent layer of engineering infrastructure. Once added, a layer stays: a Sprint 3 team is still running everything it set up in Sprints 1 and 2.

Sprint	New engineering focus	Where the book teaches it	Milestone
1	**Test-driven features, coverage, CI, production deploy.** Every feature lands with unit/integration tests (TDD, and BDD with executable Gherkin scenarios); the suite holds a coverage floor (~80%); a **CI pipeline** runs the tests and reports coverage on every commit and performs automated (e.g., AI-assisted) review on every pull request; the app is live in production.	TDD/BDD: §2.3.2, §3.4.1, §10.2.3 · coverage: §10.3 with the §10.1.3 pitfall · CI: §14.2 · review on PRs: §9.3, AI review §13.2.6	Velocity established
2	**Continuous deployment.** Merging to main *is* the release: a **CD pipeline** automatically deploys every merged change to production. This is the sprint where the team feels why the pipeline's checks exist — read the two §14.3 case studies *before* wiring it up.	§14.3 — delivery vs. deployment, and the Knight Capital / CrowdStrike case studies	≈ Viable system (§A.4)
3	**Static analysis + supply-chain security.** The pipeline gains **linting** with reports and **dependency-vulnerability scanning** (Dependabot-style); the deployment gets production polish (e.g., a real domain). Findings are triaged, not ignored — they become backlog items.	linting & analyzers: §9.4 · supply-chain security: §11.4, enforced as pipeline gates in §14.6 · triage & false positives: §9.4.2	Quality data flowing
4	**Pay the debt, ship the story.** Close out: **all** open vulnerability and linter findings fixed — a real, bounded exercise in technical-debt paydown — plus a short **marketing video** that sells the product to its users, and the final demo path polished.	debt paydown: §15.5 · refactoring under green tests: §15.3 · communicating value: §A.5	Final report & presentations

> **Why this order.** It is the argument of Chapters 14 and 15 enacted: first make the checks automatic (CI), then make release boring (CD), then widen the net (lint, dependencies), then pay down what the net caught. A team that tries to bolt all four layers on in the last sprint learns the Knight Capital lesson the hard way.

Every sprint: the common cycle

These run every sprint, regardless of number — each is a book discipline in miniature:

- **Backlog grooming and prioritization, MVP-first.** Groom the backlog, mark the **MVP** stories, and prioritize — MVP marking is MoSCoW's *must-have* line (§4.4.1); with only four build sprints, the appetite is fixed and scope is the variable (§4.2.4).

- **Planning Poker for sizing** (§4.3.1); split stories until they are small, testable, and assignable — INVEST (§3.4.1).
- **A customer meeting** to confirm which stories this sprint serves the customer's goals (§2.2.3 sprint planning, with the dual-hat customer in the product-owner conversation).
- **Every story has an owner and tests**; features are built full-stack with unit *and* integration tests (§10.2), red–green–refactor (§2.3.2) — keeping the small tidy-as-you-go refactors of each red–green cycle every sprint, while *large, structural* refactoring is deliberately deferred to the end-of-term debt paydown (§15.5).
- **Branches, protected main, reviewed merges** (Chapter 8). Work on branches, merge to main frequently, and let branch protection force a review of every merge — modern code review as a gate (§9.3, §14.2).
- **Acceptance criteria live on the cards as Gherkin scenarios** (§3.4.1) and are demoed as passing behavior at Demo Day (§10.2.3).
- **Release discipline.** Tag the repository at each sprint boundary (and push the tags); the tag is the auditable "what shipped this sprint."
- **Document your AI use.** Keep a running record (e.g., in the README) of where AI assisted the work, with links to the transcripts — provenance and honest verification are part of professional AI-assisted engineering (§13.2.10 and the AI-use-policy exercise in Chapter 13).
- **Peer evaluations** after every sprint — honest, specific, delivered to teammates (§A.1.2; recorded via the team review).

Demo Day rules

Demo Day is the sprint review, and four rules keep it honest:

1. **Demo deployed software from the main branch — not slides.** Share the production URL first, so the audience can follow along; the demo runs against the same URL a grader can visit five minutes later. If it is not on main and deployed, it did not ship this sprint.
2. **Walk the work, then prove it.** Say what the application is and why it exists, walk the completed stories on the board, demo each in production — and show the tests that back each one (§10.2.3: the Gherkin scenarios *are* the acceptance evidence).
3. **Anyone on the team can drive.** If only one person can operate the system, that is a bus-factor problem the demo just exposed.
4. **End with customer feedback — and a failed demo is information, not shame.** The customer's reaction goes on the record and into the backlog. A demo that breaks in public tells the team exactly where the system is fragile, two weeks before it would have cost more. The only shameful demo is a staged one.

Appetite per sprint

Each sprint has a **fixed two-week duration** and a fixed team, leaving scope as the adjustable variable (§4.2.4).[6] Plan by asking, "what is the best version of this we can demo in two weeks?" — then select and trim the work to

fit that appetite, rather than planning by estimate. When the box gets tight, practice **scope hammering**: shrink the task, not the deadline.[7] Mark nice-to-haves with a `~` on your board when you write them down, so that cutting them at crunch time is a pre-authorized decision instead of a mid-sprint argument — and keep a few pre-groomed extras ready so a member who finishes early has a next story waiting. A team that ends every sprint with a working demo and a short list of consciously dropped `~` items is executing well; a team that ends with five features at 80 percent is not.

Team and customer reviews

Every cycle ends with a recorded review, using the team review template: a keep/change/try retrospective, peer contribution ratings with evidence, and the customer feedback captured from Demo Day — which, under the dual-hat model, your team also *gives* to another team wearing your customer hat. The mechanism that makes retrospectives compound rather than repeat is the **one committed improvement**: each review the team commits to exactly *one* checkable change, and the next review's first question is whether it stuck. One improvement per cycle, actually carried out, beats five that evaporate.

> **Graduate research track.** Courses that attach a graduate research component to the project can align it to the same cadence: the **research topic and thesis statement** are due with Sprint 0-3, so the investigation runs alongside the build sprints instead of colliding with finals. For topics on AI and software engineering, the open resources for Chapter 13 — capability evaluations, delivery-performance research, coding benchmarks — make a solid starter bibliography.

End of term

The final two weeks reconverge on §A.5: teams give **final presentations** with a live demo of the deployed system, submit the comprehensive final report, and each member submits an individual write-up — what you built, what you learned, and an honest self-assessment. The marketing video from Sprint 4 doubles as the presentation's opening exhibit. If you kept your sprint reports and review records current, the final report is an act of assembly, not archaeology: the velocity data, coverage and vulnerability trends, scope cuts, and retrospective insights are already written down, one sprint at a time.

Sources

[1] Nayla Nasir, Muhammad Usman, Jürgen Börstler, and Nina Dzamashvili Fogelström, *Software engineering team project courses with industrial customers: Students' insights on challenges and lessons learned*, Journal of Systems and Software (2025). sciencedirect.com.

[2] Austin Z. Henley, *The design of software engineering course projects* (2025). austinhenley.com.

[3] Ken Schwaber and Jeff Sutherland, *The Scrum Guide* (2020). scrumguides.org.

[4] Ryan Singer, *Shape Up: Stop Running in Circles and Ship Work that Matters*, ch. 6 “Write the Pitch” (2019). basecamp.com/shapeup.

[5] Ryan Singer, *Shape Up*, ch. 11 “Get One Piece Done” (2019). basecamp.com/shapeup.

[6] Ryan Singer, *Shape Up*, ch. 3 “Set Boundaries” (2019). basecamp.com/shapeup.

[7] Ryan Singer, *Shape Up*, ch. 14 “Decide When to Stop” (2019). basecamp.com/shapeup.

- Back to Appendix A — The Team Project.
- See the course plans for week-by-week schedules, including the two-week-sprint variant.

Appendix A — Project Exercises

Unlike the chapter exercises, these are **checkpoints** for a real team project. Work them with your team, in order, across the term. Each maps to a deliverable in the course plans. Checkpoints are tagged **[team]** (working agreements and people), **[project]** (deliverable artifacts), or **[analysis]** (reflection).

Weeks 1–2 — Team and idea

1. **[team]** Write a one-paragraph *team charter*: how you will communicate, how often you will meet, how you will make decisions, and how you will handle a member who falls behind. Commit it to your repository's README.
2. **[team]** Brainstorm five project ideas. For each, note the primary user and the one problem it solves. Pick one and write two sentences on why it is *feasible in a term* and *worth doing*.

Weeks 3–4 — Proposal

3. **[project]** Fill in the project proposal template. Your backlog must use MoSCoW (Chapter 4) and contain at least one *Won't-have-this-time* item to prove you have scoped realistically.
4. **[analysis]** Identify your project's single riskiest assumption (technical, user, or team). Describe the cheapest experiment that would tell you within one week whether it holds.

Weeks 5–7 — Skeletal system (Status Report 1)

5. **[project]** Build a *walking skeleton*: a thin slice that runs end-to-end through every architectural layer (Chapters 6–7) even if each part is trivial. Submit Status Report 1.
6. **[analysis]** Draw your system's architecture as it actually is now (not as planned), using a class or component diagram from Chapter 6. Note one place where reality already diverged from the proposal, and why.

Weeks 8–11 — Viable system (Status Report 2)

7. **[project]** Reach a minimum viable product: the *Must-haves* work for a real user. Submit Status Report 2 with a test-coverage number (Chapter 10) and your current defect count (Chapter 12).
8. **[team]** Run one retrospective (Chapter 2). Record two things to keep doing and one concrete change. At the next checkpoint, report whether the change stuck.

Weeks 12–15 — Final report and demo

9. **[project]** Complete the final report, including a quality-metrics section (Chapter 12): defect-removal efficiency, at least one chart of your project data, and a short *limitations* paragraph naming the threats to validity in your numbers — a semester project yields few, unevenly measured, non-independent observations, and saying so is part of the analysis. Descriptive statistics, trends, and honest raw counts are the expected level of rigor; add a confidence interval only if your data give you enough independent observations to justify one (§12.8), and say explicitly when they do not. A stated limitation earns more credit than pseudo-precision. If your pipeline history supports it, also report your five DORA delivery metrics (§14.7.4) from your deploy log — with the same limitations discipline.
10. **[analysis]** Write an honest *individual contribution* statement for each member, then a team retrospective: what would you do differently if you started over knowing what you know now? Tie at least three lessons back to specific chapters.
11. **[project]** Prepare a 5–8 minute demo that shows the system doing real work for its primary user — not a slideshow *about* the system, but the system itself.
12. **[analysis]** Score your repository with the repository scorecard from §A.6 at the start of this phase and again before the demo. Include both scorecards in your final report with a short account of what changed: which findings you fixed, which you decided not to fix and why, and which you refuted with evidence the tool could not see. A defended disagreement counts as much here as a fix.

Appendix A — Open Resources

Free, open-licensed, or freely accessible materials to help your team run the semester project well. Types: 📘 open text · 🎓 course · 📄 primary source/standard · 🎥 video.

Running a team project end to end

- 🎓 **UC Berkeley — Engineering Software as a Service (ESaaS), project guidance** — saasbook.info. Berkeley's course materials on scoping, running, and grading agile team projects, including customer interaction and the iterate-toward-a-viable-product model this appendix follows. The companion MOOC and slides are free to use; the textbook itself is separately licensed, so link the free resources rather than copying text.
- 🎓 **MIT 6.031 — Software Construction** — web.mit.edu/6.031. Course readings under a Creative Commons license; excellent for the architecture, testing, and code-quality work in the building phase (Status Reports 1–2).
- 📄 **Alistair Cockburn — "Walking skeleton" (from *Crystal Clear*, 2004)** — alistair.cockburn.us (archived). The original description of the walking-skeleton strategy behind the skeletal-system milestone (§A.3). The book is commercially published; Cockburn's own page, preserved by the Internet Archive, is free to read.

Requirements for the proposal (A.2)

- 📄 **Volere Requirements Specification templates** — volere.org/templates. A widely used, freely downloadable template set for eliciting and recording requirements and stakeholders; useful even if you work in user stories, as a checklist of what you might be missing. Check the current terms on the site before redistributing.
- 📄 **INVEST checklist & user-story practice** — Bill Wake's original write-up is freely available online; search "INVEST in good user stories Wake." A one-page test for whether a story is ready to build (Chapters 3–4).

Team effectiveness (A.1.2)

- 📄 **Google re:Work — Guide: "Understand team effectiveness" (Project Aristotle)** — rework.withgoogle.com. Google's research finding that *how* a team works together — especially psychological safety — matters more than *who* is on it. Directly relevant to conflict and uneven contribution. The re:Work guides and tools are free to use.

- 📄 **Amy C. Edmondson — “Psychological Safety and Learning Behavior in Work Teams”** (*Administrative Science Quarterly*, 1999) — doi.org. The original research article defining psychological safety as a shared belief that a team is safe for interpersonal risk taking. The journal article is paywalled; access it through your library, or read the free re:Work guide above, which summarizes it.
- 📄 **Patrick Lencioni — *The Five Dysfunctions of a Team* (2002), model overview** — tablegroup.com. The source of **vulnerability-based trust** as the foundation of teamwork (§A.1.2). The book is commercially published; the publisher’s overview of the model is free to read.
- 📄 **NACE — Job Outlook 2025** — naceweb.org. The National Association of Colleges and Employers’ annual employer survey; teamwork consistently ranks among the attributes employers most want to see in new graduates. The report is freely downloadable.

Process (A.1.1, referenced from Ch. 2)

- 📄 **The Scrum Guide** — scrumguides.org. The definitive, short, free definition of Scrum accountabilities, events, and artifacts, released under a Creative Commons Attribution-ShareAlike license. Ideal for teams choosing a process in week 2.
- 📘 **Ryan Singer — *Shape Up: Stop Running in Circles and Ship Work that Matters*** — basecamp.com/shapeup. Basecamp’s product-development method — pitches, appetite, fixed time and variable scope, scope hammering — that the two-week-sprint variant of this project draws on. Free to read online.
- 📄 **Manifesto for Agile Software Development** — agilemanifesto.org. The four values and twelve principles, freely readable, that motivate the whole iterate-early approach here.

Curriculum context (why the course is shaped this way)

- 📄 **ACM/IEEE-CS Computing Curricula recommendations** — acm.org/education/curricula-recommendations. The professional-society guidelines that recommend a substantial team project and the parallel concepts/project tracks this course uses. Freely available.

Cross-team customer models (the dual-hat swap)

- 📘 **Austin Z. Henley — “The design of software engineering course projects”** — austinhenley.com/blog/groupprojects.html. A practitioner-professor’s survey of project-course designs, including rotating customer/lead roles and mid-course team swaps — context for why separating the customer from the builders matters. *Free to read.*
- 📄 **“Software engineering team project courses with industrial customers: Students’ insights on challenges and lessons learned”** (Journal of Systems and Software, 2025) — sciencedirect.com. Research on courses that use real external customers — the same customer–developer separation the dual-hat swap creates without needing industry partners. *Open access.*

Metrics for status reports and the final report (referenced from Ch. 12)

- 📘 **OpenIntro Statistics** — openintro.org. A free, openly-licensed statistics textbook; use it to analyze your velocity and defect data honestly rather than cherry-picking a number.
- 📄 **DORA / Accelerate metrics overview** — dora.dev. Freely available guidance on the five delivery metrics — throughput (change lead time, deployment frequency, failed-deployment recovery time) and instability (change fail rate, deployment rework rate) — a mature vocabulary for the quality dashboard in your final report.

License note

Linked resources remain under their own licenses; verify terms before redistributing any material. This page is licensed CC BY-SA 4.0.

Idea Pitch —

> **How to use this template.** Copy this file into your team's repository (for example, `docs/pitches/pitch-court-scheduler.md`), delete these instruction blockquotes, and replace every *italic prompt* with your own content. **One page maximum.** A pitch is not a proposal: it is a problem that is *rough-solved and bounded* (§2.8) — solved enough to show a feasible path, bounded enough to say what it will *not* do. Each teammate pitches at least one idea.
>
> **You are writing for another team.** Under the dual-hat model (Running the Project on Two-Week Sprints), a *different* team builds this pitch — and your team becomes its **customer**. Write so that strangers can build from it, and only pitch something you are willing to answer questions about all semester. The developer team completes the full project proposal with you after the swap.

Field	Value
Working title	…
Pitched by	*your name*
Team	…
Date	*YYYY-MM-DD*

1. Problem

Tell one concrete story of the problem happening to a specific person, and state the baseline — how they cope today. If you cannot name a real person or office who feels this pain, pitch something else.

2. Appetite

Why is this worth roughly a semester of two-week sprints — no more, no less? Time is the fixed box; scope flexes to fit it (§4.2.4). Say what a "small but real" version looks like.

3. Rough solution

Sketch the core idea at fat-marker level (§6.1.2): elements, connections, and the main user flow — strokes too thick for detail on purpose. A photo of a whiteboard sketch is ideal.

4. Rabbit holes

Where could a team burn a whole sprint? Name the technical unknowns, tricky cases, and integrations this pitch is quietly assuming will be easy.

- …
- …

5. No-gos

What is explicitly out of bounds, even if someone asks? Saying “no” here is what keeps the appetite honest.

- …
- …

6. Customer contact

Who on your team is the primary customer contact for the developers who build this, and where do they reach you (channel)? As the customer, you commit to answering elicitation questions, prioritizing stories at their sprint planning, and giving feedback at every Demo Day.

Project Proposal — <Team / Project Name>

> **How to use this template.** Copy this file into your team's repository (for example, `docs/proposal.md`), delete these instruction blockquotes, and replace every *italic prompt* with your own content. Aim for two to four pages. See Appendix A, §A.2 for what makes a proposal strong. Keep the scope small enough to finish, test, and measure.

Field	Value
Team name	…
Members	*name (role), name (role), …*
Real user / stakeholder	*who you will talk to*
Repository	*URL*
Board / tracker	*URL*
Date	*YYYY-MM-DD*

1. Problem statement

Describe the problem in two or three sentences. What is hard, slow, error-prone, or impossible today, and who feels that pain? State the problem, not your solution.

2. Users and stakeholders

Name the specific people this serves. Distinguish primary users (who use it daily) from secondary users and sponsors. How will you reach a real user to ask questions and show progress?

- **Primary user(s):** …
- **Secondary user(s):** …
- **Sponsor / decision-maker:** …

3. Proposed solution and value

_In a paragraph, describe what you will build and the single most important benefit it delivers. Finish this sentence: "A user will be able to __, *which today they cannot.*"

4. Key user stories / features (prioritized)

List the most important stories in the user's language. Prioritize with MoSCoW (Must / Should / Could / Won't) — this is your first backlog.

Priority	As a … I want … so that …	Notes
Must	…	…
Must	…	…
Should	…	…
Could	…	…
Won't (this term)	…	*out of scope, on purpose*

5. Scope boundaries

State explicitly what you will NOT do. Naming non-goals is how you keep scope finishable.

- In scope: …
- Out of scope: …

6. Intended architecture and technology

Sketch the system in broad strokes: major components and how they connect. List your main technology choices with one sentence of "why" for each. You are showing a feasible path, not committing forever.

- **Components:** …
- **Stack (with rationale):** *language/framework — why; database — why; hosting — why*
- **Sketch:** *paste a simple diagram or link to one*

7. Process and team plan

How will you work? See Chapter 2.

- **Process:** *Scrum / XP / hybrid — and why*
- **Iteration cadence:** *e.g., 2-week sprints*
- **Roles:** *coordinator, customer liaison, architecture owner, quality owner*
- **Communication:** *sync channel, async record, meeting schedule*

8. Risks and how you'll probe them

Name your two or three biggest risks and the early experiment (a "spike") that will tell you whether each is a real problem — while there is still time to react.

Risk	Likelihood / impact	How we'll probe it, and when
…	…	…
…	…	…

9. Definition of done for the term

One or two sentences: what does a successful end-of-semester system look like? This is the bar you will measure yourselves against in the final report.

Sprint Report <1–4> — <Team / Project Name>

> **How to use this template.** Copy this file into your repository (for example, `docs/sprint-2.md`), delete these instruction blockquotes, and fill in every *italic prompt*. Submit one at the end of each two-week build sprint (see Running the Project on Two-Week Sprints). The report backs up your Demo Day: everything you claim here should be visible in the deployed system or the repository. Be honest — a candid "we cut scope" beats an optimistic "almost done."

Field	Value
Sprint number	*1 / 2 / 3 / 4*
Dates	*YYYY-MM-DD to YYYY-MM-DD*
Demo link	*URL of the deployed system a grader can try*
Repository	*URL*

1. Sprint goal

One sentence: what this sprint was supposed to achieve, as you stated it at sprint planning.

2. What shipped (demo notes)

What can a user do now that they could not two weeks ago? List what you demoed (or will demo) on Demo Day, from the main branch.

- …
- …

3. Stories: planned vs. done

Your commitment against your delivery. Velocity is the running average of points completed per sprint — track it, don't spin it.

Story	Points	Planned	Done	Notes
…	…	Yes / No	Yes / Partial / No	…
…				

- **Points completed this sprint:** …
- **Velocity so far:** *points per sprint, averaged over completed sprints*

4. Quality

See Chapters 9–10.

- **CI status:** *passing? link to the pipeline*
- **Coverage:** *target and current number*
- **Open defects:** *count, with a link to the tracker*
- **Pipeline layers active this sprint:** *CI tests / PR review / CD / linting / dependency scans — per the sprint's hardening arc*
- **Findings triage:** *(Sprints 3–4) open vs. fixed linter and vulnerability findings*

4a. Process evidence

- **Sprint tag pushed:** *e.g.* `sprint2` *— verify it is checkout-able on the remote*
- **Acceptance criteria on cards:** *Gherkin scenarios present and demoable? (yes/no)*
- **AI use documented:** *README section current, with transcript links for each assist*
- **Customer touchpoints this sprint:** *planning meeting date + channel posts (URL)*

5. Scopes and hill chart

Break the remaining work into scopes and place each on the hill: uphill means "still figuring out how," downhill means "just executing" (§12.3.5). Paste a snapshot or sketch.

Scope	Position (uphill / crest / downhill / done)	Movement since last sprint
…	…	…

6. Scope cuts and nice-to-haves dropped

Two weeks is a fixed box; scope flexes. What did you hammer out to fit, and which nice-to-haves (~) did you drop or defer?

- …

7. Risks and blockers

What is threatening the next sprint, and who owns each item?

Item	Impact	Action / owner
…	…	…

8. Next sprint plan

The candidate goal and top stories for the next sprint, sized to your actual velocity — not your hopes.

- **Proposed goal:** …
- **Top stories:** …

9. Contributions

One line per member, this sprint only. Keep it current so the final report is honest.

Member	What they did this sprint
…	…

Status Report <1 | 2> — <Team / Project Name>

> **How to use this template.** Copy this file into your repository (for example, `docs/status-1.md`), delete these instruction blockquotes, and fill in every *italic prompt*. Use **Report 1** for the *skeletal system* / walking skeleton (Appendix A, §A.3) and **Report 2** for the *viable system* / MVP (§A.4). Be honest: graders reward a clear-eyed status over an optimistic one.

Field	Value
Report number	*1 (skeletal) / 2 (viable)*
Team	…
Date	*YYYY-MM-DD*
How to run / try it	*URL or one-line command a grader can use*

1. Summary

Two or three sentences: where the project stands and the single most important thing to know since the last checkpoint.

2. What works right now

Describe what a user can actually do end to end today.

- **Report 1 (skeletal):** *which single path runs end to end through every layer?*
- **Report 2 (viable):** *which must-have user stories work end to end for a real user?*

Feature / story	Status	Evidence (link, screenshot, test)
…	Done / Partial / Not started	…
…		

3. Architecture (current)

Show the system as it actually is now, not as proposed. Note where reality diverged from the plan and why (Chapters 6–7).

Diagram placeholder — paste a Mermaid diagram or link to `assets/diagrams/…`:



- **Key decisions since last report:** …
- **Divergence from proposal / previous report:** …

4. Process and progress

How the team worked this period (Chapter 2).

- **Iterations completed:** …
- **Velocity / throughput:** *stories or points per iteration*
- **Board snapshot:** *link*

5. Testing and quality (Report 2 emphasizes this)

See Chapters 9–10.

- **CI status:** *passing? link to the pipeline*
- **Tests:** *how many, what kind (unit / integration / end-to-end)*
- **Coverage:** *target and current number* (Report 1: at least one real end-to-end test)

6. Metrics so far (Report 2)

Early numbers you are collecting for the final report (Chapter 12).

- **Defects:** *open / closed*
- **Build health:** …
- **Other:** …

7. Risks, blockers, and changes

What is threatening the plan, what is blocking you, and what you decided to change.

Item	Impact	Action / owner
…	…	…

8. Plan for next milestone

Concrete, prioritized next steps toward the next deliverable.

- …
- …

9. Individual contributions this period

Keep this current every report so the final one is honest.

Member	What they did this period
…	…

Team & Customer Review <#0–4> —

> **How to use this template.** Copy this file into your repository (for example, `docs/review-2.md`), delete these instruction blockquotes, and fill in every *italic prompt* as a team, at the end of each sprint cycle (see Running the Project on Two-Week Sprints). This is the retrospective half of the cycle (§2.2.3): Demo Day inspects the *product*; this review inspects the *team*. Candor is the whole point — a review that reports "everything is fine" every cycle is a review that is not working.

Field	Value
Review number	*0 / 1 / 2 / 3 / 4*
Team	…
Date	*YYYY-MM-DD*
Attendees	*members present; customer present?*

1. Retrospective: keep / change / try

As a team, name what worked, what hurt, and what you want to experiment with. Specific beats general: "standups drifted to 25 minutes" is actionable; "communication" is not.

Keep (working — protect it)	Change (hurting — stop or fix it)	Try (an experiment)
…	…	…
…	…	…

2. Improvement committed last review — did it stick?

Copy the one improvement you committed to last time. Did you actually do it? What happened? (Review #0: write "first review" and move on.)

- **Last commitment:** …
- **Verdict:** *stuck / partially / abandoned — and why*

3. New committed improvement

Pick exactly ONE change to carry into the next cycle. One, so it actually happens. Make it checkable: someone reading it next review should be able to say yes or no.

- **We commit to:** …
- **We will know it worked if:** …

4. Peer contribution

Each member rates their own contribution this cycle and receives an averaged rating from teammates (1–5). Evidence is a link — commits, pull requests, reviewed PRs, docs, tests — not an adjective. Gaps between self and peer ratings are a conversation, not a verdict.

Member	Self rating (1–5)	Peer rating (1–5)	Evidence
…	…	…	*links*
…	…	…	…

5. Customer feedback captured

What did your customer team (and any real end users) say when they saw the demo (or the latest build)? Record their words, then your response — even if the response is "no, and here is why."

What they said	What we'll do about it
…	…

6. Escalations

Anything that needs instructor attention — a persistent contribution imbalance, a blocked dependency, a scope crisis? Raising it now is engineering; raising it in the final week is too late. Write "none" if none.

- …

Final Report — <Team / Project Name>

> **How to use this template.** Copy this file into your repository (for example, `docs/final-report.md`), delete these instruction blockquotes, and replace every *italic prompt* with your own content. This is your capstone artifact and the main thing you are assessed on — see Appendix A, §A.5. It should tie the whole course together: requirements (Ch. 3–5), architecture (Ch. 6–7), process (Ch. 2), testing (Ch. 10), and metrics (Ch. 12). Reward comes from working software, real engineering evidence, and honest reflection.

Field	Value
Team	…
Members	…
Date	*YYYY-MM-DD*
Repository	*URL*
How to run / try it	*URL or steps*
Demo	*link to video or slides, if any*

1. Problem and users

What problem did you set out to solve, and for whom? How did that understanding evolve from the proposal to now? Name your real user(s) and stakeholders (Chapters 1, 3).

2. Users and requirements

The requirements you ultimately built. Present the final, prioritized user stories / use cases, and be honest about what changed since the proposal and why (Chapters 3–5).

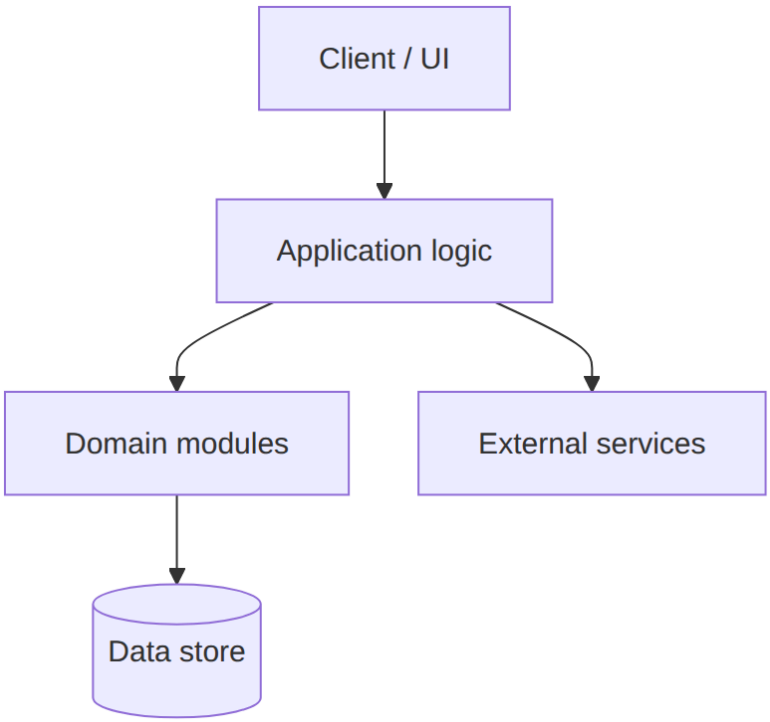
Priority	User story / use case	Delivered?	Notes
Must	…	Yes / Partial / No	…
Should	…		
Could	…		

- **What changed since the proposal, and why:** …

3. Architecture

Describe the system's structure and the reasoning behind it (Chapters 6–7). Discuss key design decisions in the vocabulary of cohesion, coupling, and patterns, and the trade-offs you accepted.

Diagram placeholder — paste a current Mermaid diagram or link to `assets/diagrams/…`*:*



- **Major components and responsibilities:** …
- **Key decisions and trade-offs:** …
- **Patterns used, and why:** …

4. Process used

How you actually worked (Chapter 2) — not the idealized version. Your iterations, cadence, board, standups, and where your process adapted mid-project.

- **Process and cadence:** …
- **What worked / what you changed:** …
- **Tools:** *repo, board, CI*

5. Testing and quality metrics

Your test strategy and the evidence behind your quality claims (Chapters 10 and 12).

- **Test strategy:** *unit / integration / end-to-end; black-box and white-box (Ch. 10)*
- **Coverage achieved:** *target vs. actual*
- **Defect data:** *found, fixed, open; where they clustered*
- **Analysis:** *what the numbers mean — descriptive statistics, trends, and raw counts first; inferential claims like a confidence interval (§12.8) only if your sample justifies them (Ch. 12). Don't cherry-pick a single flattering figure*
- **Limitations / threats to validity:** *what your numbers cannot support — small samples, definitions that changed mid-project, observations that aren't independent*

Metric	Value	What it tells us
Test coverage	…	…
Velocity (per iteration)	…	…
Open / closed defects	…	…
other	…	…

6. Results

What works, what does not, and how you know — measured against the "definition of done" from your proposal. Be specific and evidence-based.

- **Goals met:** …
- **Goals not met, and why:** …
- **Evidence:** *screenshots, logs, test output, user feedback*

7. Retrospective

The honest lessons. What surprised you, where estimates were wrong, what you would do differently. Write for the next team — the most valuable sentences begin "if we did this again, we would…". Insight is rewarded here, not spin.

- **What went well:** …
- **What went poorly:** …
- **What we'd do differently:** …
- **Biggest lesson about software engineering:** …

8. AI use and provenance

Where and how AI assisted the work, per the running log your README kept all term (Chapter 13, §13.2.10): the kinds of tasks you delegated, links to the transcripts, and — most importantly — how each AI-assisted change was verified (tests, review) before it shipped. One honest paragraph plus the log link beats a list of tools.

- **README AI log:** *link*
- **What we delegated vs. owned:** …
- **How we verified AI-assisted changes:** …

9. Individual contributions

A fair account of who did what, backed by your running log and version-control history. Keep it factual.

Member	Primary responsibilities	Notable contributions	Rough share
…	…	…	…%
…	…	…	…%

- **Notes on collaboration:** *how work was divided and reviewed; how conflicts or imbalances were handled*

10. Appendix (optional)

Links to the proposal, both status reports, the backlog, and any supporting data or diagrams.

Individual Write-up —

> **How to use this template.** Copy this file, delete these instruction blockquotes, and fill in every *italic prompt*. This is an **individual** deliverable: write it alone, in your own voice, without coordinating the content with teammates. It complements the team's final report (see Appendix A, §A.5). Graders reward **honesty over polish** — a clear-eyed account of what went wrong teaches more, and scores better, than a varnished one. One to three pages.

Field	Value
Name	…
Team / project	…
Date	*YYYY-MM-DD*

1. What I built

Your concrete, personal contributions to the system — features, modules, tests, pipeline, documentation. Be specific and link the evidence: pull requests, commits, files, the deployed feature itself. "I worked on the backend" is not an answer; "I built the scheduling module and its integration tests (PR #41, #47)" is.

- …
- …

2. What I learned

*Tie your experience to at least **three** concepts from the chapters, citing the section (for example, walking skeletons §A.3, appetite §4.2.4, coupling §6.2.2, coverage §10.3). For each: what the book said, what actually happened on your project, and what you now believe about it.*

1. *Concept (§…):* …
2. *Concept (§…):* …
3. *Concept (§…):* …

3. Working on a team

An honest reflection on the human side: how the team communicated, how conflict or uneven contribution was handled, what role you played in the dynamics — including where you were part of the problem. This section is

about insight, not blame.

4. What I’d do differently

If you started this project again tomorrow, what would you change — technically, in process, or personally? The most valuable sentences here begin “if I did this again, I would…”.

- …
- …

5. Self-assessment

The grade you would give yourself for this project, and the justification. Ground it in the evidence from sections 1–4, your team’s review records, and the version-control history — not in effort you feel but cannot show.

- **Grade:** …
- **Justification:** …

Open Curriculum Resource Map

This book is designed to support **a complete first undergraduate software-engineering course** on its own (see the chapter-to-SEEK mapping below). This map is a companion: for each chapter it lists the best **free or openly accessible** courses, textbooks, primary sources, and videos, so learners who want more depth — or instructors assembling a reading list — can reinforce every topic without buying anything. **Licenses vary and are noted where known** — "free to read" is not the same as "openly licensed"; check each before adapting.

Resources are grouped as:

- 📘 **Open textbook / notes** — full readable text, usually CC-licensed.
- 🎓 **Course / MOOC** — university courseware or a free online course.
- 📄 **Primary source** — the canonical spec, standard, or paper.
- 🎥 **Video** — recorded lectures or talks.

> **Coverage summary.** The book's fifteen chapters cover a first course. The four "anchor" open courses below reinforce most of that ground; the per-chapter tables add focused resources for the rest (estimation, use-case mechanics, architectural pattern catalogs, and the statistics in Chapter 12).

The four anchor courses

Anchor	Covers chapters	What it is	License
MIT 6.031 Software Construction — web.mit.edu/6.031, OCW 6.005 (2016)	6, 9, 10	Complete, polished reading notes on specs, ADTs, testing, code review, and design.	CC BY-NC-SA (OCW)
UC Berkeley — Engineering Software as a Service (ESaaS) — saasbook.info	1, 2, 3, 10	Free book + MOOC on agile, BDD/user stories, and testing.	Free book; CC-BY media
CMU 17-214 / 17-313 Software Engineering — 17-313 course site	2, 6, 9, 12	Open slides/readings on process, design, reviews, and measurement.	Course-open
"Software Engineering at Google" — free online	1, 9, 10, 12	Industry practices for reviews, testing, and metrics at scale.	CC BY-NC-ND

Chapter 1 — Introduction

Type	Resource	Notes
📘	ESaaS ch. 1 “Introduction to SaaS & Agile”	What SE is; why process matters.
📄	ACM Code of Ethics and Professional Conduct	§1.6 social responsibility.
📄	IEEE-CS/ACM Software Engineering Code of Ethics	Professional conduct.
📄	N. Leveson & C. Turner, “An Investigation of the Therac-25 Accidents” (IEEE Computer, 1993)	The canonical safety case study.
📄	SWEBOK Guide v4 (IEEE)	The discipline’s body of knowledge.
🎥	MIT 6.031 Lecture 1 “Static Checking”	Sets up “safe from bugs, easy to understand, ready for change.”

Covers: what is SE, requirements challenge, complexity, defects/testing intro, the iron triangle (scope/cost/time), the VW & Therac-25 case studies, ethics.

Chapter 2 — Software Development Processes

Type	Resource	Notes
📄	Manifesto for Agile Software Development	The four values, twelve principles.
📄	The Scrum Guide (2020) · PDF	Accountabilities, events, artifacts — primary source.
📘	Extreme Programming (c2 wiki / Beck summaries)	XP practices and user stories.
📄	W. Royce, “Managing the Development of Large Software Systems” (1970)	The origin of “waterfall” (and its critique).
📄	B. Boehm, “A Spiral Model of Software Development and Enhancement” (1988)	Risk-driven process.
🎓	ESaaS ch. 7 (Project Management, Agile)	Plan-and-document vs. agile lifecycles.

Covers: process vs. values, plan-vs-grow cultures, Scrum, XP, waterfall & V-model, risk, the spiral framework.

Chapter 3 — User Requirements

Type	Resource	Notes
🎓	ESaaS ch. 7 “Requirements: BDD & User Stories”	User stories, Connextra template, SMART.
📄	Volere Requirements Specification Template	Industry requirements template.
📄	ISO/IEC/IEEE 29148 (requirements engineering)	Standard (abstract free).
📘	K. Wiegers, “Software Requirements” articles	Free companion articles & templates.
📄	B. Schneier, “Attack Trees”	§3.7 security requirements.
📄	INVEST checklist for user stories	Story quality.

Covers: requirements cycle, eliciting needs, user stories/features/scenarios, goal hierarchies, security attack trees.

Chapter 4 — Requirements Analysis

Type	Resource	Notes
📄	Planning Poker (Mountain Goat Software)	Story-point estimation.
📘	M. Cohn, “Agile Estimating and Planning” free chapters/articles	Velocity, story points.
📄	MoSCoW prioritization (DSDM/Agile Business)	Must/Should/Could/Won’t.
📄	N. Kano et al., Kano model overview	Satisfiers vs. dissatisfiers.
📄	B. Boehm, COCOMO II model definition	Parametric effort estimation.
📄	Wideband Delphi	Structured group consensus.

Covers: iteration planning, anchoring bias, story points & velocity, Planning Poker, MoSCoW, Kano, value/cost/risk, COCOMO.

Chapter 5 — Use Cases

Type	Resource	Notes
📘	A. Cockburn, *Use-Case Foundation* — templates & method	The definitive use-case method.
📄	UML Use Case Diagrams (OMG UML spec)	Actors, relationships, notation.
📘	ESaaS §7 (user stories vs. use cases)	When to use each.
🎥	Use case modeling tutorials (free)	Notation reference.

Covers: actors & goals, basic/alternative flows, extension points, use-case diagrams, include/extend relationships.

Chapter 6 — Design and Architecture

Type	Resource	Notes
📘	MIT 6.031 — Specifications, ADTs, Abstraction	Modularity, coupling/cohesion, spec design.
📄	P. Kruchten, “Architectural Blueprints — the 4+1 View Model” (1995)	The 4+1 views.
📘	refactoring.guru — Design Principles & SOLID	Coupling, cohesion, OO relationships.
📄	UML Class Diagrams reference	Association, aggregation, composition, inheritance.
📘	“Software Engineering at Google” ch. on design	Real-world modular design.

Covers: role of architecture, modularity principle, coupling & cohesion, UML class diagrams, 4+1 views, describing an architecture.

Chapter 7 — Architectural Patterns

Type	Resource	Notes
📘	M. Fowler, Patterns of Enterprise Application Architecture (catalog)	Layering, MVC, and more.
📄	M. Fowler, GUI Architectures (MVC/MVP)	Model-View-Controller in depth.
📘	Microsoft — Cloud Design Patterns	Pub/sub, broker, client-server, pipes-and-filters.
📘	M. Richards, Software Architecture Patterns (free O’Reilly report)	Layered, event-driven, microkernel.
📄	Reactive/Dataflow: The Reactive Manifesto	Streams and back-pressure.
📘	Software Product Lines (SEI)	Families & product lines.

Covers: layering, shared-data, observer, publish-subscribe, MVC, dataflow/pipelines, client-server, broker, RESTful APIs, product lines.

Chapter 8 — Version Control with Git

Type	Resource	Notes
📘	Pro Git (2nd ed.)	The canonical Git reference; free, CC BY-NC-SA.
🎓	MIT Missing Semester — Version Control (Git)	Data-model-first Git.
📄	Markdown Guide · CommonMark	Markdown syntax and the standard.
📄	GitHub Flow · Trunk-Based Development	Team workflows.
📄	Chris Beams — How to Write a Git Commit Message	Commit hygiene.
🎥	“Oh Shit, Git!?!”	A recovery cookbook for common messes.

Covers: version control and the commit graph, branching/merging/rebase, remotes and pull requests, merge conflicts, team workflows, and Markdown for READMEs and documentation.

Chapter 9 — Static Checking

Type	Resource	Notes
📘	Google Engineering Practices — Code Review	How industry does code review.
📘	MIT 6.031 — Code Review reading	Static checking & review practice.
📄	M. Fagan, “Design and Code Inspections” (IBM Sys. J., 1976)	The origin of formal inspection.
📘	“Software Engineering at Google” — code review ch.	Review at scale.
📄	SonarQube / linters docs	Automated static analysis in practice.

Covers: architecture reviews, software inspections, code reviews, automated static analysis, false positives/negatives.

Chapter 10 — Testing

Type	Resource	Notes
📘	MIT 6.031 — Testing reading	Partitioning, coverage, glass-box vs. black-box.
📘	P. Ammann & J. Offutt, Introduction to Software Testing (free samples & slides)	Graph/control-flow coverage, MC/DC.
🎓	ESaaS ch. 8 “Testing”	TDD, unit/integration, test doubles.
📘	“Software Engineering at Google” — testing chapters	Test sizes, flakiness, coverage culture.
📄	NIST — Combinatorial (pairwise) testing	Input coverage & ACTS tool.

Covers: levels of testing, control-flow graphs & coverage, MC/DC, equivalence/ boundary, combinatorial testing.

Chapter 11 — Software Security

Type	Resource	Notes
📄	OWASP Top 10:2025	The industry’s shared vulnerability vocabulary (§11.2). CC BY-SA 4.0.
🛠️	OWASP Juice Shop · WebGoat	Deliberately insecure apps for hands-on practice (§11.2).
🎓	PortSwigger Web Security Academy	Free hands-on web-security labs (§11.2).
🛠️	Strix — autonomous AI pentest agents	AI security testing (§11.3). Apache-2.0; **authorized testing only.**
📄	Andres Freund’s xz disclosure · xz timeline	Primary sources for the xz-utils case study (§11.4).
🛠️	OpenSSF Scorecard · SLSA · Sigstore	Supply-chain security tooling (§11.4).
🎓	OpenSSF “Developing Secure Software” (LFD121)	Free course + certificate (§11.5).
📄	Hastings & Walcott, ISSREW 2022	The six-control continuous-verification framework (§11.4).

Covers: the CIA triad and defense in depth, the OWASP Top 10:2025, SAST/DAST/SCA and AI-assisted security testing, the open-source supply chain (Log4Shell, xz-utils, weak links, continuous verification), and building security in (SSDF).

Chapter 12 — Quality Metrics

Type	Resource	Notes
📘	OpenIntro Statistics (CC BY-SA)	Boxplots, histograms, variance, normal & t-distributions, confidence intervals, regression.
📘	Introduction to Modern Statistics (CC, OpenIntro)	Modern companion to the above.
📄	DORA / “Accelerate” State of DevOps metrics	Modern software delivery metrics.
📘	N. Fenton & J. Bieman, Software Metrics (concepts overview)	Measurement theory, scales.
📄	ISO/IEC 25010 product quality model	Forms of software quality.

Covers: meaningful metrics & scales, forms of quality, boxplots/histograms, variance/ SD, normal & t-distributions, confidence intervals, linear regression.

Chapter 13 — Software Engineering in the Age of AI

Type	Resource	Notes
📄	METR — Early-2025 AI & experienced developer productivity RCT	The ~19% slowdown trial (§13.1.4).
📄	o16g — the Outcome Engineering manifesto	Primary source for §13.4.
📄	Perry et al. — “Do Users Write More Insecure Code with AI Assistants?”	The false-sense-of-security study (§13.3).
📄	GitClear — AI code quality research	Duplication up, refactoring down (§13.3).
📄	SWE-bench · SWT-bench	Coding-agent capability benchmarks (§13.2.7).
📄	DORA	Delivery-performance research referenced in §13.2.9.

Covers: what AI changes vs. what endures, the productivity paradox, AI across each lifecycle stage, evidence on quality/security, the o16g manifesto.

Chapter 14 — Delivery: CI/CD, DevOps, and Operations

Type	Resource	Notes
📄	SEC Release No. 34-70694 — In the Matter of Knight Capital	The primary source for the Knight Capital case study (§14.3). Public domain.
📄	CrowdStrike — External Technical Root Cause Analysis (2024)	Primary source for the July 2024 outage case study (§14.3).
📄	DORA — research and delivery metrics	The five metrics: throughput (change lead time, deployment frequency, recovery time) and instability (change-fail rate, deployment rework rate) (§14.7).
📘	Martin Fowler — CI, Deployment Pipelines, Blue-Green	Canonical free articles for §§14.2–14.3.
📘	Google SRE books	Free online; reliability, release engineering, postmortems.
📄	GitHub Dependabot docs · OWASP Dependency-Check · SLSA	Dependency scanning and supply-chain security (§14.6).
📘	minimumcd.org	A concise, community definition of minimum CD practice.

Covers: SaaS & cloud, CI pipelines, continuous deployment & rollout strategies (Knight Capital and CrowdStrike case studies), packaging and DNS/TLS/the edge, security pipelines & supply chain, DORA metrics.

Chapter 15 — Maintenance and Evolution

Type	Resource	Notes
📘	Martin Fowler — StranglerFigApplication	The original naming of the incremental-replacement pattern (§15.6).
📘	Refactoring catalog · refactoring.guru	Named, behavior-preserving moves behind §15.3.
📄	Ward Cunningham — The WyCash Portfolio Management System (OOPSLA 1992)	The origin of the technical-debt metaphor (§15.5).
📄	Glass — Frequently Forgotten Fundamental Facts (IEEE Software 2001)	The maintenance-cost figure behind the chapter's framing.

Covers: what makes code legacy, characterization tests, refactoring under green tests, technical debt, strangler fig versus big-bang rewrite. (Feathers' *Working Effectively with Legacy Code* is the standard commercial citation.)

Appendix A — A Team Project

Type	Resource	Notes
🎓	ESaaS project guidance & rubric	Running a semester team project.
📄	Volere templates	Proposal & requirements docs.
📘	Google Project Aristotle (team effectiveness)	What makes teams work.
📄	ACM/IEEE CS Curricula — SE course guidance	Why a project is required.

Covers: project goals, team experience, proposal, status reports, final report.

Coverage vs. the ACM/IEEE SEEK body of knowledge

The book is designed to support a complete *first* undergraduate SE course. The table below maps each chapter to the knowledge areas of the ACM/IEEE **SE2014 / SEEK** guideline so instructors can see coverage — and its limits — directly, rather than taking a "full scope" claim on faith. SEEK is a curriculum *guideline*; no single one-semester course covers the whole discipline, and the "Depth" column is honest about that.

SEEK knowledge area	Where in this book	Depth
Professional Practice (PRF)	Ch. 1 (ethics, cases), Appendix A (teamwork)	Introductory
Software Process (PRO)	Ch. 2 (Scrum, XP, waterfall, V, spiral, Shape Up)	Solid
Requirements / Modeling & Analysis (REQ, MAA)	Ch. 3–5 (elicitation, analysis, use cases)	Solid
Software Design (DES)	Ch. 6–7 (modularity, UML, 4+1, patterns)	Solid
Software Configuration Management (CFG)	Ch. 8 (Git, branching, merging, pull requests, team workflows, merge conflicts)	Solid
Software V&V (VAV)	Ch. 9–10 (reviews, static analysis, testing, coverage)	Solid
Software Quality (QUA)	Ch. 12 (metrics, statistics, defects)	Solid
Security (SEC)	**Ch. 11 (OWASP Top 10:2025, SAST/DAST/SCA, AI security testing, supply-chain security)**; reinforced by Ch. 3 §3.7 (attack trees, STRIDE), Ch. 9 §9.4 (SAST), Ch. 14 §14.6 (security pipelines)	Solid
Software Construction (CST)	assumed as a prerequisite; touched in Ch. 9–10	Light
Software Evolution / Maintenance	Ch. 15 (legacy code, characterization tests, refactoring, repository stewardship, technical debt, strangler fig); cost-of-change theme throughout	Introductory
Emerging practice: AI-assisted SE	Ch. 13	Introductory
Computing Essentials, Math & Eng. Fundamentals (CMP, FND)	assumed prerequisites	Out of scope

Notes on coverage & gaps

Within the areas it *does* teach, the book is self-contained — no purchase is required to follow the course. This map records how much of each topic *also* has strong external open coverage, so you know where the free supplements are richest:

- **Abundant open supplements:** process/agile, requirements & user stories, design/modularity, code review & static analysis, testing & coverage, and all of the statistics in Chapter 12.

- **Thinner external coverage:** some *estimation* details (e.g. COCOMO calibration), a few *architectural-pattern* treatments, and worked *case studies*. Our own chapters supply original explanations and examples for these.
- **Deliberately light or out of scope** (per the SEEK table above): deep software *construction* and the CS/math prerequisites — appropriate omissions for a one-semester first course, but worth naming so no one mistakes this for the whole discipline. (*Maintenance/evolution* moved from this list to introductory coverage when the book added legacy code, refactoring, and technical debt — now Chapter 15; *security* moved to solid coverage when Chapter 11 added the OWASP Top 10, AI-assisted security testing, and open-source supply-chain security.)

Course Plans

Two ready-to-use schedules for a junior/senior software-engineering course built on this book — **Variant A**, a 15-week milestone track, and **Variant B**, a 16-week semester run on two-week sprints. Both run **two parallel tracks** (as the ACM/IEEE guidelines recommend[1]):

- **Concepts track** — the fifteen chapters, one theme at a time.
- **Project track** — a real team project (Appendix A), started early and iterated.

The two tracks are **tightly coupled at the start** (weeks 1–4, while teams form and scope their projects) and **loosely coupled afterward**, so each can move at its own pace.

Variant A: 15-week milestone track

Week	Concepts (chapter)	Project milestone	Key open resources
1	Ch. 1 — Introduction	Form teams; brainstorm ideas	ACM Code of Ethics; Therac-25
2	Ch. 2 — Processes (Scrum/XP); Ch. 8 — Version Control (Git); CI basics (§14.2.1–14.2.2)	Pick a process; set up repo & board with a protected `main`, plus a hello-world CI pipeline (§14.2) so every later commit lands on green	Scrum Guide; Pro Git; Fowler CI
3	Ch. 3 — User Requirements	**Project Proposal** (App. A.2)	ESaaS ch.7; Volere; INVEST
4	Ch. 4 — Requirements Analysis	Backlog + estimates (Planning Poker)	Planning Poker; MoSCoW; Kano
5	Ch. 5 — Use Cases	Use cases / stories for iteration 1	Cockburn use cases
6	Ch. 6 — Design & Architecture	Architecture sketch; class diagram	MIT 6.031; 4+1 views
7	Ch. 7 — Architectural Patterns	**Status Report 1** (skeletal system)	Fowler PoEAA; MS patterns
8	*Midterm / catch-up*	Sprint review + retrospective	—
9	Ch. 9 — Static Checking; pipeline stages & gates (§§14.1–14.2)	Harden CI: linters, code review as a gate	Google eng-practices; Fagan
10	Ch. 10 — Testing (coverage); progressive deployment (§14.3)	Test plan; coverage targets; staged rollout for the project	MIT 6.031 testing; Ammann/Offutt; SEC 34-70694
11	Ch. 10 — Testing (MC/DC, combinatorial)	**Status Report 2** (viable system)	NIST combinatorial
12	Ch. 11 — Software Security (OWASP Top 10, SAST/DAST/SCA, supply chain)	Threat-model the project; add dependency scanning	OWASP Top 10; PortSwigger; OpenSSF Scorecard
13	Ch. 12 — Metrics (quality, defects, statistics); DORA (§14.7)	Defect tracking; measure & analyze project data	OpenIntro Stats; DORA

Week	Concepts (chapter)	Project milestone	Key open resources
14	Ch. 15 — Maintenance & evolution (legacy code, refactoring, technical debt); synthesis + o16g manifesto discussion (§13.8)	**Comprehensive Final Report** (A.5)	o16g manifesto; Fowler refactoring catalog
15	Final presentations / demos	Ship & retrospective	—

> **Chapter 13 (AI) is read alongside, not as its own week.** Its per-stage sections map onto the concept weeks — assign each §13.2.x with its matching chapter (requirements weeks get §13.2.2, testing weeks §13.2.5, and so on), the evidence sections §13.1 and §13.3 (including the METR study) with week 9 or 10, and the o16g manifesto (§13.8) as the week-14 synthesis discussion. Chapter 14's delivery material is likewise distributed where the project needs it: CI basics in week 2, pipeline hardening in week 9, progressive deployment in week 10, DORA in week 13 — so week 14 belongs to Chapter 15 and to pulling the course together.

> **Flexibility on Chapter 11 (Software Security).** Week 12 gives security its own session, but the material also pairs naturally with the CI and static-analysis week (week 9) — teach it wherever your emphasis lies. Either way, the project's "static analysis + supply-chain security" sprint (Appendix A) is where students apply the OWASP Top 10 and the dependency-vetting controls to their own code. If you are pressed for time, Chapter 12's statistics half (§12.6–§12.9) is the readiest thing to move to optional reading, following its own "two paths" note.

Variant B: 16-week semester on two-week sprints

For courses that run the project on a two-week sprint cadence with the **dual-hat cross-team customer model** — each team pitches an idea that *another* team builds, and serves as customer for its own pitch (see Running the Project on Two-Week Sprints). Concepts and project interleave: four onboarding deliverables (Sprint 0-0 … 0-3), then four two-week build sprints, each ending in a **Demo Day** and a team/customer review, with a progressive engineering-hardening arc (CI → CD → lint/dependency scans → debt paydown).

Week	Concepts (chapter)	Project / sprint deliverable
1	Ch. 1 — Introduction; SaaS & the cloud (§14.1)	Think about project ideas
2	Ch. 3 — User Requirements; Shape Up (§2.8, §4.2.4)	—
3	Ch. 2 — Processes & types of agile; Ch. 8 — Version Control (Git)	Teams form; **Sprint 0-0**: idea pitches; set up repo with protected `main`
4	Ch. 6 — Design principles	**Sprint 0-1**: pitch swap + proposal with customer
5	Ch. 7 — Architectures + REST (§7.5.4); idea presentations	**Sprint 0-2**: user stories + lo-fi UI/storyboards (§§3.4–3.5)
6	Ch. 10 — Testing; CI pipelines (§14.2); DORA (§14.7)	**Sprint 0-3**: initial view, deployed
7	Demo Day; sprints & iterations (§2.2)	**Sprint 1** begins; team review #0
8	*Test 1*	—
9	Continuous deployment (§14.3: Knight Capital, CrowdStrike)	Sprint 1 due → Demo Day; **Sprint 2**; review #1
10	*Spring break*	—
11	Ch. 9 — Static checking; risk	Sprint 2 continues
12	Ch. 11 — Software Security (OWASP Top 10, SAST/DAST/SCA, supply chain); security pipelines (§14.6)	Sprint 2 due → Demo Day; **Sprint 3**; review #2
13	Legacy code & refactoring (Ch. 15); Ch. 13 AI readings	Sprint 3 continues
14	Work days; Ch. 12 metrics in practice	Sprint 3 due → Demo Day; **Sprint 4**; review #3
15	*Test 2*	Sprint 4 due
16	Final presentations	Individual write-ups; review #4

Chapters 4–5 (estimation, use cases) are woven into the sprint work as read-before material (see the sprint table in the appendix page) rather than given lecture weeks. Assessment shifts accordingly: sprint reports and team reviews replace the two status reports, and two in-term tests replace the midterm/final split.

Assessment (suggested weights)

Component	Weight
Team project artifacts (proposal, iterative progress reports/reviews, final report, demo)	45%
Individual exercises / problem sets (chapter `exercises.md`)	20%
Test 1 / midterm (concepts, first half)	15%
Test 2 / final exam (concepts, whole book)	20%

The progress artifacts differ by variant: Variant A uses the two **status reports**, Variant B uses **sprint reports and team reviews** — same weight either way.

Learning outcomes

By the end, a student can:

1. Choose and run an appropriate **process** (Scrum/XP/hybrid) for a team project.
2. **Elicit, write, prioritize, and estimate** requirements as user stories/use cases.
3. **Design a modular architecture** and describe it with UML and 4+1 views.
4. Apply the right **architectural patterns** to a problem.
5. Use **version control** — branching, pull requests, reviewed merges — and **continuous integration** to coordinate team development.
6. **Review and statically check** code and architecture for defects.
7. Design a **test suite** that meets defined coverage criteria.
8. Identify common **vulnerabilities** (the OWASP Top 10) and vet open-source dependencies for **software-supply-chain** risk.
9. Define and interpret **quality metrics** using sound statistics.
10. Design and evaluate a **delivery pipeline**: progressive deployment, automated quality gates, and delivery-performance (DORA) metrics.
11. Safely modify an **inherited system** using characterization tests, incremental refactoring, and technical-debt management.
12. Use **AI-assisted engineering tools responsibly**: specify intent, verify outputs, document provenance, and retain human accountability.
13. Work effectively on a **software team**, communicate progress, and act on the **professional and ethical responsibilities** of shipping software that affects real people.

Adapting the plan

- **10-week quarter:** merge weeks 4 ↔ 5, fold Chapter 11 (security) into week 9's CI/static-analysis session, and teach Chapter 12 on its minimum path (§12.1–§12.5); move MC/DC to optional reading. (See the one-quarter path on the front page.)
- **Bootcamp / self-study:** follow the four anchor courses in `open-resources-map.md` alongside the chapters; do the project solo or in a pair.
- **No team project:** keep the concepts track; use the checkpoints in Appendix A's project exercises as larger individual assignments.

[1] ACM/IEEE-CS Joint Task Force on Computing Curricula, *Software Engineering 2014: Curriculum Guidelines for Undergraduate Degree Programs in Software Engineering* (SE2014), which recommends a substantial team project alongside the concepts. acm.org/education/curricula-recommendations.

Contributing

Thanks for helping improve this open software-engineering book. It is meant to be a living resource, and clear, well-sourced contributions are very welcome.

Ground rules

1. **Original prose only.** Do not paste text, figures, or exercises from any copyrighted textbook. Explain concepts in your own words and cite primary/open sources. This keeps the whole book CC BY-SA-clean.
2. **Prefer open, durable sources.** When you link a resource, prefer ones that are free and openly licensed (MOOCs, university courseware, standards, papers, official docs). Add them to the chapter's `resources.md` and, if broadly useful, to `curriculum/open-resources-map.md`.
3. **Keep the house style** (see below) so chapters read as one book, not many.

Repository layout

```
.
├── README.md                     # landing page
├── SUMMARY.md                    # mdBook table of contents
├── book.toml                     # mdBook config
├── LICENSE                       # CC BY-SA 4.0 (prose) + MIT (code)
├── CONTRIBUTING.md
├── curriculum/
│   ├── open-resources-map.md     # chapter → open resources mapping
│   └── course-plan.md            # course plans (15-week and 16-week sprint variants)
├── chapters/
│   ├── 01-introduction/
│   │   ├── README.md             # the chapter body
│   │   ├── exercises.md          # individual practice exercises
│   │   └── resources.md          # curated open resources for this chapter
│   └── ...                       # one directory per chapter
├── templates/                    # reusable project & document templates
└── assets/diagrams/              # source for diagrams (Mermaid/SVG)
```

House style

- **Structure per chapter:** a short “Where we are / why this matters” opener, then numbered sections matching `SUMMARY.md`, then a footer of three bullets: **Key takeaways** (pointing at the conclusion section), a link to **Exercises**, and a link to **Open resources**.
- **Voice:** direct, second person (“you”), concrete. Explain the *why* behind each practice, not just the *how*. Use a running example or case study where it helps.
- **Diagrams:** use Mermaid fenced code blocks (`` ```mermaid ``) so they render on GitHub and in mdBook. Keep an SVG export in `assets/diagrams/` if the diagram is complex.
- **Callouts:** use blockquotes with a bold lead-in, e.g. `> **Principle.** …`, `> **Pitfall.** …`, `> **Case study.** …`.
- **Terminology:** bold a term the first time you define it. Keep to US spelling.
- **Line length:** wrap prose around 90–100 columns to keep diffs reviewable.

Making a change

1. Fork and branch: `git checkout -b improve-chapter-09-mcdc`.
2. Edit Markdown. Preview locally with `mdbook serve` if you have mdBook installed.
3. Run the link check (see `.github/workflows/ci.yml`) or at least verify new links.
4. Open a pull request describing *what* changed and *why*, and cite sources for any new claims.

Reporting issues

Open an issue for anything from a typo to a conceptual gap or a broken link. If you know the fix, a PR is even better. Suggestions for new open resources are welcome.
